Graal Technical Reference Manual

version 0.0.5, build of 4cd8153

0 Front matter and conventions
0.1 Scope
0.2 Structure and numbering
0.3 Evidence tags and citations
0.4 Table cells
0.5 Numbers, byte order and bits
0.6 Field types
0.7 Messages and their ids
0.8 Units
0.9 Versions, builds and generations
0.10 Servers, hosts and redaction
0.11 Checking and building
0.12 System map
1 Client generations and builds
1.0 Reading this Part
1.1 Build catalogue
1.2 Version tokens
1.3 Version ids
1.4 Encryption generation by build
1.5 Level-format generation by build
1.6 Era timeline
1.7 Precursors and side builds
1.8 Build profiles
1.9 Identity and platform evidence
2 Wire protocol
2.0 Reading this Part
Chapters
2.1 Framing and encodings
2.1.1 Integer and string encodings
2.1.2 Messages and terminators
2.1.3 The raw-data escape
2.1.4 Frames by generation
2.2 Ciphers and compression
2.2.1 Encryption generations
2.2.2 Generation 3 insertion
2.2.3 Generation 4 and 5 partial XOR
2.2.4 Compression choice
2.2.5 Generation 6 and script-defined transports
2.3 Login and handshake by generation
2.3.1 1998 relay (generation 1)
2.3.2 1999 GSERV login (generation 2)
2.3.3 2.x login (generation 3, type 0)
2.3.4 2.22, 2.31 and 3.x login (generation 5, type 5)
2.3.5 4.x to 6.03 login (generation 5, type 5, machine ids)
2.3.6 Server parse and reply sequence
2.3.7 Unpacked 6.037 against unpacked 6.1.1.3
2.3.8 Linux local-login baselines
2.4 PLI message reference (client to server)
2.4.1 Conventions
2.4.2 Ids sent by build
2.4.3 Level navigation
2.4.4 Props
2.4.5 Combat and level objects
2.4.6 Flags, NPCs and weapons
2.4.7 Files, scripts and server values
2.4.8 Chat, private messages and profiles
2.4.9 Client checks and housekeeping
2.4.10 Staff messages sent by game clients

2.4.11 Login-time order
2.4.12 Outgoing senders in unpacked 6.1.1.3 and 6.0.3.7
2.5 PLO message reference (server to client)
2.5.1 How a client reads a PLO
2.5.2 Ids handled by build
2.5.3 Level data
2.5.4 Players and the local player
2.5.5 Combat and level objects
2.5.6 NPCs, weapons and scripts
2.5.7 Files and update packages
2.5.8 Messages, windows and modes
2.5.9 Staff and NPC-control messages
2.5.10 Other ids
2.5.11 Unpacked Windows 6.1.1.3: script-installed handlers
2.5.12 Linux receive switches and compiled-script delivery
2.6 Props
2.6.1 Prop streams
2.6.2 PLPROP (player)
2.6.3 Body colours and attributes
2.6.4 NPCPROP
2.6.5 BDPROP (baddy)
2.6.6 Ids each parser accepts
2.6.7 Property ids in unpacked 6.1.1.3 and 6.0.3.7
2.6.8 Build-614 player encoder dispatch
2.7 File transfer
2.7.1 Requests and replies
2.7.2 The `PLO_FILE` body
2.7.3 The layout follows the server's token table
2.7.4 Requests at login
2.7.5 The client's request lists
2.7.6 Saving a download
2.7.7 Finding a local file
2.7.8 Unpacked Windows 6.1.1.3 packages
2.7.9 Exact original Windows 2.05r2 receiver
2.8 Chat and private messages
2.8.1 Local chat
2.8.2 Private messages
2.8.3 Staff messages
2.8.4 Buddy and list text requests
2.8.5 Profiles
2.8.6 Unpacked Windows 6.1.1.3
2.9 UDP peer-to-peer
2.9.1 Setup and addressing
2.9.2 Datagram
2.9.3 Routing
2.9.4 UDP ids
2.9.5 Unpacked Windows 6.1.1.3
2.10 The 6.11 server-defined protocol
2.10.1 The connector request
2.10.2 The connector script
2.10.3 Framing and encryption
2.10.4 Id tables
2.10.5 The Windows 6.1.1.3 hello
2.10.6 Unpacked Windows 6.1.1.3
3 File formats
3.1 Levels: Z3-V1, GR-V1, GLEVNW01
3.1.0 Format selection
3.1.1 Z3-V1 (.zelda)
3.1.2 GR-V1 (.graal)

3.1.3 GLEVNW01 (.nw)
3.2 GRMAP001 and gmaps
3.2.1 Three kinds of map
3.2.2 GRMAP001 as the server reads it
3.2.3 GRMAP001 as the tools write it
3.2.4 Client readers by build
3.2.5 Text big map and minimap (2.05)
3.3 GANI animations
3.3.1 File
3.3.2 Grammar
3.3.3 Sprite rows
3.3.4 Header
3.3.5 Sprite mode body
3.3.6 Movie mode body
3.3.7 Shared Mac v5 loader boundary
3.4 GOBJSET01 object sets
3.4.1 Layout
3.4.2 Shipped sets
3.5 Tile types and arrays.dat
3.5.1 Tile sheet geometry
3.5.2 Tile type values
3.5.3 arrays.dat
3.5.4 arrays2.dat
3.5.5 graal.sig and arrays2.sig
3.6 Sign text alphabets
3.6.1 Encoding
3.6.2 Corrections to earlier notes
3.6.3 GLEVNW01 signs
3.6.4 Reference decoder
3.7 Save games (graalsav)
3.7.1 Container
3.7.2 Revisions by build
3.7.3 File layout
3.7.4 Player record
3.7.5 DES-wrapped local files
3.7.6 Server account file (GRACC001)
3.8 Web level cache (GWEBL001)
3.8.1 When it is used
3.8.2 Path
3.8.3 Encrypted file
3.8.4 Plaintext
3.8.5 Level body as cached
3.8.6 Tool
3.8.7 Unpacked Windows 6.03 weapon-cache acceptance
3.8.8 Exact Linux 6.03 local-file helper
3.8.9 Linux 6.x class cache
3.9 Config and dat files
3.9.1 Where settings live
3.9.2 graalconfig.txt file rules
3.9.3 Keys in file order (2.05)
3.9.4 Key bindings
3.9.5 Registry values
3.9.6 Keys by build
3.9.7 Dropped files
3.10 Image and sound asset conventions
3.10.1 Image sheets
3.10.2 The levels tree
3.10.3 Sound and music in 2.05

3.10.4 Sound libraries in 2.17, 2.22 and 2.31
4 Client behaviour by version
4.0 Reading this Part
Chapters
4.1 Frame loop and timing
4.1.1 The tick
4.1.2 What drives the tick, by build
4.1.3 Periodic work
4.1.4 Movement updates on the wire
4.1.5 Durations in ticks
4.1.6 Clocks in the later engines
4.2 Movement and collision
4.2.1 The step
4.2.2 The wall test
4.2.3 What blocks a point
4.2.4 Refused steps, nudges and snaps
4.2.5 The blocked-step counter and the push pose
4.2.6 Ledge jumps
4.2.7 Pushing blocks and NPCs
4.2.8 Grab and pull
4.2.9 The push and pull trigger actions
4.2.10 Water and lava
4.2.11 Following links
4.2.12 Gates that stop movement (2.22, 2.31)
4.3 Combat, damage, hurt and knockback
4.3.1 Who judges a hit
4.3.2 The sword swing
4.3.3 The sword hit test (victim's side)
4.3.4 Sword damage
4.3.5 Other damage sources (2.05)
4.3.6 Hits by message
4.3.7 The hurt routine and hurt mode
4.4 Death clocks and respawn
4.4.1 Life-states
4.4.2 The death sequence (2.05)
4.4.3 Respawn
4.4.4 The level-entry save
4.4.5 Server side (GServer-v2)
4.5 Items, drops and board respawn
4.5.1 Matching a 2 × 2 object
4.5.2 Lift, carry, hide and throw
4.5.3 Sword cut
4.5.4 Chests
4.5.5 Item drops: who rolls
4.5.6 Items on the floor (2.05 rev 2)
4.5.7 Board respawn on the client
4.6 HUD, sign panel, chat text and options
4.6.1 The status display
4.6.2 The sign panel (2.05)
4.6.3 Chat text and nicknames
4.6.4 The item list (2.05)
4.6.5 Options at run time (2.05)
4.7 Sound
4.7.1 Placing a sound (2.05)
4.7.2 GS1 sound commands (2.05)
4.7.3 Which action plays which sound (2.05)
4.7.4 lift.wav and lift2.wav
4.8 Offline and training mode

4.8.1 What offline means
4.8.2 Ways in (2.05)
4.8.3 Where offline levels come from
4.8.4 The tutorial world
4.8.5 Windows 4.04r2: startup and an offline transition
4.9 The built-in editor
4.9.1 Where the editor lives
4.9.2 Switching between play and editing (2.05)
4.9.3 Undo (2.05)
4.9.4 Play, flags and the map image (2.05)
4.9.5 Saving and opening (2.05)
4.10 Other client features
4.10.1 Integrity checks
4.10.2 Animation state as others see it
4.10.3 Key emotes (2.05 rev 2, same routine in 1.39)
4.10.4 Picking up and dropping items online
4.10.5 The player list and social lists
4.10.6 Start message and disconnects
4.11 Precursors, side builds and the 4.x clients in play
4.11.1 Zelda Online and Java Graal
4.11.2 The relay room bridge
4.11.3 The Game Boy Color demo
4.11.4 The 39ster client and GraalX
4.11.5 4.04r2 and 4.20r8 live
4.11.6 2.04 to 2.05 rev 2: what changed in the code
4.11.7 The 2.05 rev 2 client's parts
4.11.8 Client features 2.00 to 2.31 by release notes
4.11.9 Crossing gmap edges (2.22, 2.31)
4.11.10 What each executable names, 1998 to 2.31
4.11.11 Android Classic and Era, 1.6 to 2.2
4.11.12 What the early Windows clients show when started
5 Game servers
5.1 GServer13 (1999)
5.2 39ster 2.1 GServer
5.3 GServer-v2 reference behaviour
5.4 Accounts and allowed versions
5.5 NPC server
6 Listserver
6.1 Requests
6.2 Replies
6.3 Login-type matrix
6.4 Server-list records
6.5 Game server link
7 RC and NC
7.1 RC login and commands
7.2 NC login
7.3 NPC, class and weapon messages
8 NPC scripting
8.0 Reading this Part
Chapters
8.1 GS1 language
8.1.1 Where scripts run
8.1.2 Compilation
8.1.3 Execution
8.1.4 Timeouts and sleep
8.1.5 Events
8.1.6 State flags
8.1.7 Flags, variables and persistence
8.1.8 Syntax and operators

8.1.9 Execution contexts
8.1.10 The script corpus
8.1.11 Message codes (exact 2.05 rev 2)
8.1.12 GS1 on Linux 3.04
8.2 GS1 reference by version
8.2.1 Argument signature letters
8.2.2 Name tables
8.2.3 Online side effects
8.2.4 Commands of 2.05 rev 2
8.2.5 Functions and variables
8.2.6 Constants
8.2.7 The command table by build
8.2.8 Superseded readings
8.2.9 Queued move consumer (exact 2.05 rev 2)
8.2.10 Exact-build object getter semantics
8.3 GS2 language
8.3.1 Where GS2 appears
8.3.2 Delivery
8.3.3 The script machine in the 4.2, 5.0 and 6.00 engines
8.3.4 Events, waits and the event queue
8.3.5 Names and the published API
8.3.6 Tools, compilers and reimplementations
8.4 GS2 bytecode
8.4.1 Transport
8.4.2 Format
8.4.3 OpenGS1 bytecode (reference reimplementation)
8.4.4 Dispatch by build
8.4.5 Built-in names in unpacked 6.1.1.3 and 6.0.3.7
8.4.6 Load-time stages and fusion
8.4.7 Scripts embedded in the Linux clients
8.4.8 Network GS2 on Linux 6.03
9 Graal3D (2000)
9.0 What ships and how it relates to 2D Graal
9.1 Protocol
9.2 Props and accounts
9.3 Levels (.x)
9.4 Camera
9.5 Collision
9.6 Client behaviour
10 Torque-era 3D (3.x to 5.12)
10.0 Reading this Part
10.1 Engine lineage
10.2 Join sequence
10.3 Handshake by build
10.4 DSO v33
10.5 Object model
10.6 3D formats
10.7 Linux 3.04: startup and compiled-script reading
10.8 Linux 3.04: packaged Torque streams and the later trees
10.9 Linux 3.04: a seven-operation executor subset
10.10 Linux 4.20r6: native-object registration
10.11 Linux 3.04: show-image wrapper constants
10.12 Linux 3.04: local value helpers
11 Tools and the formats they define
11.0 Tool map
11.1 GraalEditor
11.2 GraalShop
11.3 Generators
11.4 gif2mng

11.5 RemoteControl and NPC Control
11.6 Level editor
11.7 Other tools
Appendix A. Message index
A.1 Client to server
A.2 Server to client
A.3 Former names
Appendix B. Prop index
B.1 Player, NPC and baddy props
Appendix C. Version matrix
C.1 Wire and scripting by era
C.2 Server-to-client ids handled, 1.33 to 2.1 tests
C.3 Prop parsers
C.4 UDP ids handled
C.5 GS1 command table size
C.6 Generation 2 builds, 1.21 to 1.32
C.7 Lab runs of later real clients
Appendix D. Glossary
Appendix E. Open questions and disagreements
Appendix F. Source map
Appendix G. Evidence-tag legend
G.1 Kind-of-evidence tags
G.2 Build tags
G.3 Combining tags
Sources

0 Front matter and conventions

This chapter fixes the scope, structure, citation rules and notation of the Graal Technical Reference Manual. Every later chapter uses the terms defined here and does not redefine them.

0.1 Scope

The manual describes **Graal itself**: its clients from the 1998 relay builds through 6.x, its wire protocol, file formats, game servers, listserver, RC and NC, NPC scripting, Graal3D, the Torque-era 3D clients, and the tools that define formats.

These are out of scope:

- Any reimplementation's own design, including Seihai's boards, systems and gap lists. These notes are marked `excluded` in `manual/sources.tsv` (→ App. F).
- How the notes were produced: lab procedure, the order of discovery, and tooling for the notes themselves (the navigator, the release builders).
- Trainers and hacks, except where they document a client integrity check as behaviour (→ §4.10).

An implementer should be able to build a compatible client or server from the manual alone. Where the notes do not reach that far, the gap is listed in App. E rather than filled.

0.2 Structure and numbering

The manual is numbered the way a standard is: Parts, chapters inside a Part, and sections inside a chapter. The number tells you where a fact lives, and every cross-reference in the text uses it. The table gives each level, how its number is formed, and the file that holds it.

Unit	Form	Example	File
Part	one number	§2 Wire protocol	`manual/02-protocol/00-part.md`
Chapter	part.chapter	§2.2 Ciphers and compression	`manual/02-protocol/02-ciphers.md`
Section	chapter.section	§2.2.2 Generation 3 insertion	heading inside the chapter
Single-file part	number, then sections	§5.1 GServer13 (1999)	`manual/05-servers.md`
Appendix	letter	App. A Message index	`manual/app-a-messages.md`
Appendix entry	letter + # + number	`App. E#12`	an `### E#12` heading in App. E

Every heading that can be cited begins with its number. Part 0 is this chapter.

Cross-references. `→ §2.2.2` points to a section; `→ App. E#12` points to an appendix entry. A cross-reference replaces a repeated fact: every fact has exactly one canonical section, and other sections point to it. The checker (`tools/manual/check_manual.py`) fails on a reference that does not resolve to a heading.

Version differences go in columns or rows of the same table, never in parallel chapters. A rule that changes across builds is one table with one row per build, or per range of builds where the notes measured the whole range.

0.3 Evidence tags and citations

Every factual statement carries an **evidence tag** and the **path of the note** it comes from, in this form:

```
[tag] (path/to/note.md#anchor)
```

- The tag says what kind of evidence backs the claim. The full legend is App. G; build tags such as `[2.05]` mean "read from that client build".
- The path is relative to the repository root. The `#anchor` is optional and names the heading in the note.
- Several tags may share one path: `[srv][2.05] (protocol/codec-and-framing.md)`. Several sources are separated by `;` inside the parentheses, or written as several tag-and-path pairs.
- In a table, the tags and paths go in a **Source** column. Each body row carries its own source.
- A sentence that only arranges tagged facts, such as a cross-reference or a table caption, carries no tag.

`[inf]` marks an inference that the source note itself marked as one. The manual never upgrades an `[inf]` to a measured tag, and never adds an inference of its own without `[inf]`.

The manual adds **no new measurements**. A fact missing from the notes goes to App. E, or to a lab note in the notes first, before it can appear here.

0.4 Table cells

A table cell holds exactly one of:

Cell	Meaning
a value	measured or read, with the row's source
`unmeasured`	no build in the notes was measured for this cell; no neighbouring build's value is copied in
`disputed → App. E#n`	two notes disagree; App. E quotes both and says what would settle it
`n/a`	the field or rule does not exist in that build or generation
–	used only where a source table itself leaves a cell empty and gives no reason; treated as `unmeasured`

A value is never interpolated from builds on either side.

0.5 Numbers, byte order and bits

Numbers in this manual are written the way the source notes write them, so a value can be compared with a capture or a disassembly by eye. The table fixes how hexadecimal, byte strings, bits and byte order are shown.

Item	Convention
Hexadecimal	`0x` prefix, lower or upper case as in the source note: `0x20`, `0x04A80B38`
Decimal	no prefix; message ids are given in decimal
Byte strings	hex pairs separated by spaces: `FF FF FF`
Bit numbering	bit 0 is the least significant bit; a mask is written in hex (`0x7F`)
Bit streams	read least-significant-bit first unless the section says otherwise (→ §3.1)
Multi-byte integers	each layout names its byte order; there is no global default
`u8`, `u16be`, `u16le`, `u32le`	raw unsigned integers of 8, 16 or 32 bits, big- or little-endian
Signed values	a signed field is named `s8`, `s16le` and so on; a G-type sent as signed is its unsigned bit pattern[1]
Text	bytes as sent, with no character set implied; `\n` is the byte `0x0A`

0.6 Field types

Graal's own integer types add `0x20` to each 7-bit group so that a message stays printable and never contains `\n`[1]. Their encodings are defined once in §2.1.1. Field layouts in this manual use these short names:

Name	Short	Size	Defined in
GCHAR	`c`	1 byte	→ §2.1.1
GSHORT	`s`	2 bytes	→ §2.1.1
GINT, GINT3	`i3`	3 bytes	→ §2.1.1
GINT4	`i4`	4 bytes	→ §2.1.1
GINT5	`i5`	5 bytes	→ §2.1.1
GSTRING	`str`	c length, then that many bytes	→ §2.1.1
rest of message	`rest`	all bytes up to the terminator	→ §2.1.2
raw integer	`u8`, `u16le`, …	as in §0.5	→ §0.5
half-tile value	x×2	the value is in half-tiles; the reader divides by 2	→ §0.8

These short names follow the notation of the protocol notes[2]. A layout is written left to right in wire order, for example `c x, c y, str text`.

0.7 Messages and their ids

Graal's wire vocabulary has a few words that are easy to mix up: a message is not a frame, and an id is not the byte on the wire. The table defines the terms that Part II and the appendices use.

Term	Meaning
message	one logical unit: an id, a payload and, in most generations, a `\n` terminator (→ §2.1.2)
frame	one transport unit that carries one or more messages (→ §2.1.4)
PLI	a message from client to server; names `PLI_*`
PLO	a message from server to client; names `PLO_*`
UDP id	a message on the peer-to-peer UDP channel (→ §2.9)
logical id	the decimal id used throughout this manual
wire byte	the byte on the wire; for a GCHAR id it is the logical id + `0x20`[3]

Message and prop **names** are the conventional ones the notes use, restated rather than copied from any server source[3]. Every id appears in App. A (messages) or App. B (props) with a pointer to its canonical section.

0.8 Units

Sizes and times are given in the client's own units, so a value can be checked against the client without conversion. The table defines each unit and how it relates to the others.

Unit	Definition
pixel (px)	one pixel of the 2D game view
tile	one board cell; 16 px on a side, since a walk step of 8 px is half a tile[4]
half-tile	8 px; the unit of most coordinate fields in 2D messages[2]
board	one level's tile grid; its size per format is in §3.1
tick	one step of the client's game loop; its length per build is in §4.1
seconds	used only where the source measured wall time; ticks are not converted to seconds unless the source gives the tick length for that build

0.9 Versions, builds and generations

Five numbering schemes describe "which Graal" a fact applies to. They change on different calendars and are never mixed.

Scheme	What it names	Written as	Canonical table
build	one client executable, by its release name and revision	`2.05 rev 2`, `1.39`, `4.20r8`; in tags `[2.05]`	→ §1.1
archive name	the archive a build was found in	`5thgraalalpha`, `graal121`	→ §1.1
version token	the login version string a build sends	`GNW03081`, `G3D14097`	→ §1.2
client type	the login type value that selects a server code path	type 0, 4, 5, 6	→ §2.3
encryption generation	the transport scheme, in the GServer-v2 numbering 1–6	gen 3	→ §2.2.1
level-format generation	the level file family and revision	Z3-V1.03, GR-V1.02, GLEVNW01, GRMAP001	→ §3.1
save generation	the save-game magic	Graalsav5, Graalsv11	→ §3.7

The encryption generation numbering is that of GServer-v2, as the notes use it[1]; the build catalogue uses the same numbering[5]. PE FileVersion is recorded but is not used as a build name, because it is sometimes stale (2.05 still says 2.0.4.0)[5].

Eras. For brevity the manual groups builds into eras: the 1998 relay and Zelda Online, 1.x, NewMain/NewWorld, 2.x, 3.x, 4.x, 5.x, 6.x (including the 6.11 server-defined protocol), Graal3D (2000) and the Torque-era 3D clients. An era name is a heading aid only. A fact is always attached to the builds it was measured on, never to a whole era by default.

0.10 Servers, hosts and redaction

The repository is public, and the manual follows its redaction rules. No real server, account or address appears. The table says how hosts, ports, accounts and binary locations are written instead.

Item	Convention
Public hosts	never named; written <listserver-host>, <login-host>, <public-host>, <lan-host>, or described ("the 1999 list host")
Lab hosts	"the loopback address"; no dotted literal
Ports	lab ports only (14802, 14803, 14804, 14900, 14921–14923); a historical default port that is part of a format or protocol is given as the notes give it
Accounts	the clients' built-in guest login only; no other account names, passwords or license secrets
Binary internals	no code addresses, no disassembler names, no decompiled text, no raw string dumps
Outside projects	GServer-v2, gs2lib and similar sources are summarised and cited through the notes, never quoted

These rules come from the repository's agent rules[6] and are enforced by the checker (→ §0.11).

0.11 Checking and building

The manual is checked by scripts in `tools/manual/`, and the same scripts build the PDF. The table lists what each command does, so a contributor can run the checks CI runs.

Task	Command
check the manual	`python tools/manual/check_manual.py`
check before the final audit	`python tools/manual/check_manual.py --final` (also fails on any `pending` row in `sources.tsv`)
check that cited notes are carried	`python tools/manual/check_carry.py` (a `cited` row with no reason fails when a planned section does not cite the note, or when under 30 % of a Markdown note's distinctive terms appear in the manual; `--report` lists the lowest carries)
run the checker's tests	`python -m pytest tools/manual -q`
build the PDF	`python tools/manual/build_manual.py` (pandoc, then Chromium prints A4; default output `manual/pdf/graal-trm.pdf`; `--version X` labels the edition and names the file `graal-trm-X.pdf`; when a page is A3, it also writes the print set: `graal-trm-X-a3-inserts.pdf` with the A3 pages and `graal-trm-X-print-a4.pdf` with a placeholder page at each insert's place)

Each manual file begins with a header comment `<!-- manual: kind=K status=S -->`:

Field	Values	Effect in the checker
kind	`reference`	every body line, list item and table row needs a tag and a path, except cross-references and captions (a line ending in `:`)
kind	`front`	definitions and rules; tags and paths are checked where present
kind	`index`	headings and cross-references only
status	`stub`	only headings, cross-references and > status notes are allowed
status	`draft`, `complete`	the kind rules apply; a > `Status:` line may say what is still missing

`manual/sources.tsv` lists every note in the repository with one of three states: `pending` (not yet represented), `cited` (with the sections that cite it) or `excluded` (with a reason). App. F is its readable form.

A3 insert 1 of 1

0.12 System map

This page is printed separately on A3 landscape paper.

Print page 1 of **graal-trm-0.0.5-a3-inserts.pdf** and put it here.

In the digital edition, graal-trm-0.0.5.pdf, it is page 13.

1 Client generations and builds

This part names every client build the notes cover. Every other part keys its version columns off the build names in §1.1 and the schemes defined in → §0.9.

1.0 Reading this Part

A Graal build is named by what it sends, not by the number on its installer. Every other Part keys its version columns off the builds in this Part, and the safest key is the login token, because two builds with the same marketing number can speak different protocols and two different numbers can speak the same one. 3.04 carries 2.22's date stamp and speaks 2.22's protocol; Windows and Linux 6.03r7 send different tokens; the Windows 6.113 upload is not the Mac 6.11 bundle.[7,8,9]

Four kinds of program carry the name. From 1998 to 2005, 1st Graal to 2.31, the client is one Delphi executable, and its wire changes through the five encryption generations of §1.4. The 3.x clients of 2004 and 2005 are a C++ rewrite on the Torque Game Engine that keeps the 2.22 and 2.31 protocols. From 4.04r2 to 5.12 on Windows a Delphi launcher drives a C++ engine DLL. From 5.24, and in every 6.x build, launcher and engine are one native C++ program per platform.[10] The thirteen eras of §1.6 subdivide these four; the Torque years are Part X's subject (→ §10.0).

The line is not monotonic. 2.00 to 2.04 drop the `.nw` reader that 1.41 rev1 and NewMain 1.42 already had, and 2.05 brings it back.[5] NewWorld beta 1 was cut from the 2.05 code and beta 2 from the 2.1 test code.[11] The 6.00 betas of January 2010 are older than 5.24r3 and share its token.[8] A table sorted by version number is therefore not a table sorted by code; when the two disagree, this Part follows the code.

The evidence changes with the era. Most clients from 1998 to 2001 sit in one archive of 73 executables, so §1.1.1 can give each a hash, a link date, a FileVersion and the magics it reads.[12] Later builds are known from their tokens, lab runs and static reads, and there is no per-build hash catalogue for them.[13] Several gaps are real absences: of 2.1 to 2.17 only 2.17.1 survives as a binary, and 2.3 has no binary at all.[10] From 5.12 the Windows builds are wrapped by MoleBox or Themida, so what a static read can see depends on whether an unpacked image exists.[8]

How the Part is laid out. §1.1 lists the builds; §1.2 to §1.5 give the per-build keys a server or a reader needs (token, server version id, encryption generation, level and save magics); §1.6 groups them into eras; §1.7 covers the Java precursors and side builds. §1.8 is the practical half: for each generation, what a client has to do to *be* that build, with the machine-readable tables in §1.8.8. §1.9 gathers the identity and platform records whose job is to say what an archived file is, and how far that identity reaches.[14]

1.1 Build catalogue

1.1.1 Delphi and Java clients, 1998 to 2001

Most 1998 to 2001 executables come from one shared archive of 73 executables in 55 folders[12,15]. *FileVersion* is read from the PE version resource, not from program logic[16]. One row per executable, sorted by link date. *Compiled* is the resource-directory timestamp in local time; *Gen* is the encryption generation (→ §1.4); *Levels and saves* lists the level, map and save magics the loader compares against (→ §1.5). Default hosts and list names are in → §6.

Archive	Release	Compiled	FileVersion	MD5	Size	Token	Gen	Levels and saves
zeldaonline2	Zelda Online (Java applet)	1997–98	–	–	–	none	1998 relay (→ §2.3.1)	`Z3-V1.02/1.03`[5]
1stgraal-1	first Delphi beta	1998-09-06 23:54	–	07452f8941fdfcdce878b035156e701c	654336	none	1	Z3-V1.03/1.04 only; Graalsav1[5]
2ndgraal	beta	1998-09-16 01:33	–	f74b35fc7975a4873087f7fd2aa1d79b	777216	none	1	+ GR-V1.00–1.01; Graalsav1[5]
3rdgraal	beta	1998-09-30 01:47	–	fc78fbc96bdbc6ca22ae56d3ed8635ab	665600	none	1	GR-V1.00–1.01; Graalsav1[5]
4thgraal	beta	1998-10-05 01:33	–	e040af2e19912419ab80aa6dd202b4ea	666112	none	1	GR-V1.00–1.01; Graalsav1[5]
4thgraalalpha	beta (alpha)	1998-10-12 22:54	–	8cae23a58e1b1b10f3149bdbe53d0431	711680	none	1	GR-V1.00–1.01; Graalsav2[5]
5thgraal	beta	1998-10-19 02:27	–	0e7c3573f0a916f8b2376060e768fe96	723968	none	1	GR-V1.00–1.02; Graalsav2[5]
5thgraalalpha	beta (alpha)	1998-10-29 02:21	–	e54397e7d4f59d30b77d5cf0475335a1	739840	none	1	GR-V1.00–1.02; Graalsav2[5]
6thgraal	beta5	1998-11-09 02:08	–	53c3caf3d930534db3f44f346f5d651a	750080	none	1	GR-V1.00–1.02; Graalsav3[5]
7thgraal	beta7	1998-12-04 00:28	–	deb40c31268e499fc4436b8d0687f05d	766976	none	1	GR-V1.00–1.03; Graalsav4[5]
8thgraal	beta8	1998-12-08 01:13	–	a49aa1b773e41741f211867fd5ea55d7	775680	none	1	GR-V1.00–1.03; Graalsav4[5]
9thgraalalpha	beta9 (alpha)	1998-12-16 02:36	–	e226e1b83fd437b0aede968fb395b911	788992	none	1	GR-V1.00–1.03; Graalsav5[5]
9thgraal	beta9	1998-12-18 01:53	–	f0185ff8525a13f2f4e5997f649bc250	793088	none	1	GR-V1.00–1.03; Graalsav5[5]
10thgraal	V1.0	1998-12-22 03:24	–	f1f34b4479a8ad7614bd8f7ce56112c6	795648	none	1	GR-V1.00–1.03; Graalsav5[5]

Archive	Release	Compiled	FileVersion	MD5	Size	Token	Gen	Levels and saves
11thgraal	V1.0	1998-12-23 01:00	–	a255212621907affc94a4483c6dfc8e3	795648	none	1	GR-V1.00–1.03; Graalsav5[5]
12thgraal	V1.0	1999-01-05 01:04	–	788ae59e9419806024d14d3e5b611971	794624	none	1	GR-V1.00–1.03; Graalsav5[5]
13thgraal	V1.1 beta1	1999-01-12 01:15	–	a211a0ecf3b58daa2f46759444e18bd3	797184	none	1	GR-V1.00–1.03; Graalsav6[5]
14thgraal	V1.1	1999-01-15 02:20	–	d0999d44fcd9c226f350fcf204618885	858112	none	1	GR-V1.00–1.03; Graalsav7[5]
15thgraal	V1.1	1999-01-25 00:12	–	9a7d66dd5c8c506ad4d3e63ee4a9a1f1	878592	none	1	GR-V1.00–1.03; Graalsav7[5]
graal1.1.9	V1.1.9 Gold Edition	1999-02-11 23:50	–	0a6eef88acb4c0fce6e84eee3e76c04a	882176	none	1	GR-V1.00–1.03; Graalsav7/8[5]
graal1.1	V1.1 (readme)	1999-02-14 23:03	–	1a9580655cce686c602c01daa251c453	882176	none	1	GR-V1.00–1.03; Graalsav7/8[5]
graal121	V1.2.1	1999-05-10 18:29	–	8da2e8284dd8b315a28e2d4fccae79ac	973824	GSERV007	2	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11[5]
graal128b2	v1.28 beta2	1999-11-23 01:09	–	8e3c1a7233864da994a0db26f1811212	1196544	GSERV013	2	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11[5]
graal13b	v1.3 test	1999-11-29 02:03	–	b62aaa4d606a519e7f4689b33983429b	1311744	GSERV014	2	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11[5]
graal131	V1.31	1999-12-07 02:56	–	bff87a95a34399b2d35b243f281169a7	1318400	GSERV016	2	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11[5]
graal132	V1.32	2000-01-12 01:13	–	42b2596af4b75bcf82db03e0fc274f7d	1328128	GSERV018	2	GR-V1.00–1.03; reads Graalsav9/Graalsv10/Graalsv11, writes Graalsv11[5]
graal3d	Graal3D	2000-02-05 01:57	–	305c985a020cec6ab8d81ac95d447343	605184	none	own (→ §9)	none[5]
graal133	V1.33	2000-02-21 00:36	–	a3667ce55a5c67cf025882014b9540b0	1350144	GNW20020	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graal134	V1.34	2000-03-23 00:38	–	9e403616e73a6079ac37d65d475c81cd	1349632	GNW22030	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]

Archive	Release	Compiled	FileVersion	MD5	Size	Token	Gen	Levels and saves
graal135	V1.35	2000-04-13 23:17	–	fe9d99c4bf6e6bc9b1b127823f39646f	1389568	GNW13040	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graal136	V1.36	2000-05-12 17:34	–	018f51662c7c8862cafb9bfbd14dea41	1399296	GNW10050	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graal137	V1.37	2000-05-23 02:40	1.3.7.0	5e966a9d1a079af6d682b2618674edb1	1401344	GNW21050	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graal138	V1.38	2000-06-23 22:11	1.3.8.0	4401f7d02966af709c2df662cebe34a1	1426432	GNW22060	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graal139	V1.39	2000-08-07 02:49	1.3.8.0	62513ab884f1802a0bcd8114abe349b9	1448960	GNW07080	2	GR-V1.00–1.03; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graal_gr1411	V1.41 rev1 (-patched copy)	– (resource stamp missing)	1.4.1.1	a97740e711ffe9b698ec03712d4c06af	1506816	GNW13110	3	GR-V1.00–1.03, GLEVNW01; carries Graalsav9/Graalsv10/Graalsv11 (string scan)[5]
graalnm24	NewMain client (1.42 notes)	2001-01-10 19:55	1.4.1.1	229f05de7a43b98e223d8e1b58bffc63	1783296	GNW08011	3	GR-V1.00–1.03, GLEVNW01[5]
graalnm26	NewMain client (1.42 notes)	2001-01-12 21:19	1.4.1.1	22a08ed154f67948e01b969b68cdb05e	1784320	GNW08011	3	GR-V1.00–1.03, GLEVNW01[5]
Graal-v2.0	v2.00	2001-02-01 14:54	2.0.0.0	c93e659e2cda36379aaea4706db20b38	1812480	GNW24011	3	GR-V1.00–1.03 (no `.nw`)[5]
Graal-v2.0-rev1	v2.00 rev1	2001-02-02 19:19	2.0.0.1	30bd230e578d3a347866a87060efec20	1812992	GNW24011	3	as above[5]
Graal-v2.0-rev2	v2.00 rev2	2001-02-05 02:32	2.0.0.2	fc4bb7e3b7ca9cd8ea6693a033635534	1815040	GNW24011	3	as above[5]
Graal-v2.0-rev3	v2.00 rev3	2001-02-07 01:05	2.0.0.3	1ce5c1662b65246ae1c6559af5ff71f0	1815040	GNW24011	3	as above[5]
GraalX (`.rar`, separate upload)	2.01-era side build	2001-03-07 21:40	2.0.0.3	d3ee52ef9d22a0d99b154b47c5f56f60	1821696	GNW23021	3	GR-V1.00–1.03, **GLEVNW01**[5]
Graal-v2.01	v2.01	2001-03-08 14:59	2.0.1.0	08340a8868f1d11e0285cc70e51200fe	1815552	GNW23021	3	as above[5]
Graal-v2.01-rev1	v2.01 rev1	2001-03-15 21:10	2.0.1.0	010a32afce66729b838258809a97c0d9	1815552	GNW23021	3	as above[5]

Archive	Release	Compiled	FileVersion	MD5	Size	Token	Gen	Levels and saves
Graal-v2.02	v2.02	2001-04-15 20:48	2.0.2.0	60018a19ef671c506743826728c49718	1808384	GNW15041	3	as above[5]
Graal-v2.02-rev1	v2.02 rev1	2001-04-15 23:16	2.0.2.0	e06ff79aded885a223b933617d456577	1808896	GNW15041	3	as above[5]
Graal-v2.02-rev2	v2.02 rev2	2001-04-20 18:48	2.0.2.2	993a1829dbb6acaa2b46063f5efe636c	1807872	GNW20041	3	as above[5]
Graal-v2.03	v2.03	2001-05-07 15:31	2.0.3.0	36972c74baad09ceb33dad94b0eb1e33	1836032	GNW06051	3	as above[5]
Graal-v2.04	v2.04	2001-06-25 07:42	2.0.4.0	a40608308d4f890cb5e4efdd45b38ba4	1872384	GNW24061	3	as above[5]
Graal-v2.05	v2.05	2001-08-03 19:12	2.0.4.0 (stale)	9abce477945a1307bb2be9ed47d2f170	1891328	GNW03081	3	GR-V1.00–1.03, **GLEVNW01, GRMAP001**[5]
Graal-v2.05-rev1	v2.05 rev1	2001-08-04 03:26	2.0.5.0	2780f2f370396c580e1648696698c871	1895936	GNW03081	3	as above[5]
graalnwb1	NewWorld beta 1	2001-08-06 04:13	2.0.5.0	88d1f84c96da1dcc3942b139fbd741c9	1820672	G2N15041	3	as 2.05[5]
Graal-v2.05-rev2	v2.05 rev2	2001-08-16 17:24	2.0.5.2	f3fdf54d943fa91c89acba5bb695db99	1891840	GNW03081	3	as 2.05[5]
graal21test2	v2.1 test 2	2001-09-21 17:54	2.1.0.0	d15d877822ad4145578107857c90fb3c	1748480	GNW03081	3	as 2.05[5]
graal21test3	v2.1 test 3	2001-09-27 21:11	2.1.0.0	2116a7ba03648dcc004035534ec94de8	1753600	GNW03081	3	as 2.05[5]
graalnwb2	NewWorld beta 2	2001-10-03 20:46	2.1.0.0	af2d21066659bed6af6c77e52b2cc0e1	1613824	G2N15041	3	as 2.05[5]

Notes on the catalogue:

- The "Nth graal" archive names are not chronological: 4thgraalalpha and 5thgraalalpha were linked after their namesakes and 9thgraalalpha before 9thgraal; graal1.1 has the same size as graal1.1.9 but was linked three days later[5].
- FileVersion exists only from 1.37 on and is sometimes stale (2.05 says 2.0.4.0; 1.39 says 1.3.8.0)[5].
- The only 1.41 copy (graal_gr1411) is patched by a third party: its resource timestamp is zeroed and its list host and port may not be the original values; the token and code-level facts look untouched[5].
- GraalX is 2.01 with the GLEVNW01 reader and writer kept in: 6350 functions identical to 2.01, 7 differing, and the same command table and packet and prop ids[5].
- graal_v2.05 rev 2 is the same file as `Graal_v2.05_rev2.exe`, the main decompile target of the notes[5].

1.1.2 Compilers

Three facts settle which compiler built a client: the PE header and runtime strings, the layout of Delphi's class metadata, and which runtime classes are linked. The table reads them per group of builds. The last row is the one place the clues disagree, which is why the early builds' Delphi version stays open[5].

Builds	Compiler	Evidence
all Delphi builds	Borland Delphi, linker 2.25, DelphiX for DirectDraw, RTL copyright "1983,97 Borland"	PE header and RTL strings[5]
2nd Graal; 1.28 beta 2 and later	Delphi 4	Delphi 3+ class-metadata layout (self pointer 76, name 44 bytes before the VMT) and `TMultiReadExclusiveWriteSynchronizer` linked[5]
1st Graal; 3rd Graal to 1.1.9; 1.21	disputed → App. E#1	Delphi 2 class-metadata layout (64 and 32 bytes) but a 1997 RTL string[5]

1.1.3 Companion executables in the 1998 to 2001 archives

The 1998 to 2001 archives ship more than the client. The table lists the servers and remote-control programs found beside it, one row per file, with its link date and MD5 so a copy can be matched. The Token and Port columns show what each program expects on the wire. A server that accepts a range of tokens can serve every client in that range[5].

Archive	File	Compiled	MD5	Token	Port
graal1.1.9	Gserver.exe	1999-01-22	7882dba6ebb2ab1026de260c3043fa47	none	14802[5]
graal121	GServer.exe	1999-05-09	bfcdaae0217d953124a9b21ff7858967	GSERV007	14804[5]
graal121	remotecontrol.exe	1999-05-10	5932d6baa08289a65515342dfc4b9f5a	GSERV007	14804[5]
graal128b2	GServerv.exe	1999-11-22	8d6bff85bda6e6c9558503afa6b7259d	accepts GSERV007–013	14804[5]
graal128b2	RemoteControl1.28.exe	1999-10-20	d7150d1385ea11b812d10d30212f9757	GSERV011	14804[5]
graal13b	GServer13.exe	1999-11-29	d1d0470e53a60b8f000b9ea5da9d37b6	accepts GSERV007–014	14804[5]
graal13b	RemoteControl13.exe	1999-11-29	1f2c19ab385c8b8af6236197f98ad735	GSERV011	14804[5]
graal3d	GServer.exe	2000-02-05	aab1a34a1f196d4f4fddf12112ccbf52	none	14901[5]
graalnm24/26	RemoteControl-NewMain.exe	2001-01-08	bc2b8a2e80acc09a6bb418970cff5a64	GSERV021	14804[5]
graalnwb1	RemoteControl-NW.exe	2001-05-22	6fe24bdd4e135900fc40d7fb70bdcdd2	–	14804[5]
graalnwb2	RemoteControl-NW.exe	2001-08-27	ce454437af8880f61ac2173b0bd11047	–	14804[5]

The archives also hold gif2mng, GraalShop, GraalLevelGen and GraalLevelGenNW, NPCControl, LevelFiller with levelviewer, TerrainGenerator, PNGView, YoshiChess and Greaditor[5]. Their formats and behaviour are in → §11.

1.1.4 Later builds

Builds after 2001 are known from their tokens, lab runs and static reads; there is no per-build hash catalogue for them in the notes. Tokens are in → §1.2.

Build	Token
2.1.0.0 to 2.1.0.3	GNW31101[13]
2.1.4.0 (first with server-list tabs)	GNW30042[13]
2.1.7.0, 2.1.7.1 (2.171, `Graal_2171.exe`)	GNW22122[13]
2.1.9.0 (bz2 and new encryption)	GNW15053[13]
2.2.1.1 (extra compression byte)	GNW01113[13]
2.22 (`Graal_222.exe`)	GNW03014[13]
2.3.0.0	GNW14015[13]
2.31 (`Graal_231.exe`; editor separate, editor FileVersion 2.3.1.1)	GNW28015[13]
Linux 3.04 (Torque rewrite, 2.22 protocol)	G3D03014[13]
Mac OS X 3.153 (2.31 protocol)	G3D28015[13]
Windows 4.04r2 (`GraalEngine.dll`, March 2006)	G3D17026[13]
Mac OS X 4.20r6	G3D27076[13]
Windows 4.20r8 (`GraalEngine.dll`)	G3D20126[13]
Windows 5.00r7 (engine DLL, Jul 2007)	G3D22067[13]
Windows 5.12	G3D14097[13]
Windows 6.00 beta1	G3D03048[13]
Windows 6.00 beta2, 5.24r3, 5.24r4	G3D15010[13]
Windows 5.30r3, 5.31r4, 6.00r2, 6.00r4	G3D26090[13]
Linux 6.01r1	G3D0108A[13]
Windows 6.03r7	G3D0311C[13]
Linux 6.03r7	G3D0511C[13]

From 2.3 the editor and the online client are separate executables[13]. Graal3D (2000) is a separate client with its own server and no Graal level loaders[5]; it is described in → §9.

1.2 Version tokens

The version token is the eight-character string a client sends in its login (→ §2.3).

1.2.1 Token schemes

Graal has used five token schemes. Each row gives the form of the eight characters, the builds that use it, and how to read the digits. The date schemes are the useful ones in practice, because the token alone then dates a build to within a few days[5].

Scheme	Form	Builds	Rule
none	—	1998 betas to 1.1.9	no token is sent[5]
protocol revision	`GSERV` + 3 digits	1.21 to 1.32, RC to Jan 2001	the number is a protocol revision: 007 (1.21), 011 (RC 1.28), 013 (1.28b2), 014 (1.3 test), 016 (1.31), 018 (1.32), 021 (RC-NewMain)[5]
build date	`GNW` + DDMMY	1.33 to 2.31	DD day, MM month, Y last digit of the year; the date is on or a few days before the link date[5,17]
NewWorld	`G2N` + DDMMY	NewWorld betas 1 and 2	same date form[5]
3D era	`G3D` + date digits	3.x to 6.x	the 3.x tokens carry the same date stamps as their 2.x protocol twins (G3D03014 and GNW03014); later suffixes include hex-like letters (G3D0311C, G3D0108A)[18,13]

A revision that does not change the protocol keeps the previous token: 2.0 rev1 to rev3, 2.01 rev1, 2.02 rev1, 2.05 rev1 and rev2, and the 2.1 test builds, which still send the 2.05 token GNW03081[5]. Among those revisions only 2.02 rev2 changes the token, and it also adds the `PLI_LANGUAGE` send after login[17,19,20]. From 1.38 the token is built one character at a time in code rather than stored as a string literal[5].

A server accepts a range of tokens rather than one: GServer 1.28 accepts GSERV007 to GSERV013 and GServer13 accepts GSERV007 to GSERV014[5]. A lab GServer-v2 admits a client when its token is in the allowed-versions list, which for 2.00 is a configuration entry, not a code change[13]. Server-side token handling is in → §5.4.

1.2.2 Token table

This is the lookup table behind the schemes: every token the notes have seen, in release order, with the build that sends it. A token that maps to several builds means those builds share a protocol revision. Rows marked from server source are tokens a server accepts but no surviving binary has been read for[5,18].

Token	Build
GSERV007	1.21[5,18]
GSERV009	1.25[18]
GSERV011	1.27; RemoteControl 1.28 and 1.3[18,5]
GSERV013	1.28 beta 2[5]
GSERV014	1.3 test (1.30)[5]
GSERV016	1.31[5]
GSERV018	1.32[5]
GSERV021	RemoteControl-NewMain[5]
GNW20020	1.33[5]
GNW22030	1.34[5]
GNW13040	1.35[5]
GNW10050	1.36[5]
GNW21050	1.37[5]
GNW22060	1.38[5,13]
GNW07080	1.39[5,13]
GNW12080	1.391[18]
GNW13110	1.41 rev1[5,18]
GNW08011	NewMain client (1.42 notes)[5]
GNW24011	2.00, rev1 to rev3[13]
GNW23021	2.01, 2.01 rev1, GraalX[13,5]
GNW15041	2.02, 2.02 rev1[13]
GNW20041	2.02 rev2[13]
GNW06051	2.03[13]
GNW24061	2.04[13]
GNW03081	2.05, rev1, rev2; 2.1 test 2 and 3[13]
G2N15041	NewWorld betas 1 and 2[5]
GNW31101	2.1 (2.1.0.0 to 2.1.0.3)[18,13]
GNW01012	2.12[18]
GNW23012	2.13[18]
GNW30042	2.14 (2.1.4.0)[18,13]
GNW19052	2.15[18]
GNW20052	2.151[18]
GNW12102	2.16[18]
GNW22122	2.17 (2.1.7.0, 2.1.7.1)[18,13]
GNW21033	2.18[18]
GNW15053	2.19 (2.1.9.0)[18,13]

Token	Build
GNW28063	2.2[18]
GNW01113	2.21 (2.2.1.1)[18,13]
GNW03014	2.22 (2.2.2.0) game login[18,13]
GNW14015	2.3 (2.3.0.0)[18]
GNW28015	2.31 (2.3.1.0) game login[18,13]
GNW30123	listserver hello of 2.22 and 2.31, sent with `newmain`; not a game build (→ §6)[13]
G3D16053	3.0[18]
G3D27063	3.01[18]
G3D03014	Linux 3.04 rev 1 (3.041)[18]
G3D28015	Mac OS X 3.153[18]
G3D30123	3.x listserver login[18]
G3D17026	Windows 4.04r2[18]
G3D27076	Mac OS X 4.20r6[18]
G3D20126	Windows 4.20r8[18]
G3D22067	Windows 5.00r7[13]
G3D14097	Windows 5.12[13]
G3D03048	Windows 6.00 beta1[13]
G3D15010	Windows 6.00 beta2, 5.24r3, 5.24r4[13]
G3D26090	Windows 5.30r3, 5.31r4, 6.00r2, 6.00r4[13]
G3D0108A	Linux 6.01r1[13]
G3D0311C	Windows 6.03r7[13]
G3D0511C	Linux 6.03r7[13]

The older GServer-v2 version table writes the 1.33 to 1.38 tokens without the leading `G` (`NW20020` and so on)[18]. The gs2lib table lists `G3D28095` and later for 4.x and up without per-build rows[18].

1.3 Version ids

Server code identifies a client by an internal version id; ids sort, so 2.05 rev 2 sorts below 2.1[18].

Version id	Build	Token
CLVER_1_39	1.39	GNW07080[18]
CLVER_2_052	2.05 rev 2; sorts below CLVER_2_1	GNW03081[18]
CLVER_2_17	2.17	GNW22122[18]
CLVER_2_22	2.22	GNW03014[18]
CLVER_2_31	2.31	GNW28015[18]

The notes give version ids only for the builds above; ids for other builds are unmeasured[18].

1.4 Encryption generation by build

The encryption generation decides how a client scrambles and compresses its frames, so it is the first thing a server must know about a build. Each row gives a generation, the builds that use it and how that was established. For generations 1 to 3 the evidence is in the binaries. For 4 to 6 it comes from server source[5,1].

Gen	Builds	How it was determined
1	1st Graal to 1.1.9	no zlib linked, no decompression-error option[5]
2	1.21 to 1.39	zlib 1.0.4 linked; the 1.2 notes say "packet compression"; `nodecompressionerror` from 1.28b2[5]
3	1.41 rev1, NewMain 1.42, 2.00 to 2.05, 2.1 tests, NewWorld betas, 2.17	the connection initialiser seeds the insertion LCG and draws a 7-bit key; 1.39 lacks it[5,1]
4	2.19 to 2.21 and 3.0 to 3.01 (client type 4)	server source[1]

Gen	Builds	How it was determined
5	2.22 and later (client type 5); RemoteControl as type 6	server source; 2.22 and 2.31 send type 5[1]
6	v6	not implemented in the server source the notes summarise[1]
own	Graal3D	own protocol[5]

The schemes themselves are defined in → §2.2. Generations of the 3.x to 6.x clients beyond these rows are in → §2.3.

1.5 Level-format generation by build

Each client reads a fixed set of level and map file magics and writes one save-file magic. The table follows both through the builds. A new save magic marks a change in the saved state, and a level magic that a row lacks is a format that build cannot open[5].

Builds	Level and map magics read	Save magic
Zelda Online (Java)	Z3-V1.02, Z3-V1.03	n/a[5]
1st Graal	Z3-V1.03, Z3-V1.04	Graalsav1[5]
2nd Graal to 4th Graal	+ GR-V1.00, GR-V1.01	Graalsav1[5]
4thgraalalpha	GR-V1.00 to GR-V1.01	Graalsav2[5]
5th Graal, 5thgraalalpha	GR-V1.00 to GR-V1.02	Graalsav2[5]
6th Graal	GR-V1.00 to GR-V1.02	Graalsav3[5]
7th, 8th Graal	GR-V1.00 to GR-V1.03	Graalsav4[5]
9th Graal to 12th Graal (1.0)	GR-V1.00 to GR-V1.03	Graalsav5[5]
13th Graal	GR-V1.00 to GR-V1.03	Graalsav6[5]
14th, 15th Graal	GR-V1.00 to GR-V1.03	Graalsav7[5]
1.1.9, 1.1	GR-V1.00 to GR-V1.03	Graalsav7/8[5]
1.21 to 1.39	GR-V1.00 to GR-V1.03	reads Graalsav9, Graalsv10, Graalsv11; writes Graalsv11[5]
1.41 rev1, NewMain 1.42	GR-V1.00 to GR-V1.03, GLEVNW01	as 1.21 (1.41 rev1); unmeasured (NewMain)[5]
2.00 to 2.04	GR-V1.00 to GR-V1.03 (no GLEVNW01, no GRMAP001)	unmeasured[5]
GraalX	GR-V1.00 to GR-V1.03, GLEVNW01	unmeasured[5]
2.05 and later 2001 builds	GR-V1.00 to GR-V1.03, GLEVNW01, GRMAP001	unmeasured[5]

GR-V1.03 is the last `.graal` revision any catalogued build reads[5]. `.nw` support is not monotonic: 1.41 rev1 and NewMain 1.42 read GLEVNW01, 2.00 to 2.04 contain neither GLEVNW01 nor GRMAP001, and both return in 2.05[5]. That 2.0 branched from a pre-NewMain tree is an inference from strings[5]. The 2.05 string table holds exactly Z3-V1.03, Z3-V1.04, GR-V1.00 to GR-V1.03 and GLEVNW01 as level magics[21]. The save writer was traced only in 1.21 and 1.32; for 1.33 to 1.41 rev1 the save magics are inferred from the same string arrangement[5]. The formats themselves are in → §3.1, → §3.2 and → §3.7.

1.6 Era timeline

The manual groups builds into thirteen eras. Inside an era the clients share their engine and wire format[10]. An era letter is a heading aid; facts stay attached to builds (→ §0.9).

Era	Builds	Dates	Engine
Z	Zelda Online 1 and 2 (Java applets), GServ Standalone	May to Jul 1998	Java[10]
A	1st Graal to 1.1.9 Gold (20 builds)	Sep 1998 to Feb 1999	Delphi 2-level compiler (2nd Graal: Delphi 4), DelphiX[10]
B	1.21, 1.28 beta 2, 1.3 test, 1.31, 1.32	May 1999 to Jan 2000	Delphi 4 from 1.28b2, DelphiX[10]

Era	Builds	Dates	Engine
3D0	Graal3D	Feb 2000	Delphi, DelphiX Direct3D retained mode[10]
C	1.33 to 1.39	Feb to Aug 2000	Delphi 4, DelphiX[10]
D	1.41 rev1	Nov 2000	as C[10]
E	NewMain 1.42, 2.00 to 2.05 rev2, 2.1 tests, NewWorld betas	Jan to Oct 2001	as C, plus DirectDraw 7[10]
F	2.1 to 2.17 (only 2.17.1 survives as a binary)	Oct 2001 to 2002	as E, BASS audio from 2.17 at the latest[10]
G	2.22, 2.3 (no binary), 2.31	2004 to 2005	as F[10]
H	3.04 Linux, 3.153 Mac	2004 to 2005	C++ on the Torque Game Engine[10]
I	4.04r2, 4.20r6, 4.20r8	2006 to 2007	Delphi launcher + C++ `GraalEngine.dll` (+ `Graal3DEngine.dll`)[10]
J	5.00r7, 5.12; then 5.24 to 5.31	2007 to 2011	5.00 and 5.12 as I; from 5.24 one MSVC executable[10]
K	6.00 betas to 6.03r7, 6.11 Mac	2010 to 2014	one native C++ program per platform, no Torque layer[10]

The era table above groups builds. This second table records the three code changes that cut across those eras, found by matching functions between builds against 2.05 rev 2[11].

Change	Between
largest single code step: about 800 more functions, partly new DirectDraw 7 and grid library code	1.41 rev1 and NewMain 1.42[11]
NewWorld beta 1 cut from the 2.05 code base; NewWorld beta 2 cut from the 2.1 test code base	2001[11]
2.1 test builds are a later branch of the 2.05 code: more functions than 2.05 but fewer unique to 2.05 rev2	Sep 2001[11]

Era-by-era wire, scripting and id-set differences are tabulated in → App. C.

1.7 Precursors and side builds

These builds sit around the Delphi client line. The uploads hold 9 distinct items under 14 names[22].

Date	Build	What it is
1998-05	Zelda Online 1	Java 1.0 applet, single player, no socket code; `Z3-V1.02` and `Z3-V1.03` levels, start `level1.zelda`[22]
1998-07	Zelda Online 2 "local version"	Java 1.1 application with one-player and multiplayer modes, the GServ Standalone lobby on 14801 and the author's commented source; 51 `Z3-V1.03` levels[22]
1998-07	ZeldaEditor	Delphi 3 level editor; writes only `Z3-V1.03`[22]
1999-09	Graal Game Boy Color demo	homebrew by the Graal author, built with GBDK 2.x; header title `GRAAL`, MBC1, 256 KB ROM of which banks 0 to 5 are used[22,23]
2000-07	Java Graal	Zelda Online 2's game and client with Graal art, reading `GR-V1` levels; relay on 14802 (→ §2.3.1.3)[22]
2001-03	GraalX	a 2.01 pre-release that keeps the `.nw` reader and writer (→ §1.1.1)[22]
2014-04	39ster client 1.00	an offline fan remake on an SDL engine with GameMonkey script; reads `.nw` and `GR-V1.00` to `1.03`; NPC scripts are not run[22]

The later Android Classic and Era packages, 1.6 to 2.2, are a separate line and are not counted above. Ten supplied files reduce to seven packages (builds 6108 to 6201), each with a Java host and native engines for four architectures, read statically only (→ §4.11.11).[24]

The rules below say what the precursors contributed to Graal, and what they share with it: dialogs, the sign alphabet, data files and level reading. Each row is one inheritance or difference[22].

Rule
1st Graal grew out of ZeldaEditor: it carries the editor's sign, link, object-name and sign-code dialogs under the same class names; the baddy dialog and the main form were renamed[22]
The Graal sign alphabet (→ §3.6) is already in Zelda Online 1[22]

Rule
Zelda Online 2's options hold a 10-character save code of 60 bits (position, level index, rupees, darts, power, item and story bits, a 6-bit count of set bits), XORed with a fixed mask and transposed before writing[22]
ZeldaEditor's `objects.dat` stamp list: a name line, one width byte and one height byte, then width × height 32-bit little-endian tile ids; tile 2047 is transparent; 105 stamps in 1998, and Graal 1.21 ships the same format[22]
The Java games read `arrays.dat` in the same header form as the Delphi clients (→ §3.5.3), in a fixed order of 14 lists[22]
Java Graal reads `GR-V1` NPC lines but shows only the image, dropping the script, and skips the chest section[22]
Zelda Online, the Java client of 1997–98, keeps text-line relay traffic on port 14801 (→ §2.3.1)[25]

1.7.1 Behaviour of the Java precursors

Zelda Online 2 and Java Graal share one engine, and every number below is the same in both[26].

What	Value
main loop	50 ms per tick (20 Hz)[26]
walk	half a tile per tick while a key is held; a quarter step sideways around a block corner[26]
sword	5 ticks, hits tested on ticks 1 to 3[26]
hurt	40 ticks, half a heart[26]
pushing a wall	push pose and ledge check from tick 18 (0.9 s); a push stone moves at tick 28 (1.4 s); the counter loops from 38 back to 18[26]
bush drop	one roll of 0 to 99: 25 % a green rupee, 5 % a heart, 2 % five rupees, else nothing; a lifted bush rolls the same[26]
regrowth	none during a visit; every board change is cleared when the level loads[26]
carrying	the player is drawn with carry sprites and the object above the head, not hidden under it[26]
chat	bold serif black text with a one-pixel white offset, above the head, with no bubble; the nickname is drawn the same way under the feet[26]

1.7.2 The Game Boy Color demo

The Game Boy Color demo is a separate homebrew program, not a port of the PC client. The rules record what was seen when it ran in an emulator and what the ROM holds[23,23].

Rule
No title screen and no HUD: the ROM goes from the Nintendo logo to room 0; a sign there reads as a welcome from the author[23,22]
Seven rooms of 32 × 32 tiles in banks 4 and 5; their links use the PC level link record with a room number for the file name[23]
Select opens a carousel over a table of 31 character types, and the pick changes the player at once[26]
The text font runs space, A–Z, a–z, 0–9, then punctuation, from byte 32; it is not the PC sign alphabet[22]
Start mutes the game; the music format is open[26]

1.8 Build profiles

This section gives, generation by generation, what a client has to do to *be* a given build: how it is packaged and started, which login and license files it reads, what the player sees, and what the lab runs measured. Wire layouts are in → §2, per-build id sets in → §2.5 and → App. C, and the machine-readable profile tables in → §1.8.8.

1.8.1 GNW 1.33 to 1.39

The seven gen-2 GNW clients share one wire and one feature base; every shipped `Graal.exe` matched its catalogued hash[14].

Rule
Wire: every flush is one u16be `length` + zlib stream, with no size threshold, no cipher and no key byte (gen 2, → §2.2)[14]
Login is message id 0 (`chr(32)`, which also marks a player client): the 8-byte token, then `chr(len+32)` + account, then `chr(len+32)` + password[14]
No listserver and no server list window: the connect dialog is pre-filled with `<list-host-classic>:14900` and the guest/guest account[14]

Rule
Levels: only GR-V1.00 to 1.03 (and Z3-V1 .zelda for the offline levels); no .nw, no .gmap. The board arrives as RAWDATA (100) then BOARDPACKET (101) in its own frame. Saves are written as Graalsv11; the reader also takes Graalsav9 and Graalsv10 (→ §3.7)[14]
Files: asked for with WANTFILE (23), never UPDATEFILE (34), no large-file ids; the reply is RAWDATA then PLO_FILE (102) with no modtime. A server adds .gif to image names with no extension for clients before 2.1[14]
No tampercheck (39). A peer UDP channel for "data packages" with noudp= and fixedudpport= (default 14899) (→ §2.9)[14]
Game modes: One Player, One Player Online, and Two, Three or Four Players on one machine; local multiplayer goes away in 2.0[14]
Window 640×480 (648×508 outer); Alt+Enter toggles fullscreen. F1 options, F2/F3 load and save, F4 editor, F5 load level, F6 debugger, F7 player list; Alt+1 to Alt+7 screenshots, minimap, bytes counter, snapshot and record[14]
Server-sent props 1–9, 11 and 13 parse in the pre-2.1 forms (sword c(power+30)+str, shield c(power+10)+str, head c(len+100)+image, X/Y in half-tiles; no X2/Y2). Prop 10 must be the bow form: a server that sends a gani string there is misread and the rest of that prop stream is lost[14]

Protocol steps inside the range[14]:

Build	Change
1.34	other-player prop 34 (account name); ignores are saved by account name (ignoredaccounts.txt)[14]
1.38	C→S + 32 ITEMTAKE (the server now counts picked-up rupees) and 33 WEAPONADD; − 25; S→C + 43 DEFAULTWEAPON; UDP + 24. Up to 1.37 the client uploads each held weapon with id 25 (name, image and the full script); from 1.38 it sends 33 as chr(0) + chr(default weapon id)[14]
1.39	profile window: C→S 80 PROFILEGET, 81 PROFILESET, S→C 75 PROFILE; own prop 34 ACCOUNTNAME, other-player prop 33[14]

gs2lib also knows GNW02070 (1.38 rev1) and GNW12080 (1.39 rev1); neither build is in the archive[14].

What the player sees change[14]:

Build	HUD, tiles and windows	Controls and rules
1.33	state.png 224×81 with the letters A, S, D on the item buttons; HUD digits at the bottom of sprites.png (128×484). Player list (F7) with GPs, Guild, Normal and Ignore; More Options has 13 checkboxes (vsync, joystick, music, sound, reverse stereo, save downloaded images, external chat, compatibility draw, hide decompression errors, fixed UDP port, no UDP, ignore mass messages, no DirectInput). The server message (41) shows as raw HTML	fixed keys: arrows or IJKL, D item, S or Space attack, A action, M map, Q item list, P pause, Tab chat[14]
1.34	F6 debugger rebuilt (Next, Run, Stop next call, Stop next script; this.vars, vars, flags and NPC attribute panes), offline only	hitting a mounted player costs the horse one bush, and at zero the rider is pushed off; two players share a horse offline; one sound per type per frame, eight at once; canbepulled separate from canbepushed[14]
1.35	a CHAT label in state.png marks the focused player in local multiplayer. Player list popup gains Buddy (buddyaccounts.txt); "Don't use HTML for PMs"; PMs render as HTML; a "(Senior GP)" tag	[14]
1.36	"Limit the length of nicknames to [50]"; an "(RC)" tag	server cap rises from 220 to 28000 players; scripts read bombs[], arrows[], items[], explos[], horses[][14]
1.37	Español and Italiano added	volume falls off with distance; showimg on normal NPCs; playlooped, stopsound, array literals, true, false, hasweapon()[14]
1.38	button letters removed from the art (the client draws A/S/D itself); digits move into state.png (224×97; sprites.png 128×452); 81 tiles change in pics1.png; editor gains "Show blocking tiles"; external chat removed; Swedish added; gif2mng ships	up to three keys per action in graalconfig.txt (up, down, left, right, item, attack, action, map, switchtochat, itemlist, pause); Alt+8/Alt+9 zoom; Ctrl+letter emotion icons from levels/emotions (AFK, BRB and Zzz stay until the player moves); a nickname equal to the account shows as *nick; server-checked pickup; npcs[], hitnpc, setlevel2, changeimgpart, test*(); explosions capped at 1024; PMs saved to pms/[14]

Build	HUD, tiles and windows	Controls and rules
1.39	7 tiles change; Profile and Options buttons in the player list; a Player List Options window; a Profile window (real name, age, sex, country, ICQ, e-mail, homepage, favourite hangout, quote); "Don't connect levels", "Ignore toalls", "Don't load heads"; Français added	Ctrl+C, K and Y; 100 lines of chat history on Up and Down; weapon list clickable; dead players don't block; editor paints by dragging and turns a dropped image into an NPC[14]

These rules are what changes across the seven GNW builds of 2000 and what a server must do to admit them. The last two rows come from live captures, not from the binaries[14,14].

Rule
The GS1 command table grows 88, 93, 93, 104, 106, 109, 111 over 1.33 to 1.39; `carryobject` and `throwcarry` are in the 1.34 table but announced only in the 1.37 notes, so being in the table does not prove a command works (→ §8.2)[14]
A server that sends an id outside the build's known set hits the default branch and the client ignores it (seen for 47, 82, 156, 168, 190 and 194)[14]
GServer-v2 beta4 needs `generation = classic` and the GNW tokens in `allowedversions.txt`, checks the account with a listserver although the client connects directly, and kicks 1.33 for id 25 ("Unknown Player Packet: 25"); a proxy that drops 25 lets 1.33 play. 1.39 logs in and plays[14]
After the level loads, the first id 2 carries props 0, 11, 13, 18, 31 and 33 in both 1.33 and 1.39; id 31 `PACKETCOUNT` goes about every 10 s[14]

1.8.2 2.0x steps, the 2.1 test builds, NewWorld and NewMain

What each 2.0x step adds, matched against the binary[27]:

Step	Feature (vendor notes)	Binary evidence
2.01	levels preloaded when walking on the map; map-reading animation; chat set with `setplayerprop #c` stays 3 s; `hitobjects`	PLI 35, 36; PLO 46, 156; UDP 25[27]
2.02	hats from five gani attributes; `#i()` images in signs; string parsing (`strlen`, `startswith`, `indexof`, `#e`, `#T`, `tokenize`); `local.` flags not sent; `issparringzone`/`nopkzone`; free-server limits	PLO 47; zlib import; the server learns the language from 2.02 rev2 (PLI 37)[27]
2.03	`triggeraction`, `actionpushed`/`actionpulled`, `move dx,dy,time,options`	PLI 38; PLO 48, 165; UDP 26[27]
2.04	`replacegani`, `addtiledef`/`addtiledef2`, `removetiledefs`, `setshape`/`setshape2`, `attachplayertoobj`, showimg text, server-side showimg, `weaponscount`	PLO 166; 26 removed; one class added for object sets[27]
2.05	`setmap` applied at once; window title with server name; the `.nw` text level (`GLEVNW01`)	PLO 168; one class added, `TADP`, the anti-debug component (→ §4.11.6)[27]
2.1 test	new tile art, start window and server list (F8), an image engine without GDI, `ascii()`, `tiles[x,y]`, predefined NPCs with `#COMMENTS`/`#EDIT`, smilies in PMs, a tabbed editor	PLO 49, 169[27]

The 2001 builds branch more than their version numbers suggest. Each rule places one branch, either the 2.1 tests, NewWorld or NewMain, against 2.05 by its token, the ids it handles and its classes[28].

Rule
2.1 test2 and test3 keep the 2.05 token `GNW03081`, login and C→S set (39 ids); they add S→C 49 `PLAYERWARP2` and 169 `UPGRADEURL` (61 ids handled). test3 adds skip-reload checks in LEVELBOARD, LEVELNAME and PLAYERWARP; smilies come from `levels/smilies/smiliesdef.txt` (`code,filename`)[28]
The 2.1 test exe is about 140 KB smaller than 2.05 rev 2 despite more functions, because of the rewritten image engine[28]
NewWorld beta1/beta2 (token `G2N15041`) branch from the 2.02 line: beta1 handles 53 S→C ids (no baddies, chests, horses or freeze), beta2 adds 49 and 169 and the 2.1 editor classes; C→S has 34 ids. New vocabulary: `nw*` time variables, `layer`, `arraysnw.dat`, `bodycolors.txt`, `bodynw.png`, `controlnpc.txt`; the classic baddy names and `arrays.dat` are dropped[28]
NewMain 24 and 26 (token `GNW08011`) handle 51 S→C ids (0–45, 75, 100, 150, 151, 153) and the 2.00 C→S set; they carry the Delphi DDE classes and the old editor windows, and are a separate pre-2.00 branch, not a revision of 2.00[28]
2.00 to 2.03 have 186 classes; 2.04 and 2.05 add one each; the 2.1 test swaps four editor classes for four tabbed ones (188); NewMain has 193[27]

1.8.3 2.17, 2.22, 2.3 and 2.31

The four Graal Reborn executables can be compared side by side. The rules below cover the socket classes, the cipher generation, the login and what each build adds[29].

Rule
The four Graal Reborn executables (2.05 rev 2, 2.17, 2.22, 2.31) are Delphi builds with one base socket class and game-server and listserver subclasses[29]
2.17 still uses the gen-3 cipher of 2.05; 2.22 and 2.31 are gen 5, sending the login frame in fixed mode 3 and switching to dynamic mode after the flush (→ §2.2)[29]
From 2.17 the account name is kept obfuscated in memory with `dec(b) = (~b − 10) & 0xFF` and decoded only to build the login line; 2.31 does the same for the password. The 2.18 note "new password encryption" is about the saved-password store[29]
From 2.17 the listserver address comes from `arrays2.dat` (lines: host, port, site, registration URL, translation URL, two fallbacks), checked against the 128-byte `arrays2.sig`; `graal.sig` checks the executable the same way ("Graal.exe corrupted"). 2.05 has neither file (→ §6)[29]
No 2.3 executable was uploaded; 2.3 values are inferred from 2.31, GServer-v2 and the notes. GServer-v2's default `allowedversions.txt` enables only 2.22's token[30]
The Reborn `Graal_222.exe` is dated 2009 and `Graal_231.exe` 2007, and the 2.17 build beside them is rebranded "Eradia Online", so text and server-list edits by the community cannot be ruled out[30]
Tampercheck (39) re-sends differ: 2.17 once at login; 2.22 at login, on each gmap edge crossing and after each client-initiated level-link request; 2.31 at login and about every 3.2 s. 2.31 also sends 24 about every 5.1 s and 46 on every level entry; 2.22 sends 24 = 9 every 5.1 s only while another player is in its level[30]
2.22 login in the lab: 37 `English`, 39, 34 `UPDATEFILE` for the default sword, shield, body and bomb images, 23, then 2; it answers S→C 182 with 44[30]
GServer-v2 sends a `-gr_zlib_fix` weapon to clients 2.21 to 2.31 that pads chat made of long runs of one or two characters; it works around a client compression bug, which a faithful client should keep or tolerate[30]
2.31 does not request the map file after `setmap`, so GServer-v2 pushes every BIGMAP file at login for 2.31 but not for 2.3[30]

What the player sees[30]:

	2.22	2.31
Start window	Training · Options · Get an account! · Start	adds Editor (launches `GraalEditor.exe`); Training stays[30]
Server-list tabs	Gold · Silver · Bronze (possibly a Reborn edit)	Graal Gold · Graal 3D · Classic · Hosted; a 3D server asks for Graal3D or starts `Graal3.exe`[30]
`graal://`, `graal3://`	not registered	registered; a link sends `serverstartparams` to the server's Control-NPC[30]
Editor	full level editor in the client (toolbar, tile and object panels, dialogs, F6 debugger); 29 forms	gone (18 forms); F4 does nothing; F1 shows four buttons instead of six[30]
Options → General	"Save downloaded images (to \levels\webgifs)"	"Use filename cache" (`cachefilenames`, `FILENAMECACHE.txt`)[30]
Classic client-side NPC mode (honours `HASNPCSERVER`)	yes	removed[30]
Positions	half-tile X/Y	X2/Y2/Z2 for players and NPCs[30]
Fixes in the notes		NPCs crossing gmap borders; float-to-string (#v prints integers of more than 6 digits); a new timer replaces the busy loop at 99 % CPU; a B-tree object lookup[30]

The comparison above is of what the player sees. The rules below cover the start window, the HUD files and the GS1 words that each step adds[30].

Rule
2.17's tabs are Graal Gold · Classic · Hosted. Only tabs that hold a server are drawn: the same lab server appears under Bronze in 2.22 and Hosted in 2.31[30]
The HUD is identical in 2.22 and 2.31 (A/S/D slots, counters, spinning hearts, MP bar); 2.31 no longer references `ss_arrow.png`, `ss_bomb.png`, `graalmap.png`, `headsconfig.txt`, `loadgmaps.txt`, `recordpos.txt` or `shield1.gif`, all offline or editor files[30]
GS1 from 2.17 to 2.22 adds `gravity`, `textwidth`, `textheight`, `isthisflagset`, `isplayerflagset` (replacing `isflagset`) and a #R message code; 2.22 to 2.31 adds `timevar2`, `downloadsize`, `onwall2`, `onwater2`. `downloadsize` is in the 2.22 notes but only in 2.31's table, so a 2.22 profile treats it as 0[30]

1.8.4 Graal 3.x: Linux 3.04 and Mac OS X 3.153

The 3.x clients look like a new engine but speak the 2.x protocol. The rules say what the program is built from, where its names come from, which tokens it sends and how its licence files point it at a listserver[7].

Rule
3.x is a C++ rewrite on the Torque Game Engine, not Delphi: menus, prefs and key binds are compiled TorqueScript (`.dso`, version 33); levels, NPCs, GS1, ganis and the Graal protocol are native code[7]
Linux 3.04 (Jan 2004) is i386 ELF, UPX-packed and stripped, with SDL 1.2, OpenAL and fmod; Mac 3.153 (Feb 2005) is PowerPC Mach-O on Carbon. Both were built by "Linux Cyberjoueurs". Names come from RTTI and allocation-tracking source file names, not symbols[7]
The tokens `G3D03014` (3.04, listed by servers as 3.041) and `G3D28015` (3.153) carry the 2.22 and 2.31 date stamps, and the clients speak those protocols; earlier 3.x tokens are `G3D16053` (3.0) and `G3D27063` (3.01) (→ §1.2)[7]
The listserver host is not compiled in: `license/license.graal` names it (`<list-host-3x>`, port 14922, site, signup and translate URLs, two fallbacks) and is RSA-1024 signed (PKCS#1 v1.5 over SHA-1, e = 17) in `license/sig.license`; `sig.graal` signs the packed executable; all three verify against the key embedded in the client[7]
The server name's prefix picks the tab: `P` Graal Gold, `3` Graal 3D, `H` Hosted, `U` not listed, none Classic[7]
Neither build sends 39 after login, and the account and password go as plain strings, with no `~b − 10` step[7]
The 3.151 Linux installer upload is 139 bytes short: an ASCII-mode transfer turned every CR LF into LF, so only its metadata (version 3.151, the 3.153 data layout) survives. The two Mac uploads are byte-identical (3.153), and the 3.04 installer and tgz hold the same game files[7]

Profile[31]:

	3.04	3.153
C→S sent	0–24, 26–30, 32–38, 40, 41, 43, 80, 81 (complete)	3.04's set plus 46, 50, 93, 96–98, 152, 154–158 (2, 3, 15, 24, 29 and 80 assumed)[31]
S→C handled	79	88[31]
Own props written	0–76 except 25, 27–30, 34, 36, 50, 51	0–80 except 25, 27–30, 34, 36, 50, 51, 77[31]
Precise move	no	X2/Y2/Z2 and `M0VE2`[31]
Start window	Nick, Language, Options, Tutorial, Account, Password, Get Account, Start; tabs Graal Gold, Classic, Graal 3D, Hosted with a direct-connect field; options Graphics, Audio, General; no editor	adds a 3D mode, snow and xmas themes, the RC file browser and script uploads, server-side class scripts (131–134); prefs ship with `noudp = 1`[31]
GS1	496 names; every 2.05 command except `putobject`, `setbackpal`, `setbody`, `setbacktile`, `drawovertrees`, `setbacktile2`	not counted[31]

Lab runs: 3.04 on Linux in a loopback namespace, 3.153 on Mac OS X 10.4 in qemu[32,33].

Rule
The listserver login is one plain zlib frame holding id 7 (key, `G3D30123newmain`) and id 1 (account, password); the client accepts a gen-5 reply[32]
The first game frame holds 5 (login), 37 `English` and 34 for `bigmap.txt` and `minimap.txt`; the first props are 0, 18, 31, 33, 52 `English`, 75 `linux` or `mac`, and 76 = 1250[32,33]
Requests for its own images come in batches of at most five[32]
3.04 sends nothing periodic over TCP; its 5.1 s message (UDP id 15 with the byte 9) goes to peers over UDP. 3.153 sends 24 about every 16 to 18 s and walks with props 78/79 (X2/Y2), never 15/16[32,33]
3.04 play: walk 10 tiles per second, X or Y sent every 100 ms, `idle` 0.1 to 0.15 s after release; `push` after about 0.9 s against a wall; `sword` lasts 0.25 s; a dropped item is removed by the client with 13 after 8.3 s; a thrown bush sends no 11; chat is white with a one-pixel blue shadow and no bubble, and Tab opens the chat bar[32,32]
3.04 keys (Torque `config.cs`): arrows, A/S/D items, Q, P and M windows, T microphone, F3 options, F5 load level, F7 player list, F8 server list[32]
3.04 offline (Tutorial) loads `tiledefsOffline.txt` (`picso.png,offlinetutorial,1,0,0`), a terrain-only sheet; classic bushes, stones and pots draw wrongly and cannot be cut or lifted offline[32]
The 3.04 UDP channel is the 2.05 one unchanged: id 0 movement every 100 ms, 22 chat, 26 trigger action (UDP only); one client stopped sending UDP to its peer after 11 to 20 s for an untraced reason; an unfocused player shows as `name (paused)`[32]
The 3.04 process is named 3 by the UPX stub, keeps prefs under `~/.garagegames/graalexe/`, and scans its folders only at startup[32]

1.8.5 Graal 4.x: Windows 4.04r2 and 4.20r8, Mac 4.20r6

Program, packaging and licence

From 4.x, Windows Graal is two programs: a Delphi launcher and a C++ engine DLL. The rules describe that split, the libraries the engine links, its start-up quirks and how it finds the login server[34].

Rule
On Windows 4.x is a Delphi launcher `Graal4.exe` (window, DirectDraw, input, web panel, external windows, URL handlers, `graal4config.txt`, `graal4console.log`) over an MSVC C++ `GraalEngine.dll` with about 2,000 exports, handed its callbacks through about 115 `graal_set…` calls. The Mac client is one PowerPC binary from the same source[34]
The engine source tree is `K:\src\graalv4\…`; crypto is libtomcrypt (not Crypto++ as in 3.x), with zlib, bzip2, JasPer, giflib, libjpeg and libpng[34]
At start-up the engine makes 0 to 1,023 heap allocations of random size, creates the player object, then frees them, so the player sits at a different address each run[34]
There is no separate listserver protocol: the client connects to the `license.graal` host (`<login-host-4x>`, port 14900) as a game server named `Login` and gets the list inside that session (→ §6)[34]
Windows adds `sig.graallib` for the engine DLL; the check runs exe, DLL, license, and any missing file or bad signature stops the client ("The license file(s) have not been found", "The license file is invalid!"). Same RSA key as 3.x[34]
Saved passwords: accounts starting `guest` are never saved; the password is DES-encrypted under a key built from the machine (MAC and `DigitalProductId` bytes) and stored as 1 + 48 characters, so it decrypts only on the saving PC; 4.04r2 keeps one `password` value, 4.20 adds per-account `<account>_pass`[34]
The 4.04r2 installer is a Thinstall (2003) virtualised bundle: an outer loader found through the PE `LoaderFlags` field, an inner stub, and a compressed directory of 151 records (14 folders) with blocks tagged stored, LZ77, LZ77 XOR 0xF0 or range-coded; the extracted files verify against their signatures (→ §11)[34]
4.04r2 ships `Graal4.exe`, `GraalEngine.dll` (3 March 2006), `Graal3DEngine.dll`, `fmod.dll`, `ExeReplacer_temp.exe` and `graalv4_installfiles.txt`; its `license.graal` and `pics1.png` are byte-identical to Mac 4.20r6's[34]
`ExeReplacer` swaps in a new executable and restarts it for the `GRPKG001` update system; packages live in `localversions.txt` and are driven by S→C 105, 106, 193, 196 (→ §8.3)[34,34]

Third-party engines

Some surviving 4.x engines are not original builds. The rule below identifies one modified engine, so its traces are not taken as Graal's own behaviour[34].

Rule
The "Packet Logger" 4.20r8 engine is a third-party modification: a one-byte patch skips the signature check, and a hook in zlib's `inflate` calls `g4_PackLog.dll`, which logs only incoming zlib frames, truncated to the compressed length, as `[id]rest`; it works only on Windows XP SP2 addresses[34]

4.04r2 against 4.20, and the 3D plug-in

These rules set 4.04r2 against 4.20r8 and say how the 3D plug-in shares work with the 2D engine. The last row records how far the recovered 4.04r2 collection goes[34].

Rule
4.04r2 against 4.20r8: no layered board change (107), 108 or 196; one saved password; 4.20r8 adds a network thread, an XML parser, browser and array GUI controls, reconnect and "cheat window" checks and `MRandomLCG`; 4.04r2 still has `decryptGraalMemoryDES`[34]
3D in 4.04r2 is the `Graal3DEngine.dll` plug-in, loaded for a server whose name starts `3 `; it takes over every `graal_set*` callback and owns the window and OpenGL, while the 2D engine keeps game logic and the Graal connection (→ §10)[34]
The later recovered Windows 4.04r2 collection separates the client shell, main engine and original-carrier 3D engine. Its load-only 3D capture establishes module loading, not 3D gameplay initialization or a working rebuild; it does not turn the earlier packer-stub export into an engine export[35]

Mac 4.20r6 in the lab

The one Mac 4.x client was run in an emulator. The rule records what its login and traffic showed that the static read had missed[33].

Rule
Mac 4.20r6 in qemu: the login line is `mac,"",<md5>,""` (the MAC hash came out empty under qemu); it sends 34 and 46, which the static list missed; no flag 18 for the lab's movement weapon; chat is white with a blue shadow; Apple's software renderer needed a shim dropping two AGL attributes[33]

Mac PowerPC start-up and input

The Mac 4.04r2, 4.20r6 and v5 PowerPC clients share one start-up design: a Carbon event-loop timer calls the engine entry once, which runs initialization, then an update method until it returns zero, then shutdown, and quits the loop. On v5 the derived initialization opens a 640×480 window titled `Graal` and builds `DefaultCursor` from `defaultCursor.png`. Each registers 12 window and 15 application Carbon events.[36]

One translator turns Carbon input into engine records. Key down, repeat and up become values 1, 1 and 0 through a byte lookup table; characters are converted one byte at a time, so there is no Unicode path; a modifier emits a press or a release only when its bit changes; mouse movement is either window-local absolute or relative, with the wheel on its own code. On v5 the keyboard fallback builds a command string starting `e:`. Nothing was run, so live delivery, rendering and script execution are not established.[36,36]

1.8.6 Graal 5.x: Windows 5.00r7 to 5.31r4

Program, packers and licence files

The 5.x builds change how they are packaged more than how they play. The rules follow the launcher and engine split, the packers and the licence files from 5.00r7 to 5.31r4, and flag one third-party patch[37].

Rule
5.00r7 (July 2007) keeps the 4.x split: Delphi `Graal.exe`, MSVC 6 `GraalEngine.dll` (about 2,170 exports, source tree `graallib`, own SHA-1, RSA, DES, Blowfish and MD5), `Graal3DEngine.dll`, `ExeReplacer.exe`; a failed engine start shows "The execution file is corrupted"[37]
5.12 to 5.24 and the 6.00 betas are wrapped by MoleBox, which serves its bundled files through hooked file APIs; 5.30r3, 5.31r4 and the later 6.x builds use Themida / WinLicense 2.x and bundle nothing. From 5.24 the launcher and engine DLL are one MSVC executable and the source tree is `quattroengine\quattro_lib`[8]
License files: 5.00r7 four (`license.graal`, `sig.graal`, `sig.graallib`, `sig.license`); 5.12 two (`license.graal`, `sig.license`); from 5.24 none, with the three login hosts compiled in. Every Windows build still connects to the `Login` server on 14900[8,37]
The 5.00r7 "CRC patch" is third-party: 8 changed bytes in the license loader skip the not-found, invalid and five-line checks, so any `license.graal` is accepted[37]
`GR-V1.05` adds tile layers and `GRESCR01` caches DES-encrypted class and weapon scripts whose key arrives in clear with each script (S→C 140, 197; → §3.1, → §8.3)[37]

Login, tokens and wire

These rules cover what every 5.x build sends at login and where their framing differs. The 5.12 rows matter to a server, which has to match that build's compression and iterator step[8].

Rule
Every build logs in by itself as guest at the login screen, with a fixed guest password shared by all the Windows builds; the machine list gains a fifth field, the quoted Windows version (`"6.2 9200 "` under Windows 8 and later)[8]
5.30r3, 5.31r4, 6.00r2 and 6.00r4 expire: on today's date they log "This is a beta release of Graal..." and exit, so they run only with the clock set back for the whole session; 6.03r7 has no such check[8,4]
5.12 sends even short bodies zlib-compressed; 5.24r3 and later send them raw, as 2.22 does[8]
5.12 steps the gen-5 iterator `len / 4` rounded down for compressed frames, one step short of every later build whenever a compressed frame under 16 bytes is not a multiple of 4; a server for a 5.12 profile has to step the same way (→ §2.2.4)[4]
Stock gs2lib does not know `G3D15010` (5.24, 6.00 beta2) or `G3D03048` (6.00 beta1); it then sends the old file layout and 5.24 saves PNG data as a file name[38]

The login lobby, chat and HUD

A 5.x client starts in a lobby on the login server and draws little until it is warped to a game server. The rules say what is drawn and when, and how chat appears[38].

Rule
Login lobby mode: while the current server name starts with "login", the draw-layer mask is cut, so the client draws tiles, chests and image NPCs but no characters and no HUD, and never asks for sprites, heads or bodies. S→C 178 (`PLO_SERVERWARP`) to a server with another name ends it (→ §2.3)[38,4]
After the warp the client asks for `tiledefs<servername>.txt` with the space as `%032`; 5.31r4 and 6.03r7 draw no tiles before the warp[4]
There is no built-in chat bar: the engine looks for a GUI control named `ChatBar`, which the server's scripts supply; Escape asks "Do you really want to quit Graal?"[38,4]

Rule
Chat is drawn above the head with a blue outline and no bubble, on one unwrapped line; the client clears it itself 2.97 to 2.99 s after the server set it. 5.24 draws in Arial, 6.03 in Tempus Sans ITC 24[4]
HUD: 5.00r7 and 5.12 letter the item boxes A, S, D and print the sword level; 5.24r4 under Wine draws neither (unconfirmed on Windows). The player list is a native window "Players: N online" with Staff, Buddies, Guild, Normal, Ignore and Offline[38]

Play

Play in 5.x and 6.x was measured in the lab against 2.05. The rules give the timings that held, the few new actions seen on the wire, and one freeze traced to Wine[38].

Rule
Movement timings in 5.00r7, 5.12, 5.24r4, 5.30r3 and 6.03r7 equal 2.05's: walk 0.5 tile per 50 ms tick, push after 18 blocked ticks (0.9 s), sword 5 ticks, lift 6 ticks, hidden walk 0.1 tile per tick (→ §4.2)[38,4]
The only new mannerisms on the wire are the trigger actions `pushed,0` (0.55 s into a push, then every second) and `pulled,1` (0.85 s into a pull); hurt pushes the player 5 tiles in 5 ticks and lasts about 1.55 s; swimming is as fast as walking[4]
Lifting a pot leaves its base (1770, 1771, 1786, 1787) and the server drops a heart (S→C 22); a pot cannot hide the player, a bush or stone can (status 22, `hiddenstill`)[4]
Under Wine 6.00r2 and 6.00r4 freeze after the first image; on real Windows with the date faked they play, so the freeze is a Wine fault[4]

1.8.7 Graal 6.x: 6.00 to 6.03, Mac 6.11

What 6.x is

6.x drops both the Delphi launcher and the Torque layer. The rules describe the program on each platform, how it finds the login hosts and what it sends to identify the machine[39].

Rule
6.x is one native C++ program per platform from the `graallib` code base, with no Delphi launcher and no Torque layer; Linux links SDL_mixer, X11, OpenGL and avahi; 6.03 moves zlib to 1.2.5[39]
Unpacked Windows 6.00r2 (product version 6.002, compiler date 12 June 2011) and 6.00r4 (6.004, 26 June 2011) are that program under the source tree `quattro_lib`: OpenGL drawing after a Direct3D 9 capability check, FMOD sound, GS2 shell scripts, `TServer*` world objects and a `TSocket` game connection; `ExeReplacer.exe` is still launched for packages (Torque names that remain → §10.1.8)[40,41]
No `license.graal`: the three login hosts are compiled in (6.03 as `host:14900` entries walked on failure)[39]
Linux machine ids: MD5 of the `eth0` MAC, MD5 of the `/etc` inode number, an empty field, and the kernel release and version[39]
Windows 6.03r7 sends `G3D0311C` and Linux 6.03r7 `G3D0511C`, so a server that accepts one need not accept the other; the token belongs to a protocol revision (four builds from 5.30r3 to 6.00r4 share `G3D26090`)[8]
The 6.00 betas (Jan 2010) are older than 5.24r3, and 6.00 beta2 sends 5.24's token, so the two lines shared one code base in early 2010[8]
Hosts, ports, tokens and URL parts are stored scrambled and decoded at run time by a per-byte decoder that depends on length and position, the same in 6.01, 6.03 and 6.11[39]
6.11 (Mac, 2014) fetches a connector script over HTTPS and lets the server set framing, ciphers and id tables (→ §2.10)[39]

The packed 6.037 and 6.113 uploads

Two later Windows uploads are still wrapped by their packer. The rules say which builds they are, what the packed listings show, and what an unpacked listing of the 6.1.1.3 image later recovered[42].

Rule
The packed uploads of Windows 6.037 (2012, Linux Cyberjoueurs) and 6.113 (2014, Eurocenter Games) decompile as packer stubs. 6.037 is the known 6.03r7 executable. 6.113 is not the Mac 6.11 bundle version[42,9]
Those two listings are the Themida / WinLicense shell (blank section names, two random 8-letter names, no `.text`). The strings `Themida`, `Oreans`, `SecureEngine` and `WinLicense` are in the unpacked 5.30r3, 5.31r4, 6.00r2 and 6.00r4 listings, and the 6.037 and 6.113 exports do not contain the C++ core[43]
Windows 6.1.1.3, re-read after the 2026-11-07 gap archive, is still that stub: version string `6.113`, fixed version 6.1.1.3, image timestamp 8 August 2014, one import `SetCurrentDirectoryA`, export directory `QPlay.exe`. The archive added no unpacked engine (→ §10.1.9)[44,44]
A later unpacked listing of that same 8 August 2014 image recovers the `quattro_lib` engine (about 9,300 functions, `TSocket`, `GuiGraalCtrl`). Packer section names and Oreans strings remain (→ §10.1.8)[45,45]

Rule
That unpacked image still has the generation-5 LCG and no compiled-in token. Outgoing sends test a slot against `0xFE` and then call an onSend event. `onSendNPCProperties` is absent. The GS2 switch matches unpacked 6.00, including cases 54 and 121 and folds 234–242, and still has no case 48 or 49 (→ §2.10.6, §8.4.4)[46,47]
Torque field names stay absent. `GuiTilesCtrl` is registered, and `GuiGraal3DCtrl` is not a string; the shell looks up `GraalControl3D`. Five new `t2d…(` constructors are file text, not a `SimObject` console (→ §10.1.8)[45]
That 6.113 listing has no login token, key byte, frame builder or property parser. Do not copy Windows `G3D26090` or either 6.03 token onto it. The Linux note and the Windows captures disagree on the key byte and on the 6.03 token[48,48]
A running 6.113 client on port 14900 sends the clear hello `GNP1905C`, header `01 02 00 01 17 fd`, zlib `78 9c`, then `00 01 00 01 06 fc` and 256 session bytes. The server header is `01 02 00 01` plus `21 fd` or `22 fd`. The rest is ciphertext, so no generation-5 login line and no list or property id is visible (→ §6.1.3)[49]
The full 6.113 capture (241 s, 46225 frames) repeats that hello on TCP 14900 (two answered sessions, server marker 22) and on TCP 14882 (one session, server marker 17, 9639663 bytes from the server). No UDP datagram is to or from those remotes. No UDP port in the file is one greater than a TCP port (→ §2.9, → §10.2)[50]
Unpacked Windows 6.1.1.3 does not store `GNP1905C`. `sendraw` can write those eight bytes with no header, and the six-byte header is the `setprotocol` framer (L counts the header plus the body). Unpacked 6.0.3.7 stores `g3d0311c` and sends it, upper-cased, as id 5 (→ §2.10.5)[51]
A full 6.113 capture the same day has TCP 14900 (three completed sessions, each starting `GNP1905C`, and one unanswered SYN) and no UDP to those remotes. UDP 14900, 14901 and 14899 have no datagrams. The UDP in the file is other traffic on the same interface, including a short-header flow on UDP 443 to a separate remote[50]

Serving a 6.x client in the lab

These two rules are what a lab server needs to admit a 5.x or 6.x client, and the one known server crash it must avoid[4].

Rule
A server for 5.x/6.x needs a game server name that does not start with "login" (or a 178 warp), the tokens `G3D15010` and `G3D03048`, and must serve `basepackage.gupd` (a minimal `GRPKG001` with `NAME`, `VERSION`, `PLATFORM`, `MODE` will do) and `tiledefs<server>.txt`; the Themida builds fetch every image, gani and sound[4,4]
GServer-v2 beta4 crashes on every 6.00 login while registering the GS1 weapon `-gr_movement`, taking the GS2 bytecode branch for a weapon with no bytecode[4]

1.8.8 Profile tables

The profiles above also exist as machine-readable tables, one per range, that a client can load to behave as one exact build. Each `[[version]]` holds a build's full values, `[version.features]` its switches and `[version.assets]` its HUD and tileset revision[14].

Table	Builds
`v1998-v11.toml`	1st Graal (Sept 1998) to 1.1 (Feb 1999)[52]
`v121-v132.toml`	1.21 to 1.32[53]
`v133-v139.toml`	1.33 to 1.39[54]
`141-217.toml`	1.41 rev1 to 2.1 test, with NewMain and GraalX[55]
`2.22-2.31.toml`	2.22, 2.3, 2.31[56]
`3.04-3.153.toml`	3.04, 3.153[57]
`4.04-4.20.toml`	4.04r2, 4.20r6, 4.20r8[58]
`5.00-6.03.toml`	Windows 5.00r7 to 6.03r7[59]

Lab measurements kept as tables beside them:

Table	What it records
`lab-200-listserver-handshake.tsv`	the 2.00 listserver handshake, step by step[60]
`lab-boardmodify2-186-trailer.tsv`	how 2.31 treats extra bytes after an S→C 186 board block[61]
`lab-enterlevel-231.tsv`	when 2.31 sends C→S 46: at login and on gmap cell crossings[62]
`lab-gs13-respawn.tsv`	respawn delays of board changes, and which client saw them[63]
`lab-levelchange-46-222-231.tsv`	level changes in 2.22 and 2.31: request, server reply and id 46 order[64]
`lab-serverwarp178.tsv`	how 2.31 reacts to variants of the S→C 178 fields[65]
`lab-showimg24-relay.tsv`	C→S 24 payloads and how the server relays them to other players[66]

1.9 Identity and platform evidence

These records say what an archived file is, and where that identity stops. None of them is a behaviour reading; each keeps the scope its source gives it.[67]

1.9.1 Archived macOS executables

Each row is one archived macOS client, named by its archive label. It gives the processor the main executable targets, the minimum OS X version it declares, and how far its identity is proven. The Identity limit column is the one to read before using a row as evidence for a particular release[67].

Archived input label	Main-executable architecture	Declared minimum OS X	Identity limit
v3.04; v3.15r2	PowerPC	unmeasured	Same main-executable digest and matching regular-file contents/names across two extracted copies of 718 files each; historical 3.153 attribution remains unlinked to these exact inputs (→ App. E#609)[67,68]
v4.04r2; v4.20r6	PowerPC	unmeasured	Distinct main executables[67]
v5.12; four unknown-v5 copies	PowerPC	unmeasured	Five main-executable digests match; regular-file contents/names match across all five extracted copies of 73 files each; original containers and filesystem metadata unmeasured[67,68]
v6.11r1	Intel 64-bit	10.7.0	Metadata requirement; virtual graphics and runtime compatibility remain unmeasured by this inventory[67]

The PowerPC headers declare legacy graphics and application-framework dependencies but no exact minimum OS version. Neither these dependencies nor the Intel build's declared minimum establishes successful rendering, input or gameplay. The separately captured Tiger runs retain their own scope (→ §1.8.4, → §1.8.5).[67]

Unresolved archive-label and resource identity: → App. E#609.

1.9.2 Linux network and bootstrap gates

Seven exact Linux input identities have observed accepted local logins on one pinned patched lab server, while visual results and later execution gates remain separate. The derived ledger records exact client hashes, shared tokens, per-gate statuses and sanitized source lineage. A not-established cell concerns its specified network cohorts, not every experiment in this repository.[69]

The later 6.03 cohort separately records class parsing, registered-weapon execution and a suspended continuation with proxy-origin bodies; the 3.04 cohort separately records source-weapon effects and persistence. These observations do not establish normal-server GS2 compilation, exact sleep timing, a positive disk-cache hit or complete gameplay. Cache contracts remain explicitly static-only where marked.[69,69]

1.9.3 The imported Windows v4 carrier

The imported report’s supplied `4.20r8` alias and the retained collection’s `4.04r2` label refer to a carrier with SHA-256 `71378394469bcf16038ee3b335da786afcdf357fe4e8dd6329a05ba926a5e4bf`. A separate retained-resource checkpoint reports FileVersion `4.0.4.2`, ProductVersion `4`, OriginalFilename `Graal.exe` and ProductName `Graal`. This corroborates byte identity and metadata, not official release provenance. Keep the packed/stub carrier separate from subsequently recovered engine/shell/3D components; no behavior transfers from an alias. No target execution or original package authentication occurred.[70]

2 Wire protocol

> Status: draft. Each chapter carries its own status line.

2.0 Reading this Part

The Graal wire protocol is one message vocabulary carried by several transports. From 1.21 to 6.03 a message is an id byte, a payload and a newline, and the id is logical: the wire byte is the id plus `0x20`.[3,2] What changes from build to build is the frame around those messages, the cipher over the frame, and which ids each side sends and handles. A reader of this Part should first fix the build's encryption generation (→ §1.4), because that decides how to read every byte that follows.[1]

Printable integers, and no lengths. Graal's own integer types add `0x20` to every 7-bit group, so an encoded value never contains a newline and a message can end at `\n`.[1] Bodies that may hold any byte go behind the raw-data escape (§2.1.3). The same economy shows in prop streams: a stream is `(id, value)` pairs with no per-prop length, so a reader must know the encoding of every id up to the highest it expects, and in 2.05 to 2.31 one unknown id early in a stream hides every prop after it.[71,71]

Six generations of transport. The 1998 builds speak text lines through a relay that knows nothing about the game. 1.21 to 1.39 add zlib frames and no cipher. 1.41 rev1 to 2.17 insert one junk byte into each client message from a shared LCG. 2.19 and later apply a partial XOR after compression, and 2.22 and later choose their compression per frame.[72,1] In 6.11 the client has no fixed protocol at all: a connector script fetched at login sets the framing, the cipher and both id tables.[39] §2.1 and §2.2 give the frames and ciphers, and §2.3 gives each generation's login.

Ids are a per-build set, and silence is normal. The client-to-server (PLI) and server-to-client (PLO) references in §2.4 and §2.5 give each id's layout where a note measured or read it, and say `unmeasured` otherwise. Every handler in 2.05 to 2.31 checks a minimum length and drops a short message silently, and an id with no case falls to a shared default arm and is ignored.[2,3] A server that sends an id a build does not handle therefore sees no error, only missing behaviour, which is why the per-build id sets in §2.4.2, §2.5.2 and App. C matter.[29]

Beside the game connection. Files ride the same connection, with request lists and replies whose layout follows the server's token table (§2.7).[73] From 1.28 beta 2 to 3.04 the clients also run a UDP channel between players that the game server never sees; the server only relays each player's address (§2.9).[74,75] The strongest evidence in this Part is the 2.05 rev 2 client read against GServer-v2's source and lab captures; later rows say which build and which kind of evidence they rest on, and an exact-build reading holds for that build only.[2]

Chapters

- → §2.1 Framing and encodings (`01-framing.md`)
- → §2.2 Ciphers and compression (`02-ciphers.md`)
- → §2.3 Login and handshake by generation (`03-login.md`)
- → §2.4 PLI message reference (client to server) (`04-pli.md`)
- → §2.5 PLO message reference (server to client) (`05-plo.md`)
- → §2.6 Props (`06-props.md`)
- → §2.7 File transfer (`07-file-transfer.md`)
- → §2.8 Chat and private messages (`08-chat-pm.md`)
- → §2.9 UDP peer-to-peer (`09-udp.md`)
- → §2.10 The 6.11 server-defined protocol (`10-server-defined.md`)

2.1 Framing and encodings

Notation is fixed in → §0.5 to → §0.7. Ciphers that act on messages or frames are in → §2.2; the 1998 text relay, which has no binary framing, is in → §2.3.1.

2.1.1 Integer and string encodings

Graal's own types add `0x20` to every 7-bit group, so an encoded value is printable and never contains `\n`[1].

Type	Short	Bytes	Encoding, first byte first	Range
GCHAR	`c`	1	`v + 0x20`	0–223[1]
GSHORT	`s`	2	`(v >> 7) + 0x20`, `(v & 0x7F) + 0x20`	0–28767[1]
GINT (GINT3)	`i3`	3	three 7-bit groups, high first, each `+ 0x20`	0–3682399[1]
GINT4	`i4`	4	four 7-bit groups, high first, each `+ 0x20`	unmeasured[1]
GINT5	`i5`	5	first group 4 bits (`v >> 28`), then four 7-bit groups, each `+ 0x20`	32-bit[1]
GSTRING	`str`	1 + n	GCHAR length, then n bytes	n ≤ 223[1]

The encodings above are exact for values in range. The rules below cover what happens at the edges: overflow, signed values and over-long strings, plus the 1998 relay's use of the same one-character form[1].

Rule	Builds
On overflow the server clamps rather than wraps (a GSHORT high byte is at most 223)	server[1]
A signed value is sent as its unsigned bit pattern	server[1]
In the login message, an account or password of 224 bytes or more is sent as length byte `0xFF` followed by its first 223 bytes	4.20r6, 4.20r8, 5.00r7, 6.01r1, 6.03r7[34,37,39]
Numbers in 1998 relay payloads use the same +32 single-character encoding as GCHAR	1st Graal to 1.1.9[72]

2.1.2 Messages and terminators

Inside a frame, the unit is the message. Each rule below gives one property of how messages are written and split, with the builds it was confirmed for. The id rules decide which ids can be sent at all[74,3].

Rule	Builds
A message is `GCHAR(id)`, the payload, then `\n`	1.21 to 6.x[74,1,39]
Several messages are concatenated in one frame	1.21 to 6.x[1,74]
The wire byte of the id is the logical id + `0x20`; an id above 223 cannot be sent as one GCHAR	all binary generations[3]
The sender refuses a payload that already contains `\n`; binary payloads use the raw-data escape (→ §2.1.3)	1.39, 2.05[1,76,77]
An outgoing message containing `\n` is refused with "Illegal characters in the outgoing data"	6.01r1, 6.03r7[39]
The receiver splits an inflated frame on `\n`	1.21 to 2.05[74,1]
An id the receive dispatcher has no case for is ignored silently; 2.05 has 59 cases	2.05[1]
Every 2.05 handler checks a minimum length first and drops a short message silently	2.05[2]

2.1.3 The raw-data escape

A newline ends every message, so binary data needs a way round it. The rules below describe that escape: the server announces a length, and the receiver takes that many bytes as one message. The last row is the client's own form of it[1].

Rule	Builds
`PLO_RAWDATA` (100, wire `0x84`) carries an `i3` length; the next *length* bytes are read as one message, newline or not	server; 1.21 to 6.x clients[1,74,34]
The server sends 100 in front of `PLO_BOARDPACKET` (101) and `PLO_FILE` (102)	server[1]

Rule	Builds
2.05 has a case for 100 in its main dispatch but none for 101 or 102, consistent with 101 and 102 only arriving behind 100	2.05[1]
In the other direction, before a payload sent as id 93 the client sends id 50 with an i3 length, capped at 0x37FFFF	4.20r8, 5.00r7[34,37]

The board and file layouts behind 100 are in → §2.5 and → §2.7.

2.1.4 Frames by generation

Encryption generations are defined in → §2.2.1; builds per generation are in → §1.4.

Generation	Builds	Client → server frame	Server → client frame
1	1st Graal to 1.1.9	text lines ending in \n, no length (→ §2.3.1)	same[72]
2	1.21 to 1.39	u16be length, then a zlib stream of that length; every flush is compressed, with no size threshold; zlib 1.0.4	same[74]
3	1.41 rev1 to 2.17	u16be length + zlib stream of the whole out buffer; gen-3 insertion applied per message before compression (→ §2.2.2)	u16be length + zlib, no extra byte, no cipher[1,29,1]
4	client type 4 (2.19 to 2.21, 3.0, 3.01)	u16be length + bz2 body, then partial XOR (→ §2.2.3)	same[1]
5	2.22 and later; RemoteControl type 6	login frame: u16be length + zlib, no type byte, no XOR (fixed mode 3); later frames: u16be (len + 1), a type byte, the body (→ §2.2.4)	same layout, type byte present from the first frame[1,29,1]
5, long frames	6.01r1, 6.03r7	a frame of 0xFFFF bytes or more is sent as length 0xFFFF followed by a 4-byte big-endian length	unmeasured[39]
own	Graal3D	→ §9	→ §9[5]

2.1.4.1 Flush and buffer limits

A client does not send each message as soon as it is made. It queues them and flushes a frame at a size limit or at the end of a tick, and it drops the link if too much is left unsent. The rules give those limits per build. A server that sends faster than a build drains will see that build disconnect[77].

Rule	Builds
1.39 flushes when the queue would pass 4095 bytes and writes the frame to the socket in 256-byte slices; it drops the link when more than 256 KiB of compressed output is pending	1.39[77]
Messages collect in an out buffer, compressed and flushed at 0x1000 bytes or at the end of the tick	2.00 to 2.05, 2.17[1,78,29]
The link is dropped when more than 0x40000 bytes are queued unsent	2.00 to 2.05, 2.17, 2.22, 2.31[1,78,29]
The same limit is 0x800000 bytes	RemoteControl (gen 5)[29]
Buffers flush at 0x1000	6.01r1, 6.03r7[39]
The client links zlib 1.1.2 (Delphi ZLib unit)	2.05[1]
zlib is a compiled-in unit in 2.00 to 2.01 rev1 and from 2.03 on; 2.02, 2.02 rev1 and 2.02 rev2 import an external zlib.dll and need it beside the program; the wire format is the same	2.00 to 2.1 test[79]
The NewWorld betas and NewMain 24 and 26 use the same transport and login shape as 2.00 to 2.05	NewWorld, NewMain[79]
The 1998 relay read buffer is 1 KB	1st Graal to 1.1.9[72]
A UDP peer message is not framed this way (→ §2.9)	1.28 beta 2 and later[1]

2.1.4.2 zlib stream header

The two bytes that open a zlib stream record the compression level. This matters only to a server or proxy that builds frames for a real client or listserver to accept[80].

Rule	Builds
Real clients compress at the best zlib level: the stream starts 78 DA	2.05, 2.17.1, 2.22, 2.31[80]
A listserver accepts gen-5 frames that start 78 9C (default level); whether a classic frame with that header is accepted is unmeasured	public listserver[80]

For the exact original 2.05r2 FILE path, the counted collector preserves every announced raw byte, including a final LF within the count; the layout and persistence qualifications are in → §2.7.9[81].

2.1.4.3 Exact-input Mac PowerPC receive gate

Four independently inspected Mac receivers (shared v3, labels 4.04r2 and 4.20r6, shared v5) wait for a complete body after a two-byte unsigned big-endian length that excludes the prefix. A zero configured mode consumes the first body byte as selector and removes it from the transform length; a nonzero configured mode applies directly. Modes 1/2 are raw, 3/4 follow zlib and 5/6 the bzip2-linked path; even modes XOR before decompression. This is static receive evidence, not negotiation, send symmetry, live decoding or v6 evidence.[82]

2.1.4.4 Exact build-614 outgoing special-length correction

In the supplied Windows amd64 WorldsSetup64-614 unpacked plugin, the special internal-ID 93/163 sender path adds two to body length, applies a signed upper-cap comparison at 3,670,015, emits the inspected biased seven-bit-group operations and recursively invokes the sender with internal ID 50. This corrects body-length-only prose for that exact input. Static checks do not establish original packed provenance, production wire IDs, helper completeness or runtime/extreme-size behavior.[83]

2.1.4.5 v7 template counter check

In seven supplied unpacked Windows amd64 v7 native engines (labelled 548 to 701750), the template receiver compares the counter with previous+1, masked to 8, 16, 24 or 32 bits, only when the stored previous value is non-negative as a signed 32-bit number; a negative stored value is accepted without comparison. A mismatch marks an error and returns false. Immediate socket close, reachable negative states and runtime behaviour are not established.[83]

2.2 Ciphers and compression

Frame layouts are in → §2.1.4. Which build uses which generation is in → §1.4.

2.2.1 Encryption generations

The numbering is GServer-v2's (→ §0.9).

Gen	Selected by	Users	Scheme
1	web client	web client; the 1998 relay builds have the same "no compression" property	none, uncompressed[1,72]
2	client type 0 whose first 8 login bytes are a known token	1.21 to 1.39	zlib, no cipher[1,74]
3	client type 0 whose first 8 bytes are not a known token (the key byte comes first)	1.41 rev1 to 2.17; RC, NC and NPC-server connections in the server's numbering	zlib frame; one junk byte inserted into each client-to-server message (→ §2.2.2)[1,29]
4	client type 4	2.19 to 2.21, 3.0, 3.01	bz2, then partial XOR (→ §2.2.3)[1]
5	client type 5; RC2 (type 6)	2.22, 2.31, 3.04, 3.153, 4.x, 5.x, 6.01, 6.03, Delphi RemoteControl	compression byte, then partial XOR (→ §2.2.3, → §2.2.4)[1,7,34,8,39]
6	v6	unmeasured	not implemented in the server source the notes summarise (→ §2.2.5)[1]

The table above says which generation each build uses. The rules below add what it leaves out: the remote-control socket, which uses no cipher, the fact that generation 2 encrypts nothing, and the random-number step that generations 3 to 5 share[1].

Rule	Builds
The NC socket logs in as type 3 and stays zlib with no cipher	Delphi RemoteControl[1]
Nothing in generation 2 is encrypted; the DES code in those clients is used only for local files	1.21 to 1.32[74]
All 2.x clients share one LCG, `it = it * 0x08088405 + key`, seed `0x04A80B38`; the multiplier is Borland's `Random` constant, but the key is added where Borland adds 1	2.05, 2.17, 2.22, 2.31[29]
The shared multiplier with the RTL `Random` is a toolchain coincidence, not shared state	2.05[1]
The same seed is also used for web-level cache file encryption from 1.3 on; that use is not the connection cipher (→ §3.8)	1.3 test and later[5]

2.2.2 Generation 3 insertion

Client to server only. Server-to-client traffic for a gen-3 client is plain zlib[1].

State. `it` is a u32 seeded `0x04A80B38` when the connection object is created. The key `k` is `Random & 0x7F`, sent once as a GCHAR in the login message (→ §2.3.3)[1,78].

Per message, starting with the second message of the session (the login message is sent clean; the messages that follow it in the same first frame, such as `PLI_LANGUAGE`, already carry the byte)[1]:

Step	Operation
1	`it = it * 0x08088405 + k` (mod 2^32)[1]
2	`len` = message length including the trailing `\n`; `pos0 = (it & 0xFFFF) mod len` (0-based)[1,78]
3	insert the byte `((it >> 16) & 0x7F) + 0x20` at 0-based index `pos0`, before compression[1]

The steps above are the whole algorithm. The properties below are what a decoder needs to get right in practice, and what the lab sessions confirmed[1].

Property
`pos0` can be 0, which puts the junk byte before the id byte; a decoder must not take offset 0 as the id until the byte is removed[1]
The inserted byte is never `\n` and never lands after the terminating `\n`[1]
Insertion is per message, after the receiver's decompression, not over the whole frame[1]

Property
The server mirrors the step and deletes the byte at 0-based `(it & 0xFFFF) mod len`, where `len` is the split-off line without `\n` but with the junk byte, which equals the client's length[1]
The server's own gen-3 encoder always inserts `)` instead of an iterator-derived byte; harmless, because the receiver discards it[1]
Three lab 2.05 sessions: every message after the login had the predicted byte at the predicted index, several at index 0, no mismatches[1]

Generation 3 starts somewhere between 1.39 and 1.41. The table pins that boundary and records which later builds were checked against the formula[5].

Builds	Insertion
1.39	absent[5]
1.41 rev1, NewMain 1.42	present (seed and 7-bit key in the connection initialiser)[5]
2.00	the formula above, read byte for byte[78]
2.01 to 2.04	same call structure and endpoints as 2.00 and 2.05[78]
2.05, 2.05 rev2	the formula above[1]
2.17	the same gen-3 send code[29]

2.2.3 Generation 4 and 5 partial XOR

The server/2.22 contract below is not universal across later receivers; the independently inspected Mac PowerPC qualification follows the table. Same LCG and key as generation 3, with one iterator per direction, both seeded `0x04A80B38`[1,29].

Step	Operation
1	on send, compress the body first (→ §2.2.4)[1]
2	`groups = min(ceil(len / 4), limit)`, with `limit` 12 for an uncompressed body (`0x02`) and 4 for zlib or bz2 (`0x04`, `0x06`)[1]
3	for each group: advance `it` once, XOR the 4 body bytes with the little-endian bytes of `it`[1]
4	a short last group still advances `it` and is XORed byte by byte with the low bytes of `it`[1,84]
5	on receive, undo the XOR first, then decompress[1]

The step count above is not taken on trust. Each row is one body of evidence that the rule holds, and the last row shows the alternative reading failing[1].

Evidence for the step rule
27 of 27 server-to-client frames from four public 2.22 game servers and both listserver frames, including zlib and bz2 bodies with a partial last group[1]
610 lab frames, both directions[1]
a real 2.22 client sent 580 uncompressed and 17 zlib frames; all 17 compressed bodies had a length that was not a multiple of 4 and decode only under this rule[1]
ten 5.x and 6.x Windows captures decode with this rule[8]
for these 2.22 captures, `floor(len / 4)` groups for compressed bodies is wrong and leaves the iterator one step behind[1]

Builds agree on the cap but not always on rounding. The table gives the number of iterator steps each build takes for a compressed body. A row that differs from `ceil(len / 4)` needs a server that steps the same way[1].

Builds	Step count for zlib and bz2 bodies
2.22, 2.31	`ceil(len / 4)`, capped at 4[1]
3.04, 3.153	`ceil(len / 4)`, capped at 4[7]
4.04r2, 4.20r6, 4.20r8	cap 4; rounding unmeasured[34]
5.12	disputed → App. E#2[4,8]
5.24r3 and later 5.x, 6.00 to 6.03r7	`ceil(len / 4)`, capped at 4[8,4]

2.2.3.1 Exact-input Mac PowerPC receive qualification

For the four exact Mac inputs identified by digest in the source note, let n be body length after any mode selector is removed, before decompression. Raw mode 2 processes min(ceil(n / 4), 12) groups, including a partial final group. Compressed modes 4/6 process min(floor(n / 4), 4) groups; an incomplete tail is unchanged and advances no state. Before each processed group, state is multiplied by 134775813 and increased by an additive object word, retaining the low 32 bits. Its bytes are XORed least-significant first despite the big-endian target. State persists through the inspected receiver's iterations.[82]

Initial seed, additive-word provenance, reset/rekey writers, negotiation and outgoing symmetry remain untraced. No live frame result or Windows 5.12 capture resolution follows; E#2 remains open.[82]

2.2.4 Compression choice

The type byte of a generation-5 frame (→ §2.1.4):

Code	Body	XORed
1, 2	raw	only 2[29]
3, 4	zlib	only 4; 3 is the fixed "zlib, no cipher" mode of the login frame and of the whole NC session[29]
5, 6	bz2	only 6[29]

Which code a client picks for each flush of `n` bytes:

Builds	`n < 0x30`	`0x30 ≤ n < 0x1000`	`n ≥ 0x1000`
2.22, 2.31, RemoteControl	`0x04` if zlib is actually shorter, otherwise `0x02`	`0x04`	`0x06`[29,1]
3.04, 3.153	`0x02` or `0x04`	`0x04`	`0x06`[7]
4.x	`0x02`	`0x04`	`0x06`[34]
5.12	`0x04` even for short bodies	`0x04`	unmeasured[8]
5.24r3 and later	`0x02`, like 2.22	unmeasured	unmeasured[8]
6.01r1, 6.03r7	`0x02`	`0x04`	`0x06`[39]

The rules below complete the frame layout: what the length counts, why the login frame is different, what has not been seen, and how the older generations choose[1].

Rule
The u16be length counts the type byte[1]
The login frame carries no type byte and no XOR, because the key travels inside it; the socket switches to dynamic mode after it is flushed[1,29]
No client-originated bz2 frame has been captured[1]
Generations 2 and 3 compress every flush with zlib[74,1]

2.2.5 Generation 6 and script-defined transports

Generation 6 exists only as a name. The rules say what the late clients actually use, and how a script sent by the server can swap the transport out[1].

Rule	Builds
Generation 6 is named in the server source but not implemented there	server[1]
The classic transport of 6.01 and 6.03 is generation 5, unchanged	6.01r1, 6.03r7[39]
A server-sent connector script can replace the framing, the cipher (RC4, AES or TLS) and both id tables; generation 5 stays the default until the script switches protocol (→ §2.10)	6.11 Mac[85,10]

2.3 Login and handshake by generation

Tokens are listed in → §1.2.2, frames in → §2.1.4 and ciphers in → §2.2. The listserver exchange that comes before a game login is in → §6; RC and NC logins are in → §7.

2.3.1 1998 relay (generation 1)

2.3.1.1 Connections

Before 1999 there is no game server, only a relay. The rules below describe that relay and what a 1998 client does on it[72].

Rule	Builds
The client connects over TCP to a lobby and relay server that knows nothing about the game; it groups connections into named games, gives each an 8-character id, and forwards `D` lines between partners	1st Graal to 1.1.9[72]
A client opens two connections to the same port: one in the fixed game name and one in a room named after the current level; both pair with the other player	1.1.9 (captured); 1st, 3rd, 4th alpha, 5th Graal (live)[72]
Every message is a text line ending in `\n`; the first character is the command	1st Graal to 1.1.9[72]
There is no login token and no peer-to-peer mode	1st Graal to 1.1.9[72]
Levels are not sent over the relay; web levels are fetched over plain HTTP/1.0 by a separate small client	1st Graal to 1.1.9[72]
Default server: the loopback address on 14801; from 3rd Graal the connect dialog is pre-filled with a relay on 14802; 4th and 5th alpha have no pre-filled relay host	1st Graal to 1.1.9[72]

The socket code under the relay protocol changed twice in 1998. The table gives the layer each group of builds uses. It matters only when one of these builds is run against a modern server, where the blocking connect and the resolver order show[72].

Builds	Socket layer
1st, 2nd Graal	ICS `TWSocket`, event driven[72]
3rd Graal to 1.1.9	plain WinSock 1.1: blocking `connect`, then `FIONBIO`, then polled `recv`; host tried with `inet_addr`, then `gethostbyname`[72]
15th Graal on	as above, plus a hostname-to-address cache in front of the resolver[72]

2.3.1.2 Lobby lines

Each lobby line is one character of command followed by its arguments. The table lists every line both ways, client to server first. Together they cover joining a game, pairing with another player and leaving[72].

Direction	Line	Meaning
C→S	`N<game>#<name>`	join `<game>` as a player; the clients always send the name `def` and send the nickname later inside `D`[72]
C→S	`P<id>`	ask to play with player `<id>`, or accept a request[72]
C→S	`C`	leave the current party[72]
C→S	`-`	leave the available list; sent on a deliberate disconnect[72]
C→S	`D<data>`	relay `<data>` to the party (→ §2.3.1.4)[72]
S→C	`I<id8>`	your id: `0000` plus a 4-digit counter[72]
S→C	`+<id8>`	player in the same game; if it is the only known peer the client answers `P<id8>` at once[72]
S→C	`-<id8>`	player left[72]
S→C	`P<id8>`	partnership confirmed; the client then pushes its whole state to the peer[72]
S→C	`C<id8>`	the partner left the party[72]
S→C	`D<data>`	relayed data[72]
S→C	`NR`, `NL`	sent right after `N` by the 1.1.9 server: `NR` = relay levels, `NL` = local levels (the client then starts in `onlinestartlocal.graal`); a server with local levels sends `NR` then `NL`[72]
C→S	`O`, `S`, `V<id>`	observer, super user, observe a player; accepted by the 1.1.9 server[72]

Direction	Line	Meaning
C→S	0<game># <name>	in Knorr's GServ Standalone (1997): join as an observer, who sees the players of that game already in a party[22]
C→S	S<name>	GServ Standalone: super user, who sees every player of every game as name (game)[22]
C→S	+	GServ Standalone: available again after -[22]
C→S	V<id8>	GServ Standalone: watch a player and receive their party's D traffic[22]
C→S	H<score>	high score; GServ Standalone ignores it[22]
S→C	+<id8> <name>	GServ Standalone sends the name directly after the id; D is relayed unchanged, without the sender[22]

2.3.1.3 Game names

Clients only meet on the relay if they join the same game name. The table gives the name each build sends, which is why builds of different months cannot see each other[72].

Builds	Name sent with N
1st, 2nd Graal	Zelda2[72]
3rd, 4th Graal	Graal[72]
4thgraalalpha	Graal2 (room prefix G2)[72]
5th Graal, 5thgraalalpha	Graal3[72]
6th Graal	Graal4[72]
7th to 9th Graal	Graal5[72]
10th to 12th Graal	Graal6[72]
13th Graal	Graal7[72]
14th, 15th Graal, 1.1, 1.1.9	Graal8[72]

In 1st Graal, Zelda2 is used by one branch only; the other, chosen when the network client is created, builds the name from the level file's base name as Z + base + 2 (level1.zelda gives Zlevel12); a live 1st Graal joined Zhouse12 and Zelda2 (table below)[86,87].

The constant is the fallback. On a level the client sends G<n> plus the level's location (a web level's URL without http://), so each level is its own room and players meet only on the same level and a compatible build[72].

Build	Level room	Game-wide room
Zelda Online 2 (Java, 1998)	Z + level base name + 1.6 (Zlevel11.6)	none[22,26]
1st Graal	Z + level base name + 2 (Zhouse12)	Zelda2[88]
3rd Graal	G + level file (Ghouse1.graal)	Graal[88]
4th Graal alpha	G2 + level file (G2level13.graal)	Graal2[88]
5th Graal	G3 + level file	Graal3[88]
Java Graal (July 2000)	G2 + level file	none; a Graal2 mode exists but is never selected[88,22]

The rules below come from live runs that put different builds on one relay. They show which builds can meet, and what a bridge between rooms must keep[88].

Rule
Every Delphi beta opens two relay connections, one per level room and one game-wide; the Java clients open one[88]
Java Graal dials its own codebase host on 14802 and speaks 4th Graal alpha's dialect: the two met and chatted on a stock GServ[88]
Java Graal and 5th Graal met through a proxy that renamed the G2 level room to G3; the D lines needed no change, since each side ignores the types it does not know[88,88]
A renaming bridge must keep the level part; mapping a level room onto a game-wide room would put players on different levels together[88]
When two clients each answer the other's + with P, GServ links the pair twice and delivers every later D line twice; Zelda Online 2 and Java Graal both did this[26]
D types: Zelda Online 2 uses P C N S M H B T D E O W A; Java Graal drops S (colours) and adds L (level name), so Java players show default colours[22]
The level-leader rule (earliest login time, ties to the lower id) comes from Zelda Online 2[22]

2.3.1.4 `D` payloads

```
D <sid:2> <type:1> <payload>
```

Field	Encoding
`sid`	sender id as two base-128 characters, `(c0 - 32) * 128 + (c1 - 32)`; the receiver writes it back as `0000` plus 4 zero-padded digits; the server does not stamp the sender[72]
coordinates	half tiles, `(c - 32) / 2`[72]
knock-back and fine offsets	`(c - 96) / 16`[72]

Game traffic between two players goes inside `D` lines, one letter per kind of event. The table shows how the set of letters each build sends grew, build by build[72].

Builds	Types sent
1st Graal	`A B C E F G H L M N O P R S T W` (can also receive `D`)[72]
2nd to 4th Graal	adds `Q`[72]
4thgraalalpha to 5thgraalalpha	adds `I U X Y Z`[72]
6th Graal	adds sending `D`[72]
7th Graal	adds `0` to `4`[72]
8th Graal, 9thgraalalpha	adds `5`[72]
9th to 14th Graal	adds `6`, `7`[72]
15th Graal, 1.1, 1.1.9	adds `8`[72]

This is the dictionary for those letters. Each row gives the payload fields and what the event means. Numbers inside use the one-character encoding of §2.1.1[72].

Type	Payload	Meaning
`A`	x, y, dir/flags, kind, one more byte	arrow fired; in a 1.1.9 capture, level NPCs firing every 0.5 s produced steady `A` lines from both clients, because each client runs the level NPCs itself[72]
`B`	x, y, w·8+h, then w·h tiles as two base-128 characters each	board modify[72]
`C`	text	chat line[72]
`D`	one character per object id	listed baddies or NPCs are dead[72]
`E`	x, y, item index	item laid down[72]
`F`	x, y, a, b	bomb laid[72]
`G`	x, y, item index	item picked up[72]
`H`	object id, dx, dy, damage (dx, dy in the /16 encoding)	baddy hurt[72]
`I`	none	player left the level or went to a link; cleared by the next `P`[72]
`L`	level name	current level[72]
`M`	repeated 6-byte records: id, x, y, sprite, dir·4+mode, state	baddy movement batch[72]
`N`	text	nickname[72]
`O`	carried object id (space = none)	carry object[72]
`P`	x, y, sprite (animation frame·4 + direction, 0 up, 1 left, 2 down, 3 right)	position and animation, sent every 100 ms while moving[72]
`Q`	NPC id	`playertouchsme` for an NPC on the receiver's side[72]
`R`	x, y	object at x, y removed[72]
`S`	count, then that many colour indices (4 in 1st Graal, 5 from 4th alpha)	sprite colours[72]
`T`	10 characters, base 128, little end first	login time; the peer with the earliest time is level leader and runs baddies and NPCs[72]
`U`	x, y, kind, id	baddy spawned[72]
`W`	sprite	throws the carried object[72]

Type	Payload	Meaning
X	target id	hit or kill aimed at one player; only the named player applies it[72]
Y	sword power, shield power, power	sword and shield change[72]
Z	none	state flag[72]
0	horse image, or empty	horse on or off[72]
1, 4	x, y	horse and bush actions[72]
2	dx, dy	fireball[72]
3	len, image, script	carrying an NPC; the image can be an http:// path[72]
5, 6, 7	none	pose flags (sitting, sleeping)[72]
8	registration-key string	duplicate key check; a peer with the same key disconnects the client with a message[72]

Several D types live on in 1.21 as numbered messages with the same meaning and similar payloads: board modify, bomb, item, horse and throw[72].

2.3.2 1999 GSERV login (generation 2)

The first message after connect is an id-0 line[74]:

Offset	Field	Type	Value
0	type	c	0 = player; the server treats 1 as RemoteControl[74]
1	token	8 bytes	GSERVnnn (→ §1.2.2)[74]
9	account	str	the connect dialog defaults to the built-in guest login[74]
—	password	str	[74]
—	terminator	\n	[74]

From 1999 a real game server checks the token. The table gives the token range each surviving GServer accepts and the lowest revision it lets in as a player[74].

GServer	Accepts	Minimum for a player connection
1.21 (May 1999)	GSERV007	7[74]
1.28 (Nov 1999)	GSERV007 to GSERV013	13[74]
GServer13 (Nov 1999)	GSERV007 to GSERV014	14[74]

The rules below describe how those servers check a token and refuse a login. Both an old and a newer token are refused the same way[74].

Rule
The server matches the token against a fixed list, stores its number (7 to 14) on the connection, then checks the minimum for the connection type; old tokens stay on the list so that RemoteControl (GSERV011) can log in[74]
Refusals and login errors (version too old, account disabled, account in use, no such account) are sent as id 16 (DISCMESSAGE)[74]
A token newer than the server's list is refused like an old one: GServer13 answers GSERV016 and GSERV018 with id 16[74]
The 1.33 to 1.39 clients use the same generation with GNW tokens and default to the built-in guest login[74,10]
1.39 sends id 0 with the token GNW07080, then str account and str password; it dials port 14900 with no listserver step; the layout of 1.33 to 1.38 is unmeasured[77]

Message ids and props handled per build in this generation are in → App. C.

2.3.3 2.x login (generation 3, type 0)

The 2.x login message is the first message on the connection. The table gives its fields in order, from offset 0. The key byte at offset 1 is what lets a server tell generation 3 from generation 2[89].

Offset	Field	Type	Value
0	client type	c	0 (wire 0x20)[89]

Offset	Field	Type	Value
1	key	c	rand & 0x7F, the gen-3 key (→ §2.2.2)[89,78]
2	token	8 bytes	e.g. GNW03081 (2.05), GNW22122 (2.17)[89,29]
10	account	str	[89]
—	password	str	[89]
—	terminator	\n	no identity string follows the password[89]

The rules below cover what surrounds the login message: how it is sent and built, and which messages follow it in each build[1].

Rule	Builds
The login message is sent without the gen-3 insertion; the messages after it carry it	2.00 to 2.17[1,78]
The token is built one character at a time in code, so no string literal exists	1.38 to 2.31[89,29]
PLI_LANGUAGE (37) follows the login, with a language name (English observed)	2.02 rev2 and later[89,10]
PLI_TAMPERCHECK (39) follows 37 (→ §2.4, → §4.10)	2.17 and later Delphi clients[29]
The account name is kept obfuscated in memory and decoded only to build the login message; the map is dec(b) = (~b - 10) & 0xFF	2.17, 2.22, 2.31[29]
The password is obfuscated the same way	2.31[29]
2.00 sends the same type 0 shape with the token GNW24011; the account and password are neither hidden nor hashed	2.00[90]
A lab GServer-v2 admits 2.00 once GNW24011 is in its allowed-versions list	2.00[13,90]

2.3.4 2.22, 2.31 and 3.x login (generation 5, type 5)

The fields are those of → §2.3.3 with client type 5. The message travels in the fixed mode-3 login frame (→ §2.2.4)[29,7].

Builds	Type	Token	After the login	Credentials in memory
2.22	5	GNW03014	37, then 39	obfuscated (→ §2.3.3)[29]
2.31	5	GNW28015	37, then 39	obfuscated[29]
3.04 Linux	5	G3D03014	37 only; no 39 (confirmed on the wire)	plain[7]
3.153 Mac	5	G3D28015	37 only; no 39	plain[7]

2.3.5 4.x to 6.03 login (generation 5, type 5, machine ids)

From 4.x the login message keeps the 2.x opening and adds machine identity. The table lists its fields in order[34].

Order	Field	Type
1	client type 5	c[34,37]
2	key	c, rand & 0x7F; wire bytes above 0x7F occur because the key goes up to 0x7F[37,8]
3	token	8 bytes[34]
4	account	str; 224 bytes or more → 0xFF + first 223 bytes[34]
5	password	str; same long-string rule[34]
6	machine list	comma-separated text, below[34]

The machine fields differ by platform. Each row gives the five fields one family of builds sends. Most are MD5 hashes, so a server sees a stable value per machine but never the raw MAC or serial[34].

Builds	Field 1	Field 2	Field 3	Field 4	Field 5
Windows 4.20r8, 5.00r7	`win`	MD5 hex of the Windows `DigitalProductId`	MD5 hex of the network adapter MAC (fallback: node part of `UuidCreateSequential`, or `UuidCreate`)	MD5 hex of the `C:\` volume serial	none
Mac 4.20r6	`mac`	hash of the primary Ethernet MAC	hash of 4 bytes of `stat("/etc")`	empty	none[34]
Windows 5.12, 5.24r3 to 6.03r7	`win`	MD5 hex of the product id (5.24 and later also read `DigitalProductId4`, so the hash differs from 5.12)	MD5 hex of the adapter MAC	MD5 hex of the `C:\` serial	quoted Windows version "`<major>.<minor> <build> <service pack>`"
Linux 6.01r1, 6.03r7	`linux`	MD5 hex of the `eth0` MAC	MD5 hex of the inode number of `/etc`	empty	kernel release and version string[39]

The rules below say what the machine fields are for and what a late client sends right after logging in[37].

Rule	Builds
The machine fields let a server ban per machine; 2.x and 3.x send no machine id	4.x and later[37,34]
PLI_LANGUAGE (37) follows, then the socket switches to dynamic mode	4.x to 6.03[37,39]
After 37 the client requests basepackage.gupd (23, twice), then arial.ttf, bigmap.txt or minimap.txt by build, then 34 for a GUI image (→ §2.7)	5.12 to 6.03r7 Windows[8]
Each build logs in by itself as the built-in guest once it reaches the login screen; the guest password is fixed per build family and is decrypted at run time on Linux 6.x	5.12 to 6.03r7[8,39]
The first connection goes to a game server named Login, not to a separate listserver; the client reaches a game only after S→C 178 (PLO_SERVERWARP) and logs in again to that host (→ §6)	4.x to 6.03[34,37,39]
6.11 asks for a connector script before the game socket; the script builds the login itself, so its token and layout are unmeasured (→ §2.10)	6.11 Mac[85]

2.3.6 Server parse and reply sequence

2.3.6.1 How a GServer-v2 parses the login

A server has to tell the generations apart from the first bytes alone. The steps below are how GServer-v2 does it, in order. Step 2 is the trick that separates generation 3 from generation 2[89].

Step	Rule
1	read the type GCHAR; type = 1 << value[89]
2	for type 0, read 8 bytes as a token; if they are not a known token (the gen-3 key comes first), rewind one byte, switch to generation 3, read the key, then the real token[89]
3	read the account and password str; an optional identity string read exists for later clients[89]
4	map the token to a version id (→ §1.3); GNW03081 sorts below 2.1, so 2.05 gets every pre-2.1 code path (no file modification times, PLPROP_GANI sent as the bow slot, the 1.41 level sequence)[89]

Step 1 turns the first byte into a connection type. The table maps each value to the kind of connection it opens[89].

Type value	Connection
0	client (type 0: generation 2 or 3)[89]
1	RC[89]
2	NPC-server[89]
3	NC[89]
4	client 2.19 to 2.21[89]
5	client 2.22 and later[89]
6	RC2[89]
8	web[89]

Other login tokens the server knows: NC NCL11012 and NCL21075, RC GSERV023 to GSERV025, NPC-server GRNS0000 and LNX00001[89].

2.3.6.2 Reply sequences

After a good login the server sends a fixed burst before play starts. Each row gives that burst for one server, in the order captured. Differences between rows are messages a server version added[74].

Server	Messages after a good player login, in order
GServer13	9 (own props), 28 (flags, only if the account has any), 14 (warp), 100 then a separate frame with 101 and the board, 6 (level name), 1 (links), 5 (signs), 39 (level modification time), 3 (NPC props with scripts), 10[74]
GServer 1.28	as GServer13 without 39[74]
GServer 1.21	9, 14, 100, 101 with the board, 6, 1, 10, all in one zlib frame; no 39[74]
any of the above, another player already on the level	8 (other player's props) right after 9[74]

Server	Messages after a good player login, in order
GServer-v2 as a `Login` stand-in, to 5.x and 6.x clients	190, the level load (14, 6, 100, 101, chests, 156, 3), staff and status lists (47, 180), start and HTML messages (179, 41), 182 (the client answers with its window titles on 44); 178 is not in this burst[85]

The bytes a public 5.x or 6.x `Login` server sends are unobserved (→ App. E#3).

2.3.7 Unpacked 6.037 against unpacked 6.1.1.3

The unpacked 6.037 and 6.1.1.3 images were compared for how they log in. Each rule is one difference found by reading strings and code, not by a capture[91].

Rule	Builds
6.037 stores `g3d0311c`, upper-cases it, and sends it as id 5 on the generation-5 framer. 6.1.1.3 has no `G3D` or `GNP` string	6.037, 6.1.1.3[91]
6.1.1.3 adds `setprotocol`, `sendraw` and `senddata`. Those names are absent from 6.037	6.1.1.3[91]
6.1.1.3 names `tclient_setplayerprops`, `tclient_setotherplayerprops`, `tclient_setnpcprops` and `tclient_setbaddyprops`. 6.037 has none of those names. The property-field bytes were not compared	6.037, 6.1.1.3[91]

2.3.8 Linux local-login baselines

Seven unique original Linux executable identities reach accepted local logins on one pinned patched lab server. Exact tokens preserve shared 4.20 revision and 6.01 architecture pairs without collapsing their executable identities. Visual results range from a tiled surface to black windows; acceptance is not gameplay, official authentication or a proven cipher-step discriminator.[92].

2.4 PLI message reference (client to server)

Field notation is in → §0.6, the message envelope in → §2.1.2, and the login message (the first message of a connection) in → §2.3. Server-to-client messages are in → §2.5, prop streams in → §2.6, and the staff tools' own ids in → §7. Every id, with its wire byte, is indexed in → App. A.

> Status: draft. Layouts are given where a note measured or read them; ids with no layout in the notes say `unmeasured`.

2.4.1 Conventions

Every client-to-server message (PLI) follows the same few rules. The table states them once, with the builds they hold for, so the chapter's per-message tables need not repeat them[2].

Rule	Builds
A PLI is `GCHAR(id)`, the payload, then `\n`, sent inside the frame of the build's generation	1.21 to 6.x[2,39]
Ids are logical; the wire byte is the id + `0x20`	all binary generations[3]
Coordinates marked x×2 are half-tiles; the receiver divides by 2	2.05 and the server[2]
Id 0 is overloaded: the login line starts with GCHAR 0 because that byte is the client type, and later `PLI_LEVELWARP` messages also use id 0; the server tells them apart by connection state	2.x[3]
In 6.x the login message is sent under id 5 (`PLI_BOMBDEL`) inside the mode-3 frame	6.01r1, 6.03r7[39]
6.x sends every PLI through one send method taking `(id, body)`; it writes `id + 32`, the body and `\n`	6.01r1, 6.03r7[39]
Before ids 93 and 163, 6.x first sends id 50 (`PLI_RAWDATA`) with a 3-byte length (three 7-bit groups, capped at `0x37FFFF`)	6.01r1, 6.03r7[39]
Before a long id 93 payload, 4.20r8 and 5.00r7 send id 50 with an `i3` length	4.20r8, 5.00r7[34,37]
A server re-emits most level-object PLIs to the rest of the level with `s player id` added in front; the PLO forms are in §2.5	server[2]

Whether a level-object PLI also goes, or only goes, over the UDP peer channel is in → §2.9.

2.4.2 Ids sent by build

The *First sent* column of the tables below names the earliest build in this table that sends the id; a later gap is noted only where the later list is complete. A list marked *floor* holds only the ids the binary passes to its send routine as constants; sends that pass the id in a register are not counted[53].

Build	Ids sent	Complete?
1.21	0, 10–13, 15–26	floor; disputed → App. E#6[53]
1.28 beta 2	0, 15–21, 23, 25, 28, 29	floor (fewer constants than 1.21 for the reason above)[53]
1.3 test	0, 14–21, 23, 25, 28–30	floor[53]
1.31, 1.32	0, 10–21, 23–30	floor; 1.32 disputed → App. E#6[53]
1.33 to 1.37	0–31	yes for constant ids; a few register sends uncounted[54]
1.38	0–24, 26–33	as above[54]
1.39	0–24, 26–33, 80, 81	as above[54]
1.41 rev1	0–24, 26–30, 32, 33, 80, 81	unmeasured[55]
1.42 (NewMain), NewMain 24 and 26, 2.00 (rev0–rev3)	0–24, 26–30, 32–34, 80, 81	unmeasured[55,93,94]
2.01 (+rev1), 2.02, 2.02 rev1	0–24, 26–30, 32–36, 80, 81	unmeasured[93,94]
2.02 rev2	0–24, 26–30, 32–37, 80, 81	unmeasured[93,94]

Build	Ids sent	Complete?
2.03, 2.04, 2.05 (rev0–rev2), 2.1 test 2 and test 3	0–24, 26–30, 32–38, 80, 81	unmeasured[93,94]
NewWorld beta 1, beta 2	the 2.1 test set without 7, 8, 11, 15, 16, 17, 20; disputed → App. E#11	unmeasured[55,28]
2.14 to 2.16	the 2.1 set plus 41	inferred; no binaries of 2.1 to 2.16 in the archive[55]
2.17 (the 2.17.1 binary)	0–24, 26–30, 32–41, 43–45, 80, 81	unmeasured[55]
2.22	0–24, 26–30, 32–35, 37–41, 43, 44, 80	floor; 36, 42 and 45 not matched to a call site[30,56]
2.31	the 2.22 set without 21, plus 46	floor[30,56]
3.04 (Linux)	0–24, 26–30, 32–38, 40, 41, 43, 80, 81	yes, every game-code send site; listserver sends excluded[31,57]
3.153 (Mac OS X)	the 3.04 set plus 46, 50, 93, 96–98, 152, 154–158	no: 2, 3, 15, 24, 29 and 80 assumed from 3.04[31]
4.04r2	1, 2, 4–10, 12–19, 21–24, 26, 28, 29, 32, 33, 35–37, 41, 44, 50, 79–81, 92, 96, 130, 152, 154, 157, 158, 161	floor[58]
4.20r6 (Mac)	the 4.04r2 set plus 34, 46, 162	floor[58]
4.20r8	1, 2, 4–10, 12–19, 21–24, 26, 28, 29, 32, 33, 35–37, 41, 44, 50, 79–81, 92, 96, 130, 152, 154, 157, 158, 161, 162	floor; some sends pass the id in a register[34,58]
5.00r7	1, 2, 4–10, 12–19, 21–24, 26, 28, 29, 32–37, 41, 44, 47, 50, 79–81, 92, 93, 96–98, 130, 152, 154–159, 161	floor; whether 11 is in the list is disputed → App. E#4[37,59]
5.12	1, 2, 11, 23, 24, 34, 37, 38, 44, 47, 157 (seen in capture)	floor[59]
5.24r4	the 5.12 set plus 158 (seen in capture)	floor[59]
5.30r3	1, 11, 37, 38 (seen in capture)	floor[59]
5.31r4	2, 37 (seen in capture)	floor[59]
6.00r2, 6.015	0–20, 22–24, 26–30, 32–38, 41, 44–48, 50, 79, 92, 93, 96–98, 130, 152, 154–159, 161, 162	inferred from the 6.01 and 6.03 reads[59]
6.01r1	the 6.03r7 set without 160	unmeasured[39]
6.034	as 6.03r7	inferred[59]
6.03r7	0–20, 22–24, 26–30, 32–38, 41, 44–48, 50, 79, 92, 93, 96–98, 130, 152, 154–162	yes[39,59]

The id table above says which ids a build can send. The rules below add the exceptions found in particular builds: ids with no sender, ids a family drops or adds, and a limit on one sender[3].

Rule	Builds
No sender was found for 25 (`PLI_UNKNOWN25`) or 31 (`PLI_PACKETCOUNT`)	2.05[3]
6.x sends none of 21 (`PUTNPC`), 25, 31, 39 (`TAMPERCHECK`) or 40 (`SHOOT`) as a constant	6.01r1, 6.03r7[39]
3.04 and 3.153 send no 39 or 44, unlike their Windows twin, and add 36 and 81	3.04, 3.153[31]
6 (`PLI_TOALL`) is a real sender in every build of the family, with the chat text cut to 200 characters before sending	2.00 to 2.1 test[95,96]
No id is removed anywhere from 2.00 to 2.1 test 3, and none of 25, 31 or any id from 39 up has a constant send, so the tamper check (39) is not part of this family	2.00 to 2.1 test[95,97]
36 and 38 look like TCP copies of UDP 25 and 26: one routine sends the UDP message, then repeats it over TCP when a flag says the server needs it; what sets the flag is unknown	2.01 to 2.1 test[95]
2.02 rev1 has no `PLI_LANGUAGE` (37) send site and sends none in the lab; rev2 is the first build that sends 37	2.02 rev1, rev2[55]
Ids 4.x adds over 3.x: 44 (window list for PLO 182), 50 (large-packet length), 157 and 161 (file requests carrying an i5 time), 158 (weapon by name), 162	4.20r8[34]

The summary lists for 1.21 and 1.32 in the 1999 protocol note disagree with the per-build profile by one id each → App. E#6.

2.4.3 Level navigation

These are the messages a client sends to move between levels and servers. Each row gives the id and name, the payload layout, the first build that sends it, and what the server does in reply[2].

Id	Name	Layout	First sent	Server action
0	PLI_LEVELWARP	c x×2, c y×2, rest = level name	1.21	warps the player and sends the level (→ §2.5)[2]
30	PLI_LEVELWARPMOD	i5 modtime, c x×2, c y×2, rest = level name	1.3 test	as 0, but the server can answer with only the changes since modtime[2]
35	PLI_ADJACENTLEVEL	i5 modtime, rest = level name; modtime is all zeros when the level is not cached	2.01	sends a gmap neighbour for preloading[2]
41	PLI_SERVERWARP	rest = server name	2.17 (2.14 inferred)	the game server asks the listserver and answers with PLO_SERVERWARP (178)[3,65]
46	PLI_ENTERLEVEL	c 1	2.31; not 3.04	none recorded[3,39]

The rules below say when a build picks one of those messages over another, and what else a warp sends[2].

Rule	Builds
2.05 sends 30 instead of 0 when it has the level cached, in memory or in its web-level cache	2.05[2]
A warp to the level the player is already in only moves the player and sends nothing, in most states	2.05[2]
A real warp also sends UDP id 24 to gmap neighbours (→ §2.9)	2.05[2]
2.17 has one send site for 30 instead of two	2.17[55]
46 is sent on every level entry (login, gmap edge crossings, doors, edge links, server warps), just before the first props message of the new level, in the same frame	2.31[3,62,98,99,64]
46 comes from one send site at the start of the routine that switches the current level, whenever connected; its GCHAR 1 is a constant in the code; 2.17.1 and 2.22 have no send site	2.31; 2.17.1, 2.22[100]
At login, 46 is the first message of the client's second frame, before 35 and 44	2.31[62]
On a gmap cell crossing the props after 46 carry the new cell in props 43 or 44 and a local coordinate that starts again at the cell edge	2.31[62]
2.22 sends no 46; 2.31 does not re-send id 39 on level entry	2.22, 2.31[3]

The client's handling of the PLO_SERVERWARP reply is in → §2.5; the listserver side is in → §6.

2.4.4 Props

Three messages carry property streams from the client: its own player, NPCs and baddies. The table gives each one's layout and what the server does with it. The streams themselves are in §2.6[2].

Id	Name	Layout	First sent	Server action
2	PLI_PLAYERPROPS	rest = PLPROP stream (→ §2.6)	1.33	stores the props and forwards them to the level as PLO_OTHERPLPROPS[2,101]
3	PLI_NPCPROPS	rest = NPCPROP stream (→ §2.6)	1.33	passed through[2]
15	PLI_BADDYPROPS	rest = c id + BDPROP stream (→ §2.6)	1.21; not NewWorld betas	re-emitted as PLO_BADDYPROPS (2)[2]

Movement has no message of its own. The rules below show how walking rides on player props, with example bodies decoded byte by byte, and when 2.05 sends which props[89].

Rule	Builds
There is no movement message: a walk step is a PLI_PLAYERPROPS carrying X (15), Y (16) and SPRITE (17), so a walk frame starts with " (0x22)	2.05, 2.17[89]
Example bodies: 1- is SPRITE 13; /[is X 59 (29.5 tiles); 0f is Y 70 (35.0 tiles); *$walk/I is GANI walk then X 41	capture of 2026-10-04; the walk frame from 2.17[89]
sprite % 4 is the direction: 0 up, 1 left, 2 down, 3 right	2.22, server[89]
2.05 sends prop groups {0, 11, 13, 18, 31, 33} at session start, {42, 15, 16} on attach or a position change, and a per-tick diff of changed props	2.05[89]

Rule	Builds
Local chat is not a message id: it is the CURCHAT prop (12) inside PLI_PLAYERPROPS, and server commands typed in chat are parsed from it	server[101]
The login props go out as one PLI_PLAYERPROPS right after the first server message, with the head (11) as an index below 100	1.41 rev1[55]
Player props stay on TCP while the UDP channel is active	2.05[2]

2.4.5 Combat and level objects

Layouts read from the 2.05 senders and the server source. The server copy (PLO) and its receiver rules are in → §2.5.

Id	Name	Layout	First sent	Server action
1	PLI_BOARDMODIFY	c x, c y, c w, c h (whole tiles), then w × h s tile, row-major; 2.05 sends exactly one run	1.33	rebroadcast as PLO_BOARDMODIFY (7)[2]
4	PLI_BOMBADD	c x×2, c y×2, c power (+8 when the bomb is the sender's), c fuse	1.33	re-emitted as PLO_BOMBADD (11)[2]
5	PLI_BOMBDEL	c x×2, c y×2	1.33	re-emitted as PLO_BOMBDEL (12) with no player id[2]
7	PLI_HORSEADD	c x×2, c y×2, c (dir + 4 × bushes), rest = image file name	1.33; not NewWorld betas	re-emitted as PLO_HORSEADD (17)[2]
8	PLI_HORSEDEL	c x×2, c y×2	1.33; not NewWorld betas	re-emitted as PLO_HORSEDEL (18)[2]
9	PLI_ARROWADD	c x×2, c y×2, c flags, c sprite, then the projectile type sent as v + 33 (0 arrow, 1 fireball, 2 fireblast, 3 nuke); flags = dir & 3, +4 first removes a matching shot, +8 owned	1.33	re-emitted as PLO_ARROWADD (19)[2]
10	PLI_FIRESPY	c (a × 8 + b)	1.21	re-emitted as PLO_FIRESPY (20)[2]
11	PLI_THROWCARRIED	empty	1.21; not NewWorld betas	re-emitted as PLO_THROWCARRIED (21)[2]
12	PLI_ITEMADD	c x×2, c y×2, c item	1.21	re-emitted as PLO_ITEMADD (22)[2]
13	PLI_ITEMDEL	c x×2, c y×2	1.21	re-emitted as PLO_ITEMDEL (23) to the other players only[2,2]
16	PLI_BADDYHURT	c id, c (dx + 4) × 16, c (dy + 4) × 16, c power	1.21; not NewWorld betas	re-emitted as PLO_BADDYHURT (27); applied only by the level leader[2]
17	PLI_BADDYADD	c x×2, c y×2, c type, c power, rest = image	1.21; not NewWorld betas	answers with PLO_BADDYPROPS (2)[2]
20	PLI_OPENCHEST	c x, c y (whole tiles)	1.21; not NewWorld betas	replies PLO_LEVELCHEST (4) with opened = 1[2]
26	PLI_HURTPLAYER	s victim, c (dx + 4) × 16, c (dy + 4) × 16, c power, optional i3 npc	1.21	sends PLO_HURTPLAYER (40) with the attacker's id[2]
27	PLI_EXPLOSION	c radius, c x×2, c y×2, c power (+8 when the explosion is the sender's)	1.31	re-emitted as PLO_EXPLOSION (36)[2]
32	PLI_ITEMTAKE	c x×2, c y×2, c item	1.38	unmeasured[2]
36	PLI_HITOBJECTS	c power × 2, c x×2, c y×2, optional i3 npc	2.01; not matched in 2.22 or 2.31	re-emitted as PLO_HITOBJECTS (46)[2]
38	PLI_TRIGGERACTION	i3 npc (0 = none), c x×2, c y×2, rest = action or action,params	2.03	re-emitted as PLO_TRIGGERACTION (48)[2,3]
40	PLI_SHOOT	unmeasured	2.17; not 6.03r7	unmeasured[55,39]

Id	Name	Layout	First sent	Server action
48	PLI_SHOOT2	unmeasured	6.01r1	unmeasured[39]

The table above gives the combat and board messages. The rules below record how they were seen on the wire, including one place where 2.05 and the server read a byte differently[2].

Rule	Builds
GServer-v2 reads the bomb's third byte as `(player << 2) | power`; 2.05 writes `power + 8`, and a receiver takes the power as `v & 7` and bit 3 as "mine"	2.05 vs server[2]
Captured bomb: x 78, y 28, power byte 9 (power 1 plus the owner flag), fuse 55	2.05[2]
The fuse is probably in 0.05 s steps (55 ≈ 2.75 s)	2.05[2]
When a bomb destroyed bushes, the other player (not the bomb's owner) sent one PLI_BOARDMODIFY per destroyed 2 × 2 patch, eight in one frame; why that player reported is not settled	2.05[2]
Captured board runs: a cut bush 2 × 2 = tiles 1834, 1835, 1850, 1851; cut grass = 677, 678, 693, 694	2.05[2]
Item drops after a bush cut came from the server (`bushitems` on); no client sent PLI_ITEMADD	2.05 with GServer-v2[2]
The bush and sign pickup path also sends PLI_BOARDMODIFY	2.05[2]
2.05 sends only weapon type 0 in id 33 (→ §2.4.6)	2.05[2]

The server's respawn of board changes is behaviour and is in → §4.5.

2.4.6 Flags, NPCs and weapons

These messages change flags, put or remove NPCs and manage weapons. Each row gives the layout, the first build that sends it and the server's action where it is known[2].

Id	Name	Layout	First sent	Server action
18	PLI_FLAGSET	`rest` = flag (`name` or `name=value`)	1.21	stores the flag[2]
19	PLI_FLAGDEL	`rest` = flag name	1.21	removes the flag[2]
21	PLI_PUTNPC	`str image, str script file, c x×2, c y×2`	1.21; not 2.31 or 6.03r7	unmeasured[2,30]
22	PLI_NPCDEL	`i3 npc id`; only for NPCs with id > 0	1.21	unmeasured[2]
29	PLI_NPCWEAPONDEL	`rest` = weapon name	1.28 beta 2	removes the weapon[2]
33	PLI_WEAPONADD	`c type`; type 0 → `c item index`; type 1 → `i3 npc id` (copy an NPC's script as a weapon)	1.38	adds the weapon and answers with PLO_NPCWEAPONADD (33)[2]

Three rules limit when those messages are sent at all[2].

Rule	Builds
Flags whose name starts with `local.` are never sent	2.05[2]
When the deleted NPC is one of the player's weapons, 2.05 sends PLI_NPCWEAPONDEL (29) with the weapon name instead of PLI_NPCDEL	2.05[2]
2.05 only sends weapon type 0	2.05[2]

2.4.7 Files, scripts and server values

The download replies and their framing are in → §2.7.

Id	Name	Layout	First sent	Server action
23	PLI_WANTFILE	`rest` = file name	1.21	sends the file (→ §2.7)[2]
34	PLI_UPDATEFILE	`i5 modtime, rest` = file name	1.42 (NewMain)	sends the file if it is newer than `modtime`[2]
47	PLI_VERIFYWANTSEND	`i5 crc32, str file name`	5.00r7	sends the file if the client's copy is stale[3,37]
50	PLI_RAWDATA	`i3 length` of the next message (→ §2.4.1)	3.153	reads the next *length* bytes as one message[39,34]

Id	Name	Layout	First sent	Server action
130	PLI_REQUESTUPDATEBOARD	`c` level-name `length`, level name, `i5` modtime, `s` x, `s` y, `s` width, `s` height	4.04r2	unmeasured[3,39]
152	PLI_REQUESTTEXT	unmeasured (reads a server value)	3.153	answers with the value[3,37]
154	PLI_SENDTEXT	unmeasured (writes a server value)	3.153	stores the value[3,37]
157	PLI_UPDATEGANI	unmeasured beyond an `i5` time (a gani request)	3.153	sends the gani script[3,34]
158	PLI_UPDATESCRIPT	`rest` = script name	3.153	sends the weapon or class script[3,37]
159	PLI_UPDATEPACKAGEREQUESTFILE	an order value (meaning unmeasured), package file name, modtime	5.00r7	unmeasured[3]
161	PLI_UPDATECLASS	`i5` modtime, `rest` = class name	4.04r2	sends the class[3,34]

The 4.x and 5.x clients reuse some of these ids for new jobs. The rules below record those extra uses[37].

Rule	Builds
5.00r7's update-package downloader uses 34 and 47 (with and without a modification time)	5.00r7[37]
158 is sent after a cache miss in the client's script cache; 4.x uses it for the getweapon path	4.20r8, 5.00r7[34,37]
154 is also used by the options and secure-URL code	5.00r7[37]

2.4.8 Chat, private messages and profiles

How chat is shown, and the server-to-client forms, are in → §2.8.

Id	Name	Layout	First sent	Server action
6	PLI_TOALL	`str` message; 2.05 sends the chat-bar text after a `toall:` prefix, trimmed, at most 200 bytes	1.33	sends PLO_TOALL (13) with the sender's id[101]
28	PLI_PRIVATEMESSAGE	`s` count, count × `s` player id, `rest` = message (gtokenized)	1.28 beta 2	sends PLO_PRIVATEMESSAGE (37) to each listed player[101]
80	PLI_PROFILEGET	`rest` = account	1.39	answers with PLO_PROFILE (75), relayed from the listserver[101]
81	PLI_PROFILESET	ten fields, each `c` length + text (→ §2.8)	1.39; not 6.03r7	forwards to the listserver[101]

The rules below say how a client chooses between a chat message and the chat prop, and warn about one field a server should not trust[101].

Rule	Builds
The chat-bar text is sent as 6 only when, lower-cased and trimmed, it starts with `toall:`; all other text goes out as the CURCHAT prop (→ §2.4.4)	2.05[101]
5.00r7 sends 6 for chat-bar `toall` commands	5.00r7[37]
Field 0 of PLI_PROFILESET is the profile owner's account, which can be another online player's; a server must not take it as proof of who is editing	2.05[101]

2.4.9 Client checks and housekeeping

These are the ids a client sends for checks and upkeep rather than for play. Several have no known server action, and the table says so rather than guess[2].

Id	Name	Layout	First sent	Server action
14	PLI_CLAIMPKER	`s` player id	1.3 test	unmeasured[2]
24	PLI_SHOWIMGPLAYER	`rest`, passed through	1.21	relays it unchanged as PLO_SHOWIMGPLAYER (32) with `s` sender id in front[2,66]
25	PLI_UNKNOWN25	as PLO_NPCWEAPONADD: name, image, script	1.21; last seen 1.37	a weapon upload; GServer-v2 beta4 kicks the client for it[102,102,53,54]

Id	Name	Layout	First sent	Server action
31	PLI_PACKETCOUNT	unmeasured	1.33; last seen 1.39	unmeasured[54]
37	PLI_LANGUAGE	rest = language name (English seen)	2.02 rev2	stored[30,39]
39	PLI_TAMPERCHECK	10 raw bytes, a constant per build (not two i5)	2.17, 2.22, 2.31; not 3.04, 3.153 or 6.x	GServer-v2 reads and discards it[3,30]
43	PLI_MUTEPLAYER	s player id, c on (1 mute, 0 unmute); one message per click in the player list	2.17; not 6.03r7	unmeasured[3,84]
44	PLI_PROCESSLIST	the client's window list, comma-separated window titles	2.17; not 3.04	the reply to PLO_LISTPROCESSES (182)[3,55]
45	PLI_PROCESSRUNNING	c flag: 1 if the server's string occurs in the client's own process list	2.17	the reply to PLO_HASPROCESSRUNNING (183)[3,39]
162	PLI_RC_UNKNOWN162	game clients: ! + s value; RCs: empty (→ App. E#5)	4.20r6	a ping: the game clients' answer to PLO_UNKNOWN108, an RC's 60 s keepalive; accept and ignore, or time it against 108[102,34,39,3]
252	PLI_SET_ENC_KEY	sets the AES or RC4 key and IV; above 223, so not sendable as one GCHAR (→ §2.10)	newer gs2lib only	unmeasured[3]
253	PLI_BUNDLE	several messages in one (→ §2.10)	newer gs2lib only	unmeasured[3]

Ids 24 and 39 were studied from captures. The rules give their timing and the exact bytes seen, and what tests showed about their meaning[3].

Rule	Builds
24 is also sent on its own about every 5.1 s with one byte (value 9)	2.31; 2.22 only while another player is in the level; 2.17 every 5 s after login[3,55,99]
The meaning of the 24 payload: receivers showed no visible reaction to the values 0, 1, 5, 9, 14, 58, 95, to the bytes xx.png, or to an empty payload, and the server neither disconnected nor logged an invalid packet; the client reads show it is the weapon-image list of → §2.4.9.1	2.31 sender, 2.x receiver[66,103]
39 is sent right after PLI_LANGUAGE at login	2.17, 2.22, 2.31[3]
39 bytes: 2.17 2a 7b ad 19 5e 23 12 43 d8 2c, 2.22 25 0e 4d 0f 96 2b 03 ed e6 31, 2.31 29 16 55 b7 60 2e 87 c9 eb 82; identical across logins, accounts and login keys	2.17, 2.22, 2.31[104]
39 re-sends: 2.17 once at login; 2.22 at login, on every gmap edge crossing and after every client-initiated level-link request, but not after a server-side warp; 2.31 at login and then about every 3.2 s	2.17, 2.22, 2.31[30,104,84,105]
How the 39 bytes are computed is a client integrity check (→ §4.10)	2.17, 2.22, 2.31[30]
Because GServer-v2 discards 39, whether a different value would be rejected is unmeasured	server[30]
44 is sent just before the login props	2.17[55]

2.4.9.1 The id 24 weapon-image list

Id 24 carries the images that the sender's weapon scripts show (showimg and the changeimg* commands), as changes against the list sent last; the relay is PLO_SHOWIMGPLAYER (32):

Rule	Builds
a record starts with 0x2a + the image index (0 to 199); tag bytes 0x20 + tag follow with their data; a byte of 0x2a or more starts the next record; an unknown tag skips one data byte; a tag with no data ends the text	2.05, 2.17.1, 2.22, 2.31[103,106]
tag 0 name (GCHAR length + bytes; empty removes the image), 1 x, 2 y (half tiles, or screen units at layer 10 and over; units unconfirmed), 3 layer − 1, 4 visible part (0x20 whole image, or 0x21 + left and top as two 7-bit bytes each + width and height, clamped to 0..16000 and 0..220), 5 red, green, blue and alpha over 200, 6 zoom over 10, 7 draw order − 0x52, 8 mode	2.17.1 to 2.31 (receiver read in 2.31)[107]

Rule	Builds
2.05 sends tags 0 to 6 only: layer only when not 1, part only when its size is above 0, colours only when one is not 1.0, zoom only when not 1.0	2.05[106]
a leading 0x29 (GCHAR 9) marks a full list; after it the receiver removes every image of that player the text did not name; with no images the full list is the marker alone, the one byte every real client was seen to send	2.05 to 2.31[103,106]
a full list goes out when a countdown wraps: every 61st call in 2.05, every 102 ticks (101 calls, the call after a send skipped) in 2.17.1 to 2.31, which fits the 5.1 s seen at 50 ms a tick; the first tick after login is a wrap; an empty text is not sent	2.05; 2.17.1 to 2.31[106,103]
a UDP copy goes out when it can (id 15 in 2.05, → §2.9); the TCP copy goes out when UDP is off for the client or does not reach some player in the level; 2.31 always sends the TCP copy; a wrap that finds the test false is lost	2.05, 2.17.1, 2.22; 2.31[106,103]
a new image gets layer 1, colours 200 each, zoom 10 and mode 0; fields other than the name apply only to an image that exists	2.31[103]
the sender's own value formatting, the "UDP reaches this player" flag of the 2.22 test, and how the images are drawn are unmeasured	2.17.1 to 2.31[108]
an earlier note's four-field form (index, x, y, name) for 2.05 and earlier is wrong for 2.05; builds before 2.05 were not read	2.05[106]

2.4.10 Staff messages sent by game clients

The RC and NC ids (51 to 119, 150, 151, 155, 156, 160) are documented with the staff tools in → §7. Some game clients send a few of them:

Id	Name	Builds	Use in a game client
79	PLI_RC_CHAT	4.x, 5.00r7, 6.x	unmeasured[34,39]
92, 93	PLI_RC_FILEBROWSER_DOWN, _UP	92: 4.x, 5.00r7, 6.x; 93: 3.153, 5.00r7, 6.x	93 carries a file-name string and one data chunk, behind id 50[39,31]
96, 97, 98	PLI_RC_FILEBROWSER_MOVE, _DELETE, _RENAME	3.153, 5.00r7, 6.x; 96 also 4.x	file browser[39,31]
155, 156	PLI_RC_LARGEFILESTART, _END	3.153, 5.00r7, 6.x	the upload queue: 155, then 93 chunks of at most 32,000 bytes, then 156; the queue pauses while about 64 KB is waiting to go out[39]
160	PLI_RC_FOLDERDELETE	6.03r7 only	folder delete[39]

The 6.11 client sends no fixed ids: a script-loaded table maps each outgoing message to a wire id, so its named send slots are in → §2.10.

2.4.11 Login-time order

The login message itself and the server's reply are in → §2.3. What the client sends next:

Build	Messages after the login message, in order
1.41 rev1	one PLI_PLAYERPROPS with the login props, right after the first server message; no file requests[55]
2.02 rev1	file requests and the login props; no 37[55]
2.17	37, then 39 once; 44 with the window titles just before the login props[55,104]
2.22	37 English, 39, then 34 for the default images (sword, shield, body, bomb), 23, and 2; 44 in answer to the server's 182[30]
2.31	37, 39, then in the second frame 46 before the props, 35 and 44[30,62]
6.01r1, 6.03r7	37 right after login; no 39[39]

2.4.12 Outgoing senders in unpacked 6.1.1.3 and 6.0.3.7

Unpacked 6.1.1.3 has two fixed sends. `sendraw` uses id −1 and appends the caller's bytes. `senddata` sends the id and the body the caller passes.[109]

Fifty-five named actions each read one slot of a 256-wide outgoing table. A negative slot does not send. Slot 254 raises the matching `onSend` script event. Any other value is the packet id. `setoutdatahandlers` fills the table with the all-ones dword, then copies the script list, and stops at 256.[109]

Unpacked 6.0.3.7 has none of those names. Its login send is packet id 5, the upper-cased token.[109]

2.5 PLO message reference (server to client)

Field notation is in → §0.6, the message envelope in → §2.1.2, and the login reply sequences in → §2.3.6. The client-to-server messages that trigger many of these are in → §2.4, prop streams in → §2.6, and file downloads in → §2.7. Every id, with its wire byte, is indexed in → App. A.

> Status: draft. Layouts are given where a note measured or read them; ids with no layout in the notes say `unmeasured`. The staff tools' own server messages are in §7.

2.5.1 How a client reads a PLO

Before the per-message tables, the rules below say how a client treats any server-to-client message (PLO): short messages, unknown ids, and the two level pointers that decide where an NPC message lands[2].

Rule	Builds
Every handler checks a minimum length first and drops a short message silently	2.05, 2.17.1, 2.22, 2.31[2,2]
An id with no case in the receive switch falls to one shared default arm and is ignored without error	2.05 to 2.31[3,29]
2.05 keeps two level pointers: the *named level*, set by every `PLO_LEVELNAME`, and the *object level*, which receives NPC messages (3, 150, 151, 166) and is set by `PLO_SETACTIVELEVEL`, or by each `PLO_LEVELNAME` when there is no NPC server	2.05[2]
Handlers for level objects (bombs, arrows, items, explosions, hits, trigger actions, horses, push-away) act only while the player stands in the named level	2.05[2]
A message the server re-emits for another player starts with `s player id`; with an unknown id, bombs, arrows, explosions and items are still shown without an owner, while fire spy, throw carried and player hits are dropped	2.05[2]
2.17.1, 2.22 and 2.31 read "comma text" through the client's comma-text parser, which allows double quotes	2.17.1, 2.22, 2.31[2]
The receive dispatcher takes the id as the first byte minus 32, checks it against a bound, and goes through an index table; the bound is id 185 in 2.17.1, 186 in 2.22 and 189 in 2.31	2.17.1, 2.22, 2.31[110]
4.20r6 and 4.20r8 take 100 and 108 before the main switch	4.20r6, 4.20r8[34]

2.5.2 Ids handled by build

For the six checked Windows x86 2.02/2.05 revisions, the corroborated entries mean distinct static receive routes, not successful runtime handling; payloads and effects require separate evidence[111].

Builds before 2.00 are in → App. C.2. *Bound* is the highest id the receive switch tests, written as in the source.

Build	Count	Ids handled
NewMain 24, 26	51	0–45, 75, 100, 150, 151, 153 (bound `0x99`)[112]
2.00 (rev0–rev3)	53	0–45, 75, 100, 150, 151, 153–155 (bound `0x9b`)[112,90,113]
2.01 (+rev1)	55	0–46, 75, 100, 150, 151, 153–156[112,113]
2.02 (rev0–rev2)	56	0–47, 75, 100, 150, 151, 153–156[112,113,111]
2.03	58	0–48, 75, 100, 150, 151, 153–156, 165[112,113]
2.04	58	0–25, 27–48, 75, 100, 150, 151, 153–156, 165, 166 (26 dropped)[112,113]
2.05 (rev0–rev2)	59	0–25, 27–48, 75, 100, 150, 151, 153–156, 165, 166, 168 (bound `0xa8`)[112,3,113,111]
2.1 test 2, test 3	61	the 2.05 set plus 49, 169[112,113]
NewWorld beta 1	53	the 2.05 set without 2, 4, 17, 18, 154, 155[28]

Build	Count	Ids handled
NewWorld beta 2	55	the beta 1 set plus 49, 169[28]
2.17.1	79	0–25, 27–49, 68, 69, 75, 100, 150, 151, 153–156, 165, 166, 168–185[112]
2.22	80	the 2.17.1 set plus 186[112]
2.31	81	the 2.22 set without 44, plus 84 and 189[112,29]
3.04 (Linux)	79	0–49, 68, 69, 75, 84, 100–102, 150, 151, 153–156, 165, 166, 168, 169, 171–176, 178–181, 184–186[7]
3.153 (Mac OS X)	88	the 3.04 set without 44, plus 67, 74, 82, 104, 131–134, 187, 189[7]
4.20r8	101	0–25, 27–43, 45–49, 67–69, 74, 75, 82, 84, 100–102, 104–108, 131–134, 150, 151, 153–156, 165, 166, 168–176, 178–187, 189–191, 193–196[34]
4.20r6 (Mac)	99	the 4.20r8 set without 41 and 196[34]
5.00r7	100	0–25, 27–43, 45–49, 67–69, 74, 75, 82, 84, 101, 102, 104–107, 131–134, 140, 150, 151, 153–156, 165, 166, 168–176, 178–187, 189–191, 193–195, 197[37]
6.01r1	91	the 6.03r7 set without 111[39]
6.03r7	92	0–25, 27–40, 42, 43, 45–49, 67–69, 74, 82, 84, 101, 102, 104–107, 109, 111, 131, 134, 140, 150, 151, 153–156, 166, 168, 170, 172–174, 176, 178–180, 182, 183, 185–187, 189–191, 194, 195, 197, 198[39]

The id table above shows which ids each build handles. The rules below explain its gaps and steps: ids nobody handles after a point, why one revision stands for its siblings, and what changed in particular builds[3].

Rule	Builds
No build in the notes handles 26 (PLO_NPCACTION) after 2.03 (it is handled from 1.28 beta 2), or 152 and 167	2.04 to 6.03[3,29]
Revisions of one version share every handler shape (2.05 rev1 differs by one constant in 8), so any revision stands for the others; on the wire only 2.02 rev2 differs, by its token and id 37	2.00 to 2.05[114]
2.00 has no active-level reference, since 156 arrives in 2.01; level-addressed messages act on the current level	2.00[115]
2.01 rewrites 0, 14 and 15 so the handler no longer waits for and then requests the level, the code side of map preloading; 154 and 155 go through a shared helper	2.01[114]
0, 6 and 14 gain checks that skip reloading a level that is already loaded	2.1 test 3[114,28]
NewMain 24 and 26 have one handler set between them, but differ from 2.00 more than any 2.0x revision does, so NewMain is a separate branch	NewMain[28]
2.31 drops 44 (HASNPCSERVER); the 2.3 release notes say Classic switched to the NPC server	2.31[29]
3.04 lacks 170, 177, 182 and 183, which 2.22 handles, and adds 84, 101 and 102	3.04[31]
4.20r8 drops nothing against 3.153 and adds 13 ids	4.20r8[34]
5.00r7's list does not include 100	5.00r7[37]
Against 5.00r7, 6.x drops 41, 75, 132, 133, 165, 169, 171, 175, 181, 184 and 193 and adds 109, 111 and 198	6.01r1, 6.03r7[39]
A later client (5.07 and up, or 6.037, as the source note marks each id) ignores 26, 41, 44, 55, 56, 60, 78, 80, 81, 83, 132, 157, 159–164, 165, 167, 171, 175, 188 and 192	5.07, 6.037[3]

2.5.3 Level data

These messages carry a level to the client: its board, links, signs, chests, baddies and NPCs. Each row gives the id and name, the payload layout and what the client does with it[2].

Id	Name	Layout	Client action
0	PLO_LEVELBOARD	a run of board changes, each `c x, c y, c w, c h + w × h s tile` (row-major), back to back; empty = "use your cached board"	parsing stops when fewer than 4 bytes are left or a run is shorter than `w*h*2+4`; a pending warp to this level completes first[2]
1	PLO_LEVELLINK	`rest = dest x y w h newx newy`, split on spaces	see the rules below[2,116]
2	PLO_BADDYPROPS	`c id` + BDPROP stream (→ §2.6)	an unknown id creates the baddy[2]

Id	Name	Layout	Client action
3	PLO_NPCPROPS	i3 id + NPCPROP stream (→ §2.6); id > 0	ids below 1000 are level NPCs, created in the object level; 1000 and up are server-wide; a new NPC gets the playerenters event if the player is in that level; in 2.05r2 the script property (NPCPROP 1) is applied only when the NPC has no script yet, though its bytes are still consumed[2,117]
4	PLO_LEVELCHEST	c opened, c x, c y (whole tiles); if not opened, also c item, c signindex	a closed chest with fewer than 5 bytes is dropped; an opened chest draws nothing by itself, so on a board with no chest tiles the short form leaves no trace[2,118]
5	PLO_LEVELSIGN	c x, c y (whole tiles) + sign-alphabet text (→ §3)	needs at least 2 bytes[2]
6	PLO_LEVELNAME	rest = level file name	sets the named level, and the object level when there is no NPC server[2]
7	PLO_BOARDMODIFY	one run: c x, c y, c w, c h + w × h s tile, row-major	needs len ≥ w*h*2+4[2]
10	PLO_ISLEADER	empty	this client runs the level's baddies and timeouts; ignored unless the player's level is the active level[2]
17	PLO_HORSEADD	c x×2, c y×2, c (dir & 3) + 4 × bushes (bushes capped at 6), rest = image	.gif is added when the image has no extension[2]
18	PLO_HORSEDEL	c x×2, c y×2; no player id	removes any horse within 2 tiles of that point[2]
22	PLO_ITEMADD	c x×2, c y×2, c item (0–24); optional s player in bytes 3–4	unknown items ignored; with the player id, a dropped sword or shield copies that player's image[2]
23	PLO_ITEMDEL	c x×2, c y×2; no player id	removes the item at exactly that point[2]
39	PLO_LEVELMODTIME	2.05: i5 modtime, exactly 5 bytes; 2.17 to 2.31: rest, unparsed	2.05 writes the level to its web-level cache (→ §3); 2.17 to 2.31 hand the rest to the active gmap or level object, if one exists[2,119]
42	PLO_NEWWORLDTIME	i4 time (b0<<21 | b1<<14 | b2<<7 | b3)	needs at least 4 bytes[2]
100	PLO_RAWDATA	i3 length (→ §2.1.3)	the next *length* bytes are one message[1,119]
101	PLO_BOARDPACKET	behind 100: i3 (8194), then c 101 + 4096 × u16le raw tile ids + \n	replaces the 64 × 64 board[2]
107	PLO_BOARDLAYER	4.20r8: layer, x, y (each under 64), width and height (1–64), then two bytes per tile	a board change on a layer; layer 0 also clears the level's pending changes; 5.00r7 raises the script level to at least 5 and passes the data to a script object[34,37]
150	PLO_NPCDEL2	c n, n-byte level name, i3 npc id	removes that NPC from that level[119]
151	PLO_HIDENPCS	empty	removes NPCs with ids 1 to 999 from the active level (id 0 excluded)[119]
156	PLO_SETACTIVELEVEL	rest = level or .gmap name	sets the object level; a .gmap is loaded and made active; 2.31 adds a follow-up step whose purpose is unresolved[2,119]
185	PLO_BOARDHEIGHTS	c mapX, c mapY, c blockX, c blockY, c w−1, c h−1, then w × h cells of 2 raw bytes, row-major	height = (b0 − 0x20 − 0x32) + (b1 − 0x20) / 128; written into that gmap cell's height grid from blockX, blockY; mapX and mapY must lie in the gmap grid[119]
186	PLO_BOARDMODIFY2	c mapX, c mapY, c x, c y, c w, c h, then w × h s tile, row-major	tiles replace that gmap cell's board from x, y; needs len ≥ 6 + 2*w*h[119]

Links need the most care of the level messages. The rules below give how a link is parsed and when the client takes it, from the binaries and from lab measurements[116].

Rule	Builds
A link needs at least 5 space-separated fields, else the client raises "Wrong link data!"; x y w h are whole tiles read as floats; newx and newy are fields 5 and 6 or empty, and the literal -1 becomes playerx or playery	2.00, 2.04, 2.05 rev2, 2.17.1, 2.22, 2.31[116]
2.17.1 and 2.22 also have a helper that builds a default link with newx and newy preset to playerx and playery, used for editor and edge cases	2.17.1, 2.22[116]
dest is field 0, so a level name with a space cannot be linked	2.00 to 2.31[116]

Rule	Builds
The client takes a link when one point enters its rectangle: the player's centre (x+1.5, y+2), 1 to 1.5 tiles ahead in the walking direction; a 1×1 link off that centre line is walked past. Measured to half a tile in straight walks only	2.05 rev2 (lab, one fork server)[120]
Only PLO_BOARDPACKET carries raw little-endian tiles; every other board message uses GCHAR headers and GSHORT tiles	server[2]
186: bytes after the tiles are not read, so a complete second record in the trailer is not applied; a payload one byte short is dropped silently with no disconnect; a block past column 63 is clipped	2.22, 2.31 (lab, injected frames; no server seen sending 186)[61,3,98]
In a two-client capture the server sent PLO_BOARDMODIFY with the reported bytes to both clients, the reporter included, and 15.0 s later (respawntime = 15) sent the original tiles back	2.05 with GServer-v2[2]
On level entry an empty PLO_LEVELBOARD followed the board packet and PLO_LEVELMODTIME	2.05 with GServer-v2[2]

2.5.3.1 Level send order (pre-2.1 server path)

When a pre-2.1 client enters a level, the server sends the messages above in a fixed order. The steps below give that order. Step 1 is the short path for a level the client already has[2].

Step	Messages
1	if the client's cached modtime matches: the board changes since then, and nothing else[2]
2	otherwise: RAWDATA + BOARDPACKET, then LEVELNAME (first level only), LEVELLINK × n, LEVELSIGN × n, LEVELMODTIME[2]
3	then board changes, LEVELCHEST × n, HORSEADD × n, BADDYPROPS × n, ISLEADER (if leader), NEWWORLDTIME, NPCPROPS × n, and OTHERPLPROPS for each player in the level[2]

2.5.4 Players and the local player

These messages describe the players: the local player and everyone else on the level. Each row gives the layout and the client's action. Note the change in the width of the player id after 1.35[2].

Id	Name	Layout	Client action
8	PLO_OTHERPLPROPS	s id + PLPROP stream (→ §2.6); at least 2 bytes; original 1.21–1.35 read a one-byte c id instead	an unknown id creates the player[2,71]
9	PLO_PLAYERPROPS	PLPROP stream for the local player, no id (→ §2.6)	applies the props[2]
14	PLO_PLAYERWARP	c x×2, c y×2, rest = level	the warp stays pending until the level data arrives[2]
15	PLO_WARPFAILED	rest = level	2.05 ignores the text and shows a short failed-warp state[2]
16	PLO_DISCMESSAGE	rest = text	stored as the disconnect reason; the server then closes the connection[2]
25	PLO_SIGNATURE	c v	1: the client quits; 0: it quits unless the connected host is the loopback address; other values ignored[119]
28	PLO_FLAGSET	rest = name or name=value	adds the flag only if it is not already set[2]
31	PLO_FLAGDEL	rest = flag name	removes the flag[2]
38	PLO_PUSHAWAY	c dx, c dy, each v/16 − 4 (±2 tiles in 1/16 steps)	ignored unless the player's level is the active level[2,3]
40	PLO_HURTPLAYER	s attacker, c dx, c dy, c power, optional i3 npc	see → §2.5.5[2]
49	PLO_PLAYERWARP2	c x×2, c y×2, c z (unused), c gmapX, c gmapY, rest = level	gmapX and gmapY apply only when the level name ends in .gmap: they are clamped to the grid and the name is replaced by that cell's level[119]
154	PLO_FREEZEPLAYER2	empty	freezes the local player for 600 ticks unless unfrozen; skipped while some dialogs are open[2,119]
155	PLO_UNFREEZEPLAYER	empty	unfreezes the local player[2]
168	PLO_UNKNOWN168	payload ignored	resets the local player; sent by the login server[3,119]
170	PLO_GHOSTMODE	c flag	2.17 to 2.31: ghost mode on when the flag is above 0; 5.00r7: stored as the connection mode, and a change resets a player field[119,37]

Id	Name	Layout	Client action
173	PLO_GHOSTTEXT	rest = text	stored as the ghost-mode text[119]
174	PLO_GHOSTICON	c flag	shows the ghost icon when the flag is above 0[119]
181	PLO_UNKNOWN181	s player id, c toggle; at least 3 bytes	sets a boolean on that player[119]
55, 56	PLO_ADDPLAYER, PLO_DELPLAYER	unmeasured	ignored by 5.07 and later[3]

2.5.5 Combat and level objects

These are the server's copies of the client actions in → §2.4.5; each carries the sender's `s player id` unless the row says otherwise.

Id	Name	Layout	Client action
11	PLO_BOMBADD	s player, c x×2, c y×2, c power, c fuse	power = v & 7; the value 8 (bit 3) marks the bomb as that player's[2]
12	PLO_BOMBDEL	c x×2, c y×2; no player id	removes the bomb at exactly that point[2]
19	PLO_ARROWADD	s player + the 5 bytes of PLI_ARROWADD	needs at least 5 bytes; the last byte is decoded as b − 33[2]
20	PLO_FIRESPY	s player, c (a × 8 + b)	only a is used[2]
21	PLO_THROWCARRIED	s player	dropped when the player is unknown[2]
27	PLO_BADDYHURT	c id, c (dx + 4) × 16, c (dy + 4) × 16, c power	applied only by the level leader[2]
36	PLO_EXPLOSION	s player, c radius, c x×2, c y×2, c power	power & 7 clamped to 1–3; 8 (bit 3) = owned[2]
40	PLO_HURTPLAYER	s attacker, c dx, c dy, c power, optional i3 npc	see the rules below[2]
46	PLO_HITOBJECTS	s player, c power, c x×2, c y×2, optional i3 npc	see the rules below[2]
48	PLO_TRIGGERACTION	s player, i3 npc, c x×2, c y×2, rest = action or action,params; more than 5 bytes	fires action on the local player's scripts when the source is within 10 tiles; split at the first comma. For the player's own weapons 2.05r2 also requires the target strictly within 2 tiles of the player's (1.5, 2) anchor on each axis and a source other than the local player; accepted events are queued and duplicates of a pending name dropped[2,117]
175	PLO_SHOOT	s player id, rest = shot data (unmeasured)	only when the current level is the active level[119]
191	PLO_SHOOT2	4.20r8: x, y, z, three angle and speed values, a gani name with optional parameters, then a script string; 5.00r7: s id + data routed to an object	a projectile[34,37]

The table above names the combat messages. The rules below give how 2.05 decodes their bytes and when a hit actually lands[2].

Rule	Builds
PLO_HURTPLAYER: dx, dy decode as (v − 0x60) / 16 (knockback −4 to 4) and power as (v − 32) / 2 hearts, capped at 1.0	2.05[2]
With 6 or more bytes after the player id, the hit came from an NPC (i3), and distance is measured from that NPC	2.05[2]
A hit applies only when the attacker (player or NPC) is within 10 tiles and the local player can be hurt at that moment	2.05[2]
PLO_HITOBJECTS: power is (v − 32) / 2, capped at 1.0; it hurts the local player only when the player's centre is within 2 tiles of (x, y) on each axis	2.05[2]
40 and 46 share one handler, with one flag that picks the layout; the same handler takes UDP id 21 (→ §2.9)	2.05[2]
The trigger-action coordinates are half-tiles, not whole tiles as in the server source	2.05[2]

How damage is applied after a hit is in → §4.3.

2.5.6 NPCs, weapons and scripts

These messages move, script and remove NPCs and hand out weapons. Rows marked unmeasured are ids the server defines but no surviving client has been read for[3].

Id	Name	Layout	Client action
24	PLO_NPCMOVED	unmeasured	unmeasured[3]
26	PLO_NPCACTION	i3 npc id, rest = script text to the end of the message (no length prefix, § line breaks)	handled by 1.28 beta 2 to 2.03; replaces the script of a known server-wide NPC (id ≥ 1000; level NPCs and unknown ids are ignored), clears its compiled script, then fires playerenters if the NPC is in the active level and the player is there; the SCRIPT prop cannot do this because it is ignored once an NPC has a script[121,115]
29	PLO_NPCDEL	unmeasured	unmeasured[3]
33	PLO_NPCWEAPONADD	str name + NPCPROP stream (normally c 0 IMAGE + str image, c 1 SCRIPT + s len + GS1 text)	a name the player already has is ignored; the weapon gets the playerenters event; a script line break is §, byte A7 from a latin-1 server and C2 A7 from a UTF-8 server[2,118]
34	PLO_NPCWEAPONDEL	rest = weapon name	removes the weapon[2]
43	PLO_DEFAULTWEAPON	c item index (0–24: a built-in weapon such as bow, bomb, sword)	adds the built-in weapon[2]
44	PLO_HASNPCSERVER	empty	the client stops applying NPC props itself[3,119]
131	PLO_NPCBYTECODE	i3 code, bytecode	script and class registration: 3.153 registers classes (joinedclasses) and public. variables from it, with 132 and 133 (→ §8)[3,7]
133	PLO_UNKNOWN133	rest = text	maybe registers a class script[3]
134	PLO_GANISCRIPT	gani text	3.153 parses the gani text in place[3,7]
140	PLO_NPCWEAPONSCRIPT	s info_length, script	class and weapon script delivery (→ §8)[3,37]
165	PLO_MOVE	i3 npc id (> 0), then complete 7-byte records: c x×2, c y×2, c (dx+50)×2, c (dy+50)×2, s duration×20, c options	passed to the NPC movement routine; an unknown NPC is not created and an incomplete trailing record is ignored[119,117]
166	PLO_SHOWIMGNPC	i3 npc id, rest = props	applies the props to that NPC in the active level[119]
189	PLO_MOVE2	i3 npc id, rest	as 165, with an extra flag set[119]
193	PLO_UNKNOWN193	i5 CRC32, rest = weapon name	raises the script level to at least 3; loads weblevels/weapons/<name>.code if its zlib CRC32 matches, else asks for the script with PLI_UPDATESCRIPT (158); 4.04r2 to 5.12[102,34,37]
194	PLO_CLEARWEAPONS	empty; precedes the weapon list	4.20r8 clears its weapon and class script lists; 5.00r7 clears every loaded level and the level caches[3,34,37]
195	PLO_LOADGANI	str gani, then the gani text	loaded as a server-sent gani script[3,34]
196	(no gs2lib name)	as 193	4.20r8 reads it and does nothing else[34]
197	PLO_LOADSCRIPT	a list type,name,saveToDisk,desKey,crc32; type weapon, npc, class or gani; the key is two i5	script delivery (→ §8)[3,37]

One rule separates the old script path from the new one[2].

Rule	Builds
For v4 and later clients the server replaces the script prop with NPCPROP_CLASS and sends GS2 bytecode; 2.05 never sees that path	server[2]

2.5.7 Files and update packages

The download sequence, sizes and failure handling are in → §2.7.

Id	Name	Layout	Client action
30	PLO_FILESENDFAILED	unmeasured	unmeasured[3]

Id	Name	Layout	Client action
45	PLO_FILEUPTODATE	unmeasured	unmeasured[3]
68	PLO_LARGEFILESTART	rest = file name	stores the name; 2.31 also clears the stored size[119]
69	PLO_LARGEFILEEND	rest = file name	if it equals the stored name, the accumulated stream is processed as one received file[119]
84	PLO_LARGEFILESIZE	i5 size	stored[119]
102	PLO_FILE	behind 100; exact 2.05r2 counted layout → §2.7.9	a received file; persistence limits are build/fixture-specific[1,81]
105	PLO_UPDATEPACKAGESIZE	package name and an i5 size	sets a size or modification time on a cached object[34,37]
106	PLO_UPDATEPACKAGEDONE	unmeasured	fires onUpdatePackageDownloaded, and onPackagesDownloadComplete after the last one; 4.20r8 restarts itself when its own executable was in a package[34,37]
109	PLO_UNKNOWN109	i5 time, then a c-length string A (local name) and a c-length string B (URL)	fetches B over HTTP and stores it as A with that mod time; a file redirect; 5.24r4 to 6.03r7[102,39]
111	PLO_SETNETCOOKIE	rest, no fields	written byte for byte to creationtime.dat in the client's folder, replacing it; nothing is sent back[39]
187	PLO_UPDATEPACKAGEISUPDATED	unmeasured	a package-update message[3]
198	PLO_SERVEROPTIONS	i5 value	stored on the connection; nothing else in 6.03 reads it[39]

2.5.8 Messages, windows and modes

Chat display and the private-message rules are in → §2.8.

Id	Name	Layout	Client action
13	PLO_TOALL	s player id, str message	shown as a toall message[101]
32	PLO_SHOWIMGPLAYER	s sender id + the sender's id 24 payload, relayed unchanged	no visible reaction to any tested payload (→ §2.4.9)[3,66]
35	PLO_RC_ADMINMESSAGE	rest = Admin <account>: + 0xA7 + text	shown as an admin message[101]
37	PLO_PRIVATEMESSAGE	s from id, rest = "type","message" as a quoted comma list; type is a label such as "Server Message:" or empty	shown as a PM[101]
41	PLO_STARTMESSAGE	rest = RTF or HTML text	trimmed and stored as servermessage.rtf or servermessage.html; if it differs from the stored copy it is rewritten and shown in the start-message viewer[119]
47	PLO_STAFFGUILDS	comma text, "guild", repeated	stored as the staff-guild list; sent at login[101,119]
75	PLO_PROFILE	repeated c n + n bytes (trimmed); field 0 = account	delivered to the open profile window of that player, or the best match in the player list; 2.05 needs at least 10 fields (→ §2.8)[119,101]
82	PLO_SERVERTEXT	unmeasured	the reply to PLI_REQUESTTEXT and PLI_SENDTEXT; 3.153 raises it to script as receivetext; 5.00r7 also feeds the buddy list with it[3,7,37]
153	PLO_SAY2	rest = text; 0xA7 = newline	shown as an NPC sign message; signs use it too[2,119]
169	PLO_UPGRADEURL	rest = URL	stored; with a disconnect, the reconnect button becomes "Upgrade" and opens it[3]
171	PLO_BIGMAP	comma text txt,png,x,y; at least 4 fields are used; x and y as floats	the big map[119]
172	PLO_MINIMAP	as 171	the minimap[119]
176	PLO_KINGDOMSMODE	empty	switches the client to Kingdoms mode[119]
177	PLO_NEWWORLDMODE	empty	switches the client to New World mode[119]
178	PLO_SERVERWARP	comma text, at least 4 fields: 0 unused, 1 server name, 2 host, 3 port	see the rules below[119,65]

Id	Name	Layout	Client action
179	PLO_RPGWINDOW	rest = text	creates the RPG window on first use and gives it the text[119]
180	PLO_STATUSLIST	rest = text	passed to the status-list window if it exists[119]
182	PLO_LISTPROCESSES	empty	the client answers with PLI_PROCESSLIST (44); 4.20r8 lists the titles of all open windows[119,34]
183	PLO_HASPROCESSRUNNING	rest = names	the client answers with PLI_PROCESSRUNNING (45): 1 if the string occurs in its own list, else 0[119]
184	PLO_TAKESCREENSHOT	rest = file name	accepted only with a .png extension; saves a screenshot under the screenshots folder; sends nothing back[119]
190	PLO_SERVERLISTCONNECTED	unmeasured	fires onServerListerConnect: the Login server has accepted the session (→ §2.3)[34,37]
108	PLO_UNKNOWN108	c sub, s value	ping: subtype 0 makes the client answer with id 162 echoing the value (→ App. E#5); subtype 1 stores the value on the connection; 4.20r6, 4.20r8, 6.01r1, 6.03r7, taken before the main switch; not 5.x[102,34]
104	PLO_UNKNOWN104	str file, c a, i5 lo, i5 hi; lo and hi joined little-endian	stores an 8-byte DES key for that lower-cased file name; a .gpake package or .enc file decrypts with it once 45, 69 or 102 arrives for the file; 3.153 to 6.03r7[102,122]

Id 178 moves a client to another server. The rules below come from lab runs of 2.31: which fields it uses, what happens when they are wrong, and how the switch is timed[119].

Rule	Builds
On 178 the client closes the old connection and connects to the host in field 2 and the port in field 3 with a fresh login (new key, same stored credentials), with no listserver step	2.31 (lab)[119,65,99]
Field 1 of 178 is not needed: a bogus name still connects; a decoy host in field 0 is never contacted	2.31 (lab)[65]
With host and port in other positions the client makes no connection, closes the old one and shows "The connection couldn't be established!"; with three fields it does nothing	2.31 (lab)[65]
The old connection closes 2 to 3 ms before the new connection's SYN	2.31 (lab)[65]
178 is split by the client's comma-text reader: white space (bytes 1 to 0x20) before an item is skipped; a quoted item ends at its closing quote, two quotes in a row are one quote, and an unclosed quote runs to the end; an unquoted item ends at a comma or a byte of 0x20 or less; a comma at the very end adds no item, an empty item between commas is kept; a NUL ends the text	2.17.1, 2.22, 2.31[123]
So a server name with a space must be quoted: x,Main Server,h,1 reads as five items and connects to the host Server	2.17.1, 2.22, 2.31[124]
Item 1 becomes the current server's name, without a two-character type prefix when its second character is a space; 2.31 also puts it in the window caption, alone if it contains "graal" in any case, else after Graal: ; the name is not used to connect	2.17.1 to 2.31; caption 2.31[124]
A real service's 178 text, item 0 in particular, has never been captured	all[108]
182 and 183 are a process scan used as an anti-cheat check (→ §4.10)	5.00r7[37]

2.5.9 Staff and NPC-control messages

The RC ids (50 to 54, 57 to 59, 61 to 67, 70 to 74, 76, 77, 103) and the NC ids (78, 80, 157 to 164, 167, 188, 192) are documented with the staff tools in → §7. Game clients handle a few of them:

Id	Name	Builds	Use in a game client
67	PLO_RC_FILEBROWSER_MESSAGE	3.153, 4.x, 5.00r7, 6.x	raised to script as a folder-log event[7]
74	PLO_RC_CHAT	3.153, 4.x, 5.00r7, 6.x	raised to script as an RC-chat event[7]
79	PLO_NPCSERVERADDR	none in the notes	two bytes (0, 2), then host,port text; 5.x ignores it[3]

2.5.10 Other ids

Some ids are defined by the server but handled by no client the notes have read. The table lists them so that their silence is on record, not guessed at[102].

Id	Name	Layout	Client action
60	PLO_UNKNOWN60	unmeasured	no game client (1.21 to 6.03r7) or staff client (1999 RCs to RC3, NPC Control) handles it[102]
81	PLO_UNKNOWN81	unmeasured	no game or staff client handles it; not an NC id, against the gserver's comment[102]
83	PLO_UNKNOWN83	unmeasured	no game or staff client handles it[102]
110	PLO_UNKNOWN110	unmeasured	no game or staff client handles it[102]
124	PLO_UNKNOWN124	unmeasured; RC3 reads nothing from the body	RC3 only: prints `received player flags`; no game client and no older RC handles it[102]
132	PLO_UNKNOWN132	unmeasured	5.07 only; 6.037 ignores it; 3.153 uses it for script and class registration[3,7]
252	PLO_SET_ENC_KEY	sets the AES or RC4 key and IV; above 223, so not sendable as one GCHAR (→ §2.10)	unmeasured[3]
253	PLO_BUNDLE	several messages in one (→ §2.10)	unmeasured[3]

2.5.11 Unpacked Windows 6.1.1.3: script-installed handlers

Receive and send registration

Windows 6.1.1.3 does not compile its receive switch. Its scripts install the handlers when they run. The rules below say how that table is filled for both directions[125].

Rule	Builds
Wire id 104 has no compiled receive case. It runs only when a script pair names it, and that pair is not in the image	6.1.1.3[125]
`setindatahandlers` installs (wire id, handler) pairs into 256 slots. 255 clears, 254 is the script sentinel, 253 and 252 save the wire id, and 0 to 94 select a native function. Index 90 is empty	6.1.1.3[125]
`setoutdatahandlers` fills 256 outgoing ids with all-ones, then copies the script list, at most 256 entries	6.1.1.3[125]

The native handler slots

Unpacked 6.1.1.3 does not compile a wire-id to handler map. A script list of numbers is converted to byte pairs and installed; a second script method zeros all 256 wire-id slots.[126]

Indexes 0 to 94 select a native function, except index 90, which is empty. A handler byte of 95 or more is ignored, apart from the four special pair bytes 252 to 255.[126]

No default pair list is compiled in. The installer writes nothing when the script list has fewer than two numbers.[126]

Unpacked 6.0.3.7 has no 95-slot native table. Wire id 104 has no compiled receive case in 6.1.1.3; the only case 104 is a GS2 opcode.[126]

2.5.12 Linux receive switches and compiled-script delivery

Linux 4.04r2

Each row below is one finding about the Linux 4.04r2 receive switch, with its exact scope and what it does not prove. Read the Limit column before using a row as a count of what the client accepts[127].

Observation	Exact scope	Limit
97 explicit receive cases	Static switch mapping with direct handler edges	Not the complete handled-message count: transport pre-dispatch remains to be included[127]
Missing switch case	No case in this particular dispatch map	Does not prove rejection of that message[127]
Message 131	Requires a three-byte identifier; a positive decoded identifier selects an object lookup, the prefix is stripped, and remaining bytes reach the script setter, context loader and compiled-segment reader	Static path only; complete identifier encoding, lookup roles and indirect initialization callback not fully established; no network fixture delivered[127]
Other script delivery and cache validation	Messages 140/197 and other weapon/class paths remain under research	The map and message-131 trace do not settle all roles, field layouts or cross-platform compatibility (→ App. E#202)[127]

Linux 4.20 and 6.x

The examined splitters pass a zero-extended tag minus 32 to receive entry. After two special arguments, the receiver selects immediate dispatch or mutex/queue machinery; immediate tables use the unmodified argument. The queue branch is not a second established protocol switch. In both 4.20 revisions, 131/193 have distinct paths, 140 takes fallback and 197 is outside the guard. In all three examined 6.x builds, 131/140/197 have handlers and 193 takes fallback. This is exact-path coverage, not universal ID retirement.[128]

In 4.20r6, case 131 checks length before/after an unreviewed helper, derives a positive three-getter identifier, removes three bytes, chooses lookup sources around 1000 and forwards the remainder through a context/program loader to compiled-segment reading. A nonempty remainder also reaches an indirect callback. Gameplay types, callback name/signature, canonical seven-bit validation, helper decryption/decompression, complete field grammars and cache behavior remain unproved. Offline review observes no wire/session/script completion; E#202 remains partial.[128]

2.6 Props

Props are the typed fields of players, NPCs and baddies. The messages that carry them are in → §2.4.4 and → §2.5; field types are in → §0.6. Every prop id is indexed in → App. B.

> Status: draft.

2.6.1 Prop streams

Players, NPCs and baddies are all described by prop streams. The rules below say how a stream is laid out, where it appears and how a parser stops. The last point matters most: one unknown id loses the rest of the message[71].

Rule	Builds
A prop stream is `(c id, value)*` running to the end of the message, with no length per prop	all binary generations[71]
Streams appear in PLI_PLAYERPROPS (2), PLO_PLAYERPROPS (9, the local player, no prefix), PLO_OTHERPLPROPS (8, `s player id` first), PLO_NPCPROPS (3, `i3 npc id` first), PLO_BADDYPROPS (2, `c baddy id` first) and, for NPC props, PLO_NPCWEAPONADD (33)	server; 2.05[71,71]
A reader must know the encoding of every id up to the highest it expects, since nothing marks where a prop ends	all[71]
Each client parser is a loop over `(id, value)` with a switch on the id; an id the switch does not know ends the loop, the rest of the message is ignored, and props already applied stay applied	2.05, 2.22, 2.31[71]
An unknown id therefore cannot crash these clients, but one unknown prop placed early hides every prop after it	2.05, 2.22, 2.31[71]
The other-player stream checks the remaining length before multi-byte props and clamps string lengths; the self stream does not length-check the fixed-size props 3, 13, 14 and 24, so a truncated one reads a few bytes of garbage before the loop ends	2.05[71]
The self stream needs at least 2 bytes left to start a prop; a lone trailing id byte is dropped	2.00, 2.05[129]
The other-player prefix is two 7-bit bytes, `(b0 − 0x20) × 0x80 + (b1 − 0x20)`; an unknown id creates the player record	2.00 to 2.31[130]
The NPC prefix is three 7-bit bytes; the handler needs more than 2 bytes; ids below 1000 are level NPCs, 1000 and up server-wide	2.00 to 2.31[131]

2.6.2 PLPROP (player)

Encodings use the short names of → §0.6; `c(v + k)` means the value plus *k*, sent as a GCHAR.

Id	Name	Encoding	Notes
0	NICKNAME	str	—[71]
1	MAXPOWER	c	heart containers[71]
2	CURPOWER	c	hearts × 2[71]
3	RUPEESCOUNT	i3	gralats[71]
4	ARROWSCOUNT	c	—[71]
5	BOMBSCOUNT	c	—[71]
6	GLOVEPOWER	c	—[71]
7	BOMBPOWER	c	—[71]
8	SWORDPOWER	c(power + 30) + str(image)	—[71]
9	SHIELDPOWER	c(power + 10) + str(image)	—[71]
10	GANI / BOWGIF	str (≥ 2.1) · pre-2.1: c(10 + len) + image, or c(bowpower) if no image	**2.05 uses the pre-2.1 form**[71]
11	HEADGIF	c(len + 100) + image	—[71]
12	CURCHAT	str	chat bubble text[71]
13	COLORS	5 × c	colour indexes in the order skin, coat, sleeves, shoes, belt (→ §2.6.3)[71]
14	ID	s	—[71]

Id	Name	Encoding	Notes
15	X	c(x × 2)	half-tile[71]
16	Y	c(y × 2)	—[71]
17	SPRITE	c	`% 4` = direction[71]
18	STATUS	c	bit flags, see below[71]
19	CARRYSPRITE	c	—[71]
20	CURLEVEL	str	gmap name when on a gmap[71]
21	HORSEGIF	str	—[71]
22	HORSEBUSHES	c	—[71]
23	EFFECTCOLORS	c	server always sends 0[71]
24	CARRYNPC	i3	npc id[71]
25	APCOUNTER	s(+1)	—[71]
26	MAGICPOINTS	c	—[71]
27	KILLSCOUNT	i3	—[71]
28	DEATHSCOUNT	i3	—[71]
29	ONLINESECS	i3	—[71]
30	IPADDR	i5	—[71]
31	UDPPORT	i3	—[71]
32	ALIGNMENT	c	AP 0–100[71]
33	ADDITFLAGS	c	—[71]
34	ACCOUNTNAME	str	—[71]
35	BODYIMG	str	—[71]
36	RATING	i3	`(rating & 0xFFF) << 9 | (deviation & 0x1FF)`[71]
37–41	GATTRIB1–5	str (≤ 223)	—[71]
42	ATTACHNPC	c(type, 0 = npc) + i3(id)	—[71]
43 / 44	GMAPLEVELX / Y	c	—[71]
45	Z	c(z + 0.5 + 50)	—[71]
46–49	GATTRIB6–9	str	—[71]
50	JOINLEAVELVL	c	1 join, 0 leave[71]
51	PCONNECTED	—	no payload; marks disconnect[71]
52	PLANGUAGE	str	—[71]
53	PSTATUSMSG	c	index into the server's status list[71]
54–74	GATTRIB10–30	str	—[71]
75	OSTYPE	str	2.19+ (`wind`)[71]
76	TEXTCODEPAGE	i3	2.19+[71]
77	UNKNOWN77	?	no client checked parses it (2.05, 2.22, 2.31); sending it aborts the rest of the stream[71]
78–80	X2 / Y2 / Z2	s: `abs(v × 16) << 1`, low bit = sign	pixel precision, newer clients only[71]
81	UNKNOWN81	c	player-list tab flag (unconfirmed). Above the highest id that 2.31 parses (80), so 2.31 and older stop reading at it[71]
82	COMMUNITYNAME	str	—[71]

2.6.2.1 STATUS bits (prop 18)

Prop 18 packs several on-off states into one byte. Each row is one bit and what it means when set[71].

Bit	Meaning
0x01	paused[71]
0x02	hidden[71]
0x04	male[71]
0x08	dead[71]
0x10	allow weapons[71]
0x20	hide sword[71]
0x40	has spin attack[71]

2.6.2.2 ADDITFLAGS bits (prop 33)

Prop 33 is a second bit field, for settings that limit what the player receives. Each row is one bit[71].

Bit	Meaning
0x01	no mass message[71]
0x02	only show one level[71]
0x04	no toall[71]
0x08	voice disabled[71]

2.6.2.3 Encodings as 2.05 reads them

Some props have encodings the server source does not spell out. The table gives how 2.05 actually reads them, prop by prop. Most of them switch between a small default number and a custom image name[71].

Prop	Rule
8 SWORDPOWER	0–4 = the default sword<n>.gif; otherwise power + 30, then str image[71]
9 SHIELDPOWER	0–3 = the defaults; otherwise power + 10, then str image[71]
10	the bow image or weapon (pre-2.1 BOWGIF), not GANI[71]
11 HEADGIF	c n below 100 → head<n>.gif; otherwise c (len + 100) + name, at most 100 bytes[71]
15, 16 X, Y	one GCHAR each, half-tiles[71]
17 SPRITE	direction + 4 × animation index; 0x80 in one special state[71]
19 CARRYSPRITE	−1 none, 0, 200, 201, or tile-based values[71]
30, 31 IPADDR, UDPPORT	feed the UDP peer channel (→ §2.9)[71]
36 RATING	rating = b0 × 32 + ((b1 & 0x7F) >> 2), deviation = (b1 & 3) × 128 + b2[71]
51 PCONNECTED	no payload, but 2.05 skips one byte after it; harmless because the player is removed at once, but a server should put no props behind it[71]
7, 23	sent as constants: bomb power * (10) and effect colours 0[71]

2.6.3 Body colours and attributes

Prop 13 colours a player's body by recolouring a palette image. The rules below say which palette entries take which colour, and what a new player gets[132].

Rule	Builds
A body picture is a palette image; before drawing, palette entry 5 takes the skin colour, 6 the coat, 4 the sleeves, 3 the shoes and 1 the belt; entry 0 always takes colour 18 (black, the outline)	2.05[132]
The five colours of prop 13 are in the order skin, coat, sleeves, shoes, belt	2.05[132]
The stock bodies follow that palette layout, and entry 2 is the clear colour	2.22 files[132]
A new player is orange, darkred, black, darkred, black (2, 5, 18, 5, 18), with body.png and head0.gif; GServer-v2 gives a new account 2, 0, 10, 4, 18 instead	2.05; server[132]
A GANI sprite whose picture is ATTR1 to ATTR5 draws the player's attribute 1 to 5, which are props 37 to 41; 2.05 reads them for other players too	2.05[132]
A player with no attribute of their own draws the GANI's DEFAULTATTR1 to DEFAULTATTR5; the stock idle.gani names hat0.png as attribute 1	2.22 files[132]

Rule	Builds
GANI attribute props are strings that GANI PARAM/ATTR placeholders resolve against; 2.05 sends and receives GATTRIB1–5 but its expression table has no `gattrib` entry	2.05[2]
On the wire, animation is only a prop: `PLPROP_GANI` (10) and `NPCPROP_GANI` (12) carry the GANI name, and the client fetches the file with `PLI_WANTFILE`	server[2]

The colours in prop 13 are indexes into a fixed table of 20 named colours. The table gives each index with its RGB value as 2.05 holds it[132].

Index	Colour	RGB
0	white	255, 255, 255[132]
1	yellow	255, 255, 0[132]
2	orange	255, 173, 107[132]
3	pink	255, 132, 132[132]
4	red	255, 0, 0[132]
5	darkred	206, 24, 41[132]
6	lightgreen	132, 255, 132[132]
7	green	0, 255, 0[132]
8	darkgreen	0, 198, 0[132]
9	lightblue	132, 132, 255[132]
10	blue	0, 0, 255[132]
11	darkblue	0, 0, 198[132]
12	brown	132, 0, 0[132]
13	cynober	0, 255, 255[132]
14	purple	255, 0, 255[132]
15	darkpurple	132, 0, 132[132]
16	lightgray	206, 206, 206[132]
17	gray	132, 132, 132[132]
18	black	0, 0, 8[132]
19	transparent	0, 0, 0[132]

The looks commands that change these props are GS1 behaviour (→ §8).

2.6.4 NPCPROP

NPCs have their own prop list. The table gives every NPCPROP id with its name and how its value is encoded. Ids with no measured encoding are marked so[71].

Id	Name	Encoding
0	IMAGE	str[71]
1	SCRIPT	s(len ≤ 0x3FFF) + text[71]
2	X	c(px / 8) = half-tiles[71]
3	Y	c(px / 8)[71]
4	POWER	c[71]
5	RUPEES	i3[71]
6	ARROWS	c[71]
7	BOMBS	c[71]
8	GLOVEPOWER	c[71]
9	BOMBPOWER	c[71]
10	SWORDIMAGE	c(power + 30) + str, or c(0)[71]
11	SHIELDIMAGE	c(power + 10) + str, or c(0)[71]
12	GANI / BOWGIF	str[71]

Id	Name	Encoding
13	VISFLAGS	c[71]
14	BLOCKFLAGS	c[71]
15	MESSAGE	str[71]
16	HURTDXDY	c(dx × 32 + 32), c(dy × 32 + 32)[71]
17	ID	i3[71]
18	SPRITE	c (pre-2.x clients get % 4)[71]
19	COLORS	5 × c[71]
20	NICKNAME	str[71]
21	HORSEIMAGE	str[71]
22	HEADIMAGE	c(len + 100) + image[71]
23–32	SAVE0–9	c each[71]
33	ALIGNMENT	c[71]
34	IMAGEPART	`s x, s y, c w, c h` (x, y clamped to 0–16000, w, h to 0–220)[71]
35	BODYIMAGE	str[71]
36–40	GATTRIB1–5	str[71]
41 / 42	GMAPLEVELX / Y	c[71]
43	Z	c(z + 50), z −50..170 (2.22, 2.31)[71]
44–47	GATTRIB6–9	str[71]
48	UNKNOWN48	not parsed by 2.05, 2.22 or 2.31; aborts the stream[71]
49–52	SCRIPTER, NAME, TYPE, CURLEVEL	str[71]
53–73	GATTRIB10–30	str[71]
74	CLASS	NPC-server class[71]
75 / 76	X2 / Y2	as PLPROP X2/Y2[71]
77	Z2	2 bytes, decoded like X2/Y2; sets the same Z (2.31)[71]

The rules below cover what the NPC table cannot show: what the script prop holds, when it is applied, and the props that the later builds add[71].

Rule	Builds
The SCRIPT prop (1) carries GS1 source text, not bytecode, for pre-4.0 clients	server; 2.05[71]
SCRIPT is applied only while the NPC has no script yet; a second SCRIPT for the same NPC is ignored	2.05[71]
NPCPROP 43 is Z, `c (z + 50)` with z from −50 to 170, from 2.22 on; NPCPROP 77 is Z2 in 2.31	2.22, 2.31[71]

2.6.5 BDPROP (baddy)

Baddies, the built-in enemies, have a short prop list of their own. The table gives each BDPROP with its encoding and any build differences[71].

Id	Name	Encoding and notes
0	ID	—[71]
1 / 2	X / Y	c(tiles × 2); 2.22 clamps to 0–63 and also takes it as the start position[71,133]
3	TYPE	0 graysoldier … 9 dragon (see GS1 baddy names)[71]
4	POWERIMAGE	c(power) + str(image); an empty image means the default for the type[71]
5	MODE	0 walk, 1 look, 2 hunt, 3 hurt, 4 bumped, 5 die, 6 swamp soldier, 7 frog (hare image) jumping, 8 octopus ink, 9 dead. On mode 5 the server switches to 9 after 2 s; 9 removes the baddy on the client[71,133]
6	ANI	c; in 2.22 this is the AI step counter, not an animation frame (44 when looking about starts, 16 when the hunt starts, 0, 3, 2, 1 while chasing)[71,133]
7	DIR	c(a × 4 + b): two directions in one byte, `a = v >> 2` the body, `b = v & 3` the head[71,133]

Id	Name	Encoding and notes
8–10	VERSESIGHT / VERSEHURT / VERSEATTACK	str each[71]

The baddy verses come from the backslash-separated verse line of the level file (→ §3)[71].

The whole 2.22 baddy (types, AI steps, damage, who owns which prop online) is in `client-internals/baddies-222.md`[133].

2.6.6 Ids each parser accepts

An id outside a parser's accepted set ends that message's stream (→ §2.6.1).

Stream	Build	Accepted ids	Stops at
self (9)	2.00	0–35; disputed → App. E#8	36 and up[129]
self (9)	2.04	between the 2.00 and 2.05 sets; not re-read	unmeasured[129]
self (9)	2.05	0–42	43 and up, including 50 and 51[71]
self (9)	2.17.1, 2.22	through 74	50, 51, 53[129,71]
self (9)	2.31	through 80, with X2, Y2, Z2 at 78–80	50, 51, 53, 75–77[71]
other players (8)	2.00	0–35	36 and up[130]
other players (8)	2.05	0, 2, 8–13, 15–21, 24, 30–42, 50, 51	1, 3–7, 14, 22, 23, 25–29, 43–49, 52 and up[71]
other players (8)	2.22	highest 76	1, 3–7, 14, 22, 25–29[71]
other players (8)	2.31	0, 2, 8–13, 15–21, 23, 24, 30–76, 78–80	1, 3–7, 14, 22, 25–29, 77[130]
NPC (3, 33)	2.00	0–36; disputed → App. E#8	37 and up[131]
NPC (3, 33)	2.05	0–40	41 and up[71]
NPC (3)	2.17.1	0–40, 44–47, 53–73	41–43, 48–52[131]
NPC (3)	2.22	highest 73, including 43	48–52[131,71]
NPC (3)	2.31	highest 77	48–52[71]
baddy (2)	2.05	0–10	11 and up[71]

Because an unknown id ends a stream, a server has to know which ids each client parses. The rules below give that set for 2.05 and what was checked in 2.22 and 2.31[71].

Rule	Builds
A server that sends a 2.05 client another player's MAXPOWER (1), RUPEES (3), ID (14), HORSEBUSHES (22) or a counter (25–29) loses every prop after it in that message; send those to the player only, or last	2.05[71]
2.05 knows PLPROP 0–42, 50 and 51 only; it sends 0–24, 26, 31–33, 35 and 37–42, and only receives 25, 27–30, 34, 36, 50 and 51	2.05[71]
2.05 sends props 0, 11, 13, 18, 31 and 33 together at session start, and 42, 15, 16 together on attach or a move	2.05[71]
No client checked parses PLPROP 77 or 81, or NPCPROP 48	2.05, 2.22, 2.31[71]
The 2.22 send-side prop builder also knows 75 and 76	2.22[129]
From 2.3 GServer-v2 puts X2 and Y2 before X and Y in other-player props	server[30]

Prop ranges for the 1.21 to 1.32 builds are in → App. C.6, and parser growth from 1.33 to the 2.1 tests in → App. C.3. Two notes disagree on the 2.00 ranges → App. E#8.

2.6.7 Property ids in unpacked 6.1.1.3 and 6.0.3.7

The outgoing player encoder in both unpacked images walks ids 0 through 83. Twelve of those ids have no case: 3, 25, 27, 28, 29, 30, 34, 36, 50, 51, 77 and 83. The baddy encoder walks 0 through 10, with a case for each. The two builds use the same id lists.[134]

`tclient_setplayerprops`, `tclient_setnpcprops`, `tclient_setbaddyprops` and `tclient_setotherplayerprops` in 6.1.1.3 each store one string. No numbered NPC id table is in either image.[134]

2.6.8 Build-614 player encoder dispatch

The supplied Windows amd64 build-614 unpacked plugin uses 83 selector entries for IDs 0–82 and a 43-entry target table. Default IDs are 3, 25, 27–30, 34, 36, 50, 51 and 77; 72 nondefault dispatch inputs do not prove 72 emitted/accepted properties. Selected ID 8 adds 30, clamps 10–50, substitutes zero for 30 and applies bias 32; ID 9 adds 10, clamps 10–13, substitutes zero for 10 and applies bias 32. For IDs 15/16 the retained observations identify doubling, constants 0.5 and 0.0001, a floor-style helper and byte bias 32, but do not establish exact parenthesization, floating precision, conversion range or integer/byte-narrowing order. No reproducible encoding formula or near-boundary result is supplied. Names/getters, inbound decoding, runtime behavior and original-to-unpacked lineage are not independently established by these static checks.[135]

2.7 File transfer

The request messages are laid out in → §2.4.7 and the replies in → §2.5.7; the raw-data escape is in → §2.1.3. This chapter gives the sequence and the version rules.

> Status: draft.

2.7.1 Requests and replies

A client asks for a file in one of three ways. Each row gives a request, the reply when the server sends the file, the reply when it does not, and the builds that use that request[73].

Request	Reply when the file is sent	Reply otherwise	Builds
23 `PLI_WANTFILE` (name)	100 `RAWDATA` + 102 `FILE`	30 `FILESENDFAILED` (name)	1.21 to 6.x[73]
34 `PLI_UPDATEFILE` (`i5` cached modtime, name)	100 + 102 when the modtime differs	below 2.1: 30; 2.1 and later: 45 `FILEUPTODATE`	1.42 to 6.x[73]
47 `PLI_VERIFYWANTSEND` (`i5 crc32`, name)	100 + 102 when the client's copy is stale	unmeasured	5.00r7, 5.24, 6.x[73,73]

The rules below cover the edge cases of those replies, which differ either side of 2.1, and say that game clients never upload[73].

Rule	Builds
Below 2.1 the server adds `.gif` to a requested name with no extension	server[73]
Below 2.1, 30 also means "already up to date"; 2.05 handles 45, but the server does not send it to 2.05	server; 2.05[73]
Game clients cannot upload; RC upload uses the file-browser ids 89 to 98 and 65 to 67 (→ §7); disputed for 6.x → App. E#9	server[73]

2.7.2 The `PLO_FILE` body

The following historical rows describe server version branches; the exact original 2.05r2 receiver differs → §2.7.9[81].

Client version (by the server's token table)	Layout behind 100
historical server branch below 2.1	`c 102`, `str filename`, then the file bytes; no modtime and no trailing `\n`; the RAWDATA length is 1 + 1 + name + data[73]
2.1 to 2.13	`c 102`, `i5 modtime`, `str filename`, the bytes, then `\n`, in one piece[73]
2.14 and later	as 2.1, split into 32000-byte chunks; a file over 32000 bytes is wrapped in 68 `LARGEFILESTART`, 84 `LARGEFILESIZE` and 69 `LARGEFILEEND`[73]

The rules below give the size limit on a single file, and how a large file arrives in chunks and is put back together[73].

Rule	Builds
A client before 2.14 cannot receive a file over 64000 bytes; the server answers 30 instead	server[73]
A 541,248-byte file reached 5.00r7 as 68, 84, then 102 chunks each with modtime and name, then 69, and was saved byte for byte	5.00r7 (lab, stock gs2lib)[73]
On 69 the client compares the name with the stored one and processes the accumulated stream as one file (→ §2.5.7)	2.17.1 to 2.31[119]

2.7.3 The layout follows the server's token table

Which file layout a client gets depends on how the server ranks its token, not on what the client can parse. The rules explain that ranking and what an unranked token causes[73].

Rule	Builds
Every version branch in the server's file code tests the version id, which comes only from the login token looked up in the server library's table; a token that is allowed but not in the table gets version 0, below every real version	server[73]

Rule	Builds
Such a client gets every pre-2.1 path: PL0_FILE without modtime or \n, 30 for up to date, .gif added to names, the pre-2.1 level sequence with no 156, and no large-file messages	server[73]
A version's rank is its position in the table; the server compares positions	server library[73]
The table holds, in order: G3D22067 (5.00r7), G3D14097 (5.12), G3D26090 (5.30r3, 5.31r4, 6.00r2, 6.00r4), G3D3007A (6.01x), G3D2505C (6.034, 6.035), G3D0311C (6.03r7 Windows), G3D0511C (6.03r7 Linux); all sit above the 2.14 slot	server library[73]
G3D15010 (5.24r3, 5.24r4, 6.00 beta2) and G3D03048 (6.00 beta1) are not in the stock table	server library[73]
The client always reads PL0_FILE as i5 modtime + str filename + data; given the old layout it takes the name's length byte and first 4 characters as the modtime, and the 5th character as the length	5.24[73]
For sword1.png that gives the error "Couldn't open …1.png‰PNG… for writing"; with a table that knows G3D15010 the same server's files were saved normally	5.24r4 (lab)[73]
A server for 5.x and 6.x clients must know every token it allows	server[73]
A client cannot tell the server which layout to use	all[73]

2.7.4 Requests at login

Each build asks for different files when it enters a server. The table gives the first requests seen per build. The late builds always start with the base package[73].

Build	Requests seen
2.05	for each file a level references (for example head25.png, sword1.png, walk.gani): 34 when cached, else 23; the server answers 100 + 102 or 30[73]
2.22	34 for the default images (sword, shield, body, bomb), then 23 (→ §2.4.11)[30]
5.24r4, 5.30r3, 6.03r7	basepackage.gupd first; with an empty cache as 23, twice[73]
5.00r7	basepackage.gupd as 47 when that server's copy is cached and as 23 when not; the cache is kept per server name[73]
5.x	then 34 for idle.gani, sprites.png, body.png, head0.png, shield1.png and the guigames_* images, and 23 for the fonts (arial.ttf, arialbd.ttf) and tiledefs<server>.txt[73]

Update packages and their messages (105, 106, 187) are in → §2.5.7.

2.7.5 The client's request lists

A client does not ask for a file twice. The rules below describe the three lists it keeps, how many requests it allows in flight, and when a name moves from one list to another[136].

Rule	Builds
The client keeps three name lists: wanted (asked with 23), update-asked (asked with 34) and settled (a reply came back)	2.00, 2.05 rev 2, 2.17.1, 2.22, 2.31[136]
No local copy: the name is appended to the wanted list unless already there, and sent while that list holds at most four names	same[136]
A local copy: the name is appended to the update-asked list unless it is there or settled, and sent while that list holds at most four; so each local file is checked at most once per connection	same[136]
The time in 34 is the local file's last-write time in Unix seconds; at login it matched the files' times for the sword, shield and body images	same; 2.22 (lab)[136]
Builds with no id 34 never check a local copy	1.33 to 1.41[136]
Any reply (the file, up to date, send failed, or the end of a large file) removes the name from both lists and adds it to the settled list; if the name was among the first four, the name now fourth goes out; the two lists release independently	same[136]
Four of each kind were on the wire before the first reply	2.22 (lab)[137]
The client never compares times; the server decides	same[137]
A name answered with 30 is not asked for again on that connection: a missing step sound was asked for once however often the player walked	2.22 (lab)[136]

Rule	Builds
Images and ganis: a local file is used and checked, a missing one asked for; sounds: asked for only when missing, never checked; levels: no file transfer (board messages and the level modtime instead, → §3.8); a gmap: checked when present, asked for when missing	2.05 rev 2, 2.22; gmap 2.22 (lab)[136]
In 2.05 rev 2 and 2.22 the request side was read from the matching reply-side code and the lab	2.05 rev 2, 2.22[138]

2.7.6 Saving a download

The rules below say when a download is written to disk, where it goes, and which modification time it keeps[139].

Rule	Builds
A received file is held in memory and written to disk only when `saveimages` is on (default on, → §3.9.3)	2.05 rev 2 to 2.31[139]
An empty or missing stream is never saved: nothing is written or deleted and the loaders are not told	1.39, 2.00, 2.05 rev 2, 2.17.1, 2.22, 2.31[140]
A file the client already has is replaced where it is; a new one is placed by name, compared without case	2.00 and later[139]
From 2.1 the saved file gets the time from the reply, so the next check carries the server's time; before 2.1 it keeps the download time	2.1 and later; earlier[139]

Where a new download is saved, by build:

Builds	Folders
1.33 to 1.41	everything in `levels/webgifs`; 1.39 saves only `.gif`, `.png` and `.mng`, deletes an old copy first and sets no time[140]
1.42 NewMain	the earlier build adds `ganis`, `heads`, `swords`, `shields`; the later adds `sounds` for `.wav` and `bomys`[140]
2.00, 2.01	`sounds` for `.wav`; under `levels`: `bomys` (names starting `bomy` or `b_`), `ganis` (`.gani`), `emoticons` (names starting `emoticon`), `heads`, `bodies`, `swords`, `shields` (names containing those words), else `webgifs`[140]
2.02 to 2.05 rev 2, both 2.1 test builds	as 2.00, plus `hats` (`hat` then a digit as the fourth character)[140]
2.17.1, 2.22, 2.31	as 2.05, plus `maps` for `.gmap` and for `.txt`, `.gif`, `.png`, `.mng` names containing `map`; a 2.22 gmap download was saved as `maps/<name>`[141,139]
2.1 to 2.16	no binary; when `maps` arrived is unknown (→ App. E#10)[140]

2.7.7 Finding a local file

Before asking the server, a client looks for the file on disk. The rules below give how it matches names and in which order it searches, for 2.05 rev 2 and 2.22[142].

Rule	Builds
Names are folded A to Z only, then compared byte for byte; accented letters do not match across case	2.05 rev 2, 2.22[142]
A file of that name in the current directory wins first: the game folder in 2.22, the `levels` folder in 2.05 rev 2 (or a world opened from elsewhere)	2.05 rev 2, 2.22[142]
2.22 indexes the game folder at startup, files before subdirectories in listing order, skipping top-level `pms`, `profiles` and `weblevels` and `.code` files; for a duplicate name the newer last-write time wins, and an equal time keeps the first found	2.22[142]
2.05 rev 2 looks up on demand: the current directory, then a remembered path, then a depth-first search of an opened world's folder and the game folder with the same skips; the first hit wins and is remembered	2.05 rev 2[142]
For two different files of one name, 2.22 draws the newer and 2.05 rev 2 the first found	2.05 rev 2, 2.22[143]

2.7.8 Unpacked Windows 6.1.1.3 packages

The unpacked 6.1.1.3 image was searched for package and download names. The two rules record what was found and what was not[144].

Rule	Builds
The only `.gupd` file name in the image is `basepackage.gupd`. `package_mainfiles.gupd` is not in the listing. `game_console.log` is a path the client opens, and the log is not in the repository	6.1.1.3[144]
`onSendUpdatePackage` and `onSendDownloadFile` are present. A numeric request id for `basepackage.gupd` was not recovered	6.1.1.3[144]

2.7.9 Exact original Windows 2.05r2 receiver

The 2.05 rev 2 file receiver was read exactly. Each row names one property of it and gives the detail. The last rows say how a server was checked against it, and how far that check goes[81].

Rule	Detail
counted layout	FILE opcode, five encoded metadata bytes unconditionally, encoded one-byte filename length L, L name bytes, content; for raw count N, content length is N − 7 − L[81]
LF boundary	collector bypasses normal delimiter subtraction; all N bytes reach FILE copy, including a terminal LF inside N; outside-count bytes are not content[81]
baseline mismatch	tested server commit 1b0a6e omitted prefix before 2.1, counted an added LF and emitted no empty FILE; local fixes correct those paths without changing the receiver specification[81]
synthetic checks	two profiles × five fixtures improved 0/10 to 10/10 filename/content checks; not ten original-client runs[81]
original-client evidence	unchanged 2.05r2 entered local world, requested all fixtures, sent movement properties and saved exact no-LF/one-LF/binary-LF files; two-LF valid-GIF persistence required repeating unchanged server record to already-requested name[81]
residuals	initial two-LF caching and empty persistence unresolved; unsolicited alias not cached; byte-copy does not establish general retry/registration/deduplication[81]
fixture	isolated Wine/display namespaces, loopback only/no routes, unchanged executable and completed cleanup; labeled 2.31 donor resources, not authentic full 2.05 distribution[81]
startup qualification	only copied world greeting emptied to bypass HTML/Wine stall; milestone 6 and equal level objects; default greeting unchanged; native Windows/cause unproved[81]
limits	other clients, large-file continuation and malformed inputs remain unqualified; no server patches published in this increment[81]

2.8 Chat and private messages

The message layouts are in → §2.4.8 (client to server) and → §2.5.8 (server to client). How the client draws chat and nicknames is behaviour, in → §4.

> Status: draft.

2.8.1 Local chat

Local chat rides on a player prop, not on a message of its own. The rules below give how it is sent, how the server reads commands out of it, and the length limit[101].

Rule	Builds
Local chat is not a message: it is the CURCHAT prop (12) inside PLI_PLAYERPROPS; the server forwards it to the level as PLO_OTHERPLPROPS	server; 2.05[101]
Server commands typed in chat (`setnick`, `/sethead` and the like) are parsed by the server from the same prop	server[101]
When the chat-bar text, lower-cased and trimmed, starts with `toall:`, 2.05 sends the rest as PLI_TOALL (6) instead	2.05[101]
A line longer than 256 characters is cut to its first 253 characters plus `...` when sent; the receiver draws whatever arrives	2.05[145]
When the player's own line times out (3 s after the first move), the client sends an empty CURCHAT	2.05[145]
A typed `showname` is not sent to the server	2.05[145]
Chat also goes peer to peer as UDP id 22 (→ §2.9)	2.00 to 2.05, 3.04[75]

2.8.2 Private messages

Private messages are real messages with a list of recipients. The rules below cover addressing, the kinds of mass message that share the format, and who may block or be blocked[101].

Rule	Builds
A PM names its recipients by player id (`s count`, then the ids), so one message can go to several players	server; 2.05[101]
Mass PMs and guild PMs use the same message with different type labels, such as `"Server Message:"` or `"Guild message:"`	server[101]
Players in jail can PM only RCs	server[101]
ADDITFLAGS bit `0x01` blocks mass messages and `0x04` blocks toall (→ §2.6.2.2)	server[71]

2.8.3 Staff messages

Staff reach players through a few messages of their own. The rules below give those messages and point to the staff tools' separate logins[101].

Rule	Builds
An admin message reaches a game client as PLO_RC_ADMINMESSAGE (35), `Admin <account>:` followed by `0xA7` and the text	server; 2.05[101]
The staff-guild list is sent at login as PLO_STAFFGUILDS (47)	server; 2.05[101]
The staff tools run their own logins: RC `GSERV023` to `GSERV025`, NC `NCL11012`, RC2 as type 6 on generation 5 (→ §7)	server[101]

2.8.4 Buddy and list text requests

Buddy lists and the cross-server tabs are text requests that the game server passes on to the listserver. The rules below give the words used each way and what the server keeps, which is nothing[146].

Rule	Builds
Buddy requests go in PLI_SENDTEXT as the tokens GraalEngine, lister, then verifybuddies, addbuddy or deletebuddy, then the account list	2.17+; server[146]
The server forwards those three options to the listserver, never for guest accounts, and stores nothing itself	server[146]
The listserver answers with PLO_SERVERTEXT (82) carrying weapon, lister, buddylist, then a comma list of accounts	listserver[146]
PLI_REQUESTTEXT types pmservers, pmserverplayers <server>, pmunmapserver and pmguilds go to the listserver for the cross-server and guild tabs	server[146]
A real 2.22 client sent no buddy request on add or delete; it only wrote buddyaccounts.txt, so the token layout on the wire is unmeasured (→ §4.10.5.2)	2.22[146]
PLI_MUTEPLAYER (43) is s playerId, c 1/0; GServer-v2 registers no handler, so it counts as an invalid packet, and more than 5 invalid packets disconnect the client; two sends from a 2.22 client did not	2.17+; server[146]

2.8.5 Profiles

A profile is a list of short text fields relayed from the listserver. The rules below say how it is sent and read, and how 2.05 stores it[101].

Rule	Builds
PLI_PROFILESET and PLO_PROFILE are a run of c length + text fields, read until the message ends	2.05[101]
The profile is relayed from the listserver (→ §6)	2.05[101]
2.05 sends PROFILESET from the profile window's Send button, which also works in another online player's window; Close on your own profile only keeps the edits locally	2.05[101]
On PROFILE, 2.05 needs at least 10 fields, drops field 0 and saves fields 1 to 9 to a local per-account profile file	2.05[101]
The in-game stats in the same window (online time, kills, deaths, rating and so on) are not in this message; they most likely come from the player's props	2.05[101]

The table gives what each profile field holds, by position. Field 0 names the owner and is the one a server should check[101].

Field	Content
0	account, the profile's owner[101]
1	real name[101]
2	age[101]
3	sex: - (unknown), male or female[101]
4	country[101]
5	ICQ number[101]
6	e-mail[101]
7	homepage[101]
8	favourite Graal hangout[101]
9	favourite quote: the quote box's lines joined with spaces, trimmed, cut to 220 characters[101]

Each text field is trimmed before sending[101].

2.8.6 Unpacked Windows 6.1.1.3

In 6.1.1.3, chat from other players reaches the scripts as an event. The two rules say how it arrives and why no fixed id can be given for it[147].

Rule	Builds
Another player's line arrives as the script event onToall, raised by tclient_settoallmsg on the TCP incoming path. The UDP callback does not raise it	6.1.1.3[147]
No compiled PLO id is assigned to that event. The wire id is the script pair, which is not in the image	6.1.1.3[147]

2.9 UDP peer-to-peer

A channel between clients that the game server never sees; the server only relays each player's address. The ids handled per build are in → App. C.4; the TCP twins of the level-object messages are in → §2.4.5.

> Status: draft.

2.9.1 Setup and addressing

From 1.28 beta 2, clients can send some traffic straight to each other over UDP, past the server. The rules below say which builds have the channel, how it is set up and how peers find each other's ports[75].

Rule	Builds
No open-source server implements this channel, because the server never sees it	server[75]
The first build that links `bind`, `sendto` and `recvfrom`, and calls the feature "direct data packages", is 1.28 beta 2	1.28 beta 2[74]
Every 2.0x build, NewWorld and NewMain included, imports the same socket calls and has one send and one receive wrapper	2.00 to 2.1 tests[148]
At connect the client binds a UDP socket on `fixedudpport` when `fixedport` is set, otherwise on an ephemeral port; `noudp` turns the channel off	2.05[75]
1.31 adds `noudp=` and `fixedudpport=` to the client config; the catalogue gives 14899 as the fixed-port default	1.31[74]
`noudp` defaults to on	2.22, 2.31[30]
The client sends its own port as PLPROP UDPPORT (31, i3); a peer's address is PLPROP IPADDR (30, i5, a 32-bit address) plus UDPPORT, both arriving in PL0_0THERPLPR0PS	2.05[75]
A datagram is accepted only from the address and port of a known player	2.05[75]
A 241 s full capture of a running 6.113 client has 10643 UDP datagrams. None of them is to a host that answered the Graal TCP hello on 14900 or 14882. Ports 14899, 14900 and 14901 are not UDP ports in that file, and no UDP port is one greater than a TCP port	6.113[50]

2.9.2 Datagram

A UDP datagram is a stripped-down TCP message under a one-byte XOR. The rules below give the layout and how the key is chosen, with a capture that confirms it[75].

Rule	Builds
A datagram is seq (1 byte), then c UDP id and the fields, in the same G-type encodings as TCP; no zlib, no length prefix, no gen-3 insertion	2.05[75]
The whole datagram is XORed with one key byte: `recipientId XOR 1`	2.05, 3.04[75]
In code the key is the low byte of the recipient's id field XOR the same field of the local player object, which reads 1 in practice	2.05[75]
In a three-player capture (ids 2, 3, 4) every datagram to player 2 used key 3, to player 3 key 2 and to player 4 key 5, whoever sent it	2.05[75]
seq counts from 0 per sender–recipient pair and wraps mod 256; a duplicate seq is dropped	2.05, 3.04[75]
1.28 beta 2 sends single, uncompressed, unframed messages straight to another player's address and port	1.28 beta 2[74]
The 2.05 receive wrapper drops a bit test on a length value and moves a field offset compared with 2.00–2.04; the datagram layout does not change	2.00 to 2.05[148]

2.9.3 Routing

A client decides per action whether to use UDP, TCP or both. The rules below give that choice and what stays on TCP regardless[75].

Rule	Builds
A level-local action goes over UDP only when every other player in the level has a UDP port; otherwise it goes over UDP to the peers that have one and over TCP to the server	2.05[75]

Rule	Builds
The TCP copy is also sent when the server has an NPC server (PLO_HASNPCSERVER)	2.05[2]
This rule covers the level-object messages only; player props stay on TCP	2.05[2]
With UDP active, both clients still sent X, Y and sprite props over TCP; which condition picks the TCP copy when the server is in the level is not traced	2.05[75]
One player per level, the level leader picked with PLO_ISLEADER (10), sends the level NPCs' arrows to the others	1.28 beta 2[74]

2.9.4 UDP ids

The ids are their own space, not PLI or PLO numbers; the wire byte is again the id + `0x20`[75,149].

Id	Meaning	Payload	TCP twin
0	movement	six GCHARs: X × 2, Y × 2, SPRITE, STATUS, CARRYSPRITE (−1 none), and a letter that follows the animation	props 15–19[75]
10, 11	bomb add, bomb delete	unmeasured beyond the TCP twins	PLI 4, 5[75]
12	arrow add	the 5 bytes of relay A in 1.28 beta 2	PLI 9[75,74]
13	fire spy	unmeasured	PLI 10[75]
14	explosion	unmeasured	PLI 27[75]
15	showimg	four GCHARs (index, x, y, name length), the image name, then a few trailing bytes; 3.04 sends one byte 9 about every 5.1 s per peer	PLI 24[75,75]
16, 17	horse add, horse delete	unmeasured	PLI 7, 8[75]
18	throw carried	unmeasured	PLI 11[75]
19, 20	item add, item delete	unmeasured	PLI 12, 13[75]
21	remote hit from GS1 `hitplayer`, sent only to the victim	`c (dx + 4) × 16, c (dy + 4) × 16, c halfhearts`, optional attacker-object bytes	PLO 40 shape[75]
22	chat	text, at most 253 bytes	CURCHAT prop[75]
23	level name	raw level name, no length byte; applied as CURLEVEL (20); no 2.05 sender	prop 20[75]
24	entered level	`c x × 2, c y × 2`, raw level name; sent to peers in the 3 × 3 gmap neighbourhood	props 15, 16, 20[75]
25	hit objects	unmeasured	PLI 36[75]
26	trigger action	for example NPC id 0, x × 2, y × 2, `leftmouse`	PLI 38[75]

The id table above lists the UDP messages. The rules below are what captures showed about their timing and fields[75].

Rule	Builds
Movement goes out about every 100 ms while moving, then less often	2.05, 1.28 beta 2[75,74]
The last movement byte was 55, 41 and 51 (`W`, `I`, `S` as characters) for walk, idle and sword, so it looks like the first letter of the gani name	2.05, 3.04[75]
Showimg is re-sent about every 3 s while shown	2.05[75]
21's dx, dy is the direction from the script's `fromx, fromy` to the victim, clamped to −4..4; the receiver decodes `(b − 0x60) / 16` and damage `(b − 0x20) / 2` capped at 1.0; it is sent only when the victim is in the same level	2.05[75]
A newcomer sends one empty id 22 to each player already in the level	1.28 beta 2 to 1.32[74]
Ids 0, 15 and 22 go through a broadcast helper (prepend the id, send to every peer with a UDP port); the other ids through a peer-send wrapper	2.00, 2.05[149]
UDP 25 and 26 first appear in the same builds as their TCP twins, PLI 36 (2.01) and PLI 38 (2.03)	2.01, 2.03[149,148]
The Torque-era 3.04 client kept the channel byte for byte	3.04[75]

Rule	Builds
Whether 2.1 to 2.31 used the channel on the wire is unmeasured; 2.22 also used UDP for voice chat	2.1 to 2.31[75]

2.9.5 Unpacked Windows 6.1.1.3

In 6.1.1.3 the UDP channel survives only as code that scripts may call. The two rules say what is in the image and what the captures show[150].

Rule	Builds
A datagram socket opener is present. The game connect path does not call it. Script can, through the method that reports "Binding of port"	6.1.1.3[150]
The 6.113 captures have no client datagram on the TCP 14900 remotes. That matches the connect path, and it leaves the script opener in the image	6.1.1.3[150]

2.10 The 6.11 server-defined protocol

In 6.11 the client has no fixed protocol: a script fetched at login sets the framing, the cipher and both id tables. The classic generation-5 transport (→ §2.2) stays the default until that script switches it.

> Status: draft. The connector script itself was never captured (→ App. E#3), so the wire layout a 6.11 server uses is not in the notes.

2.10.1 The connector request

Mac 6.11 does not know how to log in on its own. It first fetches a connector from the official host. The rules below describe that request[39].

Rule	Builds
On login the client sets the server name to `Login` and requests `con.png` and `con.gs` over HTTPS and `conf.gs` over plain HTTP from the official connector host (`<login-host>`)	6.11 Mac[39]
Each step is retried up to twice, using a second host whose first label differs; after three failures the client shows a warning	6.11 Mac[39]
The query is `?p=` + a comma-joined list `g=<client id>,p=mac,v=<version>,b=<build date and time>`, DES-ECB encrypted under a compiled-in key (whole 8-byte blocks only), base64-encoded with the client's own alphabet, then URL-encoded	6.11 Mac[39]
The HTTPS connection checks the server against a certificate bundle compiled into the client, and sends `User-Agent: Graal/<version>`	6.11 Mac[39]

A lab run of the unmodified Mac 6.11r1 client in a Lion guest with no internet confirmed the order. It looked up the connector host and then the `con2` host, never the 4.x login or list hosts. It sent a TLS 1.2 ClientHello on 443 with no SNI, only the `signature_algorithms` extension and RSA-only suites including NULL and RC4. When that failed it sent `GET /conf.gs?p=…` over HTTP/1.0 on port 80 with `User-Agent: Graal/6.111`, and with no reply it showed "Couldn't connect to the server". A lab 6.11 client therefore needs a `conf.gs` responder that returns a connector script; it cannot connect to a gserver directly.[39]

2.10.2 The connector script

The connector is a script, and it takes over the connection. The rules below say how it is loaded, what it is given, and what that leaves to the server[39].

Rule	Builds
The reply is loaded as the script object `StartScript_Connector`, and its `onCreated` runs; `scriptip`, `scriptsslcipher`, `scriptsslsubject` and `scriptsslissuer` are set first so the script can check where it came from	6.11 Mac[39]
If no connector script loads, a built-in `StartScript_Fail` is used	6.11 Mac[39]
The connection object gives the script `connecttogameserver`, `setprotocol`, `sendraw`, `senddata(id, data)`, `flush`, `setsslparameters`, `getsslcipher/subject/issuer`, `setencryptionparsekey`, `setencryptionout(type, key, iv)`, `getpassword`, `setloginaccount`, `requestbasefiles`, `setoutdatahandlers`, `clearindatahandlers`, `setindatahandlers`, `defaultinhandler(id, data)` and `tclient_setencryptionin(type, key, iv)`	6.11 Mac[39]
The script builds and sends the login itself, so the token, ids, header layout and cipher of 6.11 live in the server's script, not the client	6.11 Mac[39]

2.10.3 Framing and encryption

The connector can redefine the frame and turn on a cipher. The rules below give the template language for the header and the ciphers on offer, and confirm that 6.01 and 6.03 have none of it[39].

Rule	Builds
`setprotocol(template)` replaces the generation-5 framing with a header described by the template; the header is as long as the template	6.11 Mac[39]
Each template letter marks a field and repeating it sets the width, at most 4 bytes, big-endian: I packet counter (starting at 1), L length, T packet id, E another field	6.11 Mac[39]

Rule	Builds
In that mode the stream can be wrapped in RC4 or AES-CBC (blocks padded with 0xFF), set separately per direction; other names turn it off; TLS is also possible	6.11 Mac[39]
None of this exists in 6.01 or 6.03: no connector, no RC4 or AES, no script-set id tables	6.01r1, 6.03r7[39]
gs2lib names PLI_SET_ENC_KEY and PL0_SET_ENC_KEY (252, setting an AES or RC4 key and IV) and PLI_BUNDLE and PL0_BUNDLE (253, several messages in one); both are above 223, so they cannot be sent as one GCHAR	server library[3]

2.10.4 Id tables

The connector also sets which wire id means what, in both directions. The rules below describe the two tables and the native handlers they point to[39].

Rule	Builds
setoutdatahandlers loads a 256-entry table from internal message number to wire id; 0xFE sends the message to a script event instead of the wire	6.11 Mac[39]
setindatahandlers takes (wire id, handler) pairs; a handler below 0x5F is one of 95 native handlers, 0xFE sends the packet to the script, 0xFF ignores it, and 0xFC/0xFD set two special ids	6.11 Mac[39]
The native handlers are also exposed to script by name, about 90 tclient_* functions, from tclient_setplayerprops and tclient_setlevelboard to tclient_setghostsnear and tclient_setpackageend	6.11 Mac[39]
Slot 54 and slots above 56 have no event; the login and language messages are sent by the script itself; the slot order is not the wire order, so a 6.11 server must send the table	6.11 Mac[39]

The named send slots, each with the event 6.11 raises when its slot holds 0xFE:

Slot	Event	Slot	Event	Slot	Event
0	onSendWantWeaponScript	19	onSendTakeExtra	38	onSendLevelWarp[39]
1	onSendWantClassScript	20	onSendRemoveExtra	39	onSendLevelWarpModtime[39]
2	onSendWantGaniScript	21	onSendOpenChest	40	onSendDownloadFile[39]
3	onSendToAllChat	22	onSendShot	41	onSendBoardModify[39]
4	onSendIsPKer	23	onSendBaddyProperties	42	onSendBoardModify2[39]
5	onSendCarryThrown	24	onSendBaddyHurt	43	onSendPreloadLevel[39]
6	onSendAddHorse	25	onSendWeaponImage	44	onSendPlayerProperties[39]
7	onSendDelHorse	26	onSendRCChat	45	onSendNPCProperties[39]
8	onSendAddBomb	27	onSendRequestText	46	onSendFlag[39]
9	onSendDelBomb	28	onSendSetText	47	onSendUnsetFlag[39]
10	onSendServerWarp	29	onSendFileDeletion	48	onSendProjectile[39]
11	onSendTriggerAction	30	onSendFolderDeletion	49	onSendUpdatePackage[39]
12	onSendFireSpying	31	onSendFileRename	50	onSendPrivateMessage[39]
13	onSendExplosion	32	onSendFilesMove	51	onSendUploadStart[39]
14	onSendWeaponHit	33	onSendHaveWindow	52	onSendSaveFile[39]
15	onSendPlayerHurt	34	onSendEnterLevel	53	onSendUploadEnd[39]
16	onSendDeleteNPC	35	onSendWantImage	55	onSendPingAnswer[39]
17	onSendDeleteWeapon	36	onSendImageUpdate	56	onSendWindowList[39]
18	onSendExtra	37	onSendImageUpdateCRC		[39]

2.10.5 The Windows 6.1.1.3 hello

Unpacked Windows 6.1.1.3 has the same script methods. The port 14900 capture's first bytes are the clear token GNP1905C and then a six-byte header (→ §1.8.7).

Rule	Builds
GNP1905C is not a stored string, scrambled or clear, and graalversion (the number 6.113) is not turned into the digits 1905. How those digits are chosen was not found	6.1.1.3[51]

Rule	Builds
`sendraw` appends the caller's bytes with no packet id and no header, which is the path that can write those eight bytes	6.1.1.3[51]
`setprotocol` writes an I/L/T/E header. L is the header length plus the body length. E 1 is zlib and E 2 is bzip2 at block size 5. The template is not in the image	6.1.1.3[51]
A 6-byte header plus the capture's 273-byte zlib member is 279, and `01 17` sits in the captured header. The cipher after that member was not found	6.1.1.3[51]
Unpacked 6.0.3.7 stores `g3d0311c`, upper-cases it, and sends it as id 5. 6.1.1.3 has no such string	6.0.3.7 and 6.1.1.3[51]

2.10.6 Unpacked Windows 6.1.1.3

The unpacked Windows image still contains the generation-5 LCG: seed `0x04A80B38`, step times `0x08088405` plus 1. It has no compiled-in `G3D` or `GNP` token. Each outgoing sender compares its own slot with `0xFE` and then calls a script event; the Mac slot numbers in the table above are not the order of the names in this image. `onSendNPCProperties` is the one Mac event name this image does not have. `StartScript_Connector`, `StartScript_Fail` and the three `scriptssl*` fields are present. `setprotocol`, `setindatahandlers` and `con.gs` are not plaintext strings.[46,46,46]

3 File formats

> Part index. Each chapter carries its own status line.

- → §3.1 Levels: Z3-V1, GR-V1, GLEVNW01 (`01-levels.md`)
- → §3.2 GRMAP001 and gmaps (`02-gmap.md`)
- → §3.3 GANI animations (`03-gani.md`)
- → §3.4 GOBJSET01 object sets (`04-gobjset.md`)
- → §3.5 Tile types and arrays.dat (`05-tiles-arrays.md`)
- → §3.6 Sign text alphabets (`06-sign-text.md`)
- → §3.7 Save games (graalsav) (`07-saves.md`)
- → §3.8 Web level cache (GWEBL001) (`08-caches.md`)
- → §3.9 Config and dat files (`09-config.md`)
- → §3.10 Image and sound asset conventions (`10-assets.md`)

3.1 Levels: Z3-V1, GR-V1, GLEVNW01

A Graal level is one 64×64 board of tiles plus its links, baddies, NPCs, chests and signs. Three file families carry it: the binary Z3-V1 (`.zelda`) and GR-V1 (`.graal`) formats, and the text GLEVNW01 (`.nw`) format. The wire form of a board is in → §2.

> Status: draft. The 2.05 GR-V reader and writer and the 2.31 reader are measured; the 2.05 loader's Z3-V1 path and every build after 2.31 are not.

3.1.0 Format selection

Graal has three level formats, and the same file can carry the wrong extension. The rules below say how the client and the server pick a loader, and why a reader has to trust the magic over the name[21,151].

Item	Rule
magics in the 2.05 string table	`Z3-V1.03`, `Z3-V1.04`, `GR-V1.00` to `GR-V1.03`, `GLEVNW01`; each set appears three times, one copy next to the GWEBL001 cache code (→ §3.8)[21]
server loader	picks the loader from the extension `.nw`, `.graal` or `.zelda`; for an unknown extension, from the 8-byte magic[151]
server magic fallback	`GLEVNW01` → text loader; `GR-V1.01`, `GR-V1.02`, `GR-V1.03` → binary Graal loader; `Z3-V1.03`, `Z3-V1.04` → Zelda loader[151]
real files	a reader must decide by the magic, not the extension: `.zelda` names hold GR-V1 files and `.graal` names hold `Z3-V1.04` files[21,152]
`.zelda` names	694 of 801 distinct `.zelda`-named files in the old builds are `GR-V1.00` to `GR-V1.03`; 1st Graal's 16 levels are all `Z3-V1.04` under `.graal` names[152,153]
client magics	every client tested, 1st Graal to 2.31 and the NewWorld betas, carries `Z3-V1.03` and `Z3-V1.04` and no third `Z3-V1` revision; the 1st Graal and 1.39 loaders compare the 8 magic bytes only[152]
2.05 writer	always writes `GR-V1.03` for a level that is not saved as `.nw`[21]
2.31 writer	writes `GR-V1.04` only when the level has a height map, `GR-V1.03` otherwise[154]

A level name with no extension is a separate question. The rules below say what clients do with such a name, what the editor adds, and which builds were read to establish it[155].

Rule	Detail
bare names	no client adds `.zelda` to a level name without an extension, and none swaps `.zelda` for `.graal` when a file is missing; a link or warp loads the name as written[155]
editor	the editor's open and save buttons (and from 1.3 the snapshot save) add `.graal` to a name typed without an extension[155]
built-in names	1st to 9th Graal compare the current level with fixed names such as `level1.zelda` and `cave2.zelda` or warp to them[155]
scope	read in 26 Windows builds, 1st Graal to 1.32 and 2.05; that the loader uses the name unchanged is inferred[155]

Which builds accept which magics is in the build catalogue (→ §1.1).

3.1.1 Z3-V1 (.zelda)

Z3-V1 is the Zelda Online level family. Its board coding is that of GR-V1 (→ §3.1.2.1); only the code width and the sections differ.

3.1.1.1 Layout

A Zelda-era level is a run of sections in a fixed order with no index. The table gives the sections in file order and the shape of each, so a reader can walk the file from top to bottom[21].

Order	Section	Shape
1	magic	`char[8]`: `Z3-V1.02`, `Z3-V1.03` or `Z3-V1.04`[21]
2	board	bit stream until 4096 tiles are produced (→ §3.1.2.1)[21]
3	links	text lines until # or an empty line (→ §3.1.2.3)[21]

Order	Section	Shape
4	baddies	binary records until FF FF FF (→ §3.1.1.2)[21]
5	signs	text until an empty line or end of file (→ §3.1.2.6)[21]

Z3-V1 has **no NPC section and no chest section**[21].

3.1.1.2 Revisions

The three Z3 revisions differ in the width of a board code and in the baddy record. Each row gives one revision and the builds that read it[21].

Magic	Code width	Baddy record	Readers
`Z3-V1.02`	11 bits	4 bytes, last byte always `\n`	only the 1998 Java Zelda Online games; no real file is in the notes[21,156]
`Z3-V1.03`	12 bits	4 bytes: `x y type` then an empty line rest; no verse line	Zelda Online, Delphi and later clients, server[21]
`Z3-V1.04`	12 bits	`x y type` then the verse line (→ §3.1.2.4)	Delphi and later clients, server[21]

The rules below come from parsing real Z3 files. They cover details the layout leaves open: number formats, arrival coordinates and the bytes of a baddy[21].

Rule	Detail
link numbers	`Z3-V1.03` links are written with decimals (`31.0 42.0 2.0 1.0 43.5 41.0`), so the numbers are floating point[21]
negative arrival	a negative `newx` or `newy` means "keep the player's own coordinate"[21]
baddy bytes	the three raw bytes `x y type` are taken before the line end is looked for, because `0x0A` can be an `x` or a `y`[21]
baddy types	values 0 to 6 occur; Zelda Online's names for them are green, blue and red soldier, shot soldier, swamp soldier, jumping hare, octopus[21]
link destinations	no destination name contains a space; `Z3-V1.03` links name `.zelda` or `.editor` files, `Z3-V1.04` links `.graal`, `.zelda` or `.editor`[157]
verses	every `Z3-V1.04` record carries the verse line, most with all three verses empty; no `Z3-V1.03` record has one[158]
a GR-V1 reader on Z3-V1	integer link parsing drops 124 decimal links, a computer line end left in the stream shifts every later record, and a negative arrival is taken as a literal coordinate[159]
sample	the rules were checked against 139 distinct real files[21,160,161]

3.1.2 GR-V1 (.graal)

3.1.2.0 Layout

A Graal binary level keeps the Zelda layout and adds sections at the end. The table lists the sections in file order, with the revisions that carry each one[21].

Order	Section	Shape	Revisions
1	magic	`char[8]`: `GR-V1.00` to `GR-V1.04`	all[21,154]
2	board	bit stream until 4096 tiles are produced (→ §3.1.2.1)	all[21]
3	links	text lines until # or an empty line	all[154]
4	baddies	3-byte records, each followed by a verse line, until FF FF FF	all[154]
5	NPCs	text lines until # or an empty line	`GR-V1.00` and later[154]
6	chests	text lines until #	`GR-V1.01` and later[154]
7	signs	everything left (2.05); up to the first empty line (2.31)	all[154]
8	heights	empty line, then a `HEIGHTS` block (→ §3.1.2.7)	`GR-V1.04`; accepted after any magic by 2.31[154]

The board is a packed bit stream with run-length codes, not a 64×64 array of 2-byte tiles; only `.nw` text and the wire board form store one 16-bit value per tile[21].

3.1.2.1 Board bit stream

The board is 64 × 64 tile ids packed as a bit stream with run-length codes. The rules below give how to unpack it, from the bit order through to the run codes[21].

Rule	Detail
bit order	bytes are appended to a little-endian bit buffer; each code is the bottom `width` bits[21]
flag bit	the top bit of the code: `0x800` at 12 bits, `0x1000` at 13 bits[21]
flag clear	the code is a tile id[21]
flag set	a control code: `count = code & 0xFF`; if `code & 0x100` is set the next run is a double run[21]
default count	`count = 1`: the next tile is written once[21]
single run	the next tile is written `count` times, then `count` resets to 1[21]
double run	two tiles A and B are read and `A B` is written `count` times (`{double 4} 5 6` → `5 6 5 6 5 6 5 6`), then the count resets[21]
fill order	tiles fill `board[x + 64*y]` row by row, layer 0 only, until 4096 tiles are produced[21]
double flag	the Java Zelda Online reader keeps the double flag until a double run completes, the GR-V1 reader clears it at the next control code; no real file shows a difference (0 of 663)[161]
tile range	a 12-bit file cannot address tiles ≥ 2048, because the flag takes the top bit; that is why 1.02 widened the code to 13 bits for the 4096-tile `pics1`[21]

Code width per magic:

Magic	Width
`Z3-V1.02`	11[21]
`Z3-V1.03`, `Z3-V1.04`	12[21]
`GR-V1.00`, `GR-V1.01`	12[21]
`GR-V1.02`, `GR-V1.03`	13; the 2.05 loader compares the magic against 1.02 and 1.03 to pick it[21]
`GR-V1.04`	13[154]

3.1.2.2 Board writer (2.05)

The 2.05 writer walks the 4096 tiles in order and picks one case at each position[154]:

Case	Codes emitted
the next tile equals this one	`0x1000 |` n, then the tile; n counts 2 to 64 equal tiles[154,21]
tiles `i, i+1` equal `i+2, i+3` (pattern AB AB)	`0x1100 |` n, then A, then B; n counts up to 64 pairs[154]
otherwise	the tile, `& 0xFFF`[154]
packing	13 bits per code, least significant bit first; the last partial byte is flushed[154]
check	`write_board` in `tools/file-formats/gwebl.py` follows these rules and re-encodes 144 cached levels and 13 tutorial levels byte for byte[154]
full runs	single runs of exactly 64 occur in the embedded tutorial levels[21]

3.1.2.3 Links

Each link is one text line. The rules below give its fields and how a name with spaces, a missing target and a relative arrival point are handled[21].

Rule	Detail
line	`dest x y w h newx newy`[21]
spaces	the level name may contain spaces; the tokens before the last six are rejoined into `dest`[21]
missing level	a link to a level that does not exist is dropped[21]
arrival	`newx` and `newy` may be `playerx` or `playery`, optionally followed by a signed number (`playery+10`)[21]

Rule	Detail
arrival in the corpus	of 18,710 arrival fields: 12,035 plain numbers (integers and half tiles), 6,652 bare `playerx`/`playery`, 21 keyword plus signed offset (`playerx-15`, `playery-1.5`) in 14 levels, 2 stray words; whether a client evaluates the offset is open. Destinations end `.graal` 9,203 times, `.nw` 140, `.enigma` 10 and with no extension [162]
terminator	a line # or an empty line[21]
negative arrival	805 of 5381 distinct GR-V1 links in the old builds have a negative arrival; that 2.x reads it as "keep the player's coordinate", as Z3-V1 does, is inferred[157]

How the client resolves an arrival expression is in → §4.

3.1.2.4 Baddies

Each baddy is three raw bytes and a line of verses. The rules below give the record, why its line end needs care, and how the list ends[21].

Rule	Detail
record	`i8 x, i8 y, i8 type`, then a `\n`-terminated verse line[21]
verse line	three verses separated by `\`: sight, hurt, attack
line end	the client reads up to a `\n` that comes after at least 3 bytes, so the binary bytes may contain `0x0A`[154]
terminator	`FF FF FF` followed by an empty line; the client stops at a record whose first byte is ≥ `0x80`[21,154]
x of 0	a valid baddy; real levels have them; a reader that stops at x = 0 loses them and reads the rest of the baddy bytes as NPC lines[156,163]
corpus	902 baddies in 99 of 3113 levels; types 0 to 9 all occur[163,164]
short record	the reference decoder also ends the section at a record shorter than 3 bytes[156]

3.1.2.5 NPCs and chests

NPCs and chests are short records. The table gives the fields of each in order[21].

Record	Shape
NPC	`GCHAR x, GCHAR y, image#script`; the script uses `0xA7` for a newline; an empty image is a script-only NPC[21]
chest	`GCHAR x, GCHAR y, GCHAR item, GCHAR signindex`[21]

Coordinates in these records are whole tiles (GCHAR), not half-tiles[21]. NPC bytes are single bytes and can lie outside 0 to 63, as 165 NPCs in the corpus do. A level with no chests still ends the section with a bare `#`. Chest items 0 to 11, 16, 17, 19 and 24 occur, plus one 78, outside the 25-name table; 343 of 375 chests have sign index −1, which is "no sign"[165,166]. The chest item index table is in → §8.

3.1.2.6 Signs

Signs are stored in the sign alphabet, not in plain text. The rules below give the record, the alphabet, the line-break token and where the section ends in each build[21].

Rule	Detail
record	`GCHAR x, GCHAR y`, then text in the sign alphabet[21]
alphabet	the same as signs on the wire: `byte - 32` indexes the sign table (→ §3.6)[154]
line break	the token `v*eUWf + 0x80` decodes to `#K(13)` and a newline; it is the explicit line-break marker, not a space[154]
end	2.05 reads signs to end of file; 2.31 stops at the first empty line, because a `HEIGHTS` block can follow[154]
signs at (0,0)	common in real levels and hold dialogue text, so they are probably messages that NPC scripts show by index rather than signs on the map[154,166]
no signs	a level with no signs ends after the chest #, with no empty line; in 3113 files nothing follows the signs[165]

3.1.2.7 GR-V1.04 height block

GR-V1.04 is GR-V1.03 plus an optional height map[154].

```
<empty line>
HEIGHTS
h00,h01,…,h08
```

```
…                       9 lines of 9 comma-separated decimals
h80,h81,…,h88
HEIGHTSEND
```

Rule	Detail
position	after the signs and one empty line[154]
reader	accepts the block after the signs whatever the magic says; matches `HEIGHTSEND` case-insensitively; keeps at most 9 rows of 9 values[154]
board	unchanged: 13 bits, the same run codes[154]
same overlay	the 9×9 grid is the `HEIGHTS` overlay that GLEVNW01 carries and TerrainGenerator writes (→ §3.1.3.5)[154]
meaning	a 9×9 grid fits the corners of an 8×8 grid of 8×8-tile cells on a 64×64 level[154]
where	in 2.31, GR-V parsing appears only in the web-level cache code, the one place that client both writes and reads GR-V1.04 (→ §3.8)[154]

3.1.2.8 Embedded tutorial levels (2.05)

The 2.05 executable carries the offline tutorial as 13 `RCDATA` resources, all plain `GR-V1.03`[154]:

Resource	Level name
`TUTORIALA1` to `TUTORIALA4`	`tutorial_a-1.graal` to `tutorial_a-4.graal`[154]
`TUTORIALB1` to `TUTORIALB4`	`tutorial_b-1.graal` to `tutorial_b-4.graal`[154]
`TUTORIALB2H`	`tutorial_b-2_house.graal`[154]
`TUTORIALC1` to `TUTORIALC4`	`tutorial_c-1.graal` to `tutorial_c-4.graal`[154]

2.05 carries its tutorial inside the executable. The rules below say when it opens those levels and how they fit together, and give the tool that pulls them out[154].

Rule	Detail
selection	when the client is not connected and the requested level name starts with `tutorial`, it opens the matching resource instead of a file on disk[154]
grid	the levels form a 3×4 grid joined by edge links; the B2 house, B4 and B2 hold all the NPCs, chests and signs[154]
2.22	2.22 embeds 9 tutorial levels; all parse with the expected link grid, chest items and sign text[156]
extraction	`tools/file-formats/extract_tutorials.py Graal.exe OUTDIR --json` writes the 13 `.graal` files and a parsed JSON for each[154]

3.1.2.9 Reference decoder

`formats/grv1-decoder/grv1.py` decodes `GR-V1.00` to `GR-V1.04` and `Z3-V1.02` to `Z3-V1.04` with the rules above; NPC scripts have `0xA7` turned into newlines, and `#K(13)` is dropped from sign text[156].

3.1.3 GLEVNW01 (.nw)

3.1.3.1 File

The `.nw` level is plain text. The rules below give its general form and how it travels[151].

Rule	Detail
form	line-oriented text; the first line is the signature `GLEVNW01`[151]
editing	official notes say to edit it in an editor that does not wrap lines[151]
transfer	servers send the same payload as for a binary `.graal`; only the on-disk form is larger[151]

3.1.3.2 BOARD

```
BOARD x y width layerindex tilesdata
```

Rule	Detail
meaning	one horizontal run of `width` tiles on the background, starting at `(x, y)`[151]

Rule	Detail
tilesdata	`width*2` characters; each tile is a 12-bit index into the tile list (`pics1.png`), two base64 characters, high 6 bits first[151]
alphabet	`ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/`; `tile = index(left) << 6 | index(right)`[151]
examples	`AA` is tile 0, `AB` tile 1, `//` tile 4095[151]
trailing =	the client alphabet in the server's level code appends a =, which is not part of the 64-character alphabet[151]
rejection	a line is ignored unless it has 6 tokens, the run stays inside x 0 to 63, and `tilesdata` has at least `width*2` characters[151]
storage	the tile goes to `levelTiles[x + y*64]`; `layerindex` is parsed and not used[151]

3.1.3.3 Other records

Apart from the board, a `.nw` level is a list of keyword records. The table gives each record's shape and the rules that a writer must keep[151].

Record	Shape	Rules
link	`LINK dest x y width height newx newy`	`dest`, newx and newy must not contain spaces[151]
sign	`SIGN x y`, text lines, `SIGNEND`	the body may be empty[151]
NPC	`NPC image x y`, script lines, `NPCEND`	`-` if the NPC has no image; x and y may be floating point[151]
baddy	`BADDY x y type`, verse lines, `BADDYEND`	type is a name or 0 to 9; verses may be shortened, but `BADDYEND` is required[151]
chest	`CHEST x y item signindex`	item is a name or 0 to 24, resolved by the server's item table; an unknown item drops the chest[151]

3.1.3.4 GSERVL01

The 3.x clients know a second text signature. The rules below say what it is, which parser reads it and which keywords it adds[7].

Rule	Detail
magic	`GSERVL01`, a second text-level signature known to the 3.x clients[7]
parser	read by the same parser as GLEVNW01[7]
keywords	`BOARD`, `LINK`, `CHEST`, `SIGN`, `NPC`…`NPCEND`, `BADDY`…`BADDYEND`, `HEIGHTS`…`HEIGHTSEND`, and `REPLACENPCS`… `REPLACENPCSEND`, which the client skips[7]
later builds	4.x lists it among the magics it knows, as in 3.x[34]

3.1.3.5 Other writers of the same keywords

Three tools write level text, and only two of them produce what the loader accepts[151]:

Writer	Output
GraalEditor	opens `*.graal` and `*.nw` with one filter and treats both as text; does not require the signature; `BOARD 0` is the tile layer as 64 lines of 64 decimal tile indices, not a base64 run; heights are a separate overlay, not a second `BOARD`; keywords `LINK`, `SIGN`, `CHEST`, `NPC`, `BADDY`, `LEVELNAMES` with fields in the order the dialogs collect them[151]
GraalEditor compatibility	the decimal `BOARD 0` text is not what the loader accepts; it must be rewritten as GLEVNW01 base64 runs before it loads, and a client `.nw` is not a decimal grid[151]
TerrainGenerator	GLEVNW01 with 64 rows of tile 0 (each row 64 <y> and 128 A), a `HEIGHTS` block of nine lines of nine numbers closed by `HEIGHTSEND`, then a `LINK` on each edge that has a neighbour; no NPC section[151]
DungeonGenerator	GLEVNW01 with base64 `BOARD` runs for 64 by 64 tiles, edge `LINK` lines and `NPC` blocks; stamp id 0 is replaced by a 4 by 4 ground fill before the board is stored, and stamp id 1 is a hole and is not written[151]

The tools themselves are in → §11.

3.2 GRMAP001 and gmaps

A gmap is a grid of levels that the client walks across without a level warp. Its file is `GRMAP001` text. The 2.05-era text big map and minimap, which name a grid of levels for the map screen only, are a different format and are in → §3.2.4. How a client crosses a gmap edge on the wire is in → §4.10.

> Status: draft. The server reader and three tool writers are summarised; the 2.05 reader is read statically and may be dormant; how 2.17 and later clients lay out the gmap overview is unmeasured.

3.2.1 Three kinds of map

Three different mechanisms are called a map, and they do not mix. Each row names one, the file behind it, the server option that turns it on and what the player sees[167].

Kind	File	Server option	Effect
level links	none; links inside each level (→ §3.1.2.3)	none	walking into a link rectangle makes the client ask for a level warp[167]
server big map	`bigmap.txt`-style text grid	`maps =`	tells the server which levels are adjacent, so players see and chat across edges; the client does nothing special[167]
gmap	`GRMAP001`	`gmaps =`	the client walks continuously across level borders[167]
map-screen overlays	text grid and image	`bigmap =`, `minimap =` (four comma fields `text,image,x,y`)	the server sends the overlay settings at login to 2.1 and later clients; the notes warn that setting `bigmap` breaks gmaps[167]

3.2.2 GRMAP001 as the server reads it

A `.gmap` file is a list of keyword lines. The table gives each key the server reads, with its arguments and meaning[168].

Key	Arguments	Meaning
`GRMAP001`	none	signature, first line[168]
`WIDTH`, `HEIGHT`	n	grid size in levels[168]
`GENERATED`	one argument	read and stored; not used to rebuild names on the server side[168]
`LEVELNAMES` … `LEVELNAMESEND`	one grid row per line, quoted, comma-separated names	fills the grid row-major; names are lowercased for lookup[168]
`MAPIMG`	file	overview image[168]
`MINIMAPIMG`	file	minimap image[168]
`NOAUTOMAPPING`	none	flag[168]
`LOADFULLMAP`	none	flag: preload every level[168]
`LOADATSTART` … `LOADATSTARTEND`	name list	levels to preload[168]

The rules below cover how those lines are split, how the world coordinates are formed, and how a client gets the file[168].

Rule	Detail
tokens	each line is split on whitespace after `\r` is stripped[168]
name rows	lines inside the `LEVELNAMES` block are parsed as quoted CSV[168]
coordinates	position in the world is `cell*64 + local`; local = global − 64 × cell[167]
transfer	a client fetches the `.gmap` file itself as a file transfer when it does not have it[167]

3.2.3 GRMAP001 as the tools write it

Three writers emit the signature, each with a different set of keys; `GENERATED` does not mean the same thing in all three[169].

Writer	Keys in order	Level names
DungeonGenerator (short form)	GRMAP001, WIDTH, HEIGHT, GENERATED <bottom-right level name>; no LEVELNAMES block	LEVELSTART, a column letter and a 1-based row; the grid is rebuilt from the last level's name in GENERATED[169]
TerrainGenerator (generator form)	GRMAP001, WIDTH, HEIGHT, GENERATED 1, GENSEED, GENBASE, GENEVENBORDERS (0 or 1), GENHEIGHT, GENCHAOS, LEVHEIGHT, LEVCHAOS, MAPIMG <basename>.png, one level name per row, HEIGHTMAP … HEIGHTMAPEND, RANDOMSEEDS … RANDOMSEEDSEND (one seed per level, when present)	basename, a column A then AA, a zero-padded row, and .nw; GENERATED 1 is a flag here[169]
GraalLevelGen	GRMAP001, WIDTH, HEIGHT, then a LEVELNAMES list of quoted, comma-separated file names	as listed[169]

The map tools write keys the server does not read. The rules below say what those keys hold and where the per-level heights go[169].

Rule	Detail
MAPIMG	the overview the client shows on the map list; the PNG TerrainGenerator saves from its preview dialog is a separate file[169]
HEIGHTMAP	the height field; a sibling _heights.txt holds the same numbers without the gmap wrapper[169]
per-level heights	each level file then carries a 9 by 9 HEIGHTS block (→ §3.1.2.7)[169]

3.2.4 Client readers by build

Not every client reads a gmap the same way. Each row gives one build's reader and the map path it uses; rows marked unmeasured have only lab captures of the crossing[168].

Build	Gmap reader	Map path in use
2.05 rev 2	has the GRMAP001 string, a reader that knows WIDTH, HEIGHT, GENERATED, GENSEED, GENBASE, GENEVENBORDERS, GENHEIGHT, GENCHAOS, LEVHEIGHT, LEVCHAOS, MAPIMG, HEIGHTMAP … HEIGHTMAPEND, RANDOMSEEDS … RANDOMSEEDSEND and HEIGHTS, and a map class; no static caller of the reader was found, so it may be dormant	the text big map and minimap below; no gmap warp handler and no gmap cell prop name[168]
2.22	unmeasured	gmap crossing captured in the lab (→ §4.11.9)[167]
2.31	unmeasured	gmap crossing captured in the lab (→ §4.11.9)[167]
Windows 4.04r2	separate coarse HEIGHTMAP and named per-level HEIGHTS branches; the latter normalizes a level filename, looks it up and stores samples in its height record with nine-by-nine capacity	no new rendering or server run; a separate Mac 4.04r2 file fixture contains one four-by-four coarse grid and nine named nine-by-nine grids[170]

For Windows 4.04r2's matching-level branch, storage is limited to nine rows and nine values per row with fixed nine-value strides. Fewer than 81 supplied values do not by themselves cause rejection; excess values are not stored by this branch. Missing-cell defaults and numeric-error handling remain unmeasured.[170]

An earlier version of the gmap note said 2.05 had no GRMAP001 string; that was wrong, and the build catalogue also lists it for 2.05[168].

3.2.5 Text big map and minimap (2.05)

2.05 reads the big map and minimap as text grids; the GS1 commands setmap and setminimap load the same kind of file from a script[168].

Map	File	Image	Built-in integers
big map	bigmap.txt	map.png	10, 16, 1688, 1024[168]
minimap	minimap.txt	map2.png	664, 896[168]
tutorial (offline)	tutorialmap.txt	tutorialmap.png	3, 4, 192, 486[168]

The older big map and minimap are text grids of level names. The rules below give how 2.05 reads them and the shape of the shipped file[168].

Rule	Detail
lines	plain text, one map row per line, each parsed as a comma-separated list with quoting allowed[168]
cells	each cell is normalised and appended to one flat list; a cell is a level file name, and x marks an empty cell[168]
shape	the column count is the number of cells on the first line; the row count is total cells ÷ columns[168]
shipped file	the shipped `bigmap.txt` is 20 columns × 21 rows of quoted level names[168]
server-list override	an entry with more than the usual 8 fields replaces these: field 8 big-map file, 9 its image, 10 and 11 two integers, 12 minimap file, 13 its image, 14 and 15 two integers[168]
integers	stored as points in the player object; probably image offsets or scale, which fits the defaults, but their use is not traced[168]
later builds	the server-pushed forms `PLO_BIGMAP` (171) and `PLO_MINIMAP` (172, for example `zodiacminimap.txt,zodiacworldminimap3.png,10,10`) replace this; they carry file names plus a default x,y for heads of players off the map[168,167]

The server-list entry layout is in → §6.

3.3 GANI animations

A GANI file is the text animation format that GraalShop reads and writes and that clients load from `levels/ganis/`. This chapter gives its grammar as the editor reads it; how a client plays a GANI is in → §4.

> Status: draft. The grammar is from the GraalShop reader and writer and checked against client sample files; the separate exact-input Mac v5 loader result is scoped in §3.3.7.

3.3.1 File

A gani is a text file of keyword lines. The rules below give the general form and the samples it was checked against[171].

Rule	Detail
form	line-oriented text; the first line is the discriminator[171]
case	the parser lowercases each line before comparing keywords; the writer emits uppercase keywords and space-padded columns[171]
samples	client samples under `levels/ganis/` and `levels/bomys/` match the sprite dialect, including files that omit the signature[171]

The first line tells sprite mode from movie mode. Each row gives one signature, the mode value it sets and the body that follows[171].

First line	Mode value	Body
`GANI0001`	0	sprite table, header, `ANI` … `ANIEND` (sprite mode)[171]
`GANI0002`	1	sprite table, header, `MOVIE` … `MOVIEEND` (movie mode)[171]

3.3.2 Grammar

The grammar as the editor accepts it[171]:

```
file        := signature? sprite* header* body
signature   := "GANI0001" | "GANI0002"
sprite      := "SPRITE" id sheet x y w h description
             | "ATTACHSPRITE"  id parent dx dy
             | "ATTACHSPRITE2" id parent dx dy
header      := "LOOP" | "CONTINUOUS" | "SINGLEDIRECTION"
             | "SETBACKTO" name
             | "BACKCOLOR" "#" hex6
             | "FRAMES" count
             | "DEFAULTHEAD" file | "DEFAULTBODY" file
             | "DEFAULTPARAM" n value | "DEFAULTATTR" n value
             | "COLOREFFECT" id args | "ZOOMEFFECT" id args
             | "ROTATEEFFECT" id args
             | "STRETCHXEFFECT" id args | "STRETCHYEFFECT" id args
             | "EFFECTMODE" id args
body        := ani | movie
ani         := "ANI" step* "ANIEND"
movie       := "MOVIE" actor* "MOVIEEND"
actor       := "ACTOR" name frame* "ACTOREND"
frame       := "FRAME" index token ("," token)*
step        := placement ("," placement)* | "WAIT" n | "PLAYSOUND" name x y
placement   := spriteId x y | "PARAM" n x y | "ATTR" n x y
script      := "SCRIPT" rawlines "SCRIPTEND"
```

3.3.3 Sprite rows

Sprite rows cut images out of sheets. The rules below give their fields, the column widths the writer uses, and how attached sprites work[171].

Rule	Detail
SPRITE	id, sheet, x, y, w, h, description[171]
columns	the writer pads the id to 4, the sheet name to 14, and x, y, w and h to 4, then writes the description[171]
ATTACHSPRITE, ATTACHSPRITE2	hang a sprite on a parent sprite with a pixel offset; they are not a second sheet[171]

Client files therefore look like this[171]:

```
SPRITE    0         SPRITES    0    0   24   12 shadow
SPRITE  200            HEAD    0    0   32   32 head up
SPRITE   20           SWORD   62    0   28   28 sword
```

3.3.4 Header

The header keywords set how an animation plays. Each row gives a keyword and its meaning; some are written in only one mode[171].

Keyword	Meaning
LOOP, CONTINUOUS	playback flags[171]
SINGLEDIRECTION	each step holds one row instead of four; written only in sprite mode[171]
FRAMES	frame count; written only in movie mode[171]
SETBACKTO	names the GANI to return to[171]
BACKCOLOR	a #rrggbb fill behind the editor preview[171]
DEFAULTHEAD, DEFAULTBODY	reload the head and body images[171]
DEFAULTPARAM, DEFAULTATTR	seed the 30-row attribute grid[171]
COLOREFFECT, ZOOMEFFECT, ROTATEEFFECT, STRETCHXEFFECT, STRETCHYEFFECT, EFFECTMODE	stored as an id plus arguments and applied at blit time[171]
SCRIPT … SCRIPTEND	copied through as text; the editor does not execute it[171]

3.3.5 Sprite mode body

In sprite mode the body is a list of steps. The rules below say what a step can hold and how directions are laid out[171].

Rule	Detail
bounds	ANI opens the step list and ANIEND closes it[171]
step	a comma-separated placement row, a WAIT hold, or a PLAYSOUND name x y cue[171]
placement	a sprite id plus x and y, or a PARAM or ATTR slot plus x and y[171]
directions	a step holds four direction rows, or one row when SINGLEDIRECTION is set[171]
wait	the wait counter starts at 1; each WAIT adds to it[171]

3.3.6 Movie mode body

In movie mode the body is a list of actors, each with its own frames. The rules below give that structure[171].

Rule	Detail
bounds	MOVIE opens the actor list and MOVIEEND closes it[171]
actor	ACTOR name, a run of FRAME lines, ACTOREND[171]
actor kind	not a keyword in the frame line but an integer on the actor: 0 CHAR, 1 SPRITE, 2 SOUND, 3 TEXT[171]
frame	an index and a comma-separated token list; the tokens become key=value strings on the actor's frame record[171]

3.3.7 Shared Mac v5 loader boundary

The shared PowerPC v5 main executable, including an archive copy labelled 5.12, was inspected statically; matching bytes do not authenticate release labels. ANI collects comma-delimited placement records into a group requiring four records, or one with SINGLEDIRECTION. ANIEND accepts a sufficient group and discards an incomplete one. End of input destroys a still-current group rather than establishing its commitment.[172]

WAIT adds duration to an existing group. PLAYSOUND stores a resource reference, the current duration and two default fields; both optional numeric arguments must exist to replace those defaults. These storage observations do not prove units, scheduling, audible playback or rendering. SCRIPT collects text through SCRIPTEND or end of input; this loader body does not run an interpreter. Native runtime acceptance, full placement grammar, numeric/tokenizer helpers and GS1/GS2 execution remain untested here.[172]

3.4 GOBJSET01 object sets

An object set is a list of named multi-tile stamps that an editor places on a board. It carries tile ids only; what a tile does is decided by the tile type table (→ §3.5).

3.4.1 Layout

```
GOBJSET01
OBJECT width height name
<height lines of width*2 base64 characters>
OBJECTEND
…
```

Rule	Detail
signature	`GOBJSET01`[173]
tile packing	the same 12-bit base64 packing as GLEVNW01 (→ §3.1.3.2)[173]
name	may contain spaces[173]
no other fields	there is no layer, link, NPC or type byte[173]
placing	placing a stamp writes its tile ids onto the board; collision is decided later by the type table[173]

3.4.2 Shipped sets

The client ships five object sets. The table gives what they hold and decodes the first stamp as a worked example. The last row points to where Bush collision is discussed[173].

Item	Detail
sets in the client tree	`objectsStandard`, `objectsHouse Inside`, `objectsCastle`, `objectsSnow`, `objectsVolcano`[173]
objects	288 named multi-tile objects across the five sets[173]
first stamp	`objectsStandard` begins with the 2×2 "Bush", base64 `ACAD` / `ASAT` → ids 2, 3, 18, 19[173]
Bush collision	file-membership correction → §3.5.3.1; runtime collision remains unmeasured → App. E#100[174]
generator stamps	in DungeonGenerator, stamp id 0 is a 4×4 ground fill and stamp id 1 is a hole that is not written (→ §3.1.3.5)[173]

3.5 Tile types and arrays.dat

A board stores tile ids; what a tile does is looked up in separate tables. This chapter gives the tile type values, the tile sheet geometry, the `arrays.dat` table file and the signed `arrays2.dat` with its `.sig` files. How movement uses these values is in → §4.2.

> Status: draft. Only part of the 25 `arrays.dat` tables is traced to a gameplay use; the signature key encoding is not pinned down (→ §3.5.5).

3.5.1 Tile sheet geometry

A tile id is a position on a sheet and nothing more. The rules below give the sheet's size and the formula from id to picture. The sheet is laid out in blocks, so the formula is not a plain row and column[175].

Rule	Detail
id	a board stores a tile id 0 to 4095; the client does not inspect the tileset image for collision[175]
sheet	`pics1.png` is 2048×512, palette, 128 by 32 tiles of 16 px[175]
id to picture	`col = (id % 16) + (id // 512) * 16`, `row = (id // 16) % 32`, source rectangle `(col*16, row*16, 16, 16)`: the sheet is eight side-by-side blocks of 512 ids, each 16 columns by 32 rows, filled row by row; an earlier rule `col = id % 128`, `row = id / 128` is `disputed → App. E#102`[176]

3.5.2 Tile type values

The official tile type names, as documented for `setshape2` in the 2001 feature notes[175]:

Type	Official name
0	nonblocking[175]
2	hurting underground[175]
3	chair[175]
4	bed, upper side[175]
5	bed, lower side[175]
6	swamp[175]
7	lava swamp[175]
8	near water[175]
9	near lava[175]
11	water[175]
12	lava[175]
20	throw-through (fences)[175]
21	jumping stone[175]
22	blocking tile[175]

3.5.2.1 A 4096-byte type table

A 4096-byte table copied from the standard tileset, one type byte per id, is kept in an outside client's source and written as `tiledefs.txt`: decimal, row-major, 128 values per line (the width of `pics1.png`)[175]. Its counts:

Value	Count	Official meaning
0	823	nonblocking[175]
3	40	chair[175]
4	4	bed, upper[175]
5	4	bed, lower[175]
6	12	swamp[175]
7	8	lava swamp[175]
8	30	near water[175]

Value	Count	Official meaning
11	47	water[175]
12	67	lava[175]
20	129	throw-through fence[175]
21	18	jumping stone[175]
22	2914	blocking[175]

The rules below add what the counts leave out: the types missing from the table, how its first row reads, and how that outside client tests a tile. It is not the official client's method, which follows in §3.5.2.2[175].

Rule	Detail
absent types	types 2 and 9 are in the official list but do not appear in this table[175]
first row	the first data line, ids 0 to 127, starts `0,0,20,20,…`: the top-left two tiles walk and the next two are walls[175]
that client's tests	wall if the type byte is ≥ 20 or the foot is outside the 64×64 board; water if it is 11; walkable otherwise[175]

3.5.2.2 Sheet types and the type-0 allow lists (official client)

The official client decides walkability differently for each kind of sheet. The rules below come from live tests: how sheets are listed and chosen, and the two allow lists that decide a type-0 tile[177].

Rule	Detail
tile definition list	each entry is `image,prefix,type` with optional `x,y` offsets; the default is `pics1.png` of type 0, stock selection `pics1.png,,0`[177,175]
sheet order	type 0 is the old Graal tile order (`pics1`), type 1 the new order; `addtiledef2` paints a rectangle on top of the base sheet; the offline tree holds `picso.png,offlinetutorial,1,0,0`[176]
sheet choice	a level may reference several sheets; the active one is selected when the level is entered[177]
type-0 sheet	a tile walks only if its 12-bit id is in one of two internal allow lists (the "nowall" lists); an id in neither blocks[177,178]
allow lists	built when the sheet loads; not stored in `tiledefs.txt`, which names only the sheet, prefix and type; they are arrays 0 and 1 of `arrays.dat` (→ §3.5.3)[177,178]
type 1 and 2 sheets	consult a fixed 0/1 body-block grid; one extraction counted 1725 blocked and 2371 walkable cells[177,175]
type-0 mask	`disputed → App. E#101`[177]
`setshape2`	blocks the body when the shape-tile type is greater than 19[175]
throw	throw handling does not stop on type `0x14` (20)[175,177]
water	type 11 does not block; it changes the displayed tile and may set a swimming flag[177]

The Bush-value membership correction is recorded in → §3.5.3.1; runtime collision remains open → App. E#100[174].

3.5.3 arrays.dat

`arrays.dat` is a tree of integer arrays with a small header on each. The rules below give the encoding needed to walk it[178].

Rule	Detail
integers	big-endian 32-bit[178]
array header	`header = (kind << 16) | count`[178]
kind 0	`count` int32 values follow[178]
kind 1	`count` nested arrays follow, each with its own header[178]
order	the client reads exactly 25 arrays in a fixed order into 25 globals at startup[178]
missing file	the client quits with "File not found"; the file is not signed or checked[178]
2.22 file	11500 bytes; parses to exactly 25 arrays with nothing left over[178]
tile ids	all tile ids in the tables index the 4096-tile `pics1` sheet[178]

The table lists the top-level arrays of the 2.05 file in order, with each one's size and what it is known to hold. Rows that say "use not traced" are arrays whose reader has not been found[178].

#	Shape	Content
0	934 ints, max 4089	tile ids; with array 1, the legacy walkability test: in the old tile mode a tile blocks unless it is in array 0 or array 1[178]
1	116 ints	tile ids, second list for that test[178]
2	133 ints	tile ids; use not traced[178]
3	76 ints	tile ids; use not traced[178]
4	40 ints	tile ids; use not traced[178]
5, 6	4 ints each	single 2×2 tile blocks (708–711, 724–727); use not traced[178]
7	79 ints	tile ids; the legacy tile-type lookup maps a tile's position in this list to 4 classes (positions 0–11, 12–19, 20–62, 63+); the newer tile-type path returns types 6 to 9 for the same classes[178]
8	4 lists, one per facing	ledge tiles (type 21): a ledge jump needs the tile to be in the list for the player's facing[178]
9 to 12	small nested lists of 2×2 tile groups	level-object code: cut and lift[178]
13	4 ints	one 2×2 tile block, used with array 14[178]
14, 15	small nested lists of 2×2 tile groups	level-object code[178]
16	2 × 2×2 blocks	stone blocks: entry 0 pushable, entry 1 pullable[178]
17	124 × 4	packed records used by the horse object[178]
18	nested	use not traced[178]
19	nested	used by the leap object[178]
20, 22, 23	nested	used by the baddy AI object[178]
21	4 × 4	packed values; use not traced in detail[178]
24	356 ints, packed	used by the item, bomb and extras objects[178]

Meanings read from the uses in the 2.22 client[179]:

#	Content (2.22)
0	passable tile ids; any other tile is a wall (→ App. E#101, E#608)[179]
1	liquid tile ids, water and lava[179]
3	the liquid tiles that are lava[179]
9, 10	cuttable 2×2 objects and the tiles a cut leaves, indexed type, column, row: the bush `2 3 / 18 19` and `420 421 / 436 437`[179]
11, 12	liftable 2×2 objects and the tiles a lift leaves: five types, among them the bush and the rock `34 35 / 50 51`[179]
13	closed chest tiles 1784, 1785, 1800, 1801[179]
14	open chest tiles 829, 845 / 830, 846[179]

A loader that reads only the first four arrays as flat lists is right for 0 to 3 and misses the nested arrays 9 to 12[179].

3.5.3.1 Bounded packaged-file membership

One packaged `arrays.dat` was read exactly to settle which tile ids are in its first two lists. Each rule records part of that reading and how far it reaches[174].

Rule	Detail
identity	11,500-byte input, researcher-supplied SHA-256 b65905ad1e1d1e48d2b7841525c749272426e5160d187f05e8d6b37946296927; appears in 2.22-labelled and 2.3-labelled packages and a loose copy; release identity and runtime loading are unverified[174]
structure	25 top-level arrays, no trailing bytes; list 0/list 1 mean the first two structural positions, not runtime categories[174]
first lists	934 and 116 internally unique values; 15 shared; union 1,035 values within 0–4095; list 1 contributes 101 values absent from list 0[174]
Bush values	2, 3, 18 and 19 are absent from both lists, also in all 25 successfully parsed distinct inputs; file membership does not establish runtime collision[174]

Rule	Detail
collection validation	53 occurrences, 26 hashes; 25 parse completely and reproduce original bytes on re-encoding; an unsupported 39ster-labelled representation is excluded; other inputs have different shapes and counts[174]
remaining scope	no consuming path or runtime movement was checked; earlier consumer evidence retains its original build scope; mask provenance and one-list versus two-list consumption remain open → App. E#100, E#101, E#608[174]

3.5.4 arrays2.dat

2.05 does not reference `arrays2.dat`, `graal.sig` or `arrays2.sig` and has no signature check[178]. In 2.17, 2.22 and 2.31 the file is plain text with CRLF lines, numbered from 0; 2.22 needs at least 5 lines[178].

Line	Use
0	main listserver host[178]
1	listserver port[178]
2	web host, joined into a URL template[178]
3	registration page URL[178]
4	translation service URL[178]
5, 6	fallback listserver hosts, optional[178]

A fan-server copy of the file points line 0 at its own `<listserver-host>` with port 14922 and lists two other list hosts as fallbacks[178]. The 2.05 list host is hidden in the executable instead (→ §6).

3.5.5 graal.sig and arrays2.sig

Two 128-byte signature files guard the executable and `arrays2.dat`. The rules below give what is known of the key, the order of checks and the algorithm, and which parts are judged from names alone[178].

Rule	Detail
size	both files are 128 bytes, the size of an RSA-1024 signature[178]
key	built at startup from an obfuscated hex string assembled one character at a time; each decoded byte is `not(b) - 10`[178]
order	`graal.sig` is verified against the client's own executable file, then `arrays2.sig` against `arrays2.dat`[178]
algorithm	an RSA component configured with a SHA-1 signing digest and a PKCS#1 scheme, judged from the component's class and property names; no signature has been verified[178]
failure	the client shows "The Graal execution file is corrupted. Please check for viruses or other programs that manipulate..." and exits[178]
builds	2.17, 2.22 and 2.31 embed the same two key strings and the same error text[178]
open	checking the shipped signatures against the decoded key with exponents 3, 17 and 65537 did not reproduce PKCS#1 padding; the key encoding is not pinned down[178]
fan-server 2.22	that executable has the three branches that raise the verify error rewritten so verification always passes; the 2.17 and 2.31 executables do not[178]
consequence	an unpatched 2.17, 2.22 or 2.31 client refuses an edited `arrays2.dat` or executable; editing `arrays2.dat` alone fails the signature[178]

The check as a client integrity behaviour is in → §4.10.

3.6 Sign text alphabets

Binary sign text uses one packed alphabet. The same encoding is used in the sign sections of `.graal` and `.zelda` files (→ §3.1.2.6) and on the wire in `PLO_LEVELSIGN`; GLEVNW01 text signs are plain text[180].

3.6.1 Encoding

Sign text is stored and sent in Graal's own alphabet, not in ASCII. The rules below give the record and the alphabet, code range by code range[180].

Rule	Detail
record	`GCHAR x`, `GCHAR y`, then the text bytes[180]
text byte	a GCHAR: `byte - 32` indexes a fixed 97-entry alphabet, the same string in the 2.05 client and in GServer-v2[180]
codes 0 to 61	`A–Z`, `a–z`, `0–9`[180]
codes 62 to 96	punctuation, space (code 89, byte `0x79`) and newline (code 96, byte `0x80`); several slots are # placeholders[180]
glyph codes	one or two codes stand for # plus a symbol: #A #B #X #Y (button icons), #u #d #l #r (arrows), #x #y #z, #h and #4 (two codes each), and `#.`; code 86 is a literal #[180]
other characters	written as the escape `#K(n)`, with n the decimal character code spelled in alphabet codes[180]
controls	bytes under 32 are controls and are not shown[180]
line break	the editor writes CRLF as `#K(13)` and the newline code; in a raw dump that is the bytes `v*eUWf` and a byte with the high bit set[180]
in the corpus	2,381 signs in 808 levels, 684 of them at (0, 0) (text kept by index); 498 `#K(n)` escapes with n other than 13, most often 95 (_), 164, 61 (=), 42 (*), 36 ($), 64 (@), 94 (^), 124 (|), 134, 233, 169 and 43 (+)[166]

The alphabet, from code 0[154]:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!?-.,#>()#####"####':/~&### <####;\n
```

3.6.2 Corrections to earlier notes

Earlier notes decoded signs wrongly. Each row pairs an earlier claim with its correction, so a reader who meets the old reading knows what replaced it[180].

Earlier claim	Correction
two alphabets, one for files and one for the wire	one alphabet indexed by `byte - 32`[180]
a `char + 89` file rule with "subtract 6 from A–Z"	it only looked right for some characters[180]
`v*eUWf` + high byte = space	it is `#K(13)` + newline, a line break[180]
garbled house-outside signs	viewers that drop non-ASCII bytes produce that appearance[180]

NPC scripts in the same file are plain Latin-1 with `0xA7` as the newline, not sign-encoded[180].

3.6.3 GLEVNW01 signs

Signs in a `.nw` level do not use the alphabet. The two rules give the record and its plain text[180].

Rule	Detail
record	`SIGN x y`, the text, `SIGNEND`; the body may be empty (→ §3.1.3.3)[180]
text	plain; no alphabet transform[180]

3.6.4 Reference decoder

The repository has a decoder for the alphabet. The table lists its functions and the test vectors it was checked against[180].

Function	Use
`decodeSignRecord(line)`	a raw sign line or a `PLO_LEVELSIGN` payload → `{ x, y, text }`[180]
`decodeSignText(payload)`	the payload after the coordinates; by default the CR from `#K(13)` is dropped[180]
`encodeSignText`, `encodeSignRecord`	the reverse[180]
tests	`formats/sign-decoder/test.js` and `vectors.json`, from 11 real signs in GR-V1.03 levels[180]

3.7 Save games (graalsav)

Clients from 1st Graal (September 1998) to the 1.4x builds write offline save games; from 2.00 the server owns the save[181,182]. This chapter gives the save container, its revisions, the DES-wrapped local files and the server's account file.

> Status: draft. Field names rest on 13th Graal's script variables; the Graalsav9 and Graalsv10 layouts come only from readers; the DES wrapping has not been round-tripped on a real save.

3.7.1 Container

Every revision is a Delphi `TWriter` value stream, not a hand-made binary layout. Each value starts with a one-byte type tag[181]:

Tag	Type	Body
2	Int8	1 byte[181]
3	Int16	2 bytes, little-endian[181]
4	Int32	4 bytes, little-endian[181]
5	Extended	10-byte x87 float[181]
6	String	1-byte length and the characters[181]
12	LString	4-byte length and the characters; strings of 256 bytes or more[181]
8, 9	False, True	none[181]

A save file is a list of tagged values. The rules below say how to read the tags and where the field names come from[181].

Rule	Detail
integer width	the writer picks the smallest of Int8, Int16 and Int32, so a reader must go by the tag[181]
first value	always the magic string: `Graalsav1` to `Graalsav9`, `Graalsv10`, `Graalsv11`[181]
field names	taken from 13th Graal's script variables (`playerrupees`, `playerdarts` ...), whose storage lines up with that build's save record; other builds are matched by position[181]

3.7.2 Revisions by build

The save format was revised many times. Each row gives one magic, the builds that write and read it, the file extension, and whether it is DES-wrapped[181].

Magic	Written by	Read by	Extension	DES-wrapped
Graalsav1	1st, 2nd, 3rd, 4th	same	`.save`	no[181]
Graalsav2	4th alpha, 5th, 5th alpha	same	`.save`	no[181]
Graalsav3	6th	same	`.save`	no[181]
Graalsav4	7th, 8th	same	`.save`	no[181]
Graalsav5	9th alpha to 12th	same	`.save`	no[181]
Graalsav6	13th	same	`.save`	no[181]
Graalsav7	14th, 15th	same	`.save` (14th), `.gsave` (15th)	from 15th[181]
Graalsav8	1.1, 1.1.9	7 and 8	`.gsave`	yes[181]
Graalsav9, Graalsv10	no build in the set	1.21 to 1.32 readers	`.gsave`	yes[181]
Graalsv11	1.21 to 1.41 rev 1; writer traced in 1.21 and 1.32, later builds by string references	9, 10 and 11	`.gsave`	yes[181]

The rules below explain the odd steps in that table and correct an earlier catalogue entry[181].

Rule	Detail
4th alpha	built later than 4th, which is why it writes the newer revision[181]
1.33 to 1.41 rev 1	reference the three magics as 1.32 does (two references to `Graalsv11`, one each to `Graalsav9` and `Graalsv10`), so they write `Graalsv11`; the record itself was not re-read[181]
catalogue correction	earlier catalogue rows listed 1.21 to 1.39 as "Graalsav9"; a search for `Graalsav` misses `Graalsv10` and `Graalsv11`[181]
v9 and v10	presumably match saves from a 1.2x build that is not in the archive[181]
15th Graal	saving only works on a level from one named community server or on the online start level[181]
1.21 on	the client refuses to save online: "The server saves your game if you have an own account."[181]
2.00 on	the server owns the save; 2.05 contains "Save games are not supported anymore", and the offline F2/F3 save games ended with 1.41[182]

3.7.3 File layout

```
magic                     String
mode                      Integer   0 one player, 1 two players, 2 online, 3/4 three/four players (14th
on)
[mode 2]      String, String        two online-session strings
player record × n         n = 1, 2, or the player count in 3/4-player modes
[Graalsav2+]  flag list   Integer count, then count × String
```

Rule	Detail
layout	as above[181]
online strings	nickname and server[181]
Graalsv11 header	`magic`, then `Integer players-1`, then the records and the flag list[181]
6 to 7	the change is in the header: the four-player mode[181]

3.7.4 Player record

3.7.4.1 Graalsav1

The first revision is the simplest. The table gives its fields in file order with their types. Rows tagged as inferred have a field whose meaning is not read directly[181].

Order	Type	Field
1	String	level[181]
2	String	nickname[181]
3, 4	Float	x, y[181]
5	Integer	dir[181]
6	Integer	fullhearts[181]
7	Float	hearts[181]
8	Integer	rupees[181]
9	Integer	darts, default 5[181]
10	Integer	bombs, default 10[181]
11	Integer	glovepower, default 2[181]
12	4 × Boolean	flags; the first defaults to true[181]
13	4 × Boolean	flags[181]
14	4 × Integer	colours, the same four values the relay S message sends (→ §2.3.1)[181]

3.7.4.2 Graalsav2 to Graalsav8

The record starts as Graalsav1 (level, nickname, x, y, dir, fullhearts, hearts, rupees, darts, bombs), then[181]:

Field	2	3	4	5	6, 7, 8
Integer glovepower	yes	yes	yes	yes	yes[181]
Integer shieldpower, swordpower	yes	yes	yes	yes	yes[181]
Integer shootpower	n/a	n/a	yes	yes	yes[181]
one Boolean	yes	yes	n/a	n/a	n/a[181]
5 × Integer colours	yes	yes	yes	yes	yes[181]
Integer headset	yes	yes	yes	yes	yes[181]
horse: String image, Integer bushes	n/a	n/a	n/a	yes	yes[181]
carried NPCs: Integer n, then n × (String image, String script)	n/a	n/a	n/a	yes	yes[181]
trailing Boolean	n/a	n/a	n/a	n/a	yes[181]
trailing Integer	n/a	n/a	n/a	n/a	8 only; the 1.1.9 loader reads it only when the magic is 8 or above[181]

The power order in the file is glove, shield, sword, then shoot; the colour count goes from 4 to 5 between Graalsav1 and Graalsav2[181].

3.7.4.3 Graalsav9 to Graalsv11

The player is one string in the network player-props encoding[181]:

```
String  props          concatenation over prop index i: chr(i+32) + the prop's encoded value
[v9]    String horse image, Integer bushes
Integer n, then n × (String image, String script)        carried and inventory NPCs
[v11]   Integer w, then w × String                        NPC weapon names
[v11]   Integer selected weapon index
```

Rule	Detail
sources	`Graalsv11` is from the writer; the `[v9]` and `[v11]` conditions are from the reader[181]
props codec	the same as the generation 2 protocol (→ App. B); a props decoder for generation 2 messages reads these saves[181]

3.7.5 DES-wrapped local files

From 5th Graal the client carries a small DES class and two 8-byte keys, `AXBORG15` and `VERGRAAL`. They protect local files against editing; nothing on the network is encrypted in these builds[181].

3.7.5.1 `.code` files (5th Graal on)

```
24 bytes  DES-ECB(AXBORG15, P)
24 bytes  DES-ECB(VERGRAAL, P)
```

Rule	Detail
files	`guildname.code` (behind the `playeringuild` script check); in 15th Graal to 1.1.9 also `reg.code`, the registration key[181]
P	a 24-byte buffer holding a Pascal short string: a length byte, then up to 23 characters[181]
reader	decrypts both halves, requires them to be equal, and returns the string; a short file or differing halves read as `###`[181]
key broadcast	in 15th Graal to 1.1.9 the registration key is also sent as relay type 8, so two players with the same key are noticed (→ §2.3.1)[181]

3.7.5.2 `.gsave` (15th Graal on)

```
u32le     n = length of the padded plaintext
n bytes   DES-ECB(AXBORG15, plaintext)
n bytes   DES-ECB(VERGRAAL, plaintext)
```

Rule	Detail
plaintext	the `TWriter` stream of §3.7.1, padded to a multiple of 8 with random bytes[181]
loader	decrypts both copies, compares them byte for byte, and parses only if they match[181]
DES variant	the class takes a second constructor argument (7) whose role is not pinned down; the 2.05 DES is textbook DES-ECB with each key byte bit-reversed, and the same handling is probably at work here, so a round trip should try the keys bit-reversed first[181]
open	no real save has been round-tripped[181]

3.7.5.3 Saved passwords (1.3 test to 2.22)

The client saves account passwords in the registry under `HKEY_CURRENT_USER\Software\Graal\Client`, as `password` and as `<account>:pass` per account[183].

Rule	Detail
cipher	DES-ECB with standard tables, each key byte bit-reversed before the key schedule; the same routine as the `.gsave` DES[183]
key	one fixed 8-byte key, the same in every copy of the client, so a saved value is as good as plain text to anyone who can read the registry or an export of it[183]
builds	the same key and form from 1.3 test (Nov 1999) to 2.05; a value saved by 1.31 or 1.32 in the lab decrypts the same way[183]
encrypt	cut the plaintext to 23 bytes; build 24 bytes: the length, the text, random padding; encrypt as three DES blocks; write each byte as two characters `(b >> 4) + 0x20`, `(b & 0x0F) + 0x20`[183]
decrypt	input under 48 characters, or a first decrypted byte of 0 or 24 and above, gives an empty string[183]
same text, new value	the random padding makes every save of the same password differ[183]
opting out	"Don't save the passwords" in More Options deletes the values when the client closes[183]
2.22	a 49-character value that starts with a marker character is decrypted with a second key held by the options form; any other value with the first key, so old saves still load; new saves use the marker and the second key[184]
2.22 account name	stored with the byte transform of the login (→ §2.3.3)[184]
other uses	2.05 builds its hidden listserver host the same way, from a key and a 48-character text assembled one character at a time (→ §6.1.1); the web-level cache uses the same DES with per-block keys (→ §3.8)[184,183]
what it protects	nothing; these schemes only keep the strings out of a casual string dump[184]

3.7.6 Server account file (GRACC001)

From 2.00 the save is the server's account file; the client receives it as its login burst (→ §2.3)[182].

Rule	Detail
file	`accounts/<name>.txt`; text, first line `GRACC001`, then `KEY  value` lines, CRLF on write[182]
player keys	NAME, NICK (at most 223 characters), COMMUNITYNAME, LEVEL, X, Y, Z, MAXHP, HP, RUPEES, ANI, ARROWS, BOMBS, GLOVEP, SHIELDP, SWORDP, BOWP, BOW, HEAD, BODY, SWORD, SHIELD, COLORS (5 comma-separated), SPRITE, STATUS, MP, AP, APCOUNTER, ONSECS, IP, LANGUAGE, KILLS, DEATHS, RATING, DEVIATION, LASTSPARTIME[182]
repeated keys	FLAG (`name=value`), ATTR1 to ATTR30, WEAPON, CHEST (an `x:y:level` style string)[182]
admin keys	BANNED, BANREASON, BANLENGTH, COMMENTS, EMAIL, LOCALRIGHTS, IPRANGE, LOADONLY, FOLDERRIGHT (repeated), LASTFOLDER[182]
new account	the server clones `accounts/defaultaccount.txt`, applies `startlevel`, `startx` and `starty` from the server options if set, and saves the new file[182]
guest	load-only, with a random `pc:` community name[182]
saving	on disconnect for accounts that are not load-only, and every 300 s while connected; not saved if the listserver login failed[182]

The client's other local files from 2.00 on are its config files (→ §3.9) and the web-level cache (→ §3.8)[182].

3.8 Web level cache (GWEBL001)

The web-level cache is a local copy of a server level, keyed by the level's modification time, so the client can skip downloading a level it already has. The format was traced in the 2.05 rev 2 reader and writer and confirmed on 144 real cache files from a 2.22 install: every one decrypts, passes the checksum, parses and re-encrypts byte for byte[185].

3.8.1 When it is used

The web-level cache keeps copies of server levels on disk. Each row gives one event in its use and what the client does[185].

Event	Behaviour
write	when the server sends the level's 5-byte modification time (`PLO_LEVELMODTIME`, id 39), the client saves the level it holds with that time[185]
read	when the client enters a level and a cache file exists, it builds the level from the file[185]
benefit	the server then presumably only needs to send levels that have changed[185]
mismatch	a file whose header does not match the current server is deleted[185]

3.8.2 Path

```
<exe dir>\weblevels\<server host>\<level>-<server port>.code
```

Rule	Detail
`<level>`	the level's file name, lowercased, with any path removed[185]
fallback	the reader also tries `<level>.code` without the port[185]
example	`weblevels\<public-host>\bank.nw-14803.code`[185]

3.8.3 Encrypted file

```
s32le      N           padded plaintext length + 8
N-8 bytes  ciphertext  DES-ECB, one fresh key per 8-byte block
8 bytes    checksum    bytewise sum (mod 256) of all plaintext blocks, per byte position
```

Rule	Detail
layout	as above[185]
padding	the plaintext is padded with `0x0A` to a multiple of 8[185]
seed	`RandSeed = 0x04A80B38`; then for each byte c of `<level>`, `RandSeed = RandSeed * c` with 32-bit wrap[185]
block key	8 calls to Delphi `Random(256)`, drawn fresh for every block in file order[185]
`Random(n)`	`RandSeed = RandSeed * 0x08088405 + 1`, then `(RandSeed * n) >> 32`, unsigned[185]
DES key bytes	bit-reversed, as in the 2.05 DES elsewhere (→ §3.7.5.2)[185]
strength	the encryption needs only the level name; a wrong name shows up as a checksum mismatch[185]

3.8.4 Plaintext

```
"GWEBL001"
GCHAR len + server host          must equal the connected host
GCHAR hi, GCHAR lo               server signature, (hi-32)*128 + (lo-32)
5 bytes                          level modification time, 5 × 7-bit GCHAR, big-endian
GCHAR len + level name           must equal <level> above
level body                       GR-V1.03 (2.05), or GR-V1.03 / GR-V1.04 (2.31)
```

Rule	Detail
layout	as above[185]
signature check	passes when the stored value equals the value the server sent in `PLO_SIGNATURE` (id 25), or when the stored value is 1 and the server sent 73, GServer's usual value[185]
test files	all 144 hold 73[185]
modification time	decoded as a 35-bit integer it is Unix time; the test files decode to dates a little before their file timestamps (`bank.nw` decodes to 1247727798, 16 July 2009, and the file is dated 26 July 2009)[185]

3.8.5 Level body as cached

The body is an ordinary GR-V level (→ §3.1.2), but the writer fills only some sections[185]:

Section	Cached
board	yes, the 13-bit run-length stream[185]
links	yes, `dest x y w h newx newy` lines, then `#`[185]
baddies	no; terminator only, `FF FF FF 0A`[185]
NPCs	no; empty, `#`[185]
chests	no; empty, `#`[185]
signs	yes, `x+32, y+32, sign text, \n`[185]
heights	2.31 only: a `HEIGHTS` block when the level has a height map, which also switches the magic to `GR-V1.04` (→ §3.1.2.7)[185]

3.8.6 Tool

The repository has a tool for these cache files. The table lists its commands and what each produces[185].

Command	Effect
`python3 tools/file-formats/gwebl.py FILE.code`	summary[185]
… `--json`	full parse[185]
… `--raw plain.bin`	the decrypted stream[185]
module	`encrypt_stream` re-creates files byte for byte; also `read_board`, `write_board` and `parse_grv` for plain `.graal` data[185]

3.8.7 Unpacked Windows 6.03 weapon-cache acceptance

In the exact inspected unpacked Windows 6.03 input, the reader first clears success. Missing key or cache flag other than 1 returns empty. At least eight bytes and GRESCR01 pass only preliminary checks. After stripping the magic, nonzero expected CRC is compared with CRC32 of the stored body **before transformation**; mismatch returns empty. Expected CRC zero skips comparison and may return transformed bytes, but success remains false. Returned bytes and cache acceptance are separate.[186]

The inspected weapon caller constructs its weapon-cache.code path and branches on success: false dispatches request logical ID 158; true hands returned script bytes onward. For exactly eight magic bytes the stored body is empty; its CRC with the inspected zero initialization is zero. Expected CRC zero therefore leaves success false, and any nonzero expected CRC mismatches. Both follow the request branch if surrounding caller gates permit it.[186]

Independently authored header-only, wrong-flag and missing-key vectors have static-derived expectations, not original-client executed results. This does not establish downstream framing, a positive cache, class-cache or all-caller behavior, decoded-container validity, transformation compatibility or GS2 execution. The input is an unpacked analysis image, not an authenticated runnable distribution; no new client/server run occurred. Other builds retain their own contracts.[186]

3.8.8 Exact Linux 6.03 local-file helper

In the exact inspected Linux 6.03r7 x64 local cache helper, the output status starts false and a nonnull key/context plus mode exactly one are required. At least eight bytes and the GRESCR01 prefix pass recognition only. For a nonzero expected CRC, the helper compares CRC-32 over the stored body after the prefix before invoking its inverse-side block helper. Mismatch returns empty, while equality sets status. Expected CRC zero skips this comparison and leaves status false even if transformed bytes are returned. Returned bytes, status and a valid compiled program are separate outcomes. This is retained static file-format evidence, not a newly observed successful cache read. No original-client execution, transformation algorithm, full malformed-file contract or cross-build equivalence is established.[187]

3.8.9 Linux 6.x class cache

On Linux 6.03r7 x64 the class-cache writer transforms the body and then adds the `GRESCR01` prefix, and the reader checks the CRC before the inverse transform. The CRC therefore covers the stored, transformed body. Notes that say this build checks the plaintext after decryption are wrong for this input.[188]

Packet 197 parses its whole body as one metadata list that includes the expected CRC, with comma, quote and escape handling. When that CRC is zero, the class path skips the validator, deletes the cache entry and may load an empty program. When it is non-zero, it must equal the stored body's CRC, and a header-only file's empty body has CRC zero. So a header-only file is never a class cache hit on this path. On a miss the client sends logical 161 with the stored-body CRC as five offset-integer bytes and then the class name; the field is a CRC, not a modification time.[188,188]

Linux 6.01r1 x86 shows the same split: packet 140 carries a descriptor and a body, and packet 197 carries metadata, and both lead to the same class store. That store uses the modern class-code namespace, not the web-level cache in → §3.8.1–§3.8.5. Nothing here was run; fixtures, malformed files and script execution are untested (→ App. E#202).[188]

3.9 Config and dat files

The client keeps its settings in `graalconfig.txt` next to the executable and its login details in the registry. This chapter gives both as 2.05 rev 2 reads and writes them, checked against a real `graalconfig.txt` written by 2.22, which uses the same keys in the same order[189]. The table files `arrays.dat` and `arrays2.dat` are in → §3.5.

> Status: draft. 2.05 is traced and 2.22 compared; the keys of 3.x and later clients are unmeasured.

3.9.1 Where settings live

A 2.05 client keeps its settings in two places. Each row gives one store, where it is and what it holds[189].

Store	Location	Holds
`graalconfig.txt`	next to the executable	every option on the More Options and Player List Options dialogs, the head, language, font, joystick and key bindings[189]
registry	`HKEY_CURRENT_USER\Software\Graal\Client`	`nickname`, `accountname`, `password`, and one `<account>:pass` value per account[189]

3.9.2 graalconfig.txt file rules

The rules below say how `graalconfig.txt` is read and written. The writing rule matters most: the whole file is rewritten, so a key the client does not know is lost[189].

Rule	Detail
form	a plain key=value list; key lookup is case-insensitive and order does not matter on read[189]
read	once, when the options form is created at startup[189]
written	in full every time the More Options or Player List Options dialog closes, every key in the fixed order of §3.9.3; hand-added keys are lost on the next save[189]
missing file	the built-in defaults stay, and `language` is set to the English name of the Windows user locale (for example `English` or `German`)[189]
boolean	`true` when the lowercased value is exactly `true`, anything else `false`; written as `true` or `false`[189]
missing boolean	reads as `false`, not as the default; a partial file therefore switches off `musicon`, `soundfxon`, `saveimages`, `nohtmlimages` and `joystickon`[189]
integer	converted as decimal; a conversion error, an empty value or a missing key keeps the default[189]
string	used only when non-empty[189]

3.9.3 Keys in file order (2.05)

MO is the More Options dialog and PL the Player List Options dialog; labels are the dialog captions as the client spells them.[189]

Key	Type	Default	Dialog label	Effect
`waitvertical`	bool	false	MO "Wait for vertical blank (slower but smoother)"	vsync on flip[189]
`musicon`	bool	true	MO "Music support"	turning it off stops the music player[189]
`usewinamp`	bool	false	MO "Use WinAmp for music"	music goes through Winamp (→ §3.10.3)[189]
`soundfxon`	bool	true	MO "Sound FX's enabled"	sound effects[189]
`reversestereo`	bool	false	MO "Reverse stereo channels"	swaps channels[189]
`saveimages`	bool	true	MO "Save downloaded images (to \levels\webgifs)"	turning it on writes the images already in memory to disk[189]
`dontsavepms`	bool	false	PL "Don't save PMs/Profiles"	as labelled[189]

Key	Type	Default	Dialog label	Effect
compabilitydraw	bool	false	MO "Draw in compatibility mode (very slow)"	spelling as in the client[189]
nodecompressionerror	bool	false	MO "Don't show decompression errors"	as labelled[189]
head1, then head	string	Fung Si Yun	none	read in that order, so head wins; only head= is written; the 2.22 file still holds the default[189]
joystickon	bool	true	MO "Joystick support"	as labelled[189]
nodirectinput	bool	false	MO "Don't use DirectInput"	as labelled[189]
joyselected (also reads joyselected1)	string	(none)	Controls frame	(none), Joystick1 or Joystick2[189]
keyset_<action>_joystickbutton	string	per action	Controls frame	one of Joy_Up, Joy_Down, Joy_Left, Joy_Right, Button1 to Button12[189]
keyset_<action>_key0 to _key2	string	per action	Controls frame	three key slots per action, by key name (Up, Num 8, Space, Tab …); empty means unbound[189]
language	string	English	Connect frame language box	interface language[189]
font	string	Tempus Sans ITC	MO "Font:"	chat and name font; the 2.22 folder ships this font file[189]
fontsize	int	24	MO "Size:"	font size[189]
fixedport	bool	false	MO "Use fixed port for data packages:"	fixes the UDP port[189]
fixedudpport	int	14899	MO port box	used when fixedport is true[189]
noudp	bool	false	MO "Don't use UDP for data packages"	changing the UDP options restarts the UDP socket (→ §2.9)[189]
nomassmessages	bool	false	PL "Ignore mass messages"	as labelled[189]
nohtmlinpms	bool	false	PL "Don't use HTML for PMs"	as labelled[189]
limitnicknames	bool	false	MO "Limit the lenght of nicknames to"	as labelled[189]
nicknamelimit	int	50	MO length box	the limit[189]
dontconnectlevels	bool	false	MO "Don't connect levels"	turning it on drops the cached neighbour levels[189]
notoalls	bool	false	MO "Ignore toalls"	turning it on also clears the toall history[189]
nohtmlimages	bool	true	PL "No images in PMs"	as labelled[189]
sortplayersbyname	bool	false	PL "Playerlist: sort by nickname"	as labelled[189]
dontloadlistheads	bool	false	MO "Don't load heads for the playerlist / map"	as labelled[189]
dontsavepasswords	bool	false	MO "Don't save the passwords"	see §3.9.5[189]
dontusedirectx7	bool	false	MO "Don't use DirectX7"	as labelled[189]
fullscreenmode	string	1024x768	MO "Fullscreen mode:"	one of 640x480, 800x600, 1024x768, 1280x1024, 1600x1200; any other value keeps the default[189]

3.9.4 Key bindings

Key bindings sit in the same file. The rules below give where the block goes, the eleven actions and how conflicts between keys are resolved[189].

Rule	Detail
position	the joystick and key block is written right after head=[189]
actions	11, in file order: `up`, `left`, `down`, `right`, `item`, `attack`, `action`, `map`, `switchtochat`, `itemlist`, `pause`[189]
duplicate key	a key name already bound earlier in the file is treated as `-` (unbound), so one key cannot drive two slots[189]
rebinding	binding a key that another slot holds moves it, and the other slot gets the old key[189]
default joystick buttons	in action order: `Joy_Up`, `Joy_Left`, `Joy_Down`, `Joy_Right`, `Button1` to `Button5`, `Button9`, `Button10`; the 2.22 file matches[189]
2.22 default keys	arrows or IJKL, D, S or space, A, M, Tab, Q and P[190]
key names	2.05 holds only the bare prefix `keyset_` as a string, so it probably builds the binding key names at run time[190]

3.9.5 Registry values

The account details live in the registry. Each row gives one value and when it is read and written[189].

Value	Read	Written
`nickname`	at startup	when the dialogs close[189]
`accountname`	at startup; a comma list whose first entry is the current account	when the dialogs close[189]
`password`	at startup, decrypted	when the dialogs close, unless `dontsavepasswords` is true[189]
`<account>:pass`	at startup for the current account, and again when the account box changes	as `password`[189]

Two rules cover how passwords are cleared and how they are encrypted[189].

Rule	Detail
clearing	when More Options closes with "Don't save the passwords" ticked, the client deletes `password` and every value whose name ends in `:pass` from the key[189]
encryption	password values are encrypted with a fixed DES key, the literal that older notes called a password mask[189,190]

3.9.6 Keys by build

The config file grew between 2.05 and 2.22. The table compares the two builds group by group[189].

Key or group	2.05	2.22
the keys of §3.9.3	read and written	present, same order[189]
`automapping`, the `micro*` and `voice*` voice-chat keys, `midivolume`, `mp3volume`, `radiovolume`, `sfxvolume`, `startinfullscreen`, `quickplayerworlds`	n/a	present[190]
`noudp`	default false	`noudp=true` in the compared file[190]
total	about 29 keys in the string table	86 keys in the compared file[190]
`serveraddr`, `loadcolorsfromserver`, `gamemode`	not read or written	unmeasured[189]

The three keys in the last row come from an older public key list[189]. The static survey places them in earlier clients, not later ones: `gamemode` from 1.1 to 1.41, `serveraddr` from 1.21 to 1.39, `loadcolorsfromserver` from 1.21 to NewMain[191].

3.9.6.1 Plain key strings added and dropped, 1998 to 2.31

From 1.38 some keys are built one character at a time, so a build can read a key that is missing here; three-letter fragments such as `ekv` are such keys[191].

Build	Added	Dropped
6th Graal	`joystickon`, `musicon`, `reversestereo`, `soundfxon`, `waitvertical`	[191]

Build	Added	Dropped
8th Graal	saveimages	[191]
1.1	editnickname, gamemode	[191]
1.21	accountname, loadcolorsfromserver, password, serveraddr, serverport	[191]
1.28 beta 2	compabilitydraw, externalchat, language, nodecompressionerror	[191]
1.3 test	font, fontsize	accountname, password[191]
1.31	fixedport, fixedudpport, noudp	editnickname[191]
1.32	nomassmessages	[191]
1.33	nodirectinput	[191]
1.35	nohtmlinpms	[191]
1.36	limitnicknames, nicknamelimit	serverport[191]
1.38		externalchat[191]
1.39	dontconnectlevels, dontloadlistheads, nohtmlimages, notoalls, sortplayersbyname	[191]
1.41 rev 1	dontsavepasswords, fullscreenmode, usewinamp	serveraddr[191]
NewMain 1.42	dontsavepms, dontusedirectx7, head, joyselected	gamemode[191]
2.00		loadcolorsfromserver[191]
2.1 test 2	startinfullscreen	[191]
2.17.1	automapping, the micro* and voice* keys, midivolume, mp3volume, quickplayerworlds, radiovolume, sfxvolume	musicon, soundfxon, usewinamp[191]
2.31	cachefilenames	[191]

3.9.7 Dropped files

A file dragged onto the window is opened by type. The rules below give the gate and what each kind of file becomes[190].

Rule	Detail
gate	a file dragged onto the client window is handled only when an internal flag is set, the same flag that enables the editor drop target[190]
.txt	spawns a script NPC[190]
.gif, .png, .mng	spawn an image NPC[190]
anything else	loaded as a level[190]
joystick	the device is activated when an internal bit is set; the WinMM joystick identifier is stored apart from the main window object[190]

3.10 Image and sound asset conventions

This chapter lists the files a client install carries, the image sheets the engine expects, and which sound and music files each backend plays. How the client positions and mixes a sound in play is in → §4.7.

> Status: draft. The asset tree is from one official install; the 2.22 extension-to-backend dispatch and what 2.05 does after a missing sound arrives are untraced.

3.10.1 Image sheets

The client draws from a few large image sheets. The table gives each sheet's size and role[176].

File	Size	Role
`pics1.png`	2048×512	standard tiles, 4096 ids of 16 px (→ §3.5.1)[176]
`picso.png`	1024×512	offline tileset, sheet type 1[176]
`sprites.png`	128×452	sprite sheet[176]
`letters.png`	—	bitmap font[176]

The tile definition line that selects a sheet is in → §3.5.2.2.

3.10.2 The levels tree

Files on disk under `levels/` in the measured install[176]:

Directory	Files	Contents
`ganis/`	93	86 `.gani` (→ §3.3) plus images[176]
`midis/`	49	music[176]
`heads/`	35	33 png, 2 mng[176]
`weapons/`	33	weapon images, 2 txt[176]
`bomys/`	29	bomy images and ganis[176]
`emoticons/`	29	png and mng[176]
`webgifs/`	26	downloaded images, mostly png (→ §3.9.3 `saveimages`)[176]
`backpals/`	23	background palettes[176]
`bodies/`	20	body images[176]
`images/`	19	other images[176]
`smilies/`	15	—[176]
`baddies/`	10	one sheet per baddy[176]
`shields/`	7	—[176]
`swords/`	6	—[176]
`npcs/`	5	script text[176]
`horses/`	4	—[176]
`hats/`	4	—[176]
`tutorial/`	4	—[176]
`letters/`	2	`blueletters`, `graal2002letters`[176]
`snapshot/`	2	`ss_arrow`, `ss_bomb`[176]
`tiledefs/`	1	the offline tile definition line[176]

The table lists the other top-level items of the client's `levels` tree and points to where each is described[176].

Top-level item	Contents
`maps/`	`bigmap.txt` and minimap material (→ §3.2.5)[176]
`arrays.dat`, `arrays2.dat`, `.sig` files	→ §3.5.3 to §3.5.5[176]

Top-level item	Contents
`bass.dll`, `basswma.dll`, `hvdi.dll`	sound, WMA add-on and voice libraries (→ §3.10.4)[176]
`readme.txt`	documents HawkVoice (LGPL), joystick calibration and the classic controls: F1 options, F4 editor, F5 load, arrows or IJKL walk, S or space strike, A act, M map[176]

3.10.3 Sound and music in 2.05

2.05 plays sound through three back ends. Each row gives what is played and which back end plays it[192].

What	Backend
sound effects (`.wav`)	DelphiX wave list on DirectSound[192]
music, Windows MCI path	an invisible media player component (MCI)[192]
music, Winamp path	a component that remote-controls a running Winamp 1.x or 2.x[192]
music volume	`midiOutSetVolume` (MCI path) or Winamp IPC messages[192]

The rules below give the sound options, how a back end is chosen, and how a sound file is named, found and fetched[192].

Rule	Detail
options	`musicon`, `usewinamp`, `soundfxon`, `reversestereo` (→ §3.9.3)[192]
backend switch	chosen at game start and again whenever More Options closes; switching stops the current track and frees the old backend; turning music off stops the track[192]
loop prefix	a leading + on a file name means looped; it is stripped[192]
missing file	online, the client asks the server for the file by the same path that fetches images; the sound plays on a later trigger once the file has arrived[192]
dispatch	`.wav` → effect path, only when `soundfxon`; any other extension → music path, only when `musicon`; `.mid` is the common case[192]
named effects	fifteen effects are known by name: `arrow`, `axe`, `bomb`, `crush`, `extra`, `fire`, `horse2`, `item`, `lift`, `lift2`, `put`, `steps`, `sword`, `thunder`, `water` (`.wav`)[192]
effect cache	each file is loaded once and cached by name; `stopsound` finds the cached wave by file name, ignoring the path[192]
MCI types	`.mid` goes to the MCI sequencer; other types depend on the MCI drivers installed[192]
MIDI loop	a `.mid` track replays when it ends, so MIDI music loops until `stopmidi` or another track[192]
Winamp types	in Winamp mode music is whatever Winamp's input plugins accept, MP3 included[192]
Winamp start	the component finds the player by window class `Winamp v1.x`; if none is running it reads the Winamp path from `Software\CLASSES\Winamp.File\shell\Play\command` and starts it[192]

The voice cap, positional pan and volume rules and the GS1 sound commands are in → §4.7.

3.10.4 Sound libraries in 2.17, 2.22 and 2.31

From 2.17 the BASS library replaces most of that. The table compares 2.05 with 2.17 to 2.31 area by area. Several rows are judged from imported names, and are tagged as inferred[192].

Area	2.05	2.17, 2.22, 2.31
library	DelphiX, MCI, Winamp	`bass.dll`, imported with the same functions in all three: samples, tracker music, file streams, URL streams, stream length and position, channel attributes and recording; `winmm` is still imported, with MIDI device queries added[192]
effects	DirectSound wave list	BASS samples[192]
music files	MCI (MIDI) or Winamp	BASS streams and tracker modules; MIDI still through MCI[192]
internet radio	n/a	BASS URL streams, `radio` and `radiovolume`, judged from names[192]
Winamp	`usewinamp`	removed: no component, key or window class[192]

Area	2.05	2.17, 2.22, 2.31
volume options	none	`midivolume`, `mp3volume`, `radiovolume`, `sfxvolume`, `voicevolume`[192]
voice chat	n/a	microphone capture through BASS recording, encoded with the `hvdi.dll` HawkVoice codec; an `emoticonmicro.png` icon shows over a talking player[192]
voice options	n/a	`microon`, `microinputdevice`, `microactivatebyvolume`, `microactivationlevel`, `microactivatekey`, `voicecodec`, `voicevolume`[192]
WMA	n/a	2.17.1 loads `basswma.dll` at run time by name and uses its stream functions; 2.22, 2.31 and GraalEditor 2.3.1.1 neither import nor name it, so the shipped DLL is a leftover and WMA music is not supported there[192]
extension dispatch	→ §3.10.3	unmeasured[192]

4 Client behaviour by version

> Status: draft. Each chapter carries its own status line.

4.0 Reading this Part

Graal's game rules run on the client, and they run on one clock. One game tick is 50 ms, and the static survey of every Delphi executable finds that interval wherever a frame timer is present.[10,193] This Part therefore states durations in ticks, and §4.1 fixes how each build drives the tick before any later chapter counts with it. The 5.x and 6.x clients measured on the wire keep 2.05's walk, push, lift and sword timings in ticks, so the counts carry well beyond the Delphi line.[4]

The client is the referee. A victim's own client judges another player's sword from that player's props, applies the damage, and reports its own hearts; the attacker only broadcasts its props.[194] The same holds for bombs and arrows the server relays.[195] A server that wants to reproduce a build's play does not compute combat; it relays the messages each build expects and leaves the rules to the client. Where the authority did move, the move is dated: item drops are rolled by the client up to 1.41 and by the server from NewMain 1.42.[196]

2.05 rev 2 is the reference build. Most rules here were read in full in 2.05 rev 2, then checked against 2.17.1, 2.22 and 2.31, and against 1st Graal and 1.39 where those builds were read.[197] A row names the builds it covers. A rule given for 2.05 is not assumed for 1.39 or for 3.x, and the chapters' status lines say which builds are read in full and which only as timed on the wire. The Torque-era 3.x and 4.x frame loops are still unmeasured (→ App. E#301).[10]

How the Part is laid out. §4.1 gives the tick and the periodic work. §4.2 to §4.5 follow a player through movement, combat, death and the items on the board. §4.6 to §4.9 cover what the player sees and hears, offline play and the editor, and §4.10 the remaining client features, including the integrity checks. §4.11 covers the builds outside the 2.x Delphi line and the code changes inside it. Open behaviour points are App. E#300 to E#323.[198]

Chapters

- → §4.1 Frame loop and timing (`01-loop-timing.md`)
- → §4.2 Movement and collision (`02-movement-collision.md`)
- → §4.3 Combat, damage, hurt and knockback (`03-combat-damage.md`)
- → §4.4 Death clocks and respawn (`04-death-respawn.md`)
- → §4.5 Items, drops and board respawn (`05-items-board.md`)
- → §4.6 HUD, sign panel, chat text and options (`06-hud-ui.md`)
- → §4.7 Sound (`07-sound.md`)
- → §4.8 Offline and training mode (`08-offline.md`)
- → §4.9 The built-in editor (`09-editor.md`)
- → §4.10 Other client features (`10-feature-specs.md`)
- → §4.11 Precursors, side builds and the 4.x clients in play (`11-side-builds-labs.md`)

4.1 Frame loop and timing

This chapter fixes the tick that every count in Part IV uses, how each build drives it, and the periodic work that runs off it. Durations of individual actions are in the chapter for each action (→ §4.1.5).

> Status: draft. The notes cover configured intervals and runtime timing separately; the three early builds in App. E#300 have static configuration and a callback-execution count, not a measured cadence; the 3.x and 4.x Torque-era loops are not in the notes (→ App. E#301).

4.1.1 The tick

Everything in Part IV is counted in ticks. The rules below fix the tick's length and show that it holds from the first Delphi build to 6.x[10].

Rule	Detail
length	one game tick is 50 ms (20 ticks per second)[10,197]
same in every Delphi build	the static survey of all 57 Delphi executables finds a 50 ms frame interval wherever a frame timer is present[193]
tick counter	2.05 and 2.22 keep a per-player tick counter that rises by one every 50 ms whatever the player does[199]
5.x and 6.x	a moving 5.24r4, 5.30r3 or 6.03r7 client sends its position every 50 ms; their walk, push, lift and sword timings equal 2.05's in ticks[4]
Graal3D	driven by a 50 ms DelphiX timer[10]

4.1.2 What drives the tick, by build

The tick is the same everywhere, but what drives it changes. Each row gives a group of builds and the timer or loop that runs its frame[193].

Builds	Driver
1st Graal to 4th Graal	a DelphiX timer on the main form, interval 50 ms[193]
4th Graal alpha, 5th Graal, 5th Graal alpha	main adventure form declares an enabled DelphiX timer, interval 50 ms, active-only disabled, maximum lag ten, with a statically bound callback. A debugger count shows the callback runs (61, 61 and 60 hits in 3 s); actual cadence and runtime idle installation remain unmeasured → App. E#300[200,200]
6th Graal (beta5) to 1.41 rev1, and Graal3D	a DelphiX timer on the main form, interval 50 ms[193]
NewMain 1.42 to 2.31 (every 2.0x, NewWorld betas 1 and 2, the 2.1 tests, 2.17.1, 2.22, 2.31)	no form timer; the frame runs from the application's idle handler behind a 50 ms gate[193,10]
2.31	in addition, the main form carries a 50 ms timer that runs the 64-tick integrity cycle (→ §4.1.3)[201]
3.x, 4.x	unmeasured[10]
Windows 4.04r2	still unmeasured, but the core's clocks are traced: `getTicks` is `GetTickCount` and `timevar2` scales it by 0.001; legacy `timeout` counts service passes (set ×20 + 0.0001 floored, read ×0.05); a GS2-style pass advances its script clock 0.05 per pass and drives `istimeout` and scheduled events → App. E#301[202]
Mac PowerPC 4.04r2, 4.20r6 and v5	a Carbon event-loop timer calls the engine entry once; it runs an initialize method, then an update method repeatedly while it returns non-zero, then shutdown, and quits the event loop. The pump sleeps 1 ms when idle. The update rate is not measured[36]

4.1.2.1 The frame function (idle-driven builds)

In the idle-driven builds one function runs each frame. The rules below give its order and two consequences a remake must copy: no catch-up after a stall, and input read once per tick[203].

Rule	Detail
order	after the 50 ms gate opens, one frame handles clock skew, network traffic, then drawing[203]

Rule	Detail
no catch-up	a stall costs one frame; the client does not run extra ticks to catch up, and ignores any step count its pacer supplies[203]
input sampling	key state is read once per tick: a key released and pressed again inside one tick is not seen as a new press, and a tap shorter than a tick can be lost[198]

4.1.3 Periodic work

Some work runs every tick and some on a longer cycle. Each row gives the period, the work and the builds it was read in[198].

Period	Work	Builds
every tick	the player state machine steps (walk, push, hurt, death and other counters in §4.2 to §4.4)	all measured builds[198]
every tick	the tile under the player is checked for water and lava, except while the sword swings, while riding, and when the player is not under local control	2.05[204]
every tick	the probe point for links is taken and each link of the level is tested (→ §4.2.11)	2.05[205]
64 ticks (3.2 s)	a counter modulo 64 runs the integrity cycle: tick 1 a debugger probe and a 5000 ms timer armed, tick 10 the first tamper-check word, tick 20 a scan of top-level window titles and classes against a blacklist, tick 30 the second word, tick 31 `PLI_TAMPERCHECK` sent (→ §2.4, §4.10)	2.31 only; 2.17.1 and 2.22 have no such timer[201]
about 5.1 s	a one-byte id 24 message (→ §2.4); tying it to the 5000 ms timer above is an inference	2.31; 2.22 only while another player is in the level[201]
5.0 s	the server's `PLO_NEWWORLDTIME`, seen alongside the client's id 24	2.22 against a public server[199]

4.1.4 Movement updates on the wire

How often a moving client reports itself; the message layouts are in → §2.4.

Builds	What is sent while a key is held
1.1.9 (relay)	a `D` line of type P on the level connection every 100 ms: x and y in half-tiles, then frame·4 + direction (→ §2.3)[206]
1.21	one id 2 (player props) message every 100 ms with X, Y and SPRITE[206]
1.28 beta 2	UDP id 0 to each peer every 100 ms: X, Y, SPRITE, STATUS and CARRYSPRITE (→ §2.9)[206]
1.3 test against GServer13	one id 2 message every 100 ms with X and SPRITE; speed 0.5 tile per tick[206,207]
2.22	the X prop only on whole tiles (every other tick), plus the final half-tile when the player stops; the gani prop switches to the walk gani with the first step and back to idle 100 ms after the last[199]
2.22	a script that changes a client flag every tick makes the client send that flag every tick, one message each, uncoalesced[199]
NewWorld beta	the idle gani is re-sent about every 100 ms, because the control NPC sets it each tick; 2.05 sends it only now and then[208]
5.24r4, 5.30r3, 6.03r7	the position (x2/y2) every 50 ms[4]
2.05	unmeasured as a rate; a hit sends one props message with CURPOWER, GANI and SPRITE in the hit's tick, then the knockback positions[209]

4.1.5 Durations in ticks

The canonical place of each tick count:

- → §4.2.5 blocked steps before the push pose (18) and the push cycle (20)
- → §4.2.6 the ledge jump (8 ticks)
- → §4.2.8 the pull cycle (18 ticks)
- → §4.2.10 swimming and the lava burn (101 ticks)
- → §4.3 sword, spin, hurt and knockback

- → §4.4 the death timer

4.1.6 Clocks in the later engines

These are exact-build static and debugger readings of how a later engine advances its clock. None of them is a measured frame cadence (→ App. E#301).[210]

Linux 4.20r6: clock updates and the `isgraal3d` predicate

Six unchanged-startup updater returns in Linux 4.20r6 have a zero branch byte. The initial zero clock is seeded within the observer's wall-clock interval; the next five values each match binary-double previous value plus 0.05. Debugger pauses prevent inferring a 50 ms frame period, 20 Hz rendering, movement timing or sleep latency.[210]

Registration directly binds global read-only `isgraal3d` to the getter for that predicate byte, whose ELF default is zero. Three direct-reference writer owners are identified; the set-to-one branch follows a `3` input prefix. Indirect/alias/module writes remain possible. With the byte zero and the clock nonzero, it adds 0.05 per call; if the clock is zero, or the byte is nonzero, it refreshes from the wall-clock helper. The probe observed one seed and five increments; it does not prove that the clock can never be reset or reseeded. No live nonzero branch, 3D connection or rendering is observed. Inputs/signatures/executable/mode byte stay unchanged; the untruncated capture retains executable equality and verifies cleanup, with no server. E#301 remains open for frame-driver and walk measurements. Separate callback adapters provide no complete ABI, module activation or Windows reader result.[210,210,210]

Windows x64 v7, input 701750: managed and native update

The separately identified Windows x64 recovered inputs show managed Update calling native QuattroUpdate and conditional FixedUpdate forwarding. QuattroUpdate increments a counter, calls a wrapper bound to GetTickCount and stores the return as 32 bits; an inferred void pseudocode type does not override caller dataflow. Guarded QuattroFixedUpdate iterates a registered collection and dispatches onFixedUpdate with an empty argument string. Update3D wraps a one-byte state getter rather than establishing scene simulation. These are selected-range static observations: no measured cadence, wrap/catch-up policy, complete collection/lifecycle/indirect-call semantics, build-614/v6 equivalence, authenticated release provenance or independent original-to-recovered derivation. Exact image identities remain owner-held. E#301’s older-client driver question remains open.[211]

4.2 Movement and collision

This chapter gives the walking step, the wall test, what blocks a point, the blocked-step counter and the actions that hang off it (push pose, ledge jump, pushing blocks and NPCs), grab and pull, water and lava, link following, and the gates that stop movement. Tile type values and the `arrays.dat` lists are defined in → §3.5; this chapter only says how movement uses them. Units are tiles and 50 ms ticks (→ §4.1.1). Facings are 0 up, 1 left, 2 down, 3 right.

> Status: draft. The rule is read in full for 2.05, 2.17.1, 2.22 and 2.31; the exact 1st Graal geometry is now read statically, while 1.39 still lacks its numeric tables; 3.x and 4.x are not in the notes. Open points are App. E#302 to E#309.

4.2.1 The step

Walking is built from one step per key per tick. The rules below define that step: order, facing, vector and units[212].

Rule	Detail
one key, one step	each arrow key held is one step per tick; with several held, the steps run in the order up, left, down, right, each from the position the previous one left; a diagonal is two steps[212]
facing	the step first sets the player's facing to its direction[212]
step vector	(0, -0.5), (-0.5, 0), (0, 0.5), (0.5, 0) by facing, times a speed factor[212]
positions	doubles, in tiles[212]
first tick	the first tick after the key goes down already moves the player[198]
release	letting go switches to `idle` frame 0 in the next tick; there is no slow-down and no finishing step[198]
walk animation	`walk` advances one frame per tick, 8 frames, 8-tick cycle[198]

4.2.1.1 Speed factors

The step vector is scaled by a speed factor. Each row gives one factor and the state that selects it[213].

Factor	When	Builds
1.0	on foot, carrying or not (carrying does not slow the walk)	2.05[213,198]
1.2	a high speed state	2.05[213]
0.2	hidden under a carried object: 0.1 tile per tick	1.39, 1.41, 2.05, 2.22[198,214]
about 0.2	hidden: 1.5 px per tick, steps of 2 and 1 px alternating	5.x, 6.x[4]
0.4, 0.8	exist in the walking code; not tied to states (→ App. E#302)	2.05 to 2.31[215]
swimming	the same 0.5 tile per tick as walking	1.39, 1.41, 2.05, 2.22, 5.x, 6.x[198,4]

4.2.1.2 Walking speed by build

Walking speed was timed in live runs across the builds. Each row gives the step per tick and the resulting speed. The column stays the same from 1998 on[216].

Build	Step per tick	Speed
1st Graal	0.5	10 tiles/s[216]
1.0 (10th)	0.5	10 tiles/s[216]
1.1 (15th)	0.5	10 tiles/s[216]
1.32	0.5	10 tiles/s[216]
1.39	0.5	10 tiles/s[198]
1.41 rev1	0.5; a second copy of y moves in 0.25 steps (the view or the drawn position)	10 tiles/s[216]
2.00	unmeasured	unmeasured[216]
2.05 rev2	0.5	10 tiles/s[216]
2.17.1, 2.31	0.5 (read from the walking routine)	10 tiles/s[212]
2.22	0.5 (live and from the walking routine)	10 tiles/s[199]

Build	Step per tick	Speed
1.3 test (GServer13)	0.5	10 tiles/s[207]
5.24r4, 5.30r3, 6.03r7	8 px (0.5) per tick	10 tiles/s[4]

4.2.2 The wall test

4.2.2.1 Probe points (2.05 to 2.31)

Six points per facing, offsets from the sprite's top-left, read at the position the step would reach. They are numbered 0 to 5 in the order given:

Facing	Fixed coordinate	The six points along the edge
up	y + 1.0	x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475[212]
left	x + 0.5	y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975[212]
down	y + 3.0	x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475[212]
right	x + 2.5	y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975[212]

Whether a step is allowed is decided by testing points on the player's body. The rules below give where those points lie, how a point maps to a tile, and what lies off the board[213].

Rule	Detail
body box	the points lie on the leading edge of a two-tile square, x + 0.5 to x + 2.5, y + 1 to y + 3[213]
point to tile	`floor(v + 0.0001)`: a point on a tile line reads the tile that starts there; a value a hair under a whole number reads the tile above[212]
off the board	a tile outside 0 to 63 is the neighbour level's on a gmap, or a fixed tile outside a gmap[212]
off the board, default tileset	the wall test checks bounds first and calls off-board a wall; the type tests skip the bounds check and read id 632, which is in no list of the 2.22 `arrays.dat`[217]

4.2.2.2 The same test in earlier builds

Earlier builds were checked against the 2.05 test. Each row gives one build and how far its rule is known; several are live readings rather than reads of the code[218].

Build	Rule
1st Graal	exact-input static step/probe/nudge and stepped-axis snapping rules are in → §4.2.2.3; cadence and exceptional gates remain unmeasured[218]
1.1 (15th)	after bumping furniture x carried a 0.025 offset (26.925) while walking, which fits a nudge rule of the 2.05 kind; not read[216]
1.39	six probe offsets per facing step; the numeric tables are unread (→ App. E#307)[214]
1.39	walking into a character NPC's side nudged the player along it at 0.125 tile per tick, and `playertouchsme` did not fire on that contact[198]
2.05, 2.17.1, 2.22, 2.31	as §4.2.2.1 to §4.2.4; the four agree on every table and constant[212]

4.2.2.3 Exact 1st Graal geometry

1st Graal's movement was read exactly from its executable. These rules give its coordinate system, camera and starting position. The input-scope row says which file they apply to[218].

Rule	Detail
input scope	exact Windows x86 executable identified in the source note; no 1.39 or later-build equivalence is established[218]
position	separate little-endian doubles for world X/Y; integer parts select cells of a 64×64 board, supporting tile units[218]
camera	ordinary 640-pixel-wide viewport: x + 1.5 − 20 horizontally, y + 2 − 14 vertically, clamped to the board; screen displacement is not world displacement[218]
initial position	new-game/restart stores (24.5, 30) via the coordinate setters; second local player adds 12 to X[218]

Offsets below are ordered probes 0–5 from the candidate after adding the facing step; true means blocked[218].

Facing	Candidate delta	Ordered offsets
up	(0, −0.5)	(0.6, 1.05), (1.4, 1.05), (1.6, 1.05), (2.4, 1.05), (2.525, 1.05), (0.475, 1.05)[218]
left	(−0.5, 0)	(0.55, 1.1), (0.55, 1.9), (0.55, 2.1), (0.55, 2.9), (0.55, 3.025), (0.55, 0.975)[218]
down	(0, +0.5)	(0.6, 2.95), (1.4, 2.95), (1.6, 2.95), (2.4, 2.95), (2.525, 2.95), (0.475, 2.95)[218]
right	(+0.5, 0)	(2.45, 1.1), (2.45, 1.9), (2.45, 2.1), (2.45, 2.9), (2.45, 3.025), (2.45, 0.975)[218]

The second table gives what 1st Graal does when a step is blocked: the sideways nudge, its conditions, and the refusal[218].

Rule	Detail
nudge axis/size	up/down: ±0.125 X; left/right: ±0.125 Y[218]
positive nudge	probe 0 blocked and probes 2, 3, 4 clear[218]
negative nudge	otherwise, probes 0 and 1 clear, probe 3 blocked and probe 5 clear; no new six-probe collection follows either nudge[218]
refusal	any of probes 0–3 blocked selects pushing and undoes the original facing delta; sideways nudge remains and clears the push count, otherwise the count increments[218]
snapping convention	for nonnegative coordinates, trunc is truncation toward zero; the helpers explicitly select it; all snaps affect the stepped axis[218]
up/down snap	when Y has a positive fractional part: up sets Y to trunc(Y); down sets Y to trunc(Y + 1)[218]
left/right snap	when X + 0.5 has a positive fractional part: left sets X to trunc(X + 0.5) − 0.5; right sets X to trunc(X + 1.5) − 0.5[218]
counters	count ≥ 38 becomes 18; at 18 a passing ledge predicate selects an eight-tick hop; a count-28 helper can suppress position commits, with full semantics untraced[218]
limits	static constants/branches only; no new runtime cadence, all-state-gate or complete object-list validation; 1.39 tables remain open → App. E#307[218]

4.2.3 What blocks a point

Tested in this order; the first hit blocks:

#	Test
1	a list of objects with two flags and a player-sized body box, checked only when a client option is on (unidentified → App. E#303)[212]
2	the level's NPCs that have a blocking shape or image[212]
3	a list of 1.5 × 2 boxes with an enabled flag, x + 0.25 to x + 1.75, y to y + 2 (unidentified → App. E#303)[212]
4	chests: each covers x to x + 2, y to y + 2, edges included, opened or not[212]
5	the tile, which blocks unless it is in the passable list or the liquid list of `arrays.dat` (arrays 0 and 1, → §3.5.3)[212]

4.2.3.1 How the tile is answered, by tileset

What counts as a wall depends on the tileset type. Each row gives one type and the rule that answers the wall test for it[217].

Tileset type	Wall
0 (default, `pics1.png`)	not in array 0 and not in array 1; looked up by binary search of each sorted list, with no 4096-entry table built[217]
1, 2 (new kind)	a fixed rule on the tile id: id 32; rows 26 and 27 with column 2 or more; rows 30 to 47, 84 to 95, 116 to 127; row 128 or more with `row / 16` odd (row = id / 16, column = id % 16)[217]
5	no board rule; only NPC shapes have types[217]
any	a `setshape2` NPC shape the player stands on answers first[217]

Consequences on the default tileset:

Rule	Detail
lava is liquid	the liquid list holds lava too, so lava is walkable and the swim test is true on it[217]
deep water	a tile in none of arrays 0 to 3 is a wall: the dark deep-water tile 3644 blocks, and `push` comes on the 18th refused step[217,209]

Rule	Detail
shallow water	a tile only in the passable list is walked on, not swum in[217]
several lists	a tile can be in several lists; in the 2.22 file 6 of the 133 throw-through tiles are passable, and the 9 lava tiles that are not liquid are all passable, so they are walked on and still burn[217]
thrown objects	a thrown object stops on a tile that is not type 20 and that the wall test calls a wall[217]
scripts	`onwall(x, y)` is this wall test; `onwater(x, y)` is the type 11 test, true on lava; on the classic board that test is the id range 64 ≤ id < 288, matching the type 11 step of the rule above[219]; `tiletype(x, y)` returns -1 on the default tileset, 1 on type 5, and the fixed rule on types 1 and 2 (an NPC shape's type wins when above 1); 2.00 to 2.04 carry the same three names, their code not compared[217]

4.2.4 Refused steps, nudges and snaps

When a step is refused the client does more than stop. The steps below are applied in order: a corner nudge, taking the step back, and a snap flush to the wall[212].

Step	Rule
decision	points 0 to 3 decide: all four free keeps the step, any blocked refuses it[212]
1. corner nudge	if point 0 is blocked and points 2, 3 and 4 are free, add +0.1 across the edge (+x for up and down, +y for left and right); else if points 0 and 1 are free, point 3 is blocked and point 5 is free, add -0.1; the nudge is added to the stepped position[212]
2. take the step back	subtract the step vector times the speed factor; the nudge stays[212]
3. flush snap	along the facing only, when the coordinate has a fraction above 0: up `y = floor(y)`; left `x = floor(x + 0.5) - 0.5`; down `y = floor(y + 1)`; right `x = floor(x + 1.5) - 0.5`; there is no wall test on the snapped position[212]
parent	on a platform object or with a parent, the snaps are relative to the parent's position[212]
after every step	accepted or refused, y snaps to `floor(y + 0.0625)` if its fraction is under 0.0625 or over 0.9375, and x the same on `x + 0.5` (`x = floor(x + 0.5 + 0.0625) - 0.5`); then the position is stored, so a refused step still moves the player by the nudge and the snaps[212]
state	a refused step puts the player in the pushing mode (mode 1)[213,220]

Consequences of the rule:

Rule	Detail
left and up	the probe is inside the body box, so the body stops with its edge on the wall's tile line without a snap[212]
right and down	the probe is the tile line one tile beyond the body; the step that would put the body's edge on a wall's line is refused, and the flush snap moves the body that half-tile[212]
gaps	a two-wide body fits a two-tile gap and not a one-tile gap[212]
corners	pressing against the end of a wall slides the body round it at 0.1 tile per tick, ten ticks per tile[212]
sliding along a wall	holding Down into a wall and adding Right walks right at full speed with `walk`; the push pose ends in that tick[198]
live confirmation	the stopping positions are read from the binary and not walked on a live 2.22 (→ App. E#304)[215]

4.2.5 The blocked-step counter and the push pose

Refused steps are counted, and the count drives the push pose, ledge jumps and pushing. The rules below give how the count rises and resets and when the pose changes[220].

Rule	Detail
counting	a step counts when it is refused and no corner nudge was applied; a free step or a nudge sets the count to 0, and so does the first refused step after a free one[220]
shared counter	the blocked-step count and the grab's pull cycle share one per-player counter[220]
running in place	a refused step still advances the walk frame; the gani stays `walk` until the push pose or a jump takes over[209]

Rule	Detail
push pose	push replaces walk on the 18th blocked tick (0.9 s); at 38 the count goes back to 18 and the gani restarts at frame 0[198]
push frames	5 frames × 4 ticks = the 20-tick wrap, so the pose loops seamlessly[198]
sliding resets	after a slide the count goes on from 1, so the full 18 ticks pass again before push; pressing into the wall again after letting go starts at 1[198]
on the wire	push is sprite steps 14 + frame (SPRITE 11, 12, 13, 11, 12)[220]

By build:

Build	Push at	Wrap	Push frame length
1st Graal	count 18 (the ledge test); push frames while the count is above 17 and not carrying	above 37 back to 18	unmeasured[221]
1.39	18	38 to 18	about 2 ticks (video)[198,214]
1.41	18	38 to 18	4 ticks (push.gani)[198]
2.05 rev 2	18	38 to 18	4 ticks[198]
2.22	the tick after 18 blocked ticks	37 back to 18	5 × 4 ticks, looping; a lab read of the frame field showed it cycling 0 to 3 each tick (→ App. E#305)[198,199]
5.24r4, 5.30r3, 6.03r7	0.85 to 0.90 s after the walk started	unmeasured	unmeasured[4]

4.2.6 Ledge jumps

At certain counts the client checks for a ledge to jump down. The rules below give when that check runs, where it looks and where the player lands[220].

Rule	Detail
when	the ledge test runs at blocked count 18, and again every 20 steps (the wrap)[220]
test point	from the player's position by facing up, left, down, right: (1.5, 0.5), (0, 2), (1.5, 3.5), (3, 2)[220]
ledge tile	the tile there must be a ledge tile for that facing: array 8 of arrays.dat on the default tileset (→ §3.5.3), or rows 30 and 31, columns 8 to 15, on a tileset of type 1 or 2[220]
landing	from (x + 1.5, y + 2): (0, -5), (-6.5, 3), (0, 7), (6.5, 3); the landing point must pass the walking test (not a wall, not a chest), or the player just keeps pushing[220]
sound	jump.wav at (x + 1.5, y + 2) when the jump starts[220]
flight	8 ticks from the position after the refused step, the first in the same tick as the start; keys are not read until landing; the blocked-step counter counts down 7 to 0 as the jump timer[220,209]
gani	none of its own: the walk (or push) gani stays; no sprite of its own on the wire[209,220]
mode	mode 2 for the 8 ticks[220]

The flight, per tick i = 0 to 7:

Facing	Position
up	y0 - [1.5, 2.7, 3.8, 4.5, 5.0, 5.2, 5.2, 5.0][i]; live from y0 = 14: 12.5, 11.3, 10.2, 9.5, 9.0, 8.8, 8.8, 9.0[220,209]
down	y0 + [-0.2, 0.2, 0.5, 1.5, 2.8, 4.0, 5.4, 7.0][i][220]
left, right	x0 -/+ [1, 2, 3, 3.8, 4.5, 5.2, 5.8, 6.5][i], y0 + [-0.2, -0.5, 0, 0.5, 1.1, 1.7, 2.3, 3.0][i][220]

Two builds were compared with that rule. Each row gives what differs from 2.05[221].

Build	Difference
1st Graal	at count 18 a passing ledge test sets the hop mode and the counter to 8; the hop follows a facing table for 8 ticks[221]
2.22, 2.31	the same counts, probe points, flight curve and tables as 2.05; jump.wav plays only while the current gani is a built-in name (→ §4.2.12)[220]

4.2.7 Pushing blocks and NPCs

Pushing a block is tried at a later count than the ledge test. The rules below give when it is tried, which tile counts as a block, and where the block may move[220].

Rule	Detail
when	at count 28 (checked before the step is counted, so on the 29th blocked step), and every 20 after[220]
trigger	the try first fires the trigger action pushed at the ledge test point, with the facing as its parameter (→ §4.2.9)[220]
block	the tile under that point is compared with the four tiles of the pushable block (array 16, entry 0, → §3.5.3); only that one tile is compared, and its place in the block gives the block's top-left[220]
target cells	from the block's top-left: up (0, -1) (1, -1); left (-1, 0) (-1, 1); down (0, 2) (1, 2); right (2, 0) (2, 1); none may be a wall, a chest, water, a chair or the lower half of a bed[220]
move	each old cell first takes the tile two cells ahead, then the block is written one cell ahead, so the row the block leaves gets the floor that was in front of it[220]
after	online, the 4 × 4 area round the old spot goes to the server as a board modify; the player moves one tile in the facing; `stonemove.wav` plays[220]

4.2.7.1 Pushed NPCs

When there is no block, the push may move an NPC instead. The rules below give how the NPC is found, the permission it needs and how its path is checked[220].

Rule	Detail
lookup	with no block found, the level's NPC hit test at the point: a character NPC in x + 0.5 to x + 2.5, y + 1 to y + 3; a `setshape2` NPC by its shape's tile types; a gani NPC by its animation; any other by its image box `[x, x + w/16) × [y, y + h/16)`, where a transparent pixel is a miss for plain image NPCs[220]
permission	the NPC needs `canbepushed` (GS1 command 44, cleared by 45, → §8.2)[220]
edge test	with the NPC left out of its own test, every point along the edge it moves into is checked, one per tile, `t` while `t < x + w` (or `y + h`): up (t, y - 1), left (x - 1, t), down (t, y + h), right (x + w, t); sizes in pixels / 16, character and gani NPCs count as 48 × 48[220]
blocked-point test for NPCs	other players (online only), blocking NPCs, two level object lists, and wall tiles; water does not stop an NPC[220]
move	the NPC moves one tile, the player one tile the same way, `waspushed` fires on the NPC (`actionpushed` has already fired), and the block sound plays[220]
water	a character NPC that crosses a water edge gets the splash and switches between `swim` and `idle`[220]
server	nothing is sent for the NPC move; the `pushed` trigger action is the only message[220]
2.22	the NPC lookup also searches the neighbouring map parts on a gmap[220]

4.2.8 Grab and pull

Grabbing and pulling use the A key. The rules below give what A does first, what can be grabbed and how holding and pulling work[220].

Rule	Detail
A key order	throw what is carried; else lift (→ §4.5); else grab[220]
grab test	by facing (1.5, 0.9), (0, 2), (1.5, 3.1), (3, 2): a wall tile, or an NPC that blocks; nothing is grabbed when another player stands there[220]
grab on NPCs	2.05 grabs a blocking NPC only when a per-NPC flag (most likely the character-shape flag) is clear; 2.22 and 2.31 grab any blocking NPC, and search the neighbouring map parts[220]
holding	the grab lasts while A is held; arrows do not move or turn the player; mode 5[220]
pull cycle	the arrow away from the wall sets the counter to 2 and starts `pull`; the counter runs 2 to 19 and back to 2 (18 ticks); a pull is tried whenever the arrow is held at 19; letting go of any arrow sets the counter to 1[220]
pull try	fires `pulled` one tile out from the body, (1.5, 0), (-0.5, 2), (1.5, 4), (3.5, 2) by facing, with the opposite facing as parameter, then reads the tile at the grab point[220]
pullable block	array 16, entry 1 (→ §3.5.3); the two cells behind the player must be free, from the block's top-left: up (0, 4) (1, 4); left (4, 0) (4, 1); down (0, -3) (1, -3); right (-3, 0) (-3, 1); the block moves one tile towards the player (each old cell first takes the tile two cells behind), the player steps back one tile, and `lift.wav` plays, or `lift2.wav` by a player flag (→ App. E#306)[220]

Rule	Detail
pulled NPCs	with no pullable block, an NPC with canbepulled (command 46, cleared by 47) is pulled: the two cells behind the player must be free from the whole-tile grab cell (up (0, 3) (1, 3); left (3, 0) (3, 1); down (0, -3) (1, -3); right (-3, 0) (-3, 1)), then the edge facing the player is checked (up (t, y + h), left (x + w, t), down (t, y - 1), right (x - 1, t)); NPC and player move one tile back and waspulled fires; nothing goes to the server[220]
futile pull	on anything else (a wall, a rock too heavy for the glove, a push block) nothing moves; pulled still fires every 18 ticks and pull keeps playing[220]
on the wire	grab is sprite step 19 (SPRITE 14); pull steps 20 + frame (SPRITE 15, 16, 17)[220]

Pose timing by build:

Build	Pull frames	Arrow released during a pull	A released after a pull
1.41	unmeasured	back to grab	grab stays until the next action[198]
2.05 rev 2	3 frames × 2 ticks	counter to 1; the pose stays pull, animating	pull loops until the next key moves the player[198]
2.22	3 × 2 (pull.gani)	stays pull, looping	as 2.05[198]
5.x, 6.x	unmeasured	unmeasured	unmeasured; grab and pull start in the next tick, the stone stays put, dust is drawn at the feet[4]

4.2.9 The push and pull trigger actions

Each push and pull try is announced as a trigger action. The rules below give who sees it locally, how it goes to the server and peers, and how often it repeats[220].

Rule	Detail
local	NPCs at the point see actionpushed or actionpulled[220]
server	PLI_TRIGGERACTION (38, → §2.4): GINT 0 (no NPC), GCHAR x*2, GCHAR y*2, the action name, then , and the parameter as a decimal string when there is one (pushed,0)[220]
peers	2.05 also sends it to UDP peers as UDP id 26 (→ §2.9)[220]
cadence, 2.05	one try on the 29th blocked step, then every 20 steps (push); every 18 ticks (pull)[220,220]
cadence, 5.x and 6.x	pushed,0 about 0.55 s after push starts, then every 1.0 s; pulled,1 0.85 s into a pull[4]

4.2.10 Water and lava

The water check runs every tick at one point on the body. The rules below give when it runs and what happens on entering and leaving the water[204].

Rule	Detail
check	every tick, except while the sword swings, while riding, and when the player is not under local control[204]
entry point	the switch to swim happens on the tick the point (x + 1.5, y + 2) is on a liquid tile; leaving happens when it is off, and walk comes back at frame 0[198]
entering	when not already swimming: a splash (break effect 5) at the player's position; two more splashes when the player was not walking (a warp or a knock); then swim mode with swim, or lava when the tile type is 12[204]
leaving	one splash, then back to standing[204]
splash sound	water.wav at (x + 1, y + 1) from the effect's anchor; every other break effect plays crush.wav[204]
swim mode	mode 10: set on a type 11 tile; cannot be hurt; the sword needs no horse[222]
status bit	while crossing water 1.28 beta 2 sets STATUS bit 0x04 and sends prop 18[206]

Swimming by build:

Build	Standing still in water	Swim frames
1.39	only the head and a water ring are drawn; the ring keeps alternating, the head stays still	n/a (no ganis)[198]

Build	Standing still in water	Swim frames
1.41	`swim` frame resets to 0 and freezes	unmeasured[198]
2.05 rev 2	keeps animating; does not reset when the player turns	8 frames × 2 ticks (16-tick stroke)[198]
2.22	keeps animating; the frame resets to 0 at the stop, then runs on	8 × 2[198]
5.x, 6.x	unmeasured; `swim` from the first water step, only the head with a ripple, sideways strokes throw splashes	unmeasured[4]

4.2.10.1 Lava burn

Lava hurts on a slow cycle. The rules below give the test point, the counter and which tiles burn[217].

Rule	Detail
test	once per player tick, one point, the position plus (1.5, 2), against the lava list (array 3) only; the order of the two offsets is inferred[217]
counter	on lava a counter runs 0 to 100 and wraps to 0 on the next tick: one burn every 101 ticks; a tick off lava sets it to 0[209,217]
burn	half a heart through the hurt routine, with a flag that skips the usual "can be hurt" checks (→ §4.3); reading that call as the hurt routine is inferred[217]
tiles	the bright-red lava tile 2882 is swum in (`lava gani`) and burns; the dark-red lava floor 2958 is walked on and burns[209]

4.2.11 Following links

Links are tested every tick. The rules below give how the client builds the neighbour table and tests links in order[205].

Rule	Detail
neighbours	on entering a level while a big map is loaded, the client fills a 3 × 3 table of level names round it from the big map's text grid (→ §3.2.5); cells off the grid stay empty; the step is skipped under an unidentified flag (→ App. E#308)[205]
probe	each tick, the player's position plus a per-facing offset (offsets unread → App. E#308)[205]
order	each link of the level in order; the first that fires is taken; offline the client loads the level, online it asks the server[205]

On a big map the client adds four edge links of its own. Each row gives one edge, the condition that fires it and where the player arrives[205].

Link	Fires when	Arrival
west edge: x 0, width 1, destination = west neighbour	probe x ≤ 0 facing left, or player x < -2.5, or (neighbour not loaded) probe in the rectangle	x = player x + 64, y unchanged[205]
east edge: x 63, width 1, destination = east neighbour	probe x ≥ 64 facing right, or player x > 63.5, or (not loaded) probe in the rectangle	x = player x - 64[205]
north edge: y 0, height 1, destination = north neighbour	probe y ≤ 0 facing up, or player y < -3.0, or (not loaded) probe in the rectangle	y = player y + 64[205]
south edge: y 63, height 1, destination = south neighbour	probe y ≥ 64 facing down, or player y > 63.0, or (not loaded) probe in the rectangle	y = player y - 64[205]
any other link	probe in the rectangle; a probe within 1 tile of an edge (x or y ≤ 1 or ≥ 63) counts only while the player faces out across that edge	the link's new-x and new-y, evaluated as expressions; a missing or -1 value means `playerx` / `playery`[205]

The rules below complete the link behaviour: the seam between neighbours, the case with no big map, and the editor's defaults[205].

Rule	Detail
seam	an edge link between big-map neighbours keeps the player's exact offset across the seam (east off x 63.6 arrives at x -0.4), whatever its new-x says[205]
no big map	without a big map, or for a level that is not the neighbour, the link's own coordinates apply, and `playerx` on an east-edge link arrives on the east edge of the destination[205]

Rule	Detail
editor defaults	a new edge link gets new-x `0` on the east edge, new-y `0` on the south edge, new-x `61` on the west edge and new-y `61` on the north edge; the other coordinate stays `playerx` / `playery`[205]
level-entry messages	what the client sends on each level entry is in → §2.4[201]

4.2.12 Gates that stop movement (2.22, 2.31)

2.22 and 2.31 add flags that freeze the player. The rules below list those gates and note that 2.05 has none[220].

Rule	Detail
stop gates	walking, the ledge test, the A key and other gameplay code do nothing while either of two server-set flags is on; server messages 176 and 177 set them (→ §2.5.8); reading them as the full-stop messages is an inference[220]
connection flag	a further flag on the server connection also stops gani changes, footsteps and the A key; its meaning was not found[220]
built-in gani gate	`jump.wav`, `stonemove.wav`, the pull sound and footsteps play only while the current gani is a built-in name (→ §4.10); the jump, push and pull still happen under a custom gani; sound names are bare (`jump.wav`) instead of under `sounds/`[220]
2.05	none of these gates[220]

4.3 Combat, damage, hurt and knockback

This chapter gives the sword swing and the spin, who judges a hit, the gates a hit must pass, the damage each source takes, the shared hurt routine, hurt mode, knockback and the hurt pose. The combat messages themselves are in → §2.4 and → §2.5.5; the STATUS bits are in → §2.6.2.1. Death is → §4.4.

> Status: draft. Read in full for 2.00 to 2.05 rev 2, 2.17.1, 2.22 and 2.31; 1st Graal and 1.39 for the main rules; 3.x to 6.x only as timed on the wire. Open points are App. E#309 to E#315.

4.3.1 Who judges a hit

In Graal the victim's client, not the server, decides whether it was hit. The rules below set out that model and the units hearts are counted in[194].

Rule	Detail
the victim decides	the victim's own client judges another player's swing from that player's props (position, direction, sprite) and applies the damage; the attacker only broadcasts its props[194,223]
bombs and arrows	the victim's client also judges a server-sent bomb (PLO_BOMBADD, 11), explosion (36) or shot (PLO_ARROWADD, 19), whoever owns it; no hit message is sent, only the victim's props message (CURPOWER and the `hurt` gani) and the knockback positions; a client's own bomb hurts its owner[195,224]
hearts on the wire	the victim reports its own hearts: in the hit's tick one props message with CURPOWER, the `hurt` gani and one more prop; nothing from the server asks for it, and the server's own CURPOWER is not echoed[209,195]
units	CURPOWER is in half-hearts, MAXPOWER in hearts; the client clamps CURPOWER to MAXPOWER × 2 before subtracting[195]
killer report	when a hit by another player's sword, piece or shot, or a UDP hurt, leaves the hearts at 0 or below, the victim sends PLI_CLAIMPKER (14) with the attacker's id (→ §2.4)[225,224]
integrity	hearts, AP and the heart cap are stored obfuscated twice and cross-checked; a mismatch raises a "memory problems" error (→ §4.10)[225]

4.3.2 The sword swing

The sword swing is a mode with its own gani and timing. The rules below give when it can start, where the tip probe lies, and how long it lasts[194].

Rule	Detail
preconditions	weapons allowed by the level or server, sword power not 0, not swimming without a horse, not already in the spin mode[194]
mode and gani	mode 6; plays `sword` (`ridesword` on a horse) and the sword sound, then checks for a cuttable bush at the sword tip[194,204]
tip probe	one tile beyond the middle of the leading edge: up (x + 1.5, y), left (x - 0.5, y + 2), down (x + 1.5, y + 4), right (x + 3.5, y + 2)[213]
timing	`sword` in the next tick, 5 frames of one tick, then `idle`; the bush is cut on the first swing frame; the player does not move while it plays, even with an arrow held; walking resumes the tick after, from walk frame 0[198]
repeat	holding S does not swing again; each swing needs a new press; holding S gives no charge[198]
on the wire	the swing's sprite steps are 9 to 13 (SPRITE 6 to 10)[194]
no sword	a player with sword power 0 started from the editor still plays `sword` and its sound, but no blade is drawn and nothing is cut[198]
carrying	the sword key with an object carried throws it instead (the same throw as the action key, → §4.5); no swing precondition is checked on that path[226,198]
`hidesword n`	sets a counter of n seconds (clamped 0 to 30, × 20 ticks) that counts down each tick; while it is 0 or more STATUS carries bit 0x20; it does not stop the local swing, only refuses the hit at the victim's end[227]
`hideplayer n`	the same clamp and scale on its own counter, which drives STATUS bit 0x02 (hidden)[227]

By build:

Build	Swing	Cut
1st Graal	hit frames 6 to 10	unmeasured[221]
1.39	counter 1 to 6 (5 ticks), no gani	cuts the bush[198]
1.41	`sword` 5 ticks	on the first frame[198]
2.05 rev 2	`sword` 5 × 1 tick	on the first frame[198]
2.22	`sword`	the 2 × 2 leaves board change goes to the server in the same frame as the sword gani; idle follows 240 ms later[199]
5.24r4, 5.30r3, 6.03r7	`sword` 5 ticks, `idle` 0.25 s later	the cut (board change) in the same tick as the swing starts[4]

4.3.2.1 The spin attack

A second press during a swing turns it into a spin. The rules below give the timing window, the length of the spin, and what it ignores[198].

Rule	Detail
start	a second S press while `sword` is still playing, with the has-spin status (STATUS bit 0x40); not a hold-to-charge[198,228]
new press	S must be up for at least one tick in between; a press on swing frame 2, 3 or 4 works; a press after the swing has ended starts a normal swing[198]
timing	the swing is cut off: `spin` frame 0 shows in the tick after the second press; 13 frames of one tick (650 ms), then `idle`; the facing at the start is stored[198]
mode	mode 13; a press during the spin is ignored and queues nothing[198,222]
wire	the client sends `GANI=spin` once at the start and `GANI=idle` at the end, and asks for `spin.gani` the first time[198]
2.22	the same: a second press on frame 3 cut the swing off, `spin` ran 13 × 1 tick, a third press was ignored[198]
1.39, 1.41	unmeasured (offline neither build could be given the has-spin status)[198]
what it hits	which bushes the spin cuts and whether it hits players or NPCs: unmeasured[198]

4.3.3 The sword hit test (victim's side)

A swing hits the local player only when every gate passes. In 2.05 rev 2 order:

#	Gate
1	the attacker's player id is known[229]
2	the attacker is on sprite steps 9 to 13 (SPRITE 6 to 10)[230]
3	the victim can be hurt: not in hurt mode, not dead, not swimming, hearts above 0, and normal play or a sparring-zone level[231]
4	the victim's AP is not exactly 100[230]
5	the level has not run `noplayerkilling`; when the local player's own attack is refused by it, the client prints `nopkzone`[231]
6	the attacker has not already been judged in this second of the day (a per-attacker stamp, stored before the guild test); its place in the sword path is `disputed → App. E#310`[229]
7	guild: the first bracketed text in the attacker's nickname (a `(`, then the first `)` after it with at least one character between) does not occur anywhere in the victim's nickname, case-sensitive[231]
8	the attacker's STATUS has neither 0x02 (hidden) nor 0x20 (sword hidden)[231]
9	the attacker is not kicking: 2.05 by the animation name `kick`, 2.17.1 and later by an object id of 3[230]
10	reach box: the victim's centre 0 to 4 tiles ahead of the attacker's centre along its facing (both ends count) and strictly under 2 tiles to the side; centres are (x + 1.5, y + 2); skipped while the victim is in hurt mode[194,223]

By build:

Rule	1st Graal	1.39	2.00	2.01	2.02	2.03	2.04 = 2.05 rev 2; 2.17.1, 2.22, 2.31
reach box	up to 4 ahead, sideways under 2; the 0 and 4 edges unread	0 to 4 ahead, sideways under 2	0 to 4, both ends; under 2	=	=	=	=[221,214,223]
tested only in	the normal life-state	unmeasured	play state exactly 0 (its own victim test; no paused/editor test)	the shared can-be-hurt test	shared	shared	shared[221,223]
kick test	unmeasured	unmeasured	absent	by animation name	=	=	by name (2.05); object id 3 (2.17.1 on)[223,230]
no-kill flag	unmeasured	unmeasured	on the player, cleared on level load	on the player	on the player	on the player	on the level, read through a tamper-checked byte[223]
`playerhurt` event on a hit	n/a	unmeasured	absent	present	present	present	present[223]
props path: per-player link field equal	n/a	unmeasured	no	no	no	no	yes (→ §4.3.6)[223]

4.3.4 Sword damage

Sword damage comes from one prop. The rules below give the formula, how the prop is encoded and what the formula leaves out[232].

Rule	Detail
formula	half-hearts taken = the attacker's sword power as the victim last received it in SWORDPOWER (prop 8), limited to -20 to 20; a power-3 sword takes 1.5 hearts[232]
prop 8 on the wire	power 0 sends byte 0 (no image); power 1 to 4 with the matching default image (`sword1` to `sword4`) sends the power itself; anything else sends power + 30 (10 to 50) followed by the image name; the receiver reads a byte below 10 as the power and otherwise byte - 30 (→ §2.6)[232]
negative power	heals, but only at AP 40 or more; hearts stop at the maximum[232]
not in the formula	the victim's shield power, item levels and the attacker's AP were not read into it[233]

The damage store changed twice. Each row gives a group of builds, how they store the received power and what a negative power does[221].

Build	Sword power store	Negative power
1st Graal	fixed half a heart; the sword does not grow	n/a[221]
2.00, 2.01, 2.02	the received power as is, in a 6-bit slot (0 to 63)	cannot occur: it wraps in the store (inferred from the mask)[223]
2.03 to 2.31	power + 20, limited to -20 to 20	heals at AP 40 or more[223,232]

4.3.5 Other damage sources (2.05)

Every source calls the same hurt routine (→ §4.3.7) with a power in hearts and a knockback direction.

Source	Hearts	Hit test
explosion piece, type 1 (`putbomb 1`, `putexplosion`, the blast of a fireblast or nuke)	0.5	the piece's centre, offset by one tile, less than one tile from the player's centre (x + 1.5, y + 2) on both axes; the lab pattern is `disputed → App. E#311`[224]
explosion piece, type 2 (`putbomb 2`, the super bomb)	0.5	as type 1[224]
explosion piece, type 3 (`putbomb 3`, the jolt bomb)	1	as type 1[224]
arrow (`shootarrow`)	0.5	the shot box against the player's box (→ §4.3.5.2)[224]
fireball (`shootfireball`)	1	as an arrow[224]
fireblast (`shootfireblast`)	0.5, then a blast of reach 2, type 1	as an arrow[224]
nuke (`shootnuke`)	1, then a blast of reach 2, type 1	as an arrow[224]
thrown object (`waspelt`)	1, or 0.5 when the player has exactly 1 heart	lands within 2 tiles of (x + 0.5, y + 1) on both axes[224]
baddy touch	0.5, or 1 for the golden warrior (type 7)	the player's centre within one tile of the baddy's point on both axes; the baddy may say its "hurts the player" verse[224]
lava	0.5 every 101 ticks (→ §4.2.10.1)	the point (x + 1.5, y + 2) on lava[209]
`PLO_HURTPLAYER` (40)	its power byte - 32 in half-hearts, capped at one heart; 0 gives hurt mode and knockback with no hearts	→ §4.3.6[232,195]

Swords are not the only source of damage. The rules below cover ownership, overlapping hits, and baddy touch damage[224].

Rule	Detail
ownership	a bomb, blast, arrow or thrown object can belong to another player; online, a piece counts as a player's when it has an owner still in the player list; a player's source does not hurt in a level that ran `noplayerkilling`; a source with no owner (a baddy's, the level's, or one whose owner has left) always hurts[224,224]
one blast, one hit	overlapping pieces take hearts once, because the second finds the player in hurt mode[224]
1.39 baddy touch	0.5 hearts (1 for type 7), within 1 tile on both axes of the baddy centre[214]
2.22 baddies	touch as above, judged by each client for its own player only, with the 2 s hurt lock-out spacing the hits; a player's sword hits a baddy inside a box that depends on the player's facing, and types 0, 1, 2 and 7 block with a shield when they face the swing (knockback 0.8 instead of 2.0); online, a client that is not the leader reports its hit with `PLI_BADDYHURT` (16)[133]
1st Graal	sword and baddy touch half a heart; baddy centre (x + 1, y + 1); two projectile types (sprites 65 and 261) half a heart, the others one heart; an arrow (sprites 107 to 110) flying opposite the player's facing, with the player's sprite index below 6, is blocked[221]

4.3.5.1 Bombs and blasts

Bombs and blasts are built from one command and a cross-shaped blast. The rules below give the powers, the shape and the explosion command[224].

Rule	Detail
`putbomb power,x,y`	powers 1 to 3 are the bomb, super bomb and jolt bomb of the built-in weapons (inferred from the weapon scripts); a local fuse is 60 ticks; water under (x + 1, y + 1) makes a splash and no bomb[224,224]
going off	power 3 plays `thunder.wav`, any other `bomb.wav`; power 2 makes five type-2 blasts (four of reach 2 at the diagonal corners 2 tiles out, one of reach 4 in the middle); any other power one blast of reach 2 with the bomb's power as its type[224]
blast shape	a cross: a centre piece and n pieces in each direction (n at most 15); the last piece of an arm uses the end picture; each piece lives 11 ticks and may hit on any of them[224,224]
`putexplosion radius,x,y`	one blast of type 1 with that reach[224]
knockback	along the piece's arm[224]
another player's bomb (11)	at least 4 bytes; power `v & 7`, bit 8 marks it as the sender's; the fuse byte is in ticks (the sender's copy is 5 ticks shorter than a local `putbomb`); put in the sender's level; nothing is sent back[224]
another player's explosion (36)	at least 4 bytes; power `v & 7`, 0 read as 1 and above 3 read as 3; one cross, never the super-bomb split; one of two sounds by whether the power is 3 (that they are `thunder.wav` and `bomb.wav` is inferred)[224]

4.3.5.2 Shots

Shots fly one tile per tick. The rules below cover local and remote shots, their flight and their hit box[224].

Rule	Detail
local shots	the four shoot commands make a flying object of type 0 (arrow), 1 (fireball), 2 (fireblast) or 3 (nuke) at speed 1 and play `arrow.wav`[224]
another player's shot (19)	at least 5 bytes: x×2, y×2, flags, sprite, type (`byte - 33`); flags `& 3` direction, 4 first removes a matching shot already held, 8 marks it as the sender's; arrows use sprites 107 + direction[224]
flight	1 tile per tick; past -2 or at 64 and beyond it moves to the neighbouring level of the loaded gmap block, or is gone[224]
hit box	size from the sprite table entry v (array 24): width `((v >> 6) & 63) / 16`, height `(v & 63) / 16`; hits when `x + w > px + 0.5`, `x < px + 2.5`, `y + h > py + 1`, `y < py + 3`[224]
skipped	while the shot bounces back, for a player's shot (one's own, or one received in a `noplayerkilling` level), or when the player cannot be hurt[224]
shield	the player faces the shot, shield power above 0, the shot is type 0 or 1, and the animation is one of two built-in names (inferred `idle` and `walk`); power 1: an arrow turns back for 3 ticks with a sound, any other sprite is gone; power 2 or more: the shot is sent back the other way and re-sent as a new shot[224]
walls	whether arrows and fireballs pass walls is open (→ App. E#313)[224]
lab	an injected shot hurt only when its last byte was `0x23` or more; meaning open (→ App. E#312)[195]

4.3.6 Hits by message

Two server messages deliver hits. The table compares them rule by rule: their payload, the gates they pass and the range they use[195].

Rule	`PLO_HURTPLAYER` (40)	`PLO_HITOBJECTS` (46)
payload	attacker id, dx, dy, power; an optional trailing GINT3 NPC id	power, then the hit point's two bytes (half-tiles)[195,229]
gates 1 to 6 of §4.3.3	yes, without the attacker status test	yes[229]
link field	the attacker's layer/parent link integer equals the victim's (new players start at 0; a horse does not change it)	as 40[229,142]
range	squared distance between the two X/Y positions at most 100 (lab: hits at 100.0, none at 110.25)	the player's centre (x + 1.5, y + 2) strictly under 2 tiles from the hit point on each axis[195,229]
power	`byte - 32` half-hearts, capped at 2	capped at one heart[195]
knockback	`(byte - 0x60) / 16` per axis, normalised to the 5-tile push; a zero vector goes 5 tiles east	the victim's centre minus the hit point[195,229]
NPC id present	the hit is refused completely (no damage, no knockback); with only 2 trailing bytes it still hits	unmeasured[195]
unknown attacker	refused	refused[195]

Many of the hit gates arrived in 2.01. Each row gives one gate and whether each group of builds has it[223].

Gate	2.00	2.01, 2.02, 2.03	2.04 = 2.05 rev 2
id 46 handled	no	yes	yes[223]
AP 100 immune	no	yes	yes[223]
no-kill flag	no	yes	yes[223]
once per attacker per second	no	yes	yes[223]
guild test	yes	yes	yes[223]
link field equal	no	no	yes[223]
range and power cap	10 tiles, one heart	=	=[223]

4.3.7 The hurt routine and hurt mode

Every accepted hit goes through one hurt routine. The rules below give its input, when it refuses, what happens to hearts and the mode it sets[232].

Rule	Detail
input	a power in hearts and a knockback vector[232]
refused	while the player cannot be hurt: in hurt mode (3), swimming (10), dead, or in the editor or paused outside a sparring zone; lava's call skips these checks[225,217]
hearts	the heart change is minus the power; a heal (negative power) applies only at AP 40 or more; hearts at 0 or below start the death (→ §4.4)[232]
mode	hurt mode (3) with the counter at 40, for every accepted hit, including power 0 and an ignored heal[233]
event	the script event `playerhurt` fires[225]
countdown	each tick the counter drops by one; at 0 the mode returns to normal; no new hit for the whole 40 ticks (2 s)[229,198]
arrows during hurt	held arrows are ignored while the counter is above 29; from the 12th tick after the hit (counter 28) a held arrow walks again and `walk` replaces `hurt`[198]
carrying	1.39: a positive-power hit may force a drop of the carried object; 1st Graal: the carry is dropped[214,221]

4.3.7.1 Knockback

A hit pushes the player back. The rules below give the vector, the steps and what happens when the way is blocked[229].

Rule	Detail
vector	stored scaled to one tile; a zero vector becomes one tile east[229]
steps	while the counter reads 39 down to 35 after its decrement, the player moves one tile per tick along the vector: five tiles in 250 ms on open ground[229]
blocked	after each step twelve body-box points are tested with the walking rules (→ §4.2.3); if any is blocked the step is undone and tried again the next tick[229]
inside a blocking NPC	every step is refused and the player does not move[198]
sword direction	the victim's centre minus the attacker's point (halved before normalising, where 2.00 and 2.02 keep it)[229,223]
live	a touch NPC moved the player from (29.5, 48) to (28.79, 52.95), 5.0 tiles, along the line from the NPC image centre to the player's centre[198]

The twelve points, offsets from (x, y), three per edge of a box 0.6 to 2.4 across and 1.1 to 2.9 down, centred on (x + 1.5, y + 2):

Point	x offset	y offset
1	0.6	1.1[234]
2	1.5	1.1[234]
3	2.4	1.1[234]
4	0.6	1.1[234]
5	0.6	2.0[234]
6	0.6	2.9[234]
7	0.6	2.9[234]
8	1.5	2.9[234]
9	2.4	2.9[234]
10	2.4	1.1[234]
11	2.4	2.0[234]
12	2.4	2.9[234]

4.3.7.2 Hurt mode by build

Hurt mode was measured across builds. Each row gives one build's lock-out, knockback, pose and blinking. The 1.39 knockback is disputed between the binary and a live run[221].

Build	Lock-out	Knockback	Hurt pose	Blinking
1st Graal	40 ticks; walking refused while the counter is 30 or more	5 steps; baddy and projectile hits an axis unit step from the attacker's facing, a sword hit the centre-to-centre delta; no zero-vector rule	the hurt body while the counter is above 30	odd counters draw a single frame[221]
1.39	40 ticks; a hit is also refused in mode 7	5 steps from the binary; 4 tiles live: `disputed → App. E#309`	about 10 ticks, the back with the arms out	after the pose, every other tick to 40[214,198]
1.41	40	5 steps	`hurt` 31 ticks	from the hit, every other tick[198]
2.05 rev 2	40	5 steps	`hurt` 31 ticks (`hurt.gani` 1 frame, WAIT 30), keeping the facing; 12 ticks when an arrow is held	the body drawn on alternate ticks, only the shadow on the off ticks, for all 40 ticks, after `hurt` has given way[198]
2.22	40	4.9 tiles on the same test	12 ticks with an arrow held	unmeasured[198]
5.24r4, 5.30r3, 6.03r7	unmeasured	5 tiles in 5 ticks	`idle` 1.54 to 1.55 s after the hit	blinks[4]

4.4 Death clocks and respawn

This chapter gives what the client does when its hearts reach zero, the death timer, the dead bit on the wire, the respawn key, the hearts and the destination after respawn, and the level-entry save that picks the destination. What the server does on the dead bit is in → §5. Damage is in → §4.3.

> Status: draft. Read for 1st Graal, 1.39, 2.00 to 2.05 rev 2, 2.17.1, 2.22 and 2.31; timed live for 2.05 and 2.22. Open points are App. E#314 and E#315.

4.4.1 Life-states

The client keeps a life-state alongside its mode. Each row gives one value and what it means; the death and respawn rules below depend on it[235].

Value	Life-state
0	playing[235]
1	editor or pause[235]
2	dead[235,214]
4	a state in which any key restarts, with no timer test (1st Graal, 1.39); not named[221,214]

4.4.2 The death sequence (2.05)

Death in 2.05 runs as a short fixed sequence. The steps below are in order, from the trigger to the dead pose[225].

Step	Rule
trigger	any change that leaves hearts at 0 or below starts the sequence once; a per-frame check also forces the dead state whenever hearts are 0 or below[225]
1	the death timer starts at 0 and counts up one per tick; it stops at 30 (1.5 s)[225]
2	the life-state becomes dead, the mode returns to 0 and the player faces down[225]
3	the dead gani plays and the script event `playerdies` fires[225]
4	controls are ignored until the timer passes 27 (1.35 s)[225]
5	STATUS (prop 18) carries the dead bit 0x08 only while the life-state is dead and the timer is at least 28, so other players see the death about 1.4 s late[225]
6	no dead sprite goes on the wire: SPRITE shows idle plus direction[225]

On the wire:

Rule	Detail
server CURPOWER 0	the client at once sends the dead gani and no CURPOWER; 27 ticks later STATUS 0x1C (dead bit set)[209]
2.22	dead gani and one more prop, no CURPOWER; then STATUS 28; the board goes black and an "End" scroll in the sign style appears under the player[195,199]
offline	burning to 0 hearts offline shows the End death scroll[209]
baddies	a leader client stops sending baddy props the moment it dies[199]

4.4.3 Respawn

There is no automatic respawn. The rules below give the key that ends death and where the player comes back[225].

Rule	Detail
key	once the timer has passed 27, any movement or action key respawns the player; there is no timer-only respawn, no respawn message and no server permission step[225,199]
destination	the last saved level-entry point (level, x, y, direction; → §4.4.4)[225]
nothing saved, online	x 30.5, y 30.0, facing down, in the current level[197,214]

Rule	Detail
nothing saved, offline	the default start level `tutorial_b-4.graal` (axes of its start position unread)[197,236]
online warp	the client re-sends the level at the destination, so the server sees an ordinary level warp, not a "died" message; it does not wait for a reply[225,209]
2.05 frame	`PLI_LEVELWARP` (0) to the same level at the login point, and in the same frame CURPOWER, GANI `idle`, X, Y and STATUS with the dead bit cleared[209]
2.22 frame	the level warp, `PLI_TAMPERCHECK`, any script flags, then props with CURPOWER (full hearts), GANI `idle`, X, Y, SPRITE and STATUS; the server echoes the position and the hearts[199]
dead bit	cleared when the life-state returns to playing[225]
repeat	a server that answers every respawn with CURPOWER 0 puts the client into a death loop[209]

4.4.3.1 Hearts after respawn

The hearts a player gets back depend on AP. Each row gives an AP band and the hearts restored, the same in 2.05 and 2.22[225].

AP	Hearts (2.05, 2.22)
below 20	3[225,195]
20 to 39	min(5, max hearts)[225,195]
40 or more	max hearts[225,195]
MAXPOWER 0	2.05 still reports CURPOWER 6 (3 hearts): the respawn hearts are not capped at a MAXPOWER of 0[209]

4.4.3.2 By build

Death was read in four builds. Each row is one part of the sequence, with how each build does it, so the columns can be compared across[221].

Rule	1st Graal	1.39	2.05 rev 2	2.22
on death	hearts clamped to 0, counter and mode cleared, dead state; the relay object is notified; no inventory spill, no sound	hearts clamped to 0, counter 0, mode normal, dead state; `dead.wav`; the inventory partly spilled as floor items (rupees in 100/30/5/1 bands, arrows and bombs alike)	→ §4.4.2	→ §4.4.2[221,214]
timer	death clears to 0; dead-state updater increments while below 30, then stops; sprite/key predicates remain above 27; elapsed duration not newly measured	counts 0 to 30 and stops	0 to 30	unmeasured[221,237,214]
drawing	the board replaced by a filled rectangle and a text panel; the body uses sprite 157 once the counter is above 27	death sprites cycle between counters 4 and 27; a cleanup runs above 29	the dead gani	the "End" scroll[221,214]
dead bit 0x08	n/a (no STATUS prop)	only while dead and the counter is above 27	at 28 or more	sent[221,214]
key	above 27 (one key index does not restart)	above 27	above 27	a key, after 34 s in the run[221,214,199]
destination	the new-game setup: `house1.graal` at (24.5, 30), facing 2; a second local player 12 tiles to the east	the saved entry, else (30.5, 30.0) online, else the built-in start-level table offline	→ §4.4.3	the level's entry point[221,214,195]
hearts	3, and the heart cap set to 3	max hearts, no AP bands (1.39 has no AP display; the bands arrive with 1.41)	by AP	by AP[221,214]

4.4.4 The level-entry save

Run after every level entry, and at login; saving stores the new level's name, the player's x and y, and the direction as it is[238].

Step	Condition	Result	Builds
1	the level left is empty (login)	save	all read[239]
2	index(new) × index(old) in the current gmap's level list is negative (-1 = absent; an index of 0 gives 0, so the first list entry behaves like "absent")	save	all read[238]
3	the name groups differ	save	2.00 (all revisions) to 2.05 rev 2[238]
4	otherwise	do not save	all read[239]

The respawn point is saved on level entry, but only when the new level is a different group from the old. The rules below give how 2.05 forms that group and which builds skip the test[238].

Rule	Detail
name group	drop everything up to the last path delimiter, lower-case, then take the text up to and including the last § (byte 0xA7) or $; with neither, the group is empty, and two empty groups are equal; the extension stays[238]
effect	plain names such as `a.nw` and `b.nw` are always the same group; the test only separates names that use the delimiters[238]
no gmap	2.05 returns -1 for both indices when there is no current gmap; 2.00 to 2.04 take the list straight from the client's state, which is inferred to give the same result[238]
2.17.1, 2.22, 2.31	no group test (step 3 absent)[238]
end-of-respawn save	the respawn routine calls the save once more, unconditionally; in the 2.22 lab, dying again before the server's warp reply still respawned at the entry point, not the death spot[238,195]
gmap list order	the list's order and the timing of its update against the save are open (→ App. E#314)[236]

Builds that carry the group test's delimiter set, by a constant scan, with the rule itself read only where marked:

Builds	Delimiter constant
1.1 to 1.36	absent[240]
1.37, 1.38, 1.39, 1.41	present[240]
2.00 to 2.05 rev 2, NewMain 24 and 26, NewWorld beta 1	present (rule read in all 2.00 to 2.05 builds)[240]
2.1 test 2 and 3, NewWorld beta 2	absent; their entry code not read (→ App. E#315)[240]
2.17.1, 2.22, 2.31	absent; entry code read, no group test[240]

4.4.5 Server side (GServer-v2)

Death is client-driven; the server reacts only to the dead bit of STATUS (prop 18) changing[241].

Event	Server action
dies (bit 0 → 1)	outside a sparring zone, adds a death and drops random gralats between `mindeathgralats` and `maxdeathgralats` as a floor item, plus items when `dropitemsdead` is set; a dead level leader hands leadership to the next player, who gets `PLO_ISLEADER` (10)[241]
revives (bit 1 → 0)	sets hearts by AP (under 20: 3; under 40: 5; else MAXPOWER; at least 0.5), pushes CURPOWER to the player and the level, and re-sends `ISLEADER` to a leader; it does not move the player[241]
`PLI_CLAIMPKER` (14)	ignored for a self-kill or an unknown killer; in a sparring zone both players get a rating update (skipped when they share an IP); elsewhere the killer gets a kill and, with the AP system on, can lose AP[241]
AP	with `apsystem` on, a per-second countdown raises AP by 1 (to 100) and reloads from `aptime0` to `aptime4` by AP band; it does not run while paused or in a sparring zone[241]
hearts	no periodic regeneration; a client CURPOWER above the server's value is refused while AP is under 40[241]
idle	`disconnectifnotmoved` with `maxnomovement` disconnects an idle player with a message[241]

4.5 Items, drops and board respawn

This chapter gives the client's side of lifting, carrying, hiding and throwing, the sword cut, chests, items on the floor, who rolls item drops in each build, and what the client does (and does not do) about board respawn. The `arrays.dat` tables named here are defined in → §3.5.3; the server's respawn timers and drop tables are in → §5.

> Status: draft. Read for 1.3, 1.38 to 1.41 and 2.05 rev 2; lab-checked on 1.39, 2.05 rev 2 and 2.22. Open points are App. E#316 to E#318.

4.5.1 Matching a 2 × 2 object

Cutting and lifting both work by matching a 2 × 2 object in the board. The rules below give the match, the probe, a change made in 2.04 and how the result is written[242].

Rule	Detail
match	take the tile under the probe point; for each table entry in order, for each of its four tiles equal to that tile, the object's top-left is the point minus that tile's column and row; the four board tiles there must equal the entry's four, all on the board; the first full match wins[242]
probe	one tile beyond the middle of the leading edge (→ §4.3.2)[213]
2.04 change	from 2.04 the whole 2 × 2 must match before anything happens; 1.3 matched only the single tile under the probe and wrote the leftover block at the derived corner[243]
writing	cut and lift share one routine: it writes the leftover tiles, then, only online, sends `PLI_BOARDMODIFY` (1) with the 2 × 2; nothing is sent if the tiles were already equal[243]
Windows 4.04r2, static	the attack-linked tile edit requires a complete two-by-two pattern; its first pattern and replacement also occur in pickup, whose category is tested by the bush property; no successful cut was captured[244]

The objects (`arrays.dat`; the same in 1.21, 1.3 test and the 2.22 install, one index lower in 1.21):

Table	#	Object	Tiles (top row / bottom row)	Leaves	Carry sprite
cut	0	bush	2 3 / 18 19	677 678 / 693 694	n/a[243]
cut	1	tall grass	420 421 / 436 437	679 680 / 695 696	n/a[243]
lift	0	bush	2 3 / 18 19	677 678 / 693 694	261[243,245]
lift	1	sign	512 513 / 528 529	1802 1803 / 1818 1819	267[243,245]
lift	2	pot (vase)	684 685 / 700 701	1770 1771 / 1786 1787	265[243,245]
lift	3	light stone	34 35 / 50 51	1834 1835 / 1850 1851	263[243,245]
lift	4	black stone	990 991 / 1006 1007	1834 1835 / 1850 1851	239[243,245]

4.5.2 Lift, carry, hide and throw

The A key lifts, carries and throws. The rules below give the key order, the glove rule, NPCs that can be carried, and the timing of a lift[220].

Rule	Detail
A key order	throw what is carried; else lift; else grab (→ §4.2.8)[220]
glove	glove power g allows the first g + 2 lift entries; every player starts at glove 1 (bush, sign, pot); `glove1` raises it to at least 2 (light stone), `glove2` to at least 3 (black stone); online the power is player prop 6; the cut-and-lift note words the steps differently: `disputed → App. E#316`[245,242]
NPCs	an NPC is lifted only when its script set `canbecarried` (command 41, cleared by 42), at the same point, with no glove check; it leaves the level's NPC list and nothing is sent[245]
lift	`lift` plays at once and the board tile becomes its leftover at once; a counter runs 4 down to 0 and the carried flag is set on the 5th tick; `carrystill` follows on the 7th tick (6 ticks of `lift`)[198]
carry	`carry` (8 frames, one per tick) at the normal 0.5 tile per tick; the object is drawn over the head, the head and arms hidden behind it[198]
carried image	the carried bush is the plain bush sprite, not the board's tiles (a bluish board bush is carried green)[199]

Rule	Detail
hide	D while carrying a bush, light stone, black stone or NPC hides the player under it (mode 14); D with a pot or sign does nothing; STATUS gets bit 0x02; the carry sprite prop does not change; only the object is drawn, where the player stands[245,198]
hidden walk	`hiddenstill` standing, `hidden` walking at 0.1 tile per tick (→ §4.2.1.1); `hidden.gani` is one frame, so the frame does not advance[198]
hidden on the wire	`hiddenstill` is step 24, sprite 18 (as `carrystill`); `hidden` steps 25 to 28, sprites 19, 20, 19, 18[245]
hits while hidden	the hit gate refuses hits on a hidden player unless a player byte that was not traced is set[245]
leaving	D again, or A, goes back to `carrystill` in the same tick; out from under a bush (not an NPC) the client starts break effect 0 (the leaves) at (x + 0.5, y + 1)[245]
throw	A, or S (also from hiding): `idle` in the same tick, the carried and hidden flags cleared, the carry sprite back to -1[198,226]
flight	a thrown pot flies about 6 tiles in 9 ticks, its shadow along the ground; it breaks into four pieces that spread over 4 ticks; a landing does not touch the board, it only breaks into leaves (`crush.wav`, or `water.wav` for the water leap)[198,243]
D key while carrying	does not throw; with carry sprite 239, 261 or 263 it hides (above)[226]
stone without glove	A at a black stone with glove 1 does nothing: no lift and no grab[198]

By build:

Build	Lift	Hide
1.39	counter 4 to 0, carried on the 5th tick; no lift pose; carried bush id 261	hidden flag; walk 0.1 per tick[198]
1.41	`lift` 6 ticks, then `carrystill`	hidden flag, but the pose stays `carrystill` / `carry`, the legs cycling at 0.1 per tick; `hidden.gani` is shipped but not played[198]
2.05 rev 2, 2.22	as above	`hiddenstill` / `hidden`; D toggles[198,199]
Windows 4.04r2, static	static sequence → §4.5.2.1; elapsed duration unmeasured	state relationships → §4.5.2.1; captured behavior unmeasured[244]
5.24r4, 5.30r3, 6.03r7	`lift` and the board change in the key's tick, `carrystill` 0.30 s later; a throw sends id 11 and `idle` in the same tick	hidden at about 1.5 px per tick; D with a pot does nothing[4]

4.5.2.1 Windows 4.04r2 static transition limits

Windows 4.04r2 was read statically for the same actions. Each row gives one relationship, what was seen and the limit on what that proves[244].

Relationship	Static observation	Limit
Pickup	Transitional action and countdown initialization precede a later update setting the carrying flag; the carried category and boolean carrying property read different state	Countdown initial value five does not establish total frames or elapsed duration; glove-bound semantics remain unmeasured[244,244]
Hiding	Bush and two stone categories, or a carried-object pointer, permit selection of a hiding-related state and animation; its movement branch starts with a smaller magnitude	Physical keys, distance per real-time tick and captured hiding behavior remain unmeasured[244]
Release	Creates a level carry entity and clears player carry fields under animation-dependent conditions	Separate exit-helper conditions remain untraced; no universal drop/unhide rule established[244]

These are static findings, not gameplay confirmation; older and later lab timings remain version-specific. Open questions: → App. E#611.[244]

4.5.3 Sword cut

A sword swing cuts what is at its tip. The rules below give when that happens, what is left behind and how bombs differ[242].

Rule	Detail
when	at the start of a swing, at the probe point, against the whole cut table with no glove[242]
effect	the four tiles become the entry's leftover tiles (array 10) through the shared write routine, then the leaves effect; the leaves fly up and out for about 8 ticks[242,198]
bombs	"bombs don't destroy half bushes" (2.05)[243]
bomb blast, 1.x	the blast's four test points cut cut-entry 0 and destroy lift entries 1 to 3 (sign, pot, stone)[196]

4.5.4 Chests

Chests are records in the level, not tiles. The rules below give how they look, how they block and how they are opened[242].

Rule	Detail
look	a chest record has no draw code in 2.05: a chest looks like its board tiles; the record adds the open action and a solid 2 × 2 footprint, opened or not[242]
2.22 drawing	the full (closed) form of PLO_LEVELCHEST (4) is drawn as a closed chest and the short form as an open one, each a 2 × 2 block over the board with no board tile needed[195]
solid	a closed chest stops the player when the body probes touch its 2 × 2 box; pushing against it plays push[195]
opening	only facing up, from below, with the action key; facing down, left or right nothing is sent[195]
on opening	at once: asks for chest.wav, sends PLI_OPENCHEST (20) with the chest's top-left in whole tiles, draws the chest open locally without waiting; about 1.2 s later sends message 13 with the point one tile above the chest[195]
sound	chest.wav, also when the chest has a sign; no item sound follows[246]
record bytes	each + 32: opened flag, x, y, and only if not opened the item index and the sign index (→ §3.1)[242]
the reward	what the server sends back for a known chest is unmeasured[195]
2.05 opening sides	unmeasured[195]

4.5.5 Item drops: who rolls

Who rolls a dropped item moved from the client to the server at 1.42. Each row gives a group of builds, who drops, and what the client sends[196].

Builds	Who drops	What the client sends
4th Graal alpha to 1.37	the client rolls	the roll as UDP 19 to the other players only; a pickup is applied locally, with UDP 20 and PLI_ITEMDEL (13); a timeout sends nothing[196]
1.38 to 1.41	the client rolls	the roll as UDP 19 and PLI_ITEMADD (12); online a pickup sends PLI_ITEMTAKE (32) and the server gives the item with props; a timeout sends PLI_ITEMDEL (13) (1.39 read and captured; 1.41 read for the roll only)[196]
NewMain 1.42 to 2.31	the server (PLO_ITEMADD, 22); offline nothing drops	pickup PLI_ITEMTAKE (32); timeout PLI_ITEMDEL (13)[196]

The rules below give the evidence for that change and what follows from it, offline and online[196].

Rule	Detail
the change	at 1.42: "bush rupees and pot hearts are server-side"; the roll is gone from NewMain 24 and 26 and still in 1.41[196]
arrows and stones	no client ever rolls arrows, and no client drops anything for a stone[196]
2.05 offline	the cut routine makes only the leaves and the lift routine only the carried object, so an offline 2.05 bush or pot gives nothing[243]
server drop timing	on a 2.22 server with pot drops on, the heart arrived in the same frame as the lift, at the pot's spot, and stayed when the pot was thrown[199]

4.5.5.1 The 1.x roll

One `Random(100)` per drop, with the Delphi generator `seed = seed × 134775813 + 1`, result `(seed × range) >> 32`:

Roll	Item
0 to 24	green rupee (25 %)[196]
25, 26	blue rupee (2 %)[196]
27 to 31	heart (5 %)[196]
32 to 36	bombs (5 %)[196]
37 to 99	nothing (63 %)[196]

Before 1.42 the client rolled the drop itself. The rules below give the table it used, what rolls, when and where[196].

Rule	Detail
same table	in every build from 4th Graal alpha to 1.41; from 1.38 the item is picked by its number in the item table (0, 1, 5, 3) instead of by name[196]
what rolls	every cut entry (tall grass too); a lifted bush (lift entry 0); a bush destroyed by a blast; a pot (lift entry 2) always leaves a heart, lifted or blasted; sign and stones nothing[196]
when	at the lift, not when the thrown pot breaks[243]
where	at the object's top-left tile[196]
new item	life 164 and the announce flag[196]
1.3 to peers	UDP id 19, `chr(x·2+32) chr(y·2+32) chr(item+32)`, sent straight to each peer, only for item kinds marked in a per-item table; the board change still goes to the server over TCP[243]

4.5.6 Items on the floor (2.05 rev 2)

An item on the floor has a life that counts down. The rules below give that life, when it can be picked up, the event it fires and how it blinks[196].

Rule	Detail
life	from 164, down 1 per tick; below 0 the item goes and, online, the client sends `PLI_ITEMDEL`; lab: 8.20 s from the drop[196]
pickup	the item is not permanent (life below 100 at creation, such as a chest's display item with life 24), its life is below 160 (not in its first four ticks), and `
event	at life 140 `playerlaysitem` fires[196]
blink	not drawn while the life is below 37 and odd[196]
bounce	raised by `
draw	the shadow, sprite 42, at (x + 0.5, y + 1 + h / 2 - 0.5) for an item h tiles high; the item centred on (x + 1, y + 1)[196]
sprites by item number	136, 137, 138, 67, 143, 139, 145, 351, 65, 350, 314, 315, 321, 323, 322, 324, 325, 328, 329, 330, 331, 332, 333, 334, 349[196]
floor items	`PLO_ITEMADD` makes an item only for the announce-flag items: 0 to 5, 7, 8, 19 and 24 (rupees, bombs, darts, heart, bow, bomb, gold rupee, spin attack)[196]
offline value	green 1, blue 5, red 30, gold 100 rupees (up to 9,999,999); bombs and darts 5 each (up to 99); a heart one whole heart; sound `extra.wav`[196]
online pickup	`PLI_ITEMTAKE` (32) with the item's three bytes; the server answers with props (for example RUPEESCOUNT)[196]
1.3	the pickup test is an overlap of pixel boxes, and a timeout is silent[196]
1.x pickup box in tiles, draw order against players	unmeasured (→ App. E#317)[196]

4.5.7 Board respawn on the client

Cut and lifted objects come back only when the server says so. The rules below give how the client handles a restore and what it sends after a push[243].

Rule	Detail
no client regrowth	no client regrows a cut or lifted object; the 1.3 and 2.05 board-modify record is built to send one change and has no timer; an offline 2.05 board polled for 633 s after a lift never changed back[243]

Rule	Detail
restore	a restore arrives as `PLO_BOARDMODIFY` (7) and is handled as any board modify; the timers and the echo rules are the server's (→ §5)[243]
where	the restore puts the object back where it was lifted, since the board changed only at the lift[243]
push and pull blocks	the 2.05 client sends the 4 × 4 area round the old spot (→ §4.2.7)[220]
re-entry offline	whether 2.05 reloads a level from disk on re-entry, which would bring a cut bush back, is unmeasured (→ App. E#318)[243]

4.6 HUD, sign panel, chat text and options

This chapter covers what the client draws over the game view: the status display, the sign panel, chat text and nicknames over players, and the item list window, plus what the in-game options change while playing. The file formats behind them are in → §3.6 (sign text), §3.9 (config) and §3.10 (sheets).

> Status: draft. The HUD, sign panel and item list are read from 2.05 rev 2; chat text is read from every Delphi build and checked live on 2.05 rev 2. The player list window is in → §4.10.

4.6.1 The status display

4.6.1.1 Sheet

The status display is drawn from one sheet. The rules below give its size and how it changed between builds[247].

Rule	Detail
sheet	every build from 1.41 on ships a 224 × 145 `state.png`[247]
2.22 against 2.05	the 2.22 sheet matches the 2.05 rev 2 and 2.1 test 3 sheets pixel for pixel except one 27 × 14 patch at (190, 60), which the status routine does not use[247]
buttons	1.41's sheet has three blank buttons at x 158, 180 and 202; the 2.0-era NewMain sheets have one, at x 202, which the routine draws in all three slots[247]

4.6.1.2 Layout (2.05 rev 2)

Positions are from the top-left of the game view; the routine's base point is (15, 15). Each part is drawn only when its `showstats` bit is set:

Part	`showstats` bit	Sheet rect (x, y, w, h)	Screen position
rupee, bomb and arrow icons	0x02	(80, 33, 120, 16), one strip	(274, 30)[247]
rupee count	0x04	digit d at (14d, 81, 14, 16)	1 to 7 digits, no leading zeros, 12 px apart, centred on x 280, y 48[247]
bomb count	0x08	same digits	two digits at x 326 and 338, y 48[247]
arrow count	0x10	same digits	two digits at x 374 and 386, y 48[247]
hearts	0x20	big (40·half, 0, 40, 44); small (80 + 31·half, 0, 31, 33)	→ §4.6.1.3[247]
buttons	0x01	(202, 0, 22, 33)	x 15, 80 and 145, y 30[247]
key names	0x01	text	on each button, 26 pt Helvetica, white with a black copy at +1, +1; smaller for longer key names; the item-list key drawn small at (147, 27)[247]
glove icon	0x01	sprite 145 for power 2, sprite 325 on top for power 3	(40, 29); the power printed at (60, 31) when 2 or more[247]
sword icon	0x01	sprite 17, from the player's sword image	(105, 31); the sword power printed at (125, 31)[247]
selected weapon	0x01	the weapon's whole image	x 170, centred on y 45[247]
alignment bar	0x40	frame (0, 97, 130, 22); fill (0, 119, AP, 10)	(15, 65); fill at +22, +6[247]
magic bar	0x80	frame (0, 49, 130, 22); fill (0, 71, MP, 10)	(15, 90); only with weapons on and MP above 0[247]

The table above gives the layout. These rules add the parts drawn by the same routine, the sprite rectangles it uses and the weapons flag[247].

Rule	Detail
other bits	0x100 and 0x200 belong to the player icons and the stacked weapon image list, drawn by the same routine but not part of the status display[247]

Rule	Detail
sprite rects	sprites 17, 145 and 325 come from `arrays.dat` array 24 (→ §3.5): 17 is (94, 36, 28, 28) of the sword image; 145 is (28, 212, 28, 32) and 325 is (0, 212, 28, 32) of `sprites.png`[247]
weapons flag	with weapons off only the first button, its key name and the alignment bar are drawn; the same flag gates the S and D keys[247]

4.6.1.3 Hearts

Hearts are drawn big or small depending on how many the player has. The rules below give which hearts are shown and where each row goes[247].

Rule	Detail
which	one heart per container while power remains; empty containers are not drawn; heart i is full when power $- 1 \geq i$ and half when $i <$ power[247]
big hearts	while power is 4 hearts or less; from (W − 198, 24), 45 px apart, W the view width[247]
small hearts	above 4 hearts, 35 px apart; new rows start at hearts 5, 11, 16, 22 and 27; row $k + 1$ is $20(k + 1)$ px lower and shifted $\text{round}(17.5(k + 1)) - 35$ px sideways; the block moves 10 px down while power is below 6[247]

4.6.1.4 The selected weapon

One list holds every weapon. The rules below give how entries are added, selected and deleted[247].

Rule	Detail
one list	server weapons (`PLO_NPCWEAPONADD`) and built-in weapons (`PLO_DEFAULTWEAPON`) share one list in arrival order; a built-in weapon is skipped when the list already has its name (→ §2.5)[247]
adding	every add selects the new last entry[247]
selecting index i	the first entry from i on, wrapping round, whose name is not empty and does not start with `-`; if every entry is hidden, i stays selected[247]
deleting	the selection keeps its index if it still exists, otherwise it moves to the new last entry[247]
built-in weapons	of the 25-entry item table, only ids 7 bow, 8 bomb, 12 superbomb, 20 fireball, 21 fireblast, 22 nukeshot and 23 joltbomb are weapons; the name is the item name with a capital first letter[247]
images	`wbowi1` to `wbowi4` and `wbomb1` to `wbomb3` (`.gif` in 2.05); 2.22 asked for the same names as `.png`[247]

4.6.1.5 Not measured

Some of the status display has not been read. The table lists what is still unmeasured, so a reader does not take the 2.05 layout as proven for other builds[247].

Item	State
2.22 and 2.31 status routines	unmeasured; their sheets and a 2.22 screenshot match 2.05[247]
builds before 2.05, including 1.41's three button cells	unmeasured[247]
font, colour and anchor of the glove and sword power text	unmeasured[247]

4.6.2 The sign panel (2.05)

4.6.2.1 Drawing

Sign text is drawn from a bitmap font. The rules below give the sheet, the glyph advances and the glyphs that draw frames and buttons[248].

Rule	Detail
sheet	`letters.png`, 256 × 224, 16 glyphs a row, each 16 × 32; glyph g at (g mod 16 × 16, g div 16 × 32); loaded once at start; the glyph order is the sign alphabet of → §3.6[248]
advances	each glyph has its own advance, twice the client's half-width table (I 6 px, space 8, most letters 12)[248]
frame glyphs	96 to 99 corners (top-left, top-right, bottom-left, bottom-right), 100 and 101 top and bottom edge, 102 and 103 left and right edge[248]
button glyphs	91 to 94 the A, B, X and Y buttons; 77 to 80 up, down, left, right; 71 to 75, 87, 88 and 67 the other button pictures[248]

Rule	Detail
fill colours	the pixels at (254, 223) and (255, 223) are the outer fill and, 8 px further in, the inner fill; a fill in the sheet's transparent colour is skipped; cream and cream on the 2.22 sheet, inner white on 1.39[248]
size	24 cells wide and 9 tall (16 px cells), centred across the view[248]
vertical place	640-wide view: y 256, or y 128 when the player is more than 14 tiles down the view; 800: 18 tiles, y 320 or 160; 1024: 22 tiles, y 448 or 192; which two values are compared is inferred[248]
frame order	top and bottom edges every 16 px, sides every 32 px, corners over them, then the two fills from (x + 16, y + 24)[248]
text	three lines a page from (x + 16, y + 16), 32 px apart; the text is split at newlines[248]
# codes	`#A #B #X #Y #u #d #l #r #h #x #y #z #4 #.` draw button pictures; # and any other letter draws glyph 86 and moves 16 px; `#i` and a character shows an image from the level's list[248]
other characters	a character outside the glyph order is drawn in the system font, in the colour at (253, 223)[248]
check	not compared against a screenshot of a real client[248]

4.6.2.2 Keys while reading

While a sign is open the keys do different things. The rules below give each key's effect[248].

Rule	Detail
own state	reading is its own input state: the player does not walk, swing, lift or use an item; switch-to-chat still works, and while the chat bar has focus the sign keys are skipped[248]
map key	ends reading at once[248]
up	turns back three lines, unless on the first page[248]
other game keys	left, down, right, D, S, A, item list and pause turn to the next three lines; when the page on show is not full, reading ends instead, so a sign of exactly three lines shows an empty second page before it closes[248]
edge	a key counts only when no game key was down on the tick before; a held key does not repeat (→ §4.1.2.1)[248]
side effects	`nextpage.wav` on each page turn (→ §4.7.3); `playerendsreading` runs when reading ends[248,246]

4.6.2.3 NPCs with no image

An NPC with no image draws nothing in play. The rules below say what is drawn instead in each case[248].

Rule	Detail
in game	an NPC with no image draws nothing of its own, only its text if any[248]
gani and character NPCs	drawn from the gani, or the way players are drawn[248]
placeholder	the placeholder picture appears only in the level editor's view (→ §4.9)[248]

4.6.3 Chat text and nicknames

The chat line is player prop 12 (→ §2.6.2); the chat and PM messages are in → §2.8.

4.6.3.1 Font

The chat font changed three times. Each row gives a group of builds, the face, the cell height and the shadow rule[145].

Builds	Face	Height (px, cell)	Shadow
1st Graal, 4th, 9th, 15th, 1.1.9, 1.21	`Dialog`	28	always blue[145]
1.31, 1.38	`Dialog`	28	contrast rule[145]
1.39, 1.41	`Helvetica`	24	contrast rule[145]
NewMain 1.42, 2.00 to 2.05 rev 2, 2.17, 2.22, 2.31	`Tempus Sans ITC`, installed from the client folder	24	contrast rule[145]

The rules below say how the chat text is drawn, how the config overrides the font, what the height value means and what replaces a missing face[145].

Rule	Detail
drawing	straight onto the game view with the Windows canvas, bold, no bubble and no box[145]
config	from 1.31 on, `font=` and `fontsize=` replace the defaults (→ §3.9.3)[145]
height	the value is the cell height in pixels, not a point size; for Tempus Sans ITC a 24 px cell is an em of about 18.4 px, which the live capture matches[145]
missing faces	`Dialog` does not exist on Windows and is mapped to a sans serif, normally Arial; `Helvetica` maps to Arial; neither was checked on real Windows[145]
bold	Tempus Sans ITC has only a regular face, so Windows emboldens it: each glyph smeared one pixel right, each advance one pixel wider[145]

4.6.3.2 Colour and position

Chat is white with a shadow. The rules below give the two drawing passes and how the shadow colour changed at 1.31[145].

Rule	Detail
colour	chat is white[145]
passes	shadow at (+1, +1), then text at (0, 0)[145]
shadow, 1.21 and earlier	always RGB (0, 0, 255)[145]
shadow, 1.31 on	red and green 0 if the text's red or green is above 0xE0, otherwise 0xFF; blue equals the text's blue; the same routine draws nicknames and number pop-ups[145]
x	centred on the player's x + 1.5 tiles[145]
y	the top of the text cell at the player's y − 1 tile, or y − 2 while carrying (the carry part read, not checked live)[145]
clip	to the level view rectangle[145]

4.6.3.3 Wrapping

Long chat lines wrap from 1.31 on. Each row gives a group of builds and its wrapping rule[145].

Builds	Rule
1.21 and earlier	one centred line, clipped, no wrapping[145]
1.31 and later	text wider than the view (640 px in 2.05) is split at spaces; a word that does not fit starts the next line, and a word too long for a line stands alone, unbroken[145]
1.31 and later	lines stack upward so the last line sits on the anchor; each line is centred on its own, then pushed left to fit the right edge and right to start at 0 or more[145]
6.x	wraps only text of 11 or more characters, breaking after any of `.,;+-?!`[145]

4.6.3.4 How long chat stays (2.05 rev 2)

The rules below give how long a chat line stays on screen in 2.05 rev 2, the length limit, and the special texts that show something else[145].

Rule	Detail
own line	stays while the player stands still (over 7 minutes seen); the first direction key starts a 60-tick (3 s) countdown, after which the line goes and the client sends an empty chat prop[145]
another player's line	shows for at least 3 s from arrival and until that player's empty chat arrives; an empty line only waits out what is left of the 3 s, it does not start a fresh 3 s; with no empty line the text stays[145]
length	a line longer than 256 characters is cut to 253 characters plus `...` when sent; the receiver draws what arrives[145]
showap	a chat of exactly `showap` shows the player's AP instead of the text[145]
other uses of the slot	while a per-player counter runs it shows two numbers joined by `/`; in some sprite states it shows an item image or a carried number; the chat is then not drawn; these branches were read but not named[145]

4.6.3.5 Nicknames (1.31 to 2.31)

Nicknames are drawn by the chat routine. The rules below give where, when the player's own name shows, and how it is cut[145].

Rule	Detail
position	drawn by the chat routine, centred on x + 1.5 tiles, the top of the text cell at y + 2.5 tiles[145]
other players	always shown[145]
own name	shown for 60 ticks (3 s) after setup (so at login), after an AP change, and after typing exactly `showname`, which is not sent[145]
length	cut to `nicknamelimit` (default 50) only when `limitnicknames` is on (off by default)[145]
colour	follows AP (prop 32, → §2.6.2); 1.21 has no AP colours; the same ramp is in every build from 1.31 to 2.31[145]

The AP colour ramp, each step `floor(n × k / d)` with a 0.0001 bias:

AP	Colour (R, G, B)
0	(64, 0, 0)[145]
1 to 26	(128 + (ap − 1)·127/25, 64 − (ap − 1)·64/25, same as G), dark red to red[145]
27 to 49	(255, (ap − 26)·255/24, same as G), red to pale pink[145]
50 to 55	white[145]
56 to 60	(255 − (ap − 55)·127/5, 255, same as R), white to light green[145]
61 to 70	(128 − (ap − 61)·128/10, 255, 128 + (ap − 61)·127/10), light green to cyan[145]
71 to 99	(0, 255 − (ap − 70)·255/29, 255), cyan to blue[145]
100	(224, 196, 0), gold[145]
live check	AP 0, 20, 40, 58, 65, 85 and 100 drew exactly (64,0,0), (224,16,16), (255,148,148), (179,255,179), (77,255,178), (0,124,255) and (224,196,0); AP 50, the value before any AP is sent, was white[145]

4.6.4 The item list (2.05)

The item list is a window of weapon cells. The rules below give the key that opens it, what it holds and how the cells are laid out[249].

Rule	Detail
key	the `itemlist` action (index 9; default Q) opens and closes the list; it opens from the normal state only when a player flag allows it, flag not identified[249]
content	the player's built-in and NPC weapons; a `/` in a weapon name makes a folder, and the current level's folders come first as 32 × 32 cells; names starting with `-` are skipped when selecting[249]
window	framed, from (16, 80) to 16 px from the right and bottom edges; cells from (48, 112) to 48 px from the right and 64 px from the bottom[249]
cells	max(image width, label width) + 10 wide, image height + label height + 15 tall, packed in the first free place left to right then down; image centred 5 px from the top, label centred below; a weapon with no image gets a 32 × 32 placeholder; the selected cell is filled RGB(255, 212, 128); font size 16; scroll arrows when cells are cut off[249]
arrows	stretch the selected cell to the edge in that direction and take the overlapping cell with the closest near edge; moving selects the weapon at once and sends nothing[249]
other keys	item key or Enter opens a folder, the attack key goes up a folder, other game keys and Esc close the list, a click selects a cell[249]
Delete	removes the selected weapon; online it sends `PLI_NPCWEAPONDEL` (29) with the name, or `PLI_NPCDEL` (22) with the NPC id for an NPC that is not a weapon (→ §2.4)[249]

4.6.5 Options at run time (2.05)

The file, its keys and its parse rules are in → §3.9; this table gives what each option does while playing.

Options	Effect
`musicon`, `soundfxon`, `reversestereo`, `usewinamp`	sound and music (→ §4.7)[250]
`noudp`, `fixedport`, `fixedudpport`	the UDP channel (→ §2.9)[250]
`nomassmessages`, `notoalls`, `nohtmlinpms`, `nohtmlimages`, `sortplayersbyname`, `dontloadlistheads`	the player list and messages (→ §4.10)[250]
`dontconnectlevels`	stops loading the neighbour levels of a gmap ahead of time[250]
`limitnicknames`, `nicknamelimit`	cut drawn nicknames (→ §4.6.3.5)[250]
network	none of the options is sent to the server[250]
dialogs	More Options, Player List Options and the Controls frame; closing either dialog rewrites the whole file[250]
later builds	2.17 and later add volume keys and an Audio/Microphone tab (→ §3.10.4)[250]

4.7 Sound

This chapter gives how the client places, limits and triggers sounds while playing. The backends, the options and the file types are in → §3.10.3 and §3.10.4.

> Status: draft. Read from 2.05 rev 2 and checked in a running 2.05 rev 2 client with an audio device; the 2.17 to 2.31 BASS clients are read only as far as → §3.10.4 goes.

4.7.1 Placing a sound (2.05)

Every sound goes through one entry point, called from about two dozen places: GS1 commands, gani `PLAYSOUND`, pickups, bombs, swords and baddies (→ §3.10.3 for the loop prefix, missing files and dispatch):

Rule	Detail
arguments	the file, the level offset x and y, and the x and y of the sound in tiles[192]
listener	the player at (x + 1.5, y + 2)[192]
gmap	a sound from a neighbouring gmap level is first shifted by ±64 tiles (→ §3.2)[192]
pan	(sound x − listener x) × 150, clamped to ±10000 DirectSound pan units; `reversestereo` negates it[192]
volume	a negative attenuation growing with the distance on both axes, clamped at −10000 (hundredths of a dB)[192]
looped effects	kept in the player's looped-sound list; leaving a level or entering the editor stops every looped effect in it[192]
voices	at most 8 effects play at once; a ninth request is dropped[192]
reuse	a cached effect is replayed from its buffer with the new pan, volume and loop settings[192]
after a missing file arrives	what 2.05 does then is unmeasured[192]

4.7.2 GS1 sound commands (2.05)

The commands' syntax is in → §8.2; their effect on the sound system:

Command	Effect
`play file`	one-shot at the NPC's position, its x and y plus half its image size[192]
`playlooped file`	the same, with the loop prefix[192]
`stopsound file`	stops a cached effect found by file name; the path is ignored[192]
`stopmidi`	stops the current music, MCI or Winamp[192]
`setmusicvolume l,r`	MIDI-out volume (MCI) or Winamp volume and pan, each 0 to 1[192,251]
2.17 and later	the same commands plus `musicpos` and `musiclen`[251]

4.7.3 Which action plays which sound (2.05)

Matched by the client's lookup of each file under `sounds\` while the action ran:

Action	Sound
walking on the ground	`steps.wav` when a move starts, then `steps2.wav` about every 0.2 s[246]
swimming	`water.wav` about every 0.8 s (→ §4.2.10)[246]
riding a horse	`horse.wav` and `horse2.wav`, alternating about every 0.4 s[246]
sword swing	`sword.wav`; with the battle axe, `axe.wav` (→ §4.3)[246]
lifting	`lift.wav` or `lift2.wav` by the gender byte (→ §4.7.4)[246]
getting hurt	the same `lift.wav` or `lift2.wav`, from `hurt`, an explosion, a thrown ball or a baddy hit[246]
dying	`dead.wav` (→ §4.4)[246]
laying a bomb	`put.wav`[246]

Action	Sound
bomb or explosion	`bomb.wav`: `putbomb` of any power, `putexplosion`, `putexplosion2` power 2, the end of a fireblast or nuke[246]
jolt explosion	`thunder.wav` (`putexplosion2` power 3)[246]
launching a projectile	`arrow.wav` for arrows, fireballs, fireblasts, nukes and `shootball`, and for arrows from shooting baddies and the dragon[246]
fire spy	`fire.wav` (`spyfire`)[246]
a thrown object or a leap landing	`crush.wav`; `water.wav` for the water leap (type 5)[246]
picking up a laid item	`extra.wav` (→ §4.5)[246]
opening a chest	`chest.wav`, also when the chest has a sign; no item sound follows[246]
turning a sign page	`nextpage.wav` (→ §4.6.2.2)[246]
pushing a stone block	`stonemove.wav` about once a second while it moves (→ §4.2.7)[246]
ledge jump	`jump.wav` (→ §4.2.6)[246]
frog	`frog.wav` when it hops[246]
baddy walking	`steps.wav`[246]
baddy dies	`compudead.wav`[246]
baddy draws its sword	`swordon.wav` at its call site; six baddy types tried, none played it[246]
an arrow hitting the player	`arrowon.wav` at its call site; not reached in the lab[246]
`item.wav`, `item2.wav`	used only in the sign-reading routine and a server-message handler; the chest and laid-item tests did not play them[246]
another player's bomb or item, online	`bomb.wav` or `thunder.wav`, `item.wav`, `extra.wav`, from server messages[246]
gani `PLAYSOUND`, GS1 `play` and `playlooped`	the named file[246]

4.7.4 lift.wav and lift2.wav

Two lift sounds exist, and one byte picks between them. The rules below give the byte, its default, the command that clears it, and an earlier note they correct[246].

Rule	Detail
choice	one byte on the player object picks the lift and hurt sound: set gives `lift.wav`, clear gives `lift2.wav`[246]
default	new players start with the byte set[246]
`setgender female`	the sound changed to `lift2.wav`; that the command clears the byte is inferred from that[246]
correction	an earlier note took `lift2.wav` for heavy objects and `fire.wav` for the fireball; both were wrong[246]

4.8 Offline and training mode

This chapter gives how the client plays without a game server: the ways in, where the player lands, where offline levels come from and what the embedded tutorial holds. The tutorial's level resources are in → §3.1.2.8; the level formats in → §3.1.

> Status: draft. Read from 2.05 rev 2 and checked live on 2.05 rev 2, 2.22 and 2.31; the special single-player level names are open (→ App. E#319).

4.8.1 What offline means

Offline is not a separate mode in the code. The rules below say what defines it and what stays the same as online[252].

Rule	Detail
condition	offline means the client has no game connection object; every check that refuses the editor, enables the tutorial resources or sends file requests tests that one condition[252]
current level name	the global that holds the current level name is set only by the server's set-level message, so it stays empty in offline training; the level is still drawn and walked[253]
same rules	entering an offline level stores the level name and arrival point on the player; collision, signs, links and scripts run the same paths as for a disk level online[253]

4.8.2 Ways in (2.05)

A 2.05 player can get offline in four ways. Each row gives one way in and what it does[246].

Way	Detail
Training	a button on the start screen; starts the embedded tutorial at `tutorial_b-4.graal` and makes no connection[246]
Play offline	a button of the reconnect dialog, beside Reconnect and Open serverlist; choosing it frees the game socket[252,246]
command line	`Graal.exe level.graal` starts with a local player and opens the file in the editor; Play then runs it offline (→ §4.9)[252]
F5 Load level	while online the client warns that loading a level disconnects from the server, then loads the file locally[252]

4.8.2.1 The reconnect dialog

The reconnect dialog is the usual way into offline play. The rules below give when it appears and what Play offline does in each case[252].

Rule	Detail
texts	shown when the connection cannot be established, is dropped, or the server is full[252]
no listserver	pressing Start with no listserver shows the connection error and stays on the server list, which has no Play offline button[246]
refused game port	the dialog appears; Play offline goes to the tutorial start, reading no level file from disk[246]
dropped after login	the dialog appears; Play offline reloads the current level from the game folder with the player where they stood[246]
half-closed socket	a normal close from the server side left the socket half-closed and showed no dialog; only a reset or socket error brought it up[246]

4.8.2.2 Where the player lands on Play offline

Where Play offline puts the player depends on what is on disk. Each row gives one case and its result[252].

Case	Result
the current level exists in the local folder	the player stays on it at the same coordinates[252]
it does not	the player goes to `tutorial_b-4.graal` at (32, 60); the big map switches to `tutorialmap.png` and the minimap to `tutorialmap2.png`, both with `tutorialmap.txt`; a first-offline-start flag is set[252]

Case	Result
the level still fails to load	a warning names the level and the client stays at the reconnect dialog[252]
dropped on a level not on disk	not repeated live; the code falls back to the tutorial[246]

4.8.3 Where offline levels come from

Offline levels come from the executable or the game folder. The rules below give the lookup order, show that the embedded tutorial wins, and say what changes online[252].

Rule	Detail
order	offline, a name starting with `tutorial` is read from the executable's resource (→ §3.1.2.8); any other name goes through the file-name cache to disk[252]
embedded wins, 2.05	with a different `tutorial_b-4.graal` in the game folder, Training still loads the embedded level and does not look on disk[246]
embedded wins, 2.22	with `offlinetutorial_b-2.graal` in the game folder, the client looks the name up once but still loads the embedded copy[246]
online	names starting with `tutorial` are not special and go to the server like any level, so a server can host its own[252]
tile definitions	every client tested reads `levels/tiledefs/tiledefsOffline.txt` on Training, since the file is `tiledefs` + server name + `.txt` and the offline server name is `Offline`; the shipped file targets only `offlinetutorial*` levels, so it changes nothing in 2.05's tutorial[246]
special names	`level1.graal`, `YoshiClanR2.zelda` and `YoshiClanFields.zelda`, the old single-player start levels, run an extra setup step on load; what it does is unmeasured (→ App. E#319)[252]

4.8.4 The tutorial world

4.8.4.1 2.05

The 2.05 tutorial is a small world of linked levels. The rules below describe its grid and what each level holds[252].

Rule	Detail
grid	3 columns (a to c, west to east) by 4 rows (1 to 4, north to south), plus `tutorial_b-2_house.graal` linked from b-2 at (22, 26); edge links keep the other coordinate and land at 0 or 61[252]
scenery levels	a-1 to a-4, c-1 to c-4 and b-1: 2 or 3 links, no NPCs, chests or signs[252]
b-4, the start	8 NPCs, 1 chest, 3 signs: a welcome sign, a gate NPC that opens when the player says the word the sign gives while standing in a band of rows, a `setmap` and `setminimap` NPC, two light NPCs and two character NPCs; the chest holds `glove1`[252]
b-3	a sign telling the player to cut the trees with the sword[252]
b-2	5 NPCs, 2 signs: an arrow box, a door that opens on the `opendoors` flag with `chest.wav`, character NPCs[252]
b-2 house	6 NPCs, 6 chests, 7 signs: a shopkeeper, a gate that slides open on `housegateopen` with `stonemove.wav`, lamps and lights; the chests hold battleaxe, lizardshield, spinattack, nukeshot, fullheart and superbomb[252]
signs at (0, 0)	not drawn on the board; chests and `say n` show them by index[252]
end	the last sign sends the player to F8, the server list, to go online[252]
map	`tutorialmap.txt`, set by the start level's `setmap` NPC[246]

4.8.4.2 Later builds

Later builds replace the tutorial and the start screen. Each row gives a group of builds and what changed[252].

Builds	Detail
2.17, 2.22, 2.31	9 embedded `GR-V1.03` levels, `offlinetutorial_a-1.graal` to `offlinetutorial_c-3.graal`[252]
2.22, 2.31	Training starts on `offlinetutorial_b-2.graal`, the centre of the grid; its NPCs play `cauldron_bubbles.wav` and `fire_torch0` to `fire_torch2` as soon as it loads; the maps are the client's own `bigmap.txt` and `minimap.txt` plus per-level map parts[246]

Builds	Detail
start screens	2.05: Training, Options, Start; 2.22 adds Get an account!; 2.31 adds Editor, which launches the separate GraalEditor (→ §11.6)[246]
GraalEditor 2.3.1.1	carries the same 9 names and `tiledefs0ffline.txt`[252]

4.8.5 Windows 4.04r2: startup and an offline transition

Windows 4.04r2 was started under a test harness. Each row gives a condition, the state before, what was observed, and the limit on what it proves[254].

Condition	Before	After or observed result	Limit
Controlled startup, no server or login	Client initialization	Native game/UI window construction, successful DirectDraw initialization call, and opens of configuration, cache, GUI assets and four supplied animations	Window visibility, successful decoding and a rendered gameplay scene are unmeasured by these hooks[254]
Harness invokes an internal offline transition on the UI thread	No adventure object	Offline-mode flag set; adventure object allocated; observed message-loop callbacks continue for 20.031 seconds after the call returns	Instrumented invocation, not a normal user action or documented API; a playable level, movement and bush interactions remain unmeasured by this capture[254]
Run R01: instrumented transition, then a tutorial board from a Mac-distribution fixture	No adventure or level state	Adventure and level state allocated; 539 shell ticks at a median of about 50.27 ms; no visible game and no periodic core update or render hook	A separate run from the row above; it measures the shell timer, not the core simulation rate, a frame rate, a natural start-up or gameplay[254]

These observations do not establish that 4.04r2 uses the 2.05 entry conditions or arrival rules in → §4.8.1–4.8.2. Earlier unsuccessful harness attempts are not successful offline sessions.[254]

Remaining scene and mechanic validation: → App. E#610.

4.9 The built-in editor

This chapter gives what is particular to the editor built into the 2.05 client: how play and editing switch, what entering the editor does to the game, and what it writes. The editor's window, dialogs and objects, and the later standalone GraalEditor, are in → §11.6; the level formats it reads and writes are in → §3.1.

> Status: draft. Read from 2.05 rev 2, with Generate map, Play and Edit flags checked live on 2.05 rev 2.

4.9.1 Where the editor lives

Where the level editor lives changed between 2.05 and 2.31. Each row gives a build and its editor[255].

Builds	Editor
2.05	a second mode of the main game window, not a separate program[255]
2.31	the client has no editor of its own; the start screen's Editor button starts `GraalEditor.exe` from the game folder as a separate process (→ §11.6)[246]

4.9.2 Switching between play and editing (2.05)

Four keys move a 2.05 player between play and editing. Each row gives a key and what it does in game mode[255].

Key	In game mode
F4	opens the editor; refused while connected, with a message that the editor cannot be opened online (→ §4.8.1)[255]
F5	the Load level dialog; online it first warns that loading disconnects from the server (→ §4.8.2)[255]
P or the Play button	back to game mode on the current level (→ §4.9.4)[255]
Close	closes the current level tab, asking to save a modified level[255]

The rules below give what happens on the way into the editor, how a level on the command line is opened, and which keys a focused field blocks[255].

Rule	Detail
teardown	entering the editor stops looped sounds (→ §4.7.1), frees the level's NPC and baddy instances, and opens the player's level as a tab re-read from disk, so board changes made while playing are not carried over; the view centres on the player[255]
command line	`Graal.exe path\level.graal` skips the login screen, changes into the file's folder, creates a local player at (30, 30.5) and opens the file in the editor[255]
editor keys	L, S, P and M (links, signs, play, generate map) are ignored while a combo box in the side panel has focus; the full key list is in → §11.6[255]
NPC with no image	shown with a placeholder picture only in the editor's view (→ §4.6.2.3)[248]

4.9.3 Undo (2.05)

Undo covers board tiles only. The rules below give what is recorded, how deep the stack goes and what an undo restores[255].

Rule	Detail
scope	board tiles only, hence the button's name, Undo board copy; links, signs, NPCs, baddies and chests are not undoable[255]
record	before any board write: the rectangle (x, y, width, height) and the tiles it overwrites[255]
stack	one per level tab, at most 128 records; the oldest is dropped when full[255]
undo	pops the newest record, writes the old tiles back, re-selects the rectangle (or clears the selection if it was the whole board) and marks the level modified[255]

4.9.4 Play, flags and the map image (2.05)

The rules below cover the editor's Play button, its flag list and how it builds the map image[246].

Rule	Detail
Play saves first	Play on a named, modified level writes it to disk ("Level saved") before the Level options dialog asks for the start position; a level with no name must be saved under a name first; 2.22 does the same (→ §11.6)[246,255]
Edit flags	edits the client's in-memory flag list, not anything in the level; the list survives Play and a return to the editor, but is never written to disk or the registry and is gone after a restart[246]
Generate map (M)	after a prompt that it may take minutes, walks the level list of the loaded big map (`bigmap.txt` or the list a `setmap` NPC loaded) in list order, row by row[246]
Generate map, each level	read from disk, its NPCs run into a 1024 × 1024 view, shrunk to 56 × 56 px; a missing or unreadable entry is a blank square; links are not followed, so a level not in the list (such as the tutorial house) is skipped[246]
Generate map, output	offered in a save dialog as `graalmap.png` in the level's folder; the 3 × 4 tutorial list gave a 168 × 224 image; the level that was open is restored afterwards[246]

4.9.5 Saving and opening (2.05)

The editor writes a level by its extension. The rules below give the format chosen, how the board is packed and two edge cases[255].

Rule	Detail
format by extension	`.nw` is written as `GLEVNW01` text (→ §3.1); any other extension, normally `.graal`, always as `GR-V1.03`; the 2.05 editor cannot write older versions[255]
`GR-V1.03` board	the writer emits a single run for two or more equal tiles and a double run when `A B A B` starts at the cursor, otherwise a plain tile (→ §3.1)[255]
baddy section	ends with `FF FF FF 0A` on write; the reader does not need it (→ §3.1)[255,255]
URL levels	saving a level opened from a URL falls through to Save as[255]
opening	accepts `Z3-V1.03`, `Z3-V1.04`, `GR-V1.00` to `GR-V1.03` and `GLEVNW01`; anything else is refused as an unsupported file type[255]
chest items	in index order: greenrupee, bluerupee, redrupee, bombs, darts, heart, glove1, bow, bomb, shield, sword, fullheart, superbomb, battleaxe, goldensword, mirrorshield, glove2, lizardshield, lizardsword, goldrupee, fireball, fireblast, nukeshot, joltbomb, spinattack (→ §4.5.4)[255]
new chest	defaults to item 1 (bluerupee) and no sign (−1)[255]

4.10 Other client features

This chapter collects the client features that sit outside the core game loop: the integrity checks, animation state as other players see it, key emotes, the player list and its social lists, and what the client does with the server's start message and disconnect texts. The message layouts are in → §2.4 and §2.5.

> Status: draft. The 2.05 parts are read from 2.05 rev 2; the 2.17 to 2.31 parts come from lab captures and reads of those clients. Open points are App. E#320 to E#322.

4.10.1 Integrity checks

4.10.1.1 The tamper check (2.17 to 2.31)

From 2.17 the client proves to the server that its code is unchanged. The rules below give the message, how its bytes are laid out and what the CRC words cover[256].

Rule	Detail
builds	new in 2.17; absent from 2.05 and 1.x[256]
message	`PLI_TAMPERCHECK` (39), 10 raw bytes, sent right after `PLI_LANGUAGE` (37) in the first frame after login; the per-build values are in → §2.4.9[256]
what the bytes are	two 32-bit words A then B, each written as 5 bytes in a form that is neither `GINT5` nor masked: only the first and last byte are range-limited, so the middle three can fall below 0x20[257]
the words	both are standard reflected CRC-32 values over the client's own image in memory, chained across ranges; A covers 8 or 9 ranges of the code, B one large code range and one block of static data[257]
constant	nothing depends on the login key, account, level or time, so the value is a constant per build and changes only if the executable's bytes change; all six lab values were reproduced offline from the executables[257]
2.17	sent once at login; nothing more in about 90 s[257]
2.22	re-sent by the level-change routine, so on every gmap edge crossing and after every link request the client starts, but not after a server-side warp; no timer[257,256]
2.31	re-sent every 3.2 s by the 64-tick integrity cycle of → §4.1.3; not re-sent on level entry[201]
client check	the clients only send it; nothing on their receive side checks it[257]
servers	the open-source server reads and discards it; four public servers accepted a 2.22 login without it for at least 20 s, which shows only that they do not enforce it at login (→ App. E#320)[256,256]

4.10.1.2 Process list and window scan

The server can also ask for a process list, and 2.31 scans windows on its own. The rules below give both checks and when they run[256].

Rule	Detail
process list	the server sends `PLO_LISTPROCESSES` (182) once during login to 2.17 and later; the client can answer `PLI_PROCESSLIST` (44) with its running programs (→ §2.4.9)[256]
2.31 window scan	tick 20 of the 64-tick cycle compares the titles and classes of top-level windows with an obfuscated blacklist; the list is not decoded and what the client does on a match is unknown (→ App. E#321)[256,258]
2.31 debugger probe	tick 1 of the cycle probes for a debugger and arms a 5000 ms timer[201]

4.10.1.3 What outside input tools look like

The recovered trainers of the period are summarised here only for what the client's checks could see of them:

Rule	Detail
how they act	every recovered sample types keys into a classic client window or writes text into it through window messages; none speaks the game protocol[259]
memory	no memory-writing or remote-thread imports were found in the unpacked samples, so none of them changes the client's image; the tamper check of → §4.10.1.1 cannot see them[260,261]
what the server sees	only the resulting actions (dig, use, walk, chat), at the rates the tool's waits allow[259,262]

Rule	Detail
window titles	some tools look for the client window by title, and some show their own named window; such a window is the kind of thing the 2.31 scan compares, but whether its list names any of them is unknown (→ App. E#321)[259,260]

4.10.1.4 The recovered trainers, tool by tool

The trainers fall into four groups: AFK chat bots, AutoIt input macros, chat and costume script spam, and one archive blocked as malware and never opened; a caps-lock utility filed with them is not a Graal tool[261]. Their loops were read statically and none was run; the waits are client-side waits, not server cooldowns, so they are the rates a server would see[259].

Tool	What it sends	Rate and waits	Window check
AutoD (2001)	d	every 1 ms, forever	none[259]
AutoS	s#	no delay, forever	none[259]
AutoIdle	a fixed pass of 48 left and right arrow taps	no sleep	none[259]
Graal Auto Key	nothing to the game: renames the active window to `Graal Auto` after 2 s, then exits; the other RpgWarrior tools look for that title	once	renames[259]
WaterGet v1.1	Down twice, a	key delay 300 ms, then 2000 ms per pass	none[259]
WaterGet v1.0	S nineteen times 1000 ms apart, then two idle directions twice each	2000 ms, 500 ms between directions	`Graal Auto` must be active[259]
Random Walker	one of: nothing, Up, Down, Left or Right five times, or S five times, drawn at random	key delay 200 ms	`Graal Auto`[259]
Graal Prayer	Tab, then the chat text `/use_skill praying` and Enter three times 2000 ms apart, a weapon shortcut, s ten times, another shortcut and the idle direction; a helper sends Up ten times	key delays 0 to 100 ms, sleeps of 500 to 2000 ms	`Graal Auto` (the helper none)[259]
Graal Load	each line of a chosen text file written into the client window, then Return; the bundled `bomb.txt` is 164 lines of costume commands and a countdown	150 ms per line	finds the window by its text[259]
AFK Bot	three phrases in turn (`I am AFK` and two more) into the focused chat box	one every 3000 ms	none; 3 s to click the chat box or it stops[259]
AFK Scroll Bot	one phrase scrolled across chat, first character moved to the end each tick	100 ms tick	none[259]
Graal Text Sender 1.3.5	operator text in six modes (word scroller, middle in-out letters, word stay, two words repeat, letter-by-letter to-alls and shout-outs)	100 ms timer, per-mode waits set by the operator	looks up the client window; declares `GetAsyncKeyState` but never calls it, so it is not a key logger[259]

The trainers recovered from the period were read one by one. Each rule is one tool or archive and what it turned out to be[261].

Rule
The AutoIt tools are compiled AutoIt 2.60 to 2.64 scripts packed with UPX, using only input and window-message calls (`keybd_event`, `mouse_event`, `SendMessageA`, `PostMessageA`, `FindWindowA`, `BlockInput`); the VB6 tools import only the VB runtime[261,260]
Digger's archive is truncated (36,111 of 41,893 bytes of a UPX image), so its loop is lost; its howto says to equip a shovel, stand on grass and focus the client[259]
The miner archive is blocked by the file host as malware and was never downloaded, so nothing is known of it beyond its size[261]
A static pass over the remaining images found the same input-only pattern and nothing that speaks the game protocol[262]

4.10.2 Animation state as others see it

4.10.2.1 2.05: the sprite prop

In 2.05 the sprite prop carries a step number as well as the direction. The rules below give its form, when the step changes and the table from step to value[263].

Rule	Detail
form	prop 17 (→ §2.6.2) is direction + 4 × S[263,264]
when S changes	only while the active gani is the default `def` gani and the step is 38 or lower; other ganis leave S alone, and a name the sender does not know gives 0; read from the sender, not confirmed on the wire[263]
hurt	S = 32, so the prop is 0x80 + direction[263]
step to S, steps 0 to 38	0, 4, 5, 4, 3, 2, 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 11, 12, 14, 15, 16, 17, 23, 18, 19, 20, 19, 18, 21, 22, 21, 18, 24, 25, 26, 27, 28, 29; steps 9 to 13 are the sword swing (→ §4.3)[265]
gani to step	idle and def: frame; walk and walkslow: frame + 1; kick 9; sword, ridesword and spin: 9 + (frame mod 5); push 14 + frame; grab 19; pull 20 + frame; lift 23; carrystill and hiddenstill 24; carry and hidden 25 + frame; shoot 33; ridestill 34; ridefire 35; rideeat 36; ride 35 + (frame and 1); sit 37; sleep 38; hurt and ridehurt 39; dead 40, not sent[266]
built-in ganis	31 names, index 0 to 30: idle, walk, walkslow, kick, sword, ridesword, spin, push, grab, pull, lift, carrystill, hiddenstill, carry, hidden, shoot, ridestill, ridefire, rideeat, ride, sit, sleep, hurt, ridehurt, dead, maps1, maps2, maps3, swim, lava, carried[267]
not on the wire	swimming, lava and death are sent as S 0 plus direction; death travels only as the STATUS dead bit (→ §2.6.2.1, §4.4); there is no swim sprite, so another 2.05 client cannot tell a swimmer from an idle player[264]
STATUS as 2.05 sets it	0x01 editor or paused; 0x02 hidden (mode 14 or the hidden timer); 0x04 male; 0x08 dead, only once the death timer reaches 28 ticks; 0x10 weapons allowed; 0x20 sword hidden while the `hidesword` timer runs; 0x40 has the charged spin (→ §2.6.2.1)[268]
short codes	the client maps gani-name parts to one letter (attack A, carry C, dead D, hurt H, idle I, kick K, lift L, ride R, sit T, spin U, swim V, sword S, walk W, def X and others) to shorten cached gani file names[269]

4.10.2.2 2.1 and later

From 2.1 the gani travels as a name. The rules below give that prop and where its cadence is measured[263].

Rule	Detail
gani prop	prop 10 is the gani name as a string; a receiver that lacks `<name>.gani` asks the server for it[263]
2.22 cadence	→ §4.1.4[199]

4.10.3 Key emotes (2.05 rev 2, same routine in 1.39)

Ctrl and a letter shows an emote. The rules below give the keys, how the image is found, where it is drawn and that nothing is sent[263].

Rule	Detail
keys	Ctrl+A to Ctrl+Z, with no text field focused and the player neither dead nor locked[263]
lookup	the letter is lowercased and the client takes the first loaded file whose name starts with `emoticon_<letter>` and ends in png, mng or gif; there is no letter-to-name table, so the mapping is whatever files exist[263,264]
drawing	at the player's x + 2, y − 5 (a little lower when a second flag is set) for 50 ticks (2.5 s)[263]
network	the key path sends nothing; `PLI_SHOWIMG` (24) is sent only by the script command `showimg` (→ §8.2)[263]
others seeing it	unmeasured: captured SHOWIMG frames re-sending an AFK image about every 3 s are not explained by the key path (→ App. E#322)[263]
earlier builds	1.38 introduced Ctrl+letter emotes, and 1.39 added letters[263]

4.10.4 Picking up and dropping items online

Items are picked up and dropped through four messages. The rules below give each one and what the server does with it[270].

Rule	Detail
2.05 pickup	sends `PLI_ITEMDEL` (13) with the item's half-tile position and changes its own props itself; the server only removes the item and tells the others[270]
the other form	`PLI_ITEMTAKE` (32) also removes the item, and the server applies it to the player's props and echoes them[270]
drop	`PLI_ITEMADD` (12) with the position and item[270]

Rule	Detail
chest	PLI_OPENCHEST (20) with whole-tile x and y; the opened state is per account (→ §4.5.4)[270]

4.10.5 The player list and social lists

4.10.5.1 2.05 (F7)

The 2.05 player list is opened with F7. The rules below describe its window, its popup and what each action does[271].

Rule	Detail
window	a stay-on-top form with a multi-select list, a caption "--- N online ---" and Profile and Options buttons; the Options window has 5 check boxes and the profile edit window 10 fields[271]
popup	right-click selects the row under the cursor and shows Message (default), History, Buddy, Ignore, Profile; Buddy and Ignore are checked when every selected player is in that list[271]
open	double-click or Enter opens the read window if a PM from that player is waiting, otherwise the compose window[271]
Message	one PM to all selected rows, reusing an open window for the same set; at most 50 recipients (→ §2.8)[271]
Profile	the first selected player only; online it sends PLI_PROFILEGET (80), which the listserver answers (→ §6)[271,272]
Ignore	toggles each selected account in `ignoredaccounts.txt` and saves the buddy list too; local only; ignored players' chat and PMs are dropped[271,272]
Buddy	toggles `buddyaccounts.txt`; Buddy and Ignore exclude each other[271]
auto reply	the client holds the text "Your messages are ignored."; when it is sent was not traced[271]
world click	there is no right-click handler on a player in the game view; the only player popup is the list's[271]
no player action form	the person-action form is the editor's NPC script editor (→ §4.9), not a trade or duel menu[271]

4.10.5.2 By build

The player list grew one feature at a time. Each row gives the build where a feature arrived[272].

Builds	Change
1.33	popup Message, Ignore, History; sections GPs, Guild, Normal, Ignore[272]
1.34	ignore by account name; ignored players drawn red[146]
1.35	Buddy, kept in `buddyaccounts.txt`[272]
1.39	Profile and the Player List Options window[272]
2.14 and later	a Staff section named from PLO_STAFFGUILDS (47); the status list PLO_STATUSLIST (180) with the player's own index in prop 53[146]
2.22	sections Staff, Buddies, Guild, Normal, Ignore; popup Message, History, Buddy, Ignore, Mute, Profile[146]
2.22 Buddy and Ignore	local only: they write `buddyaccounts.txt` or `ignoredaccounts.txt`, one account per line, delete an empty buddy file, move the entry in the list, and send nothing[146]
2.22 Mute	sends PLI_MUTEPLAYER (43) with the player id and 1 or 0, one message per click; the entry does not move[146]

4.10.6 Start message and disconnects

The rules below cover the messages a server sends around login and logout: the start message, the RPG window, their order, and how a disconnect is explained[273].

Rule	Detail
start message	PLO_STARTMESSAGE (41) carries the server message as one line of HTML; 2.05 has a form for it; whether it shows at once or on request is unmeasured[273]
RPG window	PLO_RPGWINDOW (179), 2.1 and later: quoted, comma-separated lines naming the server and its software[273]
login order	level data, big map, minimap, RPG window, start message, an empty PLO_SERVERTEXT (82)[273]
disconnects	every refusal, kick and ban reaches the client as PLO_DISCMESSAGE (16) with the text, `\r` for a line break, then the socket closes; the client then shows the reconnect dialog (→ §4.8.2.1)[274]
upgrade	from 2.14, PLO_UPGRADEURL (169) with a disconnect turns Reconnect into an Upgrade button[274]
server texts	the refusal texts and the order of the server's login checks are server behaviour (→ §5)[274]

4.11 Precursors, side builds and the 4.x clients in play

This chapter covers how the builds outside the 2.x Delphi line behave: the Java precursors, the relay room bridge, the Game Boy Color demo, the 39ster fan client, and the 4.04r2 and 4.20r8 clients run live. Their place in the build list is in → §1.7 and → §1.8.

4.11.1 Zelda Online and Java Graal

The Java precursors were run in the lab. Each rule records what one of them does when played[22].

Rule
Zelda Online 1 (class files dated 16 May 1998) runs at scale 2 in a 512×448 box and fetches levels, `arrays.dat`, images and sound relative to its codebase. The rar ships 36 `Z3-V1.03` levels[22]
Zelda Online 1 hard-codes its story per level name: people on `level13`, `level28`, `level22` and `cave2`, chests on `level21`, `level24`, `village3`, `level16` and `level29`, gated by progress flags[22]
Zelda Online 1 already holds the sign glyph-code list `A B X Y u d l r h x y z 1 2 3 4 .` beside the 97-entry alphabet (→ §3.6)[22]
Zelda Online 2 (files dated 18–26 July 1998) opens a start window with One-Player and Multiplayer modes and a server field defaulting to loopback, refuses to run on Java 1.0, and reads files from the working folder[22]
A second `ZeldaMain (1).class` differs from the shipped one only in the "free stone" tile left by a lifted stone: 1023 instead of 584. The readme warns that the local version's tile indexes differ from the online server's[22]
The Zelda Online 2 options hold a nickname, a show-own-name option, cap, cap-edge, belt and coat colours and the 10-character save code; the save code packs x and y (6 bits each), level index (9), rupees (10), darts (6), power in half hearts (5), four ability bits, four story-chest bits and a 6-bit set-bit count (→ §1.7)[22]
The Zelda Online 2 copy in the "very first graal" zip carries a player's edits of 13–15 February 2002: four new levels, four changed ones, and an `objects.dat` with 109 stamps[22]
ZeldaEditor (Delphi 3, linked around 25 July 1998) has a toolbar with New, Open, Save, Save As, Undo, Cut, Copy, Paste, Delete, "Turn Selection Into A Warp Field", Edit Links, Edit Baddies and Edit Signs, a stamp palette, and keeps window positions in `ZeldaEditor.ini` as `Name:left,top,width,height`[22]
GServ Standalone ("(C) Stefan Knorr 1997, based on JccVsd") is a game-neutral lobby on 14801: ids are `0000` plus a counter that wraps at 9999, the log is `GServProto.txt`, and a `ChessSync` thread sends every "Chess" player a one-character timer line each second (→ §2.3.1)[22]
The Zelda Online 2 client joins `Z` + level base + `1.6`, sends `C` and `-` on a level change and joins the new room as `def`; the real nickname follows in `D`. It answers the first `+` with `P` and pushes its state[22,26]
The client clamps baddy knock-back to ±4 before encoding, but one of the four clamps writes the wrong axis, so a large positive vertical knock-back overwrites the horizontal value[22]
When two clients see each other at once both send `P`, GServ links the pair twice, and every later `D` line arrives twice (1144 relayed lines for 572 sent); a server that emulates GServ need not copy this[26]
Java Graal: five classes in `Graal.jar`, all dated 2 July 2000 and signed; a 512×418 Java 1.1 applet on a Cyberjoueur page beside Bomberman, chess, Reversi and Tetris; starts on `level13.graal`; has a `nickname:` chat command; the 2002 rip holds 180 readable levels and 25 damaged ones, many `.zelda` files carrying `GR-V` headers[22]
Java Graal sends the Zelda Online 2 `D` types without `S` (colours) and with `L` (level name), so Java players show in default colours to others[22]
Zelda Online's `arrays.dat` is generated by `makeArrays.java`: the passable list `nowall`, then `swamp`, `jumpstone`, the `bushobj`, `liftobj` and `chestfields` lists each with a replacement tile, `movestone`, and sprite placement tables (`spritevalues`, `hurtsprites`, `compusprites`); `addSprites.java` appends the sprite crop rectangles packed into integers. The later tile-type tables are in → §3.5[25]
Zelda Online 1 walks one half-tile per key-down event: no held-key polling, so a held key repeats only through OS key repeat; facing turns before the wall test, the wall-push count tries a stone on the 29th blocked event and wraps 38 → 18, every key-up clears the sword latch and push and grab progress, and the worker sleeps a fixed 50 ms without catch-up[275]
Zelda Online 1 keys: arrows, `i j k l` or `8 4 2 6` move; space, `s` or `5` sword; `a` or `1` lift/grab; `d` bow; no pause key and no uppercase mapping. A key that closes a sign is processed further, so it can also move or attack[275]
Zelda Online 2 (from source, not run) polls held keys on 50 ms ticks that subtract the work time, starts at (22.5, 36) with 3 hearts and 5 arrows, and adds `P` pause and case-insensitive letters[275]
Zelda Online 2 combat (from source): sword hits on the middle 3 of 6 counts; damage sword 1, player arrow 3, contact 0.5 with an unnormalized knock-back; enemy hit points green/swamp 2, blue/shooting 3, red 4, octopus 1; bush drops green currency below 25 %, a heart below 30 %, blue below 32 %; currency caps at 999 and arrows at 30[275]

Rule
Zelda Online 2's relay leader is chosen by comparing each peer with the local login time rather than the best so far, so peer order can change the leader (bytecode agrees)[275]
The Java level codec is LSB-first with 11-bit codes for Zelda 1.02, 12-bit for Z1.03/1.04 and GR1.00/1.01, 13-bit for GR1.02/1.03; 399 files decoded to 4,096 tiles each[275]
The July 1998 Zelda editor saved an empty board as a 206-byte `Z3-V1.03` file with every tile 1023 (one run)[275]

Constants of the shared Java engine, beyond those in → §1.7.1[26]:

What	Value
shield bump	4 ticks[26]
lifting	5 ticks[26]
baddy moves	sent by the level leader every 2 ticks (100 ms)[26]
push frames	advance every 4 ticks; letting go or turning resets the counter to 0[26]
liftable kinds	four; kinds 0 to 2 always liftable, kind 3 (heavy stone) needs the stronger glove; kind 2 always leaves a heart[26]
board changes	each relayed as a `B` line; the level leader sends all current changes to every newcomer[26]
chat font	TimesRoman bold 20 pt, black over a white copy one pixel right and down, centred at the player's x + 12 pixels times the zoom; the nickname is drawn the same way three tiles lower[26]

4.11.2 The relay room bridge

Java Graal (2000) and the Delphi betas talk to the same GServ lobby but join different rooms. Run live against Knorr's own GServ[88]:

Pair	Relay	Meet	What went across
Java Graal + 4th Graal alpha	stock GServ	yes	both drawn, nicknames, chat both ways[88]
Java Graal + 5th Graal	stock GServ	no	different rooms (`G2<level>` and `G3<level>`)[88]
Java Graal + 5th Graal	GServ behind a bridge that renames `G2` to `G3`	yes	both drawn, nicknames, walking and chat both ways, a level change and back[88]

A relay bridge let builds of different months meet. The rules below record how it works and what got in the way[88].

Rule
Every Delphi beta opens two relay connections, one to a room per level and one to a fixed game-wide room; the Java builds open one. Room names per build are in → §2.3.1[88]
The bridge passes every line unchanged except the room in `N<room>#<name>` and `0<room>#<name>`; the rename must keep the level part, or players on different levels share a room[88]
5th Graal sends `D` types `T P Y C N S`, Java Graal `T P C N D`; each ignores the types it does not know, so the `D` dialect lines up from Zelda Online 2 to 1.1.9[88,88]
Under Wine the betas lose the arrow keys, because DelphiX's DirectInput keyboard format lists one object and Wine fills only listed objects; with `WINEDLLOVERRIDES="dinput=d"` DelphiX falls back to `GetKeyboardState` and walking sends a `D #P…` line every 100 ms while a key is held[88]
From 3rd Graal the server box is greyed out at Cyberjoueur's fixed address on 14802; 4th alpha and 1st Graal keep an editable address on 14801[88]

4.11.3 The Game Boy Color demo

`Graal V1.1 Adventure Demo (PD) [C].gbc` is 262,144 bytes; its sources reassemble to a byte-identical ROM[23].

Rule
Built with GBDK 2.x and LCC: 161 functions, of which 41 are GBDK library or crt0 routines[23]

Rule
Everything runs in the VBlank interrupt: OAM DMA, scrolling, the window slide and music every VBlank, game logic every second one (about 30 Hz); `main` switches the CGB to double speed and returns[23]
Cartridge type `$02` (MBC1+RAM) with RAM size 0; CGB flag `$80` (runs on both models); banks 6 to 15 are all $FF[23]
Bank 0 core game; bank 1 entities, AI, lifting, camera, music; bank 2 graphics (two 256-tile sets, an 82-tile font, walk bitmaps, five BG palette sets, 172 sprite tiles); bank 3 objects, effects, the character table; banks 4 and 5 rooms[23]
A room is one 32×32-tile BG map (256×256 pixels), written whole to VRAM; positions are 16-bit half pixels, a tile is 16 units; directions are 0 up, 1 left, 2 down, 3 right, as on the PC; sprites are 8×16, sorted by Y each tick[23]
Room header (13 bytes): map pointer, tileset (0 outdoor, 1 indoor), BG palette set, then count and pointer for links, texts and NPCs; the bank is room / 5 + 4[23]
Map: 1,024 tile ids then 1,024 CGB attribute bytes; collision reads a second WRAM copy that changes when things are lifted[23]
Link (8 bytes): x, y, width, height, new x, new y (−1 keeps a coordinate), then the target room as 16 bits[23]
Text (4 bytes): x, y and a string pointer; a text at (0, 0) is shown only by index; signs are read only while facing up into a solid tile[23]
NPC (16 bytes): x, y, direction, character type, two unknown bytes, text index, five movement bytes (mode and patrol bounds), a 16-bit flag word (bit 0: turn and chase once lined up) and a 16-bit event number. Event 1, on the captive in room 4, is the only event[23]
Each tileset has a 256-bit walkable bitmap and a second 256-bit set read when something is put down; its meaning is open[23]
Seven rooms: six outdoor rooms in a 2×3 grid (0 start, 2, 3 with two chasers, 4 the captive, 5 "Hello!", 6 the house) and room 1, the house inside; room 0's sign reads "Welcome to Graal! Demo version by Stefan Knorr"[23]
The text box is three lines of 18 characters, padded so words break right; in play only a d-pad direction other than the facing one turns the page; capitals are drawn red[23,23]
31 character types, each with a name, three OBJ palettes and a two-frame 16×16 head (Fung Si Yun, Pach Soldier, Ninji, Lunar, Squall, Vincent, Mandi, Ryu, Zoraas, Mad Max, Pachukite, Matt, Brandon, Bob, Athena, Lizardon, Oldman and recoloured Gold, Red, Green, Blue, Fire, Silver, Theif and Water variants); NPCs and the player pick one by number[23]
At most eight 50-byte entities, entity 0 the player; states 0 idle, 1 walking, 2 reading, 3 sword, 4 knocked back, 5 chasing, 8 carrying, 9 lifting; movement mode 1 is a random walk, 2 to 4 patrol[23]
Controls: d-pad walks, B swings the sword (every character, three colours), A is the character's item (lift and throw for Fung Si Yun, a bomb for Ninji, a boomerang for Ryu, nothing else), Select opens the carousel, Start toggles music[23,23]
Lifting compares the top two tile ids two tiles ahead with four liftable kinds (three kinds of bush and stone, and a dropped object never seen in play)[23]
Eight one-shot sound effects on the pulse and noise channels, skipped during music; the music player reads its song from `$A000` in cartridge RAM, which the ROM never writes, so Start mutes the game until reset[23,23]

Seen in an emulator (PyBoy, CGB mode, 60 Hz frames)[23]:

What	Value
walk	4 pixels per logic tick, 2 per frame, 120 per second; a room takes about 2 s to cross[23]
against a wall	the walk cycle keeps running in place; no push pose[23]
sword	an arc from one side to the front, about 24 frames; hits knock back and blink; nothing dies, no damage, no game over[23]
lift	the object becomes its lifted tiles at once, is drawn where it stood for 10 ticks, then over the player's head; the carrier faces the screen with arms up and the object hides the head[23]
throw	flies about 6 ticks, breaks into four leaf pieces for about 8 ticks; nothing is written back to the map[23]
respawn	none while in the room (a stump stayed two minutes); the room reloads from ROM on re-entry[23]
Ninji's bomb	sits about 110 frames, flashes about 70, then a plus-shaped blast about three tiles each way[23]
Ryu's boomerang	about four tiles out and back, about one second[23]
NPCs	room 3's first soldier patrols the top at 1 pixel a frame; both soldiers chase at a little over 1 pixel a frame once lined up; Vincent and Squall wander in room 6[23]
the captive	after its first text it walks two tiles left, lifts and throws the bush, and the second text opens; the player also starts as Fung Si Yun, so the text reads as if the player was meant to start as someone else[23]

4.11.4 The 39ster client and GraalX

Two side builds were read for comparison. The rules below describe the 39ster fan client and GraalX[22].

Rule
39ster's client 1.00 (23 April 2014): `GameMaker.exe` is a small C++ engine on SDL 1.2 with zlib and stb_image, unrelated to YoYo's GameMaker; all Graal logic is about 1,500 lines of GameMonkey script in `main.gm`[22]
It reads `.nw` and `GR-V1.00` to `1.03` levels and `.gani` files, converts tile ids into the 2.x `pics1.png` layout, starts on `level13.nw`, has no socket code, and shows NPC images without running their scripts; it bundles 1,733 `.nw` levels and 1,321 GIFs from community worlds. Its `arrays.dat` is a plain-text tile-type table[22,22]
GraalX has the same 28 forms as 2.0 rev3 and 2.01 and differs from 2.01 only in two control positions on the main form[22]
39ster movement (from its script, not run-tested): 3 pixels per nominal update (2 from sitting), diagonals × 0.75, time factor = last frame ms ÷ (1000/60) capped at 4, vertical resolved before horizontal, single-axis slide at 1.5 × the factor with 16-pixel snapping[275]
39ster tile categories: ground, blocking, throw-through, water, chair, jump, swamp, shallow water; blocking, jump and throw-through count as walls; eight GANIs with 1, 8, 5, 8, 1, 1, 3 and 5 frames; a link latch blocks repeat warps until the player leaves the link area[275]
GraalX saved an unchanged `GLEVNW01` level byte-identical (10,319 bytes; one case, no edits)[275]

4.11.5 4.04r2 and 4.20r8 live

The real 4.04r2 client (token `G3D17026`, reported by GServer-v2 as 4.042) and a restored 4.20r8 engine were run under Wine against GServer-v2 through a gen-5 logging proxy. Ganis and art came from the lab server, so anything a gani draws is the server's look[276].

Rule
The draw-layer mask is 0 while the lowercase server name starts with `login` and all layers (0x7ff) otherwise; ghost mode (S→C 170) forces 0x500. A 178 warp to `x,<name>,<host>,<port>` (field 0 unused, fewer than 4 fields ignored) ends the lobby (→ §1.8.6)[276]
In the Login session the game still runs: walking, chat and the sword send as normal; only drawing is cut[276]
Tab opens a dark red chat bar at the bottom of the view and Enter sends (prop 12); the bar stays open after Enter, Tab closes it and sends an empty prop 12, Esc asks to quit. Letter keys without Tab are game keys[276,276]
Chat is white bold sans above the head with a pure blue (#0000FF) copy 1 px down and right; no bubble[276]
Walking sends X2/Y2 (78/79) every 50 ms in half-pixels with the sign in the low bit, 16 half-pixels (half a tile) per tick; the first step carries GANI `walk` and SPRITE; each tick sends only the axes that moved; `idle` goes alone one tick after the last step; X/Y (15/16) are never sent[276,276]
Push starts about 0.9 s after the player stops (4.20r8: 0.85 s); `pushed,0` follows 0.55 s later and then once a second; a click sends `leftmouse` at the clicked point[276,276]
A on something that cannot be lifted plays `grab`; S sends 34 `sword.gani`, 157 `sword`, prop 10 `sword`, `sword.wav`, back to `idle` about 250 ms later; D with the bow sends 157 `shoot`, prop 10 `shoot,wbow1.png`, `arrow.wav` and C→S 9; with bombs C→S 4[276,276]
A cut sends C→S 1 with the 2×2 stump tiles and `crush.wav`; a lift sends the same, then `lift` with `item.wav` and `lift.wav`, `carrystill` and carry sprite 1 about 300 ms later. The client keeps no respawn timer[276]
The carried object is the gani's ATTR3 sprite, drawn last 22 px above the body, so it covers the top of the head. D hides the player (status 0x16, `hiddenstill`, `hidden`), hidden walking averages 1.6 px per tick; A throws with C→S 11 and an empty payload[276]
Coming out of hiding with A sends `carrystill` and status 0x14; a second A throws; S throws straight from hiding. A thrown object breaks 250 to 350 ms after the throw, with no packet marking where it landed[276]
After a reconnect while hidden the server had saved `hiddenstill` and status 22 with nothing carried; only the shadow drew until the first step. A server should clear the hidden bit and gani when it saves or loads an account[276]
The client runs GS1 weapon scripts but sends only `client.` string flags to the server: a probe that set `gr.probe`, `client.probe` and `probeflag` sent only C→S 18 `client.probe=2`[276]
After login both builds send 37, 23 `basepackage.gupd` twice, 23 `bigmap.txt` (Login session only), 157 `idle`, 152 `GraalEngine,pmguilds` and `pmservers`, 154 `GraalEngine,lister,options,…` and `verifybuddies`, 46, 44, then 2[276]
Update packages: with no copy the client asks `basepackage.gupd` with 23 twice and saves it in `updatepackages/` (and per server in `updatepackages/<server, space as %032>/`); on the next run it sends C→S 47 with the file's CRC-32, while within one run it keeps asking with 23. `bigmap.txt` is asked once per run[276]
Prop 0 in the own props is the start screen's Nickname field, not the account; losing focus sends status 21 and regaining it 20; 24 is `GCHAR 9` every 5.142 to 5.151 s[276]
4.20r8 survives only in the packet-logger build; putting back the 8 original bytes at the hook's call site gives a plain engine, which GServer-v2 accepts with `G3D20126` and which behaves as 4.04r2 for chat, walking, pushing, sword and items[276]

Lifting with glove power 3 on the lab level[276]:

Object	Glove	Board after lifting	Carry sprite	Item drop (GServer-v2)	D hides	Back after
bush (2)	1	677 678 / 693 694	1	sometimes	yes	14.2 to 14.5 s[276]
pot (684)	1	1770 1771 / 1786 1787	5	a heart, every time	no	15.0 s[276]
stone (34)	2	1834 1835 / 1850 1851	3	none	yes	15.0 s[276]
black stone (990)	3	1834 1835 / 1850 1851	byte 0xe9 (201), not decoded	none	yes	14.3 s[276]

The respawn times are GServer-v2's 15 s timer checked once a second, not the client's (→ §4.5.7)[276].

4.11.6 2.04 to 2.05 rev 2: what changed in the code

The three builds closest to 2.05 rev 2 were matched function by function on address-free instruction streams; unmatched counts are an upper bound on real churn[277].

Pair	Functions (old / new)	Identical	Changed	Only old	Only new
2.05 rev1 → rev2	6522 / 6529	6345	94	83	90[277]
2.04 → 2.05 rev2	6471 / 6529	5799	558	114	172[277]

2.04, 2.05 rev 1 and 2.05 rev 2 were compared function by function. Each rule is one difference in code or classes[277].

Rule
2.05 rev1 to rev2 moves about 20 game functions with identical class layouts and GS1 table: player counting reads both tests through one object (the "playerscount is fixed" note), the tutorial no longer creates a map object at start-up, one level-reset path drops a game-mode-5 flag and takes a loop bound from the main form instead of 632, and one of three §-to-CR/LF helpers is dropped[277]
rev2 is linked against a rebuilt VCL: its library units differ in shape from rev1's, while rev1's library matches 2.04's[277]
2.05 adds the classes `TGraalMap` (gmaps) and `TADP`, and drops `TEditorBaddy` and `TEditorLevel`[277]
`TDrawableObject` gains a virtual method in 2.05, so every level-object class's slot numbers move by one: 2.04 `X.virt_N` is 2.05 `X.virt_(N+1)`[277]
`TAdventureLevel` grows from 96 to 184 bytes (gmap and `.nw` state), `TAdventureCommand` from 68 to 76, and `TServerPlayer`, `TPlayer` and `TEditorNPC` shrink by 4[277]
The `GLEVNW01` reader and writer are new code beside the binary reader; the writer emits one `BOARD 0 <y> 64 0 <tiles>` line per row, and a new chest-line parser reports "Wrong chest data!" (→ §3.1)[277]
New GS1 in 2.05: `loadmap`, `setz`, `updateterrain`, `setbacktile2` and an argument form of `setbacktile`; the table holds 161 commands, and only `loadmap` has an executor case (→ §8.2)[277]
2.05 writes PMs to `pms\messages_<name>.txt`, adds a `drawaslight` draw check, a script-editor button that handles `/* */` comments and tabs, and an editor routine clamping `playerx`/`playery` to 0–61; 2.04's `tempsitc.ttf` installer is left uncalled and the `Courier New` default is gone[277]
Anti-debug: 2.05, its revisions, the 2.1 test builds and both NewWorld betas link the third-party `TADP` component with `Kill_Debugers`, `Stop_Execution`, `Anti_Procdump` and `Kill_Monitors` all on; it walks processes, modules and heaps and stops the client when it finds a debugger, loader or monitor (names such as `FPLOADER.EXE`). Builds up to 2.04 do not have it (→ §4.10.1)[277]

4.11.7 The 2.05 rev 2 client's parts

2.05 rev 2 (`GNW03081`) is a Delphi build on DelphiX (DirectDraw, DirectSound, DirectInput) with zlib 1.1.2; its classes are named by RTTI[278].

Area	Classes and role
network	`TGraalClient` (socket, out buffer, zlib flush, u16be frame); `TServerListClient` (listserver link, no cipher, handles ids 0 and 1); `TAdventureClient` (game link, gen-3 send, a receive jump table of 59 ids, the login builder); `TWebClient` (HTTP for web levels and `openurl`); `TAdventureServer` (offline stand-in)[278]
socket rules	the game socket keeps the gen-3 iterator, the key and a first-message flag; disconnecting renames the server `Offline`; messages flush at 0x1000 bytes and the link drops at 256 KB queued[278]

Area	Classes and role
game model	TAdventure (main window and loop); TAdventureLevel, TBoard, TBoardModify; TLevelObject and its extras, leaps, bombs, explosions, balls, flying objects and carried objects; TServerPlayer with TPlayer and TEditorNPC; TComputer (baddy); TAdventureHorse; TDisplayedNPCImg, TTempWeaponImg; TAdventureWeapon; TTileDef, TTempTile; TGraalMap; the TGraalAni* gani model; TCachedStream[278]
script engine	TAdventureCommand, TAdventureOperation and TAdventureVariable parse-tree nodes with TArray, TInteger, TDouble and TStringPosList values, run by a recursive tree walk (→ §8.1)[278]
forms	connect, reconnect, server list, load level, server message, short message, player list and its options, profile, options, more options, links, sign codes, TPersonAction (the NPC "Set Npc's Action" editor, not a player menu), chest editor, debugger; editor and client are one executable until 2.3 (→ §4.9)[278]
other code	a Delphi DES class (UDesCipher) for saved passwords, the hidden listserver host and the web cache, not the wire; GIF, MNG and JPEG decoders[278]

2.05 and 2.22 hide strings and passwords in several ways. The rules below give each scheme and why none of them protects anything[184].

Rule
2.05 hides the listserver host as a DES ciphertext and key built one Chr() at a time, and the port and newmain as plain characters built the same way; a hosts entry for the host is enough to redirect it[184,184]
2.22 keeps its in-memory copies of the three listserver hosts and port, and the account name in the config and before login, through dec(b) = (NOT b − 10) & 0xFF; the plain values come from arrays2.dat, so a memory search for the names finds nothing[184]
2.22 saved passwords: a 49-character value starting with a marker is decrypted with a second key held by the options form, anything else with the first, so old saves still load; new saves use the second key (→ §3.7)[184]
None of these schemes protect anything; they keep strings out of a casual strings dump[184]

4.11.8 Client features 2.00 to 2.31 by release notes

The shipped newfeatures2001.txt to newfeatures2005.txt are named by release year: 2001 covers 2.00–2.12, 2002 2.13–2.17, 2003 2.18–2.21, 2004 2.22 and 2005 2.3–2.31[279].

Version	Release-note highlights	On the wire
2.00	online only: 2/3/4-player mode and save games removed; bush rupees and pot hearts go server-side; warpto <account> for GPs	[279]
2.01	level preloading on maps; hitobjects as the fair replacement for hitplayer; openurl2	36/46[279]
2.02	hats and signs with images; string parsing; free-server limits on light effects and complex ganis	[279]
2.03	triggeraction; / chat commands in RC and NC; the built-in zlib.dll replaced by in-game code "because it has been abused"	38/48[279]
2.04	gani PLAYSOUND PARAM1, replacegani, sound and animation changes	[279]
2.05, 2.05 rev 2	mostly bug fixes; server name in the window title	type 0, gen 3, 59 S→C ids[279]
2.1	new start window and server list with account info; an image engine without GDI; gani scripts (SCRIPT … SCRIPTEND); string arrays; play2; auto-mapping; big map and minimap set by the game server (bigmap=, minimap=); ghost mode for free players; Alt+0 log window; graalversion	170–174[279]
2.12	Graal2001/Graal2002 client compatibility; multi-level editing; wraptext, wraptext2	[279]
2.13	editor control-NPCs; a new RC with file browser and server list, RC and NC in one program (→ §7)	[279]
2.14	two server-list tabs; voice chat; player-list status icons (playerlisticons=); serverwarp <world>; files over 64 KB (downloadpos); gmaps auto-download	180, 178, 68/69[279]
2.15	bug fixes, "enabling Graal2002"; showimg2	[279]
2.16	real transparency (seteffectmode); gani attributes #P10–#P30; a UDP check with a warning when UDP is blocked	props 54–74[279]
2.17	Graal Kingdoms speed-ups; level heights stored in level files instead of the gmap	185; adds 39 and the signed arrays2.dat[279]
2.18	"new password encryption" for stored passwords; preloaded levels on entering a server	[279]
2.19	released alongside v3; new start screen and server list; "better compression" (bz2); this. strings on the client; on NPC-server servers the client stops sending non-client. strings	GNW15053, gen 4[279]

Version	Release-note highlights	On the wire
2.2	multi-byte sign text (Korean, Chinese, Japanese); server-side tile changes on gmaps via `updateboard2`	186[279]
2.21	"network traffic reduced drastically"; `triggeraction 0,0,clientside,…` reaches a weapon's client-side part	`GNW01113`, compression byte[279]
2.22	a third server-list tab for playerworlds under construction (gold and VIP only); `gravity`; `downloadsize`	gen 5, `GNW03014`[279]
2.3	editor and online client split; `FILENAMECACHE.txt`; a new timer; B-tree NPC lookup; classic client-side mode removed; Graal3D tab; `graal://` with `serverstartparams`; full-precision positions; `timevar2`; more hacking checks	44 dropped; 189 and 84 in 2.31[279]
2.31	float-to-string fix, keyboard config saved, gmap NPC border crossing fixed	`GNW28015`[279]

A feature announced as "enabled later" often shows in a client's receive switch one or more versions before servers used it (full-precision moves in 2.3, level heights in 2.17), so a handled id does not prove servers of that era sent it[279].

4.11.9 Crossing gmap edges (2.22, 2.31)

A real 2.22 client walked east, south, west and north across a 2×2 gmap of open-grass levels on a lab server; the gmap file format is in → §3.2[167].

Rule
Three different "maps" must not be mixed: level links (S→C 1, walking into one sends 0 or 30); bigmap-style server maps (`maps =`, a text grid the server alone uses to know which levels are adjacent); and gmaps (`gmaps =`, `GRMAP001`), where the client walks continuously. The `bigmap =` and `minimap =` options are a fourth thing: overlay settings sent as S→C 171 and 172 at login, and setting `bigmap` breaks gmaps[167]
At login the server sends S→C 49 (x×2, y×2, z+50, cell x, cell y, then the gmap name), `LEVELNAME` (6) with the level and again with the gmap name, 174, 10 and 156 with the gmap name; the client fetches the `.gmap` itself and sends 35 (`i5` modtime + name, 0 on a cold cache) for the three neighbour levels[167]
A crossing uses only ordinary props, with no level-warp message: the cell prop (43 or 44) changes in the same props message as the wrapped coordinate (local = global − 64 × cell), and position props go out only on whole-tile steps[167]
2.22 sends flag sets `gr.x` and `gr.y` on every half-tile step carrying the global position (past 63.5) from the server's movement weapon, and re-sends 39 on each crossing; going west, 39 goes at global x 63.5 and the next props carry X = 63.0 and cell 0[167]
The server answers each crossing in one frame: 6 with the new level, 39 (level modtime, only when not cached), 0 (an empty board), 6 with the gmap name, 174, 10, 156[167]
2.31 sends 46 (`GCHAR 1`) first, then the props with the cell prop and wrapped coordinate, and does not re-send 39 at the crossing (→ §2.4)[167]
UDP id 24 ("entered level") goes to peers in the 3×3 neighbourhood; the server keeps 39 per level so the next entry can use 30 or 35 with a cached board[167]

4.11.10 What each executable names, 1998 to 2.31

A static survey lists, per build in link order, what each executable adds (+) or drops (−) against the one before. From 1.38 some config keys are built one character at a time, so a key can be read while missing from the list; garbled three-letter keys (`ekv`, `u5j`, `d4n`, `kxs`, `azg`) are such fragments[191].

Build	`graalconfig.txt` keys
6th Graal (beta5)	+ `joystickon`, `musicon`, `reversestereo`, `soundfxon`, `waitvertical`[191]
8th Graal (beta8)	+ `saveimages`[191]
1.1	+ `editnickname`, `gamemode`[191]
1.21	+ `accountname`, `loadcolorsfromserver`, `password`, `serveraddr`, `serverport`[191]
1.28 beta2	+ `compabilitydraw`, `externalchat`, `language`, `nodecompressionerror`[191]
1.3 test	+ `font`, `fontsize`; − `accountname`, `password`[191]
1.31	+ `fixedport`, `fixedudpport`, `noudp`; − `editnickname`[191]
1.32	+ `nomassmessages`[191]
1.33	+ `nodirectinput`[191]

Build	graalconfig.txt keys
1.35	+ nohtmlinpms[191]
1.36	+ limitnicknames, nicknamelimit; − serverport[191]
1.38	− externalchat[191]
1.39	+ dontconnectlevels, dontloadlistheads, nohtmlimages, notoalls, sortplayersbyname[191]
1.41 rev1	+ dontsavepasswords, fullscreenmode, usewinamp; − serveraddr[191]
NewMain 1.42	+ dontsavepms, dontusedirectx7, head, joyselected; − gamemode[191]
2.00	− loadcolorsfromserver[191]
2.1 test 2	+ startinfullscreen[191]
2.17.1	+ automapping, microactivatebyvolume, microactivatekey, microactivationlevel, microinputdevice, microon, midivolume, mp3volume, quickplayerworlds, radiovolume, sfxvolume, voicecodec, voicevolume; − musicon, soundfxon, usewinamp[191]
2.31	+ cachefilenames[191]
Graal3D	bodytex, bodyx, fixedport, fixedudpport, headx, noudp, serveraddr, serverport[191]

Each build names its windows in the executable. The table follows the Delphi forms that each build adds, from 1st Graal to 2.31[191].

Build	Delphi forms (windows)
1st Graal	+ TADVENTURE, TCONNECTFRAME, TEDITBADDY, TEDITSIGNS, TLINKLIST, TLINKOPTIONS, TLOADLEVEL, TOBJECTNAME, TOPTIONS, TSHOWCODES[191]
2nd Graal	+ TCHESTEDIT, TEDITPERSON, TPERSONACTION[191]
3rd Graal	+ TEDITFLAGS[191]
5th Graal	+ TMOREOPTIONS[191]
6th Graal (beta5)	+ TOPENURL[191]
9th Graal alpha	+ TRECONNECTFRAME[191]
1.1	+ TCONTROLSFRAME[191]
1.21	+ TDEBUGGERFRAME[191]
1.28 beta2	+ TEDITCHARACTER, TMESSAGEEDITOR, TPLAYERLIST[191]
1.32	+ TRTFMESSAGE[191]
1.39	+ TPLAYERLISTOPTIONS, TPROFILEEDIT[191]
1.41 rev1	+ TSERVERLISTFRAME, TSERVERMESSAGE, TSHORTMESSAGE, TWAABOUT; − TRTFMESSAGE[191]
2.04	+ TOBJECTSET[191]
NewWorld betas	− TCHESTEDIT, TEDITBADDY[191]
2.1 test 2	+ TEDITNPC; − TEDITPERSON, TOBJECTSET, TPERSONACTION, TSHOWCODES[191]
2.17.1	+ TABOUTDLG, TLOGFRAME, TRPGFRAME, TTILESARRAYFRAME; − TWAABOUT[191]
2.31	− TCHESTEDIT, TDEBUGGERFRAME, TEDITBADDY, TEDITCHARACTER, TEDITFLAGS, TEDITNPC, TEDITSIGNS, TLINKLIST, TLINKOPTIONS, TOBJECTNAME, TTILESARRAYFRAME (the editor leaves the client)[191]
Graal3D	TADVENTURE, TCONNECTFRAME, TEDITLOOK, TMOREOPTIONS[191]

The rules below follow the other names an executable carries: sounds, images and fonts, build by build[191].

Rule
Sound names in the executable: 4th Graal alpha has arrow, axe, bomb, chest, crush, dead, extra, frog, lift, put, sword, swordon; 7th adds water; 8th adds arrowon, compudead, fire, goal, horse, horse2, item, item2, jump, lift2, nextpage, sign, steps, steps2, stonemove, thunder; 1.1.9 Gold drops horse, horse2, steps2; 1.34 brings back horse2; NewMain 1.42 drops dead, goal, sign (→ §4.7)[191]
The NewWorld betas drop chest, compudead, frog, item2, jump, stonemove and swordon, which 2.05 rev2 and 2.17.1 carry[191]

Rule
Images: 1st Graal names `letters`, `link`, `map`, `opps`, `pics1`, `state` (`.gif`); 3rd adds `heads.gif`; 5th adds `sprites.gif`; 1.21 brings the baddy, head, shield, sword, bomb and bow images; 1.33 moves the core sheets from `.gif` to `.png`; 2.00 adds lights, gates, doors and the tutorial maps; the NewWorld betas swap to `bodynw.png`, `pics1nw.png`, `spritesnw.png`, `newsword.png`; 2.17.1 moves the remaining item images to `.png` and adds the Kingdoms body parts, `picso.png` and `graal2002letters.png`; 2.31 drops `graalmap.png`, `shield1.gif`, `ss_arrow.png`, `ss_bomb.png` (→ §3.10)[191]
Fonts named: `Courier`, `MS Sans Serif`, `System` from 1st Graal; `Courier` dropped in 6th; `Courier` and `Courier New` from 1.34; `Arial` from 2.17.1[191]
In the shared Drive archive (73 exes in 55 folders) every PE timestamp is the same dummy 1992 date, so only file-version strings date the builds; it holds 2.00 and its revisions 1 to 3 but no 2.1.x release, 2.17, 2.22 or 2.31 (→ §1.1.1)[12,12]
Under Wine the 2.0x exes from that archive stop at "Cannot open file ... commands.rtf" until another build's data files are put beside them[12]

4.11.11 Android Classic and Era, 1.6 to 2.2

The later Android packages are their own line and were only read statically, so nothing in this section is projected onto the period clients. Seven distinct packages were found among ten files; the host details come from Classic 1.7 and the engine details from Classic 1.8 x86.[24]

The Java host owns the lifecycle, the EGL surface, input and audio, and calls the native frame entry over and over; it swaps buffers only when the engine says a frame is ready. Touch types are 1 for down, 2 for move and 3 for up, and other pointers are sent as moves while one changes.[24]

Classic 1.8 gates each frame on elapsed time, 50 ms by default. A slightly late frame moves the deadline on by one interval; a very late one resets it to now, so frames are not caught up. A passing frame handles input, then graphics state, then timers, then drawing. This is a configured gate, not a measured rate.[24]

The wall test checks NPCs, then bombs, then chests, and only then tiles; an option adds other players. The board is 64 by 64 tiles of two bytes, and the neighbour accessor wraps coordinates to their low six bits. Animation part offsets are divided by 16 in the world modes read. Outgoing global chat is cut to 220 bytes and prefixed with one byte equal to the length plus 32; incoming messages go through a 256-entry handler table.[24,24,24,24]

Classic 1.8 and later ship GANIs that are not plain text, and decode trials did not reproduce known plain text, so no decoder is claimed.[24]

4.11.12 What the early Windows clients show when started

The research owner started the v0, v1 and v2 Windows clients offline in a Wine lab and reviewed the screenshots. This is a check of what each build draws, not a timing or movement measurement.[280]

All 18 v0 builds show their start screen and 16 reach an offline scene; 4th Graal alpha and 5th Graal draw only part of it, 9th Graal stays at the start screen and 9th Graal alpha stops on a missing `heads.gif`. Only 12 run summaries record a successful Start against 16 drawn scenes.[280]

17 of 18 v1 builds reach a scene; 1.32 and later start in the welcome cave, and 1.19 stays at the start screen. In v2, 2.04 and the three 2.05 revisions draw the tutorial; 2.00, 2.01 rev 1 and 2.03 stop on a missing `objects.dat`; 2.02 and its revisions show a black window; the stock 2.22 warns of a corrupted executable. Scene and mechanic validation stays open (→ App. E#610).[280,280]

5 Game servers

This part describes the game servers that Graal clients talk to: the original 1999 GServer13, the 2006 39ster 2.1 remake, and GServer-v2 as the reference for later behaviour. Login frames are in → §2.3, ciphers in → §2.2, and the reply sequence after a good login in → §2.3.6. The listserver is in → §6, RC and NC in → §7.

> Status: draft. No official 2000–2002 GServer binary is in the archive, so the 2.x-era server rules here come from GServer-v2 and the 39ster remake, not from an original server.

5.1 GServer13 (1999)

GServer13 is the server shipped with the 1.3 test client in the `graal13b` archive; it was run with its own client under Wine on the loopback address[207].

5.1.1 Options and defaults

GServer13 opens a start dialog before it listens, and saves the values to `serverconfig.txt`[207]:

Key	Dialog label	Default
`serverport`	port	14804[207]
`respawntime`	Respawn time (bushes etc.) in sec	10[243]
`horselifetime`	Horse lifetime in sec	10[243]
`apincrementtime`	Alignment increment time in sec	1200[243]
`baddyrespawntime`	Baddies respawn time in sec	60[243]

GServer 1.28 beta 2 (`GServerv.exe`) has the same four timer keys[243]. An RC of 2000 can set the same four timers over the wire (→ §7.1.4).

5.1.2 Accounts

GServer13 creates every new account from the same template. Each row gives one field of a new account and its default, read from a live run[207].

Field of a new account (Edit accounts)	Default
level and position	`onlinestartlocal.graal` at x 30.5, y 30[207]
AP, MP	50, 0[207]
hearts	3 of 3[207]
rupees, bombs, arrows	0, 10, 5[207]
glove, bomb, bow power	1, 1, 1[207]
sword, shield	1 `sword1.gif`, 1 `shield1.gif`[207]
head, colours	`head0.gif`; orange, white, blue, red, black[207]
weapons, spin attack	weapons enabled, no spin attack[207]

The rights levels are no rights, Graalian Police, Trusted Admin and Administrator[207]. The 1.3 archive does not ship `onlinestartlocal.graal`; the lab copied it from the 15th Graal build, which uses the same `GR-V1.03` format[207]. The account files of this server family are in → §5.4.1.

5.1.3 Login and play

The rules below follow a client through a GServer13 session: the first frame, a refused login, and what arrives after a good one[207].

Rule
The client's first frame is a u16be length and zlib of the id-0 login line with `GSERV014`, account and password (→ §2.3.2)[207]
A wrong account is answered with id 16 and the text "An account with the name and password you typed in doesn't exist."; the client offers Reconnect or Play offline[207]

Rule
After a good login the board goes in a separate zlib frame (100, then a frame with 101 and 8192 bytes); the full reply order is in → §2.3.6.2[207]
The client then asks for each NPC image with id 23; the server answers id 30 (file not sent) for images it lacks[207]
The client sets flags from NPC scripts with id 18 and adds horses with id 7 (from `puthorse`); the server deletes each horse with id 18 about 9.5 to 9.8 s later, matching the 10 s horse lifetime on a 1 s tick[207]
The level leader (the player who sends level NPCs' arrows to the others over UDP) is picked by the server with id 10, sent at the end of the first player's login reply and to the next player when the leader leaves[74]
While a key is held the client sends id 2 with X (prop 15) and SPRITE (17) every 100 ms[207]
Chat is id 2 prop 12; the server relays it; 1.3 draws it with no bubble[207]
Horse deletes and bush regrowth land on the same fractional second, so the server checks its timers once a second[207]

5.1.4 Board changes and respawn

A board change goes through a fixed set of steps on GServer13. The steps below are in order. Step 3 is the one a client author must know, because the sender gets no echo[243].

Step of `PLI_BOARDMODIFY` (1)	Rule
1	needs at least 4 bytes, a current level, and `len ≥ 4 + 2·w·h`[243]
2	writes the new tiles into the server's 64 × 64 board and keeps the old tiles[243]
3	if a tile changed, relays `PLO_BOARDMODIFY` (7) with the client's bytes to every other player in the level; **the sender is not echoed**[243]
4	a change of exactly 2 × 2 gets a countdown `Trunc(Random · R/2 + 3R/4)` with R = `respawntime`; any other size never respawns (1 × 1, 2 × 1, 3 × 3, the 4 × 4 push/pull block)[243]
5	there is no tile check: any 2 × 2 change respawns, whatever the tiles[243]
6	the record joins the "changed since you left" list of every player who left the level since loading it; an earlier record with the same `x, y, w, h` is replaced[243]

Respawn runs from a one-second timer. The rules below give the timer, what a restore sends and when it lands[243].

Respawn tick	Rule
timer	a 1000 ms timer walks every loaded level, empty or not, and decrements each countdown[243]
restore	at < 1 the server removes the record, sends `PLO_BOARDMODIFY` (7) with the **old** tiles to everyone in the level, the original sender included, and writes them back into its board[243]
timing	for R = 10 the countdown starts at 7..12 and the first tick comes 0 to 1 s after the change, so the restore lands 6 to 12 s later; 7 and 12 are half as likely as 8 to 11[243]
measured	60 of 60 2 × 2 changes respawned after 6.21 to 11.98 s, mean 9.21 s; other sizes never respawned; restored tiles were the level's own tiles[243,63]
re-entry	a player with no change list gets the level as on a first visit; otherwise only the queued changes go out and the list is dropped; the exact packet is hard to read[243]
disconnect	the player's change lists are freed[243]

Item drops from cut bushes and pots are rolled by the 1.3 client, not by GServer13[243]; the client side is in → §4.5.

5.1.5 Scripts

GServer13 has player and NPC props, including an NPC nickname field, but no GS1 command table and no `setcharprop` or `showcharacter` strings; NPC scripts of this era are carried to clients and run there[281]. GServer 1.34's Windows binary lists `showcharacter` but not `setcharprop`[281]. Server-side scripting is in → §5.5.

5.2 39ster 2.1 GServer

An open remake of the 1.39 server from December 2006, targeting 2.x clients; one Windows/POSIX C++ process with no script VM[282]. The `ash/` notes describe this tree as written[283].

5.2.1 Process and data

The 39ster server is a small threaded program. Each row gives one part of its process or data, with the rule that governs it[283].

Item	Rule
listen port	14822 by default[283]
threads	a main thread (accept loop with a 1 s `select`, `gameTime` + 1 each second, idle kick after 300 s) and one thread per connection; shared heaps, no locks[282]
player cap	`maxPlayers` 100, never enforced; player ids come from a free list[282]
data files	`data/` is walked once at boot; a file is found by basename, first match wins[284]
accounts	`accounts/<name>`, plain text; loading is a stub that spawns every player on `level13.graal`[282]
weapons	`Weapons.txt`, INI style[282]
level formats	GR-V1.01 (12-bit codes), GR-V1.02 and 1.03 (13-bit codes) (→ §3.1); no `.nw` loader[285,282]

5.2.2 Connections and login

Each row gives how the 39ster server frames and reads one kind of connection. The RC row shows how much of the RC side is only a stub[284].

Item	Rule
frame	u16be length (raw, not a G-integer) + zlib; the inflated body is `\n`-separated messages[284]
player (type 0)	the login line carries a key; every later line drops one byte chosen by the LCG (→ §2.2.2)[282,285]
RC (type 1)	same framing, no descramble; RC ids 51 to 87 are in the table but their bodies are empty stubs[282]
login line	`c` type, `c` key (player only), version (7 bytes for a player, 8 for RC), 1 byte skipped, `str` account, `str` password; the login line itself is not scrambled[284]
descramble	iterator starts at `0x04A80B38`; per line `iterator = iterator · 0x8088405 + key`, then the byte at `(iterator & 0xFFFF) mod count` is dropped[285]
G-integers	1 to 5 bytes, 7-bit groups high first, each + 32 (→ §2.1.1); widths used: 2 for player id and script length, 3 for rupees, kills, deaths and NPC id, 5 for file time[285]
head image	`c` `(100 + n)` then n bytes; an inbound length below 100 is a classic head index, `head{n}.gif`[285]

Besides the named messages, the server sends three special ids. Each row gives one, its role and its payload[284].

Server id	Role	Payload
100	announced length	G-integer size of the next packet[284]
101	raw board	4096 native `short` tiles[284]
102	file	5-byte G-integer modification time, `c` name length, name, raw bytes[284]

The header comments of the 2006 tree give wrong opcode numbers after 83; the `msgFuncs[89]` table and the C enums are the reliable source[283].

5.3 GServer-v2 reference behaviour

GServer-v2 is an open reimplementation, not an original server[243]. Where this part says "GServer-v2", it means the behaviour read in its source or seen in the lab; it is the reference for what 2.x and later clients accept, not a record of the official servers.

5.3.1 Server source lineage

Several server source trees survive. Each row gives one tree, the era it belongs to and how its code is laid out[286].

Tree	Era	Shape
`linuxgserver131`	1.31	one `gserver.cpp` monolith, about 9k lines with zlib headers[286]
`sidrax-v1.4-srv-src-1.3.9`	1.3.9	`gserver.cpp`, `levels.cpp`, `npcs.cpp`, MySQL accounts[286]

Tree	Era	Shape
`oldwin32-gserver1.39source`	1.39	same family, Win32[286]
`gservernwsrc`	NW, about 1.4	objects and Pascal strings[286]
gs2emu (`d3ecab4`)	Reborn 2.x	typed classes, about 18k lines[286]
Reborn binaries 2.1.0 to 2.4.0, GServer revisions 29, 31, 51	2.x, 2.17 line	binaries and default worlds only[286]

The trees group into three eras. Each row gives what an era changed in how a server is built[286].

Era	What changed
1.3.x monolith	one listen socket and one world; login, movement and chat in one file; Pascal strings (`c` length + bytes); MySQL or flat-file accounts; generation 3 at most[286]
gs2emu / Reborn 2.x	`servers.txt` runs several worlds in one process; typed players (client, RC, NPC-server); generations 3, 4 and 5 chosen by the login type; `GRACC001` account files and `GLEVNW01` levels; RC file browser; listserver link (→ §6.5); optional NPC-server process[286]
a Python rewrite of gs2emu, also called Ash	same on-disk layout and wire protocol; generations 3 to 5, client, RC and NPC-server logins, listserver link, accounts, levels with baddies, RC file browser, gmaps, baddies, NPC-server relay; no horses, no weapon bytecode; default port 14812[286,287]

Two different projects carry the name Ash (→ App. E#400).

5.3.2 Login and receive path

The rules below give how GServer-v2 reads a frame and a login, and how the login type picks the generation[288].

Rule
Every multi-byte integer on the player socket is big-endian; frames are u16be length + payload[288]
The login line is `c type`, `c key` (clients, and RC when the generation is above 3), 8 version bytes, `str account`, `str password`; an NPC-server adds a `s` listen port[288]
`type = 1 << value`; the type chooses the generation (client type 0 → 3, type 4 → 4, type 5 and RC2 → 5) and the full type table is in → §2.3.6.1[289]
The key resets both iterators; the start value per generation is `0, 0, 0x04A80B38, 0x04A80B38, 0x04A80B38, 0` and the limit is −1 (→ §2.2)[289]
Inbound generation 1 is plain, 2 and 3 are zlib, 4 and 5 are XOR then decompress; a generation-3 client line has its inserted byte removed before the id is read; the server's outbound generation 3 inserts nothing[289,289]
Login checks, in order: version in `allowedversions` (→ §5.4.2), IP ban, RC account on the staff list, NPC-server password; then the server asks the listserver with `SVO_VERIACC2` and continues when the answer comes (→ §6.5)[289]
An NPC-server login is answered with the NC maps, levels and weapons[289]
A trailing identity string (`win`, hardware ids, OS) is read and forwarded to the listserver[290]

5.3.3 What real clients require of a server

These rules come from running real clients against GServer-v2 builds and fixing what they refused; each is a client requirement, not a server choice[291].

Requirement	Clients
Send `PLO_SETACTIVELEVEL` (156) during the level load; without it the client waits on the loading screen	2.05 (and every Original-type client from 2.01 in the lab fix)[290,291]
Send the first `PLO_LEVELNAME` before the board; after it, the client stays on the loading screen	2.05[291]
Read the rest of the login frame: the client sends `PLI_LANGUAGE` in the same frame as the login line, and dropping it leaves the generation-3 iterator one step behind for the whole session ("Invalid PlayerProp" on 2.17.1)	2.05, 2.17.1[291]
Treat a lone `PLI_PACKETCOUNT` (31) as an ordinary message: these clients send one every 10 s, and a server that reads the next frame as a login kicks them	1.33 to 1.39, and any 2.x client that sends 31 alone[291]
Accept id 25 (the client's weapon upload)	1.33 to 1.37[291]
Send prop 10 in the bow form, not as a gani	1.33 to 1.39[291]
Do not save the hidden status bit (`STATUS` 0x02) into the account; a client that quit hidden logs in invisible	2.x (v4 lab)[291]

Requirement	Clients
Answer a 2.x client's zlib data error: 2.21 to 2.31 have a zlib bug on some chat; GServer-v2 either pads chat with a hidden clientside weapon (`-gr_zlib_fix`) or retries the frame as raw DEFLATE and rolls the generation-5 iterator back one step	2.21 to 2.31[290]
A 5.24 client given the pre-2.1 file path saves PNG data as a file name and never gets files over 64000 bytes; with its own version slot it gets the modification time and the large-file messages	5.24r4[291]

Real 2.05 rev 2, 2.17.1 and 2.22 reach the game and walk on a GServer-v2 build with these fixes, and real 1.33 to 1.39 play on its classic server[291].

5.3.4 Version-dependent paths

GServer-v2 serves clients differently by version. Each row gives a path that depends on the version id and the rule behind it[291].

Path	Rule
version id	the token is looked up in a table; the position in the table is the version id, and comparisons are by id[291]
unknown token	an unknown token allowed by config gets id 0, sorts below everything, and is served every pre-2.1 path[290]
level send below 2.1	the older GServer-v2 master sends a separate 1.41 level sequence with no 156; the newer branch has one level path for every version[290]
default files	`PLI_UPDATEFILE` for a hard-coded list (`idle.gani`, `walk.gani`, `sword?.png`, `body.png`, the stock sounds, `pics1.png` and others) is answered "up to date" without checking times; `PLI_WANTFILE` has no such shortcut[290]
default file to a pre-2.1 client	one build answers `PLO_FILEUPTODATE` (45), the other `PLO_FILESENDFAILED` (30); 2.05 accepts either[290]
player class	chosen from the login type byte (type 0 is the Original class that 2.05 and 2.17 use), not from version config[290]
`generation` setting	server-wide; changes props, baddies, items and NPC details; 1.x clients need `classic`, and 2.x clients on a `classic` server get mangled packets[290,291]

5.3.5 Wire encodings the server writes

Each row gives one encoding the server writes and its limits. A client author can check a decoder against these[288].

Item	Rule
G-integers	c up to 223, s up to 12255, i3 up to 1 568 735, i4, and i5 for file size and time (→ §2.1.1)[288]
Pascal string	c length + raw bytes (the text is not offset)[288]
board	`PLO_RAWDATA` with i3 8194, then `PLO_BOARDPACKET` and 8192 raw tile bytes (64 × 64 u16)[288]
file (2.1+)	`PLO_RAWDATA` with the total, then `PLO_FILE`: i5 time, c name length, name, bytes (→ §2.7)[288]
props	the prop encodings are in → §2.6[288]

5.3.6 Word filter

`rules.txt` is a word filter. The table gives its file, its header and the fields of a rule[273].

Item	Rule
file	`config/rules.txt` is the word filter, not a rules text players see[273]
header	optional `WARNMESSAGE` and `SHOWWORDSTORC`, then `RULE` … `RULEEND` blocks[273]
rule fields	`CHECK` (pm, chat, nick, toall), `MATCH` (? is a wildcard), `PRECISION` (number or percent), `WORDPOSITION` (full, start, part), `ACTION` (log, tellrc, replace, warn, jail, ban), optional per-rule `WARNMESSAGE`[273]
applied to	chat (prop 12), PMs, toall and nickname changes[273]
actions	`replace` swaps the matched letters for *; `warn` replaces the whole message with the warn text; `log` and `tellrc` go to the server log and RCs; `jail` and ban are stubs and do nothing[273]
chat echo	when a chat line was changed, the server sends the new CURCHAT back to the sender so its own bubble shows the filtered text[273]

5.3.7 Board respawn

GServer-v2 respawns board changes on its own terms. The rules below give which changes respawn, after how long, and how a new change clears old ones[243].

Rule
A change respawns only when its top-left tile before the change is one of 11 listed ids (bush, tall grass, pot, sign, stone, black stone and five others); otherwise it is kept with no timeout[243]
A change of any size can respawn, after exactly `respawntime` (default 15 s)[243]
A new change deletes stored changes it fully covers[243]
The change is relayed to the whole level, sender included[243]
Restore swaps old and new tiles and keeps the record, so late joiners get the restored state[243]
Drops, for clients at the 2.1 slot or above: bush, grass or swamp tiles with `bushitems` drop item `rand() % 6` `tiledroprate` % of the time (default 50); a pot with `vasesdrop` drops a heart[243]
With `clientsidepushpull` on, a 4 × 4 change of a push-pull stone is refused and the board is sent back[243]
The real 2.x GServer's timer, tile list and drop rates are unknown; no 2000–2002 official server binary has been found[243]

The 1.42 release moved bush and pot drops to the server[243].

5.4 Accounts and allowed versions

5.4.1 Account stores

Servers keep accounts in different files. Each row gives a server, a file or field, and its layout[292].

Server	Store	Layout
1999 gserver tree	`accountslist.txt`, one account per line	`str` name, `str` password, a flag byte, a 3-byte file number[292]
1999 gserver tree	flag byte	bit 0 only-load, bits 1–3 admin level, bit 4 blocked[292]
1999 gserver tree	`accounts/accN.txt`	name, password, a prop list (`id + 32` and a value), the flag byte, two zero counts, weapons, action flags, opened chests[292]
1999 gserver tree	`accounts.txt`	read if present, imported, then removed[292]
GServer 1.39 with MySQL	`accounts` table	`accname`, `encrpass` (traditional DES `crypt`), `email`, `blocked`, `onlyload`, `adminlevel`, `adminworlds`, `lastused`[293]
gs2emu / GServer-v2	`GRACC001` files	text account files[286]

The rules below record what the 1999 and 1.39 servers did with account data: stat pages, count files and the chat commands players could use[292].

Account rule
The 1999 stat pages rank accounts by props 0 (nickname), 1 (max hearts), 3 (rupees, three 7-bit bytes), 27 (kills), 28 (deaths), 29 (online seconds) and 32 (alignment), and skip only-load accounts[292]
GServer 1.39 servers wrote `playercount.txt`, `accountcount.txt`, `serveruptime.txt` and top lists as `nickname:value` lines for web pages[293]
The 1.39 chat commands listed for players: `sethead`, `setsword`, `setshield`, `setnick`, `showname`, `showap`, `showdeaths`, `showkills`, `showonlinetime`, `showguild`, `showadmins`, `unstuck me`, `toguild:`, `toall:`[293]
GServer-v2's stock default account has `MAXHP 3.00`, which its loader reads as 0 hearts[291]

5.4.2 Allowed versions

The 1999 servers accept a fixed list of `GSERV` tokens with a minimum per connection type (→ §2.3.2). GServer-v2 reads the list from a file:

Rule
Each line of `config/allowedversions.txt` is a token or a range `A:B`; a range allows every token whose position in the version table lies between A's and B's, both included[294]
A token not in the table gets id 0 and lies in no range; a bare unknown token on its own line also resolves to 0, and so admits every unknown token[294,291]

Rule
The refusal is "Your client version is not allowed on this server." followed by "Allowed:" and the list, each token printed by its table name and a range as `A - B`[294]
A server prints its list only to a client it refuses[294]
The stock file of the older master enables only `GNW03014` (2.22)[294]
The newer branch groups ranges by `generation`: `classic` = 1.41r1 only, `newmain` = 2.22 to 2.31, `modern` = 5.12 to 6.037; stock, it refuses 2.05 and 2.17[290,291]
The listserver does not check the version; whether a build gets in is up to each game server's list[294]

A server lets a client in only if its token is on the allowed list. The table maps each late token to its build and the name the version table gives it[294].

Token	Build	Name in the version table
`G3D22067`	5.00r7	5.07[294]
`G3D14097`	5.12	5.12[294]
`G3D26090`	5.30r3, 5.31r4, 6.00r2, 6.00r4	5.31x[294]
`G3D3007A`	6.01x	6.015[294]
`G3D2505C`	6.034, 6.035	6.034/6.035[294]
`G3D0311C`, `G3D0511C`	6.03r7 Windows, Linux	6.037, 6.037 (Linux)[294]
`G3D15010`	5.24r3, 5.24r4, 6.00 beta2	not in the table[294]
`G3D03048`	6.00 beta1	not in the table[294]
`GNW03081`	2.05 rev 2	2.05r2 (newer branch only)[290]

Public servers print their allowed list when they refuse a client. Each row gives one refusal, the token that was sent and the list printed back[294].

Refusal seen from a public server	Client token sent	Allowed list printed
a GServer-v2 server reporting 4.0.0	`GNW14015` (2.3)	2.22, 5.07[294]
a second server (signature 73)	`GNW03081` (2.05)	2.22, and the range 1.41r1 to iPhone Belote 1.5[294]

The second server refused 2.05 although `GNW03081` lies inside its printed range in the current version table (→ App. E#404). No 5.x or 6.x client has logged in to a public server[294].

5.5 NPC server

5.5.1 The 2001 NPC server

The 2001 NPC server runs NPC scripts away from the game server. The rules below give how it connects, how its main loop runs and how it starts[295].

Rule
The NPC server is a separate program that logs into the GServer like a player and owns the NPCs of a configured set of levels[295]
Its main loop runs every 0.1 s: read from the GServer, run all NPC scripts, push changed NPC and player attributes, then sleep; 0.1 s is the smallest useful server-side `sleep`[295]
Startup: load options, connect to the GServer, then (if connected) load the level list, then load NPCs from its database[295]
From the GServer it receives player attributes, temporary level state (bombs, explosions, bush removal, dropped items), PMs, game time, levels to reload, and `server.` flag changes; the GServer decides what reaches clients[295]
To the GServer it sends NPC props, player props changed by scripts, PM replies, weapon grants and warps[295]
The packet ids and port of the GServer–NPC-server link are unmeasured[295]
Scripts run on the server by default; `//#CLIENTSIDE` splits a script, and the part after it is sent to clients[295]
Server-only events: `created`, `initialized`, `playerenters`, `playerleaves` (not on disconnect), `playertouchsme` (needs `setshape`), `playerchats`, `npcwarped`, `exploded`[295]
The server never loads images, so `setshape 1,w,h` gives an NPC its touch rectangle[295]

Variable names carry their scope in a prefix. Each row gives one prefix, what it is scoped to and whether it is saved[295].

Variable	Scope	Persisted
`name`	current script or class	no[295]
`this.name`	current NPC	yes, for database NPCs[295]
`level.name`	the NPC's level	no[295]
`server.name` (number)	this NPC server only	no[295]
flags	current player	yes, in the account[295]
`client.` flags	current player, also sent to the client, which can change them	yes[295]
`this.` flags	current NPC	yes, for database NPCs[295]
`server.` flags	world-wide, shared with other NPC servers of the world, never sent to players	yes[295]

The NPC server's commands fall into groups. Each row gives a group and its commands[295].

Command group	Commands
flow and core	`destroy`, `if`, `while`, `for`, `break`, `continue`, `sleep`, `set`, `unset`, `setstring`, `message`[295]
NPC appearance and state	`setgif`/`setimg`, `setgifpart`/`setimgpart`, `setshape`, `drawoverplayer`, `drawunderplayer`, `dontblock`, `blockagain`, `show`, `hide`, `noplayeronwall`, `showcharacter`, `setcharprop`, `setcharani`, `setchargender`, `putbomb`, `putexplosion`, `putexplosion2`, `setarray`[295]
server-side additions	`canwarp`, `canwarp2`, `cannotwarp`, `callnpc`, `with (players[i])`, `with (getplayer(acct))`, `with (npcs[i])`, `with (getnpcs(name))`, `join`, `warpto`, `tokenize`, `tokenize2`[295]
acting on the current player	`setplayerprop`, `setani`, `setgender`, `say`, `enableweapons`, `disableweapons`, `toweapons`, `addweapon`, `removeweapon`, `setlevel2`, `freezeplayer2`, `unfreezeplayer`[295]
string helpers	`strlen`, `startswith(part,str)`, `indexof(part,str)` (−1 if absent), `#e(start,len,str)`, `#T(str)` (trim), `tokenscount`, `#t(i)`[295]

Some commands exist only on the NPC server. Each row gives one and its rule[295].

Command	Rule
`noplayeronwall`	`onwall` and `onwall2` stop counting players on this NPC[295]
`canwarp`	database NPCs only; the NPC warps through links exactly as a player would[295]
`canwarp2`	database NPCs only; for wandering overworld NPCs: normal links, checks `onwall` at the destination, refuses small links such as house doors[295]
`with`	swaps the current player or NPC for the block; commands are skipped if the target does not exist; local variables still resolve to the caller; `sleep`, `break` and `continue` inside the block leave the swapped context in place[295]
`join class`	loads `class.txt` from the scripts folder; an NPC holds its own script then each joined class; a class loaded by another NPC is shared, so updating it updates every member; later scripts can override functions[295]
`warpto`	moves the NPC to another level; database NPCs only[295]
`addweapon`	like `toweapons`, but the weapon script comes from the database and the weapon must already exist[295]
`tokenize`, `tokenize2`	`tokenize` splits on spaces and commas; `tokenize2` adds extra delimiters[295]

Client support for these commands per build is in → §8.1 and → §8.2; the PM admin interface and the NC socket that replaced it are in → §7.3.

5.5.2 Message codes on a script-running server

On a server that runs scripts, message codes are expanded before use. The rules below give how `#n` is expanded in `setcharprop` and what it changes[281].

Rule
In a `setcharprop` value, #n expands to the nickname of the player who triggered the script; `#n(id)` picks another player by id[281]
`setcharprop` always expands its second argument before applying it[281]
When the first argument is #n, the expanded text becomes the NPC's nickname and its `showcharacter` nickname, and the NPC nickname prop is marked for sending[281]

Rule
#F is the current player's level name, which can differ from #L (the NPC's level)[295]
A capture that proves the server-side #n expansion on a script-running server is still open[281]

5.5.3 Later NPC servers

Later servers handle NPCs in their own ways. Each row gives one server and how it runs NPC scripts[290].

Server	Rule
GServer-v2, newer branch	adds a GS1 NPC server, tested on the 2.x generation; the V8 GS2 NPC server stays optional[290]
GServer-v2, newer branch, own notes	1.38 to 1.411 and the 2.x/3.x generation are supported client-authoritative; clientside GS1 works up to 5.007; server-side GS2 is not supported[290]
the Python gs2emu rewrite	announces the NPC server to clients and relays the address query (→ §7.2)[287]
any GS1 NPC server	the 2.22 built-in baddies can run as server-owned NPCs: one AI step per 0.1 s `timeout`, the client's probe points tested with `onwall`, touch damage with `hitplayer`, and ganis built from `arrays.dat`; a worked gray soldier is not yet tested live[296,133]

6 Listserver

The listserver gives a client the list of game servers and, from 2.22, checks its account. Two links exist: client ↔ listserver (§6.1 to §6.4) and game server ↔ listserver (§6.5)[297]. Framing and ciphers are in → §2.1 and → §2.2. From 4.x the first connection goes to a game server named `Login` instead (→ §2.3.5).

> Status: draft. Real-client captures exist for 2.00, 2.05, 2.17.1, 2.22 and 2.31 against a lab listserver only; the public-service runs used a reimplementation's frames. No 2000–2002 listserver binary or capture exists.

6.1 Requests

6.1.1 Where each build looks for the list

Each build finds its server list in its own way. The table gives the host and port each group of builds uses, and the list name it asks for[80].

Builds	Host and port	List name
1.33 to 1.39	no listserver; the player types a game-server address	n/a[80,291]
1.41	unmeasured: only a patched copy survives	unmeasured[80]
1.42	`<listserver-host>` built in, port 14921	`newmain`[80]
2.00 to 2.05	built in as a DES-hidden string assembled one character at a time, port 14921; `arrays2.dat` is not read	`newmain`[297,80]
2.1 test builds	port 14922 built in	unmeasured[80]
2.17 to 2.31	`arrays2.dat`, checked against `arrays2.sig`: line 0 host, line 1 port (14922), lines 5 and 6 fallback hosts, tried in order (→ §3.5)	`newmain`[297,297]
RC 1.41 (2002)	a DES-hidden host to which the code appends the top-level domain, port 14922	`rc`[298]

The 2.22 client keeps the hosts from `arrays2.dat` in memory under a one-byte transform, so a plain memory search misses them[297].

6.1.2 Client → listserver ids

These are the messages a client sends to the listserver. Each row gives the id, its name and its layout[297].

Id	Name	Layout
0	V1VER	`rest` = list name, such as `newmain` (RC 1.41: `rc`)[297,298]
1	SERVERLIST	`str account`, `str password`[297]
4	V2VER	`c` key + `newmain`; selects the 2.2 bz2 mode[297]
5	V2SERVERLISTRC	`c` key (RC)[297]
7	V2ENCRYPTKEYCL	`c` key + `GNW30123` + `newmain`[297]
223	GRSECURELOGIN	a Graal Reborn extension[297]

6.1.3 Login shapes

The listserver login differs by client generation, not by build; inside a generation every observed build sends the same bytes[80].

Shape	Builds seen with a real client	Client → listserver
A: list name only	2.00 (client side), 2.05 (port 14921)	one plain zlib frame: id 0 + `newmain` + `\n` (9 bytes decoded, 17 on the wire); no account, password, token or key; the client then closes[80]
B: id 0 then id 1	2.17.1 (port 14922 from its signed `arrays2.dat`)	one plain zlib frame holding id 0 `newmain` and id 1 (`str account`, `str password`); no token, key or cipher[80,297]
C: id 7, then generation 5	2.22, 2.31 (port 14922)	frame 1, plain zlib, 18 bytes decoded: id 7, one key byte, `GNW30123`, `newmain`, `\n`; frame 2, generation 5 with compression byte `0x02`: id 1, XORed with the key[80]

Shape	Builds seen with a real client	Client → listserver
D: 6.113 on port 14900	Windows 6.113, one owner-supplied capture	not a list frame: token `GNP1905C`, then ciphertext; no id 0, 1 or 7 and no `newmain`[49]
E: 6.113 full capture	Windows 6.113, the full capture the same day	the same hello on TCP 14900 and on TCP 14882; still no clear list id. The hosts that answer send no UDP[50]
generation 4	2.19 to 2.21	id 4 `c` key + token, then bz2 frames with partial XOR (limit 4)[297,291]
RC 1.41	RC 1.41 (2002)	id 0 `rc`, then id 1 `str account`, `str password`[298]

The rules below say which tokens go to the listserver and which go to a game server, and what the classic frames can tell about the build[297].

Rule
`GNW30123` is the listserver token of 2.22 and 2.31, shared by both; it is not a game build token (2.22 logs in to a game server as `GNW03014`, 2.31 as `GNW28015`)[297,80]
Per-build tokens such as `GNW24011` (2.00), `GNW03081` (2.05) and `GNW22122` (2.17.1) go to the game server, never to the listserver[80]
The classic frames (A and B) carry no field that could name a build[80]
The real clients compress at zlib's best level (stream starts `78 DA`); a `78 9C` stream with the same deflate body is answered by the public service for shape C[80]
The 2.05 list connection derives from the game connection's base class and keeps its base send path, which has no generation-3 insertion[297]
A real 2.00 sends one frame (id 0 + `newmain`), makes no second listserver connection, and accepts a 2.17-style two-frame reply from a lab forwarder[297,98,60]
After the list, 2.17 logs in to the game server as type 0 with a key, `GNW22122`, account and password, in generation 3[297]
What 1.41, 1.42, 2.01 to 2.04 and 2.1 to 2.16 send is unmeasured; 1.42 and 2.01 to 2.04 share 2.00's built-in list data and are expected to send shape A[80]

6.2 Replies

6.2.1 Listserver → client ids

These are the listserver's replies. Each row gives the id, its name and its layout; two are disputed and point to App. E[297].

Id	Name	Layout
0	SVRLIST	`c count`, then one record per server (→ §6.4)[297]
1	NULL	disputed → App. E#401[297,298]
2	STATUS	disputed → App. E#401[297,298]
3	SITEURL	URL[297]
4	ERROR	text; the connection closes[297,298]
5	UPGURL	donate or upgrade URL[297]

The rules below give how 2.05 handles those replies and fills its server window[297].

Rule	Builds
The receive handler switches on ids 0 and 1 only	2.05[297,278]
A client fills the list as soon as id 0 arrives after its id 0 `newmain`; it does not wait for id 1, and closes the socket about 2 s after it gets the list	2.05[297]
The window shows the fields in record order: name, language, description, URL, version, players, address, port	2.05[297]
The list client acts on ids 0 and 1 only; any other id, including 2, 3 and 5, is ignored	2.00[299]
Id 1 carries one server record in the id 0 record layout and appends it to the list; when it is then the only row, it is selected	2.00[299]
After id 0 the client fills the list and closes the list socket; a row is drawn only with at least 6 fields, and the detail view needs at least 8	2.00[299]
RC 1.41 reads 0 (server list), 1 (one more server entry), 2 (end, disconnect) and 4 (error text, disconnect), and shows name and player count	RC 1.41[298]

Rule	Builds
The 2004 RC logs in with client id 5, an empty token and an empty list name, then `str account`, `str password`; it shows name and player count from an id 0 list, and id 4 text in a message box	RC 2004[300]
2.22 shows the status lines and the Show More and Donations buttons, and files tagged servers under tabs (→ §6.4)	2.22[291]

6.2.2 Reply frames seen

Replies were captured from three kinds of login. Each row gives one login shape and the frames that came back[297].

Login	Reply
shape B from a real 2.17, lab listserver	two plain zlib frames: id 0 (the list); then id 2 status, id 3 site URL, id 5 donation URL[297]
shape C, port 14922 of the public service	two generation-5 frames with compression byte `0x04` (zlib): id 0 list; then id 2 status, id 3 site URL, id 5 upgrade URL[301]
shape C, port 14923 of the public service	three generation-5 frames: the legacy list first, sent straight after the id 7 frame before the client's id 1, then the two frames above[301]
classic (shape B frame), port 14923 of the public service	one plain zlib frame, id 0 only, no status, site or upgrade lines, no trailing NUL bytes; the server then waits[301]

The table gives details of those reply frames that a decoder must match: trailing bytes, the status frame and its count[301].

Detail	Value
trailing bytes	generation-5 frames end with two `0x00` bytes after the last `\n`; the plain legacy frame does not[301]
status frame	id 2 text of two lines joined by CR (a welcome line and "N server(s) online"), id 3 site URL, id 5 upgrade URL, `\n` after each[301]
status count	the number in the status text is the server's own text and need not match the row count[301]
timing, port 14922	both frames about 0.4 s after the client's second frame, on a link with a 0.2 s round trip[301]
timing, port 14923, shape C	the legacy list within a few ms of the handshake; the two current frames about 0.35 s after the client's id 1[301]
timing, port 14923, classic	about 0.2 s after the login[301]

6.2.3 The classic `serverlist3` listserver

The classic `serverlist3` listserver survives as source. The rules below give its ports, framing and how it filters the list[302].

Rule
One process with two listen ports, one for clients and one for game servers, set by `clientsport` and `serversport` in a `key=value` config; also `playerlimit`, `databaseserver` and `showmore`[302]
Frame: u16be length + zlib; lines start with `id + 32`; strings are `count + 32` and the bytes; an inflated buffer past 16383 bytes or an outgoing buffer past 4095 bytes counts as a broken link[302]
Client id 0 is the edition text, which filters the directory to rows whose edition contains it and that were written in the last 10 minutes; id 1 is account and password; other ids are ignored; there is no key, token or cipher[302]
Passwords are checked locally against a `crypt` hash of the first 8 characters with a two-character salt[302]
A hard failure (unknown account, wrong password, ban) is id 4 with the text, then the link closes[302]
A success is id 0 (the directory) and id 2 (status text), then the link closes; the trial and free-server texts also send id 3 first with the `showmore` URL[302]
Status texts: a welcome line, a staff sentence, a remaining-time sentence, the trial sentence or the free-server sentence[302]

6.3 Login-type matrix

On 2026-10-25 the public listserver was asked for its list 38 times, once per port for 19 version profiles of a reimplementation, with no game-server login[303]. The service chooses its reply by **login type**, not by version or token[303].

Login sent	Profiles	Port 14922	Port 14923
classic: one plain zlib frame with id 0 `newmain` and id 1	1.41, 1.42, 2.00, 2.01, 2.02, 2.02 rev 2, 2.03, 2.04, 2.1, 2.12 to 2.17	connection accepted, no reply and no close for the 5 s timeout, in all 15	legacy list in all 15: one plain zlib id 0 frame, 2 or 4 rows[304,303]
generation 5: id 7 + `GNW30123` + `newmain`, then id 1	2.22, 2.3, 2.31, and the 2.05 profile of the run	modern list (8 rows) with status, site and upgrade lines	the legacy list first, then the modern list (12 or 17 rows in all) with the status lines[304,303]
any	2.05 profile	port 14921: connection refused at once	n/a[305,301]

The public service was tested with each login type. The rules below say which port gives a list and what varied between runs[303].

Rule
A port that gets a list: a classic login on 14923; a generation-5 login on 14922 (14923 also answers it)[303]
The legacy list's row count varied between runs (2 or 4) with no link to the version; the 2-row form carried `P` and `H` prefixes and the 4-row form none[303,80]
The 2.05 rows of the matrix are a generation-5 login, not a classic 2.05 login; a classic 2.05 login was not run[80]
No real client of these versions was captured against the public service; the runs say which login type the service answers on which port, not what a real 1.41 to 2.17 client sends[303]
The 2001-era server tools list clients on 14922 and older clients on 14923 (`clientPortOld`)[293]

Why the public 14922 ignores a classic login, and what it does with a real 2.17.1, is open (→ App. E#403). Whether the 14923 legacy list is a real 2000-era reply is disputed (→ App. E#402).

6.4 Server-list records

Each server in the list is one record of eight fields. The table gives each field's content in order[301].

Field	Content
—	each record starts with `c` field count, 8 in every capture[301]
1	type prefix and name[297]
2	language[297]
3	description[297]
4	URL[297]
5	version text[297]
6	player count, as text[297]
7	address, as text[297]
8	port, as text[297]

Every field is a `str`[301]. The list body ends with `\n` after the last record[301].

Type prefix	Tier or tab	HQ level in the lab listserver
`U`	hidden	0[297,291]
`H`	bronze (hosted)	1[297,291]
none	silver, shown as "Classic" by 2.17.1	2[297,291]
`P`	gold	3[297,291]
`3`	3D	4[297,291]

The rules below give how prefixes appear, how a client uses a record to connect and one asset check that blocks it[297].

Rule
Pre-2.2 clients get no prefix for bronze[297]
Connect needs a selected record with at least 8 fields and takes the address and port from fields 7 and 8; with either empty it shows "The connection could not be established!" without dialling (2.00)[299]

Rule
A missing core asset (`objects.dat`, `arrays.dat` or one of the main images) sets a fatal flag at Start that is never cleared; the server list still works, but Connect then returns before creating the player, leaving a grey game window with no socket (2.00)[299,299]
The public 14923 legacy list sent untagged rows in its 4-row form; 3 of 4 rows also appear in the current list with the same address, port and version text[301]
A 2003 fan listserver sets the P prefix for a server marked `Favorite=Yes`, so the client files it under Gold[293]
Field bytes are passed through as sent: a captured address field had lost its leading characters, and a name carried three literal `0x3F` bytes[301]

6.5 Game server link

Game servers talk to the listserver over a separate link. The rules below give its framing, port and opening lines[288].

Rule
Listserver packets on this link have no length prefix: `id + payload + \n`[288]
Game servers register on port 14900 in the Reborn listserver[293]
A GServer-v2 opens with a plain `REGISTERV3` line, then zlib frames: `NEWSERVER`, `SERVERHQPASS` / `SERVERHQLEVEL`, `SETPLYR` and `PLYRADD` per player, `VERIACC2` per login, and `SENDTEXT` with its allowed versions; the older master never pings[290]
The newer GServer-v2 sends `SETIP` every 60 s as a keep-alive, and its `VERIACC2` carries a s identity length[290]
Id 16 from a GServer-v2 is a pong and 99 is the listserver's ping[291]

The table gives the ids on that link, in each direction[288].

Direction	Id	Name
server → list	0	SETNAME[288]
server → list	7	SETPLYR[288]
server → list	14	PLYRADD[288]
server → list	15	PLYRREM[288]
server → list	16	PING[288]
server → list	17	VERIACC2[288]
server → list	22	NEWSERVER[288]
server → list	23	SERVERHQPASS[288]
server → list	24	SERVERHQLEVEL[288]
list → server	8	ERRMSG[288]
list → server	11	VERIACC2[288]
list → server	15 to 17	FILESTART3, FILEDATA3, FILEEND3[288]
list → server	19, 20	request and send text[297]
list → server	29	PM player[297]
list → server	30	assign PC id[297]
list → server	99	PING[288]
list → server	100	RAWDATA[288]

The other server-to-list ids (0 to 31) set description, language, version, URL, address, port and player count, verify guilds, get and set profiles, request files, the list, server info and buddies, and PM a player[297]. No open listserver serves files to a game server[290].

Classic `serverlist3` game-server link
The first line is the login: kind 0 or 1, then `GLSER001` or `GLSER002`, then the world name; the socket address must match the address stored for that world[302]
Id 4: player add or update (2-byte id, a flag byte, c length, name); with flag bit 1 set and kind 0 the listserver answers id 7 with the player id and, when queued, "N ghost(s) before you"[302]
Id 5: player leave (2-byte id)[302]
The public directory is a database table, not built from these messages; the link carries presence, the ghost queue and the address check[302]

The Reborn listserver keeps its data in tables. The rows below give what it stores and how it hashes passwords[293].

Reborn listserver data
Tables for users (account, password, salt, activation, ban, profile fields), servers (name, address, port, players, description, URL, language, a two-character type, version, uptime), per-server HQ password and level, IP bans, guilds and guild members[293]
Account passwords are `md5(md5(password) + salt)` with a three-character salt[293]

7 RC and NC

RemoteControl (RC) is the staff client for a game server; NPC Control (NC) is its connection to the NPC server (→ §5.5). Both are Delphi programs that reuse the game client's connection class[298]. The game-server login types are in → §2.3.6.1, framing in → §2.1.4 and ciphers in → §2.2. Staff ids that game clients also handle are in → §2.4.10 and → §2.5.9.

> Status: draft. Every layout here is a static read of the RC binaries, except those in → §7.1.8, which one capture of the 2004 RC against a GServer-v2 fork shows. The checks that would settle the rest are listed in the protocol note's loopback list.

7.1 RC login and commands

7.1.1 Builds

RemoteControl went through many builds. Each row gives one archive, its date, its login token, its number of forms and what it added[306].

Archive	Title date	Login token	Forms	Adds
rc133	none	`GSERV014`	7	accounts, players, send, update levels[306]
rc139rev2	2000/8/19	`GSERV019`	15	chat window, flag lists, toalls, SQL account list, options, warp[306]
rc1025	2000/10/25	`GSERV020`	15	same forms[306]
rc62101	2001/01/27	`GSERV021`	15	same forms[306]
rc8212001	2001/08/20	`GSERV022`	18	rights, comments, local ban; server options leave the start dialog[306]
rc201	2002/01/04	`GSERV022`	18	new main window layout[306]
rc141	2002/05/02	`GSERV023`	26	NPC Control, server list, file browser, short message popup[306,298]
RemoteControl in the Graal Reborn folder (2004)	n/a	`GSERV024`	25	RC and NC in one program, file browser, server list[307]

The archive names do not sort by date: rc141 is the newest of the 2000–2002 set[306]. RemoteControl builds of 1999 log in with `GSERV007` (shipped with 1.21) and `GSERV011` (shipped with 1.28 and 1.3)[74]. A GServer-v2 also knows `GSERV025` (→ App. E#405).

7.1.2 Login

The RC login changed in 2004. Each row gives a group of builds, the first message and the framing during and after login[306].

Builds	First message	Mode during login	Mode after
1999 to rc141 (2002)	type 1, token, `str account`, `str password`; no key byte	u16be length + zlib, no cipher, no type byte	unchanged[306,298]
2004	type 6, `c key (rand & 0x7F)`, `GSERV024`, `str account`, `str password`	u16be length + zlib, no cipher, no type byte	generation 5 (type byte + partial XOR)[307]

The rules below explain the RC2 name, how the tokens are stored and the defaults in the connect dialog[307].

Rule
Type 6 is what the server calls RC2; the 2004 Delphi RC already uses it, so "RC2" in server tables means the generation-5 RC login, not the later GTK program[307]
The 2004 tokens are plain string constants, unlike the game clients' tokens built one character at a time[307]
The connect dialog of every build has a built-in default account; the start dialog up to 2001/01/27 defaults to the loopback address and port 14804[306]
From 2002/05/02 the address fields are gone and the server is picked from the listserver (→ §6.1.1)[306]
In the 39ster 2.1 remake an RC connection (type 1) skips the per-line descramble (→ §5.2.2)[282]

Rule
A GServer-v2 refuses an RC whose account is not on the staff list[289]

Right after login the RC sends one `PLI_PLAYERPROPS` (2):

Builds	Props
2000/8/19 on	0 (NICKNAME) = the nickname from the options; closing the player list with "Go in away mode" sends it again with `-away` appended[306]
rc141 (2002)	33 (ADDITFLAGS) = 1 only when "Ignore mass messages" is ticked; 0 (NICKNAME); 53 (PSTATUSMSG) = the selected status index[298]
2004	a short message whose prop list is not traced[307]

7.1.3 Framing

RC framing is the game framing without the cipher. Each row gives a group of builds and its rule[307].

Builds	Rule
all	messages are `c id` + payload + `\n`, G-integers + `0x20`, batched and flushed at 0x1000 bytes[307,298]
2000 to 2002	u16be length + zlib (zlib 1.1.2); no cipher, type byte, key byte or per-message insertion[306]
2002	large uploads: id 50 (`PLI_RAWDATA`) with an `i3` length, a flush, then one frame of `c id` + header + raw bytes with no `\n`; receiving id 100 switches the reader to raw mode for that many bytes[298]
2002	more than 256 KB queued unsent drops the socket[298]
2004, RC socket	client-side generation 5: each flush picks raw, zlib or bz2 by size, writes a type byte, and XORs the first 12 or 4 dwords with the `0x04A80B38`-seeded LCG plus the key, one step per 4-byte group present; send and receive keep separate iterators with the same key (→ §2.2.3)[307]

7.1.4 Start dialog and server timers

Up to 2001/01/27 the start dialog asks for four server timers, each with a "don't change" box; each value goes out after login as an `i4`, only when its box is unticked[306]:

Field	Default	Id	Name
Respawn time (bushes etc.) in sec	10	55	RC_RESPAWNSET[306]
Horse lifetime in sec	10	56	RC_HORSELIFESET[306]
Alignment increment time in sec	1200	57	RC_APINCREMENTSET[306]
Baddies respawn time in sec	60	58	RC_BADDYRESPAWNSET[306]

These are GServer13's four timer options (→ §5.1.1). From 2001/08/20 the timers are gone from the dialog and 55 to 58 are no longer sent[306].

7.1.5 Client → server ids by build

Constant ids at the RC socket's send call sites; ids sent through a computed value are not included, so each row is a floor[306,307].

Build	Ids
rc133	1, 51–64[306]
2000/8/19, 2000/10/25	1, 2, 6, 28, 55–58, 61–64, 67–79, 82[306]
2001/01/27	1, 2, 6, 28, 55–58, 61–64, 68, 70–74, 76–79, 82[306]
2001/08/20	1, 2, 6, 28, 61–64, 68, 70–79, 82–88[306]
2002/01/04	1, 2, 6, 28, 61–64, 68–79, 82–88[306]
2002/05/02	1, 2, 6, 28, 51–54, 61–64, 68–79, 82–92, 94, 96–98, plus the NC ids (→ §7.3.2)[306]
2004	2, 6, 28, 51–54, 61–64, 68–76, 78, 79, 82, 84, 86, 88–92, 94, 96–98[307]
2004, on the wire	also 83, 85 and 87 (rights, comments and ban get), with 73 and 76; the static row is a floor[300]

The later names cover server options, folder config, player props get and set, accounts, bans, rights, comments and the file browser[101]. Ids 51 to 54 changed meaning:

Id	rc133 use	Later name
51	account list: Edit (get account)	RC_SERVEROPTIONSGET[306]
52, 53	edit account: Apply	RC_SERVEROPTIONSSET, RC_FOLDERCONFIGGET[306]
54	account list: Delete	RC_FOLDERCONFIGSET[306]
59	player list: Edit (get player props)	RC_PLAYERPROPSGET[306]
60	edit player: Apply	RC_PLAYERPROPSSET[306]

From 2000/8/19 accounts move to 70 to 78, and 51 to 54 are unused until 2002/05/02 brings them back as server options and folder config; a server that talks to rc133 cannot use the later names for 51 to 54[306].

2004 file operation	Rule
folder change	id 0x5a[307]
upload	framed by ids 0x9b, 0x5d, 0x9c[307]
delete	needs the rights string to contain `w`[307]

7.1.6 Server → RC ids

The 2004 RC handles a fixed set of ids from the server. The row lists them[307].

Builds	Ids handled on the RC socket
2004	13, 16, 30, 35, 37, 55–59, 61–70, 72–74, 76, 77, 79, 100, 103, 180[307]

The table gives each server-to-RC id with its name and layout[101].

Id	Name	Layout
13	TOALL	`s player id`, `str message` (→ §2.8)[101]
16	DISCMESSAGE	`rest` = text; a disconnect box[298]
35	RC_ADMINMESSAGE	`rest` = `Admin <account>:\xa7<text>`[101]
37	PRIVATEMESSAGE	`s from id`, then a quoted, comma-separated type label and message (→ §2.8)[101]
47	STAFFGUILDS	`"guild"`, repeated, sent at login[101]
0x37	(player list)	creates or updates a player row: a 14-bit id, then a prop stream (the note gives the id in hex only); defaults are 3 hearts, `sword1.gif`, `body.png`, `onlinestartlocal.graal`[307]
74	RC_CHAT	an RC console line[101]
79	NPCSERVERADDR	`s player id`, then `ip,port` (→ §7.2.2)[298]

Players in jail can PM only RCs[101]. The layouts of the other RC ids are unmeasured in the notes[307], except those seen on the wire in → §7.1.8[300].

7.1.7 Features and local settings

RC features were added build by build. Each row gives a build and what it added[306].

Build	Feature
rc133	player list (nickname, account, level, head, id), disconnect, edit player, message, message to all, account list (add, delete, edit), update levels; the account form already carries rights (Load only, Graalian Police, Trusted Admin, Administrator, No rights, Blocked), look, position, items, weapons, opened chests, NPC flags, MP, AP, spin attack[306]
2000/8/19	chat window (RTF), server flags, toalls, SQL account list with a filter, message editor, options (ignore mass messages, no HTML in PMs, no images in PMs, away on close), warp player[306]
2001/08/20	edit rights, edit comments, local ban[306]
2002/05/02	NPC Control (NPC list, add NPC, local NPCs, classes, weapons, level list), listserver login and server list, file browser with drag and drop, short message popup[306]

RC keeps its settings in two places. Each row gives a setting and where it is stored[298].

Setting	Where
options: `nickname=`, `nomassmessages=`, `nohtmlinpms=`, `nohtmlimages=`, `attachaway=`	`controlconfig.txt` beside the exe; the later GTK RC2 uses `control2config.txt`[298,307]
account (plain) and password (DES-obfuscated, 48 printable characters)	registry `HKCU\Software\Graal\RemoteControl`[298,306]

The password scheme is a 24-byte block (length byte, up to 23 bytes of text, random padding), encrypted as three DES ECB blocks with an LSB-first key, each output byte written as two nibbles + `0x20`[298]. The same scheme, with other keys, hides the RC 1.41 and 2.05 list hosts and the 1.1 guild files[298,293].

7.1.8 Player admin on the wire (2004)

One session of the 2004 RC against a GServer-v2 fork, with one 2.22 player online; each row is a single observation[300]. Notation as in → §2.1.

Step	Rule
after login	server → RC in this order: 25, 194, 74 three times, 190 (empty), 47, 180, 103, one 55 per player online, then 74 `Currently online: ...`[300]
after login	the RC then sends 2 PLAYERPROPS twice and the server answers with 9[300]

Player administration was captured on the wire from a 2004 RC. Each row gives a direction, an id and its layout[300].

Dir	Id	Layout
S→RC	103 RC_MAXUPLOADFILESIZE	`i5` bytes; 20971520 seen[300]
S→RC	55 ADDPLAYER	`s player id`, `str account`, then props as id + value[300]
RC→S	73 RC_PLAYERPROPSGET2	`s player id`[300]
S→RC	72 RC_PLAYERPROPSGET	`s player id`, `str account`, `str world`, `c n`, n bytes of props, tail[300]
RC→S	76 RC_PLAYERPROPSSET2	`str account`, `str world`, `c n`, n bytes of props, tail, then one more byte (0 seen; → App. E#421)[300]
both	props tail	`s`, `s` (flag and chest counts, order unseen → App. E#420), `c weapon count`, `str` per weapon[300]
RC→S	83 RC_PLAYERRIGHTSGET	account as `rest`[300]
S→RC	62 RC_PLAYERRIGHTSGET	`str account`, `i5 rights`, `str IP range`, `s n` + n bytes of folder rights[300]
RC→S	84 RC_PLAYERRIGHTSSET	same layout as 62; folder lines as CSV (→ App. E#422)[300]
RC→S	85 RC_PLAYERCOMMENTSGET	account as `rest`[300]
S→RC	63 RC_PLAYERCOMMENTSGET	`str account`, comments as CSV (`rest`), one item per text box line[300]
RC→S	86 RC_PLAYERCOMMENTSSET	`str account`, comments as CSV (`rest`)[300]
RC→S	87 RC_PLAYERBANGET	account as `rest`[300]
S→RC	64 RC_PLAYERBANGET	`str account`, `c banned`, reason (`rest`)[300]
RC→S	88 RC_PLAYERBANSET	`str account`, `c banned`, reason as CSV (`rest`)[300]

The rules below record what the capture showed: which props the RC sends back, how rights map to bits, and what the ban menu offers[300].

Observation	Rule
props sent back	the RC's 76 leaves out props 0, 30, 34 and 36, adds 21, 22 and 23 as one zero byte each, and sends 29 as 0[300]
rights bits	the dialog's check boxes map to bits from bit 0; "Warpto x y" and "Admin message" gave 513 (bits 0 and 9)[300]
ban menu	the player menu has Edit Ban only: the account ban, with no IP or computer id ban[300]

7.2 NC login

7.2.1 Login and framing

NPC Control logs in on its own socket. The rules below give its first message and framing[298].

Rule
The NC socket's first message is type 3, `NCL11012`, `str account`, `str password`, with no key byte, sent right after the socket connects[298,308]
The NC socket stays in fixed zlib framing (u16be length + zlib) for the whole session: no type byte, no cipher[307,308]
The NC login did not change between rc141 (2002) and the 2004 RC[308]

Rule
A GServer-v2 also knows the NC token `NCL21075` (→ §2.3.6.1)[89]
Status bar text after login: "Connected to NPC-Server"[298]

7.2.2 Finding the NPC server

The RC never asks for an NC address[298]:

Step	Rule
1	any player whose nickname contains `(Server)` is treated as an NPC-server player[298]
2	for each, the RC sends id 94 (`PLI_NPCSERVERQUERY`): `s player id` + the text `location`, when the player appears and again whenever the NC socket drops[298]
3	the server answers id 79 (`PLO_NPCSERVERADDR`): `s player id`, then `ip,port`[298]
4	the RC opens the NC socket only if it is not already open and that player's nickname still contains `(Server)`, then sends the NC login[298]

The rules below give how NPC Control finds the NPC server and what the 2001 texts say about it[298].

Rule
The binary's default NC fields (the loopback address and 14804) are leftovers of the old start dialog and unused once discovery works[298,308]
The 2001 NPC Control text says the NewWorld game server exposed NC on port 14950; NewWorld accounts could connect, main-server accounts needed admin level 4 or higher[298]
The 2001 bundle says adding, viewing and changing classes did not work yet[298]

7.3 NPC, class and weapon messages

On the NC socket, received ids 33, 80, 157 to 164 and 167 go to the NC handler; everything else, 16 included, goes to the RC handler[298].

7.3.1 NC → client

These are the ids NPC Control receives. Each row gives the id, its name, its layout and what it opens[298].

Id	Name	Layout	Effect
16	DISCMESSAGE	`rest` = text	disconnect box[298]
33	NPCWEAPONADD	`str name`, `str image`, `s len`, `len` bytes of script	weapon script editor[298]
80	NC_LEVELLIST	`rest` = level list, one per line	level list editor[298]
157	NC_NPCATTRIBUTES	`rest` = attribute text, one per line	attribute viewer (view only)[298]
158	NC_NPCADD	`i3 id`, then props in any order and any subset: `c 50` + `str name`, `c 51` + `str type`, `c 52` + `str level`	adds or updates an NPC list row[298]
159	NC_NPCDELETE	`i3 id`	removes the row[298]
160	NC_NPCSCRIPT	`i3 id`, `rest` = script	script editor[298]
161	NC_NPCFLAGS	`i3 id`, `rest` = flags, one per line	flag editor[298]
162	NC_CLASSGET	`str name`, `rest` = script	class editor[298]
163	NC_CLASSADD	`rest` = class name	adds to the class list[298]
164	NC_LEVELDUMP	`rest` = text	dump viewer (view only)[298]
167	NC_WEAPONLISTGET	repeated `str name`	weapon list[298]
188	NC_CLASSDELETE	class name, form unmeasured	handled from 2004, not in 2002[308]

7.3.2 Client → NC

These are the ids NPC Control sends. Each row gives the id, the action that sends it and its payload[298].

Id	Name	From	Payload
3	(login)	connect	→ §7.2.1[298]
79	RC_CHAT	a console line typed in NC mode	text[298]
103	NC_NPCGET	NPC list: open, Enter, Attributes	`i3 id`[298]
104	NC_NPCDELETE	NPC list: Delete, after a confirm	`i3 id`[298]
105	NC_NPCRESET	NPC list: Reset	`i3 id`[298]
106	NC_NPCSCRIPTGET	NPC list: Script	`i3 id`[298]
107	NC_NPCWARP	Quick warp or the warp dialog	`i3 id`, `c x×2`, `c y×2`, level[298]
108	NC_NPCFLAGSGET	NPC list: Edit flags	`i3 id`[298]
109	NC_NPCSCRIPTSET	script editor: Save	`i3 id`, script[298]
110	NC_NPCFLAGSSET	flag editor: Save	`i3 id`, flags[298]
111	NC_NPCADD	Add NPC dialog	comma text: name, id, type, scripter, start level, x, y; type is one of `OBJECT`, `ANIMAL`, `POLICE`, `HORSE`, `BOAT`, `HOUSEC`[298]
112	NC_CLASSEDIT	class list: open	class name[298]
113	NC_CLASSADD	class editor: Save	`str name`, `rest` = script[298]
114	NC_LOCALNPCSGET	Local NPCs dialog	level name[298]
115	NC_WEAPONLISTGET	console button	empty[298]
116	NC_WEAPONGET	weapon list: open	weapon name[298]
117	NC_WEAPONADD	weapon editor: Save	`str name`, `str image`, `rest` = script[298]
118	NC_WEAPONDELETE	weapon list: Delete	weapon name[298]
119	NC_CLASSDELETE	class delete; sent from 2004, no button in 2002	class name[308]
150	NC_LEVELLISTGET	console button	empty[298]
151	NC_LEVELLISTSET	level list editor: Save	level list[298]

The rules below give how NC saves its texts and which reply layouts are still unmeasured[298].

Rule
All NC texts open in one shared editor window with a mode; Save sends 109, 110, 113, 117 or 151 by mode; attributes (157) and dumps (164) can only be saved locally as `.rtf`[298]
The reply id and layout after id 114 (local NPCs) are unmeasured[308]
Whether 33 carries the script length as `s` or `i3` for large scripts, and whether big set messages go through the raw-data path, is unmeasured[308]

7.3.3 The 2001 PM admin interface

Before the NC socket, admins controlled the 2001 NPC server by PMs to its player login; only senders whose nickname carries `(NPC Admin)` were answered, and an unknown command such as `help` returned the command list[295]. Multi-line commands put their payload on the lines after the first[295].

Command	Payload and effect
`add npc <name>`	next lines: id, type (up to 6 characters), scripter, level, x, y; database NPC ids are unique and at least 1000; level and x, y become the reset position[295]
`reset npc <name>`	resets every attribute but the script and returns the NPC to its start; the name is case-sensitive[295]
`update script <name>`	the following lines replace the script; a running `sleep` is cut[295]
`get script <name>`	returns an NPC's or a class's script[295]
`update class <name>`	replaces a class script, saved to the scripts folder; members pick it up at once[295]
`get npc list`	lists database NPCs (id ≥ 1000) with positions[295]
`dump vars`, `dump vars <name>`	dumps the level's or an NPC's variables[295]
`reload level list`	rereads the text file of levels this server handles[295]

The NC Add NPC dialog has the same seven fields as `add npc`, so id 111 is most likely that command moved onto its own message[308]. The RC finds the NPC server by `(Server)` in the nickname rather than `(NPC Admin)`[295].

8 NPC scripting

> Status: draft. Each chapter carries its own status line.

8.0 Reading this Part

Graal has had two script languages, and they reach the client in different forms. GS1, the language of the 1.x and 2.x clients, arrives as source text in an NPC's script prop, and the client parses it itself.[309] GS2, the language of the 4.x and later clients, is compiled before it reaches the client and arrives as bytecode or a cached `.code` file.[10] The boundary is not a clean cut. 3.x still runs GS1 natively, 3.153 already handles the NPC bytecode ids, and 4.04 rev 2 still runs GS1 weapon scripts.[10,276]

GS1 is a client-side interpreter. 2.05 does not compile GS1 to bytecode. It parses each script once, when the level loads, into a flat list of statement nodes, and every event runs that list from the top with the event's flag set.[310] There is no separate server VM for classic NPCs. A script that needs the server goes through flags, `triggeraction` and the few commands with online side effects, and a server announces an NPC server with PLO 44 to unlock the visual commands.[311,310] From 2.3 the classic client-side mode is gone and Classic moves to the NPC server.[279]

GS1 is the best-read part of the Part. §8.1 gives the language as 2.05 rev 2 runs it, and §8.2 gives every command, function and constant with its argument signature and the changes from build to build.[309] Some exact-build readings correct what the documentation implies. Constant numeric comparisons fold to zero in 2.05 rev 2, so `if (1==1)` is false while the same comparison on variables works. Message codes and event parameters follow byte-level rules that the 2000 documentation never states.[312,313] The 3,172 scraped level scripts and the archived catalogue give the frequencies that show which commands and events matter in practice.[314]

GS2 is known from its machine, not its source. The 4.2.0.8, 5.0.0.7 and unpacked 6.00 engines parse one big-endian four-segment container and run it on one stack machine. That machine finds functions by name, treats events as functions, and keeps its own clock (§8.3.3, §8.4.2).[315] GS2's syntax, its standard library and its full event model are still open, and so is which message id carries the compiled code (→ App. E#207, App. E#202).[316] A public third-party compiler and VM exist, but they are a lead for a clean-room machine, not a reading of the clients.[317]

Exact-build readings stay with their build. Many sections below record one build's behaviour on one input: the 2.05 rev 2 readings in §8.1, the Linux 4.04r2, 4.20r6, 6.01 and 6.03 readings in §8.3.4 and §8.4.4 to §8.4.8, and the installed Windows 5.00 queue. Each states its own limit. A result there is not an all-version rule, and the 4.04r2 clock is not the 5.0 clock.[318,319]

Chapters

- → §8.1 GS1 language (`01-gs1-language.md`)
- → §8.2 GS1 reference by version (`02-gs1-reference.md`)
- → §8.3 GS2 language (`03-gs2-language.md`)
- → §8.4 GS2 bytecode (`04-gs2-bytecode.md`)

8.1 GS1 language

This chapter describes GS1, the NPC and weapon script language of the 1.x and 2.x clients, as the 2.05 rev 2 client parses and runs it. The command-by-command reference, the expression tables and the per-build changes are in → §8.2. The later language is in → §8.3.

> Status: draft. The `#` message-code helper/field semantics and object-array writes and fallback details remain open; the bounded `move` consumer is described in §8.2.9, with scheduling and collision residuals (→ App. E#203, App. E#204, App. E#206).

8.1.1 Where scripts run

GS1 runs on the client. The rules below say where scripts run, what the archived corpus holds, and the three kinds of script[311].

Item	Rule
host	Classic NPCs are pure client-side; the archived script corpus shows no separate server VM[311]
corpus	the archived level scripts sort into genres; baddies (452 scripts) and hazards (192) lead, and most baddy NPCs are event skeletons that react to the player entering or touching them, an item drop, or all baddies dying[320,320]
kinds of script	a level NPC, a weapon NPC (made by `toweapons -Name`), and a child NPC made by `putnpc`[311]
delivery to 2.05	NPC and weapon scripts arrive as source text in the NPC script prop (`NPCPROP_SCRIPT`)[309]
weapon delivery	PLO 33 NPCWEAPONADD carries `str name` and an NPC prop stream with image and script; a name the player already has is ignored; 2.05 fires `playerenters` on the new weapon[228]
pre-v4 clients	the server sends the weapon script as plain GS1 text[228]
level editor	each level NPC has an image (or none) and a GS1 script; the 2.05 editor's script window has *Test* (a syntax check) and *Style*[255]
NPC server present	when the server sends PLO 44 (`PLO_HASNPCSERVER`), several visual commands are allowed online (→ §8.1.9) and the client stops applying NPC props itself[310,3]
2.3 and later	the classic client-side NPC mode is removed; Classic moves to the NPC server[279]
4.04 rev 2	still runs GS1 weapon scripts, but sends only `client.` string flags to the server; `set`, `setstring gr.*` and `setplayerprop #c` from a weapon send nothing[276]

8.1.2 Compilation

2.05 does not compile GS1 to bytecode. It parses the script text once, on level load, and interprets the result[309,310].

Item	Rule
compiled form	a flat list of statement nodes[310]
statement node	a command number, up to two string arguments, up to eight numeric expression arguments, a jump target and a pre-resolved name index (item, baddy, colour and the like)[310]
command number	600 + the command's index in the command table; `if` is 605 (→ §8.2.4)[309]
expressions	resolved through a second table of functions and variables with numeric ids 100–825 (→ §8.2.5) and a small operator table[309]
argument parsing	each command has an argument signature (→ §8.2.1); the compiler uses it to parse the arguments and to build the usage text of its error messages[310]
compiled-away keywords	`else`, `function` and closing braces never reach the executor; they become four internal node kinds: assignment or expression statement, unconditional jump, function call, and end of function body[310]
name matching	command names that end in a space in the table (`say `, `set `, `setlevel `, `setimg `, `move ` and others) require an argument; the others are matched as whole words[309]
missing files	offline, the compiler rejects a filename argument naming a missing file unless it starts with `http://`[310]
OpenGS1	OpenGS1's stack bytecode is an independent reimplementation; 2.05 does not share its opcode set (→ §8.4.3)[309]

8.1.3 Execution

A GS1 script does not run line by line on demand. Each event runs the whole script from the top. The rules below give how events, conditions and loops behave in 2.05[310].

Item	Rule
run	every event runs the whole node list from the top[310]
event test	events are flags tested with `if (name)`; the engine sets the flag before it runs the script (→ §8.1.5)[309]
`if`	exact checked 2.05 revision 2 skips its block on zero; ordinary finite nonzero computed results are true. Condition-name parsing is separate from numeric message expansion → App. E#200[312]
loops	`for` and `while` are the same loop node; the condition is checked on each pass and the loop stops after 10,000 passes, as the 2000 documentation says[310]
function calls	a call pushes a return point; nesting stops at 10,000 deep[310]
`return`	pops the call stack and resumes after the call; with an empty stack (or at the end of a function body) it ends the script[310]
string arguments	trimmed, and message codes (`#c`, `#v(...)` and others) are expanded before use, for almost every command[310]

These commands end the current script run after they execute:

Command	Condition
`setlevel`, `setlevel2`	always[310]
`destroy`	always[310]
`sleep`	non-zero argument[310]
`toinventory`	offline[310]
`hide`	on an NPC that is following the player[310]
`break`, `continue`	outside a loop[310]

Condition truth and constant-comparison folding (exact 2.05 rev 2)

In the checked original Windows 2.05 revision 2, computed `(0.5+0)` and `(-0.5)` display their nonzero values and take the true branch; `(0+0)` displays zero and is false. Bare `0.5` displays 0.5 but is false in condition context, and so is bare `1`, which displays 1 while computed `(1+0)` is true; while `created` displays zero numerically and is true. Unrecognized bare condition tokens can become named-flag tests; numeric message expansion is a separate parsing context. These observations do not establish a threshold of one or rounding[312].

A separate compiler defect folds comparisons of constant numeric leaves to zero: observed `1==1`, `1!=0`, `1<2`, `2>1`, `1=1` and `(1+0)==1` all display zero and are false. The comparison parent becomes a zero-initialized numeric leaf before evaluation; arithmetic folding evaluates before changing the parent. Dynamic self-equality and ordering controls and `this.a==1` after assigning one work, while dynamic self-inequality is false. This is not absence of comparison support or a complete operator/precedence contract[312].

The researcher ran sixteen owned cases in two offline Wine/virtual-display fixtures, with unchanged original executable bytes, no server or external network, explicit 2.31 donor assets and completed cleanup. Fifteen labels were fully readable in the second layout; its actor-obscured greater-than control was read from the identical script in the first. No absent label counted as false. Ten fresh selected instruction reads corroborate the mechanism, without live expression-node inspection. Evidence and target identity are researcher supplied. Other builds, nonfinite/malformed values, full aliases/precedence and flag-name collisions remain open[312].

8.1.4 Timeouts and sleep

Scripts that wait use a timer. The rules below give how `timeout` and `sleep` work, and the rates the archived scripts use[311].

Item	Rule
timer	a script sets `timeout = N` (seconds); when it expires the script runs with the `timeout` flag set[311]
common rates	0.05 s for AI and HUDs, 0.1–0.2 s for menus, 0.5–1.0 s for idle NPCs, in the archived corpus[311]
`sleep n`	sets the NPC's timeout to *n*, remembers the next statement, and ends the run; the next `timeout` event resumes there[310]
`sleep 0`	only clears the timer display[310]

Item	Rule
`timershow`	draws the NPC's timeout counter over it; cleared by `message` and by `sleep 0`[310]
who gets timeouts	only the level leader receives an NPC's timeout events, unless the NPC runs `timereverywhere`[310]

8.1.5 Events

The engine sets one of these flags and runs the script[309]. The scrape count is the number of the 3,172 scraped level scripts that test the event, where the notes counted it.

Event	Scrape count
`created`	unmeasured[309]
`playerenters`	2,268[309,321]
`timeout`	1,456[309,321]
`playertouchsme`	1,986[309,321]
`playertouchsother`	1,074[309,321]
`playerendsreading`	1,123[309,321]
`playerchats`	unmeasured[309]
`playersays(…)`, `playersays2(…)`	unmeasured; these are functions, not flags[309]
`washshot`	728[309,321]
`washshooted`, `washit`, `washthrown`, `waspelt`, `peltwith…`	unmeasured[309]
`exploded`	unmeasured[309]
`compusdied`	1,122[309,321]
`compsdead`	unmeasured[309]
`playerdies`, `playerhurt`, `playerhurted`	unmeasured[309]
`playerlaysitem`, `weaponfired`	unmeasured[309]
`action…`	unmeasured; any action name sent by `triggeraction`[309]
`shotbyplayer`, `shotbybaddy`	unmeasured[309]
`main`	unmeasured; sits in the same string cluster, probably a weapon's main event or entry[309]

Events documented for 2.22 but absent in 2.05: `actionleftmouse`, `actionrightmouse`, `actiondoublemouse`, `actionprojectile`, `keypressed`, `mousedown`, `playeronline`, `playerpaused`[309].

How an event reaches another script:

Mechanism	Rule
`callnpc i, event`	exact 2.05r2 queues the expanded event name with empty parameter text; comma suffixes are not split into #p values[322]
`callweapon i, event`	moves weapon *i* to the player's centre, then queues its event name with empty parameters; static-only in this follow-up[322]
`triggeraction x, y, action, params`	fires `action` with its parameters at the object on (x, y) locally, and sends it online (→ §8.2.3)[310]
`triggeraction 0,0,clientside,…`	from 2.21, reaches a weapon's client-side part[279]

Event parameters (exact 2.05 rev 2)

Event dispatch clears and fills the shared list read by `#p(index)/paramscount` from an actor's pending parameter text, using a literal comma separator. Numeric indices are expanded/evaluated/converted and bounds-checked; negative/out-of-range yields empty. This list is separate from the tokenize/tokenize2 list used by `#t/tokenscount`[322].

An actor has multiple pending event names but one parameter-text field. Each accepted distinct event adds its name and replaces that field; duplicate pending names are rejected without changing it. Any pending action-prefixed name rejects another action-prefixed request. Newly accepted actions also record source context; exact name-case rules remain untraced. Thus later accepted calls can overwrite earlier pending parameters, depending on batching; rejected requests preserve them[322].

`callnpc` resolves its caller-relative selector; `callweapon` resolves a zero-based weapon index and centers its actor on the player. Both queue expanded event names through an empty-parameter wrapper, rather than executing receivers synchronously or splitting comma suffixes into #p. Local `triggeraction` supplies separate parameter text, prefixes the action name and checks eligible receiver bodies; weapon delivery has an additional local-player proximity/source gate. Connected sending is separate from this offline chain[322].

The byte splitter ends unquoted tokens at the first comma. A quote at the start removes that quote and takes text through the next quote, retaining internal commas. Tokens and remainders trim end bytes below 33. Unquoted empty fields and trailing commas are skipped; quoted empty remains. Missing closing quote consumes the rest; there is no escape/backslash-aware quote search. Text after a closing quote need not start with comma. These mechanics are separate from script-level construction/quoting[322].

A nonnull player context rebuilds the shared list at dispatch entry; null-player early exit does not. Conditional normal cleanup clears pending actor fields but not the shared list, which may retain the current/last context until another dispatch. The nested wrapper saves/restores actor fields without a visible corresponding shared-list snapshot; this does not establish complete reentrant semantics or immutable per-event payloads. The cleanup condition's global meaning remains open[322].

Four original-client offline runs provide ten qualified result groups. Local alpha,beta delivery gives count2; unquoted empties/trailing comma are skipped, quoted empty retained, quoted beta,gamma stays one entry, and a mid-token quote does not protect its comma. A queued action followed by a distinct callnpc probe produces both handler markers with count0; two rejected-action controls retain the first value. A saved pre-sleep count2/alpha/beta resumes as count0/empty after requested sleep 0.2, without isolating cleanup versus later overwrite or measuring elapsed timing. Long labels use a readable repeat; ordinary-empty uses the earlier clear scene, with no obscured label counted empty[322].

Runs used unchanged original Windows 2.05r2 under Wine/private displays and isolated namespaces, no server, owned levels and explicit 2.31 donor assets; all supervisors exited zero and cleanup completed. Evidence is researcher supplied, using selected fresh instructions and recovered matching bodies; no live AST/pending-field snapshots. Callweapon remains static-only. Network acceptance, every event/source category, full nested-context restoration, case/escaping, precise scheduling and other profiles remain open. Only the #p producer gap is closed; broader E#203 field/scanner/formatter/renderer questions remain[322].

8.1.6 State flags

The engine keeps these flags current; a script tests them with `if`[309].

Group	Flags
NPC	`visible`, `isweapon`, `followsplayer`, `isleader`, `isonmap`[309]
carrying	`carrying`, `carries…`, `carriesnpc`[309]
player	`weaponsenabled`, `playeronhorse`, `canspin`, `playerswimming`, `playerismale`, `playerisfemale`, `playerreading`, `playerattached`, `firedonhorse`[309]
level	`issparringzone`, `sparringzone`, `nopkzone`, `lighteffectsenabled`[309]

8.1.7 Flags, variables and persistence

The rules below give how a script stores state: flags, string variables and arrays, and which of them last beyond the session[310].

Item	Rule
flags	`set name` sets a flag, `unset name` clears it; the documentation says a `server.` prefix makes it server-wide[310]
string variables	`setstring name,value` stores the flag `name=value`; an empty value deletes it[310]
arrays	`setarray var,n` resizes an array variable to *n* elements; ignored unless the variable is plain or `this.`[310]
NPC-local variables	`this.` prefix[309]
NPC save slots	`save[n]`, reached as `this.save[i]` or `npcs[i].save[i]` (→ §8.2.5)[309,311]
persistence	scripts persist state through `toweapons`, `save[n]` and `client.*` flags[311]
tokens	`tokenize` splits a string into the global token list on spaces and commas (quotes group); `tokenize2` adds delimiter characters; `tokenscount` gives the count[310,309]
2.19	`this.` strings on the client; on NPC-server servers the client stops sending strings that are not `client.`[279]

Item	Rule
4.04 rev 2	only `client.` string flags are sent to the server[276]
image ids	HUD and shop scripts in the archived corpus collide often in the `showimg` index band around 300 and above[311]

8.1.8 Syntax and operators

GS1's operators are close to C's. Each row gives a group and its forms[309].

Item	Forms
assignment	`=`, `+=`, `-=`, `*=`, `/=`, `++`, `--`[309]
arithmetic	`+ - * / %`, `^` for power[309]
comparison	`== != < > <= >=`[309]
logic	`&&`, `||`, `!`[309]
membership	`in` for ranges and sets: `x in |0,64|`, `2 in {1,2,3}`[309]
control flow	`if`/`else`, `for`, `while`, `break`, `continue`, `function name() {}` with `name();`, `return`, `sleep`, `timeout = n`[309]
string codes	exact 2.05r2 outer catalog has 44 selectors / 53 family-expanded spellings; helper/field and display meanings remain partial → App. E#203[313]
character props	`setcharprop` and `setplayerprop` take a property code (`#1`, `#2`, `#3`, `#5`, `#8`, `#C0`..`#C5`, `#P1`..`#P5`, `#c`, `#n`) as their first word (→ §8.2.4.5)[310]
not commands	`warp` and `jump` are not GS1 commands in 2.05; warping is `setlevel` / `setlevel2` or an assignment to `playerx` / `playery`[310]

8.1.9 Execution contexts

The same command can act differently depending on what runs it. Each row gives a context and the rule that changes there[310].

Context	Rule
weapon NPCs act as the player	when the running NPC is one of the player's weapons, commands that spawn things (bombs, explosions, horses, projectiles, `showimg` layers, `hitobjects`, `triggeraction`, `reflectarrow`) use the player's level and count the player as the source[310]
online gate	`seteffect`, `setcoloreffect`, `setzoomeffect`, `changeimgcolors`, `changeimgzoom`, `addtiledef`, `addtiledef2`, `removetiledefs`, `replaceani` and `attachplayertoobj` run only offline, or online when the server announced an NPC server (PLO 44); online without an NPC server they do nothing[310]
editor preview	when the in-game editor builds a preview copy of a `showcharacter` NPC, it runs that NPC's `playerenters` in a restricted mode[310]
preview whitelist	only `if`, `else`, `for`, `while`, `setgif`, `setgifpart`, `setimg`, `setimgpart`, `setmap`, `showcharacter`, `setcharprop`, `setcharani`, `setbackpal`, `setbacktile`, `setbacktile2`, `addtiledef`, `addtiledef2`, `setz`, `loadmap`, function calls and `++`/`--` style assignments run; everything else is skipped[310]
online side effects	some commands also send a TCP or UDP message when connected (→ §8.2.3)[310]

8.1.10 The script corpus

The notes hold two collections of real GS1 scripts, the evidence for the frequency counts in this part:

Collection	Content
level scrape	3,172 unique scripts from GR-V1 `.graal` levels; the scrape reported 3,187, and 15 records were lost to truncated source pages[314]
level scrape by genre	combat 1,400; interaction 602; dialogue 373; spawn 328; misc 257; object 168; transport 59[314]
archived catalogue	1,520 unique script bodies from 6,275 placements, with a body-hash catalogue; held off-repository[311]

The archived catalogue classifies about 1,519 documented scripts[311]:

Class	Approx. count
quest / dialogue	516[311]

Class	Approx. count
level prop / trigger	390[311]
farm / planting	136[311]
player weapon / projectile	135[311]
shop / merchant	111[311]
minigame	59[311]
enemy / baddy AI	44[311]
inventory / equipment UI	37[311]
HUD / stats overlay	20[311]

The commands most used in the level scrape are `hide` and `show` (in most `playertouchsme` scripts), then the level-change commands, `hurt`, `say`, and `set` / `unset`[321].

8.1.11 Message codes (exact 2.05 rev 2)

The checked outer dispatch recognizes 44 selector bytes, or 53 spellings after replacing C/P family prefixes with C0–C5/P1–P5. This includes preserved display codes and empty-output `#7`; it is not a count of implemented operations. Case matters. The byte selector inventory is `# . 1 2 3 4 5 6 7 8 A B C E K L P T W X Y a b c d e f g h i k l m n p r s t u v w x y z`[313].

Message trims end bytes below 33, expands in mode zero with the caller, stores its text and clears timer-display state. Expansion joins prefix, replacement and recursively expanded suffix without automatically rescanning inserted text. A trailing hash survives; unknown codes drop only their hash. Recognized codes consume their prefix even without valid arguments; a balanced argument extends consumption through its close. Parenthesis nesting is traced but quote/escape scanning remains partial. Strings and slicing use bytes[313].

`##` yields a hash. `#.` `#4` `#A` `#B` `#X` `#Y` `#d` `#h` `#i` `#l` `#r` `#u` `#x` `#y` `#z` preserve their two-byte display spellings. `#b` adds LF only in nonzero expansion mode; ordinary message mode zero removes it. `#E` inserts a runtime-state byte whose writer is unresolved; it was a space in the one tested context[313,313].

Character-family codes accept an optional numeric selector: omitted/zero selects the local player, minus one selects the caller only if a character, positive indices enumerate remote players then other character NPCs excluding the caller; unresolved selectors yield empty. Conversion uses the biased-floor helper. `#1/#2/#3/#5/#8` return sword/shield/head/horse/body images, stripping directories and lowercasing ASCII, retaining extensions; absent objects yield empty. `#6` returns the selected actor’s carried-NPC image; later independent static checks link its association to pickup/throw/clear paths, without full eligibility/lifetime or runtime validation (see the checkpoint below). `#7` selects but outputs empty. `#C0–#C5` index color names through six actor storage slots; the sixth slot has unproved meaning and does not extend the five-byte wire color property. Invalid C/P digits consume the extra byte and yield empty; C storage values lack a local table-size check. `#P1–#P5` return stored image-parameter text or a nonnegative integer fallback. `#c` returns unexpanded character text; `#n` returns the name and `#g` extracts its parenthesized tag without proving membership. `#a` is the character's account name, case preserved and distinct from `#n` (→ field identities below the selector list); `#m` shares the animation object with setani/setcharani[313].

`#v(expr)` recursively expands its argument in mode zero, numerically evaluates and formats a double; exact formatting remains open. `#s(name)` trims/expands the name and returns its stored string, with store normalization/case still partial. `#T(text)` trims, expands and trims again. `#e(start,length,text)` numerically evaluates the first two top-level comma parts, biased-floor converts them, expands text and slices bytes: zero-based start clamps to string bounds, negative length means remainder, overruns clip. Missing separators yield empty after consuming balanced arguments; quoted separators remain unqualified[313].

`#K(number)` returns the low byte for converted integers zero through 65535, otherwise empty; this is not Unicode. Embedded zero and display behavior are distinct. `#k(number)` asks for configured binding variant zero at indices zero through ten; layout/localization remain open. `#w/#W` return selected weapon name/image or an explicitly indexed zero-based entry after argument expansion/evaluation. Explicit out-of-range indices yield empty; omitted index checks nonnegative before list access. Weapon image names are not normalized like character equipment. `#f` normalizes the caller image; `#L` is the evaluating player's recorded level name, directory-stripped and lowercased, taking no selector and changing only when a warp commits or the server sends the self-player level property[313]. `#p/#t` expand/evaluate/convert indices and bounds-check distinct string lists; tokenize/tokenize2 produce the #t list; the separate dispatch producer for #p is established in §8.1.5[313].

Thirty qualified outputs in three usable scenes corroborate hash escaping/fallback, trailing hashes, recognized missing arguments, case sensitivity, non-rescanning of inserted text and explicit nested argument expansion. Substring bounds, numeric examples, `#K(65)`/negative input, mode-zero `#b`, name/tag extraction, empty `#7` and token indices were observed. `#K(0)` is only a box-glyph observation; empty `#p(0)` does not identify event parameters. Four original-client Wine/private-display runs used unchanged executable bytes, isolated namespaces, no server, explicit 2.31 donor assets and owned fixtures; all exited zero and cleaned up. One overlapping layout is diagnostic only and replaced, not counted among controls. Identity, static and visual evidence are researcher supplied; complete selector decoding does not fully revalidate every helper. Other builds, live remote-player ordering, malformed input, quoted scanners, exact formatting, #a/#E/#L meanings and downstream rendering remain open[313].

Independent exact-original 2.05r2 static corroboration identifies `#6` as carried-NPC image: the eligible-NPC pickup branch writes its association, the carryable flag is controlled by `canbecarried`/`cannotbecarried`, throw transfers/clears it, and `takeplayercarry` plus matching NPC removal clear it. Basename stripping and ASCII lowercase retain the extension; absence yields empty and inserted text is not rescanned. All 634 checked prior instructions matched, with additional focused decoding reported. This narrows only the association identity; no full input/collision eligibility, runtime, all-selector, receiver/property-case or object-lifetime contract is established. The prior receiver helper label is excluded as incorrect.[313]

8.1.12 GS1 on Linux 3.04

An independently authored source weapon on original Linux 3.04 produces created/playerenters flag messages and fresh-account persistence. Numeric readbacks and explicit positive comparisons work while tested bare predicates do not activate. This is a separate build scope from the Windows 2.05r2 resolution; no universal numeric threshold or complete predicate/name-lookup rule is established.[323].

8.2 GS1 reference by version

This chapter is the GS1 reference: argument types, name tables, every command of the 2.05 rev 2 table with what 2.05 does, the expression tables, constants, and how the command table grew from 1.33 to the NewWorld betas. How scripts are compiled and run is in → §8.1.

> Status: draft. Command behaviour was read from 2.05 rev 2 only; other builds are covered by table membership, not by executor reads (→ §8.2.7). Four stored signatures have an exact-input static correction; argument conversion and semantic equivalence remain open (→ App. E#201).

8.2.1 Argument signature letters

Each command in the table carries one letter per argument. The compiler uses the letters to parse and to label its usage messages[310].

Letter	Compiler's label	Accepts
`R`	number	any numeric expression[310,309]
`S`	string	a string expression: #s codes, quoted text, string variables; message codes are expanded at run time[310,309]
`M`	string	raw text up to `;`, such as flag and level names; message codes are expanded at run time[310,309]
`F`	filename	a file name[310]
`Q`	filename	a file name that names a script file[310]
`I`	itemname	one of the 25 item names (→ §8.2.2)[310,309]
`B`	baddyname	one of the 10 baddy names[310,309]
`C`	colorname	one of the 20 colour names, or a number[310,309]
`D`	direction	up `left` down `right` (0..3), or a numeric expression[310,309]
`L`	carry objectname	one of the 11 carry objects[310,309]
`O`	objectname	an entry from the editor's predefined tile-objects list (`tileobjects/`)[310,309]

The four corrected entries give stored letters only; argument conversion and equivalence of the older descriptions remain unmeasured → App. E#201[324].

8.2.2 Name tables

Several commands take a number that stands for a name. The table gives each name table with its index-to-name list[310,309].

Table	Index: name
items (`I`)	0 greenrupee, 1 bluerupee, 2 redrupee, 3 bombs, 4 darts, 5 heart, 6 glove1, 7 bow, 8 bomb, 9 shield, 10 sword, 11 fullheart, 12 superbomb, 13 battleaxe, 14 goldensword, 15 mirrorshield, 16 glove2, 17 lizardshield, 18 lizardsword, 19 goldrupee, 20 fireball, 21 fireblast, 22 nukeshot, 23 joltbomb, 24 spinattack[310,309]
baddies (`B`)	0 graysoldier, 1 bluesoldier, 2 redsoldier, 3 shootingsoldier, 4 swampsoldier, 5 frog, 6 octopus, 7 goldenwarrior, 8 lizardon, 9 dragon[310,309]
hare baddy	2.05 ships baddy`hare.gif`; it is type 5, `frog`, which 2.22 draws with the hare image (→ App. E#205)[309,133]
colours (`C`)	0 white, 1 yellow, 2 orange, 3 pink, 4 red, 5 darkred, 6 lightgreen, 7 green, 8 darkgreen, 9 lightblue, 10 blue, 11 darkblue, 12 brown, 13 cynober, 14 purple, 15 darkpurple, 16 lightgray, 17 gray, 18 black, 19 transparent; the same list indexes `PLPROP_COLORS`[310,309]
carry objects (`L`)	0 bush, 1 sign, 2 vase, 3 stone, 4 blackstone, 5 bomb, 6 hotbomb, 7 superbomb, 8 joltbomb, 9 hotjoltbomb, 10 none[310]
directions (`D`)	0 up, 1 left, 2 down, 3 right; numeric values are reduced mod 4, negative values too[310]
direction table	the four names follow the command table directly; a dump that reads two records past `hide` finds up and `right` and wrongly counts 163 commands[310,208]

8.2.3 Online side effects

When the client is connected, some commands also tell the server (TCP, PLI ids → App. A) or nearby peers (UDP ids → §2.9)[310]. A level-local action goes over UDP only when every other player in the level has a UDP port, otherwise over both[310].

Command	TCP (PLI)	UDP
`take`, `take2`, `removeitem`	13 ITEMDEL	20 item del[310]
`putcomp`, `putnewcomp`	17 BADDYADD, instead of spawning locally	n/a[310]
`putnpc`	21 PUTNPC, instead of creating locally	n/a[310]
`destroy`	29 NPCWEAPONDEL if the NPC is also a weapon, then 22 NPCDEL	n/a[310]
`toweapons`	33 WEAPONADD	n/a[310]
`hitcompu`	16 BADDYHURT	n/a[310]
`hitplayer` on a remote player	n/a	21 remote hit, to the victim only[310]
`putexplosion`, `putexplosion2`	27 EXPLOSION	14 explosion[310]
`spyfire`	10 FIRESPY	13 fire spy[310]
`hitobjects`	36 HITOBJECTS	25 hit objects[310]
`triggeraction`	38 TRIGGERACTION	26 trigger action[310]
`removebomb`	n/a	11 bomb del[310]
`removehorse`, `takehorse`	n/a	17 horse del[310]
`reflectarrow`	n/a	12 arrow, re-sent with the new direction[310]
`setplayerprop #c,...`	chat prop	22 chat[310]

8.2.4 Commands of 2.05 rev 2

The 2.05 table holds 161 commands, ops 600 to 760 (op = 600 + table index); `show` (759) and `hide` (760), the two most common commands in the scraped library, are its last two entries[309,321]. The table is the same in 2.05 rev 1 and rev 2[277].

How to read the tables below:

- *Op* is the 2.05 command number. Commands are inserted in the middle of the table in later builds, so the index of a command changes between builds and an op is valid only for 2.05[11].
- *In the table from* is the first build whose table has the command; 1.33 is the earliest build read, so "1.33" means "1.33 or earlier"[11].
- A † marks a command that is not in the 2.22 `commands.rtf`: removed, renamed (`setgif` → `setimg`, `setgifpart` → `setimgpart`) or undocumented[309].
- Positions are in tiles unless the row says otherwise; "doc" means the 2000 `npcprogramming.doc` or the `newfeatures` logs, and where the client and the documentation disagree the row gives the client's behaviour[310].

8.2.4.1 Messages, signs, sound, files

From here the command tables share one layout. Each row gives the opcode, the command, its argument types, the first build whose table has it, and what 2.05 does. This first group covers messages, signs, sound and files[309,310].

Op	Command	Args	In the table from	What 2.05 does
600	`say`	R	1.33	Shows sign *n* of the current level (index into the level's sign list). Skipped if the index is out of range or the player is already in the sign-reading state. Puts the player into the reading state.[309,310,11]
606	`play`	F	1.33	Plays a sound positioned at the NPC's centre. Online, a file that is not found locally is requested by its raw name. Doc: midis loop.[309,310,11]
635	`openurl`	M	1.33; takes arguments from 2.01	Opens the URL in the system browser (doc: with a confirm dialog, no `http://` needed).[309,310,11]
637	`stopmidi`	—	1.33	Stops the current midi.[309,310,11]

Op	Command	Args	In the table from	What 2.05 does
648	`message`	S	1.33	Sets the text drawn above the NPC. An empty string clears it. Also turns off `timershow`.[309,310,11]
676	`showfile`	F	1.33	Opens the file with its associated program, like a double-click.[309,310,11]
678	`setbackpal`	F	1.33	Copies the 256-colour palette of the given image into the tileset and into every loaded level's tile image, then repaints. Remembers the file so the same palette is not reloaded.[309,310,11]
679	`setletters`	F	1.33	Replaces the sign font image (default `letters.png`) if the file exists.[309,310,11]
702	`playlooped`	F	1.37	Same, but the file is queued as a looped sound (internally the name gets a + prefix). Stops on level exit or `stopsound`.[309,310,11]
703	`stopsound`	F	1.37	Stops that (looped) sound.[309,310,11]
711	`say2`	S	1.41 rev1	Same as `say`, but shows the given text directly instead of a level sign.[309,310,11]
739	`setmusicvolume`	RR	2.0	Sets music volume, left and right (doc: 0..1).[309,310,11]
740	`openurl2`	MRR	2.01	Opens the URL in the client's own HTML window of the given width and height.[309,310,11]

8.2.4.2 Flags and variables

This group sets and clears flags and variables[309,310].

Op	Command	Args	In the table from	What 2.05 does
604	`set`	M	1.33	Sets a flag (doc: `server.` prefix makes it server-wide).[309,310,11]
607	`unset`	M	1.33	Clears a flag.[309,310,11]
666	`setarray`	RR	1.33	Resizes the array variable named by the first argument to *n* elements. Ignored unless the first argument is a plain or `this.` variable; 2.05r2 ignores sizes outside 0–10000, zero-fills growth, keeps the prefix and frees at 0.[309,310,11]
677	`setstring`	SS	1.33	Sets string variable *name* = *value* (stored as the flag `name=value`). Empty value deletes it.[309,310,11]
742	`tokenize`	S	2.02 rev2	Splits the expanded string into the global token list on spaces and commas (quotes group).[309,310,11]
743	`tokenize2`	SS	2.02 rev2	Same, with extra delimiter characters added to the space/comma set.[309,310,11]

8.2.4.3 Control flow

This group covers control flow. Note the loop cap on `for` and `while`[309,310].

Op	Command	Args	In the table from	What 2.05 does
605	`if`	—	1.33	Evaluates the condition; jumps past the block when it is 0.[309,310,11]
634	`else`	—	1.33	Compiled away into a jump.[309,310,11]
660	`for`	—	1.33	Loop; condition checked each pass; stops after 10,000 passes.[309,310,11]
667	`sleep`	R	1.33	Sets the NPC's timeout to the value and remembers the next statement; the next `timeout` event resumes there. `sleep 0` just clears the timer display. Ends the current run.[309,310,11]
668	`while`	—	1.33	Same loop node as `for`.[309,310,11]
710	`callnpc`	RS	1.41 rev1	Exact 2.05r2 queues the expanded event name with empty parameter text; it does not split comma suffixes into #p parameters. Accepted calls can overwrite pending actor parameter text.[309,310,11,322]
712	`break`	—	1.41 rev1	Jumps to the end of the enclosing loop. Outside a loop it ends the script.[309,310,11]
713	`continue`	—	1.41 rev1	Jumps to the loop's next pass. Outside a loop it ends the script.[309,310,11]
714	`callweapon`	RS	1.41 rev1	Moves weapon *index* to the player's centre, then queues the expanded event name with empty parameters (exact-build static trace; not runtime-qualified here).[309,310,11,322]
716	`function`	—	1.41 rev1	Compiled away. Calls to a named function become a call node (max depth 10,000).[309,310,11]

Op	Command	Args	In the table from	What 2.05 does
717	`return`	—	1.41 rev1	Pops the call stack and resumes after the call. With an empty stack it ends the script.[309,310,11]
744	`triggeraction`	RRSS	2.03	Fires *action* with *params* at the object on (x, y) locally, and sends it online. From a weapon, the sender is the player.[309,310,11]

8.2.4.4 NPC appearance and behaviour

This group sets how an NPC looks and behaves. Rows marked † are in 2.05 but not in the 2.22 documentation[309,310].

Op	Command	Args	In the table from	What 2.05 does
608	`setgif` †	F	1.33	Sets the NPC image, shows the NPC, and resets the visible part to the whole image.[309,310,11]
619	`dontblocklocal`	—	1.34	Same, local only.[309,310,11]
620	`drawoverplayer`	—	1.33	Draw layer +1.[309,310,11]
631	`timershow`	—	1.33	Draws the NPC's timeout counter over it. Cleared by `message` and `sleep 0`.[309,310,11]
638	`blockagainlocal`	—	1.34	Same, local only.[309,310,11]
639	`drawunderplayer`	—	1.33	Draw layer −1.[309,310,11]
641	`canbecarried`	—	1.33	Player can / cannot lift the NPC.[309,310,11]
642	`cannotbecarried`	—	1.33	Player can / cannot lift the NPC.[309,310,11]
644	`canbepushed`	—	1.33	Player can / cannot push it.[309,310,11]
645	`cannotbepushed`	—	1.33	Player can / cannot push it.[309,310,11]
646	`canbepulled`	—	1.33	Player can / cannot pull it.[309,310,11]
647	`cannotbepulled`	—	1.33	Player can / cannot pull it.[309,310,11]
652	`putnpc`	QQRR	1.33	Offline: creates a new NPC with that image and script file at (x, y) and runs its `playerenters`. Online: asks the server instead.[309,310,11]
653	`destroy`	—	1.33	Removes the NPC from the level (and from the weapon list), sends the deletes online, ends the run.[309,310,11]
670	`timereverywhere`	—	1.33	This client gets the NPC's timeout events even when it is not the level leader.[309,310,11]
680	`setgifpart` †	FRRRR	1.33	Same, then shows only (x, y, w, h) of the image in pixels. x, y are clamped to 0..16000 and w, h to 0..220, then trimmed to the image size.[309,310,11]
686	`showlocal`	—	1.34	Same, but marked local (not synced).[309,310,11]
687	`hidelocal`	—	1.34	Same, marked local.[309,310,11]
688	`dontblock`	—	1.33	Player can walk through the NPC.[309,310,11]
689	`blockagain`	—	1.33	Blocks again and also resets the draw layer to normal (undoes `drawoverplayer`/`drawunderplayer`).[309,310,11]
719	`setimg`	F	1.41 rev1	Sets the NPC image, shows the NPC, and resets the visible part to the whole image.[309,310,11]
720	`setimgpart`	FRRRR	1.41 rev1	Same, then shows only (x, y, w, h) of the image in pixels. x, y are clamped to 0..16000 and w, h to 0..220, then trimmed to the image size.[309,310,11]
725	`setshape`	RRR	NewMain 1.42; takes arguments from 2.04	Type 0 or 1 (1 = fixed rectangle), width and height in pixels. Other types are ignored.[309,310,11]
726	`seteffect`	RRRR	1.33; out 1.34–1.41; back NewMain 1.42	Global screen tint r, g, b, a (day/night). Online gate applies.[309,310,11]
727	`setcoloreffect`	RRRR	NewMain 1.42	Per-NPC colour multiply r, g, b, a. Online gate applies.[309,310,11]
728	`setzoomeffect`	R	NewMain 1.42	Per-NPC zoom factor. Online gate applies.[309,310,11]
729	`drawaslight`	—	NewMain 1.42	Draw layer 9: above the day/night effect, below the status bar.[309,310,11]
745	`move`	RRRR	2.03	Smoothly moves the NPC by dx, dy over *time* seconds with option bits. (option bits not decoded)[309,310,11]

Op	Command	Args	In the table from	What 2.05 does
751	`setshape2`	RRR	2.04	Makes the NPC block like a small level: width and height in tiles, plus a tile-type array (literal or array variable) of w×h entries. A short or missing array clears the shape.[309,310,11]
759	`show`	—	1.33	Shows the NPC (re-applies its image).[309,310,11]
760	`hide`	—	1.33	Hides the NPC (only if it is in the current level). If it was following the player, it stops following and the run ends.[309,310,11]

8.2.4.5 Character NPCs (`showcharacter`)

This group drives NPCs drawn as player characters[309,310].

Op	Command	Args	In the table from	What 2.05 does
633	`followplayer`	—	1.33	Offline only: NPC follows the player and stops blocking. Ignored online (matches the doc).[309,310,11]
673	`showcharacter`	—	1.33	Draws the NPC as a player character (once; later calls do nothing).[309,310,11]
675	`setcharprop`	MM	1.33	Sets a character property from a message code: #1 sword image, #2 shield image, #3 head image, #5 horse image (creates the horse if needed), #8 body image, `#C0`..`#C5` colours (name or number), `#P1`..`#P5` attributes, #c chat text (not in preview mode), #n nickname. Other codes are ignored. The first word is the property code (not expanded as a message). Codes inside the value are expanded first; on script-running servers #n in that value is the triggering player's nickname, while property code #n writes the NPC's nickname — see `server-notes/setcharprop-hash-n.md`.[309,310,11]
684	`carryobject`	L	1.34	Character carries the object (0..10; none = 10). Only for `showcharacter` NPCs; switches to the carry gani.[309,310,11]
685	`throwcarry`	—	1.34	Throws the carried object if the character is in a carry sprite; bombs become live bombs.[309,310,11]
701	`takehorse`	R	1.36	Character mounts horse *index* (removes it from the level). Only for `showcharacter` NPCs.[309,310,11]
709	`setcharani`	SS	1.39	Sets the character's gani and its parameter string.[309,310,11]
733	`setchargender`	S	2.0	`male` sets male; anything else sets female. 2.05r2 trims and lowercases the raw argument without message expansion.[309,310,11]

8.2.4.6 The player

This group acts on the player. Several of these end the script run[309,310].

Op	Command	Args	In the table from	What 2.05 does
603	`hurt`	R	1.33	Hurts the player by half-hearts and knocks them away from the NPC's centre. Skipped when the player cannot be hurt right now (dead, in a protected state, or flagged unhittable on an NPC-server). 0 or negative heals (doc).[309,310,11]
610	`setlevel`	M	1.33; takes arguments from 1.38	Warps the player to that level at the current x, y. Ends the run.[309,310,11]
611	`setskincolor`	C	1.33	Player colour slots 0..4.[309,310,11]
612	`setcoatcolor`	C	1.33	Player colour slots 0..4.[309,310,11]
613	`setsleevecolor`	C	1.33	Player colour slots 0..4.[309,310,11]
614	`setshoecolor`	C	1.33	Player colour slots 0..4.[309,310,11]
615	`setbeltcolor`	C	1.33	Player colour slots 0..4.[309,310,11]
616	`setplayerdir`	D	1.33	Player direction, reduced mod 4. Accepts a name or an expression (doc says names only).[309,310,11]
618	`takeplayercarry`	—	1.33	Removes what the player carries and resets the gani.[309,310,11]
621	`enableweapons`	—	1.33	Turns player weapons on / off.[309,310,11]
622	`disableweapons`	—	1.33	Turns player weapons on / off.[309,310,11]
632	`toinventory`	S	1.33	Sets the flag and moves the NPC into the inventory. Offline it leaves the level and the run ends; online a copy goes to the inventory and the NPC stays. Only for NPCs of the current level.[309,310,324,11]

Op	Command	Args	In the table from	What 2.05 does
649	setsword	MR	1.33	Sword image and power.[309,310,11]
650	setshield	MR	1.33	Shield image and power.[309,310,11]
654	freezeplayer	R	1.33	Freezes the player for 0..30 seconds (clamped) and drops what they carry.[309,310,11]
655	sethead	F	1.33	Player head image.[309,310,11]
656	toweapons	M	1.33	Adds a copy of the NPC as weapon *name* (replacing one with the same name, in the same slot), runs its playerenters, selects it, and sends it online. A / in the name keeps only the part after it.[309,310,11]
659	hidesword	R	1.33	Hides the sword for 0..30 seconds.[309,310,11]
663	setbow †	S	1.33	Sets the bow image on the running NPC's character, not on the player. (differs from doc, which says the player's bow)[309,310,11]
669	hideplayer	R	1.33	Hides the player for 0..30 seconds.[309,310,11]
674	setplayerprop	MM	1.33	Same codes as setcharprop but on the player: #1 #2 #3 #5 #8 #C0..#C5 #P1..#P5 #c. #c sets the chat text with a 60-tick timeout and sends it online. #7 is not handled (doc lists it).[309,310,11]
683	setbody	F	1.33	Player body image (skipped if it is the same file).[309,310,11]
691	takeplayerhorse	—	1.36	Removes the player's horse.[309,310,11]
705	setlevel2	MRR	1.38	Warps to that level at (x, y). Ends the run.[309,310,11]
708	setani	SS	1.39	Player gani and parameter string.[309,310,11]
723	showstats	R	1.41 rev1	Bitmask of status-bar parts to show (doc: 1 ASD, 2 icons, 4 rupees, 8 bombs, ...).[309,310,11]
732	setgender	S	2.0	male sets male; anything else sets female. 2.05r2 trims and lowercases the raw argument without message expansion.[309,310,11]
737	enabledefmovement	—	2.0	Turns the built-in movement and controls on / off.[309,310,11]
738	disabledefmovement	—	2.0	Turns the built-in movement and controls on / off.[309,310,11]
749	replaceani	MM	2.04	Replaces one of the 31 default gani names (walk, idle, sword, ...) with another gani file. Online gate applies.[309,310,324,11]
750	attachplayertoobj	RR	2.04	Type 0 only (NPC): attaches the player to the NPC with that id. Online gate applies.[309,310,11]
752	detachplayer	—	2.04	Detaches the player if attached.[309,310,11]
753	enableselectweapons	—	2.04	Allows / blocks opening the weapon selection.[309,310,11]
754	disableselectweapons	—	2.04	Allows / blocks opening the weapon selection.[309,310,11]

8.2.4.7 Combat, items, level objects

This group handles combat, items and level objects[309,310].

Op	Command	Args	In the table from	What 2.05 does
601	lay	I	1.33; takes arguments from 1.36	Drops the item next to the NPC, to the right or below depending on where the player stands (doc).[309,310,11]
602	take	I	1.33	Picks up the first item of that type lying within the NPC's radius (half its diagonal) and gives it to the NPC.[309,310,11]
609	removecompus	—	1.33	Deletes every baddy (doc: causes compusdied).[309,310,11]
617	putcomp	BRR	1.33	Offline: spawns that baddy at (x, y). Online: asks the server.[309,310,11]
623	shootarrow	D	1.33	Fires that projectile from the NPC's edge in the given direction (reduced mod 4). Drawn on the NPC's layer, or the player's for weapons.[309,310,11]
624	shootfireball	D	1.33	Fires that projectile from the NPC's edge in the given direction (reduced mod 4). Drawn on the NPC's layer, or the player's for weapons.[309,310,11]
625	shootfireblast	D	1.33	Fires that projectile from the NPC's edge in the given direction (reduced mod 4). Drawn on the NPC's layer, or the player's for weapons.[309,310,11]

Op	Command	Args	In the table from	What 2.05 does
626	`shootnuke`	`D`	1.33	Fires that projectile from the NPC's edge in the given direction (reduced mod 4). Drawn on the NPC's layer, or the player's for weapons.[309,310,11]
627	`putbomb`	`RRR`	1.33	Bomb of power 0..2 at (x, y).[309,310,11]
628	`putexplosion`	`RRR`	1.33; takes arguments from 1.41 rev1	Explosion of radius *r* at (x, y), with sound.[309,310,11]
629	`putleaps`	`RRR`	1.33	Leaps of type 0..5 at (x, y) (bush, swamp, stone, sign, ball, water).[309,310,11]
630	`shootball`	—	1.33	Fires a ball from the NPC's centre toward the player.[309,310,11]
640	`puthorse`	`MRR`	1.33	Horse with that image at (x, y).[309,310,324,11]
643	`noplayerkilling`	—	1.33	Turns off player-vs-player damage in this level.[309,310,11]
651	`putnewcomp`	`BRRMR`	1.33	Same, plus image and power.[309,310,11]
661	`hitcompu`	`RRRR`	1.33	index, power, fromx, fromy: damages and knocks back the baddy, and sends it online.[309,310,11]
662	`hitplayer`	`RRRR`	1.33	index, half-hearts, fromx, fromy. Local player: hurt with knockback. Another player: UDP 21 to that player only (skipped if they are dead or protected). An NPC: treated like `hitnpc`.[309,310,11]
692	`removebomb`	`R`	1.36	Deletes bomb *index*.[309,310,11]
693	`removearrow`	`R`	1.36	Deletes arrow *index* (local only, doc).[309,310,11]
694	`removeitem`	`R`	1.36	Deletes item *index*.[309,310,11]
695	`removeexplo`	`R`	1.36	Deletes explosion *index*.[309,310,11]
696	`removehorse`	`R`	1.36	Deletes horse *index*.[309,310,11]
697	`explodebomb`	`R`	1.36	Detonates bomb *index* now.[309,310,11]
698	`reflectarrow`	`R`	1.36	Reverses arrow *index*. Against a character NPC it bounces off the side it hits; otherwise both directions flip. The arrow's sprite turns to its new direction and the shooter becomes the player when called from a weapon.[309,310,11]
699	`lay2`	`IRR`	1.36	Drops the item at (x, y).[309,310,11]
700	`take2`	`R`	1.36	Picks up item *index*.[309,310,11]
706	`hitnpc`	`RRRR`	1.38	NPC *index* gets the damage; its script gets `washit` style handling (no automatic push, doc).[309,310,11]
715	`spyfire`	`RR`	1.41 rev1	Spawns a fire-spy effect just in front of the player (player position plus their facing direction) and sends it online. (role of the two arguments) In 1.41 rev1 and NewMain 1.42 it parses but does nothing; it works from 2.0.[309,310,11,325]
722	`putexplosion2`	`RRRR`	1.41 rev1	Power, radius, x, y; power 3 is a jolt-bomb explosion.[309,310,11]
741	`hitobjects`	`RRR`	2.01	Hits everything at (x, y) with *power*, like a sword, and sends it online.[309,310,11]

8.2.4.8 Board, map, images

This group changes the board, the map and shown images[309,310].

Op	Command	Args	In the table from	What 2.05 does
657	`showimg`	`RMRR`	1.33	Image *index* at (x, y), on the NPC's layer +1 (player's for weapons). Re-using an index replaces it; the same file only moves. A name starting with `@` is a text image.[309,310,324,11]
658	`hideimg`	`R`	1.33	Deletes image *index*.[309,310,11]
664	`updateboard`	`RRRR`	1.33	Redraws the board rectangle (x, y, w, h) after `board[]` writes (doc: also syncs online).[309,310,11]
665	`putobject`	`ORR`	1.33	Offline only: stamps a predefined tile object on the 64×64 board at (x, y), clipped to the board, then redraws. Ignored online (doc: security).[309,310,11]
671	`setminimap`	`FFRR`	1.33	Same for the minimap.[309,310,11]

Op	Command	Args	In the table from	What 2.05 does
672	`setmap`	FFRR	1.33	Loads the big map image and its level-names file, sets the player's head-icon position, and recomputes "is on map" for every player. Both files must exist.[309,310,11]
704	`changeimgpart`	RRRRR	1.38	Visible part of image *index*; x, y clamped to 0..16000, w, h to 0..220.[309,310,11]
718	`changeimgvis`	RR	1.41 rev1	Layer of image *index*: 0, 1, 2 = under / level with / over the NPC; 3 = light layer (like `drawaslight`); 4 = screen (over the status bar).[309,310,11]
730	`changeimgcolors`	RRRRR	NewMain 1.42	Colour multiply for image *index*. Online gate applies.[309,310,11]
731	`changeimgzoom`	RR	NewMain 1.42	Zoom for image *index*. Online gate applies.[309,310,11]
746	`addtiledef`	SSR	2.04	Use image *file* instead of `pics1.png` for levels starting with *prefix*. Online gate applies.[309,310,11]
747	`addtiledef2`	SSRR	2.04	Same, but pastes the image at (x, y) of the tileset. Online gate applies.[309,310,11]
748	`removetiledefs`	S	2.04	Removes tile definitions for *prefix*. Online gate applies.[309,310,11]
758	`loadmap`	F	2.05 rev1	Loads a gmap file; if the current level belongs to it (and the editor is closed), rebuilds the level view.[309,310,11]

8.2.4.9 Parsed but ignored by the 2.05 client

These commands parse but do nothing in 2.05, because its executor has no case for them. The last column says what they do elsewhere, where that is known[309,310].

Op	Command	Args	In the table from	What 2.05 does
636	`seturllevel`	M	1.33	parsed, no executor case; doc describes it as an offline URL warp; not implemented here.[309,310,11]
681	`setfocus`	RR	1.33	parsed, no executor case; later logs mention a camera focus at (x, y).[309,310,11]
682	`resetfocus`	—	1.33	parsed, no executor case[309,310,11]
690	`setbacktile` †	R	1.34; takes arguments from 2.05 rev1	parsed, no executor case; allowed in preview mode but still a no-op.[309,310,11]
707	`canwarp2` †	—	NewMain 1.42	parsed, no executor case; warp permission is handled by the server in this era (inf).[309,310,11]
721	`drawovertrees` †	—	1.41 rev1	parsed, no executor case[309,310,11]
724	`canwarp` †	—	1.41 rev1	parsed, no executor case[309,310,11]
734	`join` †	S	2.0	parsed, no executor case; server-side class join in later versions (inf).[309,310,11]
735	`freezeplayer2` †	—	2.0	parsed, no executor case[309,310,11]
736	`unfreezeplayer`	—	2.0	parsed, no executor case[309,310,11]
755	`setbacktile2` †	RRRRR	2.05 rev1	parsed, no executor case[309,310,11]
756	`setz` †	RRRRRRRR	2.05 rev1	parsed, no executor case[309,310,11]
757	`updateterrain`	—	2.05 rev1	parsed, no executor case[309,310,11]

8.2.5 Functions and variables

Expressions resolve names through a table of functions and variables with numeric ids[309].

Ids	Group	Names
100–102	board and conversion	`board[R]`, `strtofloat(S)`, `timevar`[309]
200–218	object counts and globals	`compuscount`, `playerscount`, `bombscount`, `arrowscount`, `itemscount`, `exploscount`, `horsescount`, `npcscount`, `signscount`, `selectedweapon`, `screenwidth`, `screenheight`, `tokenscount`, `paramscount`, `actionplayer`, `weaponscount`, `levelorgx`, `levelorgy`[309]

Ids	Group	Names
400–455	maths, tests, string tests	`random(R,R)`, `sin(R)`, `cos(R)`, `arctan(R)`, `int(R)`, `abs(R)`, `arraylen(R)`, `testcompu(R,R)`, `testplayer(R,R)`, `testbomb(R,R)`, `testitem(R,R)`, `testexplo(R,R)`, `testhorse(R,R)`, `testnpc(R,R)`, `testsign(R,R)`, `tiletype(R,R)`, `onwall(R,R)`, `playersays(S)`, `playersays(R,S)`, `playersays2(S)`, `playersays2(R,S)`, `strequals(S,S)`, `strcontains(S,S)`, `hasweapon(S)`, `onwater(R,R)`, `keydown(R)`, `startswith(S,S)`, `indexof(S,S)`, `strlen(S)`, `onmapx(S)`, `onmapy(S)`, `vecx(R)`, `vecy(R)`, `getz(R,R)`[309]
600–606	`compus[i].*`	x=600, y=601, type=602, dir=603, headdir=604, power=605, mode=606[326]
640–643	`bombs[i].*`	x=640, y=641, power=642, time=643[326]
660–666	`arrows[i].*`	x=660, y=661, dir=662, dx=663, dx=664 (shadowed), type=665, from=666[326]
680–683	`items[i].*`	x=680, y=681, type=682, time=683[326]
700–704	`explos[i].*`	x=700, y=701, power=702, time=703, dir=704[326]
720–726	`horses[i].*`	x=720, y=721, dir=722, bushes=723, bombs=724, bombpower=725, type=726[326]
740–758	`this.*` and `npcs[i].*`	`x`, `y`, `width`, `height`, `rupees`, `darts`, `bombs`, `hearts`, `timeout`, `glovepower`, `swordpower`, `shieldpower`, `hurtdx`, `hurtdy`, `id`, `save[2]`, `dir`, `sprite`, `ap`; save takes an index[309]
780–781	`signs[i].*`	x=780, y=781[326]
800–819	`player*` and `players[i].*`	`playerx`, `playery`, `playerrupees`, `playerdarts`, `playerbombs`, `playerhearts`, `playerfullhearts`, `playerdir`, `playerglovepower`, `playerbombpower`, `playershootpower`, `playerswordpower`, `playershieldpower`, `playerheadset`, `playersprite`, `playerid`, `playersaysnumber`, `playermp`, `playerap`, `playerhurtdx`[309]
820–825	`player*` and `players[i].*`	`playerhurtdy`, `playerfreezetime`, `playerhurtpower`, `playerattachid`, `playerattachtype`, `playerz`[309]

The seven families above have 36 exact compiled records / 35 distinct family/member spellings. The parser matches stored suffixes and exits on its first successful record: arrow `dx=663` shadows the duplicate `dx=664` vertical getter, with no builtin `dy` spelling. This replaces the documentation-order inference for the exact checked build → App. E#204. Expression IDs are language identifiers, not binary addresses or serialized bytecode[326].

The NewWorld betas add eight time variables after the direction list. They are variables (also usable as `#v(name)` codes), not commands; the server sends the time in packet 42[208]:

Variable	Range
`nwday`	1…28[208]
`nwweek`	1…40[208]
`nwyear`	1000…[208]
`nwweekday`	1…7[208]
`nwmonth`	1…10[208]
`nwtime`	0…1439 minutes[208]
`nwhour`	0…23[208]
`nwmin`	0…59[208]

8.2.6 Constants

Several GS1 values are small numbers with fixed meanings. Each row gives one set of constants and its values[309].

Constant set	Values
baddy modes	0 walking, 1 looking, 2 hunting, 3 hurt, 4 bumped, 5 dying, 6 shooting (swamp), 7 jumping (frog), 8 shooting (spider), 9 dead[309]
player sprites	0 idle; 1–8 walk; 9–13 sword; 14–18 push; 19–22 pull; 23 lift; 24 carry idle; 25–32 carry walk; 33 shoot; 34–36 ride; 37 sit; 38 sleep; 39 hurt; 40 dead; 39 and 40 only on `showcharacter` NPCs[309]
tile types (`tiletype(x,y)`)	0 nonblocking, 2 hurting underground, 3 chair, 4 bed top, 5 bed bottom, 6 swamp, 7 lava swamp, 8 near water, 9 near lava, 11 water, 12 lava, 20 throw-through, 21 jumping stone, 22 blocking[309]

Constant set	Values
`move` options	bounded static queue/option consumer → §8.2.9; residual scheduling/collision questions → App. E#206[219]
`showstats` mask	1 ASD, 2 icons, 4 rupees, 8 bombs, and further bits the notes do not list[310]
`changeimgvis` layers	0, 1, 2 under, level with, over the NPC; 3 light layer, as `drawaslight`; 4 screen, over the status bar[310]
`putleaps` types	0 bush, 1 swamp, 2 stone, 3 sign, 4 ball, 5 water[310]
`seteffectmode` (2.22)	0 light, 1 transparency, 2 holes[309]

8.2.7 The command table by build

The executor matches commands against one table of fixed-size records[11]. In 2.05 and the NewWorld betas a record is 12 bytes: name, a flags word and an argument-signature string[208]. Other keyword lists that follow the table in some builds (`*=`, `:=`, the 1.33 soldier names, the NewWorld time variables) are not counted[11].

Build	Commands	Added over the previous row
1.33	88	n/a; earliest build read[11]
1.34	93	`dontblocklocal`, `blockagainlocal`, `showlocal`, `hidelocal`, `carryobject`, `throwcarry`, `setbacktile`; `seteffect` and `unseteffect` leave the table[11]
1.35	93	none; same table as 1.34[11]
1.36	104	`lay2`, `take2`, `takehorse`, `takeplayerhorse`, `removebomb`, `removearrow`, `removeitem`, `removeexplo`, `removehorse`, `explodebomb`, `reflectarrow`; `lay` now takes arguments[11]
1.37	106	`playlooped`, `stopsound`[11]
1.38	109	`setlevel2`, `changeimgpart`, `hitnpc`; `setlevel` now takes arguments[11]
1.39	111	`setani`, `setcharani`[11]
1.41 rev 1	126	`canwarp`, `callnpc`, `say2`, `break`, `continue`, `callweapon`, `spyfire`, `function`, `return`, `changeimgvis`, `setimg`, `setimgpart`, `drawovertrees`, `putexplosion2`, `showstats`; `putexplosion` now takes arguments[11]
NewMain 1.42	134	`canwarp2`, `setshape`, `seteffect` (back), `setcoloreffect`, `setzoomeffect`, `drawaslight`, `changeimgcolors`, `changeimgzoom`[11]
2.0	142	`setgender`, `setchargender`, `join`, `freezeplayer2`, `unfreezeplayer`, `enabledefmovement`, `disabledefmovement`, `setmusicvolume`[11]
2.01	144	`openurl2`, `hitobjects`; `openurl` now takes arguments[11]
2.02 rev 2	146	`tokenize`, `tokenize2`[11]
2.03	148	`triggeraction`, `move`[11]
2.04	157	`addtiledef`, `addtiledef2`, `removetiledefs`, `replaceani`, `attachplayertoobj`, `detachplayer`, `setshape2`, `enableselectweapons`, `disableselectweapons`; `setshape` now takes arguments[11]
2.05 rev 1, rev 2	161	`setbacktile2`, `setz`, `updateterrain`, `loadmap`; `setbacktile` now takes arguments[11,277]
2.1 tests	161	none; same table as 2.05[11]
NewWorld beta 1	161	none; same table as 2.05[11,208]
NewWorld beta 2	165	`addstring`, `insertstring`, `removestring`, `deletestring`, inserted after `loadmap`[11,208]

Every command or variable the 2.01 to 2.05 change logs name appears in that build's string table; the only undocumented additions are `actionplayer`, `lighteffectsenabled` (2.03) and the `bomy_*` animation names[27,327]. The 2.1 tests add the strings `ascii`, `defnpc`, `playeronline`, `startinfullscreen` and `tiles` (test 2) and `groundheights`, `waterheight`, `playerhp` and `playermaxhp` (test 3) outside the command table[28].

Being in the table does not mean a command does anything. Executor cases were read for 2.05 (→ §8.2.4.9) and for the NewWorld betas against their bases; other builds have not been checked[11].

8.2.7.1 NewWorld betas

The NewWorld betas add commands of their own. The rules below give the string-list commands, how lists are stored, and the commands that work in the betas but are ignored in 2.05[208].

Item	Rule
string lists (beta 2 only)	`addstring list,text` (S,S), `insertstring list,index,text` (S,R,S), `removestring list,text` (S,S), `deletestring list,index` (S,R), with `sarraylen(list)`, `lindexof(list,text)` and `#I(list,i)`[208]
list storage	a list is a normal string flag in comma form: `numbers="12 34",567,"8,9"`[208]
live in the betas, ignored in 2.05	`setfocus`, `resetfocus`, `setbacktile`, `drawovertrees`, `setbacktile2`, `setz`, `updateterrain`[208]
parsed but ignored in the betas, live in 2.05	`removecompus`, `putcomp`, `putnewcomp`, `hitcompu`; `puthorse`, `takehorse`, `takeplayerhorse`, `removehorse`; `carryobject`, `throwcarry`, `takeplayercarry`; `setsword`, `hidesword`, `setbow`; `freezeplayer`; `enabledefmovement`, `disabledefmovement`; `toinventory`; `followplayer`[208]
why	the control NPC script (`controlnpc.txt`) takes over movement, swords, carrying and horses[208]

8.2.7.2 Scripting changes from 2.1 to 2.31

From the release notes; the binaries have not been read for the command tables of these builds[279].

Version	Scripting change
2.02	string parsing[279]
2.04	gani `PLAYSOUND PARAM1`, `replacegani`[279]
2.1	gani scripts (`SCRIPT … SCRIPTEND`), string arrays, `play2`, `graalversion`[279]
2.12	`wraptext`, `wraptext2`[279]
2.15	`showimg2`[279]
2.16	`seteffectmode` (real transparency); gani attributes `#P10`–`#P30`[279]
2.17	sound through BASS, with the same GS1 sound commands plus `musicpos` and `musiclen`[251]
2.19	`this.` strings on the client; on NPC-server servers only `client.` strings are sent[279]
2.2	`updateboard2` for server-side tile changes on gmaps[279]
2.21	`triggeraction 0,0,clientside,…` reaches a weapon's client-side part[279]
2.22	`gravity`, `downloadsize`[279]
2.3	classic client-side NPC mode removed; `timevar2`[279]
2.22 (symbol table, against 2.17)	added `gravity`, `textwidth(zoom,font,style,text)`, `textheight(zoom,font,style)`, `isthisflagset`, `isplayerflagset`, and a #R message code with a string argument; removed `isflagset`, which the two new tests replace[30]
2.31 (symbol table, against 2.22)	added `timevar2`, `downloadsize`, `onwall2(x,y,w,h)`, `onwater2(x,y,w,h)`; nothing removed[30]
2.22 `downloadsize`	announced in the 2.22 notes but absent from the 2.22 symbol table; a 2.22 script sees it as undefined (0)[30]

8.2.7.3 2.05 against the 2.22 documentation

2.05 and the 2.22 documentation do not list the same commands. Each row gives one direction of the difference[309].

Item	Names
documented for 2.22, absent in 2.05	`addstring`, `insertstring`, `replacestring`, `removestring`, `deletestring`, `putnpc2`, `showtext`, `showtext2`, `showpoly`, `showani`, `play2`, `setshootparams`, `shoot`, `enablefeatures`, `seteffectmode`, `changeimgmode`, and the mouse and keypressed family[309]
in 2.05, not in the 2.22 `commands.rtf`	the † rows of §8.2.4[309]

8.2.8 Superseded readings

An earlier catalogue of GS1 commands, built from public tables and the scrape before 2.05 was read, is corrected by the 2.05 reading[310]. The corrected values are the ones in this chapter.

Earlier reading	Corrected
R is the literal string "real"	R is a numeric expression[310,321]
D is 0 south, 1 west, 2 east, 3 north	D is 0 up, 1 left, 2 down, 3 right, or a number[310,321]
`warp` and `jump` are commands	neither is a GS1 command in 2.05 (→ §8.1.8)[310]

Earlier reading	Corrected
`canwarp`, `canwarp2`, `join`, `setfocus`, `resetfocus`, `unfreezeplayer` act	they compile but do nothing in 2.05[310]
`spyfire` is a debug hook	`spyfire` is a game effect with a network message (PLI 10)[310]
the 2.05 table has 163 commands, or ends at 758	161 commands, 600–760, ending with `show` and `hide`[321,309]
2.05 runs OpenGS1-style bytecode	2.05 interprets a parse tree built from script text (→ §8.1.2)[321]

8.2.9 Queued move consumer (exact 2.05 rev 2)

The `move` command queues motion that a consumer applies tick by tick. The rules below give that consumer as read exactly in 2.05 rev 2. The scope row says how far the reading goes[219].

Rule	Detail
scope	exact Windows input identified by source; static instruction checks only; no measured scheduler frequency or cross-build generalization[219]
start/replacement	start includes current position and queued deltas; low bits 0 clear, 1 clear if endpoint is more than five Manhattan units away, 2/3 retain; at least 100 queued records forces clear; start is computed before clear and can become actor position[219]
update	first consumer update sets recorded start; remaining duration/0.05 undergoes floor(value + 0.0001) for ordinary finite in-range inputs; divide remaining deltas by that integer, then subtract applied deltas and 0.05[219]
completion	nonpositive integer bypasses normal collision probes; duration below 0.0001 also completes; set endpoint and remove record; sub-step duration may bypass probes on its first consumer update, not synchronously in the command[219]
16	face dominant delta axis, vertical on ties; probes use resulting facing[219]
probes	direction-specific character body-edge point or image/shape-derived point in tiles; not a swept box[219]
4	broad blocking checks remote players/NPCs/bombs, an unidentified collection, blocking tiles and conditional local-player body; moving NPC temporarily nonblocking; exact eligibility partial[219]
32	same category predicate as onwater; classic board recognizes IDs 64 ≤ id < 288, another path uses a table; not a generic wall test[219]
collision/8	collision cancels entire queue without attempted displacement; option 8 queues movementfinished on completion or collision, not direct event-body execution[219]
64	character auto-animation: water first; otherwise empty queue or both deltas below tolerance selects idle; at least half-tile delta on either axis selects walking, smaller selects slower walking; special profile uses its own names[219]
residuals	scheduler/catch-up, full collision eligibility, event execution, overflow/nonfinite inputs, other bits and animation playback remain open → App. E#206[219]

8.2.10 Exact-build object getter semantics

For these seven families, evaluation computes the numeric index and, for ordinary finite in-range values, uses `floor(value + 0.0001)`. It bounds-checks the local player's current-level list; absent level, negative or end/out-of-range index yields initialized numeric zero. Indices are mutable list positions, not persistent IDs; event/expression snapshot guarantees are unproved[326].

Compus, bombs, arrows, items, explos and horses expose double x/y coordinates; signs expose integer coordinates converted to floating results. Compus state and bomb power are direct integers. Bomb/item/explosion time divides an integer timer by 20, without measuring scheduler cadence. Arrow direction uses displacement, horizontal winning ties: nonnegative horizontal is right, negative left, otherwise nonnegative vertical down and negative up; zero vector reports right. Its first dx is horizontal and the shadowed duplicate vertical. Arrow type/from are raw integer state with enum/producer meanings separate[326].

Item type decodes an identity, searches 25 entries and returns the matching index or minus one (last match if duplicated); its helper checks redundant values and may take an integrity-failure path. Explosion direction maps stored state through a shared table whose full contents/invalid-index behavior remain unvalidated. Horse

dir/bombpower are direct integers; bushes/bombs use specialized decoded getters, with redundant-value checks in bushes and incomplete bombs-helper semantics. Horse type maps a stored boolean to zero/one without establishing semantic varieties. These getters are not all inert reads[326].

Two original-client offline runs contribute twelve displayed controls. Signs at (14,24)/(39,37) read correctly, negative/end indices yield zero, and 0.9998/0.99995 choose first/second sign. Up/right arrows show direction 0/dx0 and direction 3/dx1. A separate up-arrow snapshot after requested sleep 0.2 has count1, direction 0, dx0 and unassigned dy0 while y decreased4; zero dy does not prove no vertical motion or a general fallback rule. A requested power1 bomb at (20,35) reads count1/x20/y35/power1/time3. No writes, live AST or timing cadence were measured[326].

The exact unchanged original Windows 2.05r2 ran under Wine/private display in isolated namespaces without a server, using explicit 2.31 donor assets and owned fixtures. Both supervisors exited zero and cleanup completed; the count2 result is unobscured despite a nearby minimap. Identity/static/visual observations are researcher supplied. Fresh parser/table/bounds instructions and recovered matching getter/helper text support the mapping; this is not full instruction validation of every helper. Writes, setter clamps/notifications, unknown-member fallback, scanner case/quoting, enum meanings, explosion mapping, horse decoder/integrity consequences and other builds remain open[326].

8.3 GS2 language

GS2 is the script language of the 4.x and later clients. This chapter covers where it appears and how the server delivers it (§8.3.1, §8.3.2), the script machine the 4.2, 5.0 and 6.00 engines contain (§8.3.3), how events, waits and the action queue are scheduled (§8.3.4), the names and API the clients publish (§8.3.5), and the tools and third-party compilers around the language (§8.3.6). The compiled container and the opcode dispatch are in → §8.4. GS1 is in → §8.1.

> Status: draft. The machine in those three engines is in §8.3.3; syntax and the standard library stay open (→ App. E#207).

8.3.1 Where GS2 appears

GS2 replaces client-side GS1 in steps. Each row gives a group of builds and the scripting they run[279].

Builds	Scripting
2.3	classic client-side NPC mode removed; Classic moves to the NPC server[279]
2.22, 2.31	GS1 only[10]
3.x (`G3D03014`, `G3D28015`)	GS1 natively, with TorqueScript `.dso` files for menus; 3.153 handles the NPC bytecode ids[10]
4.x (`G3D17026`, `G3D27076`, `G3D20126`)	GS2 bytecode, server-pushed ganis (PLO 195), update packages `GRPKG001`[10]
4.04 rev 2	still runs GS1 weapon scripts and sends only `client.` string flags (→ §8.1.1)[276]
4.x 3D engine	the Torque console and TorqueScript are gone; every engine object is exposed to Graal's GS2 script objects; there is no TorqueScript compiler and `.dso` files are never requested[328]
5.x (`G3D22067`, `G3D14097`, `G3D15010`, `G3D26090`)	GS2, with the `GRESCR01` script cache[10]
6.x	GS2; in 6.11 even the packet id tables come from the server's script[10]

8.3.2 Delivery

How the server hands GS2 code to a client. Which message id carries the compiled code is disputed → App. E#202.

Item	Rule
NPC script	for v4 and later clients the NPC script prop is replaced by `NPCPROP_CLASS` plus a bytecode message; 2.05 never sees that path[2,228]
NPC bytecode	`PLO_NPCBYTECODE` (131), `i3` `code` then the bytecode; layout and version gate → §8.4.1[3,316]
weapon script	`PLO_NPCWEAPONSCRIPT` (140): `s` `info_length`, then the script[3]
script load	`PLO_LOADSCRIPT` (197): a list `type,name,saveToDisk,desKey,crc32`; type is weapon, npc, class or gani; the key is two `i5`[3]
3D mode	the 3D server, not the TCP game server, delivers the client's GS2 weapons and flags in 3D mode, as text commands `addweapon`, `weaponscript`, `deleteweapon` and `addnpc` (gives a named client-side object its script)[328,34]

8.3.3 The script machine in the 4.2, 5.0 and 6.00 engines

GraalEngine 4.2.0.8, GraalEngine 5.0.0.7 and unpacked 6.00r2 and 6.00r4 each contain this runner. The 3.04 Linux client is TorqueScript menus plus native GS1. The container is → §8.4.2. The opcode names, including opcodes 9 and 10, are in `protocol/gs2-client-vm.md`. The fusion numbers and the per-version case list stay in the note.[315]

Item	Rule
Script object	a `TGraalVar` owns one `TScriptExecutionSpace`. Player, NPC and level are `TServerPlayer`, `TServerNPC` and `TServerLevel`. A class is a `TGraalScript` in a global table, joined onto the runner. A weapon is a named variable given bytecode, with an encrypted `.code` cache[315]
Call	the name on top is looked up in this script, then in the other object's script, then in the built-in commands. Too few arguments is `Not enough parameters for`. A miss becomes the number 0. More than 9999 frames already stacked is `Callstack overrun`[315]

Item	Rule
Unqualified name	`temp`, `params`, `this`, `thiso`, `player`, `playero`, `level`, then `client` / `clientr` / `serverr`. Then `with` objects, the active frame, a property of `thiso`, the level, the universe, and `playero`. No leading `$`[315]
Events	functions. The runner queues `created`, `initialized` and `istimeout`. A leading `on` is stripped by `waitfor`. `playerenters` is the first name in the `script_event` mask[315]
Clock	one double. In 2D, each script pass adds 0.05 once the clock is non-zero. Otherwise it is seconds from the performance counter. Opcode 9 adds 1 and stops with `Loop limit exceeded` when the count is greater than 10000[315,329]
Bytecode	segment layout is → §8.4.2. A trailing `0x0A` is not required. Id 131 is a three-byte Graal integer then the bytes. Id 140 stays an envelope (→ §8.4.2) and is not a case of this engine's control dispatch[315]
Stack	tags 0..5. The value stack grows by 100 and the cap is 10000, not 256. Opcode 10 stores that height on the frame and does not jump. Other jumps are the truncated operand plus the current list's base[315,329]
5.0 and 6.00	5.0 adds opcodes 48, 49, 54 and 121 and the same 0.05 clock. 6.00r2 and 6.00r4 add 54, 121 and 234..242; the 0.05 step was not found there[315]

Android Classic 1.8

The Android Classic 1.8 x86 engine reads the same four-segment container, decoded into 20-byte records with 165 dispatcher cases. Its rules were read statically for this one build and are not assumed for the Windows engines.[330]

The machine advances the index before dispatch, and an opcode without a handler continues without a pop. Two conditional branches always pop; the short-circuit pair keeps the value when it jumps. The loop budget is 10,000 per generation, and the call stack refuses a frame at 10,000, but the value stack has no 10,000 cap here.[330]

Division and remainder by zero give 0, and remainder uses floor. Numbers compare equal within 0.0001. Inside the machine any non-zero number is true, but a boolean passed to a native call uses `floor(v + 0.0001)`, so 0.5 is true in the machine and false for a native call. Strings compare without case; substring and search count bytes. Arrays read 0 out of bounds, do not grow on write, and clamp to 0 to 10,000.[330,330,330]

The event queue refuses at 1,000 and drops `onAllRCChat` silently at the limit; `istimeout`, `created` and `initialized` go to the front. Scheduled events refuse negative delays and lowercase the name. Joining a class appends it once, raises its count and installs its event catchers. 173 native bindings are registered, 172 with recovered names.[330,330]

8.3.4 Events, waits and the event queue

The runner in §8.3.3 treats events as functions. How a queued event, a `sleep` and a pending wait are scheduled has been observed or read build by build; each reading below holds for its own build and input, and none is an all-version rule (→ App. E#207).[315]

Linux 4.04r2: event ordering

Event order was tested on Linux 4.04r2 with instrumented scripts. Each row gives one condition, what was seen and the limit of that test[318].

Tested condition	Observation	Limit
Instrumented profile-script fixture	Top-level statements run before the creation handler and again before the queued handler; a second control reproduces the order	Two tested event dispatches in one lifecycle, not an all-version rule[318]
Handler queues another event	Its following statement runs before the queued handler	Delivery is deferred in this experiment; other event sources remain unmeasured[318]
Trigger arity control	Event name alone is rejected; an additional value permits delivery	This fixture's call only, not a universal variadic signature[318]

The fixture replaces process input, not the executable file or its signature checks. Server and server commit are none. Joined classes, public calls and complete event semantics remain open → App. E#207.[318]

Linux 4.04r2: sleep continuation

`sleep` was tested the same way. Each row gives a condition, what happened and what was not measured[331].

Condition	Observation	Unmeasured
Controlled persistent main-menu script requests positive sleep	Saves the next instruction, clears the running position, restores the same snapshot and next instruction, then executes the following statement	Cancellation and replacement while suspended[331]

Condition	Observation	Unmeasured
Sleep versus another queued event	Top-level output does not recur for the continuation, but recurs before the separately queued event	Other object lifecycles[331]
Requested sleep 0.15 seconds	Requested duration is retained across save/restore	Calibrated elapsed delay, clock rate, frame dependence and timeout precision; no exact wall-clock duration established[331]

This is an input-substitution experiment with no server or server commit; later Windows fixed-step clocks are not imported. Remaining scheduler questions → App. E#207.[331]

Linux 4.20r6: selector 11, saved state and registration

Selector 11 is a handler in examined 4.20/6.x builds, unlike the 4.04 skip path; the detailed result is r6-only. It consumes one numeric and two string-like values. Saving requires a nonempty name, ordered positive numeric input and contexts; zero does not enter this path as an indefinite wait. Registration precedes caller lowercase/leading-`on` removal. Empty/current-owner, distinguished global and named routes do not establish full comparison or subscription rules.[332]

State saving transfers the next instruction, clears active execution, copies typed values and transfers/reconstructs collections. Insertion normalizes the key; an empty requested target permits comparison against the current owner's name as a conditional, asymmetric matching fallback. Canonical rewriting of stored or query targets is not established. Only the first matching pending record is removed; insertion destroys it and appends the new record. It does not establish uniqueness, complete cancellation or delivery order. Capacity grows by 100 up to 10,000; the requested single slot and alternate numeric branch values are not event/timeout success semantics. The stock corpus has no selector-11 input or public-prefix directory entry. No new runtime experiment/server ran; coercion, resume and lifecycle semantics remain open in E#207.[332]

Windows 5.00: event delivery and the action queue

In the exact backup engine from a used, updated and patched install labelled 5.00r7, delivery and execution are separate. The named route can create an execution space; the enum route needs one and an event-interest check. Both retain additional unresolved gates. Accepted actions are queued and their owner activated; script service separately checks gates/universe, updated classes and scheduled events before execution. This is not a TorqueScript loader finding.[319]

Both inspected receivers reject when the existing queue has 1,000 actions, before duplicate suppression. The named route suppresses matching route/name with null arguments; above nine pending entries its additional non-null comparison uses argument string representations, not pointer equality. Internal names istimeout, created and initialized prepend; other names append. The enum route appends. These internal names are not a proven public event mapping. Activation avoids duplicate owners and has an unresolved special front-insertion category; per-pass markers and list mutation handling do not establish fairness or reentrancy.[319]

Independently authored queue contracts cover null-argument duplicates, lifecycle insertion and overrun-before-dedup, with a synthetic recording executor only. They are static-derived, not original-client output. Allocation failure, argument conversion/collisions, class-download deferral, complete virtual gates, cross-owner ordering and actual scheduler cadence remain open. Do not import this input's limit into another build or infer 20 Hz service. Broader event semantics remain partial in App. E#207.[319]

8.3.5 Names and the published API

Names seen in clients and labs

These GS2 names appear in the notes; each row gives only what the source says about it:

Name	Kind	What the source says
`sendtext`	function	a message of four strings (`weapon, type, option, data`); the 3D plug-in's text messages have this shape[34]
`receivetext`	event	a weapon named in such a message receives it[34]
`onRCChat`	event	receives messages of type `rc`[34,328]
`onRPGMessage(data)`	event	receives the `rpg` command[34,328]
`action<name>`	event	a `t` line in an `events` message triggers it on a weapon or GUI (`weapon`, `gui`) or an NPC (`npc`)[328]

Name	Kind	What the source says
`trigger`, `getweapon`	client events	the client batches them per connection as `t …` and `w …` lines (others as `s …`) in one `events` message[34]
`onConnectionAccepted`, `onConnectRequestTimedOut`, `onConnectRequestRejected`, `onConnectionDropped`, `onConnectionError`	events on the universe	fired for each 3D connection outcome[34]
`onGraal3DLoginStep`	event	reports the three Torque mission-load phases, with the datablock and ghost events[34]
`getDatablockProperties(obj, list)`	function	lists datablock fields (name, type and attributes) for GS2 scripts[328,34]
`addtiledef("pics1.png", "", 0)`	function	called from a GS2 weapon `-System` on a 5.x/6.x lab server; the full level drew after the warp with or without that weapon[4]

Linux 4.04r2: published API membership

Linux 4.04r2 publishes a catalogue of script types and globals. The table gives what the exact input shows and where that stops[333].

Catalog fact	Exact-input result	Limit
Correct declaration grouping	176 published types and 332 globals; 38 declarations belong to the root script object rather than globals	Total declaration count unchanged; this corrects derived metadata, not runtime behavior[333]
Evidence class	Client-published scripting surface	Binding effects, conversions, inherited behavior and native C++ layout are not established by declaration membership[333]

Other catalog builds require their own correction records; these counts are not generalized. Standard-library semantics remain open → App. E#207.[333]

8.3.6 Tools, compilers and reimplementations

Tools around GS1 and GS2

One tool from the period works on GS1 and GS2 levels. The row says what it does and why it matters[334].

Tool	Relevance
LevelScriptEditor	lists every NPC of a folder of `.nw` levels and replaces identical scripts in one go; it reads like a tool for porting GS1 scripts to GS2 on a big world[334]

A third-party compiler and VM

A modern third-party compiler and VM exist for GS2. The rules below say what they are, what runs, and what they leave open[317].

Item	Rule
What it is	a public C# compiler and a public C# VM, both MPL-2.0; a lead for a later clean-room machine[317]
What runs	the compiler takes GS2 source; the engine loads bytecode, and its tests pass the compiler's bytes in[317]
Open	→ App. E#207 stays open for syntax, the standard library, and the event model. The note records a third-party tree, which is not a reading of the clients[317,316]

8.4 GS2 bytecode

This chapter covers compiled scripts on the wire. 2.05 and earlier clients receive GS1 as text and never see bytecode (→ §8.1.2). The container 4.2.0.8, 5.0.0.7, and unpacked 6.00 parse is → §8.4.2; the per-build dispatch ranges are → §8.4.4, and the load-time fusion stages → §8.4.6. The runner, including the value-stack cap and opcodes 9 and 10, is → §8.3.3. OpenGS1's layout, a reimplementation, stays in → §8.4.3 for reference. Syntax, the standard library, and the event model stay open (→ App. E#207). Which message carries the container is still disputed (→ App. E#202).

> Status: draft. The container for the builds in §8.4.2 is recorded. The runner is in §8.3.3. Syntax, the standard library, and the event model stay open (→ App. E#207). The id that carries the container is disputed (→ App. E#202).

8.4.1 Transport

GS2 reaches the client as compiled bytecode. The rules below give the message that carries it, the version gate and the framing[3].

Item	Rule
message	`PLO_NPCBYTECODE` (131, wire byte `0xa3`): `i3` code, then the bytecode[3]
version gate	sent only to clients at or above the GServer-v2 version constant `CLVER_4_0211`, and only when the bytecode is non-empty[316]
framing	wrapped in `PLO_RAWDATA` with a length prefix (→ §2.1.2)[316]
with NPC props	an NPC is sent as `PLO_NPCPROPS` plus the optional `PLO_NPCBYTECODE`[316]
2.05	the 2.05 receive switch does not handle it[3]
other ids	a second reading puts GS2 bytecode on id 197 → App. E#202[2,228]
how the client stores it	a 24-byte instruction record and an op-index program counter (→ §8.4.2). 4.2.0.8 attaches those instructions to the `TGraalVar` (→ §8.3.3)[329,315]

8.4.2 Format

4.2.0.8, 5.0.0.7, and unpacked 6.00r2 and 6.00r4 parse one container: four big-endian segments (event mask, functions, strings, opcodes) and `F0–F6` operands. It does not share OpenGS1's layout (→ §8.4.3). The runner is → §8.3.3. Syntax, the standard library, and the event model stay open → App. E#207.

Item	Rule
container	big-endian types 1–4, not GSHORT, not the `0xAC` wrapper[329]
later builds	5.0.0.7 and 6.00 add dispatcher cases; 6.00 adds folds 234–242[329]
unpacked 6.1.1.3	same four segments, 24-byte instructions, cases 54 and 121 and folds 234–242; 48 and 49 stay absent (→ §8.4.4)[47]
still open	GS2 syntax, standard library, event model[316]

What the clients run

The bytecode these clients run is a big-endian four-segment container. Segment lengths are raw 32-bit counts, so they are not GSHORT (→ §2.1.1). The container is not OpenGS1's layout (→ §8.4.3). Packet 140 and packet 197 stay the envelopes in → §8.3.2. The script object parses the segments, not the `0xAC` file wrapper in → §8.3.6. The runner that executes it is → §8.3.3.

Item	Rule
segments	types 1–4: event mask, functions, strings, opcodes; markers `F0–F6`[329]
4.2.0.8 weapons	`weblevels .code` by CRC32; no `GRESCR01`; 195 is a gani[329]
5.x and 6.00	`GRESCR01` cache; the wire header is unchanged from → §8.3.2[329]
6.113	no container and no opcode switch in the Windows listing. It adds nothing past 6.00's cases 54 and 121 and folds 234–242, and it cannot show whether 48 and 49 returned[335]

Item	Rule
unpacked 6.1.1.3	the switch is present and adds no case past that 6.00 set; 48 and 49 stay absent (→ §8.4.4)[47]
compiler lead	the MPL-2.0 checklist in → §8.3.6, compared with these clients[329]

The opcode names, the peephole, and the bytes the client runs which that compiler does not emit stay in the note[329]. The effect of each of those bytes is[336].

Linux 4.20r6: the container reader

The r6 reader uses big-endian four-byte framing/directory numbers and NUL-terminated names. Removing a leading `public.` marker and forwarding a flag does not define runtime visibility. Markers 240–242 select string entries (one/two-byte unsigned indexes; four-byte form checks nonnegative representation); 243–245 supply signed integers; 246 uses an incompletely reviewed numeric-text converter. Other bytes append tagged records; marker branches modify the latest record. Corpus controls do not cover multiple-marker overwrite, and strict validator rejection is not native bounds/default behavior.[337]

Windows 5.00r7: segment words and four authored vectors

The 5.00r7 backup engine's script setter reads the segment type and length as ordinary big-endian 32-bit words, not the printable packet integers. A `public.` prefix in the function table sets visibility 1 and `publicsoap.` sets visibility 2; the default is 0. Visibility 2 is stored metadata and does not show a callable SOAP service.[202]

Four hand-written vectors make the layers testable without an original payload. A weapon request is logical 158 with the name; a class request is logical 161 with the stored-body CRC as five offset-integer bytes and then the name; `00 00 00 01 00 00 00 04 00 00 00 01` is a container with only an event mask, so nothing in it can run; and a 4.04r2 GS1 `setstring client.` flag should leave as logical 18. These are predictions from the static reading, before outer framing and encryption. None was captured or sent to a client.[202]

8.4.3 OpenGS1 bytecode (reference reimplementation)

OpenGS1 defines a stack-based bytecode VM for GS1[316]. It is an independent reimplementation: 2.05 does not share its opcode set and does not run bytecode at all[309]. How the notes first read this format is → App. E#208.

File layout, in order:

Part	Content
1	u32 body offset[316]
2	u32 string-constant count, then length-prefixed strings[316]
3	u32 number-constant count, then f32 values[316]
4	the bytecode body, starting at the body offset[316]

A packed value is 4 bytes:

Field	Bits	Content
`valueType`	4	constant number, constant string, constant array, or named[316]
`value`	16	an index into the constant table or the name table[316]
unused	12	n/a[316]

Opcodes, by name; the notes give no numeric values:

Opcode	Meaning
`OP_PUSH`	push a packed value[316]
`OP_ASSIGN`	S(1) = S(0)[316]
`OP_ARR_SET`, `OP_ARR_GET`	array access[316]
`OP_ADD` … `OP_POW`	arithmetic[316]
`OP_INC`, `OP_INCPUSH`, `OP_DEC`, `OP_DECPUSH`	increment and decrement[316]
`OP_CALL`, `OP_CMD_CALL`	function call, command call[316]
`OP_JMP`, `OP_JAL`, `OP_RET`	control flow[316]
`OP_EQ`, `OP_LT`, `OP_GT`, `OP_LTE`, `OP_GTE`	comparisons[316]

Opcode	Meaning
`OP_NOT`, `OP_AND`, `OP_OR`	logic[316]
`OP_JEZ`, `OP_JNZ`	conditional jumps[316]
`OP_STOP`	halt[316]
`OP_DBG_OUT`	debug output[316]

8.4.4 Dispatch by build

Each build guards its opcode dispatch over its own range, and the range grows with the engine: Linux 4.04r2 ends at 190, both Linux 4.20 revisions guard through 233, and the three Linux 6.x inputs map a prefix through 242.[338,339] The selectors above the serialized range are fused internal tags made after loading (→ §8.4.6), so they are not wire bytes.[337]

Linux 4.04r2

The Linux 4.04r2 bytecode dispatch table was read exactly. Each row gives one observation and its limit[338].

Item	Exact-build observation	Limit
Dispatch table	Range ends at 190; 191 indexed slots, of which 119 select a non-skip path; slot 11 selects the common skip path	Later fused-opcode range is outside this table; helper semantics not all recovered[338]
Extracted stream coverage	Every opcode in fourteen recovered streams selects a non-skip slot	Does not establish execution of every stream[338]
Slot-11 insertion control	Tested event order and completion are unchanged	One context; later Windows wait behavior is not assigned to this value[338]
Static frame helper	Depth guard, overrun path, frame-like allocation, prior-position save, push and active-state change	Field meanings remain inferred; no successful new live stop confirms this helper[338]

Later-build opcode and frame descriptions retain their own identities. Unrecovered helpers and correspondence remain open → App. E#207.[338]

Linux 4.20 and 6.x

Both 4.20 revisions guard selectors through 233 (234 positions: 80 common fallback and 154 other targets), compared with 4.04's range through 190. The three 6.x inputs share executable-target sets and cache native destinations per decoded instruction. Their inspected prefix through 242 has 167 executable-mapping pointers and 76 zeros; a 243-entry extent is inferred. Adjacent non-code data/zeros are not safe no-ops, and internal selectors are not automatically wire bytes.[339]

Five unchanged startups reach executor entry, without hitting the new frame candidates. Three separate 6.x captures each single-step one selector-23 cached jump after its cached/static/register destinations agree. Termination follows the jump; handler completion and event scheduling are unmeasured. Eight untruncated captures retain identity and verify owned cleanup, with no server. E#207 remains partial.[339]

Unpacked Windows 6.1.1.3

The unpacked Windows 6.1.1.3 image walks the same four segment types and stores a decoded instruction in 24 bytes. Operand markers `F0–F6` keep the 6.00 widths. The dispatcher adds no case past 242. Cases 54, 121 and folds 234–242 are present. Cases 48 and 49 are not. A `.code` file of at least eight bytes is accepted when the magic is `GRESCR01`, the cache flag is 1, and the CRC32 of the bytes after the magic matches.[47,47,47]

Rule	Builds
A `.code` file shorter than eight bytes is rejected. The magic is `GRESCR01`, and the CRC32 covers the bytes after those eight	6.1.1.3[340]
The dispatcher has 152 cases, from 11 through 242, with no case 48, 49 or 200. Cases 54, 121, 201–242 are present. Unpacked 6.037 has the same set	6.1.1.3, 6.037[340]
Those cases include the opcodes the Preagonal compiler does not emit. A full builtin table was not extracted	6.1.1.3[340]

Mac OS X 6.11r1

The Mac 6.11r1 runner was read on its own, so the paragraph and table below describe that one input. It runs decoded 40-byte records rather than the serialized bytes, and each record caches its native handler, so a handler jumps straight to the next one. A preparation pass appends opcode 164 as an end marker. The table was read through a 256-entry window whose entries 243 and 244 are zero; its true length is unknown.[341]

The table lists only the behaviours that were read handler by handler; it is not an opcode map, and the opcode-7 handler is still unread.[341]

Item	Mac 6.11r1 observation
read handlers	0 fall through, 1 jump, 2 numeric branch and pop, 6 call, 8 sleep, 9 loop check, 11 wait for, 20 number, 21 string, 24 one, 27 pi, 164 end marker[341]
loop check	default limit 10000, overridable from the owner's context; the counter lives on the opcode-9 record and resets with recompilation; no general per-instruction fuel was found[341]
sleep and waitfor	snapshot of function, index, stack, frames and object context; the deadline is the manager's clock plus the delay; status 2 after the snapshot[341]
call	arguments run down to the tag-0 marker; access and parameter count are checked; can suspend on `classesdownloaded` and retry the call[341]
status	0 normal end, 1 missing prerequisites or failed loop check, 2 suspension[341]

Nothing was run in a guest, so the clock's unit, the order of due snapshots and the native call interface are open (→ App. E#207).[341]

8.4.5 Built-in names in unpacked 6.1.1.3 and 6.0.3.7

Both unpacked images store the eighteen math names `abs`, `atan`, `atan2`, `ceil`, `cos`, `floor`, `log`, `max`, `min`, `pi`, `pow`, `random`, `round`, `sin`, `sqrt`, `sqrt1_2`, `sqrt2` and `tan`.[342]

Of the names Preagonal's compiler turns into opcodes, these images contain `abs`, `cos`, `log`, `max`, `min`, `pow`, `random`, `sin`, `format` and `sleep`. They do not contain `arctan`, `exp`, `vecx`, `vecy`, `arraylen` or `tokenize`.[342]

`putcomp` and `putnewcomp` are in 6.0.3.7 and not in 6.1.1.3. `setprotocol`, `sendraw` and `senddata` are in 6.1.1.3 and not in 6.0.3.7.[342]

8.4.6 Load-time stages and fusion

Linux 4.20r6: post-load fusion

A jump-index pass and adjacent-pattern fusion rewrite selected records to internal tags 200–233 while retaining collection positions and advancing past matches; a current first tag 11 stays unchanged. This explains a corpus maximum of 189 alongside a larger internal range, without asserting transmitted fused values or a recovered compiler. Offline review runs no client/server. Malformed-input behavior, numeric grammar and public/joined semantics remain open in E#207.[337]

Linux 6.01 x86: decoded program and saved state

In the exact Linux 6.01r1 x86 review, the reader creates decoded records with initially empty native targets. A postpass constructs fused internal tags and caches a registry pointer on one selected path; the executor separately prepares jump targets and adds a termination record. These stages do not define new source-language or serialized opcodes.[343].

Selected saved-state handling mixes copy, transfer and selective cloning, forms a deadline from a scripting clock plus positive input, and shows registration, first-match removal and replacement calls. The call-frame layout differs from 4.20. This is not a uniform deep-copy or portable snapshot contract. Trigger/restore execution, ownership of every field, subscription lifetime, cancellation order, clock cadence and public result values remain unproved. Complete export delivery did not reconstruct the full indirect handler table; no new runtime result or cross-architecture equivalence follows.[343].

8.4.7 Scripts embedded in the Linux clients

Linux 4.04r2: embedded scripts and live linkage

Linux 4.04r2 carries scripts inside its executable. Each row gives one piece of evidence about them, what it showed and the limit of that reading[344].

Evidence	Observation	Limit
Bounded embedded-zlib extraction	Fourteen valid four-segment candidates, 129 named entries and 15,799 decoded instructions; instruction bytes round-trip and indexes are in range	Structural validation is not corpus-wide execution[344]
Profile-script input	A corrected read of 9,530 live bytes matches the recovered payload digest; the same run enters the native executor	One loader/executor linkage, not all streams or complete profile execution[344]
Native reader/writer recovery	Operations available for focused review	Does not itself supply a full format specification[344]

The wrong-representation hash attempt remains superseded; only the corrected read supplies the match. Historical server-delivered content, cross-platform equivalence and 3D gameplay are not established. Remaining container/execution questions → App. E#207.[344,344]

Across Linux builds

Bounded extraction finds 14 containers in 4.04r2, 13 in each 4.20 revision and each 6.01 architecture, and 12 in 6.03r7 x64. The 4.20 pair and 6.01 architecture pair each agree on all 13 inflated payloads; 6.03 is an exact subset of 6.01. Its missing scanned payload has nine entries and 247 instructions, without proving feature removal. Five payloads match between 4.04 and 4.20.[345]

Entry/instruction totals are 119/15,208 for each 4.20 corpus, 55/5,053 for each 6.01 corpus and 46/4,806 for 6.03. All 78 occurrences validate four unique segments numbered 1–4, candidate entry/string bounds and exact instruction-byte reconstruction; maximum observed tag is 189. Neither 3.04 duplicate yields an accepted bounded zlib stream. Roster alignment remains a candidate comparison. Extraction runs no client/server and proves neither execution nor native semantic equivalence; E#207 remains partial.[345]

8.4.8 Network GS2 on Linux 6.03

Three exact Linux 6.03r7 x64 probes establish class parsing, explicitly registered weapon instruction execution and later continuation at the saved next position. The proxy-origin bodies and corrected descriptor-length boundary are independently reviewed. This is not normal server compiler/writer validation, a script return-value oracle, exact sleep timing, cache-hit proof or rendered gameplay.[346].

9 Graal3D (2000)

Graal3D is a separate 3D side project from February 2000, with its own Delphi client (`Graal3D.exe`) and its own server (`GServer.exe`). It does not talk to the 2D GServer or the list server. The Torque-based 3D of the 4.x clients is a different program (→ §10).

> Status: draft.

9.0 What ships and how it relates to 2D Graal

Graal3D is a separate program from 2000 with its own server. The facts below say what the archive holds and where it sits among the 2D releases[347].

Fact
The archive `graal3d.7z` holds the client, the server, `graal3dconfig.txt`, `serverconfig.txt`, two levels (`level1.x`, `startlevel.x`), eight player-model meshes and their BMP textures[347]
Both programs carry resource stamps from 2000-02-05, between 2D releases 1.32 (`GSERV018`) and 1.33 (the first `GNW` token)[347]
The client is Delphi with DelphiX `TDXDraw` in Direct3D retained mode (z-buffer, textures), a 50 ms `TDXTimer`, and zlib 1.1.2 linked[347]
Neither program has `.graal`/`.nw` loaders, GANI, tiles or NPC scripting[347]
The server's default TCP port is 14901, its levels live in `levels\`, and accounts in `accounts.txt`
The server holds at most 200 players; a player id is the lowest free slot from 1 to 200[347]
Every Delphi class in the Graal3D server also exists in the 2D `GServer13.exe` of 1999-11-29; the 2D server has 23 more (NPC, baddy, chest, horse, level-link and board-modify classes, and their deltas)[347]
The account record is 44 bytes in both servers; the level object shrinks from 8,244 bytes (a 64×64 board) to 24 bytes (name, file text, players in it, players already sent the file)[347]
The per-observer "unsent props" tracker, the admin forms and the guild-tag check are carried over from the 2D server[347]
The client keeps the 2D client's class names for the main form, the socket object, the connect window, the options window, the self-check and the DES helper; the level, NPC, weapon and script classes are gone[347]
Each frame tick the client reads TCP, reads UDP, diffs its own player props and queues the changes, then flushes the TCP send buffer[347]
The client windows are the main 3D view with a chat line, the connect window (nickname, host, port 14901, the built-in guest account), "Hardware options" (fixed UDP port, "Don't use UDP") and "Player look"[347]
The server windows are a start dialog, the main window (player count, "Edit accounts", "Update levels", a tray menu), an account list and editor, an online player list, a message window and a level-reload window[347]

9.1 Protocol

9.1.1 Encoding and frames

Graal3D borrows the 2D codec. The rules below give its encodings and how a TCP frame is laid out[348].

Rule
The text codec is the 2D one: `c` is one byte + 32, `sN` is an N-byte integer with 7 bits per byte, most significant first, each byte + 32, and `str` is `c(len)` then the bytes (→ §2.1)[348]
A TCP frame is a 2-byte big-endian length, then one zlib block holding one or more messages, each `c(id)` + payload + `\n`[348]
Both sides compress when the queue passes 4096 bytes or on the frame-tick flush; the client sends in 256-byte slices and inflates into a buffer of up to 256,000 bytes[348]
A client drops the link when more than 256 KiB of compressed output is pending[348]
There is no key byte and no per-packet scramble[348]
Text with line breaks (level files, admin messages) writes each `\n` as § (0xA7)[348]
The client sends its login line and its first props line in one frame; the server answers each event with its own frame, so a server may flush per event[348]
Packet ids and the prop table are numbered from scratch, not shared with 2D; positions are 3D, the angle is one byte and colours are 24-bit[347]

9.1.2 Login

The rules below give the Graal3D login in order, from the server's first message to acceptance or refusal[348].

Rule
On TCP accept, before login, the server sends id 6 with `c(0)`[348]
The client's first line is `c(0)`, the version `GR3DV002`, `str(account)`, `str(password)`; the server reads the version at bytes 2 to 9[348]
`GR3DV001` is recognised and rejected with id 5 and the text "You need at least Graal3D beta2 to play on this server."[348]
A missing, in-use or disabled account gets id 5 with the matching text, then the server closes[348]
The server still parses the rest of a rejected frame and applies the props line to a throwaway player; a server need not copy this[348]
Id 1 with the same layout is an admin login; it needs admin level 1 or more and gets the account list (50) and the player list (55)[348]
A normal account already in use is rejected; a load-only account may log in many times at once[348]

Login order on the server[348]:

Step	Server action
1	Gives the lowest free id, links the account and applies its saved props; a non-empty saved nickname is relayed at once to the others as 2 + prop 12[348]
2	Sends 3 with props 0–5[348]
3	Exchanges 2 with all of props 0–17 between the new player and every other player[348]
4	Sends the player to their level (→ §9.3.2)[348]
5	Tells admin sessions with 55[348]
6	Handles the client's first props line like any other, so the others see props 12 and 15 again[348]

9.1.3 Server → client ids

These are the messages the Graal3D server sends. Each row gives the id, its name in the notes, the payload and what the client does[348].

Id	Name in the notes	Payload	Client action
0	LEVEL	level text with § for \n, or empty	saves and loads it as a `.x` mesh; empty means "you already have it" (→ §9.3)[348]
1	LEVELNAME	rest = file name	stored as the current level name[348]
2	OTHERPLPROPS	`c(player id)` + props	creates the player on first sight, then applies the props[348]
3	PLAYERPROPS	props	applies to the local player[348]
4	leader	empty	sent to the first player in a level and to the next when the first leaves; sets a flag nothing reads[348]
5	DISCMESSAGE	rest = text	shown when the socket closes; empty gives "You have been disconnected from the server."[348]
6	session	`c(v)`	stores a value nothing reads; the server sends 0 on accept[348]
7	FILESENDFAILED	rest = file name	not handled[348]
8	ADMINMESSAGE	rest, § = line break	shown in an "Administrator message" box[348]
16	2D disconnect message	text	not handled; sent only by the server's player-list kick button, so that kick arrives silently[347]
50–59	admin replies	→ §9.2.3	admin sessions only[348]
100	RAWDATA	`s3(len)`	flags the next line as raw, then discards it[348]
101	FILE	`str(name)` + bytes	the server's answer to request 2; unused by this client[348]

9.1.4 Client → server ids

These are the messages a Graal3D client sends. Each row gives the id, the payload and the server's action[348].

Id	Payload	Server action
0	login (→ §9.1.2)	[348]

Id	Payload	Server action
1	props	applies, then relays (→ §9.2.1)[348]
2	file name	looks in `levels\`; sends 100 + 101 if the file is under 64,500 bytes, else 7; the shipped client never sends it
51–64	admin commands	→ §9.2.3[348]

9.1.5 UDP between clients

Like 2D Graal, Graal3D sends some traffic between clients over UDP. The rules below give how peers find each other and what goes over the channel[348].

Rule
Unless `noudp` is set, the client binds one UDP socket on port 0 or on the fixed port from its config, and announces the bound port in prop 15[348]
The server adds the peer's address as prop 16; a client keeps as peers the players flagged `50 = 1` that have both a port and an address[348]
Datagrams have no length prefix and no compression[348]
First byte `c(0)`: the values of props 1–5 back to back without prop ids (9 bytes), a position update; 8 bytes or fewer is ignored[348]
First byte `c(1)`: chat text, at most 200 bytes; longer text is cut and ends in `...`[348]
A datagram is accepted only from the address and port of a known player[348]
Position goes to each peer only when props 1–5 changed since the last tick; chat goes right after Enter, and also to the server as prop 13 in the next props diff[348]
In 2D clients of the same time UDP went to the server; in Graal3D it goes peer to peer (the 2D peer channel is → §2.9)[347]
UDP between two real clients was not seen in the lab, because Wine has no retained-mode `d3drm.dll`[349]

9.2 Props and accounts

9.2.1 Player props

The same ids are used in both directions and inside UDP; a props payload is a run of `c(prop id)` + value with no separators[348].

Id	Value	Field	Default	Client sends
0	c	player id (1–200)	—	no[348]
1	s2	x, `(v − 8000) / 2`	0	yes[348]
2	s2	y, same scale	0	yes[348]
3	s2	z, same scale	0	yes[348]
4	c	angle, v × 2 degrees	0	yes[348]
5	s2	animation tick counter (→ §9.6.3)	0	yes[350]
6	str	body model	`body.x`	yes[348]
7	str	head model	`head.x`	yes[348]
8	str	body texture, reloaded on change	`body.bmp`	yes[348]
9	str	head texture	`man.bmp`	yes[348]
10	4 × 6 bits	arm colour, 24-bit RGB	`FF0000`	yes[348]
11	4 × 6 bits	shoe colour	`800000`	yes[348]
12	str, at most 200	nickname, optional `(guild)`	—	yes[348]
13	str, at most 200	chat text	—	yes[348]
14	str	level file	`level1.x` in the client, `startlevel.x` in the server	no[348]
15	s3	UDP port, 0 = none	0	yes, its bound port[348]
16	5 bytes: 4 bits, then 4 × 7 bits	IPv4 address: the in-memory address word read as a little-endian integer	—	no; the server fills in the TCP peer address[348]
17	4 × 6 bits	hand colour	`FDCC47`	yes[348]

Id	Value	Field	Default	Client sends
50	c	1 = this player is in your level (add as UDP peer), 0 = not	—	server only[348]
51	none	player left: remove	—	server only[348]

Players are described by props, as in 2D. The rules below give what some props mean and how they are sent each tick[348].

Rule
Prop 10 colours the two arms, 17 the two hands and 11 the two feet[348]
Each tick the client re-encodes every prop it sends, compares with the last tick and sends the changed ones as one id-1 message; the first send carries the full look (6–12 and 15)[348]
The wire range of props 1–3 is −4000 to 4095.5 in half-unit steps, which covers the ±4000 world clamp (→ §9.5)[350]

9.2.2 Server relay rules

The server checks and relays props. The rules below give how it treats nicknames and guild tags and who it relays to[348].

Rule
On prop 12 the server splits `nick (guild)` and keeps the tag only if `guilds\guild<guild>.txt` lists the account; if the text had a tag, the result is echoed back with 3; a plain nickname gets no echo[348]
The nickname then goes to every player (2) and every admin session (59)[348]
Prop 15 goes to every player as 2 + prop 15[348]
Prop 14 from a client is stored and relayed to the level, but does not move the player; only the server moves players, at login and when an admin edits an online player (60)[348]
Prop 16 from a client is ignored while the player has a live socket[348]
All changed props in the line go as one 2 line to the players in the same level, leaving out props 1–5 and 13 when both players have a UDP port; when nothing is left, nothing is sent[348]
Each observer has its own dirty set, so a player who was skipped gets the missing props later[348]
On disconnect every player gets `c(id)` + prop 51 and admin sessions get 56 + `c(id)`[348]

9.2.3 Admin protocol

The admin protocol is read from the server alone; no Graal3D admin client is in the archives[347]. Each command needs a minimum admin level (0 none, 1 and 2 staff, 3 full), and an admin can act only on accounts at or below their own level unless they are level 3[348]. An account id is a 28-bit number sent as 4 bytes of 7 bits[348].

Id in	Payload	Effect or reply
51	account id	reply 54, the account record[348]
52	account id, `str` name, `str` password, `str` props, c flags	edit account; 51 (changed) and 52 (renamed) to admins[348]
53	account record	add account; 50 to admins[348]
54	account id	delete account; 53 to admins[348]
59	`c(player id)`	reply 57, that player's account[348]
60	`c(player id)` + account record	edit an online player's account[348]
61	`c(player id)`	kick: the player gets 5 "The server administrator has disconnected you." and the others get prop 51[348]
62	`s2(count)` + `str` names	reload these levels from disk[348]
63	text	message to all players (8) as `Administrator <account>:§<text>`[348]
64	`c(player id)` + text	message to one player (8)[348]

Admin sessions get their own messages. Each row gives an id the server sends out, its meaning and its layout[348].

Id out	Meaning	Layout
50	account added	account id, `c(folder)`, name to end of line; one per account after an admin login[348]
51	account changed	account id, `c(flags)`; sent when a bound player leaves and the record is saved[348]

Id out	Meaning	Layout
52	account renamed	unmeasured[348]
53	account deleted	unmeasured[348]
54	account record	account id, `str(password field)` (came back empty), `str(props)`, `c(flags)`[348]
55	player joined	`c(player id)`, `str(account)`, prop 14[348]
56	player left	`c(player id)`[348]
57	player's account	unmeasured[348]
58	player's account renamed	unmeasured[348]
59	player props	`c(player id)` + props: the nickname, the full props on login, prop 14 on level entry[348]

The rules below explain the folder byte and how the admin ids line up with the 2D ones[348].

Rule
The folder byte in 50 fits 1 for a normal account plus 4 × the admin level, and 0 for a load-only account; the formula is fitted to three accounts[348]
55 and 56 match the 2D add- and remove-player ids; the names of the other admin ids are descriptive, not recovered[348]
The admin kick path (61) sends 5, which the client shows; the player-list kick button sends the 2D id 16, which it ignores[347]
Commands 51–54, 59, 60 and 62 were not exercised live[349]

9.2.4 Account and config files

The server keeps accounts and settings in text files. The rules below give their layout[351].

Rule
`accounts.txt` holds one account per `\n`-terminated line in the +32 codec: `str(name) str(password) str(saved props) c(flags)` and seven `c(0)` bytes; a hand-written file is accepted[351]
The saved props are the player's prop run, props 1–14, 16 and 17 (no 0 or 15); empty until the player first leaves, written back when a bound player leaves or is edited, and applied at the next login, nickname and chat text included[351]
Flags: bit 0 = load only (the session is not tied to the record, props are not saved back, many logins may share it), bits 1–3 = admin level, bit 4 = blocked[348]
The server UI names the levels No rights (0), Graalian Police (1), Trusted Admin (2) and Administrator (3)[351]
The seven trailing bytes are the empty count-prefixed lists of the 2D account record; nothing in Graal3D fills them[351]
The server rewrites `accounts.txt` whenever an account changes[351]
`guilds\guild<name>.txt` lists one account per line; a nickname keeps its `(name)` tag only if the file exists and lists the account, as in the 2D server[351]
`serverconfig.txt` holds `serverport=14901`[351]
The client stores nickname, account and password in the registry under `HKCU\Software\Graal3D`, section `Client`; the password is three DES blocks under a key built into the exe, written as 48 printable characters (obfuscation, not protection; the key is not given here)[351]

Client config `graal3dconfig.txt`, `key=value` lines read at start and written back on change[351]:

Key	Shipped value	Meaning
`serveraddr`	`<public-host>`	server host or address; the connect window defaults to the loopback address[351]
`serverport`	14901	TCP port[351]
`fixedport`	`false`	bind UDP to `fixedudpport` instead of a random port[351]
`fixedudpport`	14898	fixed UDP port[351]
`noudp`	`false`	no UDP socket; position and chat go only through the server[351]
`bodyx`, `headx`, `bodytex`	`body.x`, `head.x`, `body.bmp`	props 6, 7 and 8[351]
`armcolor`, `shoecolor`, `handcolor`	`FF0000`, `800000`, `FDCC47`	props 10, 11 and 17[351]
`accountname`, `password`	—	also accepted by the reader[351]

9.3 Levels (.x)

9.3.1 Level files

A Graal3D level is a DirectX mesh file. The rules below say what it holds and how the two shipped levels differ[351].

Rule
A level is a DirectX retained-mode `.x` text file (`xof 0302txt 0064`); there is no tile layer, NPC or link data[351]
`startlevel.x` is the server's start level and `level1.x` the client's default; the two hold the same scene and differ only in line ends and trailing spaces[351]
The shipped scene is one 139-vertex mesh `obj` with nine materials and seven textures (`grass`, `way`, `stones`, `wall`, `tower`, `platform`, `clouds`)[351]
The server caches each level's text and sends it inline in id 0; the client saves it under the level name in its own folder and loads it with Direct3D RM[351]
Textures are not transferred: the BMPs a level names must already be in the client folder[351]
The client's saved copy was byte-identical to the server's file in the lab[348]

9.3.2 Level change

Changing level takes a few server steps. The steps below are in order[348].

Step	Server action
1	Finds the level in its cache or loads `levels\<name>`[348]
2	Sends 1 (name), then 0 with the text, or 0 empty if this player was already sent that level; the "already sent" list is per player, so a second login gets the full text again[348]
3	For each player already in the level, sends both sides 2 with `50 = 1` and the props not yet seen[348]
4	Sends 4 if the player is alone in the level[348]
5	Tells admin sessions with 59[348]
—	On leaving a level the others get `50 = 0`[348]

9.3.3 Player model and textures

The player is built from eight meshes. The rules below give the meshes and their textures[351].

Rule
The player is eight `.x` meshes (`xof 0303txt 0032`): body, head, two arms, two hands, two feet, each with normals, texture coordinates and a material[351]
`body.x` is textured with `body.bmp` (128×128, 24-bit; `body1.bmp` is an alternative) and `head.x` with `man.bmp` (64×64)[351]
Mesh and texture names travel as props 6–9, so they may point at any file present on every client; colours are set as material colours at run time[351]
Textures are plain Windows BMPs, 24-bit or 8-bit RLE, mostly 128×128 (`clouds.bmp` is 256×256)[351]

9.4 Camera

The camera follows the player at fixed offsets. Each row gives one quantity or rule with its value[352].

Quantity or rule	Value
Follow distance	220 behind the player[352]
Look height	80 above the player on Y[352]
Gap closing per tick	factor `((d − 220) / 220)² / 10` with the sign of `d − 220`, d the horizontal distance; skipped if the factor is 0 or 100 or more[352]
Y follow per tick	`(player_y + 80 − camera_y) / 10`[352]
Turn	up to ±5° per tick toward the player, more when the player is behind the camera[352]
Aim	LookAt the player, then pitch −0.15 rad about the camera's local X axis[352]
Follow toggle	Numpad 0; with follow off the camera stays where it is[352]
Facing sent	prop 4 is the angle of the camera-to-player direction, sent as degrees / 2[352]
Surface	640×456 pixels[352]

Quantity or rule	Value
Field	the Direct3D RM initial field 0.5, a square window; the exe sets no field[352]
Opening	`2 × atan(0.5)` ≈ 53.13° on either axis of that window[352]
Front plane	1 (library initial value)[352]
Back plane	disputed → App. E#500[352]
Lab	none of these figures is lab-confirmed; Wine's retained-mode stubs break camera frames[352]

9.5 Collision

Collision uses a small set of numbers. Each row gives one quantity and its value[353].

Quantity	Value
Body radius (horizontal half-extent)	5[353]
Upper probe height	+35 above the feet[353]
Upper probe lateral offsets	0, +10 and −10 on the strafe axis only[353]
Max step height	30[353]
Walkable face	normal Y > 0[353]
Feet-mesh attach	normal Y > 0.2; steeper walkable faces snap the point but skip the attach[353]
Wall already clear	new signed distance ≥ 3[353]
Wall started inside	previous signed distance < −5[353]
World clamp, x and z	±4000; a step that would leave is dropped[353]
Sliding	subtract `distance × normal` from the test point[353]

Order of tests for one attempted step:

Step	Action
1	Build the candidate position from the walk, run or jump delta (→ §9.6)[353]
2	Load every face of the level mesh, filled at level load, and transform its vertices to world space[353]
3	Run the body probes against every face; a blocked probe stops the rest[353]
4	Reject the step if a probe is blocked, otherwise keep the (possibly slid) position[353]
5	Drop the step if x or z leaves ±4000[353]

The rules below give where the collision probes lie and how each set is used[353].

Rule
Ground probes: four points in the feet plane at ±5 on the facing axis and ±5 on the strafe axis, a plus shape, not a filled disc[353]
Upper probes: Y + 35 at lateral 0, +10, −10; they use only the wall and ceiling path[353]
A face is used only if the probe's (x, z) lies inside its horizontal projection, by a winding test that tries both edge orders[353]
Walkable face, ground probes only: signed distance > 30 lands on the face; < −30 fails (step too high); otherwise the probe slides onto the plane, with the mesh attach when normal Y > 0.2[353]
Wall or ceiling (normal Y ≤ 0): the face is dropped if the new distance is ≥ 3, the previous distance < −5, or the move has zero length; otherwise the new point slides onto the plane[353]
No floor friction, extra ground stickiness or ceiling thickness shows in the routine's constants[353]
A disc of radius 5 is an approximation of the plus-shaped probe set, not what the binary runs[353]
Collision is not lab-confirmed; Wine's face query is a stub[353]

9.6 Client behaviour

9.6.1 Controls

Graal3D is played on the number pad. Each row gives a key and its action[350].

Key	Action
Numpad 8 / 2	forward / back, relative to the camera's view direction[350]

Key	Action
Numpad 4 / 6	strafe left / right[350]
Numpad 5	jump[350]
Numpad 0	camera follow on or off[350]
Enter	send the chat line[350]
Ctrl+Enter	fullscreen on or off[350]
F1 / F2	Hardware options / Player look[350]
Esc	quit[350]
—	movement keys are polled only while the chat line lacks focus and the window is active[350]

9.6.2 Walking, running and jumping

The rules below give walking speed, running and the jump[350].

Rule
Walking speed is 8 units per 50 ms tick (160 per second)[350]
After 25 ticks of walking without stopping the speed becomes ×1.5 (12 per tick); stopping resets the count[350]
Holding only "back" walks backwards along the facing at key-down, not following the camera[350]
While walking, y gets `abs(sin(3.14 × 2 × (anim − 1) / 40)) × 3` added (the program uses 3.14, not π)[350]
The jump climbs by a fixed table of 10, 10, 9, 8, 6, 3, 1 units per tick (47 in 7 ticks), walks the table back down with negative speeds, then falls at −10 per tick until it lands[350]
Releasing the key early starts the fall at once from the top of the table, so a tap gives a short hop; a new jump needs the key released and pressed again[350]

9.6.3 Animation counter

Prop 5 drives the walk and jump poses. The rules below say how it counts[350].

Rule
Prop 5 is a tick counter for the walk and jump poses, not a pose id[350]
Starting to walk sets it to 0 or 20 at random, so players do not step in time; a jump sets it to 40[350]
Where it advances each tick was not traced; that standing still freezes it is an inference[350]

9.6.4 Chat

Chat in Graal3D is a log, not bubbles. The rules below give how lines are added and drawn[350].

Rule
There are no chat bubbles; a chat line (prop 13 or a UDP chat datagram) is added to a chat log as `nickname: text`[350]
The log is drawn at the bottom left of the view, x = 5, newest line at y ≈ 451, older lines above, in white 14 px "Dialog"[350]
Each line stays 10 seconds (200 ticks)[350]
A line over 200 characters is cut and ends in `...`[350]
The "Player look" window edits props 6–11 and 17 and saves them in `graal3dconfig.txt`[350]

9.6.5 Lab coverage

Not every part of Graal3D was confirmed in the lab. The facts below say what was and what was not[349].

Fact
Lab-confirmed on 2026-10-30: the accept and reject flow, the login order, the full props exchange, level name, text and leader, prop 50, the UDP relay rule, the guild-tag echo, prop 14 not moving the player, saved props, load-only logins, the disconnect messages, the admin login replies and `accounts.txt`[349]
Not confirmed: UDP between two real clients, admin commands 51–54, 59, 60 and 62, and the two kick buttons[349]
Ids 4 and 6 set client values that nothing reads[347]

10 Torque-era 3D (3.x to 5.12)

From 2004 to 2007 the Graal client was built on GarageGames' Torque Game Engine (TGE). Torque gave the console, the script VM, the GUI, the resource manager, the renderer and, for 3D servers, the UDP netcode; Graal's 2D game, GS1/GS2 and the Graal TCP protocol stayed native code on top. The February 2000 DirectX Graal3D is unrelated (→ §9).

> Status: draft.

10.0 Reading this Part

The Torque years are best understood as three different programs that share the name "Graal" and some of the same code. Before treating any table below as describing "the 3.x–5.x client", fix which of the three a build is.[354]

First, the monolith. The 3.x clients (3.04 Linux, 3.151 Linux, 3.153 Mac) are Torque applications. TGE boots them, compiled TorqueScript draws every menu, and Torque's canvas holds the screen. Graal's 2D game is one native GUI control, `GuiGraalCtrl`, inside that canvas. The engine reports TGE version 1100 and still carries the stock game classes it does not use.[354,355] 3.04 has none of Graal's net-layer changes. 3.153 already has them, which makes 3.153, not 3.04, the first build whose UDP wire a compatible 3D server has to speak.[356]

Second, the split programs. From 4.04r2 to 5.12 on Windows, a Delphi launcher drives `GraalEngine.dll`, a C++ 2D engine that keeps Torque's GUI classes but draws only through callbacks into its host. Torque proper survives only in the optional `Graal3DEngine.dll`, which takes over drawing when the player joins a 3D server.[354,357] Inside this era the 3D DLL changes character. 4.04r2's DLL is near-stock TGE with datablocks and `Player`. By 4.20r6 that game layer is gone and every 3D thing is a Graal `GameObject`. The break falls between the February and July 2006 tokens.[358,356] So "the 4.x 3D client" is two incompatible object models, and the protocol number (19, 21, 24) is the reliable way to tell them apart.[359]

Third, after Torque. From 5.24r3, and in every 6.x build, there is no Torque engine. What survives is the GS2 GUI vocabulary: Torque's control class names, plus its field names up to 5.31r4, re-implemented by Graal's own `quattro_lib`.[360,360] The 6.x reads gathered in §10.1.6 and §10.1.9 are here to document that absence. The 6.x engine itself is Part I's subject (→ §1.8.7).

What is measured, and what is not. The UDP handshake is the strongest material in this Part. 5.12 was driven live against a lab server through challenge, connect, accept and its first data packets, and the minimal accept it takes is known.[359] Everything after the accept rests on static reads, and no Graal client has been seen drawing a 3D world: under Wine, 5.12's launcher faults when it switches to 3D.[358] No 3D content shipped with any client, because missions, terrain and shapes always came from the server, and no Graal 3D server or level data is in the archives.[358] The open points that would change an implementation are listed in App. E#501–E#503: the 4.04r2 class CRC, and whether 4.04r2 loads `.mis` and `.dso` files.

How the later sections are scoped. §10.7 to §10.12 record exact-build observations on Linux 3.04 and 4.20r6: script reading, packaged streams, a seven-operation executor subset and native-call boundaries. Each is deliberately narrow. A finding there holds for that input and that path, and it is not a general statement about TorqueScript execution in Graal.[361]

10.1 Engine lineage

10.1.1 Builds

The Torque era spans several builds and platforms. Each row gives one build with its date, its shape, how much of Torque it carries, its 3D world, its game string and its menu scripts[356].

Build	Date	Shape	Torque part	3D world	Game string / protocol	Menu scripts
1998 to 2.31	—	Delphi	none	none	n/a	n/a[356]
3.04 Linux	Jan 2004	one C++ binary (SDL, OpenAL)	full TGE, version number 1100	TGE classes, no 3D scripts shipped	`Torque Game Engine Demo` / 9	TorqueScript `.dso`[362]
3.153 Mac	Feb 2005	one C++ binary (Carbon, PPC)	full TGE with Graal's net-layer changes	stock TGE world and `main3D.cs` client scripts	`Graal3D` / 19	TorqueScript `.dso`[362]
4.04r2 Win	Mar 2006	Delphi launcher, `GraalEngine.dll` (2D) and `Graal3DEngine.dll` (3D plug-in)	2D engine on Torque base classes; the 3D DLL near-stock TGE	stock TGE with datablocks	`Graal3D` / 19	GS2 and native[362]
4.20r6 Mac	Sep 2006	one C++ binary (PPC)	Torque core and netcode, Graal game layer	`GameObject` model, no datablocks	`Graal3D` / 21	GS2 and native[356]
4.20r8 Win	Jan 2007	2D engine on Torque base classes	—	packet-logger upload has no `Graal3DEngine.dll` (later file-version 4.2.0.8 export: → §10.1.5)	unmeasured	GS2 and native[356,357]
5.12 Win	Oct 2007	Delphi launcher and one engine DLL, still named `Graal3DEngine.dll`	Torque core and netcode; the game layer is Graal's own	`GameObject` and `GameShape*`, no datablocks, no `Player`/`ShapeBase`	`Graal3D` / 24	GS2 and native[362]
5.24r3 and later	2010+	engine linked into `Graal.exe`	no Torque 3D (what Windows 6.00 still names → §10.1.8)	none	n/a	GS2[358]
6.x	2010–2014	native C++ per platform	no Torque 3D; GS2 keeps Torque control names (→ §10.1.8)	none	n/a	GS2[362,40]

10.1.2 Engine version and changes

The facts below give the Torque version each build reports and what Graal changed or added to the engine[358].

Fact
3.04 reports version 1100 (formatted "1.100"), uses TGE's default game string, the home folder `~/.garagegames/` (3.151 changed it to `~/.graal`) and TGE source paths, so 3.x is built on a TGE 1.1–1.2 era code base[358]
3.153 is the same engine for PowerPC and Carbon, plus the community resources `AdvancedCamera`, `fxFoliageReplicator`, `twSurfaceReference` and `volumeLight`, a 3D Studio loader and a Quake 3 `.md3` loader[358]
The 4.04r2 `Graal3DEngine.dll` is near-stock TGE (`Player`, `AIPlayer`, vehicles, projectiles, `InteriorInstance`, `TSStatic`, the `fx*` replicators, every matching datablock) and imports 623 symbols from `GraalEngine.dll`[358]
The 4.04r2 DLL matches open TGE 1.x in its out-of-band packets, packet header, Huffman table, rate limits, DTS/DIF/TER versions and class list, and lacks TGE 1.4's lighting, so it is of the TGE 1.2/1.3 line[328]
5.12 keeps Torque's core (console, sim objects, `NetConnection`/`GameConnection`, ghosting, `SceneObject`, terrain, interiors, DTS, sky, sun, water, the editors) but drops the TGE game layer: no datablocks, no `GameBase`/`ShapeBase`/`Player`, vehicles, projectiles, items, `ScriptObject` or audio emitters[358]
4.20r6 already has the 5.12 model, so the TGE game layer was dropped between 4.04r2 (Feb 2006 token) and 4.20r6 (Jul 2006 token)[356]
3.04 has no Graal changes in Torque's net layer and none of the Graal net classes[356]
Graal's net-layer changes (class table in the accept, 12-bit class ids, 14-bit length prefixes, BigData and terrain events) are already in 3.153[356]

The Graal changes a compatible peer adds to open TGE[328]:

Change
a class table in the connect accept, with 12-bit class ids in the server's numbering (→ §10.3.3)[328]
a 14-bit length prefix on every event, ghost update and datablock[328]
a class id on every ghost update and a raw index width[328]
12-bit datablock ids (TGE uses 10)[328]
a server time and client-reported state in the game packets (→ §10.3.5)[328]
mission acks without a sequence number (4.04r2)[328]
the BigData and terrain events (→ §10.5.3)[328]
GS2 in place of TorqueScript, from 4.x on[328]

10.1.3 Rendering

The facts below give the display devices every build has and how each platform draws[358].

Fact
Every build has Torque's display devices: OpenGL, a Direct3D wrapper (`opengl2d3d.dll` / `glu2d3d.dll`) and Voodoo2 (full screen only); the minimum mode is 640×480[358]
3.x uses SDL and OpenGL on Linux, AGL and DrawSprocket on Mac; 4.x and 5.12 on Windows draw 2D with DirectDraw/Direct3D 7 in the Delphi launcher and host the Torque canvas for 3D (`GuiGraal3DCtrl`)[358]
In 4.04r2 the loaded plug-in registers its own handler for every drawing callback of the 2D engine, so in 3D mode the Torque DLL owns the window and the 2D game draws through Torque's canvas[34]
Scene lighting is Torque's, cached per mission and run as the last mission-load phase[358]
Under Wine with software OpenGL the 5.12 Torque side starts on switching to 3D, but the launcher faults re-initialising DirectDraw and stops driving the engine's frame export; the 3D view was not seen[358]

10.1.4 Era A: what 3.04 contains

3.04 Linux compiles stock TGE beside a Graal game. The stock game classes are still in the binary; Graal's 2D game is the console class `GuiGraalCtrl`, not a replacement of `Player`.[355]

Fact
`GuiGraalCtrl` registers 31 methods, in order `moveUp`, `moveLeft`, `moveDown`, `moveRight`, `doA`, `doS`, `doD`, `doQ`, `doP`, `doM`, `doMicro`, `setChat`, `setPassWord`, `openServerList`, `selectServer`, `selectServerCategory`, `startOfflineMode`, `connectToSelectedServer`, `serverDirectConnect`, `privateMessageSend`, `playerListOpenPlayer`, `playerListProfile`, `updateLoadLevelFields`, `loadLevel`, `updatePlayerProfile`, `updatePlayerMuted`, and the four volume and options setters[355]

Fact
Those methods have an empty usage string. Counts include the command name and, for a method, the object. `moveUp` is 3–3 (one script argument). `openServerList` is 2–2 (none). `loadLevel` is 5–5 (three). `updatePlayerProfile` is 12–12 (ten). A maximum of 0 means no upper limit[355]
`setNickName`, `setAccountName`, `setLanguage` and `retrieveAccountFields` are not on this table. Nickname, account and language are `$pref::Graal::` variables[355]
Graal's C++ draw types are `TDrawableObject`, `TLevelObject`, `TGaniObject`, `TServerPlayer`, `TDrawableSurface` and `TSurfaceOwner`, plus `TGraalConnection`. None of them is a Torque console class, and `TGraalConnection` is not a child of `NetConnection`[355]
`ShapeBase` still has 79 decoded methods here, `fxSunLight` 51 and `GameConnection` 30. The classes 5.12 drops are the TGE game layer (`Player`, vehicles, items, projectiles), not the console or the editors[355]
The console surface is 857 functions and methods, almost all of them stock TGE[355]

The game components

Linux 3.04 keeps stock Torque (`GameConnection`, `NetConnection`, and the `sim/netConnection.cc` / `game/gameConnection.cc` sources) beside Graal's own 2D game. `TGraalConnection` is a separate C++ type with no base class. The binary has no `Graal3D` string and no `BigData` event.[363]

Component	What the 3.04 units do
Session (`adventure.cpp`, `GuiGraalCtrl`, levels, the local player)	Wakes once, opens the three license files, loads the HUD sheets, and logs a local level entry only for seven file magics[364]
Drawing	The `TDrawableObject` family, `pics1.png` and tiledefs, gani text, polygon show-images[365]
NPCs and script	`playerenters` and the other trigger names, bow baddies, weapon rectangles, `shapetiletypes`[366]
Graal TCP	Login id 5 then language id 37, the server list into `AdventureGui`, the player list and profiles[363]
Sound	fmod at mix rate 44100, wav samples, mid modules, and the mp3-class streams[367]

10.1.5 Era B: the Windows 4.2.0.8 split

Two exports named as the 4.04r2 and 4.20r8 executables are one packer stub, byte for byte, with no Graal or Torque class. The file-version 4.2.0.8 archive is the client that can be read, and it includes `Graal3DEngine.dll`, which the packet-logger upload did not.[35,368,357]

A later original-carrier 4.04r2 recovery supplies a separate shape-based 3D component that retains `ShapeBase`; it is distinct from the later 4.2.0.8 `GameObject` image below. Its repaired import directory cannot independently verify the earlier named-import count. Recovery availability and load-only limits: → §1.8.5.[35]

Fact
The launcher registers 94 `graal_set…` callbacks and loads `GraalEngine.dll`. The 3D DLL adds 19 names the launcher does not register, including `CreateGraal3DGameObject`, `GetGraal3DTerrainHeight` and `SendGraal3DScriptEvent`[357]
`GraalEngine.dll` keeps Torque's `GuiCanvas` and `GuiControl` and has no `SimObject`, `ActionMap` or `.dso`. The game is `TAdventure`, `TShowImg` and `TGraalAni`. Scripts are `TGraalVar` and `TScriptExecutionSpace`[357]
The engine builds a `TGraalVar` for `StartScript_GraalGui`, `StartScript_MoveMap2D`, `StartScript_MoveMap3D` and the other start scripts, and it special-cases the control names `GuiGraalCtrl` and `GuiGraal3DCtrl`[357]
`Graal3DEngine.dll` is the `GameObject` world: `GameConnection`, `TerrainBlock`, interiors, two login phases (`onGraal3DLoginStep`), and `TerrainChunkEvent`. `ShapeBase` and Torque's `Player` are absent[357]

How the three programs run

The unpacked 4.2.0.8 launcher, `GraalEngine.dll` and `Graal3DEngine.dll` are three programs. The launcher resolves 94 `graal_set` slots and calls `renderGraalCanvas` from the main form's frame; `renderGraal` is resolved and never called. A 3D start loads the 3D DLL, which keeps drawing, tiles, text, panels and the clipboard, while the launcher takes back Flash and the external windows.[369,369]

`TAdventure::timer` runs from the canvas frame and pumps the client, nearby-player expiry and keys. `TGraalVar` stores a float, a string, or a link, and `TScriptExecutionSpace` refuses a paused owner. `loadGraalGui` builds the `StartScript_` objects in a fixed order. `GuiGraalCtrl` and `GuiGraal3DCtrl` are not constructed by the script builder. The DLL has no `SimObject` console and no `.dso`.[370,370,370]

GameObject is a SceneObject with script properties for shape, motion, mounting and interiors. onGraal3DLoginStep is raised for ghost objects (step 2), then mission lighting (step 3), and again when GUIContainer3D is installed (step 6). A terrain chunk is a compressed square; the byte layout is not in the decompile. Interior::read rejects a non-zero first value. ShapeBase and Torque's Player are absent.[371,371,371,371]

A loose 4.20 engine core, recovered from an archive labelled 4.20r8 and kept apart from the 4.2.0.8 export, has two render entry points that each check their target first: the adventure entry returns without drawing when there is no adventure, the canvas entry returns without drawing when there is no canvas and otherwise asks for one frame, and setting the drawing callback only stores it. These rules do not show what paint draws or when start-up is ready to draw.[372]

10.1.6 Era B: Windows 5.0.0.7 and the later v5 exports

File version 5.0.0.7 is a Delphi launcher, GraalEngine.dll, and Graal3DEngine.dll. The launcher resolves 95 graal_set* names and loads the engine before it registers them. A zero return from graal_engineInitialize shows The execution file is corrupted.[373,373]

Against the 4.2.0.8 launcher's 94 names, this launcher adds the colour-matrix pair, fullscreen get and set, graal_setShowGraalWindow, screen capture, Flash clipping and forced texture transparency. It no longer resolves the external history, log and PM setters, stretched graphics, the screenshot setter, Flash panel size, or graal_setLogGraalMessage.[373]

The engine defines 118 setters. The 23 the launcher does not resolve are the 3D bridge (connect, GameObject, terrain height, script events, window lock) and three external-window setters neither host calls. SimObject and .dso are absent from GraalEngine.dll.[374,374]

TAdventure is the session. A server name whose second character is a space loses its first two characters, and a name that starts with 3 asks the launcher through graal_setStartGraal3D and then connects the 3D side at listed port + 1. Any other server puts GUIContainer on GuiCanvas. The timer calls handleClient, which drops a dead link and raises onPlayerLogout.[374]

A script object is a TGraalVar with a TScriptExecutionSpace. runScript walks that space's actions on a machine borrowed from the universe. The factory refuses to build GuiGraalCtrl or GuiGraal3DCtrl as an ordinary object.[374,374]

In the 3D DLL, GameObject's property table is parented on SceneObject. Two mission steps download ghosts and then light the mission, and the following step sets the canvas to GUIContainer. A terrain chunk load reads a compressed file under levels3d/heights for that server. The connect check reads 4 bytes and accepts only 24; any other value returns CHR_PROTOCOL. An interior file's first 4 bytes must be 0, and an interior resource's first 4 bytes must be 44.[375,375,375,375,375]

The rest of that v5 export

One static read of a v5 export (nothing run). File-version strings below are the version resources in that export[376]:

Fact
Graal.exe 5.0.0.7 is a Delphi launcher. It loads GRAALENGINE.DLL and GRAAL3DENGINE.DLL, resolves graal_engineInitialize and 95 graal_set* names, and shows a warning that the execution file is corrupted if initialisation fails[376]
GraalEngine.dll defines TAdventure, TGraalVar, TShowImg, TGraalAni, GuiGraalCtrl and Torque GUI classes (GuiCanvas, GuiControl). It defines 118 graal_set* setters. SimObject, GameConnection, ActionMap and .dso are absent[377]
The launcher resolves the Flash, external-window, login and graal_setStartGraal3D setters. The 3D DLL calls the OpenGL and GameObject setters, then graal_engineInitialize. Three external-window setters are defined and named by neither host[377]
Graal3DEngine.dll in this export is the GameObject era: SimObject, GameConnection, ActionMap, TerrainBlock, terrain chunks, GameShapeDTS / DIF / FT. ShapeBase, PlayerData and a datablock download phase are absent. Interior read strings are present[378]
Mission logs are a ghost-object download (step 2), mission lighting (step 3), lighting done (step 4), the initial control object (step 5), then step 6 attaches GUIContainer[378]
A check writes CHR_PROTOCOL unless the tested value is greater than 23 and less than 25[378]
The packed 5.12 exe's version resource is 5.1.2.0 and the decompile is a stub. That zip has no Graal3DEngine.dll[379]
The packed exes named 5.2.4.3 and 5.24r4 have version resources 5.243 and 5.244. Both decompiles are stubs[380,381]
freetype6.dll is FreeType 2.1.10.2079. The fmod.dll version resource says FMOD 3.75 and the recovered code is a packer stub[382,383]
Unpacked 5.30r3 and 5.31r4 are one exe each, file versions 5.303 (compile string Oct 18 2010) and 5.314 (compile string Jan 5 2011), source tree quattro_lib. graal_set, SimObject and GameConnection are absent. GuiGraalCtrl, GuiGraal3DCtrl and the Torque GUI field names remain[384,385]

10.1.7 Which modules each era still has

A module is present when that image registers it, recovers methods, records the C++ type, or emits a `Class::method` error. A stub is the name alone. The 4.2.0.8 and 5.0.0.7 launchers are omitted below: both listings have DirectDraw and none of these modules.[360,360]

Era	What of Torque is in the image
3.04 Linux	Console, `SimObject`, `ActionMap`, `GuiCanvas`, `GuiControl`, `GBitmap`, terrain, interiors, DTS, `GameConnection`, `ShapeBase`, `Player`, vehicles and datablocks. Graal's console class is `GuiGraalCtrl`[360]
4.2.0.8 and 5.0.0.7 engine DLL	`GuiCanvas`, `GuiControl` and `GBitmap`. No console, `SimObject` or `ActionMap`. The game types are `TAdventure`, `TGraalVar`, `TScriptExecutionSpace` and `TShowImg`[360,360]
4.2.0.8 and 5.0.0.7 3D DLL	`SimObject`, `ActionMap`, terrain, interiors, DTS, `GameConnection` and `GameObject`. No `ShapeBase`, `Player`, vehicles or datablock class. 5.0.0.7 adds the names `GameShapeDTS`, `GameShapeDIF` and `GameShapeFT`[360,360]
5.30r3 and 5.31r4	No Torque world and no `GuiCanvas`. Control names remain, and so do `horizSizing`, `vertSizing`, `minExtent`, `fontType`, `fillColor`, `makeFirstResponder` and `pushToBack`[360]
6.00r2 and 6.00r4	The same control names, without those field names. `TAdventure` does not occur[360]
Packed 5.12, 5.24, 6.037 and 6.113	Packer listings. None of the module names[360]

`GameConnection` in 3.04 uses the stock method names. `BigDataStartEvent` and `TerrainChunkEvent` on the 4.2.0.8 and 5.0.0.7 3D DLLs are the Graal patch. `GameObject`, `GuiGraalCtrl`, `TAdventure`, `TGraalVar`, `TScriptExecutionSpace` and `TShowImg` are Graal-only. A stock name whose body was not diffed against a TGE tree is uncompared.[360]

10.1.8 After Torque: what the 6.x images still name

This section records absence. Each 6.x image below was read to establish which Torque names survive once Graal's own engine replaced Torque. The engine itself is described in §1.8.7. The table gives the cut, and the builds follow in date order.[386]

What	Who
Torque console, `SimObject`, `GameConnection`, and UDP on the listed port plus one	v3 to v5[386]
`quattro_lib`, `TSocket`, `TServerPlayer`, OpenGL, `fmod`, `GRESCR01` and the GS2 switch	unpacked 6.037 and 6.1.1.3[386]
`GuiWindowCtrl` and `GuiControlProfile` remain as strings. `SimObject` and `GameConnection` do not	both unpacked images[386]

Windows 6.00r2 and 6.00r4

Unpacked Windows 6.00r2 (product version 6.002, 12 June 2011) and 6.00r4 (6.004, 26 June 2011) are Graal's `quattro_lib` program[40,41]. The drawing, GUI, script and socket split is in §1.8.7.

What remains	In these two builds
Torque 3D classes and formats (`GameConnection`, `TerrainBlock`, `Interior`, DTS, `SimObject`, `.mis`, `.ter`, `.dts`)	absent[40]
Torque GUI field names (`horizSizing`, `vertSizing`, `minExtent`, `fontType`, `fillColor`, `makeFirstResponder`, `pushToBack`)	absent; those names are a 5.24r3 finding[40,354]
`GuiControl`, `GuiControlProfile`, `GuiScrollCtrl`, `GuiWindowCtrl`, `GuiMLTextCtrl` and the other Torque control names	live GS2 controls, each with its own draw routine[40]
`GuiGraal3DCtrl`	live type in the GUI walk, same branch as `GuiGraalCtrl`[40]
`GuiTilesCtrl`	absent; tiles are the `Draw_Tiles` pass[40]
`MissionGroup`	script object looked up by name[40]
`onMouseYawPitch`, and a builtin described as changing the colour of the 3D terrain	script text; no terrain block[40]
`levels3d`, `graal3://`	a content folder, and an option-list string with no recovered call[40]

Unpacked Windows 6.037

The unpacked 6.037 listing is one GUI executable, product version `6.037` (fixed words 6.0.3.7), compiler banner 16 November 2012 18:58:51. The code section holds `quattro_lib` and `TSocket`. It is not the packed 6.037 shell.[387]

What the listing shows	Unpacked 6.037
Program	One `quattro_lib` executable. No Delphi launcher, no engine DLL, no SDL. `ExeReplacer.exe` is still launched[387]
Packer residue	Seven sections, including a blank name, two eight-letter names and `.SCY`. The export directory is still named `QPlay.exe`. The entry is the C runtime cookie setup[387]
Connection	`TSocket`, newline refused, key byte masked to seven bits, gen-3/5 seed, next field 3. No plaintext `G3D` token. The receive switch is the 92-id Linux 6.03r7 set[387]
TLS	CyaSSL is called with the SSLv3 client method. `zlibVersion` returns `1.2.3`; inflate callers pass `1.2.5`[387,387]
GS2	Cases 54, 121 and 200–242. No case 48 or 49, and nothing above 242, as in unpacked 6.00. `GRESCR01` on a `.code` file of at least eight bytes[387]
`GuiGraalCtrl`	Still the type the GUI walk drops[387]
`GuiGraal3DCtrl`	Absent. The shell looks up `GraalControl` and `GraalControl3D`[387]
`GuiTilesCtrl`	Constructed. Unpacked 6.00 does not have the name (above)[387,40]
`horizSizing` and the other Torque field names	Absent, as in unpacked 6.00[387]
Torque modules	`GameConnection`, `SimObject`, DTS, `TAdventure` and SDL are absent. `-printmap` writes Torque 2D datablock text and does not run a module[387]

Unpacked Windows 6.1.1.3

A later listing of the same 8 August 2014 executable decompiles the engine. The upload is a Ghidra C listing and an HTML listing, not a raw executable and not `Graal.exe` plus a DLL. The version resource, the timestamp and the virtual image size match the stub in §10.1.9. The header entry is the Visual C++ security-cookie initializer, and about 9,300 functions recover.[45,45]

What this listing shows	Unpacked 6.1.1.3
Version	String `6.113`, fixed words 6.1.1.3, Eurocenter Games, copyright 2014. Compiler strings `Aug 8 2014` and `17:16:33`. Image timestamp 8 August 2014, 13:19:46 UTC. Linker major 9, minor 0[45]
Shape	One 32-bit GUI image, seven sections. `.text` is marked initialized data. `.SCY` is marked code. Virtual image 6,504,448 bytes[45]
Engine	`quattro_lib` / `Quattro Play`, `TSocket`, `TServerPlayer`, `GuiGraalCtrl`, OpenGL, FMOD, `GRESCR01`. Original file name `Graal.exe`[45]
Packer left behind	Export directory still named `QPlay.exe` (in `fbndyrix`). Import directory in `.SCY`. Blank section still holds `Themida`, `WinLicense` and Oreans driver names[45]

Against unpacked 6.00 (above), this image still has no `SimObject`, `GameConnection`, `TerrainBlock`, `MissionGroup`, `TAdventure`, `graal_set` or `horizSizing`. The GUI walk compares `GuiGraalCtrl`. `GuiGraal3DCtrl` is not a string here; the shell looks up `GraalControl3D` beside `GUIContainer`. `GuiTilesCtrl` is registered beside `GuiControl`, which the 6.00 notes recorded as absent. A file reader matches five `new t2d…(` constructors, and `-printmap` takes a datablock file and a level file. That is text, not a Torque console. The hold on writing the module stands.[45]

10.1.9 Packer stubs and version resources

Several uploads named after real clients are packer shells, not the client. They are listed here so that nobody reads them as engine evidence. Their version resources are still useful for identification.[388]

The packed listings

The two v4 exe listings are one file. The program is the Thinstall outer loader: `TS_LOADER` or the module path, `LoaderFlags` as the table offset, magic `0x82D0B188`, a stored block or the range coder, then relocations, imports and a jump to the unpacked entry. The import directory adds one import from each DLL of the wrapped program (registry, shell, save dialog, image list, OLE, joystick, printer). Winsock, DirectDraw, OpenGL and `GraalEngine.dll` are not in that listing.[389,389,389]

Packed 5.12 keeps Delphi form resources (`TGRAALWINDOW`, `TPLAYERLIST`, and the options, profile, URL, controls and short-message forms) plus `DVCLAL`. Its recovered code does not call `LoadLibraryA`. The packed 5.2.4.3 and 5.24r4 exes bind only `LoadLibraryA` and `VirtualAlloc` and name no form.[390,390]

`fmod.dll` in the 5.0.0.7 archive is the same UPX listing as the 4.2.0.8 copy. The export names are FMOD's sample, stream, 3D, DSP and music API, including net streams, and the recovered body is the decompressor.[390]

The packed 6.037 and 6.113 exes are distinct. Each export directory is named `QPlay.exe` and lists a CyaSSL client, zlib, and a hash and cipher set; the 6.113 names add TLS 1.0 through 1.2, DTLS and AES-CBC. The bytes at those exports are not that library, and neither listing names a Torque class.[391,391,391]

Version resources

Packer-stub uploads from the v4, v5 and v6 decompile sets still carry a version resource. The recovered code is not the client.[388]

The 4.0.4.2 exe stub and the 5.1.2.0 stub share one string table: four-part file version, a one-character product version, description Graal Online client, internal name Graal client, company Linux Cyberjoueur, no year. From 5.243 on, the file and product strings are the collapsed form (5.243, 5.244, 6.037, 6.113) and the fixed words stay four-part (5.2.4.3, 5.2.4.4, 6.0.3.7, 6.1.1.3). Those later Graal stubs name Linux Cyberjoueurs or Eurocenter Games and a copyright year (2010, 2012, 2014).[388]

Upload	Code	File version string	Fixed words	Product version string	Original file	Company and year
Graal 4.04r2.exe and Graal v4.20r8[Windows].exe (one decompile)	stub	4.0.4.2	4.0.4.2	4	Graal.exe	Linux Cyberjoueur, no year[388]
Graal 5.12.exe	stub	5.1.2.0	5.1.2.0	5	Graal.exe	Linux Cyberjoueur, no year[388]
Graal 5.2.4.3.exe	stub	5.243	5.2.4.3	5.243	Graal.exe	Linux Cyberjoueurs, 2010[388]
Graal v5.24r4.exe	stub	5.244	5.2.4.4	5.244	Graal.exe	Linux Cyberjoueurs, 2010[388]
fmod.dll in the 4.2.0.8 and 5.0.0.7 archives (one decompile)	stub	3.75	3.7.5.0	3.75	fmod.dll	Firelight Technologies, 1994-2005[388]
Graal 6.0.3.7.exe	stub	6.037	6.0.3.7	6.037	Graal.exe	Linux Cyberjoueurs, 2012[388]
Graal 6.1.1.3.exe	stub	6.113	6.1.1.3	6.113	Graal.exe	Eurocenter Games, 2014[388]

Packed Windows 6.1.1.3

The 2026-11-07 gap archive did not contain an unpacked 6.113 image. The Windows file is still one GUI executable whose export directory is named `QPlay.exe` and whose only import is `SetCurrentDirectoryA`, which the stub does not call.[44,44]

What the listing shows	6.1.1.3
Version	String `6.113`, fixed words 6.1.1.3, Eurocenter Games, copyright 2014, image timestamp 8 August 2014. No version token[44]
Connection, GS2 VM, property parsers, file cache, GUI classes	Absent. The recovered code is the dword walk and a breakpoint[44]
Export names	CyaSSL through TLS 1.2, DTLS, AES-CBC, HMAC, SHA-256, and zlib. The bytes at those exports are not the library[44]
Login, framing, packet ids, property streams, `GRESCR01`	Not in this image. Neighbour notes disagree on the key byte, the 6.03 token, ids 100 and 108, and the oldest level magics[48]
GS2	No opcode switch. Unpacked 6.00 already has cases 54 and 121 and folds 234–242, and does not have 48 or 49. This file cannot extend that[335]
Torque names	Console, `SimObject`, `GameConnection`, `ShapeBase`, `Player`, `graal_set`, `horizSizing`, `GuiTilesCtrl` and `TAdventure` are already gone in unpacked 6.00. `GuiGraalCtrl` and `GuiGraal3DCtrl` are that build's playfield. This listing has none of those names. Issho is v6; v7 Unity is not a profile this file supports[44]

The 6.1.1.3 decompile is 106 functions in a 227,199-byte listing. Three are the entry, which clears a breakpoint byte, mixes four kilobytes and then breakpoints. The other 103 are export names for a `CyaInt` TLS client, AES-CBC, HMAC, SHA-256 and RSA; those bodies do not implement the named calls, several names share one body, and the zlib names are not functions in the listing. The file has no GS2 switch, GUI class or socket import.[392]

10.2 Join sequence

Joining a 3D server runs the same routine in several builds. The steps below are in order[7].

Step	What happens
1	The server list marks a 3D server with the name prefix 3 (the "Graal 3D" tab)[7]
2	Choosing it runs one routine in 3.153, 4.04r2 and 5.12: strip the two-character prefix, remember UDP port = listed port + 1, set the 3D-mode flag and start the Torque side[359]
3a	3.153 creates a `SimGroup` named `MissionGroup` and runs `main3D.cs`, which creates `GameConnection(ServerConnection)` and calls `connect()` with the stored address[359]
3b	4.04r2 and 5.12 first ask the launcher (a non-zero answer cancels), then have it connect to host, port + 1, with the UDP option; 4.04r2 loads `Graal3DEngine.dll` on demand at this point[359]
4	In both 2D and 3D the client opens the normal Graal TCP connection to the listed port and logs in as usual (→ §2.3)[359]
5	The Torque handshake runs on UDP (→ §10.3), then the mission download (→ §10.3.6)[359]

The rules below give how a 3D server is split across two ports and how the client picks its own UDP port[359].

Rule
A 3D server is two servers on one host: a Graal gserver on TCP *P* and a Torque server on UDP *P* + 1[359]
The client's own UDP port is random unless the fixed-port option is on, then `fixedudpport` (default 14899)[359]
28000 is only the Torque address parser's default for an address without a port[356]
Seen live: the 5.12 client sent the Torque challenge to UDP *P* + 1 and opened TCP *P* in the same millisecond; the TCP side carried the ordinary 5.x login frame with the built-in guest account, then ids 37 and 23[359]
The 6.113 full capture does not show that pair. TCP 14900 and TCP 14882 carry the Graal hello, and neither remote has a UDP datagram. No UDP port in the file is a TCP port plus one[50]
The 3D connection lives only as long as the TCP one: when the lab TCP listener closed, the client reported "You have been disconnected from the game server." and sent a Torque disconnect[359]
In 4.04r2 a Torque time-out ("The 3D server connection has timed out") also drops the TCP session[328]
4.04r2 gives the 3D side the account name, nickname and password through the engine; no session key or cookie is passed[328]
If the 4.04r2 plug-in is missing, the launcher has the text "You need to download Graal3D at … for playing this server!"; when it shows it was not traced[328]

Rule
The 4.04r2 launcher still registers `graal3://` against `Graal3.exe`, the 3.x name[34]
Each outcome fires a GS2 event: `onConnectionAccepted`, `onConnectRequestTimedOut`, `onConnectRequestRejected`, `onConnectionDropped`, `onConnectionError`[34]

TCP traffic in 3D mode, 4.04r2[328]:

Traffic	In 3D mode
S→C 0 LEVELBOARD, 14 PLAYERWARP, 28 FLAGSET, 31 FLAGDEL, 49 PLAYERWARP2, 101 BOARDPACKET	ignored; level, warps and flags come from the 3D server[328]
level requests at start-up, getweapon, client-variable and flag sets, server-side triggers	rerouted to the 3D server (→ §10.5.3)[328]
chat, PMs, the player list, file transfer, RC	not touched by the switch, so presumably still TCP[328]
missing 3D resources	one observed resource-miss branch normalizes the name, extracts its extension and requests a download only when it differs from `.dso`, `.cs` and `.ter`; this does not establish absence of local readers (→ §2.7)[393]

10.3 Handshake by build

10.3.1 Info packets

The first bit of a datagram picks the kind: 1 is a data packet of an established connection, 0 an info packet whose first byte is the type; all integers are little-endian[359].

Type	Name	Direction	Body
26 (0x1A)	ConnectChallengeRequest	C→S	U32 connect sequence[359]
28 (0x1C)	ConnectChallengeReject	S→C	not handled by the Graal clients[359]
30 (0x1E)	ConnectChallengeResponse	S→C	U32 sequence, 4 × U32 address digest[359]
32 (0x20)	ConnectRequest	C→S	U32 sequence, 4 × U32 digest, string `GameConnection`, then the request fields (→ §10.3.2)[359]
34 (0x22)	ConnectReject	S→C	U32 sequence, string reason[359]
36 (0x24)	ConnectAccept	S→C	U32 sequence, the class table (from 3.153), U32 protocol (→ §10.3.3)[359]
38 (0x26)	Disconnect	both	U32 sequence, string reason[359]

Torque info packets come before the connection. The rules below say how they are routed and how strings in them are encoded[359].

Rule
Types below 23 go to the master-server and LAN query handler[359]
A string is Torque `writeString`: one flag bit (1 = Huffman with Torque's fixed tree, 0 = raw), a U8 length, then the codes or bytes; fields after the first string are not byte-aligned[359]
The address digest is TGE's MD5 over the client address, the sequence and a random server key[328]
There is no encryption beyond Torque's framing; the join password travels in clear[359]

10.3.2 Connect request by build

The request writes the net class group (U32, 0 = game) and the class CRC (U32), then the `GameConnection` fields[359].

Field	3.04	3.153	4.04r2	4.20r6	5.12
game string	`Torque Game Engine Demo`	`Graal3D`	`Graal3D`	`Graal3D`	`Graal3D`[356]

Field	3.04	3.153	4.04r2	4.20r6	5.12
current = minimum protocol	9	19	19	21	24[356]
class CRC	stock table, value unmeasured	unmeasured	disputed → App. E#501	unmeasured	0xFFFFFFFF (static table, never written)[356]
join password	the server-password pref	set by the engine, not traced	the account password, plain text	not traced	the account password, plain text[359]
argc (at most 16), then strings	script args	engine-set, not traced	2: account, nickname	not traced	2: account, nickname[359]
server side of the request	stock script `onConnectRequest`	stock script `onConnectRequest`	native	native	native[356]

The connect request differs by build. The rules below record a live 5.12 request decoded in full and how a 5.12 server rejects a wrong game string[359].

Rule
The live 5.12 request was 71 bytes and decoded completely: group 0, CRC `0xFFFFFFFF`, `Graal3D`, 24/24, the built-in guest password as join password, argc 2 (`guest`, empty nickname), no bits left over[359]
A 5.12 server rejects a game string other than `Graal3D` with `CHR_GAME` ("Only official Graal clients are allowed to connect."), current < 24 with `CHR_PROTOCOL_LESS`, minimum > 24 with `CHR_PROTOCOL_GREATER`, and argc > 16 with `CR_INVALID_ARGS`[359]
4.04r2 knows the same reasons plus `CHR_PROTOCOL` and `CHR_INVALID`; it rejects < 19 and a minimum ≥ 20[34]
Stock Torque rejects a CRC mismatch with `CHR_INVALID`; in 5.12 the class table in the accept takes the CRC's place[359]

10.3.3 Connect accept and class table

The connect accept carries a class table. Each row gives one field of a table entry, its width and its meaning[359].

Field	Width	Meaning
flag	1 bit	1 = another entry follows, 0 = end of table[359]
class type	12 bits	0 object, 1 datablock, 2 event (3.153, 4.04r2); 0 object, 1 event (4.20r6, 5.12; the event meaning is inferred)[359]
server class id	12 bits	0 to 255[359]
class name	string	Huffman string, such as `NetStringEvent`[359]
protocol	U32	after the end flag; must equal the build's protocol[359]

The rules below give how the client matches those entries to its own classes[359].

Rule
For each entry the client finds its own class of that name, type and group and records server id ↔ local id in two maps of 256 slots per group and type; class ids on the wire are then the server's[359]
A name the client does not know maps to "unknown" and its data is skipped[328]
Until a table is read the map is the identity (3.153)[356]
An accept with an empty table (one 0 bit) and protocol 24 is enough for 5.12 to reach "Connection established"[359]
A stock accept without the table bit is read one bit off, gives protocol 12 and is silently rejected by 5.12[359]
The class table is in 3.153 and 4.04r2 with three types, in 4.20r6 and 5.12 with two types; 3.04 has none[356]
Each build accepts exactly its own protocol (9, 19, 19, 21, 24); anything else gives `CHR_PROTOCOL`[359]

10.3.4 Live handshake (5.12)

A live 5.12 handshake was captured. Each row is one packet, with its time from the first and its direction[359].

Time	Direction	Packet
+0.000 s	C→S	challenge request, sequence 1024[359]
+0.000 s	S→C	challenge response with the lab server's digest[359]

Time	Direction	Packet
+3.009 s	C→S	connect request, 71 bytes (the 3 s gap was the lab injector's own wait)[359]
+3.009 s	S→C	connect accept: sequence, empty class table, protocol 24[359]
+3.043 s	C→S	first data packet, sequence 1, with the "max rate" flag and two 12-bit values (2080 and 368 in the stock field order)[359]
then	C→S	one 17-byte data packet a second with no events, and a ping after about 4.6 s with nothing acknowledged[359]
+18.02 s	C→S	disconnect, sequence 1024, empty reason[359]

The client's console logged the same steps: "Sending Connect challenge Request", "Got Connect challenge Response", "Sending Connect Request", "Connection established", and at the end "Issuing Disconnect packet."[359].

10.3.5 Data packets

The rules below give the header of a Torque data packet as stock TGE writes it[359].

Rule
Header (stock TGE): flag 1, a sequence-parity bit, 9-bit sequence, 9-bit highest ack, 2-bit type (0 data, 1 ping, 2 ack), 3-bit ack byte count, then the ack mask[359]
Then a flag → 10-bit update delay and 10-bit packet size, and a flag → two 10-bit max-rate values[328]
4.04r2 rate prefs: to server 32 (8–32), to client 32 (1–32), packet size 800 with an allowed range of 100–450; the update delay is 1024 / rate ms, so in practice 32 packets/s of 450 bytes[328]

Game-packet bodies after the header[328,356]:

Part	4.04r2	4.20r6, 5.12
S→C move ack	int32 last move ack	none[356]
S→C damage flash and white-out	flag → { flag → float7; flag → float7 }	none[328]
S→C control object	flag → flag (dirty) → u32 server time, int12 ghost index, the object's data; else 3 × F32 compression point	flag → flag → u32 server time (ms), int12 ghost index of a `GameObject`, its data; the first control object starts login step 5[356]
S→C camera	flag → int12 camera ghost, flag → camera data	the same; no flag clears the camera[356]
S→C field of view	flag → int8	flag → int8[356]
C→S first flag	first person	set when a connection float is 0; meaning not traced[356]
C→S position	3 × F32 position of the control object, flag → int12 its ghost index	the same, or the ghost of the object it is mounted on[356]
C→S state	flag → the control object's data; else int32 first move index, int5 move count (up to 30), the moves	flag → the control object's data; no move branch[356]
C→S field of view	flag → int8	flag → int8, clamped to 1–179[356]
Both, then	events, then ghosts	events, then ghosts[356]

Graal adds to the stock data packet. The rules below give those additions and what 5.12 reports in them[328].

Rule
The u32 server time is a Graal addition; the client keeps it as its synchronised clock[328]
In 5.12 the client always reports its own position and never sends a Torque `Move`; position is client-reported as in 2D Graal[356]
Unguaranteed events: flag 1, int12 class id, sized payload, …, flag 0; guaranteed-ordered events add a flag (next in sequence) else int7 sequence[328]
Ghosts: flag, int4 index width (sent as is; TGE sends width − 3), then per ghost flag 1, index, flag (delete), and if not deleted int12 class id on every update and the sized data; flag 0 ends[328]
New ghosts go into the client's `MissionGroup`; in 4.04r2 they wait while TCP file downloads are running[328]
In 5.12 the class id reader takes 12 bits and applies the server-to-local map; events and ghosts each have their own length wrappers[356]

10.3.6 Mission download

Mission download runs in phases, and 5.12 drops one. Each row gives a group of builds and its phases[359].

Build	Phases
3.153, 4.04r2 (stock TGE)	1 "Download Datablocks & Targets", 2 "Download Ghost Objects", 3 "Mission Lighting"[359]
5.12	1 "Download Ghost Objects", 2 "Mission Lighting"[359]

The rules below give how the server drives the phases and how the client answers[359].

Rule
The server drives the phases with `MissionStartPhaseN` remote commands and the client answers `MissionStartPhaseNAck`[359]
In 3.153 these are script functions; phase 1 acks with the sequence number, and phase 3 lights the scene and acks when lighting is done[359]
In 4.04r2 they are native; the acks carry only the command tag, no sequence number; every other `commandToClient` name is ignored[328]
4.04r2 raises `onGraal3DLoginStep` 1 to 5: phases 1, 2 and 3, lighting done (`$missionRunning = 1`), and the first control object (switch to the 3D GUI)[328]
5.12 starts the value at −1 and raises 2 (phase 1), 3 (phase 2), 4 (lighting done), 5 (first control object) and 6 (TCP file downloads finished, the `Graal3DControl` GUI attached)[356]
The 3.153 loading screen is a snowball rolling across `LoadingGui`, with the Christmas 2004 3D images[394]

The retained Windows 4.04r2 review traces native datablock/ghost/lighting phase dispatch and network-class construction into the mission group, with a deferred-preload path. This is positive network-mission evidence; separate local mission/script readers remain unresolved (→ App. E#502, → App. E#503). No new 3D session or file-loading trace was obtained.[393]

10.4 DSO v33

Graal 3.x keeps menus, options, key binds, message boxes and the 3D mission-loading flow in compiled TorqueScript; from 4.x on no `.dso` files ship[394].

10.4.1 Shipped scripts

Each build ships its own set of compiled scripts. The table gives a build and what its scripts cover[394].

Build	Scripts
3.04 Linux	39 compiled scripts in the TGE starter layout; a PM window and a player list in script; no 3D scripts[394]
3.153 Mac	44 compiled scripts (`scripts/`, `scripts2D/`, `scriptsgui/`) and 13 plain `.cs` files (prefs, key configs, nine language tables); adds the 3D client scripts[394]
4.x and later	none; menus are native or GS2[394]

10.4.2 File layout

All integers little-endian[394].

Field	Notes
U32 version	33[394]
U32 size, bytes	global string table, NUL-separated[394]
U32 count, count × F64	global float table[394]
U32 size, bytes	function string table[394]
U32 count, count × F64	function float table[394]
U32 codeLength, U32 lineBreakPairCount	[394]
code	`codeLength` words; each one byte, or `0xFF` then a U32[394]
lineBreakPairCount × 2 × U32	line and ip pairs[394]
U32 identCount, then per ident U32 string offset, U32 count, count × U32 ip	patches identifier words in the code with global-string entries[394]

Compiled TorqueScript files have tables for strings and numbers. The rules below give how they are used and one byte-order difference on the Mac[394].

Rule
Code inside a function body uses the function tables; code outside uses the global tables[394]
The Mac PPC build writes the F64 tables big-endian while the rest stays little-endian; the 3.04 Linux files are little-endian throughout[394]
An argument the body never uses gets an ident offset just past the end of the global string table, so its name is lost[394]
All 83 shipped files decode with the opcode table below and no other[394]

10.4.3 Opcodes

The numbering differs from stock TGE 1.x: two unused opcodes follow `CREATE_OBJECT` and one follows `CALLFUNC`[394].

#	Opcode	#	Opcode	#	Opcode
0	FUNC_DECL	28	SHL	56	STR_TO_UINT[394]
1	CREATE_OBJECT	29	AND	57	STR_TO_FLT[394]
2	unused	30	OR	58	STR_TO_NONE[394]
3	unused	31	ADD	59	FLT_TO_UINT[394]
4	ADD_OBJECT	32	SUB	60	FLT_TO_STR[394]
5	END_OBJECT	33	MUL	61	FLT_TO_NONE[394]
6	JMPIFFNOT	34	DIV	62	UINT_TO_FLT[394]
7	JMPIFNOT	35	NEG	63	UINT_TO_STR[394]
8	JMPIFF	36	SETCURVAR	64	UINT_TO_NONE[394]
9	JMPIF	37	SETCURVAR_CREATE	65	LOADIMMED_UINT[394]
10	JMPIFNOT_NP	38	SETCURVAR_ARRAY	66	LOADIMMED_FLT[394]
11	JMPIF_NP	39	SETCURVAR_ARRAY_CREATE	67	TAG_TO_STR[394]
12	JMP	40	LOADVAR_UINT	68	LOADIMMED_STR[394]
13	RETURN	41	LOADVAR_FLT	69	LOADIMMED_IDENT[394]
14	CMPEQ	42	LOADVAR_STR	70	CALLFUNC_RESOLVE[394]
15	CMPGR	43	SAVEVAR_UINT	71	CALLFUNC[394]
16	CMPGE	44	SAVEVAR_FLT	72	unused[394]
17	CMPLT	45	SAVEVAR_STR	73	ADVANCE_STR[394]
18	CMPLE	46	SETCUROBJECT	74	ADVANCE_STR_APPENDCHAR[394]
19	CMPNE	47	SETCUROBJECT_NEW	75	ADVANCE_STR_COMMA[394]
20	XOR	48	SETCURFIELD	76	ADVANCE_STR_NUL[394]
21	MOD	49	SETCURFIELD_ARRAY	77	REWIND_STR[394]
22	BITAND	50	LOADFIELD_UINT	78	TERMINATE_REWIND_STR[394]
23	BITOR	51	LOADFIELD_FLT	79	COMPARE_STR[394]
24	NOT	52	LOADFIELD_STR	80	PUSH[394]
25	NOTF	53	SAVEFIELD_UINT	81	PUSH_FRAME[394]
26	ONESCOMPLEMENT	54	SAVEFIELD_FLT	82	BREAK[394]
27	SHR	55	SAVEFIELD_STR	—	—[394]

The rules below give the operands of the opcodes that take them[394].

Rule
Operands: `FUNC_DECL` name, namespace, package, hasBody, end ip, argc, then argc arg idents; `CREATE_OBJECT` parent ident, isDatablock, fail ip; `ADD_OBJECT` and `END_OBJECT` one flag; jumps one target[394]
`SETCURVAR`, `SETCURVAR_CREATE`, `SETCURFIELD`, the `LOADIMMED_*` opcodes, `TAG_TO_STR` and `ADVANCE_STR_APPENDCHAR` take one operand; `CALLFUNC` and `CALLFUNC_RESOLVE` take name, namespace and call type (0 function, 1 method, 2 parent); the rest take none[394]
The operand shape is the same as stock TGE 1.x; opcodes 2, 3 and 72 never occur, so what they were is unknown[394]
The repo has a disassembler and a decompiler for this format under `torque/tools/`[394]

10.4.4 What the 3.153 scripts do

The 3.153 scripts run the menus and settings. Each row gives one script area and what it does[394].

Script area	Behaviour
start-up	`main.cs` opens the console, runs `main2D.cs` and writes default prefs (fixed UDP port 14899, font size 22, OpenAL); `main2D.cs` loads the GUI and binds and shows the 2D main menu; the 2D game is the native control `Adventure` (`GuiGraalCtrl`)[394]
menus	call native `Adventure.*` methods (nickname, account, password, language, server list, offline mode, direct connect, connect, account fields); with `$serverstartconnect` set and an account filled in, the menu goes straight to the server list, which is how `-connect` and `graal://` links join[394]
2D keys	arrows move, A/S/D act (Space = S), Q, P, M, T microphone, F5 load level, F7 player list, F8 server list; global F1 help, F2 console, F3 options, Alt+Enter full screen, Esc[394]
3D keys	WASD and arrows move and also drive the Graal player; mouse look with sensitivity clamped to 0.05–2.0 and scaled by FOV/90; Space jump, left mouse fire, right mouse first person, E free look[394]
3D start	`main3D.cs` maps nine terrain textures to detail textures and footstep sounds, loads the play, loading and download GUIs, sets net port 0, creates `GameConnection(ServerConnection)` and connects[394]
connection errors	every Torque reject code maps to a message box, including the Graal protocol-less and protocol-greater texts[394]
chat over Torque	stock `clientCmdChatMessage` / `clientCmdServerMessage`, ending in native `onChatMessage` / `onServerMessage`[394]
3.04	the key map equals 3.153's 2D map, named `moveMap`, with F3 bound in the map[394]

10.5 Object model

10.5.1 Networked classes

Each build registers its own set of networked classes. The table gives each build's set[356].

Build	Networked classes
3.04	stock TGE; no Graal net classes[356]
3.153	the TGE game, datablock and Graal event classes of 4.04r2, without `AudioProfile`[356]
4.04r2	117 class registrations, 98 networked: 49 objects, 34 datablocks, 15 events[356]
4.20r6 objects	`Camera`, `DynamicGrass`, `DynamicGrassReplicator`, `DynamicShapeReplicator`, `GameObject`, `SimpleNetObject`, `Sky`, `Sun`, `TerrainBlock`, `WaterBlock`, `fxSunLight`, `twSurfaceReference`[356]
4.20r6 events	`BigDataChunkEvent`, `BigDataStartEvent`, `ConnectionMessageEvent`, `GhostAlwaysObjectEvent`, `NetStringEvent`, `RemoteCommandEvent`, `RequestReloadEvent` (C→S), `SimpleMessageEvent`, `TerrainChunkEvent`, `TerrainRequestUpdatesEvent` (C→S)[356]
5.12	10 objects (4.20r6's without `SimpleNetObject` and `twSurfaceReference`) and 9 events (without `SimpleMessageEvent`)[356]

4.04r2 networked classes, all stock TGE unless marked[328]:

Group	Classes
shapes and players	`AIPlayer`, `Player`, `ShapeBase`, `StaticShape`, `Item`, `TSStatic`, `Shape3ds` (Graal), `Projectile`, `Debris`, `Splash`[328]
vehicles	`FlyingVehicle`, `HoverVehicle`, `WheeledVehicle`, `VehicleBlocker`[328]
cameras	`Camera`, `PathCamera`, `AdvancedCamera`[328]
world	`InteriorInstance`, `PathedInterior`, `TerrainBlock`, `WaterBlock`, `Sky`, `Sun`, `Precipitation`, `Lightning`[328]
mission and markers	`GameBase`, `Marker`, `MissionArea`, `MissionMarker`, `SpawnSphere`, `WayPoint`, `Trigger`, `PhysicalZone`, `ScopeAlwaysShape`, `SimpleNetObject`[328]
effects	`AudioEmitter`, `ParticleEmitterNode`, `FractalTree`, `fxFoliageReplicator`, `fxGrassReplicator`, `fxShapeReplicator`, `fxShapeReplicatedStatic`, `fxLight`, `fxSunLight`, `volumeLight`, `twSurfaceReference`[328]
Graal replicators	`DynamicGrass`, `DynamicGrassReplicator`, `DynamicShapeReplicator`[328]
datablocks	the stock `…Data` classes for shapes, images, items, players, cameras, vehicles, projectiles, effects, particles, audio, markers, interiors and triggers[328]

The rules below give details of those classes in 4.04r2 that a string search would miss[356].

Rule
In 4.04r2 `Item`, Sky, Sun and Path register by string address, so a plain string search misses them[356]
In 4.04r2 ShapeBase carries a Graal chat string drawn above the head, with emoticon handling; how the client's own chat reaches the 3D server was not found and probably goes over TCP[328]
Every other class registers with group mask 0 (not networked); the `GameShape*` classes ride inside a `GameObject` update[356]

10.5.2 The 5.12 `GameObject` model

All classes are script-visible through GS2 property and method tables[358].

Class	What it is	Notable members
`GameObject`	every 3D thing, a ghosted `SceneObject`	mounting, node position and transform, interior checks, drop height, box collision, the controlling client, show-images on 3D objects, `onMount`, `onUnmount`, `onAdd`, `onRemove`; script variables and attributes synced to clients[358]
`GameShapeDTS`	Torque DTS model	`playAnimation(slot, name, flags)` with flags 1 hold, 2 at end, 4 attack; `playAnimation2` with start, end, time scale and transition; pause, stop, node and material lists, `onEndAnimation`[358]
`GameShapeDIF`	Torque interior	`activateLight`, `deactivateLight`[358]
`GameShape3DS`	3D Studio model	collision hulls are meshes named `Col-…`, closed and convex[358]
`GameShapeFT`	3D text from a TrueType font	[358]
`GameShape`	base class	pixel and vertex shaders (GLSL), fractal-tree fields[358]
`GameMovementInterpolate`, `…List`, `…Rigid`	how an object moves	`move3d`, `finishMoves`, `addRigidMove`, `applyImpulse`, mass, `onImpact`, `onCollision`, `onEnterLiquid`, `onLeaveLiquid`[358]
`Camera`	the view	modes `free` and `lookat`, follow an attached object, look position and vector, Euler rotation[358]
`DynamicGrass`, `DynamicGrassReplicator`, `DynamicShapeReplicator`	ground cover	terrain alignment, slope limits, allowed surfaces, per-grid counts[358]
`Sky`, `Sun`, `WaterBlock`, `fxSunLight`, `TerrainBlock`	stock Torque	three cloud layers, fog volumes, fog storms with server-synchronised time[358]

5.12 adds a `GameObject` class that bridges 3D and 2D Graal. The rules below give its model and its GS2 properties[358].

Rule
The 2D Graal player is still the player; 3D objects can carry ganis and show-images[358]
The GS2 properties on `GameObject` are `alpha`, `attachedtoobject`, `attr`, `attrobjects`, `chat`, `collision`, `iseditorobject`, `liquidheight`, `liquidtype`, `movetype`, `nickcolor`, `nickname`, `scaledshapebox`, `shadow`, `shape`, `shapebox`, `showterrain`, `skin`, `solid`, `velocity`, `watercoverage`[356]

`GameObject` update, 5.12: thirteen flag-guarded sections; the mask bits are the server's dirty bits, and on the wire each section is one flag[356].

#	Dirty bits	Payload after the flag
1	`0x1`	string, the object's name[356]
2	`0x2` or `0x200`	3 × 32 bits (velocity, inferred)[356]
3	`0x8`	string, a setting of the movement object[356]
4	`0x10`	string, also passed to the movement object[356]
5	`0x20`	transform: position, rotation, then 32 bits; the packing is unmeasured[356]
6	`0x80`	string that creates or replaces the shape object (`shape`, inferred)[356]
7	`0x40`, with a shape object	the shape object's own data[356]
8	any of `0xF00`, with a movement object	int14 bit length, then the movement object's data[356]
9	`0x1000`	string, the chat text (with emoticons)[356]
10	always 0 from this DLL	the reader takes a string and loads it as a script or image; what a real server sent is unknown[356]
11	any of `0x3FC000`	the attribute list[356]

#	Dirty bits	Payload after the flag
12	0x400000	string, then 4 colour bytes RGBA 0–255 (nickname and nickcolor, inferred)[356]
13	0x800000	int16 of booleans; bits 0x01, 0x04, 0x08, 0x10, 0x20, 0x40 are used[356]

10.5.3 Graal events over Torque

Graal sends its own events over the Torque connection. Each row gives an event, its direction and its layout[328].

Event	Direction	Layout
BigDataStartEvent	both	int16 transfer id, three strings (s1, s2, s3, each up to 255 characters), int24 total length, int3 coding (0 none, 1 zlib, 2 bzip2)[328]
BigDataChunkEvent	both	int16 transfer id, int8 length (up to 191), the bytes[328]
TerrainChunkEvent	S→C	int4 chunk x, int4 chunk y, u32 (a stamp, inferred), int10 length, then a zlib 16 × 16 piece of heights and materials; 16 × 16 chunks make a 256 × 256 terrain[328]
TerrainRequestUpdatesEvent	C→S	256 × u32, one per chunk, telling the server what the client already has[328]
RequestReloadEvent	C→S	asks the server to resend an object[328]
SimDataBlockEvent (4.04r2)	S→C	int12 (id − 3), int12 class id, int12 index, int13 total, sized datablock data[328]
RemoteCommandEvent	both	int5 argc, tagged-string arguments; the 4.04r2 client acts only on the mission phases[328]
others	—	GhostAlwaysObjectEvent, NetStringEvent, ConnectionMessageEvent, PathManagerEvent, SimpleMessageEvent, the audio and lightning events, stock TGE[328]

The rules below give how a BigData message is shaped and how the sender compresses and splits it[34].

Rule
A BigData message has the shape of GS2 sendtext: weapon, type, option, data[34]
The sender uses zlib from 50 bytes and bzip2 from 4096 bytes, and sends the start event then up to 16 chunks per call; the TCP framing uses nearly the same limits (48 and 4096, → §2.2)[328,34]
S→C with s1 set: the weapon of that name gets onReceiveText(s2, s3, params)[328]
S→C with s1 empty, s2 is a command: set / unset a player flag; events (lines s <flag>, or t <target>,<kind>,<action>,<params> on a weapon, GUI or NPC; kind rc goes to onRCChat and torque is ignored); addweapon (with a 5-byte Graal integer, a time stamp inferred); weaponscript; deleteweapon; addnpc (a script for a named client-side object); setmaterials (terrain materials); rpg (→ onRPGMessage)[328]
In 3D mode the 3D server, not the TCP gserver, delivers the client's GS2 weapons and flags[328]
C→S the engine batches its game events as one message with s2 = "events": w <weapon> from getweapon, s <name>=<value> from client-variable and flag sets, t weapon,<name>,<params> from a server-side trigger[328]
The 4.04r2 DLL's server-side BigData handler is empty[328]
The 5.12 client streams terrain from the server instead of reading .ter files ("Loading terrain from terrain chunks."); terrain textures must be 256×256 at 24 bits[358]
The 4.04r2 client caches terrain under levels3d/terrain/ and never requests .ter over TCP[328]

10.6 3D formats

The Torque clients read Torque's 3D formats. Each row gives an extension, its format and the builds that read it[358].

Ext	Format	Builds
.dts, .dsq	Torque shape and sequence; 4.04r2 reads up to version 24; 5.12 rejects shapes that are too old	3.x, 4.04r2, 5.12[358,328]
.dif	Torque interior; 4.04r2 reads resource version 44, interior version 0	all[328]
.ter	Torque terrain, version 3 or older, from levels3d/terrain/	3.x, 4.04r2[328]
terrain chunks	streamed with TerrainChunkEvent (→ §10.5.3)	5.12[358]
.3ds	3D Studio (Shape3ds, then GameShape3DS)	3.153, 4.04r2, 5.12[358]
.md3	Quake 3 model in head, upper and lower parts, version 15	3.153, 4.04r2[358,328]
.mis	Torque mission file; 5.12 has no .mis loader and builds its MissionGroup from ghosts (inferred)	3.x; 4.04r2 disputed → App. E#502[358,328]

Ext	Format	Builds
`.ml`, `.ifl`, `.dml`	material lists and image-file lists for animated textures	stock Torque[358,328]
`.gft`	Torque cached GUI font	3.x, 4.04r2, 5.12[358,328]
`.dso`	compiled TorqueScript (→ §10.4)	3.x; 4.04r2 disputed → App. E#503[394,328]
images	`.png`, `.jpg`, `.jp2`, `.jng`, `.mng`, `.gif`, `.bmp`, `.bm8`, `.tga`, `.dds`	4.04r2[328]
`.zip`, `.gpake`	archives the resource manager opens	4.04r2[328]

The rules below give where 3D content came from and how terrain textures are mapped[358].

Rule
No `.dts`, `.dif`, `.ter` or `.mis` file ships with any Graal client: the 3D content always came from the server[358]
3.153 maps terrain textures to detail textures and footstep sound types in script (→ §10.4.4)[358]
4.04r2 ships 3D GUI skins in `levels3d/gui/`, help pages in `levels3d/help/*.hfl` and nine `languages3D/language*.cs` tables of `$language::Name = "text";` lines, probably loaded by server GUI scripts[328]

10.7 Linux 3.04: startup and compiled-script reading

The two 3.04-labelled unpacked trees are identical across 528 regular files each; the packed original is a separate input. The tested unpacked initializer fails its first observed signature check. The unchanged original passes both checks after its temporary/display environment is corrected. The comparison does not isolate packing as the cause.[395,395]

In another original-input launch, the compiled reader returns true for `common/main.cs`: post-read state has 474 code words and 39 line pairs, matching the packaged version-33, 1,903-byte compiled file. The boolean is its low return byte, not unspecified upper bits. Packaged-file identity is retained; runtime memory-payload hashing and instruction execution are unmeasured. Termination and cleanup follow the reader return. No server, offline game or Windows E#503 resolution is claimed.[395]

A separate later unchanged-packed-input probe follows the same program object in one thread from compiled reading through first code-word fetch to native handler entry; next index is one. Of 474 words, 401 nonrelocation positions match and 73 relocation positions are excluded. Termination occurs at handler entry: no handler/script completion or gameplay is proved. Earlier reader-only and unpacked-startup failures retain their own stopping points. Server/commit: none; observations and original-carrier audit remain researcher-supplied.[395]

10.8 Linux 3.04: packaged Torque streams and the later trees

Each duplicate 3.04 label has the same 39 version-33 streams, 48,310 numeric words and 733 line pairs. Independent serialization reproduces every stream byte-for-byte. Numeric line-pair meanings remain unassigned; out-of-range identifier-pool selections remain flagged, separately from the reader's empty-string fallback.[396]

A 1,463-file inventory across eight prepared trees (including eight generated launchers and both duplicate 3.04 labels; not pristine installer counts) finds no packaged `.dso` or offset-zero version-33 candidate in the six later supplied trees. All nine supplied localization script resources match across the three 4.x labels; older counterparts differ. The supplied standalone 6.x folders originally held only executables; each second prepared-tree file is a generated launcher. These are two 6.01r1 architectures and 6.03r7 x64, not three releases. The 4.x trees are 4.04r2, 4.20r6 and 4.20r8 as identified in the source note. These offline results exclude neither embedded/downloaded streams nor an engine/compiler. No process/server ran; later execution and Windows E#503 remain unproved.[396]

10.9 Linux 3.04: a seven-operation executor subset

Selected Linux 3.04 static paths establish bit-preserving immediate load, one-entry discard, consuming zero/nonzero conditional branches, absolute code-word targets, signed decimal conversion and return of the current string position. The analysis ELF and original packed executable retain distinct identities.[361].

Seven independently authored acceptance vectors distinguish formatting and branch/stack behavior. They have not been executed in the original client; fresh invocation state, errors, string lifetime and complete VM equivalence remain unproved. The implementation comparison is pinned to its inspected snapshot. No runtime or server was

started for this publication.[361].

Native-call evidence boundary

A retained exact-input Linux 3.04 static review distinguishes script-defined recursive entry from registered native callback dispatch with argument-count checks; a selected resolved-call path can rewrite its word or operand to a cached form. Some registry helpers remain outside the selected export. This is static path evidence, not callback execution or complete lookup/caching policy.[361]

Published inclusive command/method counts do not establish argument representation or callback ABI. The seven-operation executor return does not establish native callback result conversion or lifetime. Case/namespace precedence, duplicates, coercions, arity failures, cache invalidation and echo behavior remain unqualified; independent API policy tests are not original-client oracles. No new native inspection or runtime work was done for publication.[361]

10.10 Linux 4.20r6: native-object registration

In the selected Linux 4.20r6 paths, constructor stores and runtime type metadata identify concrete SimGroup and GameConnection objects, each containing a Graal scripting-variable base subobject. The creation callback registers those base-subobject pointers on the scripting side, exposing a concrete static native-object bridge. Names alone are not the identification evidence.[397].

A tail-call forwarding path does not establish an ordinary C callback ABI; an optional fast path remains unresolved. The separate r8 review shows that equal embedded payloads do not imply equal execution-state layouts. Initializer execution, live TorqueScript, GUI/input and a working 3D session remain unobserved in this scope. No runtime or server was started for publication preparation.[397].

10.11 Linux 3.04: show-image wrapper constants

Retained exact-input static findings bind showimg/showimg2 to mode 0, showani/showani2 to mode 4, showpoly/showpoly2 to mode 3 and showtext/showtext2 to mode 2, with respective flags 0/1 in every pair. These internal constants add precision to already documented command presence and polygon machinery; mode 1, signature-letter grammar, flag meaning and rendered results remain unqualified.[365]

A selected common helper uses a numeric-derived integer display key and finds or creates its indexed object. Lookup/insertion compares that key; the insertion helper rejects an equal key and otherwise inserts by position. This is a sorted integer-key container, not a generic name map. Numeric conversion edges, ownership, full helper ABI and corruption handling remain open.[365]

Local insertion rejection does not establish repeated-command behavior or console/namespace duplicate policy. These are retained static assertions, not new native execution or a general interpreter/callback contract; no cross-build behavior is inferred. Document digests are checked while underlying native observations remain researcher-supplied.[365]

10.12 Linux 3.04: local value helpers

Retained exact-input static review records a numeric accessor returning zero for null input, reading a stored double on its reviewed type/slot-absent branch and otherwise dispatching through a type-specific slot. A string-result accessor dispatches through its type-specific slot or a fallback initializer. A separate assignment helper includes scalar/numeric-array paths, element conversion and selected index-guarded writes.[366]

These are local branch observations, not a complete coercion, array/index or formatting specification. They do not establish universal variable defaults, an empty-string result, every runtime type, rounding/exceptional-value policy, parser grammar or storage lifetime. They are separate from the published Torque integer-to-string operation and do not establish a native callback result adapter or ABI. Underlying static findings remain researcher-supplied; only retained documents were reviewed for publication and no original-client conversion result is claimed.[366]

11 Tools and the formats they define

The official and community tools around Graal, covered for the files they read and write and the behaviour that bears on those files. The byte-level formats themselves are in Part III; the staff wire protocols are in → §7.

> Status: draft.

11.0 Tool map

The table maps each tool to its role, the files it defines or writes, and the section that describes it[398].

Tool	Role	Files it defines or writes	Section
Built-in level editor	editor inside each client up to 2.22	GR-V1.03, GLEVNW01, GOBJSET01 sets, map images	→ §11.6[398]
GraalEditor	standalone level editor from 2.3	GLEVNW01 with HEIGHTS, gmaps	→ §11.1[399]
GraalShop	GANI editor	GANI0001, GANI0002	→ §11.2[399]
TerrainGenerator, DungeonGenerator, GraalLevelGen	world generators	GLEVNW01, GR-V1.03, GRMAP001, bigmap.txt, GDUNG001 options	→ §11.3[400]
gif2mng	image converter	PNG, MNG	→ §11.4[401]
RemoteControl, NPC Control	staff clients	— (wire only)	→ §11.5, → §7[402]
TextConvert, Character Designer, GChar, gserverstat and others	converters, offline editors, server-side helpers	GLEVNW01, CHR:1.00, HTML leaderboards	→ §11.7[403]

11.1 GraalEditor

11.1.1 Lineage and package

The facts below give how the standalone editor came out of the client and what its package holds[399].

Fact
Up to 2.22 the editor is built into the client (F4, refused online); 2.31 has no editor forms, and its start screen has an Editor button that launches GraalEditor.exe from the game folder[399]
GraalEditor 2.3.1.1 (2005) is the same TAdventure program with the online client removed: the 2.05 main form, tool forms, toolbar hints and even game-only strings remain[399]
It keeps the whole game model and GS1 interpreter, so Play runs a level locally; it has no network client, list-server host or login token[399]
The shipped copy is a Thinstall package (magic 0x82D0B188); the Delphi image is recovered by running the stub decompressor and its block codec[404]
Version resource: company Linux Cyberjoueur, file version 2.3.1.1, internal name Graal editor, product GraalEditor[404]
The main form is TAdventure, the document TAdventureLevel, the unit UAdventureEditor[404]
Greaditor.exe (2001, in the New World beta folders) is a separate UPX-packed editor that was not unpacked[405]

11.1.2 Files

The rules below give which level formats GraalEditor opens and how it saves[399].

Rule
Open: GR-V1.00–1.03, Z3-V1.03/1.04 and GLEVNW01, with the filter Graal levels (*.graal, *.nw)[399]
Save: GLEVNW01 with a new HEIGHTS section; the GR-V1 writer strings are still present; "Level(s) saved" means several levels can be saved at once[399]
The BOARD 0 layer as GraalEditor writes it (disputed → App. E#600)
Heights are a TLevelHeights overlay per level, not a second board[404]
Sections read and written: BOARD, LINK, SIGN, CHEST, NPC, BADDY, LEVELNAMES, in the field order of the dialogs (→ §3.1.3)[404]
The editor stores no tile types; collision comes from the 4096-byte type table (→ §3.5.2)[404]

Rule
gmaps are wired in: the GRMAP001 reader plus LEVELNAMES, MAPIMG, MINIMAPIMG, NOAUTOMAPPING and the generator keys (→ §3.2); a TEditorLevelCache holds neighbours[399]
2.05 already has a dormant GRMAP001 reader with no caller[399]
Offline data: offlinetutorial_* (3×3) levels and tiledefsOffline.txt[399]

11.1.3 Additions over the 2.05 built-in editor

GraalEditor adds features over the 2.05 built-in editor. Each row gives an area and what was added[399].

Area	GraalEditor
terrain	per-level height overlay, "Terrain Height Spot", "Show terrain grid"[399]
palette	a fourth mode, Predefined NPCs, with a parameter grid[399]
NPCs	one TEditNPC window with image, script, Test, Style and Find; scripts may carry //#EDIT and //#COMMENTS markers[399]
character NPC	the same fields on four tabs (Look, Movement, Attack, General)[399]
tiles	Export Tiles Array: the selection as one line of tile indices and as a script array[399]
view and log	Hide npcs, Show terrain grid, a log window[399]
sound	BASS, with the function set the 2.22 client imports[399]
integrity	the client self-check plus RSA/SHA-1 checks of GraalEditor.sig and arrays2.sig; live, the editor starts with no GraalEditor.sig and with a corrupted arrays2.sig, so the check does not gate start-up (→ §3.5.5)[399]
unchanged	the tile undo model, the link, sign, chest and baddy dialogs, GOBJSET01 sets, right-click copy, Generate map → graalmap.png[399]

11.2 GraalShop

GraalShop is the period gani editor. The facts below say what it is and where its name comes from[399].

Fact
GraalShop.exe 1.1.1.0 ("Gani Editor", 2006) and GraalShop_old.exe (2002) are Delphi programs with one main form, "Graal Shop - The Graal Animation Editor", and an Add sprite dialog; they edit .gani files only[399]
The name is a pun on Photoshop, not a store[406]
The model holds sprites with attachments (ATTACHSPRITE, ATTACHSPRITE2), frames of placements and sounds, and in movie mode actors and actor modifiers; undo and redo are a list of edit operations[399]
Sprite mode (GANI0001): direction selector, sprite-definition grid, frame strip and canvas; play, add, delete, copy and insert frames, add sound, add, import or prune sprite definitions, reverse the animation, undo, redo, file actions; selected sprites can be moved, raised, lowered and keyed by x/y[399]
Movie mode (GANI0002): a timeline of CHAR, SPRITE, SOUND and TEXT actors; TEXT has font, zoom, size, alignment, wordwrap and shading; CHAR has ani, dir, head, body, sword, shield, horse, colours, attr and param; options Automatic Gani Setting and Graal2002 ganis[399]
Header fields are edited in a grid and written as LOOP, BACKCOLOR, SETBACKTO, DEFAULTHEAD, DEFAULTBODY, DEFAULTPARAM, DEFAULTATTR, the effect keys and SCRIPT … SCRIPTEND (→ §3.3.4)[399]
The script block is copied, not run[406]
Preview uses DirectDraw 7 and plays PLAYSOUND steps through DirectSound; default art is sprites.png, body.png, head0.png, shield1.png, ride.png, soundicon.png; the Graal folder is found through Graal.exe and FILENAMECACHE.txt, and pms, profiles and weblevels are searched[399]
GraalShop_old has sprite mode only: no GANI0002, MOVIE, ACTOR, ATTACHSPRITE2 or ROTATEEFFECT[399]
2.05 plays only the sprite-mode subset (SPRITE, ANI … ANIEND, WAIT, PLAYSOUND, LOOP, CONTINUOUS, SETBACKTO, SINGLEDIRECTION, COLOREFFECT, ZOOMEFFECT), so movie-mode ganis and attachments from the 2006 GraalShop do not play there[399]
The offline tile definition in the client tree is picso.png,offlinetutorial,1,0,0[406]

11.3 Generators

Three writers emit GLEVNW01 and GRMAP001, each with a different key set; one writer's output is not a valid input for another's loader (the key sets are → §3.2.3)[407].

11.3.1 Builds

Several level generators survive. Each row gives a program, its date and what it writes[400].

Program	Date	Writes
`TerrainGenerator.exe`	2003-12-16	`GRMAP001` generator form and `GLEVNW01` levels of tile 0 with a 9×9 `HEIGHTS` block and edge links[400]
`dungeongen.exe` and the Linux `dungeongen`	2003-05-23	`GLEVNW01` levels from `GDUNG001` options, short `GRMAP001`[400]
`GraalLevelGen_nw-gmap.exe`	2003-03-07	`GLEVNW01` and a `LEVELNAMES` gmap; GR-V1.03 when the output name does not end in `.nw`[400]
`GraalLevelGen_graal-txt.exe`	2004-01-18	GR-V1.03 levels and `bigmap.txt`[408]
`GraalLevelGen_nw-txt.exe`	2001-06-30	`GLEVNW01` levels and `bigmap.txt`[408]
`GraalLevelGen.exe`	December 2000	levels named `.graal` and `bigmap.txt`; the readme says New World format and not for 1.39 or older[409]
`GraalLevelGenNW.exe`	2001-10-08	a New World levels tree from a map image, with calendar fields and per-level options such as whether warping is allowed[410]

No generator imports a socket library; output stays in the chosen folder[409].

11.3.2 TerrainGenerator

The rules below give TerrainGenerator's options and how it builds the height field[400].

Rule
Dialog fields: grid width and height, base, height, chaos, even borders, an islands preset (even borders, the rim kept at the base) and a seed[400]
The height field is plasma midpoint displacement[407]
gmap keys in order: `WIDTH`, `HEIGHT`, `GENERATED 1`, `GENSEED`, `GENBASE`, `GENEVENBORDERS`, `GENHEIGHT`, `GENCHAOS`, `LEVHEIGHT`, `LEVCHAOS`, `MAPIMG`, one level name per line, `HEIGHTMAP`, `RANDOMSEEDS`[400]
Level names: world name, an A…Z/AA column, a zero-padded row, `.nw`[400]
A level is 64 `BOARD` rows of tile 0, a `HEIGHTS` block of nine lines of nine numbers, and a `LINK` on each edge with a neighbour; no NPC section[400]
Edge arrivals: west 61, east 0, north 61, south 0; the other axis is `playerx` or `playery`[400]

11.3.3 DungeonGenerator

The rules below give DungeonGenerator's options file and how it cuts a layout into levels[400].

Rule
`dungeonoptions.txt` starts with `GDUNG001`; a `LAYOUTFILE` of X (wall) and `.` (path) wins over `LAYOUTSTYLE`[400]
Each layout cell is 8 tiles wide and 11 tall, cut into 64×64 levels named `LEVELSTART`, a column letter and a 1-based row (`cave_A1.nw`)[400]
The gmap is three lines after the signature: `WIDTH`, `HEIGHT`, `GENERATED` with the bottom-right name[400]
Each level is one base64 `BOARD`, edge links and `NPC` blocks; stamp 0 becomes a 4×4 ground fill, stamp 1 is a hole and is not written[407]
Place names for NPCs and 88-id `TILES` blocks: `path`; `walln`/`walls`/`wallw`/`walle` (one open side, wall opposite); `wallnw`/`wallne`/`wallsw`/`wallse` (one open diagonal); the …2 corner forms (two adjacent open sides); `walld1` (open north and south) and `walld2` (open east and west); `wall` otherwise[400]
`NPC` lines take `image`, `script`, `blocking`, `underplayer`, `dominant`, `chance` (1–100, `%` optional), `place`, `x`, `y`; false `blocking` adds `dontblock`, `underplayer` adds `drawunderplayer`[400]
`ADDTESTNPCS` adds one test NPC per level running `addtiledef`, `addtiledef2` and `loadmap`[400]
`LAYOUTSYMMETRY` 1 none, 2 vertical mirror, 3 horizontal mirror, 4 both, 0 picks 1–4 from the seed; `DOUBLE 1` draws at half size and scales up; `LAYOUTORIENTATION` is stored[400]
The random styles (maze, onion, spiral, rogue, snake, square spiral) come from the linked DungeonMaker 0.9 library, which reads `layouts/rooms*`, `layouts/stats*` and `layouts/constants`; its crawler steps are unmeasured[408]

11.3.4 GraalLevelGen

GraalLevelGen builds levels from a picture. The rules below give how the picture is read[409].

Rule
The input picture is 64 pixels per level across and down, saved as gif or 8-bit png[409]
The pixel grid is `(width + 5) // 6` by `(height + 5) // 6` cells, each read at `(x × 6, y × 6)`; a colour matches when the sum of absolute channel differences is under 32[400,408]
Forest blits a 6×5 stamp of tiles 25–30, 41–46, 57–62, 73–78, 89–94 at `(x × 6, y × 6 + 1)`, shifted 3 tiles left on odd rows[400]
Other colours write one fill tile: water 12, mountains 80, flowers 4, sand 5, puddle 6, puddle stone 7, small stones 8, big stone 9, swamp 10, bushes 11; grass and unknown colours write nothing[400]
Water, mountain and sand edges are blitted from neighbour flags; the stamp-to-shore table is unmeasured[408]
nw-gmap names are `world_a-01.nw` to `world_z-99.nw`, and the gmap is `WIDTH`, `HEIGHT` and quoted `LEVELNAMES` rows; edge links use the 0/61 arrivals[400]
graal-txt names are prefix + column + `-` + row + `.graal`; nw-txt names drop the dash and end in `.nw`; column letters are a + index mod 26 with a leading letter from 26 levels across; the row is 1-based with leading zeros from 10 and 100 levels down[408]
`bigmap.txt` is one row per line of `"name","name"` (the editor's M-key map)[408]
A second run on existing levels keeps their links and NPCs and paints only the new non-grass tiles[400]

Colour table, the same in the December 2000 build and the nw-gmap build[409,400]:

Index	Meaning	RGB
0	grass, not drawn	24, 140, 24[400]
1	forest	181, 239, 189[400]
2	water	0, 0, 255[400]
3	mountains	128, 0, 0[400]
4	flowers	255, 0, 0[400]
5	sand	150, 110, 0[400]
6	puddle	0, 255, 255[400]
7	puddle stone	0, 179, 179[400]
8	small sand stones	113, 82, 0[400]
9	big sand stone	83, 61, 0[400]
10	swamp	0, 106, 0[400]
11	bushes	0, 66, 0[400]

11.4 gif2mng

gif2mng converts images for the client. The rules below say what it does[401].

Rule
A GIF converter with banner date 2004-06-03, units `UGif` and `UMNG`, zlib linked[401]
One frame becomes PNG, two or more become MNG[401]
`-b` rewrites the existing palette to the seven Graal body-colour slots and forces transparent index 2; it does not add a second palette (the slots' draw roles are → §2.6.3)[401]
The seven slot RGB values and the MNG `TERM` delay fields are unmeasured[408]

11.5 RemoteControl and NPC Control

The RC and NC wire protocols, logins and commands are → §7; the RC lineage from `GSERV014` to `GSERV023` is → §7.1. This section lists the recovered staff-client binaries and what each shows.

Binary	Toolkit	Behaviour
`RemoteControl.exe` (classic `rc.rar` line)	Delphi	server-list connect screen; registry prefs, a local startup script and password fields; NPC attributes, script and flags, NPC level list, local NPCs; player count and NPC-server status; account and player editors; deflate[402]
custom `remotecontrol.exe` and a UPX `Agret_RC.exe` variant	Delphi	the classic RC role with a blue theme and a text config with nickname and UI-colour keys; the unpacked `Agret_RC.exe` is a different build[411]

Binary	Toolkit	Behaviour
`RemoteControl-NW.exe` (New World packs, NC V2 pack)	Delphi	account and player editors and a database account search by admin level, admin worlds and block state; admin message and broadcast, edit account, update level, disconnect, ban, away-on-close, HTML in PMs[412]
`RemoteControl3.exe` (RC3 beta, also in a 5.x pack)	GTK on Windows	server list, version and password fields; list-server and separate NPC-server connections; player list with mass PM and admin message, NPC lists, level-NPC dump, folder and config pages, profile, rights, account and access editors; ban with time and reason, warps, disconnect, update level, separate NC chat; bzip2 and zlib; highlighters for Graal script, ini, Pascal and C++[413]
`NPCControl.exe` (`nc7132001`, July 2001)	Delphi	a text config of server name, port and password mirrored in the registry; NPC and weapon management, take a script from a class, edit weapon scripts, local NPCs; deflate[414]

11.6 Level editor

The built-in editor, with 2.22 as the reference and every other build as a delta. The 2.05-specific in-client behaviour is → §4.9. Formats it reads and writes are → §3.1[398].

11.6.1 Entering, window and modes (2.22)

In 2.22 the editor is a mode of the game window. The rules below give how it is entered and laid out[415].

Rule
The editor is the `TAdventure` main form in a second mode; the same exe plays and edits[415]
F4 opens it and is refused online ("You cannot open the editor in the online mode."); from a tutorial level it opens a new empty level[415]
F5 opens Load level; P or Play returns to the game on the current level, saving every modified level first since 2.13[415]
A level path as the first command-line argument skips the login screen and opens it in the editor[415]
F10 toggles the window border; Esc warns before leaving; dropping a text file creates an NPC from it[415]
Board view: 64 × 64 tiles at 16 px with scroll bars, wheel and arrow-key scrolling[415]
Toolbar order: New, Open, Save, Save as, Undo board copy, Cut, Copy, Paste, Delete, Turn selection into a warp field, Edit links, Edit signs, selection-as-text (2.16), Close, Play[415]
One tab per open level; Close asks about unsaved changes[415]
Palette modes: predefined object (stamp), tile palette, baddies/NPCs/chests, predefined NPCs (`defnpc*.txt` with an `#EDIT` grid)[415]
Blocking, foreground and terrain overlays are computed from the type table, not stored[415]
Editor control-NPCs (2.13): `editnpc<Name>.txt` scripts editor actions with `mousex`, `mousey`, `mousescreenx`, `mousescreeny` and `mousebuttons` (left 1, middle 2, right 4); only tile changes are supported[415]
M builds a PNG map image of the level, and of a gmap from 2.16 rev 7[415]
The mouse state machine (select, drag, drop, double-click by kind) is known only in outline: left drag selects or paints, right button on an object clones it, right button elsewhere flood-fills, double-click opens an object's dialog or makes a tile the background tile[415]

11.6.2 Level model (2.22)

The editor holds a level as a set of lists. Each row gives one list and what an element holds[415].

List	Element
board	64 × 64 16-bit tile ids, index y × 64 + x[415]
links	destination, x, y, w, h, new-x text, new-y text[415]
signs	x, y in tiles, text[415]
chests	x, y, item, sign index[415]
NPCs	image or none, x, y in fractional tiles, script[415]
baddies	x, y, type, three verse strings[415]
heights	optional 9 × 9 floats[415]
cache	recently loaded levels, for the link walker and gmaps (2.17 on)[415]

11.6.3 Open and save (2.22)

The rules below give how the 2.22 editor opens and saves levels[415].

Rule
The open filter is `Graal levels (*.graal, *.nw)`; several files open in their own tabs[415]
The loader branches on the first eight bytes: `GLEVNW01` text; `GR-V1.00–1.03` binary (13-bit codes for 1.02/1.03, 12-bit for 1.00/1.01); `Z3-V1.03/1.04`; anything else is rejected ("Unsupported file type!", no caller found)[415]
`GR-V1.04` is read only by the weblevels cache path, not by the editor[415]
The text reader rules (layer field ignored, row skipping, 12-bit tile ids, `LINK` defaults, `BADDY` clamp, `HEIGHTS` only when an overlay exists) are → §3.1.3[415]
Save (Ctrl+S) is save-all: it commits a floating paste, then saves every modified level with a file name plus the current one, deleting the old file and writing in place; a modified map is saved too; the status bar shows "N Level(s) saved"[415]
The extension picks the format: `.nw` writes GLEVNW01, anything else GR-V1.03; the editor never writes Z3-V1, GR-V1.00–1.02 or GR-V1.04[415]
The text writer emits `HEIGHTS` only when the overlay is marked modified (2.05 never does)[415]
Chests and baddies are written only when two obfuscated session-mode flags are both clear; otherwise only the terminators; the same flags gate the loaders; that they are a New World mode switch is an inference[415]
2.22 deletes before writing, so a crash loses the level; the 2.3 change log says the editor then writes a copy and renames it[415]

11.6.4 Editing tools (2.22)

The rules below give the editor's tools and how tiles and tile definitions are addressed[415].

Rule
Tile ids are 0–4095 on the 2048 × 512 sheet; the id → sheet mapping is → §3.5.1[415]
A tile definition line is `image,prefix,type,x,y`; type 0 is the old order and type 1 the new; type 4 and its x offsets are unmeasured[415]
Palette: click picks one tile, drag picks a block that pastes as a floating selection; double-click sets the background tile, which fills new levels[415]
Undo covers board tiles only: a rectangle snapshot before each write, at most 128 records per tab[415]
Draw mode: with one tile selected, left drag paints (1.41); flood fill with the right button (2.00), Shift for complete objects only, and the background tile when nothing is selected[415]
Turn selection into a warp field: a link over the selection with new x `playerx` and new y `playery`, then the first matching edge rule: starts in column 63 → new x 0; row 63 → new y 0; column 0 and one wide → new x 61; row 0 and one high → new y 61[415]
Stamps come from `objects<set>.txt` in GOBJSET01 (→ §3.4); add-selection inserts after the chosen stamp and Save objects rewrites the set; placing writes tile ids only; sets arrived in 2.04[415]
Edit links (L): Linkfile, X, Y, W, H, New X, New Y; Load opens the destination in a tab; the dialog refuses invalid fields with "Wrong values! Type in a filename..."; arrival text is stored as typed[415]
Whether the engine evaluates an arrival such as `playery+10` at warp time (disputed → App. E#601)
Edit signs (S): X, Y, New, Delete; a (0, 0) sign is not drawn and is used by index from chests and `say n`; text is in the sign alphabet (→ §3.6); trailing empty lines are removed[415]
Chest dialog: item, sign index, Sign button; items in index order `greenrupee, bluerupee, redrupee, bombs, darts, heart, glove1, bow, bomb, shield, sword, fullheart, superbomb, battleaxe, goldensword, mirrorshield, glove2, lizardshield, lizardsword, goldrupee, fireball, fireblast, nukeshot, joltbomb, spinattack`; a new chest is item 1, sign −1[415]
Baddy dialog: ten types (Gray, Blue, Red, Shooting and Swamp Soldier, Frog, Spider, Golden Warrior, Lizardon, Dragon) and three verses; types 4–6 disable and save empty verses; a new baddy says "Stop!" and "Ouch!"[415]
NPC dialog: image, script, Test, Style, Find; `//#EDIT name value` and `//#COMMENTS text` markers; predefined NPCs replace each named variable on drop; NPC positions survive a binary save only as whole and half tiles[415]
Multi-level editing (2.12): when an NPC calls `setmap` the neighbours show together; scroll by dragging the player or with arrows; a big cursor on maps (2.13)[415]
Other dialogs: load by name, log window, `TShowImg` (role unmeasured), variable editor, tile-array window[415]

Keys, 2.22[415]:

Key	Action
Ctrl+N, Ctrl+O, Ctrl+S	New, Open, Save (save-all)[415]
Ctrl+Z, Alt+Backspace	Undo board copy[415]

Key	Action
Ctrl+X / Shift+Del, Ctrl+C / Ctrl+Ins, Ctrl+V / Shift+Ins, Del	Cut, Copy, Paste, Delete[415]
L, S, P, M	Edit links, Edit signs, Play, map image; ignored while a side-panel combo has focus[415]
Ctrl+Q	batch re-save of every level named in the loaded map, by extension (2.17.1, 2.22)[415]
F10, Esc	border toggle, warning before leaving[415]

11.6.5 Builds as deltas from 2.22

Other builds' editors are described as differences from 2.22. Each row gives a build and its delta[405].

Builds	Delta
2.17.1	none found; same class list[405]
2.05, rev 1, rev 2	no heights, cache, tiles-array or log windows, no predefined NPCs; separate person-image and script dialogs; a sign-codes window; open filter `*.graal`; the map parser present from rev 1 but dormant; GLEVNW01 first appears; the chest clamp is gone[405]
2.0 rev 3 to 2.04	no GLEVNW01 reader or writer; `.zelda` dropped from the open filter at 2.0 rev 3 (the reader still accepts it); object sets and tile definitions from 2.04; chests clamped to 0–62 and NPCs to 0–63 on load[405]
2nd Graal to 2.0 rev 2	read GR-V1.00–1.03 and Z3-V1.03/1.04, save GR-V1.03; filter `*.graal, *.zelda`; same clamps[405]
1.28b2 to 1.33	baddy class still named opponent; character NPC dialog present[405]
before 1.28b2	no character dialog or level class; 2nd Graal adds chest and person dialogs, 3rd Graal the flags dialog; 1st Graal reads only Z3-V1 and has no chest or NPC dialog[405]
New World betas	no chest and no baddy editor or class; heights and map in beta 1; text format in the beta's own `levelformat.txt`[405]
2.1 test builds	combined NPC dialog, predefined NPCs, no sign-codes window[405]
2.31	no editor dialogs; the level model and GR-V1.04 reader remain for the game; no GLEVNW01[405]

The rules below give the loader limits that differ between builds[405].

Rule
The load clamps are compares against 62 for chests and 63 for NPCs in the 1.39, 2.00 and 2.04 loaders, absent from 2.05 on; a chest at x = 63 loads at 62 up to 2.04[405]
The baddy section ends at a record shorter than three bytes or whose first byte is 0x80 or more, with no x = 0 test, in 1.39, 2.00, 2.05, 2.17.1 and 2.22[416]
The editor writes double runs (ABAB…, up to 64 pairs) in GR-V1 boards, and a newline after each chest record[416]

Change-log dated editor features, from the shipped `newfeatures` files[405]:

Version	Features
1.41	drag painting with one tile, M map PNG, status-bar position and tile, double-click background tile, no selection outside the level, drop a text file to make an NPC, trailing sign lines trimmed, new levels use the background tile[405]
2.00	flood fill with the right button[405]
2.04	object sets; `addtiledef`, `addtiledef2`, `removetiledefs`[405]
2.1	predefined NPCs, merged sign-code and add-NPC windows, multi-tile palette selection, F4 on a tutorial opens an empty level[405]
2.12	multi-level editing on `setmap`, scrolling by player drag or arrows[405]
2.13	scrollbars on single levels, big cursor on maps, Play saves all, editor control-NPCs[405]
2.13 rev 3	dropped objects can be released and moved; caching and Play position fixes[405]
2.15	wheel scrolling, arrow scrolling on single levels[405]
2.16	selection-as-text button, `showimg` and `putnpc` in editor NPCs, map-level fixes; M on gmaps from rev 7[405]
2.3	editor and client become separate exes; write-copy-and-rename saving; filename cache[405]

11.7 Other tools

Other period tools are listed here. Each row gives a tool, what it does and the files it uses[417].

Tool	What it does	Files
TextConvert	on start, walks the current folder and subfolders and converts every GR-V1 level to GLEVNW01, links included; no socket	GR-V1.00–1.03 in, GLEVNW01 out; errors `Wrong link data!`, `Wrong chest data!`, `Decompression error!`[417]
Graal Character Designer 1.2.7 (Nov 1999) and GChar 1.0	offline character-look editors; need `headsconfig.txt`, `levels\heads\`, `levels\gifs\`, `levels\webgifs\`; import `*.chi` files and rename them `*.chr`; no socket	`CHR:1.00<name>=<head gif>,<male|female>,<five colour names>`, one line; GChar keeps `GChar.ini` (`drawmethod`, `headmode`, `graaldir`)
`gserverstat`, `gserverstat2`, `statmaker` (1999 gserver tree)	HTML leaderboards on standard output, 14 modes (kills default), top 50, load-only accounts and the account named `super` skipped; `statmaker` loops with 10 s and 600 s sleeps; no socket	reads `weapons.txt`, `accountslist.txt`, `accounts/accN.txt` (→ §5.1)[292]
Classic listserver `serverlist3`	list server ELF with MySQL; protocol → §6	config keys `clientsport`, `serversport`, `playerlimit`, `databaseserver`, `showmore`; status files beside the process[302]
Win32 `ListServer.exe`	Delphi directory helper reading an ini server list (→ §7)	ini, registry[419]
Script Editor 2.53	GS1 NPC and weapon script editor with templates and a find-NPC helper	GS1 scripts[420]
BytecodeViewer	console dump of a GS2 bytecode capture: skips the first byte, then the information section, function names, strings and bytecode, with opcode names from `BytecodeViewer_defs.txt` (GS2 bytecode → §8)	GS2 bytecode[421]
Assimilator ("GIF2PNG")	offline batch checker and converter for scripts, levels, text and animations	asset files[422]
PNGView	offline image viewer	PNG[423]
TilesEditor (39ster, Reborn line)	offline 64-bit Qt editor for levels, tiles, links, chests and NPCs (anonymous and external classes), layers, grid snap, map export	levels[424]
Recovered third-party trainers (AutoIt and VB6 macro tools such as AutoD, WaterGet and the AFK bots)	AFK chat bots, input macros and chat or costume spam that act on the stock client's window from outside; read statically, none run; groups and tool-by-tool behaviour → §4.10.1.4	—[260,259,262,261]
DClient setup	a VB6-era installer bootstrap; its cabinet is truncated, so the client cannot be rebuilt	—[425]

Recovered server binaries, described in Part V[426]:

Binary	Notes
`gserver13` (Linux, Dec 1999)	unstripped; account parse, level enter and update, guild verify, IP bans (→ §5.1)[426]
`gserver` (Linux) and `gserver.exe` (Win32), about June 2002	the same classic server with source beside it: NW level load, RC player props, MySQL accounts[427,428,429]
Matriark `gserver` (pack labelled 1.39)	stripped static build with database accounts, guilds, RC and NC sessions and leaderboard queries; the pack's bug note says head, shield and sword path checks disagree with the paths used when sending level files[430]

Appendix A. Message index

Every client-to-server (PLI) and server-to-client (PLO) id with its wire byte and the section that documents it. Names follow → §0.7; the wire byte is the id + `0x20` (→ §2.1.2). The *2.05* column says whether 2.05 rev 2 sends (PLI) or handles (PLO) the id. UDP peer ids are a separate id space (→ §2.9).

A.1 Client to server

This is the index of every client-to-server id. Each row gives the logical id, its wire byte, its name, whether 2.05 sends it, and the section that describes it[3].

Id	Wire	Name	2.05	Section
0	0x20	PLI_LEVELWARP	sent	→ §2.4.3[3]
1	0x21	PLI_BOARDMODIFY	sent	→ §2.4.5[3]
2	0x22	PLI_PLAYERPROPS	sent	→ §2.4.4[3]
3	0x23	PLI_NPCPROPS	sent	→ §2.4.4[3]
4	0x24	PLI_BOMBADD	sent	→ §2.4.5[3]
5	0x25	PLI_BOMBDEL	sent	→ §2.4.5[3]
6	0x26	PLI_TOALL	sent	→ §2.4.8[3]
7	0x27	PLI_HORSEADD	sent	→ §2.4.5[3]
8	0x28	PLI_HORSEDEL	sent	→ §2.4.5[3]
9	0x29	PLI_ARROWADD	sent	→ §2.4.5[3]
10	0x2a	PLI_FIRESPY	sent	→ §2.4.5[3]
11	0x2b	PLI_THROWCARRIED	sent	→ §2.4.5[3]
12	0x2c	PLI_ITEMADD	sent	→ §2.4.5[3]
13	0x2d	PLI_ITEMDEL	sent	→ §2.4.5[3]
14	0x2e	PLI_CLAIMPKER	sent	→ §2.4.9[3]
15	0x2f	PLI_BADDYPROPS	sent	→ §2.4.4[3]
16	0x30	PLI_BADDYHURT	sent	→ §2.4.5[3]
17	0x31	PLI_BADDYADD	sent	→ §2.4.5[3]
18	0x32	PLI_FLAGSET	sent	→ §2.4.6[3]
19	0x33	PLI_FLAGDEL	sent	→ §2.4.6[3]
20	0x34	PLI_OPENCHEST	sent	→ §2.4.5[3]
21	0x35	PLI_PUTNPC	sent	→ §2.4.6[3]
22	0x36	PLI_NPCDEL	sent	→ §2.4.6[3]
23	0x37	PLI_WANTFILE	sent	→ §2.4.7[3]
24	0x38	PLI_SHOWIMGPLAYER	sent	→ §2.4.9[3]
25	0x39	PLI_UNKNOWN25	no	→ §2.4.9[3]
26	0x3a	PLI_HURTPLAYER	sent	→ §2.4.5[3]
27	0x3b	PLI_EXPLOSION	sent	→ §2.4.5[3]
28	0x3c	PLI_PRIVATEMESSAGE	sent	→ §2.4.8[3]
29	0x3d	PLI_NPCWEAPONDEL	sent	→ §2.4.6[3]
30	0x3e	PLI_LEVELWARPMOD	sent	→ §2.4.3[3]
31	0x3f	PLI_PACKETCOUNT	no	→ §2.4.9[3]
32	0x40	PLI_ITEMTAKE	sent	→ §2.4.5[3]
33	0x41	PLI_WEAPONADD	sent	→ §2.4.6[3]
34	0x42	PLI_UPDATEFILE	sent	→ §2.4.7[3]
35	0x43	PLI_ADJACENTLEVEL	sent	→ §2.4.3[3]
36	0x44	PLI_HITOBJECTS	sent	→ §2.4.5[3]
37	0x45	PLI_LANGUAGE	sent	→ §2.4.9[3]

Id	Wire	Name	2.05	Section
38	0x46	PLI_TRIGGERACTION	sent	→ §2.4.5[3]
39	0x47	PLI_TAMPERCHECK	no	→ §2.4.9[3]
40	0x48	PLI_SHOOT	no	→ §2.4.5[3]
41	0x49	PLI_SERVERWARP	no	→ §2.4.3[3]
43	0x4b	PLI_MUTEPLAYER	no	→ §2.4.9[3]
44	0x4c	PLI_PROCESSLIST	no	→ §2.4.9[3]
45	0x4d	PLI_PROCESSRUNNING	no	→ §2.4.9[3]
46	0x4e	PLI_ENTERLEVEL	no	→ §2.4.3[3]
47	0x4f	PLI_VERIFYWANTSEND	no	→ §2.4.7[3]
48	0x50	PLI_SHOOT2	no	→ §2.4.5[3]
50	0x52	PLI_RAWDATA	no	→ §2.4.7[3]
51	0x53	PLI_RC_SERVEROPTIONSGET	no	→ §7[3]
52	0x54	PLI_RC_SERVEROPTIONSSET	no	→ §7[3]
53	0x55	PLI_RC_FOLDERCONFIGGET	no	→ §7[3]
54	0x56	PLI_RC_FOLDERCONFIGSET	no	→ §7[3]
55	0x57	PLI_RC_RESPAWNSET	no	→ §7[3]
56	0x58	PLI_RC_HORSELIFESET	no	→ §7[3]
57	0x59	PLI_RC_APINCREMENTSET	no	→ §7[3]
58	0x5a	PLI_RC_BADDYRESPAWNSET	no	→ §7[3]
59	0x5b	PLI_RC_PLAYERPROPSGET	no	→ §7[3]
60	0x5c	PLI_RC_PLAYERPROPSSET	no	→ §7[3]
61	0x5d	PLI_RC_DISCONNECTPLAYER	no	→ §7[3]
62	0x5e	PLI_RC_UPDATELEVELS	no	→ §7[3]
63	0x5f	PLI_RC_ADMINMESSAGE	no	→ §7[3]
64	0x60	PLI_RC_PRIVADMINMESSAGE	no	→ §7[3]
65	0x61	PLI_RC_LISTRCS	no	→ §7[3]
66	0x62	PLI_RC_DISCONNECTRC	no	→ §7[3]
67	0x63	PLI_RC_APPLYREASON	no	→ §7[3]
68	0x64	PLI_RC_SERVERFLAGSGET	no	→ §7[3]
69	0x65	PLI_RC_SERVERFLAGSSET	no	→ §7[3]
70	0x66	PLI_RC_ACCOUNTADD	no	→ §7[3]
71	0x67	PLI_RC_ACCOUNTDEL	no	→ §7[3]
72	0x68	PLI_RC_ACCOUNTLISTGET	no	→ §7[3]
73	0x69	PLI_RC_PLAYERPROPSGET2	no	→ §7[3]
74	0x6a	PLI_RC_PLAYERPROPSGET3	no	→ §7[3]
75	0x6b	PLI_RC_PLAYERPROPSRESET	no	→ §7[3]
76	0x6c	PLI_RC_PLAYERPROPSSET2	no	→ §7[3]
77	0x6d	PLI_RC_ACCOUNTGET	no	→ §7[3]
78	0x6e	PLI_RC_ACCOUNTSET	no	→ §7[3]
79	0x6f	PLI_RC_CHAT	no	→ §2.4.10[3]
80	0x70	PLI_PROFILEGET	sent	→ §2.4.8[3]
81	0x71	PLI_PROFILESET	sent	→ §2.4.8[3]
82	0x72	PLI_RC_WARPPLAYER	no	→ §7[3]
83	0x73	PLI_RC_PLAYERRIGHTSGET	no	→ §7[3]
84	0x74	PLI_RC_PLAYERRIGHTSSET	no	→ §7[3]
85	0x75	PLI_RC_PLAYERCOMMENTSGET	no	→ §7[3]
86	0x76	PLI_RC_PLAYERCOMMENTSSET	no	→ §7[3]

Id	Wire	Name	2.05	Section
87	0x77	PLI_RC_PLAYERBANGET	no	→ §7[3]
88	0x78	PLI_RC_PLAYERBANSET	no	→ §7[3]
89	0x79	PLI_RC_FILEBROWSER_START	no	→ §7[3]
90	0x7a	PLI_RC_FILEBROWSER_CD	no	→ §7[3]
91	0x7b	PLI_RC_FILEBROWSER_END	no	→ §7[3]
92	0x7c	PLI_RC_FILEBROWSER_DOWN	no	→ §2.4.10[3]
93	0x7d	PLI_RC_FILEBROWSER_UP	no	→ §2.4.10[3]
94	0x7e	PLI_NPCSERVERQUERY	no	→ §7[3]
96	0x80	PLI_RC_FILEBROWSER_MOVE	no	→ §2.4.10[3]
97	0x81	PLI_RC_FILEBROWSER_DELETE	no	→ §2.4.10[3]
98	0x82	PLI_RC_FILEBROWSER_RENAME	no	→ §2.4.10[3]
103	0x87	PLI_NC_NPCGET	no	→ §7[3]
104	0x88	PLI_NC_NPCDELETE	no	→ §7[3]
105	0x89	PLI_NC_NPCRESET	no	→ §7[3]
106	0x8a	PLI_NC_NPCSCRIPTGET	no	→ §7[3]
107	0x8b	PLI_NC_NPCWARP	no	→ §7[3]
108	0x8c	PLI_NC_NPCFLAGSGET	no	→ §7[3]
109	0x8d	PLI_NC_NPCSCRIPTSET	no	→ §7[3]
110	0x8e	PLI_NC_NPCFLAGSSET	no	→ §7[3]
111	0x8f	PLI_NC_NPCADD	no	→ §7[3]
112	0x90	PLI_NC_CLASSEDIT	no	→ §7[3]
113	0x91	PLI_NC_CLASSADD	no	→ §7[3]
114	0x92	PLI_NC_LOCALNPCSGET	no	→ §7[3]
115	0x93	PLI_NC_WEAPONLISTGET	no	→ §7[3]
116	0x94	PLI_NC_WEAPONGET	no	→ §7[3]
117	0x95	PLI_NC_WEAPONADD	no	→ §7[3]
118	0x96	PLI_NC_WEAPONDELETE	no	→ §7[3]
119	0x97	PLI_NC_CLASSDELETE	no	→ §7[3]
130	0xa2	PLI_REQUESTUPDATEBOARD	no	→ §2.4.7[3]
150	0xb6	PLI_NC_LEVELLISTGET	no	→ §7[3]
151	0xb7	PLI_NC_LEVELLISTSET	no	→ §7[3]
152	0xb8	PLI_REQUESTTEXT	no	→ §2.4.7[3]
154	0xba	PLI_SENDTEXT	no	→ §2.4.7[3]
155	0xbb	PLI_RC_LARGEFILESTART	no	→ §2.4.10[3]
156	0xbc	PLI_RC_LARGEFILEEND	no	→ §2.4.10[3]
157	0xbd	PLI_UPDATEGANI	no	→ §2.4.7[3]
158	0xbe	PLI_UPDATESCRIPT	no	→ §2.4.7[3]
159	0xbf	PLI_UPDATEPACKAGEREQUESTFILE	no	→ §2.4.7[3]
160	0xc0	PLI_RC_FOLDERDELETE	no	→ §2.4.10[3]
161	0xc1	PLI_UPDATECLASS	no	→ §2.4.7[3]
162	0xc2	PLI_RC_UNKNOWN162	no	→ §2.4.9[3]
252	n/a (above 223)	PLI_SET_ENC_KEY	no	→ §2.4.9[3]
253	n/a (above 223)	PLI_BUNDLE	no	→ §2.4.9[3]

A.2 Server to client

This is the index of every server-to-client id. The 2.05 column says whether 2.05 handles the id[3].

Id	Wire	Name	2.05	Section
0	0x20	PLO_LEVELBOARD	handled	→ §2.5.3[3]
1	0x21	PLO_LEVELLINK	handled	→ §2.5.3[3]
2	0x22	PLO_BADDYPROPS	handled	→ §2.5.3[3]
3	0x23	PLO_NPCPROPS	handled	→ §2.5.3[3]
4	0x24	PLO_LEVELCHEST	handled	→ §2.5.3[3]
5	0x25	PLO_LEVELSIGN	handled	→ §2.5.3[3]
6	0x26	PLO_LEVELNAME	handled	→ §2.5.3[3]
7	0x27	PLO_BOARDMODIFY	handled	→ §2.5.3[3]
8	0x28	PLO_OTHERPLPROPS	handled	→ §2.5.4[3]
9	0x29	PLO_PLAYERPROPS	handled	→ §2.5.4[3]
10	0x2a	PLO_ISLEADER	handled	→ §2.5.3[3]
11	0x2b	PLO_BOMBADD	handled	→ §2.5.5[3]
12	0x2c	PLO_BOMBDEL	handled	→ §2.5.5[3]
13	0x2d	PLO_TOALL	handled	→ §2.5.8[3]
14	0x2e	PLO_PLAYERWARP	handled	→ §2.5.4[3]
15	0x2f	PLO_WARPFAILED	handled	→ §2.5.4[3]
16	0x30	PLO_DISCMESSAGE	handled	→ §2.5.4[3]
17	0x31	PLO_HORSEADD	handled	→ §2.5.3[3]
18	0x32	PLO_HORSEDEL	handled	→ §2.5.3[3]
19	0x33	PLO_ARROWADD	handled	→ §2.5.5[3]
20	0x34	PLO_FIRESPY	handled	→ §2.5.5[3]
21	0x35	PLO_THROWCARRIED	handled	→ §2.5.5[3]
22	0x36	PLO_ITEMADD	handled	→ §2.5.3[3]
23	0x37	PLO_ITEMDEL	handled	→ §2.5.3[3]
24	0x38	PLO_NPCMOVED	handled	→ §2.5.6[3]
25	0x39	PLO_SIGNATURE	handled	→ §2.5.4[3]
26	0x3a	PLO_NPCACTION	no	→ §2.5.6[3]
27	0x3b	PLO_BADDYHURT	handled	→ §2.5.5[3]
28	0x3c	PLO_FLAGSET	handled	→ §2.5.4[3]
29	0x3d	PLO_NPCDEL	handled	→ §2.5.6[3]
30	0x3e	PLO_FILESENDFAILED	handled	→ §2.5.7[3]
31	0x3f	PLO_FLAGDEL	handled	→ §2.5.4[3]
32	0x40	PLO_SHOWIMGPLAYER	handled	→ §2.5.8[3]
33	0x41	PLO_NPCWEAPONADD	handled	→ §2.5.6[3]
34	0x42	PLO_NPCWEAPONDEL	handled	→ §2.5.6[3]
35	0x43	PLO_RC_ADMINMESSAGE	handled	→ §2.5.8[3]
36	0x44	PLO_EXPLOSION	handled	→ §2.5.5[3]
37	0x45	PLO_PRIVATEMESSAGE	handled	→ §2.5.8[3]
38	0x46	PLO_PUSHAWAY	handled	→ §2.5.4[3]
39	0x47	PLO_LEVELMODTIME	handled	→ §2.5.3[3]
40	0x48	PLO_HURTPLAYER	handled	→ §2.5.4[3]
41	0x49	PLO_STARTMESSAGE	handled	→ §2.5.8[3]
42	0x4a	PLO_NEWWORLDTIME	handled	→ §2.5.3[3]
43	0x4b	PLO_DEFAULTWEAPON	handled	→ §2.5.6[3]
44	0x4c	PLO_HASNPCSERVER	handled	→ §2.5.6[3]
45	0x4d	PLO_FILEUPTODATE	handled	→ §2.5.7[3]
46	0x4e	PLO_HITOBJECTS	handled	→ §2.5.5[3]

Id	Wire	Name	2.05	Section
47	0x4f	PLO_STAFFGUILDS	handled	→ §2.5.8[3]
48	0x50	PLO_TRIGGERACTION	handled	→ §2.5.5[3]
49	0x51	PLO_PLAYERWARP2	no	→ §2.5.4[3]
50	0x52	PLO_RC_ACCOUNTADD	no	→ §7[3]
51	0x53	PLO_RC_ACCOUNTSTATUS	no	→ §7[3]
52	0x54	PLO_RC_ACCOUNTNAME	no	→ §7[3]
53	0x55	PLO_RC_ACCOUNTDEL	no	→ §7[3]
54	0x56	PLO_RC_ACCOUNTPROPS	no	→ §7[3]
55	0x57	PLO_ADDPLAYER	no	→ §2.5.4[3]
56	0x58	PLO_DELPLAYER	no	→ §2.5.4[3]
57	0x59	PLO_RC_ACCOUNTPROPSGET	no	→ §7[3]
58	0x5a	PLO_RC_ACCOUNTCHANGE	no	→ §7[3]
59	0x5b	PLO_RC_PLAYERPROPSCHANGE	no	→ §7[3]
60	0x5c	PLO_UNKNOWN60	no	→ §2.5.10[3]
61	0x5d	PLO_RC_SERVERFLAGSGET	no	→ §7[3]
62	0x5e	PLO_RC_PLAYERRIGHTSGET	no	→ §7[3]
63	0x5f	PLO_RC_PLAYERCOMMENTSGET	no	→ §7[3]
64	0x60	PLO_RC_PLAYERBANGET	no	→ §7[3]
65	0x61	PLO_RC_FILEBROWSER_DIRLIST	no	→ §7[3]
66	0x62	PLO_RC_FILEBROWSER_DIR	no	→ §7[3]
67	0x63	PLO_RC_FILEBROWSER_MESSAGE	no	→ §2.5.9[3]
68	0x64	PLO_LARGEFILESTART	no	→ §2.5.7[3]
69	0x65	PLO_LARGEFILEEND	no	→ §2.5.7[3]
70	0x66	PLO_RC_ACCOUNTLISTGET	no	→ §7[3]
71	0x67	PLO_RC_PLAYERPROPS	no	→ §7[3]
72	0x68	PLO_RC_PLAYERPROPSGET	no	→ §7[3]
73	0x69	PLO_RC_ACCOUNTGET	no	→ §7[3]
74	0x6a	PLO_RC_CHAT	no	→ §2.5.9[3]
75	0x6b	PLO_PROFILE	handled	→ §2.5.8[3]
76	0x6c	PLO_RC_SERVEROPTIONSGET	no	→ §7[3]
77	0x6d	PLO_RC_FOLDERCONFIGGET	no	→ §7[3]
78	0x6e	PLO_NC_CONTROL	no	→ §7[3]
79	0x6f	PLO_NPCSERVERADDR	no	→ §2.5.9[3]
80	0x70	PLO_NC_LEVELLIST	no	→ §7[3]
81	0x71	PLO_UNKNOWN81	no	→ §2.5.10[3]
82	0x72	PLO_SERVERTEXT	no	→ §2.5.8[3]
83	0x73	PLO_UNKNOWN83	no	→ §2.5.10[3]
84	0x74	PLO_LARGEFILESIZE	no	→ §2.5.7[3]
100	0x84	PLO_RAWDATA	handled	→ §2.5.3[3]
103	0x87	PLO_RC_MAXUPLOADFILESIZE	no	→ §7[3]
104	0x88	PLO_UNKNOWN104	no	→ §2.5.8[3]
105	0x89	PLO_UPDATEPACKAGESIZE	no	→ §2.5.7[3]
106	0x8a	PLO_UPDATEPACKAGEDONE	no	→ §2.5.7[3]
107	0x8b	PLO_BOARDLAYER	no	→ §2.5.3[3]
108	0x8c	PLO_UNKNOWN108	no	→ §2.5.8[3]
109	0x8d	PLO_UNKNOWN109	no	→ §2.5.7[3]
110	0x8e	PLO_UNKNOWN110	no	→ §2.5.10[3]

Id	Wire	Name	2.05	Section
111	0x8f	PLO_SETNETCOOKIE	no	→ §2.5.7[3]
124	0x9c	PLO_UNKNOWN124	no	→ §2.5.10[3]
131	0xa3	PLO_NPCBYTECODE	no	→ §2.5.6[3]
132	0xa4	PLO_UNKNOWN132	no	→ §2.5.10[3]
133	0xa5	PLO_UNKNOWN133	no	→ §2.5.6[3]
134	0xa6	PLO_GANISCRIPT	no	→ §2.5.6[3]
140	0xac	PLO_NPCWEAPONSCRIPT	no	→ §2.5.6[3]
150	0xb6	PLO_NPCDEL2	handled	→ §2.5.3[3]
151	0xb7	PLO_HIDENPCS	handled	→ §2.5.3[3]
153	0xb9	PLO_SAY2	handled	→ §2.5.8[3]
154	0xba	PLO_FREEZEPLAYER2	handled	→ §2.5.4[3]
155	0xbb	PLO_UNFREEZEPLAYER	handled	→ §2.5.4[3]
156	0xbc	PLO_SETACTIVELEVEL	handled	→ §2.5.3[3]
157	0xbd	PLO_NC_NPCATTRIBUTES	no	→ §7[3]
158	0xbe	PLO_NC_NPCADD	no	→ §7[3]
159	0xbf	PLO_NC_NPCDELETE	no	→ §7[3]
160	0xc0	PLO_NC_NPCSCRIPT	no	→ §7[3]
161	0xc1	PLO_NC_NPCFLAGS	no	→ §7[3]
162	0xc2	PLO_NC_CLASSGET	no	→ §7[3]
163	0xc3	PLO_NC_CLASSADD	no	→ §7[3]
164	0xc4	PLO_NC_LEVELDUMP	no	→ §7[3]
165	0xc5	PLO_MOVE	handled	→ §2.5.6[3]
166	0xc6	PLO_SHOWIMGNPC	handled	→ §2.5.6[3]
167	0xc7	PLO_NC_WEAPONLISTGET	no	→ §7[3]
168	0xc8	PLO_UNKNOWN168	handled	→ §2.5.4[3]
169	0xc9	PLO_UPGRADEURL	no	→ §2.5.8[3]
170	0xca	PLO_GHOSTMODE	no	→ §2.5.4[3]
171	0xcb	PLO_BIGMAP	no	→ §2.5.8[3]
172	0xcc	PLO_MINIMAP	no	→ §2.5.8[3]
173	0xcd	PLO_GHOSTTEXT	no	→ §2.5.4[3]
174	0xce	PLO_GHOSTICON	no	→ §2.5.4[3]
175	0xcf	PLO_SHOOT	no	→ §2.5.5[3]
176	0xd0	PLO_KINGDOMSMODE	no	→ §2.5.8[3]
177	0xd1	PLO_NEWWORLDMODE	no	→ §2.5.8[3]
178	0xd2	PLO_SERVERWARP	no	→ §2.5.8[3]
179	0xd3	PLO_RPGWINDOW	no	→ §2.5.8[3]
180	0xd4	PLO_STATUSLIST	no	→ §2.5.8[3]
181	0xd5	PLO_UNKNOWN181	no	→ §2.5.4[3]
182	0xd6	PLO_LISTPROCESSES	no	→ §2.5.8[3]
183	0xd7	PLO_HASPROCESSRUNNING	no	→ §2.5.8[3]
184	0xd8	PLO_TAKESCREENSHOT	no	→ §2.5.8[3]
185	0xd9	PLO_BOARDHEIGHTS	no	→ §2.5.3[3]
186	0xda	PLO_BOARDMODIFY2	no	→ §2.5.3[3]
187	0xdb	PLO_UPDATEPACKAGEISUPDATED	no	→ §2.5.7[3]
188	0xdc	PLO_NC_CLASSDELETE	no	→ §7[3]
189	0xdd	PLO_MOVE2	no	→ §2.5.6[3]
190	0xde	PLO_SERVERLISTCONNECTED	no	→ §2.5.8[3]

Id	Wire	Name	2.05	Section
191	0xdf	PLO_SHOOT2	no	→ §2.5.5[3]
192	0xe0	PLO_NC_WEAPONGET	no	→ §7[3]
193	0xe1	PLO_UNKNOWN193	no	→ §2.5.6[3]
194	0xe2	PLO_CLEARWEAPONS	no	→ §2.5.6[3]
195	0xe3	PLO_LOADGANI	no	→ §2.5.6[3]
197	0xe5	PLO_LOADSCRIPT	no	→ §2.5.6[3]
198	0xe6	PLO_SERVEROPTIONS	no	→ §2.5.7[3]
252	n/a (above 223)	PLO_SET_ENC_KEY	no	→ §2.5.10[3]
253	n/a (above 223)	PLO_BUNDLE	no	→ §2.5.10[3]

A.3 Former names

An older gs2lib copy (the GServer-v2 submodule of 2025-01) uses these names for the same ids:

Former name	Id	Current name
PLI_SHOWIMG	24	PLI_SHOWIMGPLAYER[3]
PLI_MAPINFO	39	PLI_TAMPERCHECK[3]
PLI_UNKNOWN46	46	PLI_ENTERLEVEL[3]
PLO_SHOWIMG	32	PLO_SHOWIMGPLAYER[3]
PLO_FULLSTOP	176	PLO_KINGDOMSMODE[3]
PLO_FULLSTOP2	177	PLO_NEWWORLDMODE[3]
PLO_UNKNOWN183	183	PLO_HASPROCESSRUNNING[3]
PLO_UNKNOWN184	184	PLO_TAKESCREENSHOT[3]
PLO_UNKNOWN185	185	PLO_BOARDHEIGHTS[3]
PLO_UNKNOWN186	186	PLO_BOARDMODIFY2[3]
PLO_UNKNOWN190	190	PLO_SERVERLISTCONNECTED[3]

Some ids have had more than one name. The rules below record the name changes between copies of the server library[3].

Rule
The newer gs2lib copy adds PLI_PROCESSRUNNING (45), PLI_SET_ENC_KEY (252), PLI_BUNDLE (253), PLO_SETNETCOOKIE (111) and PLO_UPGRADEURL (169)[3]
An older note names 100, 101 and 102 FILELEN, TILES and FILEBLOB; the roles are those of PLO_RAWDATA, PLO_BOARDPACKET and PLO_FILE[3]
Provisional ids from early notes (login 0x01, level request 0x10, player update 0x20) are retired: the login is type 0, PLI_LEVELWARP is 0 and PLI_PLAYERPROPS is 2[3]

Appendix B. Prop index

Every player (PLPROP), NPC (NPCPROP) and baddy (BDPROP) prop id with its name and the section that gives its encoding. Prop streams are defined in → §2.6.1.

B.1 Player, NPC and baddy props

This is the index of every prop id for players, NPCs and baddies. Each row gives the kind of stream, the id, its name and the section that describes its encoding[71].

Kind	Id	Name	Section
PLPROP	0	NICKNAME	→ §2.6.2[71]
PLPROP	1	MAXPOWER	→ §2.6.2[71]
PLPROP	2	CURPOWER	→ §2.6.2[71]
PLPROP	3	RUPEESCOUNT	→ §2.6.2[71]
PLPROP	4	ARROWSCOUNT	→ §2.6.2[71]
PLPROP	5	BOMBSCOUNT	→ §2.6.2[71]
PLPROP	6	GLOVEPOWER	→ §2.6.2[71]
PLPROP	7	BOMBPOWER	→ §2.6.2[71]
PLPROP	8	SWORDPOWER	→ §2.6.2[71]
PLPROP	9	SHIELDPOWER	→ §2.6.2[71]
PLPROP	10	GANI / BOWGIF	→ §2.6.2[71]
PLPROP	11	HEADGIF	→ §2.6.2[71]
PLPROP	12	CURCHAT	→ §2.6.2[71]
PLPROP	13	COLORS	→ §2.6.2[71]
PLPROP	14	ID	→ §2.6.2[71]
PLPROP	15	X	→ §2.6.2[71]
PLPROP	16	Y	→ §2.6.2[71]
PLPROP	17	SPRITE	→ §2.6.2[71]
PLPROP	18	STATUS	→ §2.6.2[71]
PLPROP	19	CARRYSPRITE	→ §2.6.2[71]
PLPROP	20	CURLEVEL	→ §2.6.2[71]
PLPROP	21	HORSEGIF	→ §2.6.2[71]
PLPROP	22	HORSEBUSHES	→ §2.6.2[71]
PLPROP	23	EFFECTCOLORS	→ §2.6.2[71]
PLPROP	24	CARRYNPC	→ §2.6.2[71]
PLPROP	25	APCOUNTER	→ §2.6.2[71]
PLPROP	26	MAGICPOINTS	→ §2.6.2[71]
PLPROP	27	KILLSCOUNT	→ §2.6.2[71]
PLPROP	28	DEATHSCOUNT	→ §2.6.2[71]
PLPROP	29	ONLINESECS	→ §2.6.2[71]
PLPROP	30	IPADDR	→ §2.6.2[71]
PLPROP	31	UDPPORT	→ §2.6.2[71]
PLPROP	32	ALIGNMENT	→ §2.6.2[71]
PLPROP	33	ADDITFLAGS	→ §2.6.2[71]
PLPROP	34	ACCOUNTNAME	→ §2.6.2[71]
PLPROP	35	BODYIMG	→ §2.6.2[71]
PLPROP	36	RATING	→ §2.6.2[71]
PLPROP	37	GATTRIB1	→ §2.6.2[71]

Kind	Id	Name	Section
PLPROP	38	GATTRIB2	→ §2.6.2[71]
PLPROP	39	GATTRIB3	→ §2.6.2[71]
PLPROP	40	GATTRIB4	→ §2.6.2[71]
PLPROP	41	GATTRIB5	→ §2.6.2[71]
PLPROP	42	ATTACHNPC	→ §2.6.2[71]
PLPROP	43	GMAPLEVELX	→ §2.6.2[71]
PLPROP	44	GMAPLEVELY	→ §2.6.2[71]
PLPROP	45	Z	→ §2.6.2[71]
PLPROP	46	GATTRIB6	→ §2.6.2[71]
PLPROP	47	GATTRIB7	→ §2.6.2[71]
PLPROP	48	GATTRIB8	→ §2.6.2[71]
PLPROP	49	GATTRIB9	→ §2.6.2[71]
PLPROP	50	JOINLEAVELVL	→ §2.6.2[71]
PLPROP	51	PCONNECTED	→ §2.6.2[71]
PLPROP	52	PLANGUAGE	→ §2.6.2[71]
PLPROP	53	PSTATUSMSG	→ §2.6.2[71]
PLPROP	54	GATTRIB10	→ §2.6.2[71]
PLPROP	55	GATTRIB11	→ §2.6.2[71]
PLPROP	56	GATTRIB12	→ §2.6.2[71]
PLPROP	57	GATTRIB13	→ §2.6.2[71]
PLPROP	58	GATTRIB14	→ §2.6.2[71]
PLPROP	59	GATTRIB15	→ §2.6.2[71]
PLPROP	60	GATTRIB16	→ §2.6.2[71]
PLPROP	61	GATTRIB17	→ §2.6.2[71]
PLPROP	62	GATTRIB18	→ §2.6.2[71]
PLPROP	63	GATTRIB19	→ §2.6.2[71]
PLPROP	64	GATTRIB20	→ §2.6.2[71]
PLPROP	65	GATTRIB21	→ §2.6.2[71]
PLPROP	66	GATTRIB22	→ §2.6.2[71]
PLPROP	67	GATTRIB23	→ §2.6.2[71]
PLPROP	68	GATTRIB24	→ §2.6.2[71]
PLPROP	69	GATTRIB25	→ §2.6.2[71]
PLPROP	70	GATTRIB26	→ §2.6.2[71]
PLPROP	71	GATTRIB27	→ §2.6.2[71]
PLPROP	72	GATTRIB28	→ §2.6.2[71]
PLPROP	73	GATTRIB29	→ §2.6.2[71]
PLPROP	74	GATTRIB30	→ §2.6.2[71]
PLPROP	75	OSTYPE	→ §2.6.2[71]
PLPROP	76	TEXTCODEPAGE	→ §2.6.2[71]
PLPROP	77	UNKNOWN77	→ §2.6.2[71]
PLPROP	78	X2	→ §2.6.2[71]
PLPROP	79	Y2	→ §2.6.2[71]
PLPROP	80	Z2	→ §2.6.2[71]
PLPROP	81	UNKNOWN81	→ §2.6.2[71]
PLPROP	82	COMMUNITYNAME	→ §2.6.2[71]
NPCPROP	0	IMAGE	→ §2.6.4[71]
NPCPROP	1	SCRIPT	→ §2.6.4[71]

Kind	Id	Name	Section
NPCPROP	2	X	→ §2.6.4[71]
NPCPROP	3	Y	→ §2.6.4[71]
NPCPROP	4	POWER	→ §2.6.4[71]
NPCPROP	5	RUPEES	→ §2.6.4[71]
NPCPROP	6	ARROWS	→ §2.6.4[71]
NPCPROP	7	BOMBS	→ §2.6.4[71]
NPCPROP	8	GLOVEPOWER	→ §2.6.4[71]
NPCPROP	9	BOMBPOWER	→ §2.6.4[71]
NPCPROP	10	SWORDIMAGE	→ §2.6.4[71]
NPCPROP	11	SHIELDIMAGE	→ §2.6.4[71]
NPCPROP	12	GANI / BOWGIF	→ §2.6.4[71]
NPCPROP	13	VISFLAGS	→ §2.6.4[71]
NPCPROP	14	BLOCKFLAGS	→ §2.6.4[71]
NPCPROP	15	MESSAGE	→ §2.6.4[71]
NPCPROP	16	HURTDXDY	→ §2.6.4[71]
NPCPROP	17	ID	→ §2.6.4[71]
NPCPROP	18	SPRITE	→ §2.6.4[71]
NPCPROP	19	COLORS	→ §2.6.4[71]
NPCPROP	20	NICKNAME	→ §2.6.4[71]
NPCPROP	21	HORSEIMAGE	→ §2.6.4[71]
NPCPROP	22	HEADIMAGE	→ §2.6.4[71]
NPCPROP	23	SAVE0	→ §2.6.4[71]
NPCPROP	24	SAVE1	→ §2.6.4[71]
NPCPROP	25	SAVE2	→ §2.6.4[71]
NPCPROP	26	SAVE3	→ §2.6.4[71]
NPCPROP	27	SAVE4	→ §2.6.4[71]
NPCPROP	28	SAVE5	→ §2.6.4[71]
NPCPROP	29	SAVE6	→ §2.6.4[71]
NPCPROP	30	SAVE7	→ §2.6.4[71]
NPCPROP	31	SAVE8	→ §2.6.4[71]
NPCPROP	32	SAVE9	→ §2.6.4[71]
NPCPROP	33	ALIGNMENT	→ §2.6.4[71]
NPCPROP	34	IMAGEPART	→ §2.6.4[71]
NPCPROP	35	BODYIMAGE	→ §2.6.4[71]
NPCPROP	36	GATTRIB1	→ §2.6.4[71]
NPCPROP	37	GATTRIB2	→ §2.6.4[71]
NPCPROP	38	GATTRIB3	→ §2.6.4[71]
NPCPROP	39	GATTRIB4	→ §2.6.4[71]
NPCPROP	40	GATTRIB5	→ §2.6.4[71]
NPCPROP	41	GMAPLEVELX	→ §2.6.4[71]
NPCPROP	42	GMAPLEVELY	→ §2.6.4[71]
NPCPROP	43	Z	→ §2.6.4[71]
NPCPROP	44	GATTRIB6	→ §2.6.4[71]
NPCPROP	45	GATTRIB7	→ §2.6.4[71]
NPCPROP	46	GATTRIB8	→ §2.6.4[71]
NPCPROP	47	GATTRIB9	→ §2.6.4[71]
NPCPROP	48	UNKNOWN48	→ §2.6.4[71]

Kind	Id	Name	Section
NPCPROP	49	SCRIPTER	→ §2.6.4[71]
NPCPROP	50	NAME	→ §2.6.4[71]
NPCPROP	51	TYPE	→ §2.6.4[71]
NPCPROP	52	CURLEVEL	→ §2.6.4[71]
NPCPROP	53	GATTRIB10	→ §2.6.4[71]
NPCPROP	54	GATTRIB11	→ §2.6.4[71]
NPCPROP	55	GATTRIB12	→ §2.6.4[71]
NPCPROP	56	GATTRIB13	→ §2.6.4[71]
NPCPROP	57	GATTRIB14	→ §2.6.4[71]
NPCPROP	58	GATTRIB15	→ §2.6.4[71]
NPCPROP	59	GATTRIB16	→ §2.6.4[71]
NPCPROP	60	GATTRIB17	→ §2.6.4[71]
NPCPROP	61	GATTRIB18	→ §2.6.4[71]
NPCPROP	62	GATTRIB19	→ §2.6.4[71]
NPCPROP	63	GATTRIB20	→ §2.6.4[71]
NPCPROP	64	GATTRIB21	→ §2.6.4[71]
NPCPROP	65	GATTRIB22	→ §2.6.4[71]
NPCPROP	66	GATTRIB23	→ §2.6.4[71]
NPCPROP	67	GATTRIB24	→ §2.6.4[71]
NPCPROP	68	GATTRIB25	→ §2.6.4[71]
NPCPROP	69	GATTRIB26	→ §2.6.4[71]
NPCPROP	70	GATTRIB27	→ §2.6.4[71]
NPCPROP	71	GATTRIB28	→ §2.6.4[71]
NPCPROP	72	GATTRIB29	→ §2.6.4[71]
NPCPROP	73	GATTRIB30	→ §2.6.4[71]
NPCPROP	74	CLASS	→ §2.6.4[71]
NPCPROP	75	X2	→ §2.6.4[71]
NPCPROP	76	Y2	→ §2.6.4[71]
NPCPROP	77	Z2	→ §2.6.4[71]
BDPROP	0	ID	→ §2.6.5[71]
BDPROP	1	X	→ §2.6.5[71]
BDPROP	2	Y	→ §2.6.5[71]
BDPROP	3	TYPE	→ §2.6.5[71]
BDPROP	4	POWERIMAGE	→ §2.6.5[71]
BDPROP	5	MODE	→ §2.6.5[71]
BDPROP	6	ANI	→ §2.6.5[71]
BDPROP	7	DIR	→ §2.6.5[71]
BDPROP	8	VERSESIGHT	→ §2.6.5[71]
BDPROP	9	VERSEHURT	→ §2.6.5[71]
BDPROP	10	VERSEATTACK	→ §2.6.5[71]

Appendix C. Version matrix

Build and era names are defined in → §1.1 and → §1.6. Each cell points back to the canonical section; behaviour rules by build (timing, movement, combat, death) are tabulated in → §4.1 to → §4.4 and are not repeated here.

C.1 Wire and scripting by era

Each row is one era of §1.6. Across it are the era's token, cipher generation, message ids, parsed props and scripting, so two eras can be compared at a glance[10].

Era	Login token	Cipher gen	Message ids handled or sent	Props parsed	Scripting
Z	none	text lines (→ §2.3.1)	text lobby lines	n/a	none[10]
A	none	1 (no zlib)	relay lobby `N`, `P`, `C`, `D` lines; `D` payload types per build	relay `D` types	GS1-like; `this.` variables from 1.1 beta1[10,52]
B	`GSERV007` to `GSERV018`	2 (zlib, no cipher)	GSERV id ranges (→ §2.3.2)	unmeasured	GS1; NPC weapons from 1.21[10]
3D0	`GR3DV002`	own (2D codec, no key byte)	own renumbered ids (→ §9)	3D floats, 24-bit colours	none[10]
C	`GNW20020` to `GNW07080`	2	1.33: S→C 0–41, 100; C→S 0–31; 45 and 35 ids by 1.39	own 0–33/34, others 21–23 ids	GS1, 88 to 111 commands[10,14]
D	`GNW13110`	3	1.41 additions (→ C.2)	props to 36	GS1, 126 commands[10]
E	`GNW08011` to `GNW03081`; NewWorld `G2N15041`	3	2.05: 59 S→C ids	to 42 (2.05)	GS1, 134 to 161 commands; ganis from 1.42[10]
F	`GNW31101` (2.1) to `GNW22122` (2.17)	3	2.17: 79 S→C, 61 C→S; TAMPERCHECK (39) from 2.17	to 74 (gani attributes from 2.16)	GS1 and gani scripts (2.1); `this.` client strings (2.19)[10]
G	`GNW03014`, `GNW14015`, `GNW28015`; listserver `GNW30123`	5 (compression byte, partial XOR, bz2)	2.22: 80; 2.31: 81	to 74 (2.22); 80 with X2/Y2/Z2 (2.31)	GS1; classic client-side NPC mode removed in 2.3[10]
H	`G3D03014`, `G3D28015`; listserver `G3D30123`	5	79 (3.04); 88 (3.153)	as G	GS1 native, TorqueScript `.dso` for menus; NPC bytecode ids (3.153)[10]
I	`G3D17026`, `G3D27076`, `G3D20126`	5	98 to 101	unmeasured	GS2 bytecode; server-pushed ganis (195); update packages `GRPKG001`[10]
J	`G3D22067`, `G3D14097`, `G3D15010`, `G3D26090`	5	95 to 100	unmeasured	GS2; `GRESCR01` script cache; anti-cheat process scan (182, 183)[10]
K	`G3D0108A`, `G3D0511C` (Linux), `G3D0311C` (Windows 6.03); 6.11 set by script	5; 6.11 can switch to a script-defined header with RC4, AES or TLS	91 to 97	unmeasured	GS2; in 6.11 the packet id tables come from the server script (→ §2.10)[10]

Login-time extras a client must send: `PLI_LANGUAGE` (37) from 2.02 rev2[10]; `PLI_TAMPERCHECK` (39) once per session in 2.17, on gmap crossings and level links in 2.22, and every 3.2 s in 2.31[10]; machine-id MD5s after the password from 4.x on[10]. Server discovery by era is in → §6.

C.2 Server-to-client ids handled, 1.33 to 2.1 tests

Ids are logical (→ §0.7) and count only ids that reach a real handler.

Build	Change
1.33 to 1.37	43 ids: 0–41, 100[11]
1.38	+ 43[11]
1.39	+ 75 (45 ids: 0–41, 43, 75, 100)[11]
1.41 rev1	same set as 1.39[11]
NewMain 1.42	+ 42, 44, 45, 150, 151, 153[11]
2.0	+ 154, 155[11]
2.01	+ 46, 156[11]
2.02 rev2	+ 47[11]
2.03	+ 48, 165[11]
2.04	+ 166; − 26[11]
2.05 rev1, rev2	+ 168[11]
NewWorld betas	2.05 minus 2, 4, 17, 18, 154, 155[11]
2.1 tests	2.05 plus 49, 169[11]

Later builds: → §2.5 and the per-build tables in `version-profiles/plo-handled-ids-by-build.tsv`, which App. A indexes.

C.3 Prop parsers

Each prop parser grew over time. Each row gives a parser, the ids 1.33 accepts and the ids added later, with the build that added them[11].

Parser	1.33	Added later
player props	0–33	34 (1.39); 35, 36 (1.41); 37–41 (2.02 rev2); 42 (2.04); 43–45 (2.1 tests)[11]
NPC props	0–34	35 (1.41); 36–40 (2.02 rev2); 41–43 (2.1 tests)[11]
baddy props	0–10	none; the parser is gone in both NewWorld betas[11]
sparse player-prop sub-parser	21 ids up to 51	23 ids (1.39); 31 ids (2.05); role unconfirmed, likely props of other players[11]

The 1.33 player/NPC split is assumed from 1.34, where the player parser is the one that calls the sparse sub-parser[11]. Prop encodings: → §2.6.

C.4 UDP ids handled

Each row gives a build and the UDP ids it handles[11].

Build	Ids
1.33 to 1.37	0, 10–23[11]
1.38	+ 24[11]
2.01	+ 25[11]
2.03	+ 26[11]
2.1 tests	+ 27[11]
NewWorld betas	drop 16, 17[11]

UDP messages: → §2.9.

C.5 GS1 command table size

The GS1 command table grew build by build. Each row gives a build, the size of its table and the commands added[11].

Build	Commands	Added
1.33	88	—[11]
1.34	93	`dontblocklocal`, `blockagainlocal`, `showlocal`, `hidelocal`, `carryobject`, `throwcarry`, `setbacktile`; `seteffect` and `unseteffect` leave the table[11]
1.35	93	same table as 1.34[11]
1.36	104	`lay2`, `take2`, `takehorse`, `takeplayerhorse`, `removebomb`, `removearrow`, `removeitem`, `removeexplo`, `removehorse`, `explodebomb`, `reflectarrow`; `lay` takes arguments[11]
1.37	106	`playlooped`, `stopsound`[11]
1.38	109	`setlevel2`, `changeimgpart`, `hitnpc`; `setlevel` takes arguments[11]
1.39	111	`setani`, `setcharani`[11]
1.41 rev1	126	`canwarp`, `callnpc`, `say2`, `break`, `continue`, `callweapon`, `spyfire`, `function`, `return`, `changeimgvis`, `setimg`, `setimgpart`, `drawovertrees`, `putexplosion2`, `showstats`; `putexplosion` takes arguments[11]
NewMain 1.42	134	`canwarp2`, `setshape`, `seteffect`, `setcoloreffect`, `setzoomeffect`, `drawaslight`, `changeimgcolors`, `changeimgzoom`[11]
2.0	142	`setgender`, `setchargender`, `join`, `freezeplayer2`, `unfreezeplayer`, `enabledefmovement`, `disabledefmovement`, `setmusicvolume`[11]
2.01	144	`openurl2`, `hitobjects`; `openurl` takes arguments[11]
2.02 rev2	146	`tokenize`, `tokenize2`[11]
2.03	148	`triggeraction`, `move`[11]
2.04	157	`addtiledef`, `addtiledef2`, `removetiledefs`, `replaceani`, `attachplayertoobj`, `detachplayer`, `setshape2`, `enableselectweapons`, `disableselectweapons`; `setshape` takes arguments[11]
2.05 rev1, rev2	161	`setbacktile2`, `setz`, `updateterrain`, `loadmap`; `setbacktile` takes arguments[11]
2.1 tests	161	same table as 2.05[11]
NewWorld beta 1	161	same table as 2.05[11]
NewWorld beta 2	165	`addstring`, `insertstring`, `removestring`, `deletestring`[11]

A command in the table may have no executor case and be parsed and ignored; new commands are inserted mid-table, so command indexes differ between builds[11]. The commands themselves: → §8.2.

C.6 Generation 2 builds, 1.21 to 1.32

The generation-2 builds are compared here on their own. Each row gives the server ids a build handles and the props it parses and sends[74].

Build	S→C ids handled	Player props parsed	Player props sent	NPC props	Baddy props
1.21	0–34 except 30, plus 100	0–24	0–24	0–17	0–10[74,74]
1.28 beta 2	0–38, plus 100	0–31	0–24, 26, 31	0–32	0–10[74,74]
1.3 test	0–39, plus 100	0–32	0–24, 26, 31, 32	0–34	0–10[74,74]
1.31	0–40, plus 100	0–32	0–24, 26, 31, 32	0–34	0–10[74,74]
1.32	0–41, plus 100	0–33	0–24, 26, 31–33	0–34	0–10[74,74]

Prop numbers grow at the end, so a decoder can use the later tables and stop at the build's last id[74]. The meanings of ids 35 to 41 in this generation are unmeasured[74].

Client-to-server ids sent by these builds are in → §2.4.2.

C.7 Lab runs of later real clients

Each run used an unmodified client against a GServer-v2 lab on a loopback-only network, with hosts redirected by name[32,38].

Builds	How it ran	What it settled
3.04 (Linux)	the original packed executable on Xvfb with software GL	the listserver login is one plain frame holding ids 7 and 1, answered in gen 5; the game login frame is plain zlib with 5, 37 and two 34 requests, and every later frame is gen 5, including a bz2 reply[32,32]
4.20r6, 3.153 (Mac)	Mac OS X 10.4 in an emulated PowerPC machine	4.20r6 sends the login with a four-field comma list `mac`, an empty MAC hash, a 32-hex hash, and an empty field; 3.153 walks with X2 and Y2[33,33]
5.00r7, 5.12, 5.24r4	Wine on Xvfb behind a gen-5 logging proxy	the server reached through the license file is treated as the Login server, where no characters and no HUD are drawn; stock gs2lib does not know the 5.24 token and serves it the pre-2.1 file layout[38,38]

The per-build profile files and lab tables behind these cells, with the build range each covers, are listed in → §1.8.8; the full build list is in → §1.8.

Appendix D. Glossary

Short definitions of the terms the manual uses, each with the section that defines it. A glossary entry adds no fact of its own. Where a term has several meanings in the notes, the entry lists the meanings and points to where each one applies.

> Status: draft.

Term	Meaning	Defined in
1st Graal ... 15th Graal	early 1998–1999 Delphi builds named by archive order	→ §1.1.1
`ADDITFLAGS`	a bit field in the player props	→ §2.6.2
admin level	the 0–3 rights value in an account record (Graal3D: No rights, Graalian Police, Trusted Admin, Administrator)	→ §5.4, → §9.2.4
allowed versions	the server's list of version tokens it lets log in	→ §5.4
archive name	the name of the archive a build was found in, such as `graal121`	→ §0.9
arrays.dat, arrays2.dat	client data files holding the tile allow lists and other tables	→ §3.5.3, → §3.5.4
arrival	the newx/newy text of a link; a number, `playerx`/`playery`, or one of those with an offset	→ §3.1.3, → App. E#601
baddy	a level opponent with a type and three verse lines; `BDPROP` props	→ §3.1, → §2.6.5
BDPROP	a baddy prop id	→ §2.6.5, → App. B
BigData events	the two Torque events that carry Graal text and scripts over the 3D connection	→ §10.5.3
`bigmap.txt`	the text world map of older builds and generators: one row per line of quoted level names	→ §3.2.5, → §11.3.4
board	one level's tile grid	→ §0.8, → §3.1
board modify	a message that changes a rectangle of board tiles	→ §2.4.5, → §2.5.3
build	one client executable by release name and revision, such as `2.05 rev 2`	→ §0.9, → §1.1
build tag	an evidence tag naming the build a claim was read from, such as `[2.05]`	→ App. G.2
`CHR:1.00`	the one-line character file of the Character Designer and GChar	→ §11.7
class table	the list of class names and ids in a Torque connect accept, a Graal change from 3.153 on	→ §10.3.3
client type	the login type value that selects a server code path (0, 4, 5, 6)	→ §0.9, → §2.3
control-NPC	an `editnpc*.txt` script that drives the built-in editor	→ §11.6.1
DSO	compiled TorqueScript, file version 33 in the 3.x clients	→ §10.4
encryption generation	the transport scheme 1–6, in the GServer-v2 numbering	→ §0.9, → §2.2.1
era	a heading aid grouping builds; never a scope for a fact	→ §0.9
frame	one transport unit carrying one or more messages	→ §0.7, → §2.1.4
GANI	the text animation format, `GANI0001` sprite mode and `GANI0002` movie mode	→ §3.3
`GameObject`	the 3D object class of the 4.20r6 and 5.12 clients	→ §10.5.2
GCHAR, GSHORT, GINT3–5, GSTRING	Graal's printable integer and string encodings, short names `c`, `s`, `i3`–`i5`, `str`	→ §0.6, → §2.1.1
`GDUNG001`	the DungeonGenerator options file	→ §11.3.3
gen 3 insertion	the per-message byte the generation-3 client inserts	→ §2.2.2
`GENERATED`	a gmap key whose meaning depends on the writer: a level name, a flag, or absent	→ §3.2.3
GLEVNW01	the text level format (`.nw`)	→ §3.1.3
gmap, GRMAP001	a world map naming a grid of level files	→ §3.2
GOBJSET01	the stamp-set format the editors use	→ §3.4
GR-V1	the binary `.graal` level family, 1.00 to 1.04	→ §3.1.2
`GR3DV002`	the version string of the February 2000 Graal3D client	→ §9.1.2

Term	Meaning	Defined in
Graal3D	either the February 2000 DirectX program or the Torque 3D side of 3.x–5.12; the manual says which	→ §9, → §10
GraalEditor	the standalone level editor from 2.3	→ §11.1
GraalShop	the GANI editor	→ §11.2
GS1	the original NPC scripting language, interpreted from text	→ §8.1
GS2	the later scripting language, compiled to bytecode	→ §8.3, → §8.4
GServer13	the 1999 game server	→ §5.1
GServer-v2	the open-source server used as `[srv]` reference behaviour	→ §5.3
guest login	the clients' built-in guest account	→ §0.10
GWEBL001	the encrypted web-level cache format	→ §3.8
half-tile	8 px; the unit of most 2D coordinate fields	→ §0.8
`HEIGHTS`	a 9 × 9 height block in a level file; distinct from a gmap's `HEIGHTMAP`	→ §3.1.2.7, → App. E#606
leader	the player picked to run level-local actions for a level	→ §2.9.3, → §9.1.3
level-format generation	the level file family and revision	→ §0.9, → §1.5
listserver	the directory server a client asks for the server list	→ §6
load only	an account flag: the session is not tied to the record and may be shared	→ §5.4, → §9.2.4
logical id	the decimal message id used in this manual; the wire byte is the id + `0x20`	→ §0.7
message	one id, its payload and, in most generations, a `\n` terminator	→ §0.7, → §2.1.2
mission phases	the Torque `MissionStartPhaseN` steps after a 3D connect	→ §10.3.6
NC, NPC Control	the staff client for an NPC server	→ §7.2, → §11.5
NewMain, NewWorld	the 2000–2001 builds of the New World line	→ §1.1
NPC server	the server-side host for NPC scripts	→ §7
NPCPROP	an NPC prop id	→ §2.6.4, → App. B
offline mode	single-player play without a server	→ §4.8
PLI	a client-to-server message, names `PLI_*`	→ §0.7, → §2.4, → App. A
PLO	a server-to-client message, names `PLO_*`	→ §0.7, → §2.5, → App. A
PLPROP	a player prop id	→ §2.6.2, → App. B
predefined NPC	a `defnpc*.txt` script with `#EDIT` parameters, placed from the editor palette	→ §11.6.1
prop stream	a run of prop id and value pairs in one message	→ §2.6.1
raw data	the escape that carries a binary payload behind `PLO_RAWDATA`	→ §2.1.3
RC, RemoteControl	the staff client for a game server	→ §7.1, → §11.5
server-defined protocol	the 6.11 scheme in which a connector script defines the transport	→ §2.10
sign alphabet	the code table used for sign text in binary levels	→ §3.6
stamp	a predefined tile object from a GOBJSET01 set	→ §3.4, → §11.6.4
tick	one step of a client's game loop; its length is per build	→ §0.8, → §4.1
tile	one board cell, 16 px on a side	→ §0.8
tile definition	a line `image,prefix,type,x,y` swapping the tile sheet for levels whose name starts with a prefix	→ §11.6.4
tile type	the per-tile value from the type table that decides walls and water	→ §3.5.2
TGE, Torque	GarageGames' Torque Game Engine, the base of the 3.x–5.12 clients	→ §10.1
UDP id	a message on the peer-to-peer UDP channel	→ §0.7, → §2.9.4
unmeasured	a table cell value meaning no build was measured	→ §0.4
version token	the login version string a build sends, such as `GNW03081`	→ §0.9, → §1.2
warp field	a link created from an editor selection	→ §11.6.4
wire byte	the byte on the wire for a GCHAR id, the logical id + `0x20`	→ §0.7
Zelda Online	the 1997–1998 Java precursor and its `.zelda` (Z3-V1) levels	→ §1.1.1, → §3.1.1

Appendix E. Open questions and disagreements

Each entry quotes the claims, names their sources, and says what would settle it. No entry is resolved by guess (→ §0.4). Entries are numbered in the order they were added; the number never changes.

> Status: draft. Seeded from each part as it was written, and from the 0.0.2 disagreements chapter and CONSOLIDATED.md (E#600–608). Id ranges: E#1–99 front matter, Part I and Part II; E#100s Part III; E#200s Part VIII; E#300s Part IV; E#400s Parts V–VII; E#500s Parts IX–X; E#600s Part XI and the seed.

E#1 Compiler of the early Delphi builds

Two clues about the compiler of the earliest builds point to different Delphi versions. The question stays open until their runtime library is matched against candidates[5].

Item	Content
Claim A	1st Graal, 3rd Graal to 1.1.9 and 1.21 use the Delphi 2 class-metadata layout (self pointer 64, class name 32 bytes before the VMT)[5]
Claim B	the same builds carry the RTL copyright "1983,97 Borland", a year late for Delphi 2[5]
Open	which compiler built them; a Delphi-2-level compiler shipped in 1997, such as C++Builder 1's, would fit both, but it has not been checked[5]
Settles it	matching the RTL of these builds against C++Builder 1 and Delphi 2 runtime libraries[5]

E#2 Generation-5 step count of 5.12

Two reads disagree on how many iterator steps 5.12 takes for a short compressed frame. Only a capture that holds such a frame can tell them apart[4].

Item	Content
Claim A	5.12's frame decoder steps the iterator `len / 4` times, rounded down, for zlib and bz2 frames (capped at 4), and rounds up for raw frames; it encodes the same way; a proxy that re-encodes with round-down stepping lets 5.12 play against GServer-v2[4]
Claim B	every later frame of ten 5.x and 6.x Windows captures, 5.12 among them, decodes with one step per group present, rounded up[8]
Open	whether the 5.12 captures held a compressed frame short enough (under 16 bytes, length not a multiple of 4) to tell the rules apart[4]
Settles it	decoding a 5.12 capture that contains such a frame under both rules[4]

Separately, four exact Mac PowerPC receivers use whole-group compressed advancement and partial-group raw advancement. This static Mac result does not resolve the Windows capture disagreement or prove outgoing symmetry.[82]

E#3 Public `Login` server session (5.x, 6.x)

Nobody has recorded what a public 5.x or 6.x `Login` server sends. Each row is one thing still unobserved; settling them needs the owner's permission[85].

Item	Content
Unobserved	the bytes a public 5.x or 6.x `Login` server sends, including the payload of S→C 190[85]
Unobserved	the GUI script text for the server list, the account screens and the `ChatBar` control[85]
Unobserved	the file list inside that server's `basepackage.gupd`, and any later update package[85]
Unobserved	the backdrop level that server chooses[85]
Unobserved	the 6.11 connector script body, and with it the framing, cipher and login token a 6.11 client uses[85]
Settles it	a capture of a real `Login` session, which needs the owner's written permission because it touches a public server[6]

E#4 Whether 5.00r7 sends id 11

Two lists of the ids 5.00r7 sends disagree on id 11. A later static read sides with the list that includes it, and a capture is still missing[37].

Item	Content
Claim A	the 5.00r7 send wrappers send 1, 2, 4–10, 12–19, …: no 11 (`PLI_THROWCARRIED`)[37]
Claim B	the 5.00r7 profile lists 4–19, which includes 11; 5.12, 5.24r4 and 5.30r3 were seen sending 11 in captures[59]
Open	which list is right for 5.00r7; both are floors, since sends through a variable are not captured[37]
Settles it	a 5.00r7 capture of a throw, or a re-read of the send wrappers for id 11[37]
Static resolution	the 5.00r7 backup engine's carried-object throw path sends id 11 with an empty body; the earlier wrapper list missed it. Claim B holds for that engine; a capture is still missing[202]

E#5 Meaning of id 162 from the client (settled)

Three readings of client id 162 turned out to be one message, a ping answer. The entry is kept to show how it was settled[3].

Item	Content
Claim A	gs2lib names 162 `PLI_RC_UNKNOWN162`: an empty message from the RC3 beta[3]
Claim B	4.20r8 sends 162 as the answer to S→C 108[34]
Claim C	6.01r1 and 6.03r7 send 162 as a reply to a server request: `!` followed by a 2-byte value (two 7-bit groups, capped at `0x6FFF`) the server sent; probably the ping answer that 6.11 calls `onSendPingAnswer`[39]
Open	~~whether B and C are the same message, and how the RC3 beta's empty 162 relates to them~~ settled below[39]
Settles it	a capture of a 4.x or 6.x client answering S→C 108[34]
Settled	one message, a ping: 4.20r6, 4.20r8, 6.01r1 and 6.03r7 send 162 only from their 108 handler, with the same code in 4.20r8 and 6.03r7; the 2010 RC and RC3 send it empty every 60 s and have no 108 handler; a capture of the game-client answer is still missing[102]

E#6 Client-to-server constants of 1.21 and 1.32

Two reads of the 1.21 and 1.32 send sites disagree on ids 14 and 22[74].

Item	Content
Claim A	the ids visible as constants are 0 and 10 to 26 in 1.21, and 0 and 10 to 30 in 1.32[74]
Claim B	the per-build profile lists 1.21 as 0, 10–13, 15–26 (no 14) and 1.32 as 0, 10–21, 23–30 (no 22)[53]
Open	whether A is a range summary of B or B missed a send site; both are floors, since some sends pass the id in a register[53]
Settles it	a re-read of the 1.21 and 1.32 send sites for ids 14 and 22[74]

E#7 Receive-branch revision discrepancy (settled for the checked inputs)

Two tables placed ids 47 and 168 in different revisions. A static check settled it for the revisions that were read; the scope row says how far that goes[11].

Item	Content
Historical claim A	the timeline formerly placed 47 at 2.02 rev2 and 168 at 2.05 rev1; those revision labels are corrected[11]
Historical claim B	the per-build table already included 47 in 2.02 revisions 0 and 1 and 168 in 2.05 revision 0[112,115]
Resolution	the exact-input static check corroborates Claim B for Windows x86 2.02 revisions 0–2 and 2.05 revisions 0–2; counts and ID sets are in → §2.5.2[111]
Scope limit	this settles route presence in those inputs; successful runtime handling, payload semantics and first occurrence across all historical branches are not established[111]

E#8 Prop parser ranges of 2.00

Two sources disagree on whether 2.00 parses player prop 36 and NPC prop 36[11].

Item	Content
Claim A	the cross-version timeline has the player-prop parser gain 35 and 36 in 1.41, and the NPC-prop parser gain 36 to 40 only in 2.02 rev2[11]
Claim B	the 2.00 layout tables have the self and other-player streams accept 0–35 (no 36) and the NPC stream accept 0–36[129,131]
Open	whether 2.00 parses player prop 36 and NPC prop 36[131]
Settles it	a re-read of the 2.00 prop switches at ids 36[129]

E#9 Uploads from game clients

One source says game clients never upload, while the late clients send the upload ids. Which feature sends them, and whether a server accepts them, is open[73].

Item	Content
Claim A	game clients cannot upload; the RC file-browser ids are for RemoteControl only[73]
Claim B	6.01r1 and 6.03r7 send the file-browser and upload ids (93, 96–98, 155, 156) with an upload queue, and 3.153 and 5.00r7 send several of them[39,31]
Open	whether a server accepts these from a game connection, and which game feature sends them[39]
Settles it	a capture of a 6.x game client using the upload path against a server[39]

E#10 When the `maps` download folder arrived

No surviving build shows when downloads began going to a `maps` folder[431].

Item	Content
Open	the 2.1 test builds save downloads with no `maps` rule and 2.17.1 has one; no client between them has been found, so the build that added it is unknown[431]
Settles it	the save routine of any client from 2.1 to 2.16, if one turns up[432]

E#11 Client-to-server ids the NewWorld betas lack

Two reads of the NewWorld betas disagree on whether they send ids 8 and 20[55].

Item	Content
Claim A	the betas send the 2.1 test set without 7, 8, 11, 15, 16, 17 and 20[55]
Claim B	the betas send 34 ids, the 2.1 test set without 7, 11, 15, 16 and 17 only[28]
Open	the two reads disagree on 8 (`HORSEDEL`) and 20 (`OPENCHEST`); Claim B's count of 34 is 39 less five[95]
Settles it	a re-read of the beta send sites for constants 8 and 20[28]

E#12 Whether the token digits are a build date

Reading the `GNW` digits as a build date fits well, but nothing in the binaries states it[5].

Item	Content
Claim A	the `GNW` digits are DDMMY, on or a few days before the link date[5]
Claim B	the dates fit the change-log order of 2.01 to 2.05, but nothing in the binary names the format[96]
Open	the reading is a strong fit, not a stated format[17]
Settles it	a vendor note or server source that decodes the token[96]

E#100 Collision of the Bush stamp tiles

Whether a placed bush blocks the player is a runtime question. File membership alone cannot answer it[174].

Item	Content
Established file result	the inspected first two arrays exclude Bush values 2, 3, 18 and 19; the earlier membership assertion is superseded within that collection[174]

Item	Content
Existing type-table evidence	the outside client's type bytes describe walls; they do not establish the selected official client's active collision path[175]
Open	actual blocking of a placed bush for an exact client, table and selected sheet[174]
Settles it	identify that exact consumer or record a controlled scene; file membership alone does not settle runtime collision[174]

E#101 Size of the type-0 allow lists

An earlier mask count of walkable tiles cannot yet be linked to the lists read from the exact file[174].

Item	Content
Established file result	the exact-input lengths, overlap and union are in → §3.5.3.1; they do not reproduce the earlier mask count[174]
Earlier mask	the export counted 3195 non-blocking and 682 blocking tiles[177]
Open	mask producer, coverage, mapping and selected sheet remain unlinked to the inspected input; no runtime walkability total follows[174]
Settles it	link the original mask and exact file to the producing client path and sheet before interpreting or comparing runtime categories[174]

E#102 Tile id to pixel position on pics1

Two formulas map a tile id to its place on `pics1.png`. The manual follows the later one, and a 4.04r2 code read agrees with it[175].

Item	Content
Claim A	`col = id % 128`, `row = id / 128`, pixel `(col * 16, row * 16)`[175]
Claim B	`col = (id % 16) + (id // 512) * 16`, `row = (id // 16) % 32`: eight side-by-side blocks of 512 ids, each 16 columns by 32 rows; Claim A is called wrong in a correction dated 2026-10-25[176]
Open	the manual follows Claim B, the later correction; Claim A's note has not been updated[176]
Settles it	correcting `formats/tiletypes.txt`, or rendering a known board with both rules against a client screenshot[176]
Windows 4.04r2 code	the tile draw path and its invalidation path both compute Claim B's formula before optional tiledef remapping[202]

E#200 When a GS1 condition counts as true

When a GS1 condition counts as true is now settled for one exact build. Each row gives that result, its exceptions and what stays open in other builds[312].

Item	Content
Exact-build resolution	checked original Windows 2.05 revision 2 IF uses zero/nonzero for ordinary finite computed results; computed positive and negative halves are true, computed zero false[312]
Parsing qualification	bare `0.5` and bare 1 are false despite numeric displays 0.5 and 1 (computed `(1+0)` is true); `created` is true despite numeric display zero. Condition-name resolution and numeric message expansion are separate[312]
Separate defect	constant numeric comparisons fold to zero in this build, while dynamic comparison controls work; mechanism and observations carried in §8.1.3[312]
Open	other builds, nonfinite/malformed values, complete aliases/precedence and named-flag collisions[312]
Linux positive follow-up	An independently authored source weapon on original Linux 3.04 produces created/playerenters flag messages and fresh-account persistence. Numeric readbacks and explicit positive comparisons work while tested bare predicates do not activate. This is a separate build scope from the Windows 2.05r2 resolution; no universal numeric threshold or complete predicate/name-lookup rule is established.[323]

E#201 Four GS1 argument signatures in 2.05

The stored argument letters of four GS1 commands were re-read. What they mean at run time is still open[324].

Item	Content
Static resolution	exact-input stored letters are recorded in → §8.2; the follow-up agrees with the earlier reference and corrects the command reference's alternatives[324]
Superseded claim	the command reference formerly printed `M,F,R,R,R,F,R,R` and `S,S` respectively; those are not the stored sequences for the four commands in the checked input[324]

Item	Content
Open	whether differing argument descriptions express equivalent runtime semantics; conversion and coercion remain untested[324]
Settles it	targeted parser/executor observations or controlled runtime tests of argument conversion in that exact build[324]

E#202 Which message carries GS2 bytecode

Three sources name different messages as the carrier of GS2 bytecode. The Linux rows add partial results without closing the question[3,316].

Item	Content
Claim A	NPC bytecode is `PLO_NPCBYTECODE`, id 131, sent to clients at or above `CLVER_4_0211` and wrapped in `PLO_RAWDATA`[3,316]
Claim B	for v4 and later clients the script prop is replaced by `NPCPROP_CLASS` plus id 197 (`PLO_UNKNOWN197`, "GS2 bytecode"); the weapon-list note also lists 197 as the bytecode that replaces the script for GS2[2,228]
Claim C	id 197 is `PLO_LOADSCRIPT`, a list `type,name,saveToDisk,desKey,crc32`[3]
Open	whether 197 carries bytecode, a script-load descriptor, or both in different server versions, and how it relates to 131[3,2]
Settles it	a capture of a 4.x or 5.x client receiving an NPC with a GS2 script from a lab server[316]
Linux 4.04r2 partial result	A static message-131 path strips a three-byte identifier and reaches the compiled-segment reader. No network fixture was delivered; lookup roles, the indirect initialization callback, messages 140/197 and cache validation remain unresolved. The 97-case switch count is not a complete handled-ID count (→ §2.5.12)[127]
Linux cross-version follow-up	Preserve tag normalization and exact-build case behavior without publishing an unproved delivery grammar. The broader question remains partial (→ §2.5.12)[128]
Linux positive follow-up	Three exact Linux 6.03r7 x64 probes establish class parsing, explicitly registered weapon instruction execution and later continuation at the saved next position. The proxy-origin bodies and corrected descriptor-length boundary are independently reviewed. This is not normal server compiler/writer validation, a script return-value oracle, exact sleep timing, cache-hit proof or rendered gameplay.[346]
Linux 6.x class cache	On Linux 6.03r7 x64 and 6.01r1 x86, 197 is a metadata list with the expected CRC and 140 carries the body; a header-only cache file can never be a class hit; a class miss sends 161 with the stored-body CRC. Other builds and server generations stay open (→ §3.8.9)[188]

E#203 The 2.05 `#` message-code set

The full set of `#` message codes in 2.05 is partly recovered. Each row gives a part that is settled and the fields that remain open[313].

Item	Content
Partial resolution	exact original 2.05r2 outer dispatch has 44 selector bytes / 53 family-expanded spellings, including preserved display codes and empty-output #7; supported contract carried in §8.1.11[313]
Runtime controls	thirty qualified outputs from three usable offline scenes support bounded expansion behavior; fourth scene diagnostic only[313]
Field identities	#a is the account name and #L the recorded level name; #E was a space in one context, writer unresolved; a positive pickup/drop control showed #6 as the carried NPC's lowercased image name, then empty[313]
Open	the #E writer, quoted scanners, exact numeric formatting and downstream renderer semantics; other builds and live remote selectors unqualified[313]
Boundary	accepted C5 storage read does not prove a sixth clothing color or alter the five-byte wire color property; property setters have a separate grammar[313]
Exact 2.05r2 #6 follow-up	carried-NPC image association independently linked to selected pickup/throw/clear paths; runtime, complete selectors/eligibility/lifetime and receiver identity remain unvalidated; other field identities remain open[313]
Exact 2.05r2 #p follow-up	shared parameter list is produced by dispatch splitting actor pending text, distinct from tokenize; callnpc/callweapon use empty-parameter queue wrappers. Only producer linkage is resolved; other E#203 fields remain open[322]

E#204 Member names of the GS1 object arrays

The member names of the GS1 object arrays are settled for one build. What writing to them does, and how they change across builds, is still open[326].

Item	Content
Checked-build resolution	explicit suffix records and first-success parser establish the seven families' name-to-ID mappings: 36 records, 35 distinct spellings; carried in §8.2[326]
Arrow defect	dx663 binds first; duplicate dx664 vertical getter is shadowed. No builtin dy spelling; observing unassigned dy0 does not establish general fallback or no vertical movement[326]
Open	writes/setter side effects, unknown-member fallback, scanner details, enum meanings, specialized getters/integrity consequences and cross-version changes[326]

E#205 Index of the hare baddy (settled)

The hare baddy's type index is settled: it is type 5, shared with the frog[309,310].

Item	Content
Known	2.05 ships `baddyhare.gif`, so a hare baddy type exists; the baddy name table has 10 names, 0–9[309,310]
Open	~~the hare's type index~~ settled below, see `client-internals/baddies-222.md`[309]
Settles it	reading the 2.05 baddy image selection by type[309]
Settled	type 5 (`frog`): the 2.22 client draws type 5 with `baddyhare.png` and plays `frog.wav` for it; the hare and the frog are one type[133]

E#206 `move` option bits in 2.05

The `move` command's queued consumer is read for one build. The caller side and some edge cases are still open[219].

Item	Content
Established	the exact Windows 2.05r2 queued consumer now supplies static replacement, progression, collision and option behavior → §8.2.9[219]
Open	caller frequency/catch-up, complete collision eligibility, unidentified object collection, later event execution, exceptional numeric inputs and bits outside the consumer[219]
Settles it	targeted exact-input caller/state tracing and controlled runtime observations for these residuals[219]

E#207 GS2 language and bytecode format

GS2 syntax, its library and its event model are the largest open question in the manual. Each row adds a result from one build without closing it[10].

Item	Content
Known	4.x and later clients run GS2 and receive it compiled; 3.153 already handles the NPC bytecode ids[10]
Known	4.2.0.8, 5.0.0.7, and unpacked 6.00 parse one four-segment container, do not share OpenGS1's layout, and run the switch in → §8.4.2 and → §8.3.3. The value stack caps at 10000. Opcode 9 stops past 10000. Opcode 10 stores the stack height and does not jump[329,315]
Open	GS2 syntax, the standard library and the event model. The container and the dispatcher for those builds are recorded. Still open: the opcode helpers the engine note did not open, which gameplay site fires each `script_event` name, and packet id 140's own framing[315,316]
Settles it	a lab note decoding a captured bytecode message from a 4.x client session, a reading of the built-in command table and the source compiler's grammar, and a note on the GS2 documentation in the archived websites[315,316]
Linux 4.04r2 event result	Two instrumented dispatches repeat top-level execution; queued delivery follows the running handler. This partially qualifies the event question, without closing joined-class or other lifecycle semantics (→ §8.3.4)[318]
Linux 4.04r2 continuation result	A positive-sleep snapshot preserves the next instruction through restore; elapsed timing, cancellation and replacement remain unmeasured (→ §8.3.4)[331]
Linux 4.04r2 catalog correction	Correct grouping gives 176 types and 332 globals, restoring 38 root-object declarations; binding, conversion and inherited behavior remain unmeasured (→ §8.3.5)[333]
Earlier Linux 4.04r2 VM boundary	Dispatch ends at 190 and slot 11 selects a common skip path; one insertion control completes unchanged. Static frame fields and helper semantics remain qualified (→ §8.4.4)[338]
Linux 4.04r2 embedded linkage	Fourteen structurally validated script candidates include one profile payload matched to live input before executor entry; this is not corpus-wide execution or historical server-content recovery (→ §8.4.7)[344]

Item	Content
Linux cross-version follow-up	Use whole-payload equality and round-trip controls without asserting feature removal or native semantic equivalence. The broader question remains partial (→ §8.4.7)[345]
Linux cross-version follow-up	Separate static selector classifications, executor-entry stops and three observed startup cached jumps. The broader question remains partial (→ §8.4.4)[339]
Linux cross-version follow-up	Document static state saving and first-match pending-record replacement without inventing wait/timeout API results. The broader question remains partial (→ §8.3.4)[332]
Linux cross-version follow-up	Explain serialized tags versus fused internal selectors while preserving native/validator error-handling differences. The broader question remains partial (→ §8.4.6)[337]
Windows 5.00 installed-input boundary	Named/enum delivery queues separately from service; limit-before-dedup and named lifecycle priority are statically bounded for a used/patched backup input. Gates, conversions, reentrancy and runtime timing remain open (→ §8.3.4)[319]
Linux positive follow-up	Three exact Linux 6.03r7 x64 probes establish class parsing, explicitly registered weapon instruction execution and later continuation at the saved next position. The proxy-origin bodies and corrected descriptor-length boundary are independently reviewed. This is not normal server compiler/writer validation, a script return-value oracle, exact sleep timing, cache-hit proof or rendered gameplay.[346]
Linux 6.01 x86 static increment	Distinguishes reader/postpass/executor preparation and mixed saved-state handling without claiming full handler recovery, runtime restore or cross-architecture layout equivalence[343]
Mac OS X 6.11r1 runner	Decoded 40-byte records with cached handlers, an appended end marker 164, a per-record loop check with default limit 10000, snapshot-based sleep and waitfor, and calls that can suspend on `classesdownloaded`. Table extent, opcode 7, arithmetic handlers and runtime behaviour stay open (→ §8.4.4)[341]

E#208 What the OpenGS1 opcode set describes

Two notes disagree on whether the OpenGS1 opcode set describes official bytecode[316].

Item	Content
Claim A	the graphics-and-bytecode note calls the OpenGS1 VM "GS1 bytecode" and lists the full GS1 opcode set and constant-table layout as known[316]
Claim B	2.05 does not compile GS1 to bytecode; OpenGS1's opcode set is an independent reimplementation that 2.05 does not share[309,321]
Known	4.2.0.8, 5.0.0.7, and unpacked 6.00 do not use OpenGS1's layout[329]
Open	whether 3.153's bytecode matches OpenGS1, or only OpenGS1 itself[316]
Settles it	comparing a captured official bytecode message (→ App. E#207) with the OpenGS1 layout[316]

E#300 Frame driver of 4th Graal alpha, 5th Graal and 5th Graal alpha

These three early builds were first said to have no frame timer. An exact read found one in each, and their real tick timing is still unmeasured[200].

Item	Content
Superseded observation	the survey formerly reported no form timer for these three builds; the exact-input follow-up finds a configured timer in every one[200]
Static resolution	the configured timer and its static binding are described in → §4.1.2; the three survey interval cells are corrected[200]
Callback execution	a debugger breakpoint counted 61, 61 and 60 callback hits in 3 s for the three builds; this proves execution, not cadence[200]
Open	elapsed tick timing, runtime configuration changes and programmatic idle-handler installation remain unmeasured[200]
Settles it	runtime observation of those exact builds, preserving the distinction between declared interval and elapsed time[200]

E#301 Frame loop of the 3.x and 4.x clients

No one has timed the frame loop of the 3.x and 4.x clients. The rows give what static reads found so far[4].

Item	Content
Unobserved	the 3.x and 4.x Torque-era clients' tick length and loop driver; the 5.x and 6.x clients are timed only from their 50 ms position messages[4,10]
Settles it	a timed walk of a 3.x and a 4.x client on the lab server[10]

Item	Content
Windows 4.04r2 clocks	three time notions are traced statically (elapsed ticks via `GetTickCount`, a GS2 script clock advancing 0.05 per pass, legacy `timeout` pass counts); no update rate is measured[202]
Linux cross-version follow-up	Tie six observed updater returns to a directly registered state predicate without inferring frame cadence or live 3D. The broader question remains partial (→ §4.1.6)[210]

E#302 Walking speed factors other than 1.0, 1.2 and the hidden 0.2

Two walking speed factors, 0.4 and 0.8, exist in the code, but the states that use them are unknown[213].

Item	Content
Known	the walking code multiplies the half-tile step by a speed factor: 1.0 on foot, 1.2 in a high speed state; hiding gives 0.1 tile per tick[213,198]
Open	factors 0.4 and 0.8 also exist (0.8 depends on a carried object); which states use them is not traced; carrying measured live does not slow the walk[215]
Settles it	a lab walk in each candidate state (riding, shield, carrying each object kind) on 2.05[215]

E#303 Two unidentified lists in the blocked-point test

Two object lists in the blocked-point test have not been identified[215].

Item	Content
Open	the blocked-point test reads a list of objects with two flags and a player-sized body box (only with a client option on), and a list of 1.5 × 2 boxes with an enabled flag; neither is identified (signs, baddies, dropped objects and other players are candidates)[215]
Settles it	tracing what fills each list, or a lab test of walking into each kind of level object[215]

E#304 Stopping positions and the wire rounding of nudged positions

The stopping positions after a blocked step are read from code but not confirmed live, and the wire rounding of a nudged position is not captured[212].

Item	Content
Known	the flush snap and the 0.1 nudges are read from the 2.05, 2.17.1, 2.22 and 2.31 walking routines[212]
Open	the stopping positions are not confirmed on a live client; how the client rounds a position that is not on a half-tile when it sends it is not captured[215]
Settles it	a lab walk towards a rock from both half-tile alignments on 2.22, with the props messages captured[215]

E#305 The 2.22 push frame field

A memory trace of 2.22 does not match the push timing expected from 2.05[198].

Item	Content
Claim A	2.22 plays `push` as 5 frames × 4 ticks, looping, as 2.05 does[198]
Claim B	a memory trace of 2.22 against a public server showed a push frame field cycling 0 to 3 every tick from the moment the blocked count passed 18[199]
Open	whether the traced field is the gani frame or another per-tick counter; the second run used the server's own push gani[199]
Settles it	reading the gani frame field of 2.22 offline with the stock `push.gani`[198]

E#306 The flag that picks `lift2.wav` for a pull

The flag that chooses between the two pull sounds was not traced[220].

Item	Content
Open	a successful pull plays `lift.wav`, or `lift2.wav` by a player flag that was not traced[220]
Settles it	tracing the flag in 2.05's pull routine[220]

E#307 Probe, nudge and step tables of 1st Graal and 1.39

1st Graal's movement tables are now read. The 1.39 tables are still missing[218].

Item	Content
Established for 1st Graal	exact-input static step, six ordered probes, one-eighth-tile nudges and stepped-axis snapping are in → §4.2.2.3; the older unread-table and free-axis descriptions are superseded[218]
Existing 1.39 evidence	six probes and the blocked-step counter shape were read, without the numeric tables[214]
Open	1.39 numeric tables, cross-build equality, and complete exceptional gates/runtime cadence for the new static 1st Graal finding[218,214]
Settles it	recover the 1.39 tables independently and validate unresolved runtime/state gates on exact inputs[218]

E#308 Link probe offsets and the neighbour-table flag

The offsets used to probe for links, and the flag that skips the neighbour table, are unread[205].

Item	Content
Open	the per-facing probe offsets for link following; the flag that skips the 3 × 3 neighbour table; which big map 2.05 uses offline and whether 2.22 does the same with PLO_BIGMAP (171); how walking behaves past the edge before the link fires when the neighbour is loaded[205]
Settles it	reading the probe table and the flag in 2.05, and a lab crossing on 2.22 with a big map[205]

E#309 Knockback distance of 1.39

The 1.39 binary gives five tiles of knockback, while a live run measured four[214].

Item	Content
Claim A	the 1.39 binary moves the player one tile per tick while the hurt counter reads 39 down to 35: five steps[214]
Claim B	the offline 1.39 run measured four tiles of knockback on the touch-NPC test[198]
Open	whether the live run lost a step to collision or to the ±1 tick of the video timing[214]
Settles it	a fresh 1.39 knockback re-measure on open ground[214]

E#310 Where the once-per-second hit stamp sits

Two reads disagree on which hit paths share the once-per-second stamp[229].

Item	Content
Claim A	the per-attacker second-of-day stamp is shared by the sword and both hit messages, and is stored before the guild test[229,231]
Claim B	in 2.00 to 2.05 rev 2 the stamp is in the hurt-message path only; it was not found in the sword judge of any build[223]
Settles it	a lab test of two sword hits from one attacker inside one second on 2.05[223]

E#311 Blast hit zone and the occasional full-heart hit

The blast hit zone read from code does not match the pattern of hits measured in the 2.22 lab[224].

Item	Content
Claim A	a blast piece hurts when its centre, offset by one tile, is under one tile from the player's centre on both axes[224]
Claim B	in the 2.22 lab, with a bomb 0.5 or 1.5 tiles off the victim's row, hits came at y offsets +1, +3 and -3 but not at 0 or ±2, and at x offsets ±4 and +5 but not 0 or ±1: a pattern keyed on the bomb's half-tile y rather than a plain radius; a few trials took a full heart instead of a half without a clear rule[195]
Settles it	reading the 2.22 explosion routine and repeating the sweep with one bomb per trial[195]

E#312 The last byte of a shot

Lab tests suggest the last byte of a shot does more than name its type[224].

Item	Content
Claim A	the last byte of `PLO_ARROWADD` (19) is the shot type, `byte - 33`[224]
Claim B	in the 2.22 lab an injected shot hurt only when that byte was `0x23` or more; `0x20` to `0x22` did nothing, and a real arrow from another 2.22 client carried `0x21` and never hurt[195]
Settles it	reading the 2.22 shot receiver, or a capture of a real arrow that hurts[195]

E#313 Do arrows and fireballs pass walls

The code suggests arrows and fireballs fly through walls, but this has not been tested[224].

Item	Content
Open	in the 2.05 shot flow types 0 and 1 skip the blocking test that stops types 2 and 3, which would let arrows and fireballs fly through walls; the test's arguments were not recovered[224]
Settles it	a lab shot at a wall on 2.05[224]

E#314 Gmap list order and the level-entry save

Whether entering a gmap from outside saves the respawn point depends on an order that has not been read[236].

Item	Content
Open	the order of the gmap's level list and when it is updated against the entry save decide whether entering a gmap from outside saves the respawn point[236]
Settles it	a lab walk from a plain level into a gmap and back on 2.05, dying on each side[236]

E#315 The level-entry save in the 2.1 test builds

The level-entry save in the 2.1 test builds is expected to differ from 2.05's, but it was not read[238].

Item	Content
Open	2.1 test 2 and 3 lack the delimiter constant of the name-group test, so they are expected to have no group test; their entry code was not read[238]
Settles it	reading the entry routine of the two builds[236]

E#316 Glove power steps

Two readings disagree on which objects each glove power can lift[245].

Item	Content
Claim A	every player starts at glove 1, which lifts the bush, sign and pot; `glove1` adds the light stone and `glove2` the black stone[245]
Claim B	a bare hand lifts the first two entries, power 1 adds the pot, power 2 the light and black stones together[242]
Settles it	a lab lift of each object on 2.05 at glove 0, 1, 2 and 3[245]

E#317 The 1.x pickup box and item draw order

The 1.x pickup box and whether items draw under or over players are unknown[196].

Item	Content
Open	only 1.3's pixel overlap for picking up a floor item was read; the 1.x box in tiles, and whether items are drawn under or over players, are not known[196]
Settles it	reading the pickup test of 1.39, and a screenshot of a player standing on an item[196]

E#318 Level reload on re-entry offline

Whether offline 2.05 reloads a level on re-entry, which would bring a cut bush back, is untested[243].

Item	Content
Open	whether 2.05 offline reloads a level from disk when the player re-enters it, which would bring a cut bush back[243]
Settles it	a lab run on 2.05 offline: cut a bush, leave the level, come back and read the board[243]

E#319 The single-player start levels

Three level names trigger an extra setup step in 2.05 that was not traced[252].

Item	Content
Open	loading `level1.graal`, `YoshiClanR2.zelda` or `YoshiClanFields.zelda` runs an extra setup step in 2.05; what the step does was not traced[252]
Settles it	loading each name offline on 2.05 in the lab and comparing the player and level state with another level[252]

E#320 Does any server check the tamper check

No server is known to check the tamper-check value[256].

Item	Content
Open	the clients only send `PLI_TAMPERCHECK` (39); the open-source server discards it, and four public servers accepted a 2.22 login without it for 20 s; whether any server checks the value, or its periodic re-send from 2.31, is unknown[256]
Settles it	a 2.31 session on a public server with the re-send withheld or altered, which needs the user's approval for a public login[256]

E#321 The 2.31 window blacklist

The 2.31 window blacklist and the client's reaction to a match are not decoded[258].

Item	Content
Open	2.31 compares top-level window titles and classes with an obfuscated list every 3.2 s; the list and the client's reaction to a match are not decoded[258]
Settles it	decoding the list in 2.31 and a lab run with a window of a listed title open[433]

E#322 How others see a key emote

A key emote is drawn locally and not sent, yet captures show an image being re-sent. The source of those frames is unknown[263].

Item	Content
Open	the 2.05 Ctrl+letter path draws the emote locally and sends nothing, yet captures show `PLI_SHOWIMG` frames re-sending an AFK image about every 3 s; they may come from a server script calling `showimg` or from an untraced path[263]
Settles it	a lab capture of a real 2.05 client pressing Ctrl+A next to a second client[258]

E#323 Level-object routines not read (2.22)

Several 2.22 level-object routines were not read. The row lists them[434].

Item	Content
Open	the throw of a lifted object, what a cut or lift drops, the carrying pose, the chest and sign probe points, the glove property value the server sends, snapping after a step, and the swim and carry speed factors were not read[434]
Settles it	reading those routines in the 2.22 client, or a lab run per item[434]

E#400 Two projects called Ash

Two different things are called Ash in the notes. The manual keeps them apart until their archives are compared[286].

Item	Content
Claim A	Ash is a Python rewrite of the gs2emu protocol, with its own world and default port 14812; it is not the Reborn 2.4 binary[286,287]

Item	Content
Claim B	Ash is the blueprint pack of the 2006 39ster 2.1 GServer, which describes that server as written, not a rewrite; default port 14822[283,435]
Open	whether the two are one project at two stages or two projects that share a name; the manual keeps them apart (→ §5.2, → §5.3.1)[435]
Settles it	the Drive archives of both, compared by date and content[435]

E#401 Listserver reply ids 1 and 2

Listserver ids 1 and 2 are read differently by RC 1.41 and by later clients[297].

Item	Content
Claim A	id 1 is NULL (empty) and id 2 is STATUS text with `\r` as a line break; 2.17, 2.22 and the public service send status text as id 2[297,301]
Claim B	RC 1.41 (2002) reads id 1 as one more server entry and id 2 as the end of the list, then disconnects[298]
Open	whether the 2002 listserver used 1 and 2 differently from the later Reborn listserver, or the RC reads them its own way[298]
Settles it	a 2002-era listserver binary or capture, or a re-read of the 2.05 handler for id 1[297]

E#402 The 2000-era listserver reply

Whether today's legacy listserver reply matches the service of 2000 to 2002 is unknown[301].

Item	Content
Claim A	the real 2.00-era listserver reply is observed: one plain zlib frame, id 0 only, 4 rows, no status, site or upgrade lines (the public service's 14923 reply to a classic login)[301]
Claim B	what a real 2.00-era listserver sends is still unobserved; the two-frame list that 2.00 accepted in the lab came from a lab forwarder[297]
Open	whether today's 14923 legacy reply is the same as the service of 2000 to 2002[303]
Settles it	a capture of a real 2.00 or 2.05 against the 14923 service, or a 2000-era listserver binary[80]

E#403 Classic logins on port 14922

A real 2.17.1 sends a classic login to a port that does not answer it today[80].

Item	Content
Claim A	a real 2.17.1 sends a classic login (shape B) to port 14922, the port in its signed `arrays2.dat`[80]
Claim B	the public 14922 accepts a classic login and never answers it; the 2001-era tools put pre-2.2 clients on 14923[303,293]
Open	whether the 2001-era service answered classic logins on 14922, whether 2.17.1 was built for another listserver, and whether the zlib header (`78 DA` against `78 9C`) matters for a classic frame[290,80]
Settles it	a list-only capture of a pristine 2.17.1 against the public service; this touches a public server and needs the owner's written permission[6]

E#404 A public server refusing 2.05 inside its own range

A public server refused 2.05 although its printed range seems to include it[294].

Item	Content
Claim A	the server's printed range runs from 1.41r1 to iPhone Belote 1.5, which in the current version table covers every 2.x to 6.x entry, 2.05 included[294]
Claim B	the same server refused 2.05 (`GNW03081`)[294]
Open	which version table the server runs; an older table with no 2.05 entry would explain both[294]
Settles it	one 5.00r7 or 6.03r7 login to that server, with the owner's written permission[294]

E#405 RC token `GSERV025`

The server knows an RC token that no RC build read so far sends[101].

Item	Content
Claim A	the server knows the RC tokens `GSERV023` to `GSERV025`[101]
Claim B	the RC builds read send `GSERV007`, `GSERV011`, `GSERV014` and `GSERV019` to `GSERV024`; none sends `GSERV025`[74,306,307]
Open	which RC build sends `GSERV025`, and with which login type[306]
Settles it	a static read of the later RC builds in the Drive RC folder (rc2, the custom RemoteControl, the RC3 beta)[308]

E#420 Flag and chest counts in the RC props tail

The RC props tail holds two counts, and it is not known which counts flags and which counts chests[300].

Item	Content
Claim A	the tail after the props block in 72 and 76 holds two s counts, then the weapons; one counts flags and one opened chests[300]
Open	which s is which; both were 0 in the only capture[300]
Settles it	a capture of 72 for a player with some flags and no opened chests[300]

E#421 The last byte of RC_PLAYERPROPSSET2

The 2004 RC adds one byte the server's message lacks, and its meaning is unknown[300].

Item	Content
Claim A	the 2004 RC's 76 ends with one byte after the weapons, 0 in the only capture[300]
Open	what the byte means; the server's 72 has no such byte[300]
Settles it	a static read of the RC's 76 builder, or a capture after changing other dialog fields[300]

E#422 Quoting of folder rights

The 2004 RC and a server fork quote folder rights differently[300].

Item	Content
Claim A	the 2004 RC sends folder rights lines as CSV, so `rw levels/*` goes out quoted, and splits what it reads back the same way[300]
Claim B	a GServer-v2 fork returned the line unquoted, and the RC showed it as two lines[300]
Open	whether the original server stored and returned the CSV form, which would make the fork's reply a fork bug[300]
Settles it	a capture of 62 from an original 2.x GServer, or a static read of its rights reply[300]

E#500 Back clipping plane of the Graal3D client

Which back clipping plane the Graal3D client uses is unknown[352].

Item	Content
Claim A	the Direct3D retained-mode library's initial back plane is 100, and the February 2000 client sets no field of its own[352]
Claim B	a later public DelphiX fork calls `SetBack(5000)` during retained-mode init; that fork starts after 2004, so it is not the unit linked in 2000[352]
Open	which back plane the February 2000 exe uses; the static read that found no field constant did not look for `SetBack`[352]
Settles it	a static read of the exe for a `SetBack` call, or a lab run with the native `d3drm.dll`[352]

E#501 Class CRC of the 4.04r2 3D plug-in

Two reads disagree on whether 4.04r2 sends a real class CRC[34].

Item	Content
Claim A	the 4.04r2 connect request carries the stock Torque class CRC, so a server needs exactly the same list of networked classes or the client is rejected with `CHR_INVALID`[34]
Claim B	the 4.04r2 CRC table is static data holding `0xFFFFFFFF` for group 0 with no writer, so nothing computes a CRC, as in 5.12[356]

Item	Content
Open	which value a real 4.04r2 client sends; Claim B is the later, more specific read, and the 4.x plug-in note now defers to it[328]
Settles it	a capture of a 4.04r2 connect request[328]

E#502 Whether 4.04r2 loads `.mis` mission files

Whether 4.04r2 can load a local `.mis` file is open. The network path is confirmed, but a local reader is neither found nor ruled out[358].

Item	Content
Claim A	`.mis` mission files are used by 3.x and 4.04r2[358]
Claim B	the 4.04r2 plug-in does not use `.mis`; the mission arrives over the network[328]
Retained-code result	Native mission phases and network ghost construction populate the mission group, including deferred preloading; this is a positive network path, not proof that a separate local mission reader is absent[393]
Open	whether a separate 4.04r2 local mission reader exists and is reachable; the positive network path does not close that question[393]
Settles it	a static read of the 4.04r2 plug-in for the mission loader and its callers, or a capture from a 4.04r2 3D session[328]

E#503 Whether 4.04r2 reads `.dso` compiled TorqueScript

Whether 4.04r2 still reads compiled TorqueScript is open. The corrected evidence row narrows what the strings show[34].

Item	Content
Claim A	the 4.04r2 plug-in's formats include `.dso` compiled TorqueScript[34]
Historical Claim B	the earlier note inferred no TorqueScript compiler/console and no `.dso`/`.cs` use; the retained-code result narrows download exclusion to one branch and does not establish absence of local readers[328,393]
Open	whether a `.dso` reader remains in the plug-in without being reachable[328]
Settles it	a static read of the 4.04r2 plug-in for the code-block loader and its callers[328]
Corrected evidence	Short `.dso` and `.cs` literals occur in the retained 4.04r2 image and participate with `.ter` in a resource-miss download exclusion filter. The earlier defined-string export omitted them; the no-`.dso`-strings claim is withdrawn for this image. Later 5.00r7/5.12 scans are not re-evaluated[393]

E#600 Tile layer as GraalEditor writes it

Which board form GraalEditor writes has never been checked against a file it saved[404].

Item	Content
Claim A	GraalEditor 2.3.1.1 writes `BOARD 0` as 64 lines of 64 decimal tile indices, which the client GLEVNW01 loader does not accept until rewritten as base64 runs[404,151]
Claim B	the 2.05 and 2.22 built-in editors, from which GraalEditor is derived, write base64 `BOARD` rows[405]
Open	which form a file saved by GraalEditor itself holds; the claim was never checked against such a file[405]
Settles it	saving a level from GraalEditor 2.3.1.1 and reading its `BOARD` rows[405]

E#601 Whether link arrival offsets are evaluated

Whether the client evaluates an offset in a link's arrival field is untested[415].

Item	Content
Claim A	link arrival fields may hold `playerx` or `playery` plus a signed offset such as `playery+10`; 21 arrivals in a 3113-level archive have that form, and the link dialog says math and variables may be used[415]
Claim B	the editors store the arrival text verbatim and do not parse it[416]
Open	whether the client evaluates the offset when the link fires, or treats it as the bare variable[415]
Settles it	a lab run through a link with an offset arrival, or a trace of the warp code[415]

E#602 Code width of `Z3-V1` boards

Two sources give different board code widths for `Z3-V1.02`[21].

Item	Content
Claim A	`Z3-V1.02` uses 11-bit codes and `Z3-V1.03`/`.04` use 12; only the 1998 Java Zelda Online games read `.02`[21]
Claim B	the 2.05 and 2.31 readers use 12 bits for both `Z3-V1` versions they handle; the 2.05 string table lists `.03` and `.04` only[154]
Open	the two agree on `.03` and `.04`; read as a width for every Z3 file they do not, and whether 2.05's reader is ever called on a `.02` magic is unmeasured[436]
Settles it	a `Z3-V1.02` file decoded both ways, and a trace of whether 2.05 reads that magic[436]

E#603 Whole tiles and fractions in level records

Which level-record fields are whole tiles and which take fractions is not fully listed. A 4.04r2 read covers one reader[21].

Item	Content
Claim A	coordinates in the binary level records are whole tiles[21]
Claim B	`Z3-V1.03` link numbers are floating point and include halves such as `43.5`; GLEVNW01 allows fractional NPC positions; of 18,710 arrival fields in an archive, 12,035 were plain numbers, halves included[21,151,162]
Open	which fields the whole-tile sentence covers[436]
Settles it	a field list for link box, link arrival, baddy, NPC, chest and sign, each with its stated type and one real file of each magic[436]
Windows 4.04r2 reader	link boxes are floating point; arrivals stay text; baddy positions are raw bytes and NPC, chest and sign positions bytes minus 32, all whole numbers without half-tile scaling; cross-magic examples still missing[202]

E#604 Where a binary sign list ends

Where a binary sign list ends differs between readers[154].

Item	Content
Claim A	2.05 reads signs to the end of the file; 2.31 stops at the first empty line, because a `HEIGHTS` block can follow[154]
Claim B	the sign section runs until an empty line or the end of the file[21]
Open	both client behaviours fit Claim B but not each other on a file with signs, a blank line and a `HEIGHTS` block; the server layout does not say which bytes are signs there[436]
Settles it	one such file read by the 2.05, 2.31 and server readers, with the sign count each keeps[436]
Windows 4.04r2 reader	the sign reader stops after a line shorter than two bytes or at end of file and does not recognize `HEIGHTS`; only the web-level cache reader reads heights after it[202]

E#605 Client listserver frames in two vocabularies

Two descriptions of the client's listserver frames use different words and have not been set beside a capture[80].

Item	Content
Claim A	2.00 and 2.05 send id 0 and `newmain` in one plain zlib frame; 2.17 adds id 1 with account and password; 2.22 and 2.31 send id 7, a key byte, `GNW30123` and `newmain`, then a generation-5 frame with id 1[80]
Claim B	the classic Server Lister ELF acts on two client lines, an edition line (id 0) and an account line (id 1), and has no version token and no later cipher[302]
Open	whether "edition" and "list id 0 and `newmain`" are one observation, and whether "no version token" and "the client sends `GNW30123`" describe two programs; no capture has been laid beside both[436]
Settles it	loopback captures of 2.05, 2.17 and 2.22 written out frame by frame beside both descriptions[436]

E#606 Whether a gmap carries `HEIGHTS` blocks

Whether a gmap holds per-level `HEIGHTS` blocks is settled for one reader. The remaining limit says what is still unknown[437].

Item	Content
Clarified Claim A	the salvage note describes a level grid rather than a tile board; its wording now names coarse and per-level height forms with explicit Windows 4.04r2 reader scope[437,170]
Historical Claim B	HEIGHTS belongs to level files and HEIGHTMAP to the generator gmap; writer/server variants differ. Read as exclusive placement, this claim is too broad for the later evidence[169,168,170]
Bounded resolution	Windows 4.04r2 reads both coarse HEIGHTMAP and named per-level HEIGHTS inside a gmap; a separate archive-labelled Mac 4.04r2 fixture contains both forms. HEIGHTS is not confined to level files (→ §3.2.4)[170]
Remaining limit	The salvage author's historical intent, rendering, server support and original Windows fixture packaging are not established. Its wording is corrected to state the measured build scope[170]

E#607 Leftover lines in the protocol cheat sheet

An old protocol cheat sheet still carries lines that later notes withdrew[438].

Item	Content
Claim A	the cheat sheet still shows a raw byte after the version, says RC clients do not scramble, names 100, 101 and 102 as file length, tiles and file blob, and cuts frames into 1,024-byte slices; the recovery note shows guessed ids 1 (login), 16 (level request) and 32 (player update)[438,439]
Claim B	the login has no mystery byte (the extra byte is the generation-3 insertion, and RC 1.x is generation 3 too); 100, 101 and 102 are raw-data, board and file; no 1,024-byte slice is given; the login is type 0, the level request client id 0 and a walk client id 2; directions are 0 up, 1 left, 2 down, 3 right[89,1]
Open	these are withdrawals written in the later notes; the manual follows the later notes (→ §2.1–2.3), but no capture has been set beside each leftover line[436]
Settles it	a 2.05 login, a generation-3 RC session, a generation-5 flush and four labelled facings, each set beside the line it tests[436]

E#608 How many `arrays.dat` lists make a tile passable

Two summaries disagree on how many `arrays.dat` lists decide walkability, and no runtime consumer has been traced[440].

Item	Content
Existing one-list claim	the first array supplies passable tiles and all others are walls[440]
Existing two-list claim	the old-mode 2.05 path checks arrays 0 and 1[178]
Established file distinction	the operations differ for the exact inspected input → §3.5.3.1; this does not identify a runtime consumer[174]
Open	which operation each exact client path uses, and how the older summary relates to it; the new inspection did not trace consumers[174]
Settles it	trace the exact consumer or test distinguishing list-1-only values in a controlled client scene[174]

E#609 macOS archive labels and application identity

The macOS archive labels cannot yet be tied to the historical 3.153 build[67].

Item	Content
Historical claim	The earlier upload comparison explicitly identifies its Mac uploads labelled 3.04 and 3.15r2 as byte-identical copies of the 3.153 image[67]
Current observation	The newer collection's two v3-labelled main executables share a PowerPC digest, but this collection did not independently authenticate their release version; its five v5 copies also share a main-executable digest[67]
Evidence limit	The earlier published note supplies no full input digest for comparison with the current record. It remains unverified whether its image comparison concerns these exact inputs; equality of current main executables is narrower than whole-image or resource equality[67]
Resource follow-up	Regular-file contents and relative names match across two v3 extractions of 718 files each and five v5 extractions of 73 files each; timestamps, permissions, empty directories and original containers were not compared[68]
Open	Exact-input linkage to the historical 3.153 attribution; extracted regular-file equality does not authenticate historical labels or original containers[67,68]
Settles it	A sanitized comparison supplying the earlier inputs' full digests and authenticated build metadata alongside this collection's identities; broader container/metadata equivalence requires separate evidence if claimed[67,68]

E#610 Windows 4.04r2 offline initialization versus gameplay

A 4.04r2 offline transition was observed under a harness, but no gameplay was. The rows say what was seen and what is open[254].

Item	Content
Observed	Instrumentation changes the offline-mode flag, allocates an adventure object and observes continued message-loop callbacks[254]
Open	A complete rendered offline level, player movement and selected cutting, lifting, carrying or hiding behavior in the exact captured build[254]
Settles it	A version-specific controlled scene with captured user actions and before/after state for each mechanic, preserving fixture and harness identities[254]

E#611 Windows 4.04r2 pickup semantics and input qualification

4.04r2's pickup state is recovered statically, but no interaction was captured[244].

Item	Content
Observed	Static recovery separates pickup action, carried category and carrying flag; no interaction was captured successfully[244]
Open	Meaning of the pickup-bound field, configured physical key-to-action path, and asset/effect qualification of the provisional second cut pattern[244]
Settles it	A version-specific semantic read for the bound field and input route, separate asset mapping, and a complete scene capturing each selected action before and after[244]
Timing dependency	The countdown cannot supply elapsed timing without the clocks in → App. E#301[244]

Appendix F. Source map

> Status: stub. Filled by PR 15 of the work plan in issue #191. Main sources: manual/sources.tsv.

Appendix G. Evidence-tag legend

How tags are written and cited is fixed in → §0.3. This appendix lists every tag the manual uses.

G.1 Kind-of-evidence tags

Each row gives one tag and the kind of evidence it marks.

Tag	The claim comes from
`[bin]`	a static read of a program: headers, resources, strings, constants, code paths, described as behaviour
`[file]`	a file shipped with a build or found in an archive
`[doc]`	a readme, manual, release note, or a repository rule document
`[srv]`	server source, summarised through the notes
`[src]`	other source code, summarised through the notes
`[lab]`	a client this project ran in a loopback lab
`[live]`	a client this project ran and watched in play
`[cap]`	a packet capture
`[wire]`	bytes read on the wire during a loopback session
`[run]`	a loopback session log
`[shot]`	a screenshot or still from a run
`[test]`	a check of a reader or rule against real files
`[inf]`	an inference that the source note marked as one (→ §0.3)
`[GE]`	the GraalEditor notes
`[checked]`	a search of named builds for a script word or string

Source of this legend: the reading guide[441] and the agent rules[6].

G.2 Build tags

A build tag names the client build the claim was read from, written with the release number as the notes write it: `[2.05]`, `[2.22]`, `[2.31]`, `[1.39]`, `[5.12]` and so on. A build tag implies a static read or trace of that build unless another tag on the same claim says otherwise. The full list of builds and their names is → §1.1.

G.3 Combining tags

A claim can carry more than one tag. Each row gives a form of combination and what it means.

Form	Meaning
`[srv][2.05]`	both kinds of evidence agree on the claim
`[2.05][inf]`	read from 2.05, with an inference step the source marked
`[cap][inf]`	seen in a capture, with an inference step the source marked

Sources

Every fact in this manual rests on a note in the Graal_analysis repository. A superscript number in the text or a table points to the note below; the note carries the evidence, the build it was read on and how it was obtained. In the repository the same citations stay on the line they support, with their evidence tags (→ §0.3).

1. protocol/codec-and-framing.md
2. protocol/packet-layouts.md
3. protocol/packet-ids.md
4. protocol/v5v6-lab-play.md
5. client-internals/build-catalogue.md
6. AGENTS.md
7. protocol/v3x-clients.md
8. protocol/v5v6-windows-unpacked.md
9. client-internals/packed-6113.md
10. version-profiles/matrix/README.md
11. client-internals/cross-version-timeline.md
12. client-internals/drive-clients-inventory.md
13. client-internals/version-tokens.md
14. client-internals/versions-133-139.md
15. version-profiles/drive-clients-exes.tsv
16. client-internals/version/VERSION.md
17. client-internals/decomp-findings-2026-10-24-c/02-login-token-and-version.md
18. client-internals/version-id-map.md
19. client-internals/decomp-findings-2026-10-24-c/version_tokens.tsv
20. version-profiles/version-tokens-by-build.tsv
21. formats/level-binary.md
22. client-internals/precursors-and-fan-builds.md
23. client-internals/gameboy-demo.md
24. client-internals/android-classic.md
25. client-internals/zelda-online-1998.md
26. client-internals/precursor-runs-2026-10-30.md
27. client-internals/decomp-findings-2026-10-24-c/09-features-classes-vocabulary.md
28. client-internals/decomp-findings-2026-10-24-c/08-test-builds-newworld-newmain.md
29. protocol/client-versions.md
30. version-profiles/2.22-2.31.md
31. version-profiles/3.04-3.153.md
32. version-profiles/lab-304-linux-2026-10-30.md
33. version-profiles/lab-mac-tiger-2026-10-31.md
34. protocol/v4x-clients.md
35. torque/v4-404r2-exe.md
36. client-internals/mac-ppc-startup-input-2026-11-09.md
37. protocol/v5x-clients.md
38. version-profiles/lab-v5-2026-10-30.md
39. protocol/v6x-clients.md
40. client-internals/engine-600r2.md
41. client-internals/engine-600r4.md
42. client-internals/packed-6037.md
43. client-internals/themida-pack.md
44. client-internals/engine-6113.md
45. client-internals/unpacked-6113.md
46. protocol/unpacked-6113-wire.md
47. protocol/gs2-unpacked-6113.md
48. protocol/v6113-wire.md
49. protocol/v6113-list-capture.md
50. protocol/v6113-udp.md
51. protocol/v6113-hello.md
52. client-internals/version-profiles/v1998-v11.toml
53. client-internals/version-profiles/v121-v132.toml
54. client-internals/version-profiles/v133-v139.toml
55. client-internals/version-profiles/141-217.toml
56. version-profiles/2.22-2.31.toml
57. version-profiles/3.04-3.153.toml
58. version-profiles/4.04-4.20.toml
59. version-profiles/5.00-6.03.toml
60. version-profiles/lab-200-listserver-handshake.tsv
61. version-profiles/lab-boardmodify2-186-trailer.tsv
62. version-profiles/lab-enterlevel-231.tsv
63. version-profiles/lab-gs13-respawn.tsv
64. version-profiles/lab-levelchange-46-222-231.tsv
65. version-profiles/lab-serverwarp178.tsv
66. version-profiles/lab-showimg24-relay.tsv
67. version-profiles/macos-client-compatibility.md
68. version-profiles/macos-resource-identity.md
69. version-profiles/linux-network-bootstrap-evidence.md
70. version-profiles/windows-v4-import-provenance.md
71. protocol/player-props.md
72. protocol/relay-1998.md
73. protocol/file-transfer.md
74. protocol/gserv-1999.md
75. protocol/udp-p2p.md
76. client-internals/decomp-findings-2026-10-24/05-205-props-misc.md
77. client-internals/decomp-findings-2026-10-24/07-graal-139.md
78. version-profiles/gen3-insertion-200-205.tsv
79. client-internals/decomp-findings-2026-10-24-c/03-framing-and-encryption.md
80. protocol/login-frames-by-version-2026-10-25.md
81. protocol/file-205r2-prefix-lf.md
82. protocol/macos-ppc-receive-framing.md
83. protocol/v7-graal3d-framing-and-packet-tables.md
84. protocol/lab-checks-2026-10-24.md
85. protocol/login-session-known.md
86. client-internals/decomp-findings-2026-10-24/06-first-graal.md
87. client-internals/decomp-findings-2026-10-24/08-contradictions.md
88. client-internals/relay-room-bridge.md
89. protocol/login-and-walk.md
90. client-internals/decomp-findings-2026-10-24-b/04-v2.00-client.md
91. protocol/v6037-hello-diff.md
92. protocol/linux-local-login-matrix.md
93. version-profiles/pli-sent-ids-by-build-20x.tsv
94. client-internals/decomp-findings-2026-10-24-c/pli_sent_ids_by_build.tsv
95. client-internals/decomp-findings-2026-10-24-c/05-pli-client-to-server.md
96. client-internals/decomp-findings-2026-10-24-c/10-open-questions-and-corrections.md
97. client-internals/decomp-findings-2026-10-24-c/id_changes_between_builds.tsv
98. client-internals/decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md
99. client-internals/decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md
100. client-internals/decomp-findings-2026-10-25-g/02-id46-level-switch.md
101. protocol/chat-pm-rc.md
102. protocol/unknown-ids-2026-11-06.md
103. client-internals/decomp-findings-2026-10-25-g/01-id24-image-list.md
104. version-profiles/tampercheck-values.tsv
105. version-profiles/tampercheck-words.tsv

106. client-internals/id24-205.md
107. client-internals/decomp-findings-2026-10-25-g/image-text-format.tsv
108. client-internals/decomp-findings-2026-10-25-g/04-open-points.md
109. protocol/v6113-pli.md
110. client-internals/decomp-findings-2026-10-24-b/02-server-to-client-layouts.md
111. client-internals/receive-dispatch-revision-boundaries-2026-11-09.md
112. version-profiles/plo-handled-ids-by-build.tsv
113. client-internals/decomp-findings-2026-10-24-c/plo_handled_ids_by_build.tsv
114. client-internals/decomp-findings-2026-10-24-c/06-handler-changes-by-build.md
115. client-internals/decomp-findings-2026-10-24-c/04-plo-server-to-client.md
116. version-profiles/plo-levellink-layouts.tsv
117. protocol/payload-and-trigger-205r2.md
118. client-internals/decomp-findings-2026-10-25-l/04-public-server-run.md
119. version-profiles/plo-layouts-217-231.tsv
120. protocol/link-test-2026-11-04.md
121. client-internals/npcaction.md
122. formats/gpak.md
123. client-internals/decomp-findings-2026-10-25-g/comma-text-rules.tsv
124. client-internals/decomp-findings-2026-10-25-g/03-id178-comma-text.md
125. protocol/v6113-plo.md
126. protocol/v6113-handlers.md
127. protocol/linux-404r2-receive-scope.md
128. protocol/linux-cross-version-receive.md
129. version-profiles/plo-playerprops-layouts.tsv
130. version-profiles/plo-otherplprops-layouts.tsv
131. version-profiles/plo-npcprops-layouts.tsv
132. client-internals/body-colours-and-attributes.md
133. client-internals/baddies-222.md
134. protocol/v6-props.md
135. protocol/v7-graal3d-player-properties.md
136. client-internals/decomp-findings-2026-10-25-e/01-request-lists.md
137. client-internals/decomp-findings-2026-10-25-e/README.md
138. client-internals/decomp-findings-2026-10-25-e/request-rules.tsv
139. client-internals/decomp-findings-2026-10-25-e/02-save-rules.md
140. client-internals/decomp-findings-2026-10-25-k/02-save-folders-by-build.md
141. client-internals/decomp-findings-2026-10-25-k/save-folders-by-build.tsv
142. client-internals/decomp-findings-2026-10-25-e/04-name-index-case-link.md
143. client-internals/decomp-findings-2026-10-25-e/name-index-rules.tsv
144. protocol/v6113-files.md
145. client-internals/chat-text.md
146. client-internals/feature-specs/14-friends-ignore.md
147. protocol/v6113-chat.md
148. client-internals/decomp-findings-2026-10-24-c/07-udp-channel.md
149. version-profiles/udp-ids-200-204.tsv
150. protocol/v6113-udp-code.md
151. formats/glevnw01.txt
152. client-internals/decomp-findings-2026-10-25-j/03-extensions-and-magic.md
153. client-internals/decomp-findings-2026-10-25-j/by-build.tsv
154. formats/level-binary-client.md
155. client-internals/bare-level-names.md
156. formats/grv1-decoder/README.md
157. client-internals/decomp-findings-2026-10-25-j/02-links-and-arrivals.md
158. client-internals/decomp-findings-2026-10-25-j/01-record-shapes.md
159. client-internals/decomp-findings-2026-10-25-j/05-old-reader-failures.md
160. client-internals/decomp-findings-2026-10-25-j/format-stats.tsv
161. client-internals/decomp-findings-2026-10-25-j/04-loader-rules-compared.md
162. client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md
163. client-internals/decomp-findings-2026-10-25-f/03-baddy-terminator.md
164. client-internals/decomp-findings-2026-10-25-f/corpus-stats.tsv
165. client-internals/decomp-findings-2026-10-25-f/01-section-shapes.md
166. client-internals/decomp-findings-2026-10-25-f/04-sign-and-chest-details.md
167. client-internals/feature-specs/06-gmap-overworld.md
168. formats/gmap.md
169. formats/grmap001.txt
170. formats/gmap-heights-404r2.md
171. formats/gani.txt
172. client-internals/macos-v5-animation-group-lifecycle.md
173. formats/gobjset01.md
174. formats/arrays-membership-2026-11-09.md
175. formats/tiletypes.txt
176. client-internals/client-assets.md
177. formats/tile-walk.md
178. formats/arrays-and-signatures.md
179. client-internals/decomp-findings-2026-10-25-l/02-arrays-dat.md
180. formats/sign-text.md
181. formats/graalsav.md
182. client-internals/feature-specs/05-save-load.md
183. client-internals/saved-passwords.md
184. client-internals/des-and-hidden-strings.md
185. formats/gwebl001-cache.md
186. formats/windows-603-weapon-cache.md
187. formats/linux-603-local-cache-helper.md
188. formats/linux-6x-class-cache.md
189. client-internals/graalconfig-reference.md
190. client-internals/config-and-input.md
191. version-profiles/matrix/builds-deltas.md
192. client-internals/sound-and-music.md
193. version-profiles/matrix/builds-static-survey.tsv
194. client-internals/decomp-findings-2026-10-24/04-sword-hitbox.md
195. client-internals/lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md
196. client-internals/item-drops.md
197. client-internals/decomp-findings-2026-10-24/constants-205.tsv
198. client-internals/player-mannerisms-live.md
199. live-runs/2gta-222-2026-10-30/README.md
200. client-internals/early-form-timers-2026-11-09.md
201. client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md
202. client-internals/windows-404r2-500r7-static-2026-11-09.md
203. client-internals/rendering-observations.md
204. client-internals/water-and-ganis.md
205. client-internals/decomp-findings-2026-10-28/03-edge-links.md
206. client-internals/live-trace-early.md
207. client-internals/live-gserver13.md
208. client-internals/newworld-betas.md
209. client-internals/live-trace-205.md
210. client-internals/linux-420r6-clock-property.md
211. client-internals/v7-701750-bridge-clock.md

212. client-internals/decomp-findings-2026-10-25-m/01-walking-rule.md
213. client-internals/decomp-findings-2026-10-25-l/01-walking-body-box.md
214. version-profiles/matrix/rules-139.md
215. client-internals/decomp-findings-2026-10-25-m/open-questions.tsv
216. version-profiles/matrix/live-runs-2026-10-30.md
217. client-internals/decomp-findings-2026-10-28-b/01-tile-type-tests.md
218. client-internals/first-graal-movement-2026-11-09.md
219. npc-library/gs1-move-205r2.md
220. client-internals/ledges-push-pull.md
221. version-profiles/matrix/rules-1st-graal.md
222. client-internals/decomp-findings-2026-10-24/player-modes.tsv
223. client-internals/combat-200-204.md
224. client-internals/damage-sources.md
225. client-internals/decomp-findings-2026-10-24/01-death-respawn.md
226. client-internals/decomp-findings-2026-10-28/01-sword-while-carrying.md
227. client-internals/status-bit-0x20.md
228. client-internals/feature-specs/02-weapons-combat.md
229. client-internals/decomp-findings-2026-10-25-k/01-knockback-and-packet-hits.md
230. client-internals/decomp-findings-2026-10-24-d/hit-gates.tsv
231. client-internals/decomp-findings-2026-10-24-d/03-hit-gates-and-open-points.md
232. client-internals/decomp-findings-2026-10-24-d/01-sword-damage.md
233. client-internals/decomp-findings-2026-10-24-d/damage-rules.tsv
234. client-internals/decomp-findings-2026-10-25-k/knockback-probes.tsv
235. client-internals/decomp-findings-2026-10-24/life-states.tsv
236. client-internals/decomp-findings-2026-10-24-d/open-questions.tsv
237. client-internals/first-graal-death-counter-2026-11-09.md
238. client-internals/decomp-findings-2026-10-24-d/02-respawn-entry-save.md
239. client-internals/decomp-findings-2026-10-24-d/entry-save-rule.tsv
240. client-internals/decomp-findings-2026-10-24-d/name-group-constant-by-build.tsv
241. client-internals/feature-specs/04-death-respawn.md
242. client-internals/decomp-findings-2026-10-25-l/03-cut-lift-chest.md
243. client-internals/board-respawn.md
244. client-internals/404r2-carry-state.md
245. client-internals/carry-hide-and-glove.md
246. client-internals/live-checks-editor-offline-sound.md
247. client-internals/status-display.md
248. client-internals/sign-panel.md
249. client-internals/decomp-findings-2026-10-28/02-item-list.md
250. client-internals/feature-specs/09-options.md
251. client-internals/feature-specs/01-sound.md
252. client-internals/offline-tutorial-205.md
253. client-internals/offline-training.md
254. version-profiles/windows-404r2-teardown.md
255. client-internals/level-editor-205.md
256. client-internals/feature-specs/08-tampercheck.md
257. client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md
258. client-internals/feature-specs/README.md
259. recovered-hacks/behaviour-models.md
260. recovered-hacks/TRAINER_RE_NOTES.md
261. recovered-hacks/THREAT_TAXONOMY.md
262. recovered-hacks/static-2026-11-01/README.md
263. client-internals/feature-specs/11-emotes-ganis.md
264. client-internals/decomp-findings-2026-10-24/02-sprites-poses-emotes.md
265. client-internals/decomp-findings-2026-10-24/sprite-step-table.tsv
266. client-internals/decomp-findings-2026-10-24/gani-to-step.tsv
267. client-internals/decomp-findings-2026-10-24/gani-name-table.tsv
268. client-internals/decomp-findings-2026-10-24/status-bits-prop18.tsv
269. client-internals/decomp-findings-2026-10-24/gani-short-codes.tsv
270. client-internals/feature-specs/03-inventory.md
271. client-internals/decomp-findings-2026-10-24/03-player-list-popup.md
272. client-internals/feature-specs/10-right-click-menu.md
273. client-internals/feature-specs/12-motd-rules.md
274. client-internals/feature-specs/13-ban-kick.md
275. client-internals/precursor-mechanics-2026-11-09.md
276. client-internals/live-behaviour-404.md
277. client-internals/build-diff-204-205.md
278. client-internals/client-architecture-205.md
279. client-internals/feature-timeline.md
280. version-profiles/early-windows-screen-review-2026-11.md
281. server-notes/setcharprop-hash-n.md
282. ash/ARCHITECTURE.md
283. ash/INDEX.md
284. ash/BLUEPRINT.md
285. ash/ALGORITHMS.md
286. server-notes/01-lineage.md
287. server-notes/ash-overview.md
288. server-notes/03-protocol.md
289. server-notes/04-algorithms.md
290. server-notes/lab-servers-2026-10-30.md
291. tools/lab-servers/README.md
292. tools/classic-tools/gserverstat.md
293. tools/server-tools-2001.md
294. server-notes/public-allowed-versions.md
295. tools/npcserver-2001.md
296. server-notes/baddies-serverside.md
297. protocol/listserver.md
298. tools/npc-control.md
299. client-internals/v2.00-connect-path.md
300. protocol/rc-capture-2026-11-04.md
301. protocol/public-listserver-2026-10-25.md
302. tools/classic-tools/serverlist3.md
303. protocol/listserver-login-type-matrix-2026-10-25.md
304. protocol/listserver-login-type-matrix-2026-10-25.tsv
305. protocol/public-listserver-2026-10-25.tsv
306. tools/remotecontrol-lineage.md
307. tools/remotecontrol.md
308. protocol/npc-control.md
309. npc-library/gs1-reference-205.md
310. npc-library/gs1-command-reference-205.md
311. npc-library/archival-scripting-notes.md
312. npc-library/gs1-condition-truth-205r2.md
313. npc-library/gs1-message-codes-205r2.md
314. npc-library/README.md
315. client-internals/gs2-vm-from-engines.md
316. protocol/graphics-movement-bytecode.md
317. npc-library/preagonal-gs2.md
318. client-internals/linux-404r2-events.md
319. client-internals/windows-500-event-queue.md
320. npc-library/behaviours.md
321. npc-library/gs1-commands.md
322. npc-library/gs1-event-parameters-205r2.md
323. npc-library/linux-304-gs1-runtime.md
324. npc-library/gs1-signatures-205-followup.md
325. client-internals/spyfire-by-version.md

326. npc-library/gs1-object-members-205r2.md
327. client-internals/decomp-findings-2026-10-24-c/script_vocab_deltas.tsv
328. graal3d/torque-engine-4x.md
329. protocol/gs2-client-vm.md
330. protocol/gs2-android-classic-18.md
331. client-internals/linux-404r2-sleep.md
332. client-internals/linux-420r6-selector11.md
333. client-internals/linux-404r2-api-ownership.md
334. client-internals/reference-reimplementations/levelscripteditor.md
335. protocol/gs2-6113.md
336. protocol/gs2-client-only-opcodes.md
337. client-internals/linux-420r6-reader-fusion.md
338. client-internals/linux-404r2-vm-dispatch.md
339. client-internals/linux-cross-version-dispatch.md
340. client-internals/gs2-6113.md
341. client-internals/mac-611r1-gs2-runner.md
342. client-internals/gs2-builtins-6113.md
343. client-internals/linux-601-x86-program-state.md
344. client-internals/linux-404r2-embedded-scripts.md
345. client-internals/linux-cross-version-embedded-controls.md
346. protocol/linux-603-network-gs2.md
347. graal3d/README.md
348. graal3d/protocol.md
349. graal3d/lab-2026-10-30.md
350. graal3d/client-behaviour.md
351. graal3d/formats.md
352. graal3d/camera.md
353. graal3d/collision.md
354. torque/study-torque-clients.md
355. torque/engine-304.md
356. torque/wire-by-build.md
357. torque/engine-4208.md
358. torque/engine-and-formats.md
359. torque/network.md
360. client-internals/torque-surface-by-era.md
361. torque/linux-304-seven-operation-contract.md
362. torque/README.md
363. torque/304-net.md
364. torque/304-adventure.md
365. torque/304-draw.md
366. torque/304-npcs.md
367. torque/304-audio-ui.md
368. torque/v4-420r8-exe.md
369. torque/4208-launcher.md
370. torque/4208-engine.md
371. torque/4208-g3d.md
372. client-internals/windows-420-render-seams.md
373. torque/5007-launcher.md
374. torque/5007-engine.md
375. torque/5007-g3d.md
376. torque/launcher-5007.md
377. torque/engine-5007.md
378. torque/g3d-5007.md
379. torque/stub-512.md
380. torque/stub-5243.md
381. torque/stub-524r4.md
382. torque/freetype6-5007.md
383. torque/stub-fmod-5007.md
384. torque/engine-530r3.md
385. torque/engine-531r4.md
386. client-internals/issho-cut.md
387. client-internals/unpacked-6037.md
388. torque/skipped-version-stamps.md
389. torque/skipped-v4-read.md
390. torque/skipped-v5-read.md
391. client-internals/skipped-v6-read.md
392. client-internals/decompile-6113.md
393. torque/404r2-resource-filter.md
394. torque/scripting.md
395. client-internals/linux-304-startup-reader.md
396. formats/linux-torque-resource-scope.md
397. client-internals/linux-420-native-object-bridge.md
398. tools/level-editor/README.md
399. tools/graaleditor-vs-205.md
400. tools/generators-222/README.md
401. tools/gif2mng.md
402. recovered-drive-tools/ghidra-analysis/RemoteControl_classic/SUMMARY.md
403. tools/classic-tools/README.md
404. tools/graaleditor.md
405. tools/level-editor/version-deltas.md
406. tools/graalshop.md
407. tools/generators.md
408. tools/level-tools-gaps/README.md
409. tools/classic-tools/graal-level-gen.md
410. recovered-drive-tools/ghidra-analysis/GraalLevelGenNW/SUMMARY.md
411. recovered-drive-tools/ghidra-analysis/remotecontrol_custom/SUMMARY.md
412. recovered-drive-tools/ghidra-analysis/RemoteControl-NW/SUMMARY.md
413. recovered-drive-tools/ghidra-analysis/RemoteControl3/SUMMARY.md
414. recovered-drive-tools/ghidra-analysis/NPCControl/SUMMARY.md
415. tools/level-editor/222-reference.md
416. tools/level-editor/formats-crossref.md
417. tools/classic-tools/text-convert.md
418. tools/classic-tools/character-designer.md
419. recovered-drive-tools/ghidra-analysis/ListServer/SUMMARY.md
420. recovered-drive-tools/ghidra-analysis/scripteditor/SUMMARY.md
421. recovered-drive-tools/ghidra-analysis/BytecodeViewer/SUMMARY.md
422. recovered-drive-tools/ghidra-analysis/Assimilator/SUMMARY.md
423. recovered-drive-tools/ghidra-analysis/PNGView/SUMMARY.md
424. recovered-drive-tools/ghidra-analysis/TilesEditor/SUMMARY.md
425. recovered-drive-tools/ghidra-analysis/DClient_setup/SUMMARY.md
426. recovered-drive-tools/ghidra-analysis/gserver13/SUMMARY.md
427. recovered-drive-tools/ghidra-analysis/gserver-linux/SUMMARY.md
428. recovered-drive-tools/ghidra-analysis/gserver-win/SUMMARY.md
429. recovered-drive-tools/ghidra-analysis/gserver_pe_GServersource/SUMMARY.md
430. recovered-drive-tools/ghidra-analysis/Matriark_gserver/SUMMARY.md
431. client-internals/decomp-findings-2026-10-25-k/03-open-points.md
432. client-internals/decomp-findings-2026-10-25-e/03-open-points.md
433. client-internals/decomp-findings-2026-10-24-b/05-open-items.md
434. client-internals/decomp-findings-2026-10-25-l/open-questions.tsv
435. ash/ARCHIVAL-NOTE.md
436. docs/book/04-where-the-notes-disagree.md
437. protocol/format-notes-from-salvage.md
438. protocol/PROTOCOL-CHEATSHEET.md
439. protocol/login-recovery-start.md
440. CONSOLIDATED.md
441. docs/book/00-how-to-read.md